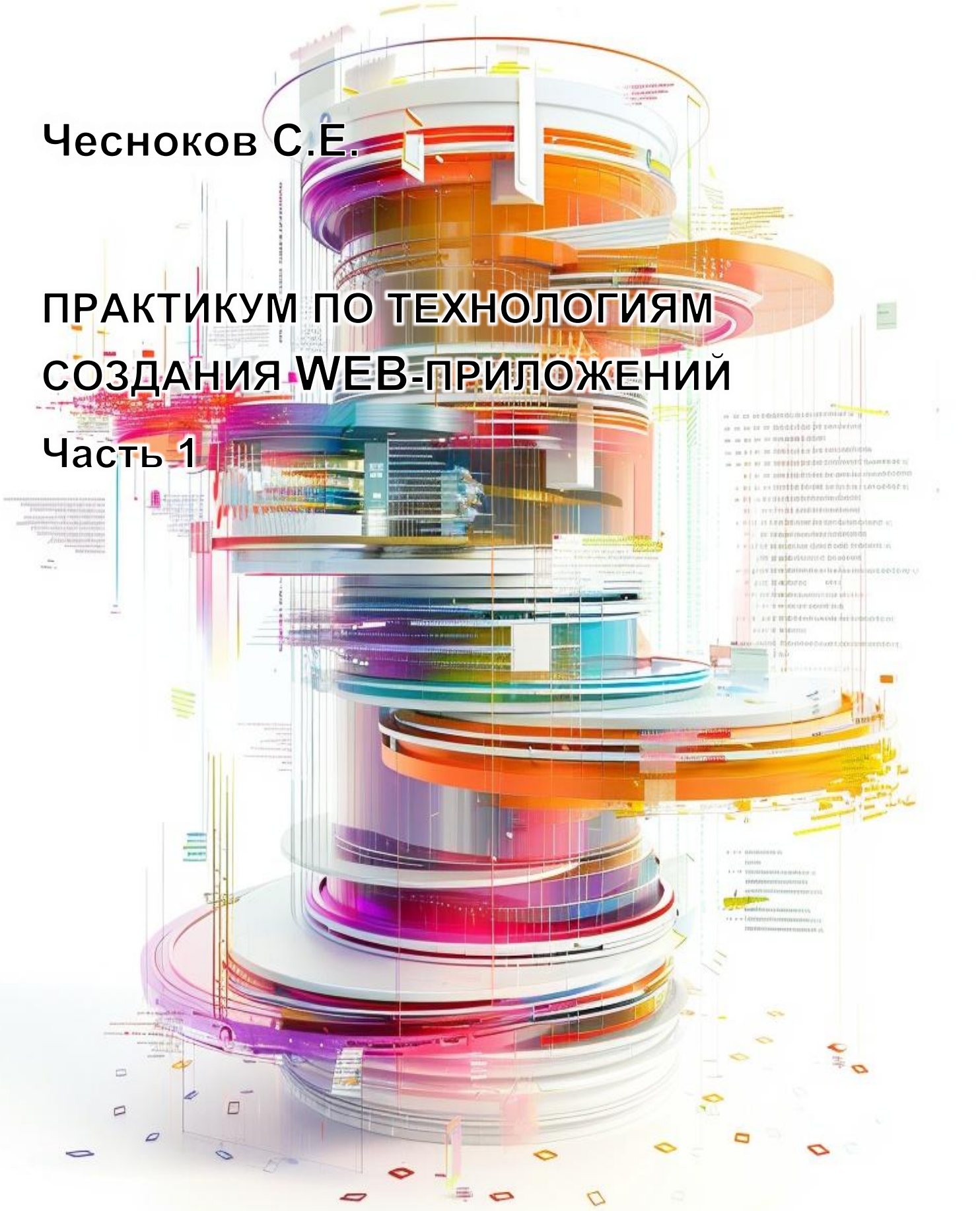
Чесноков С.Е.

ПРАКТИКУМ ПО ТЕХНОЛОГИЯМ СОЗДАНИЯ WEB-ПРИЛОЖЕНИЙ

Часть 1



УДК 681.3(07)
ББК 38.54
Ч51

Чесноков С.Е.

Ч51 Практикум по технологиям создания Web-приложений. Часть 1: Практикум / 2025. - 400 с.

В практикуме по веб-разработке студенты имеют возможность углубиться в ключевые аспекты работы с веб-технологиями: он включает в себя изучение утилит работы с сетью, что позволит участникам ознакомиться с основными инструментами для диагностики и мониторинга сетевых подключений; изучение сетевых протоколов, что является основой для дальнейшего изучения веб-приложений и их архитектуры; настройки рабочего места веб-разработчика, для закрепления навыков установки и конфигурирования необходимых программных инструментов; изучение HTML и CSS, где формируются знания по основам разметки веб-страниц с помощью HTML, создания структуры страниц и использовать различные элементов, стилизации своих веб-страниц, создавания адаптивного дизайна.

В результате прохождения работ практикума студенты смогут создавать полноценные веб-страницы, что станет основой для их дальнейшего профессионального роста в области веб-разработки.

УДК 681.3(07)
ББК 38.54

© Чесноков С.Е.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ 5
ПРАКТИЧЕСКАЯ РАБОТА 1. ДИАГНОСТИЧЕСКИЕ УТИЛИТЫ TCP/IP 7
1.1. Веб-приложения и веб-сайты 7
1.2. Адресация в сети Интернет 9
1.3. Сетевые команды 17
Задания к практической работе 34
Контрольные вопросы 36
ПРАКТИЧЕСКАЯ РАБОТА 2. АНАЛИЗ ПРОТОКОЛА HTTP 37
2.1. Протокол HTTP 38
2.2. Функции HTTP протокола 40
2.3. Зависимость от других протоколов и параметры HTTP 40
2.2. HTTP-запросы 42
2.3. HTTP-ответы 45
2.4. Анализ сетевых протоколов с использованием Wireshark 47
Задания к практической работе 56
Контрольные вопросы 68
ПРАКТИЧЕСКАЯ РАБОТА 3. НАСТРОЙКА СРЕДЫ ВЕБ-РАЗРАБОТКИ 69
3.1. Инструменты Веб-разработчика 70
3.2. Различные компоненты инструментов разработчика Chrome 71
3.3. Облачные инструменты разработчика 81
3.4. Установка редактора VS Code 84
3.5. Использование ngrok для создания тоннелей 89
Задания к практической работе 93
Контрольные вопросы 94
ПРАКТИЧЕСКАЯ РАБОТА 4. ЗНАКОМСТВО С БАЗОВЫМИ ТЕГАМИ HTML 96
4.1. Структура HTML элементов 96
4.2. Организация HTML страницы 109
4.3. Метаданные в HTML 119
4.4. Работа с SVG графикой 128
4.5. Страница 404 ошибки на сайте 131
Задания к практической работе 133
Контрольные вопросы 140
ПРАКТИЧЕСКАЯ РАБОТА 5. СТРУКТУРНЫЕ ЭЛЕМЕНТЫ HTML5 142
5.1. Структурные элементы HTML5 142

5.2. Метаданные в HTML....146
5.3. Практика написания хороших ссылок....157
5.4. Гиперссылки....160
5.5. Списки....166
5.6. Таблицы....172
Задания к практической работе....184
Чек-лист самопроверки....215
Контрольные вопросы....217
ПРАКТИЧЕСКАЯ РАБОТА 6. ВВЕДЕНИЕ В CSS3....218
6.1. Модель коробки в CSS....218
6.2. Селекторы CSS....227
6.3. Каскады....238
6.4. Наследование в CSS....250
Задания к практической работе....256
Чек-лист самопроверки....290
Контрольные вопросы....291
ПРАКТИЧЕСКАЯ РАБОТА 7. РАЗМЕЩЕНИЕ И МЕТОДЫ КОМПОНОВКИ....292
7.1. Цветовые схемы и единицы измерения....292
7.2. Методы компоновки....309
Задания к практической работе....343
Чек-лист самопроверки....411
Контрольные вопросы....411
ЗАКЛЮЧЕНИЕ....413
БИБЛИОГРАФИЧЕСКИЙ СПИСОК....414

ВВЕДЕНИЕ

Изучение основ веб-разработки является критически важным этапом для всех, кто стремится стать профессионалом в этой быстро развивающейся области. Основы, такие как HTML, CSS и JavaScript, формируют фундамент, на котором строятся более сложные концепции и технологии. Понимание этих базовых языков позволяет разработчикам создавать функциональные и эстетически привлекательные веб-страницы, что является основой для дальнейшего роста и развития в профессии.

Кроме того, знание основ веб-разработки помогает разработчикам осознать, как работают различные компоненты веб-приложений. Например, понимание протоколов, таких как HTTP, и принципов работы сетевых утилит позволяет эффективно диагностировать и решать проблемы, связанные с производительностью и доступностью сайтов. Это знание также способствует лучшему взаимодействию с другими специалистами, такими как системные администраторы и дизайнеры, что улучшает командную работу и общий результат проекта.

Изучение основ также открывает двери к более сложным технологиям и инструментам, таким как фреймворки и библиотеки, которые значительно упрощают процесс разработки. Без крепкой базы знаний, разработчики могут столкнуться с трудностями при изучении этих более сложных концепций. Таким образом, освоение основ веб-разработки не только повышает уверенность и компетентность начинающего разработчика, но и создает прочную основу для успешной карьеры в области технологий и цифровых решений.

В условиях стремительного развития технологий и постоянного появления новых инструментов, знание основ веб-разработки становится неотъемлемой частью профессионального роста. Это позволяет разработчикам адаптироваться к изменениям, осваивать новые навыки и оставаться конкурентоспособными на рынке труда. В конечном итоге, изучение основ веб-разработки — это инвестиция в будущее, которая открывает множество возможностей для карьерного роста и самореализации.

В рамках первой части практикума по веб-разработке студенты будут иметь возможность углубиться в ключевые аспекты работы с веб-технологиями через выполнение лабораторных работ. Практикум включает в себя изучение утилит работы с сетью, что позволит участникам ознакомиться с основными инструментами для диагностики и мониторинга сетевых подключений. Студенты научатся использовать такие утилиты, как ping, traceroute и netstat, что даст им понимание принципов работы сети и поможет в решении практических задач.

Следующим важным этапом станет изучение протокола HTTP. Участники практикума получат знания о методах запроса (GET, POST и других) и статусах ответов сервера. С помощью инструментов разработчика в браузере студенты будут анализировать HTTP-запросы и ответы, что позволит им лучше понять взаимодействие клиента и сервера. Этот опыт станет основой для дальнейшего изучения веб-приложений и их архитектуры.

Настройка рабочего места веб-разработчика также займет важное место в программе практикума. Студенты научатся устанавливать и конфигурировать необходимые инструменты, такие как редакторы кода, системы контроля версий и локальные серверы. Это позволит создать комфортную и эффективную среду для разработки, что является критически важным для успешной работы веб-разработчика.

Завершит практикум изучение HTML и CSS. Студенты освоят основы разметки веб-страниц с помощью HTML, научатся создавать структуры страниц и использовать различные элементы. В дополнение к этому они изучат CSS для стилизации своих веб-страниц, что позволит им применять различные визуальные эффекты и создавать адаптивные дизайны. В результате практикума студенты смогут создавать полноценные веб-страницы, что станет основой для их дальнейшего профессионального роста в области веб-разработки.

ПРАКТИЧЕСКАЯ РАБОТА 1. ДИАГНОСТИЧЕСКИЕ УТИЛИТЫ TCP/IP

Цель работы:

- **Изучение основных утилит**: ознакомление с основными утилитами для диагностики сети, такими как **ping**, **traceroute**, **netstat**, **nslookup** и **tcpdump**.
- **Анализ сетевых соединений**: определение состояния сетевых соединений и анализ маршрутов передачи данных между узлами сети.
- **Проверка доступности узлов**: использование утилиты **ping** для проверки доступности удаленных узлов и определения времени отклика.
- **Маршрутизация и трассировка**: применение **traceroute** для изучения маршрута, по которому проходят пакеты данных, и выявление возможных проблем на пути.
- **Разрешение имен**: использование **nslookup** для изучения процесса преобразования доменных имен в IP-адреса и обратно.
- **Мониторинг сетевого трафика**: изучение утилиты **tcpdump** для захвата и анализа сетевого трафика с целью выявления аномалий и проблем в сети.
- **Формирование отчетов**: составление отчетов по результатам диагностики, анализ полученных данных и формулирование выводов о состоянии сети.

Теоретическая часть

1.1. Веб-приложения и веб-сайты

Веб-приложение *— это программное приложение, доступ к которому осуществляется через веб-браузеры, что устраняет необходимость установки. Оно функционирует через Интернет, позволяя пользователям выполнять определенные задачи или получать доступ к функциям без локальной загрузки приложения.*

Веб-приложения бывают разных типов, удовлетворяя разнообразные потребности, от инструментов для совместной работы до платформ электронной коммерции:

1. **Приложения для электронной коммерции:** эти веб-приложения облегчают совершение покупок и транзакций в Интернете, позволяя пользователям просматривать товары, добавлять их в корзину и совершать безопасные платежи.

2. **Платформы социальных сетей:** веб-приложения, такие как VK, Telegram и LinkedIn, позволяют пользователям общаться, обмениваться контентом и участвовать в социальных взаимодействиях.
3. **Инструменты повышения производительности:** веб-приложения для повышения производительности, такие как Google Docs, Trello и Slack, предоставляют пользователям инструменты для совместной работы, создания документов, управления проектами и общения.
4. **Системы управления взаимоотношениями с клиентами (CRM):** веб-приложения CRM, такие как Bitrix24, Salesforce и HubSpot, помогают компаниям управлять взаимоотношениями с клиентами, отслеживать продажи и улучшать взаимодействие с ними.

***Веб-сайт** — это цифровая платформа, доступная через веб-браузеры, включающая взаимосвязанные страницы, которые передают информацию или предоставляют услуги.*

В отличие от веб-приложений, веб-сайты в первую очередь являются *информационными*, предлагая контент для потребления пользователями. Веб-сайты служат различным целям, от личных блогов до корпоративных порталов, и они подразделяются на различные типы в зависимости от их функций:

1. **Статичные веб-сайты:** эти веб-сайты имеют фиксированный контент, который редко меняется. Они обычно носят информационный характер и служат в качестве онлайн-брошюр или портфолио.
2. **Динамические веб-сайты:** динамические веб-сайты генерируют контент динамически, часто извлекая информацию из баз данных или внешних источников. Они могут включать такие функции, как вход пользователя, персонализированный контент и интерактивные элементы.
3. **Сайты электронной коммерции:** сайты электронной коммерции позволяют совершать покупки и продажи в режиме онлайн, демонстрировать товары, обрабатывать транзакции и управлять запасами.
4. **Веб-сайты блогов:** платформы блогов размещают статьи, посты и мультимедийный контент, позволяя отдельным лицам или компаниям делиться информацией, мнениями и идеями со своей аудиторией.

Таблица 1.1. Сравнение веб-приложений и веб-сайтов.

Особенность	Веб-приложение	Веб-сайт
Простая функциональность	Выполняет конкретные задачи и взаимодействия, часто динамичные.	В первую очередь информационный, предлагающий контент для потребления.
Интерактивность	Высокая интерактивность, позволяющая пользователям взаимодействовать и выполнять действия.	В целом ограниченная интерактивность, сосредоточенная на представлении контента.
Цель	Обеспечивает целевые функции, отвечающие конкретным потребностям пользователей.	Передает информацию, продвигает бренд или предоставляет услуги.

Привлечение пользователей	Поощряет активное участие с помощью таких функций, как формы, обновления в реальном времени и инструменты для совместной работы.	Предполагает пассивное взаимодействие, когда пользователи потребляют информацию без прямого взаимодействия.
Обработка данных	Обработка данных Часто подразумевает сложную обработку и управление данными, например, базами данных и учетными записями пользователей.	В основном обрабатывает статический контент, имея ограниченные потребности в обработке данных.
Примеры	Asana, Google Docs, платформы электронной коммерции, такие как Shopify.	Википедия, блоги, корпоративные информационные сайты.
Доступность	Для персонализированного взаимодействия могут потребоваться учетные записи пользователей и аутентификация.	Доступно всем пользователям без необходимости создания учетных записей.
Обновления	Обновления в режиме реального времени являются обычным делом для улучшения функциональности.	Обновления обычно подразумевают добавление или изменение контента.
Сложность разработки	Как правило, более сложный, требующий сложного кодирования и разработки для определенных функций.	Разработка часто более проста и сосредоточена на представлении контента.
Совместимость между устройствами	Может потребоваться адаптивный дизайн для обеспечения удобства использования на различных устройствах.	Обычно разрабатываются таким образом, чтобы быть доступными на различных устройствах без значительных адаптаций.

Выбор между веб-приложением и веб-сайтом — это ключевое решение для бизнеса, влияющее на его цифровое присутствие и вовлеченность пользователей. Веб-приложение может быть идеальным для компаний, ищущих ориентированные на задачи взаимодействия, в то время как те, кто сосредоточен на распространении информации, могут посчитать веб-сайт более подходящим. Объединение обоих может предложить сбалансированную и всеобъемлющую цифровую стратегию во многих случаях. Важно тщательно оценить уникальные потребности бизнеса, чтобы принять обоснованное решение, которое соответствует его целям и требованиям к вовлечению пользователей.

1.2. Адресация в сети Интернет

Для идентификации Web-серверов, маршрутизации электронной почты и многих других целей, каждая машина в Интернет обязана иметь свой уникальный номер, называемый *IP-адресом*. Термин «IP-адрес» относится к адресу интернет-протокола.

***IP-адрес** — это идентификационный номер, связанный с определенным компьютером или компьютерной сетью.*

Например:

- **IPv4**: 192.168.1.1

- **IPv6**: 2001:0db8:85a3:0000:0000:8a2e:0370:7334

При подключении к Интернету IP-адрес позволяет компьютерам отправлять и получать информацию:

- **Адрес интернет-протокола (IP)** позволяет компьютерам отправлять и получать информацию.
- IP-адрес позволяет отправлять и получать информацию нужным сторонам, а это значит, что в некоторых случаях его можно использовать *для отслеживания физического местонахождения пользователя*.
- IP-адреса генерируются с помощью иерархической системы, включающей *IANA, RIR и интернет-провайдеров*.
- К распространенным угрозам безопасности IP-адресов относятся *перехват, внесение в черные списки и DDoS-атаки*.
- Пользователи могут защитить свой IP-адрес с помощью *брандмауэров, регулярного обновления программного обеспечения, использования VPN и включения двухфакторной аутентификации*.

Типы IP-адресов

IP-адрес позволяет компьютерам отправлять и получать данные через Интернет. Большинство IP-адресов являются чисто числовыми, но по мере роста использования Интернета к некоторым адресам были добавлены буквы.

Существует четыре типа IP-адресов:

- Публичные
- Частные
- Статические
- Динамические

В то время как *публичные* и *частные* типы указывают на *местоположение компьютера* или компьютерной сети — *частные* используются *внутри сети, публичные* используются *вне сети — статические и динамические* указывают на *постоянство* или *его отсутствие*.

Пользовательские IP-адреса — IP-адреса частных лиц и предприятий, которые используют Интернет. Пользовательские IP-адреса могут быть как *публичными*, так и *частными*. Как отмечено выше, устройства используют частные IP-адреса внутри сети и публичные IP-адреса за пределами сети. Например, каждому устройству в пределах одной сети назначается уникальный частный IP-адрес, тогда как к публичному IP-адресу можно получить доступ напрямую через Интернет, и он назначается сетевому маршрутизатору интернет-провайдером.

Публичные IP-адреса бывают двух типов: *динамические* IP-адреса и *статические*. *Динамические IP-адреса* — относятся к временному адресу, назначенному устройству

интернет-провайдером. Он активен только в течение определенного периода времени, после чего истекает. Компьютер либо автоматически запросит новую аренду, либо может получить совершенно новый IP-адрес. Динамические IP-адреса обычно назначаются таким устройствам, как компьютеры, смартфоны или маршрутизаторы. Они обеспечивают определенный уровень анонимности и безопасности, поскольку IP-адрес периодически меняется.

Динамические IP-адреса затрудняют отслеживание конкретного устройства или пользователя. Когда устройство подключается к сети интернет-провайдера, ему назначается доступный IP-адрес из пула. Затем этот адрес используется в течение определенного времени, прежде чем будет возвращен в пул и назначен другому устройству. Динамические IP-адреса являются *наиболее распространенным типом адресов интернет-протокола*.

Статические IP-адреса - фиксированный адрес, который остается постоянным при назначении устройству. Такие адреса обычно используются для размещения веб-сайтов или запуска серверов. Имея фиксированный IP-адрес, пользователи могут легко подключать доменные имена к своим серверам, гарантируя, что их веб-сайты или службы всегда будут доступны. Статические адреса настраиваются вручную либо на самом устройстве, либо администратором сети. Обычно статические IP-адреса предоставляются интернет-провайдером пользователя или администратором сети и, как правило, предоставляются за дополнительную плату.

Генерация IP-адресов

1. IP-адреса генерируются автоматически с использованием интегрированного алгоритма, предоставленного *Управлением по распределению адресов Интернета (IANA)*.
2. Затем IANA распределяет блоки IP-адресов по *региональным интернет-регистраторам (RIR)*, которые, в свою очередь, географически распределяют эти блоки среди *поставщиков интернет-услуг (ISP)*.
3. Для генерации индивидуальных IP-адресов для своих клиентов интернет-провайдеры обычно используют технологию, называемую *протоколом динамической конфигурации хоста (DHCP)*. DHCP позволяет устройствам автоматически назначать IP-адрес при подключении к сети.
4. На более техническом уровне IP-адреса генерируются с использованием двух основных версий интернет-протокола (IP): *IPv4 **и** IPv6**.** Адреса IPv4 **— это** 32-битные числа*, представленные в десятичном формате с точками (например, 192.168.0.1), тогда как *адреса IPv6 **— это** 128-битные числа*, представленные в шестнадцатеричном формате (e.9., 2001:0db8:85a3:0000:0000:8a2e:0370:7334).

Следуя правилам и протоколам, установленным IANA, RIR и интернет-провайдерами, устройства могут получать уникальные идентификаторы для общения в сети.

По состоянию на 2024 год в мире насчитывалось около 3,5 млрд IP-адресов типа IPv4.

Защита IP-адреса

Защита IP-адреса имеет решающее значение для обеспечения безопасности и конфиденциальности сети и систем. Пользователи могут защитить IP-адреса с помощью *брандмауэров*, обновления программного обеспечения, использования *VPN* и включения *двухфакторной аутентификации*:

- **Используйте брандмауэр:** внедрение брандмауэра защищает IP-адрес от несанкционированного доступа. Брандмауэры действуют как барьер между сетью и внешними угрозами, фильтруя входящий и исходящий трафик на основе предопределенных правил. Настройте брандмауэр для блокировки подозрительных IP-адресов и ограничения доступа к подозрительным сайтам.
- **Постоянно обновляйте ПО:** регулярно обновляйте операционную систему, приложения и прошивку, чтобы устранить уязвимости безопасности. Устаревшее ПО может подвергнуть ваш IP-адрес потенциальным атакам. Включите автоматические обновления или регулярно проверяйте наличие обновлений из надежных источников, чтобы убедиться, что установлены последние исправления безопасности.
- **Используйте VPN:** виртуальные частные сети (VPN) шифруют ваш интернет-трафик и направляют его через защищенный сервер, маскируя ваш IP-адрес. Это помогает защитить IP-адрес от прослушивания и несанкционированного доступа. Выберите надежного поставщика VPN и включайте его всякий раз, когда идет подключение к публичным сетям или нужно получить доступ к конфиденциальной информации.
- **Включить двухфакторную аутентификацию (2FA):** настройка 2FA добавляет дополнительный уровень безопасности к аккаунтам и устройствам. Она требует от пользователей предоставления второй формы аутентификации, например, уникального кода, отправленного на их мобильное устройство, в дополнение к их паролю. Это предотвращает несанкционированный доступ, даже если кому-то удастся получить IP-адрес и пароль.

Определение своего IP-адреса

Существует несколько способов определить IP-адрес. Возможно, самый простой — ввести «*какой у меня IP-адрес*» в поисковую систему в Интернете, например Google. Есть множество сервисов, которые помогут отобразить эту информацию, например, в таком виде

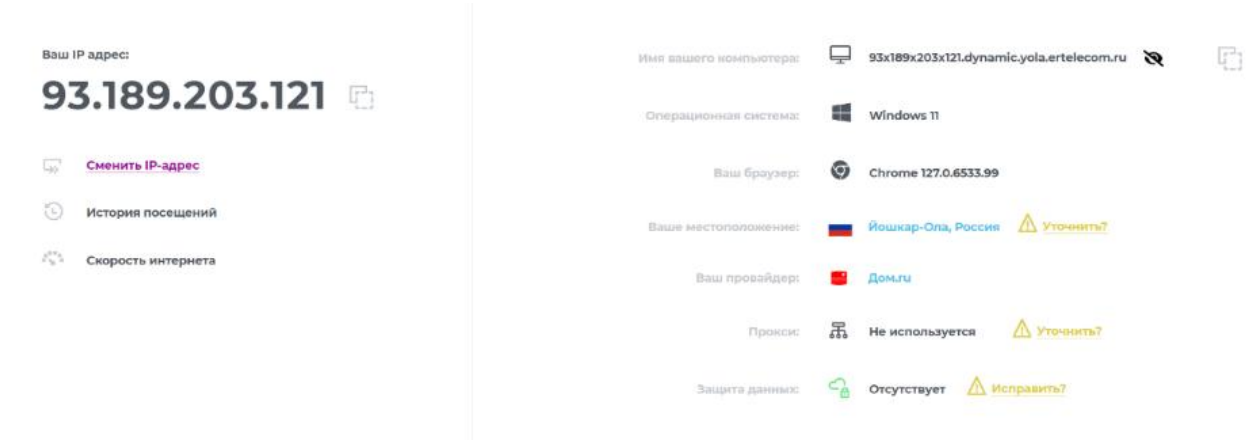


Рис.1.1. Получение IP-адреса через сервис

Для пользователей Windows можно узнать свой IP-адрес, открыв *Пуск, Система, Сеть и Интернет*, а затем выбрать сеть Wi-Fi, к которой вы подключены. В разделе *Свойства* найдите свой IP-адрес, указанный рядом с *Адрес IPv4*.

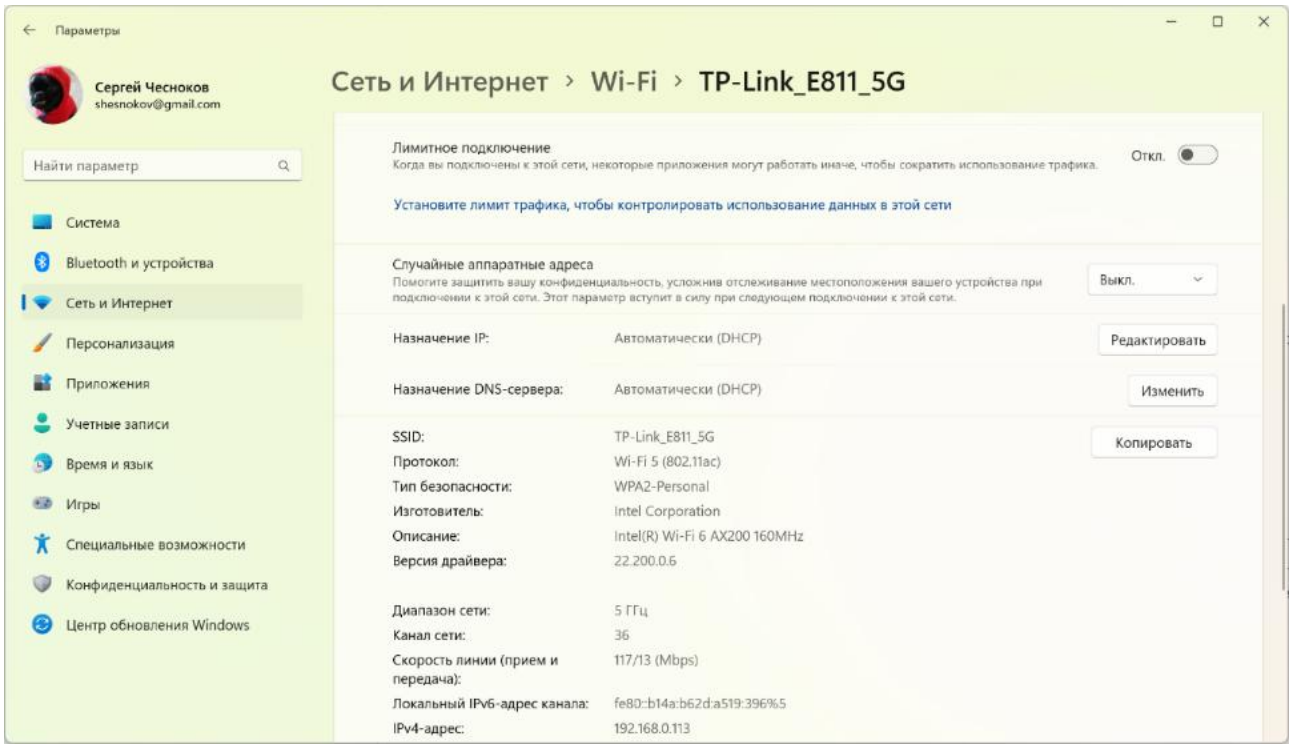


Рис.1.2. Получение IP-адреса в Windows 11

Можете сделать так, чтобы IP-адрес выглядел так, будто он откуда-то из другого места через виртуальную частную сеть.

VPN (Virtual Private Network) *— это технология, которая создает защищенное соединение между вашим устройством и интернетом. Она позволяет шифровать данные и скрывать ваш реальный IP-адрес, обеспечивая безопасность и анонимность в сети.*

VPN маскируют интернет-адрес компьютера, поскольку мы виртуально подключаемся к сети. Поэтому IP-адрес будет отображаться как VPN. Хакеры также могут подделывать IP-адреса, используя более сложные методы, которые изменяют исходный адрес в заголовке вредоносного пакета данных, чтобы имитировать доверенный источник.

Унифицированный указатель ресурса (URL)

URL расшифровывается как **U**niform **R**esource **L**ocator и используется для указания адресов во Всемирной паутине.

URL-адрес *— это основная сетевая идентификация любого ресурса, подключенного к сети (например, гипертекстовых страниц, изображений и звуковых файлов).*

URL-адрес будет иметь следующий формат − *протокол://имя хоста/другая_информация*

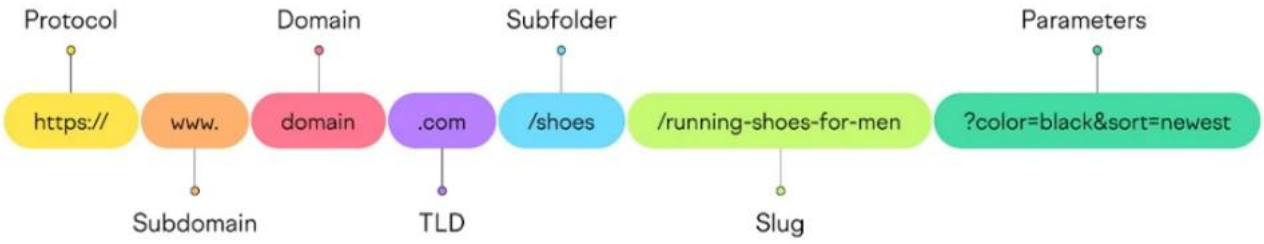


Рис.1.3. Структура URL

URL (Uniform Resource Locator) состоит из нескольких основных компонентов, каждый из которых выполняет свою функцию. Основные составляющие URL:

1. **Схема (protocol)**: указывает протокол, используемый для доступа к ресурсу. Примеры: http, https, ftp. Например: https://
2. **Хост (host)**: указывает доменное имя или IP-адрес сервера, на котором размещен ресурс. Например: www.example.com
3. **Порт (port)** (необязательный): указывает порт на сервере, который используется для соединения. Если порт не указан, используется стандартный (80 для HTTP и 443 для HTTPS). Например: :8080
4. **Путь (path)**: указывает путь к конкретному ресурсу на сервере. Например: /path/to/resource
5. **Запрос (query)** (необязательный): содержит параметры, передаваемые серверу. Начинается с символа ? и может включать несколько пар "ключ=значение", разделенных &. Например: ?key1=value1&key2=value2
6. **Фрагмент (fragment)** (необязательный): указывает на конкретную часть веб-страницы. Начинается с символа #. Например: #section1

Локальный хост

Локальный хост *— это используемый компьютер. Некоторые программисты называют его ПК. Локальные хосты могут ссылаться на местоположение компьютера, например, его «домашнюю базу» или «обратный адрес».*

При написании URL мы как будто напрямую обращаемся к другому компьютеру. С другой стороны, если вводится **localhost** — это способ работы внутри компьютера.

- IP-адрес локального хоста — 127.0.0.1. Он называется **адресом обратной связи**
- IPv6-адрес. IPv6-аналог Localhost остается **::1**
- Использование локального хоста, является **механизмом обратной связи**

Localhost использует механизм, известный как *обратная петля*, для отправки данных, которые присутствуют на той же машине, обратно на себя. Разработчики веб-сайтов и приложений используют локальный хост для экспериментов с сайтом и приложениями локально, а не публикуют их в сети. Это гарантирует, что код будет работать правильно в среде, где возможны сбои.

Размещенный (внутренний) веб-сервер *— это программное обеспечение, которое можно загрузить и установить на локальной машине или на уже установленном на ней компьютере. Он предлагает безопасное пространство для разработки, работы и развертывания веб-приложений.*

Если мы используем интернет-адрес 127.0.0.1 в адресной строке интернет-браузера, компьютер больше не будет выходить в интернет для поиска сайтов.

Компьютер смотрит на свой внутренний сервер и методы анализа скоростей такого сервера в том же компьютере, который используется. Локальный хост — это просто абстракция полнофункционального веб-сервера, который обрабатывает запрос на компьютере.

TCP/IP состоит из нескольких протоколов, из которых IP является наиболее важным для обеспечения связи между хостами на одном компьютере с использованием **localhost** в качестве ссылки. Протокол IP использует **127.0.0.1** как адрес обратной связи. Число 1 — это вариант этого адреса, который показывает, что любой элемент данных, адресованный этому значению, будет обработан внутри. Этот хорошо только для локального тестирования и разработки приложений.

Применение локальных хостов:

- **Изоляция и безопасность:** в разработке ПО и управлении сетями предпочтительно избегать воздействия внешнего мира при работе с локальным хостом. Организуясь в локальном хосте, разработчики работают в среде, изолированной от угроз, что помогает изучать приложения. Localhost полезен для внедрения нового кода или конфигураций в системах.

- **Безопасная среда для тестирования и разработки:** наличие локальной хост-среды помогает разработчикам практиковаться в приложениях, не влияя на реальные системы. Такая стратегия помогает избежать помех от ошибок или проблем во время тестирования с обычными пользователями или рабочим процессом. Ее можно использовать для экспериментов с конфиденциальными процедурами данных, моделями или функциями прототипов или возможностей, сохраняя при этом конфиденциальность соответствующей информации до тех пор, пока не возникнет необходимость ее распространения.
- **Тестирование эффективности:** разработчики могли бы легко тестировать и преобразовывать код и немедленно получать результат в своей локальной системе. Это увеличивает скорость обнаружения и исправления ошибок. Локальный хост обеспечивает удобство использования с меньшей задержкой, поскольку не требует передачи данных через Интернет на другие серверы. Такая эффективность увеличивает скорость проведения тестирования, одновременно снижая эксплуатационные расходы, необходимые для передачи данных. Localhost упрощает интеграционное тестирование, поскольку разработчики видят, как различные части приложения могут эффективно взаимодействовать без привлечения сторонних API или сервисов. Это синхронизирует весь стек и способствует правильному функционированию всех слоев.
- **Диагностика сетевых проблем:** localhost полезен, поскольку он помогает администраторам определить, откуда, скорее всего, возникла проблема, будь то сеть или устройство. Администраторы могут использовать localhost для проверки общей конфигурации сетевых параметров, таких как IP-адрес, параметры разрешения DNS и параметры брандмауэра, а также для проверки несоответствий конфигурации или конфликтов, влияющих на подключение. Для рутинного тестирования на локальном хосте можно использовать обычные программы, такие как "ping", для проверки подключения и времени отклика этого устройства. Это помогает определить корень проблемы с сетевыми адаптерами, драйверами или настройками программного обеспечения.
- **Блокировка вредоносного контента:** реализация локального хоста для блокировки нежелательного веб-контента — отличная идея, которая усиливает безопасность и защищает устройства от вредоносных сайтов.
- **Скорость и эффективность ресурсов:** некоторые преимущества использования localhost при разработке программного обеспечения и использовании сетей включают повышение скорости работы и экономию используемых ресурсов. Разработчики несут ответственность за обеспечение эффективности различных веб-сайтов, приложений или услуг, обслуживаемых на определенном сервере, для предоставления удовлетворительного пользовательского опыта.

Адрес обратной связи используется внутри устройства для направления трафика на него без указания дополнительных внешних сетей. Адреса обратной связи предназначены для IP версии 4 и IP версии 6. Они используются для тестирования и внедрения в пределах одного хоста.

Обратная связь *— это метод связи, используемый внутри устройства различными процессами.*

Этот IP-адрес очень нравится разработчикам и сетевым администраторам, особенно при тестировании сетевых приложений и диагностике существующих. Фильтр устанавливает трафик на 127.0.0.1 или ::1, что позволяет реплицировать сцены сетевого общения и изолировать другие внешние сети.

IPv4 и IPv6 — это всего лишь две версии сетевых адресов. Обе операционные системы имеют *интерфейс обратной связи с именем «lo»*. Этот интерфейс является частью операционной системы, но не соответствует ни одному физическому сетевому устройству. Адреса обратной связи настроены на интерфейс «lo».

- Адрес обратной связи для IPv4 — 127.0.0.1, а домен — localhost.
- Для IPv6 адрес обратной связи — ::1, а доменное имя — localhost6.

1.3. Сетевые команды

Сетевые команды *— это конкретные инструкции или команды, вводимые в* ***интерфейс командной строки (CLI)*** *или командную строку для выполнения различных сетевых действий. Сетевые администраторы и ИТ-специалисты могут использовать эти команды для управления, настройки, диагностики и устранения неполадок, связанных с сетью.*

Общие команды для сетей позволяют напрямую взаимодействовать с сетевыми устройствами, протоколами и службами, предоставляя больше контроля и видимости в сети. Их можно использовать для настройки сетевых параметров, тестирования сетевого подключения, анализа сетевого трафика, диагностики сетевых проблем и мониторинга производительности сети.

Команды для управления беспроводными сетями (Windows)

Команда ping *— это сетевой диагностический инструмент, который можно использовать для проверки подключения и задержки между исходным устройством и устройством назначения.*

Ping определяет, доступно ли устройство или сервер, и анализирует скорость отклика сети, доставляя крошечные пакеты данных и измеряя время на передачу и подтверждение, что делает его полезным инструментом для устранения неполадок в сети и диагностики проблем с подключением.

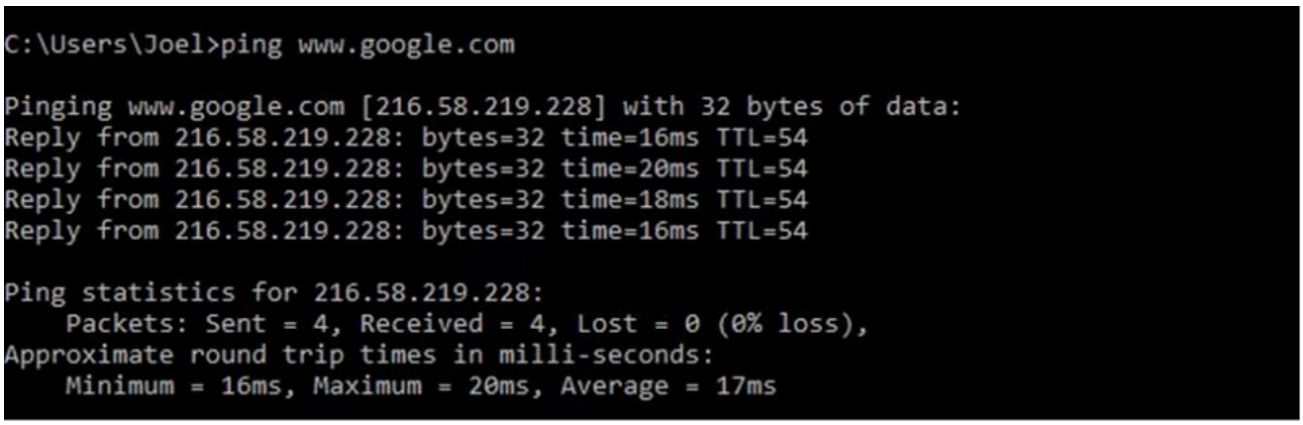


Рис. 1.4. Результат выполнения команды ping host

Команда tracert (или traceroute) *— это сетевой диагностический инструмент, который отслеживает путь пакетов данных, проходящих по IP-сети от исходного устройства до конечного устройства.*

Он дает полный анализ пути, предоставляя промежуточные маршрутизаторы или переходы на этом пути, а также время их отклика, помогая выявлять узкие места сети, проблемы с задержкой и вероятные источники сбоев, что делает его ценным для устранения неполадок сети и обнаружения аномалий сетевого пути.

Рис. 1.5. Результат выполнения команды tracert host

Команда pathping - *предоставляет обширную информацию о сетевом пути, но с более длительным временем выполнения. Анализируя маршрут и проверяя потерю пакетов на основе каждого перехода, Pathping предоставляет подробные сведения о производительности сети от вашего устройства до указанного пункта назначения.*

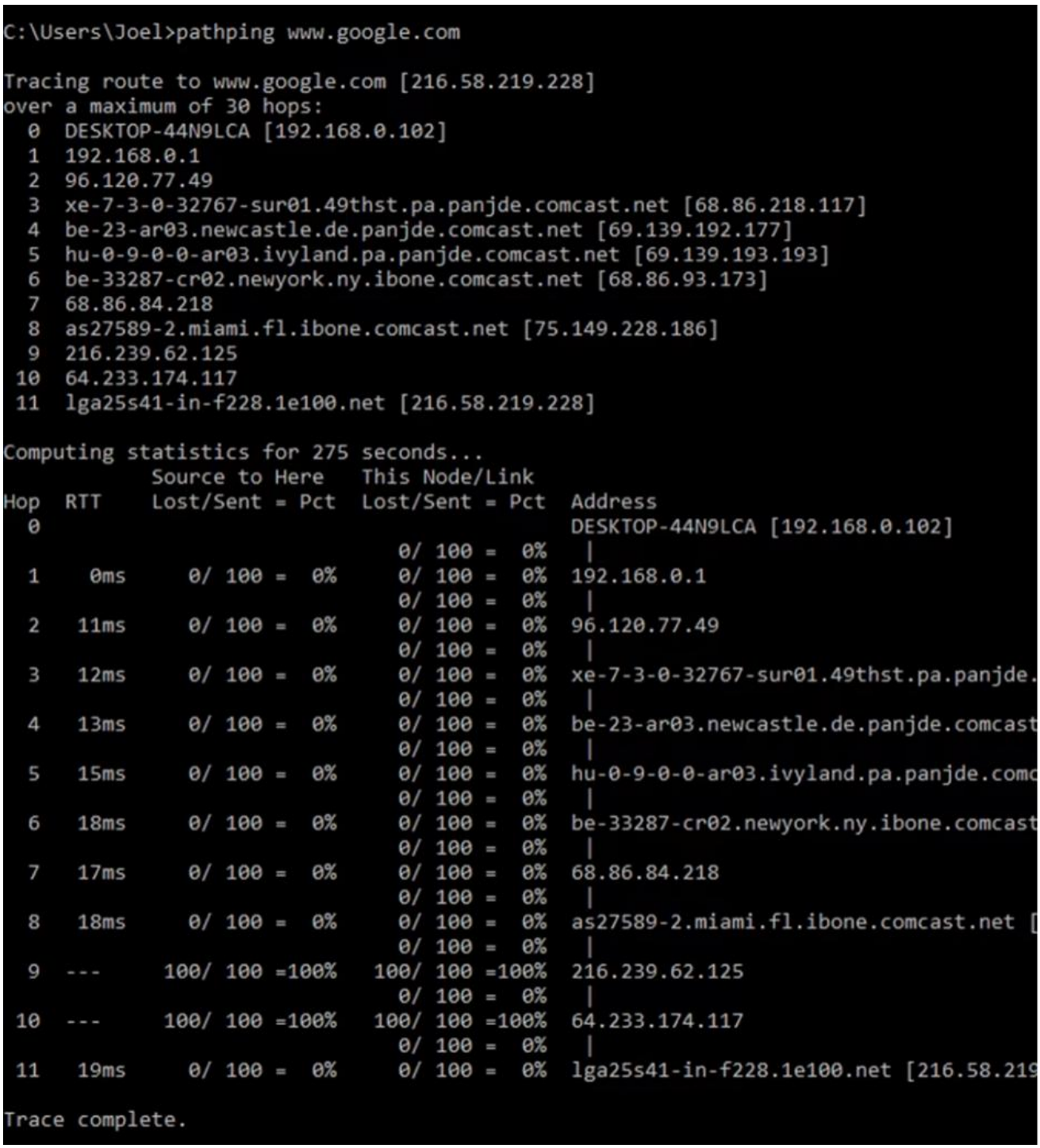


Рис. 1.6. Результат выполнения команды pathping host

Команда ipconfig - *полезный инструмент для быстрой диагностики проблем с сетевым подключением, проверки сетевых настроек и получения важной сетевой информации. Пользователи могут получить доступ к важной информации, такой как IP-адрес сетевого интерфейса (адрес протокола Интернета), маска подсети, шлюз по умолчанию и конфигурации DNS-сервера, выполнив команду ipconfig.*

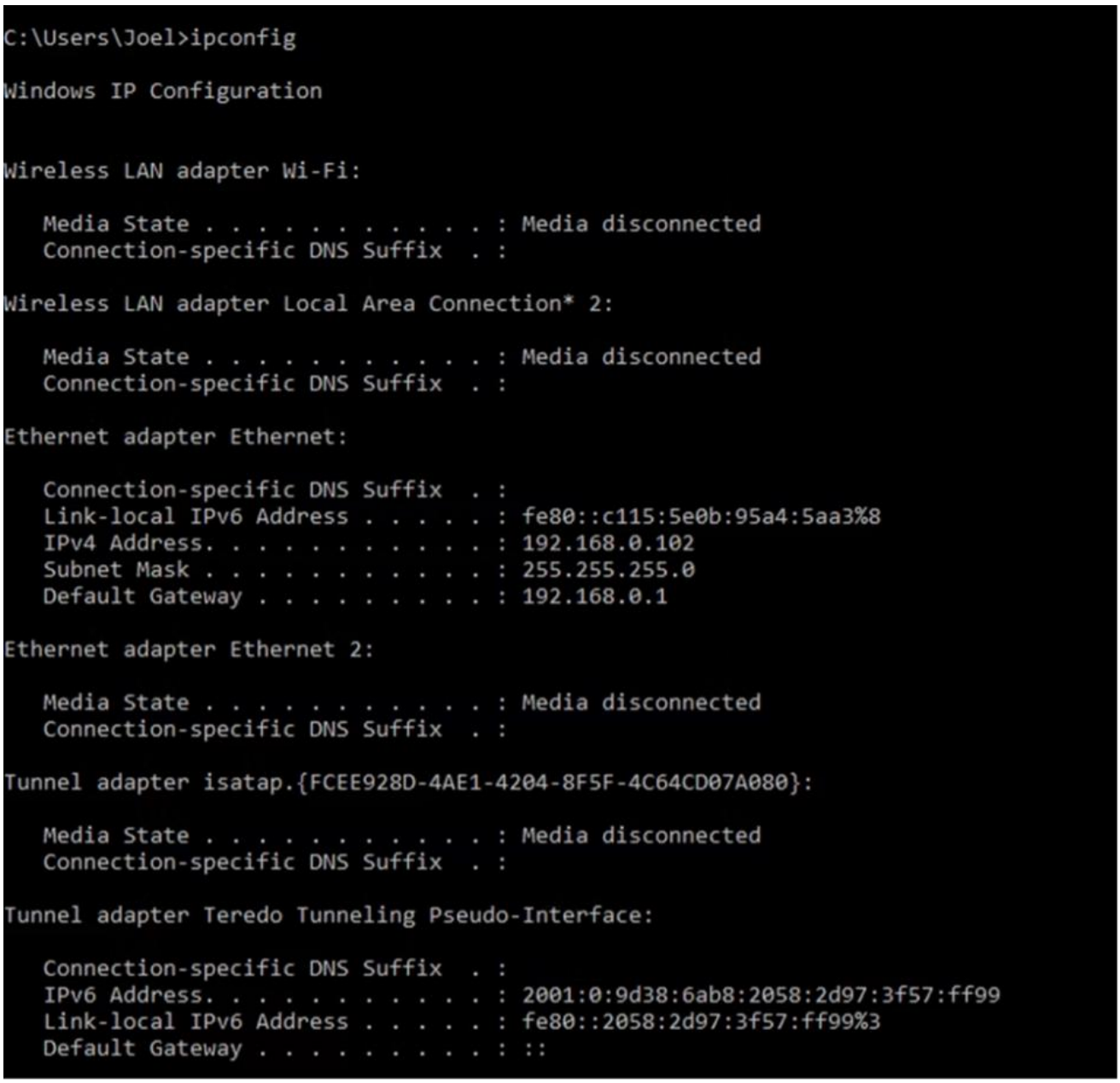


Рис. 1.7. Результат выполнения команды ipconfig

Команда getmac *– позволяет получить MAC-адрес своих сетевых устройств. Администраторы часто полагаются на эти отличительные MAC-адреса для управления сетевым доступом, разрешая или запрещая определенным устройствам подключаться к сети.*

MAC-адрес *— уникальный идентификатор, назначаемый во время производства и хранящийся в оборудовании каждого сетевого устройства, — отличает устройства в Интернете.*

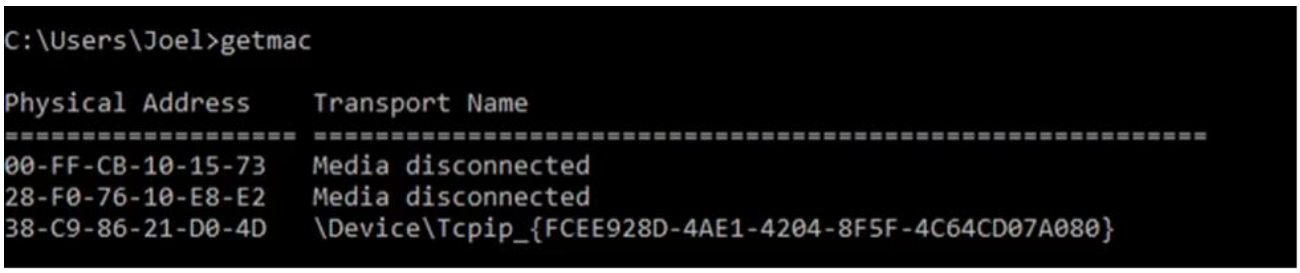


Рис. 1.8. Результат выполнения команды getmac

Команда nslookup - *используется для запроса* ***системы доменных имен (DNS)*** *для получения информации о доменном имени и IP-адресе.*

Можно получить такую информацию, как IP-адрес, связанный с доменным именем, или доменное имя, связанное с IP-адресом, используя команду Nslookup. Это полезный инструмент для быстрой и эффективной диагностики проблем, связанных с DNS, проверки конфигураций DNS и сбора информации DNS.

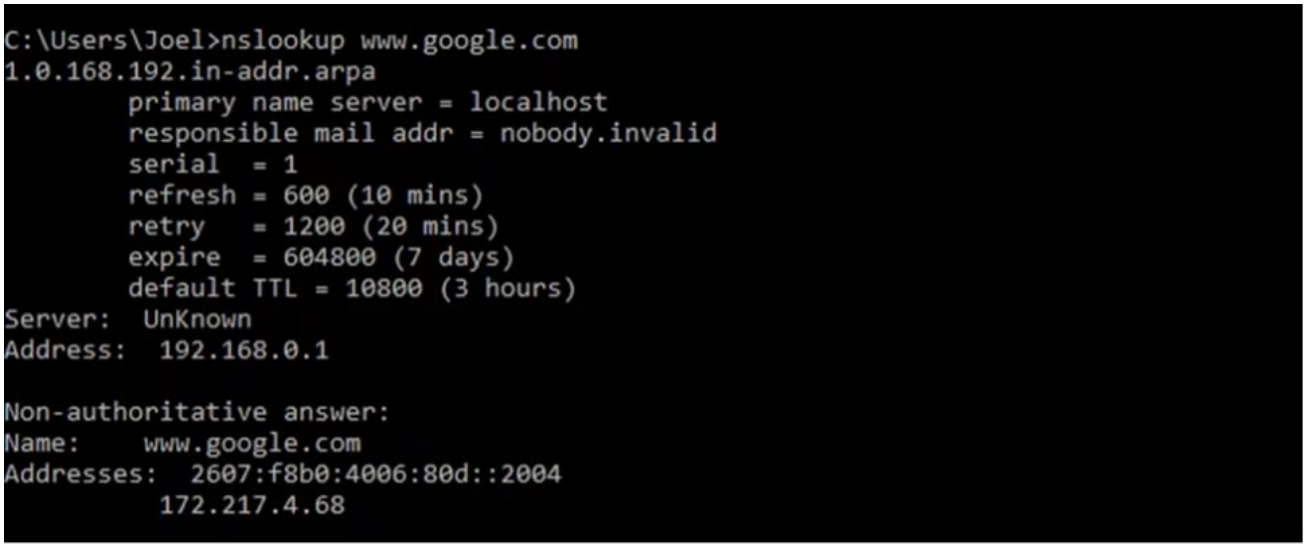


Рис. 1.9. Результат выполнения команды nslookup host

Команда netstat *— это одна из команд для устранения неполадок в сети. Команда Netstat предоставляет информацию об активных соединениях TCP, прослушиваемых портах, статистике Ethernet, таблицах маршрутизации IP и статистике IPv4/IPv6.*

При использовании без параметров она специально показывает активные соединения TCP (протокол управления передачей). Вывод этой команды полезен для устранения неполадок в сети и выявления потенциальных проблем в сетевых соединениях.

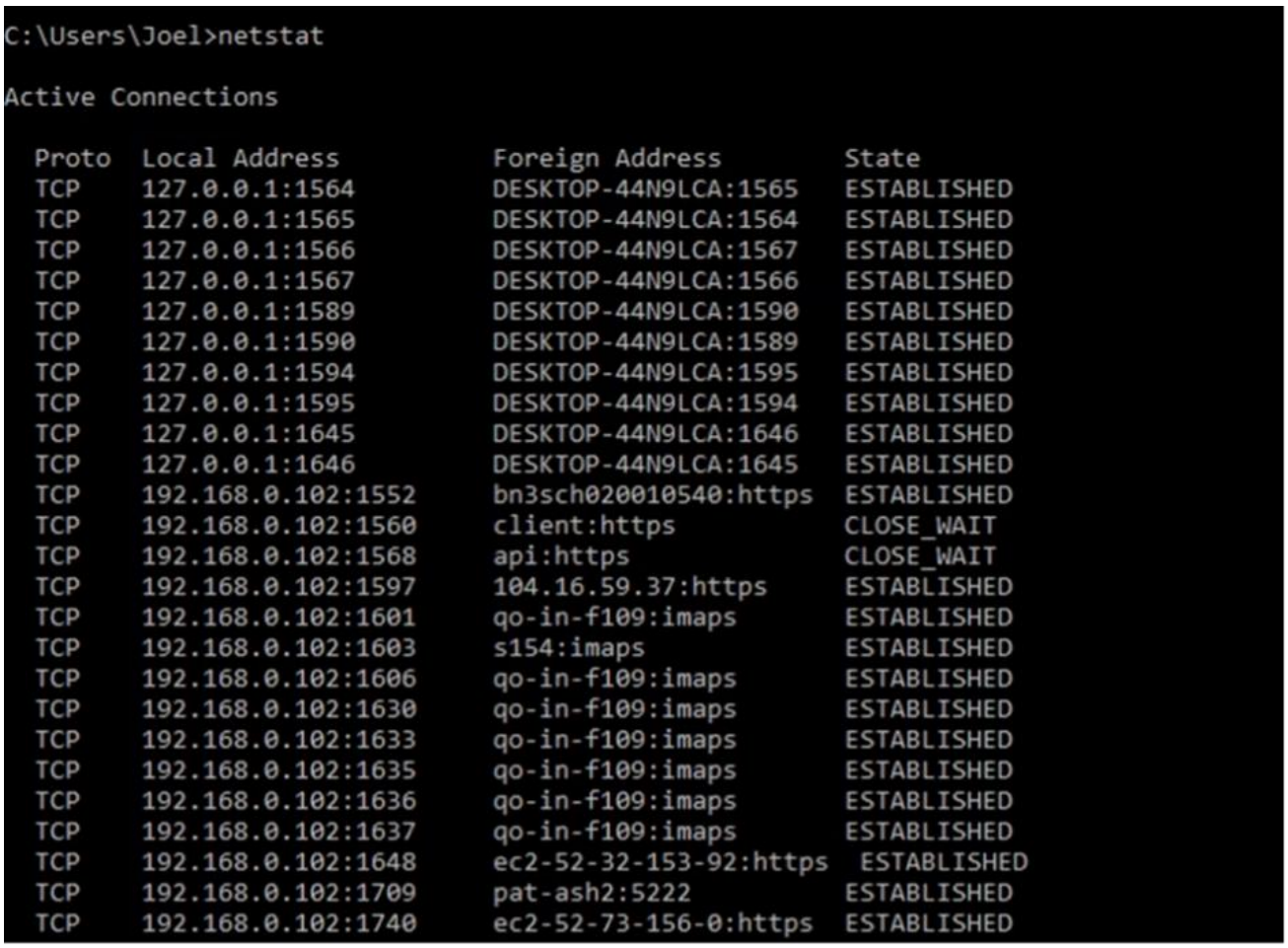


Рис. 1.10. Результат выполнения команды netstat

Команда netsh *— это мощная утилита командной строки для определения и управления различными сетевыми параметрами. Пользователи могут использовать ее для изменения сетевых интерфейсов, настройки сетевых протоколов, мониторинга сетевой статистики и выполнения множества других сетевых задач.*

Netsh особенно полезна для сетевых администраторов и опытных пользователей, которым требуется детальный контроль над сетевыми конфигурациями и параметрами.

Чтобы увидеть все контексты сетевой оболочки, выполните следующее:

```
C:\Users\Joel>netsh /?

Usage: netsh [-a AliasFile] [-c Context] [-r RemoteMachine] [-u [DomainName\]UserNam
             [Command | -f ScriptFile]

The following commands are available:

Commands in this context:
?               - Displays a list of commands.
add             - Adds a configuration entry to a list of entries.
advfirewall     - Changes to the `netsh advfirewall' context.
branchcache     - Changes to the `netsh branchcache' context.
bridge          - Changes to the `netsh bridge' context.
delete          - Deletes a configuration entry from a list of entries.
dhcpclient      - Changes to the `netsh dhcpclient' context.
dnsclient       - Changes to the `netsh dnsclient' context.
dump            - Displays a configuration script.
exec            - Runs a script file.
firewall        - Changes to the `netsh firewall' context.
help            - Displays a list of commands.
http            - Changes to the `netsh http' context.
interface       - Changes to the `netsh interface' context.
ipsec           - Changes to the `netsh ipsec' context.
lan             - Changes to the `netsh lan' context.
mbn             - Changes to the `netsh mbn' context.
```

Рис. 1.11. Результат выполнения команды netsh /?

Чтобы увидеть все сетевые команды в контексте, выполните следующее:

```
C:\Users\Joel>netsh wlan /?

The following commands are available:

Commands in this context:
?               - Displays a list of commands.
add             - Adds a configuration entry to a table.
connect         - Connects to a wireless network.
delete          - Deletes a configuration entry from a table.
disconnect      - Disconnects from a wireless network.
dump            - Displays a configuration script.
export          - Saves WLAN profiles to XML files.
help            - Displays a list of commands.
refresh         - Refresh hosted network settings.
reportissues    - Generate WLAN smart trace report.
set             - Sets configuration information.
show            - Displays information.
start           - Start hosted network.
stop            - Stop hosted network.

To view help for a command, type the command, followed by a space, and then
 type ?.
```

Рис. 1.12. Результат выполнения команды netsh wlan /?

Также выполните эту команду, чтобы просмотреть все драйверы беспроводных сетей в вашей системе и их свойства: **netsh wlan show drivers**

Команда hostname - сетевая команда Windows HostName просто отображает текущее имя компьютера Windows. Это имя, по которому ваш компьютер идентифицируется другими устройствами и серверами в вашей локальной сети. Это имя также можно найти на странице **сведений о системе графического интерфейса**, однако эта команда быстрее.

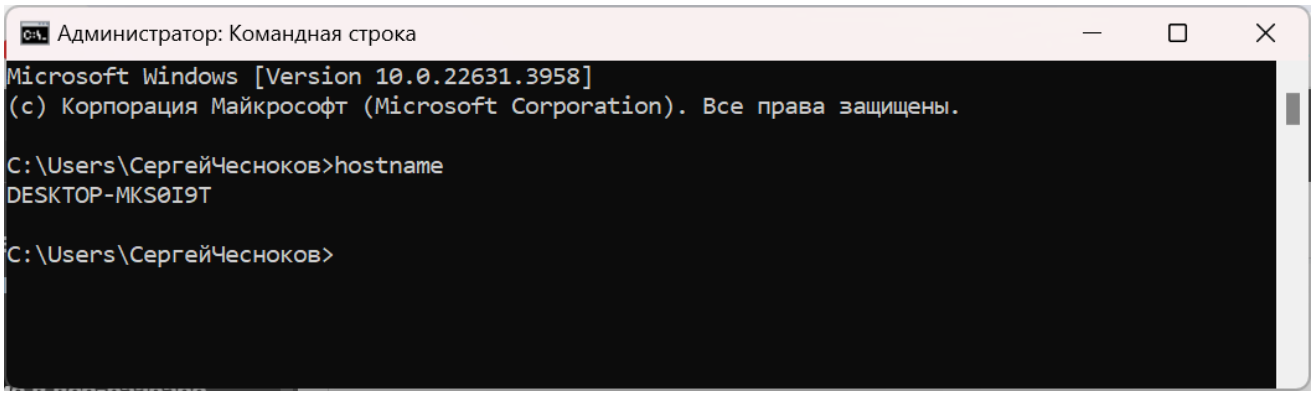


Рис. 1.14. Результат выполнения команды hostname

Команда systeminfo - *предоставляет полный обзор конфигурации ПК. Эта команда раскрывает обширные сведения, такие как версия Windows, имя хоста, идентификатор продукта, тип и количество ЦП, конфигурация ОЗУ, информация о сетевой карте и установленные исправления. Вывод довольно обширен и охватывает важные системные характеристики.*

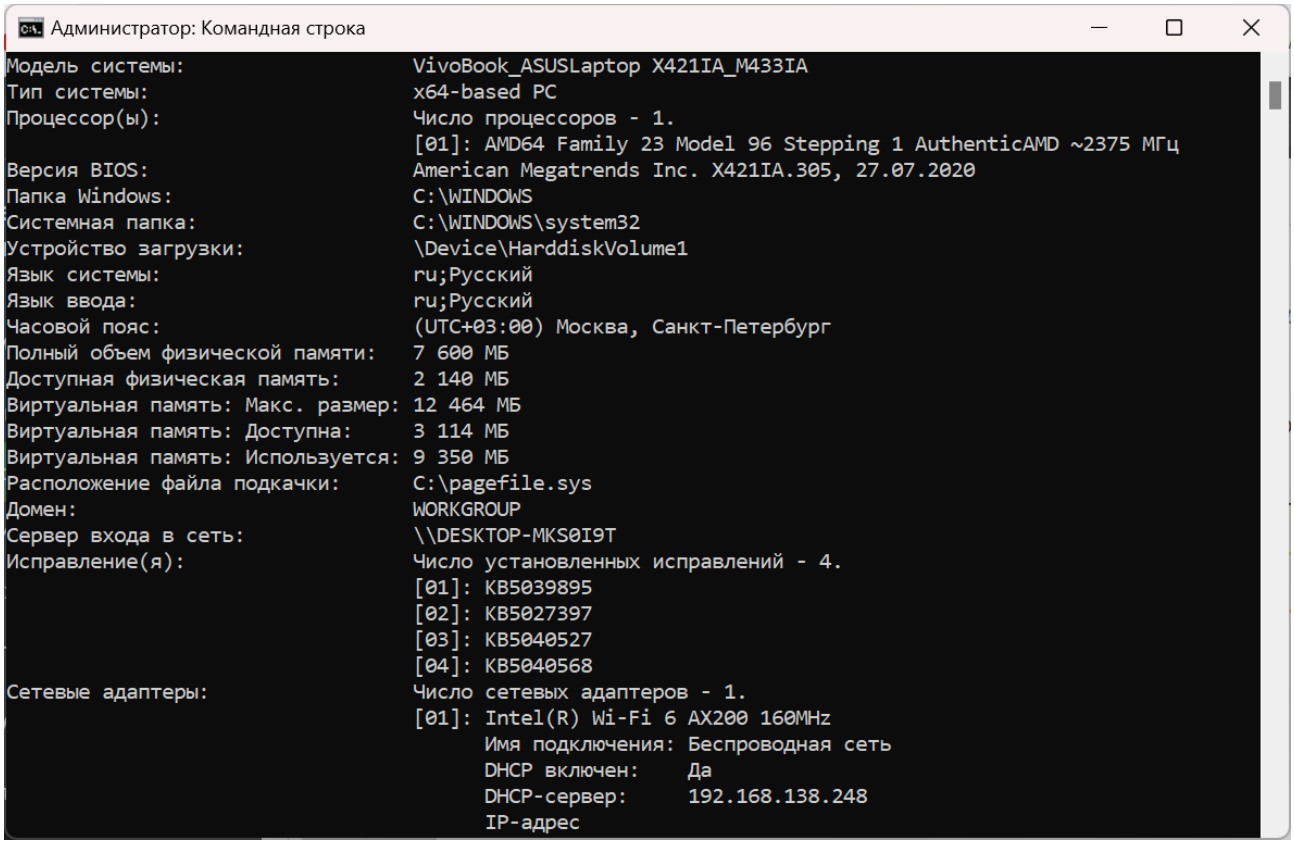


Рис. 1.15. Результат выполнения команды systeminfo

В состав TCP/IP входят диагностические утилиты, предназначенные для проверки конфигурации стека и тестирования сетевого соединения.

Таблица 1.2. Применение сетевых утилит

Утилита	Применение
arp	Выводит для просмотра и изменения таблицу трансляции адресов, используемую протоколом разрешения адресов ARP (Address Resolution Protocol - определяет локальный адрес по IP-адресу)
hostname	Выводит имя локального хоста. Используется без параметров.
ipconfig	Выводит значения для текущей конфигурации стека TCP/IP: IP-адрес, маску подсети, адрес шлюза по умолчанию, адреса WINS (Windows Internet Naming Service) и DNS (Domain Name System)
nbtstat	Выводит статистику и текущую информацию по NetBIOS, установленному поверх TCP/IP. Используется для проверки состояния текущих соединений NetBIOS.
netstat	Выводит статистику и текущую информацию по соединению TCP/IP.
nslookup	Осуществляет проверку записей и доменных псевдонимов хостов, доменных сервисов хостов, а также информации операционной системы, путем запросов к серверам DNS.
ping	Осуществляет проверку правильности конфигурирования TCP/IP и проверку связи с удаленным хостом.
route	Модифицирует таблицы маршрутизации IP. Отображает содержимое таблицы, добавляет и удаляет маршруты IP.
tracert	Осуществляет проверку маршрута к удаленному компьютеру путем отправки эхо-пакетов протокола ICMP (Internet Control Message Protocol). Выводит маршрут прохождения пакетов на удаленный компьютер.

Проверка правильности конфигурации TCP/IP

При устранении неисправностей и проблем в сети TCP/IP следует сначала проверить правильность конфигурации TCP/IP. Для этого используется утилита **ipconfig**.

Эта команда полезна на компьютерах, работающих с **DHCP (Dynamic Host Configuration Protocol)**, так как дает пользователям возможность определить, какая конфигурация сети TCP/IP и какие величины были установлены с помощью DHCP.

Синтаксис:

```
ipconfig [/all | /renew[adapter] | /release]
```

Параметры:

```
all - выдает весь список параметров. Без этого ключа отображается только IP-адрес,
    маска и шлюз по умолчанию;
renew[adapter] - обновляет параметры конфигурации DHCP для указанного сетевого
    адаптера;
release[adapter] - освобождает выделенный DHCP IP - адрес;
adapter - имя сетевого адаптера;
displaydns - выводит информацию о содержимом локального кэша клиента DNS, ис-
    пользуемого для разрешения доменных имен.
```

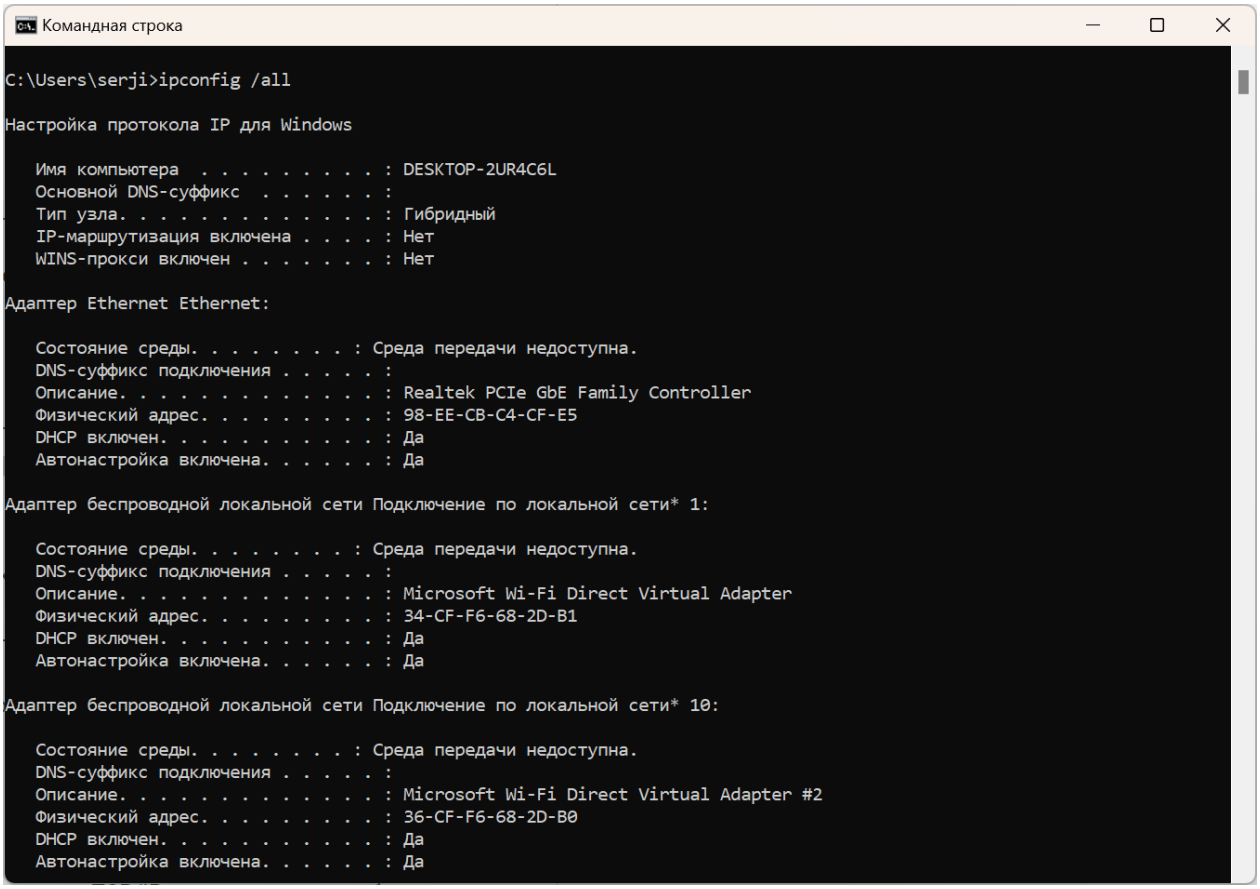


Рис. 1.13. Выполнение команды ipconfig /all

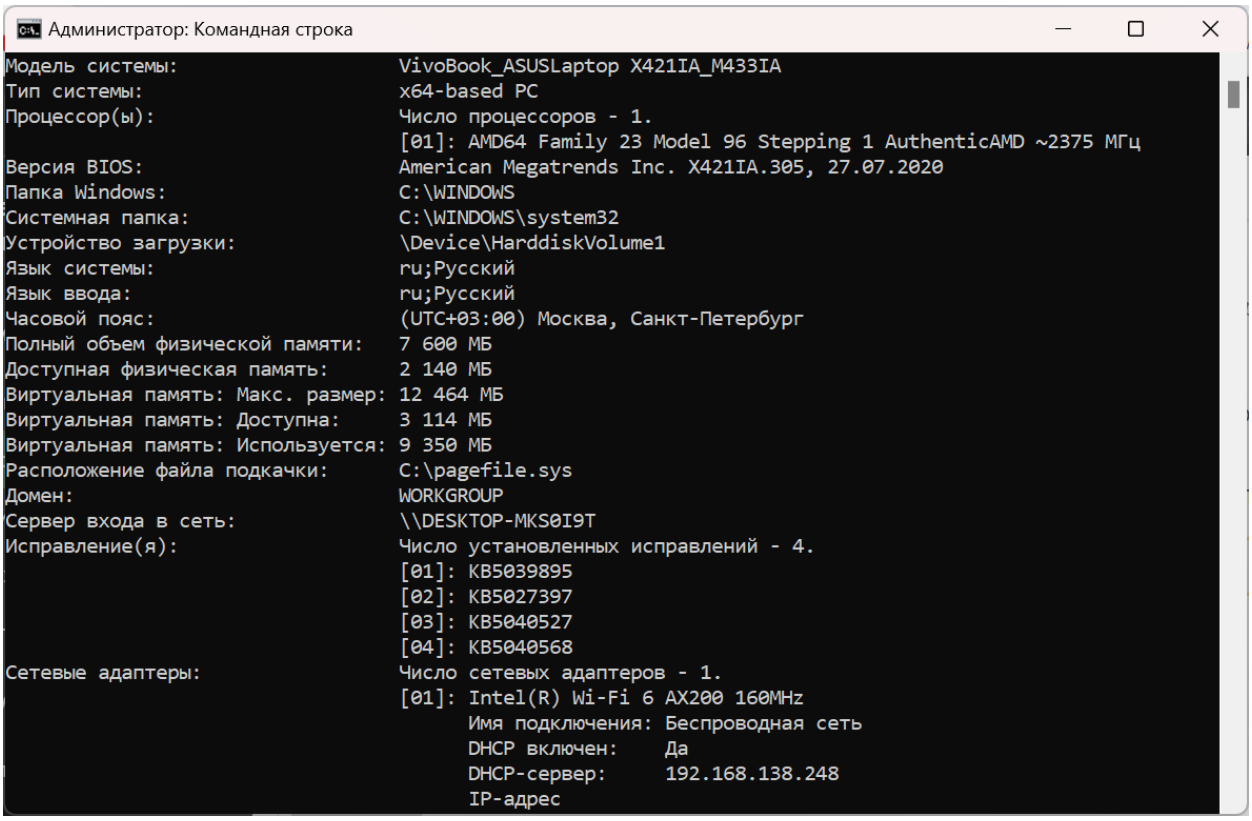


Рис. 1.14. Выполнение команды ipconfig /all (продолжение)

Таким образом, утилита ipconfig позволяет выяснить, инициализирована ли конфигурация и не дублируются ли IP-адреса:

- если конфигурация инициализирована, то появляется IP-адрес, маска, шлюз;
- если IP-адреса дублируются, то маска сети будет 0.0.0.0;
- если при использовании DHCP компьютер не смог получить IP-адрес, то он будет равен 0.0.0.0

Тестирование связи с использованием утилиты ping

Утилита **ping (Packet Internet Grouper)** используется для проверки конфигурирования TCP/IP и диагностики ошибок соединения. Она определяет доступность и функционирование конкретного хоста.

Использование ping лучший способ проверки того, что между локальным компьютером и сетевым хостом существует маршрут. Хостом называется любое сетевое устройство (компьютер, маршрутизатор), обменивающееся информацией с другими сетевыми устройствами по TCP/IP.

Команда ping проверяет соединение с удаленным хостом путем посылки к этому хосту эхо-пакетов ICMP и прослушивания эхо-ответов. Ping ожидает каждый посланный пакет и печатает количество переданных и принятых пакетов.

Каждый принятый пакет проверяется в соответствии с переданным сообщением. Если связь между хостами плохая, из сообщений ping станет ясно, сколько пакетов потеряно.

По умолчанию передается 4 эхо-пакета длиной 32 байта (периодическая последовательность символов алфавита в верхнем регистре). Ping позволяет изменить размер и количество пакетов, указать, следует ли записывать маршрут, который она использует, какую величину времени жизни (ttl) устанавливать, можно ли фрагментировать пакет и т.д. При получении ответа в поле time указывается, за какое время (в миллисекундах).

Посланный пакет доходит до удаленного хоста и возвращается назад. Так как значение по умолчанию для ожидания отклика равно 1 секунде, то все значения данного поля будут меньше 1000 миллисекунд. Если вы получаете сообщение «Request time out» (Превышен интервал ожидания), то, возможно, если увеличить время ожидания отклика, пакет дойдет до удаленного хоста. Это можно сделать с помощью ключа –w.

Ping можно использовать для тестирования как имени хоста (DNS или NetBIOS), так и его IP-адреса. Если ping с IP-адресом выполнилась успешно, а с именем – неудачно, это значит, что проблема заключается в распознавании соответствия адреса и имени, а не в сетевом соединении.

Можно воспользоваться утилитой ping:

1. Для проверки того, что TCP/IP установлен и правильно сконфигурирован на локальном компьютере, в команде ping задается адрес петли обратной связи (loopback address):

```
ping 127.0.0.1
```

Если тест успешно пройден, то вы получите следующий ответ:

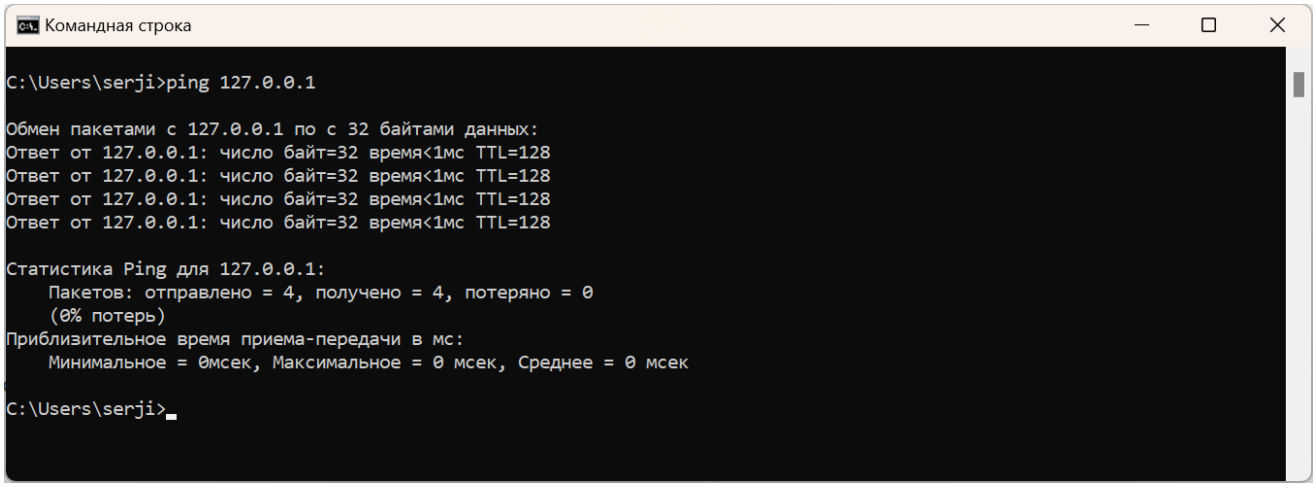


Рис. 1.15. Результат выполнения команды ping для localhost

2. Чтобы убедиться в том, что компьютер правильно добавлен в сеть и IP-адрес не дублируется, используется IP-адрес локального компьютера:

```
ping IP-адрес_локального_хоста
```

3. Чтобы проверить, что шлюз по умолчанию функционирует и что можно установить соединение с любым локальным хостом в локальной сети, задается IP-адрес шлюза по умолчанию:

```
ping IP-адрес_шлюза
```

4. Для проверки возможности установления соединения через маршрутизатор в команде ping задается IP-адрес удаленного хоста:

```
ping IP-адрес_удаленного хоста
```

Синтаксис:

```
ping [-t] [-a] [-n count] [-l length] [-f] [-i ttl] [-v tos] [-r count] [-s count] [ [-j host-list] |[-k host-list] ] [-w timeout] destination-list
```

Параметры:

-t – выполняет команду ping до прерывания. Control-Break - посмотреть статистику и продолжить. Control-C - прервать выполнение команды;
-a - позволяет определить доменное имя удаленного компьютера по его IP-адресу;
- n count -посылает количество пакетов ECHO, указанное параметром count;
- l length - посылает пакеты длиной length байт (максимальная длина 8192 байта);
- f - посылает пакет с установленным флагом «не фрагментировать». Этот пакет не будет фрагментироваться на маршрутизаторах по пути своего следования;
- i ttl - устанавливает время жизни пакета в величину ttl (каждый маршрутизатор уменьшает ttl на единицу);
- v tos - устанавливает тип поля «сервис» в величину tos;
- r count - записывает путь выходящего пакета и возвращающегося пакета в поле записи пути. Count - от 1 до 9 хостов
- s count - позволяет ограничить количество переходов из одной подсети в другую. Count задает максимально возможное количество переходов;
- j host-list - направляет пакеты с помощью списка хостов, определенного параметром host - list. Последовательные хосты могут быть отделены промежуточными маршрутизаторами (гибкая статическая маршрутизация). Максимальное количество хостов в списке, позволенное IP, равно 9;
- k host-list - направляет пакеты через список хостов, определенный в host - list. Последовательные хосты не могут быть разделены промежуточными маршрутизаторами (жесткая статическая маршрутизация). Максимальное количество хостов – 9;
- w timeout - указывает время ожидания (timeout) ответа от удаленного хоста в миллисекундах (по умолчанию – 1сек);
destination-list - указывает удаленный хост, к которому надо направить пакеты ping.

Пример использования утилиты ping:

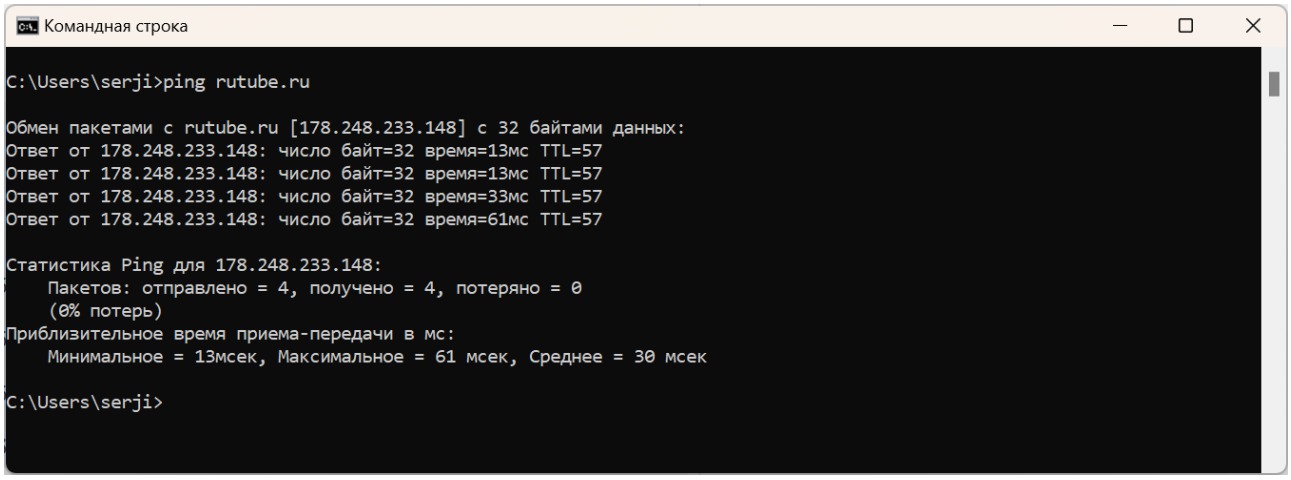


Рис. 1.16. Результат выполнения команды ping для проверки хоста

Тестирование связи с использованием утилиты pathping

Команда pathping *— это более продвинутая команда ping, которая отправляет несколько эхо-запросов маршрутизаторам между источником и пунктом назначения.*

Таким образом, это удобная команда для проверки задержки в сети и определения наличия потери пакетов. Кроме того, эта команда также вычисляет статистику за указанный период времени и выводит результат в табличном формате.

Синтаксис

```
pathping [/n] [/h <maximumhops>] [/g <hostlist>] [/p <Period>] [/q <numqueries> [/w <timeout>] [/i <IPaddress>] [/4 <IPv4>] [/6 <IPv6>][<targetname>]
```

Предоставляет информацию о задержке сети и потерях в сети на промежуточных участках между источником и местом назначения. Эта команда отправляет несколько сообщений echo Request на каждый маршрутизатор между источником и местом назначения в течение определенного периода времени, а затем вычисляет результаты на основе пакетов, возвращенных каждым маршрутизатором. Поскольку эта команда отображает степень потери пакетов на любом заданном маршрутизаторе или канале, вы можете определить, на каких маршрутизаторах или подсетях могут быть проблемы с сетью. При использовании без параметров эта команда отображает справку.

Если тест успешно пройден, то вы получите следующий ответ:

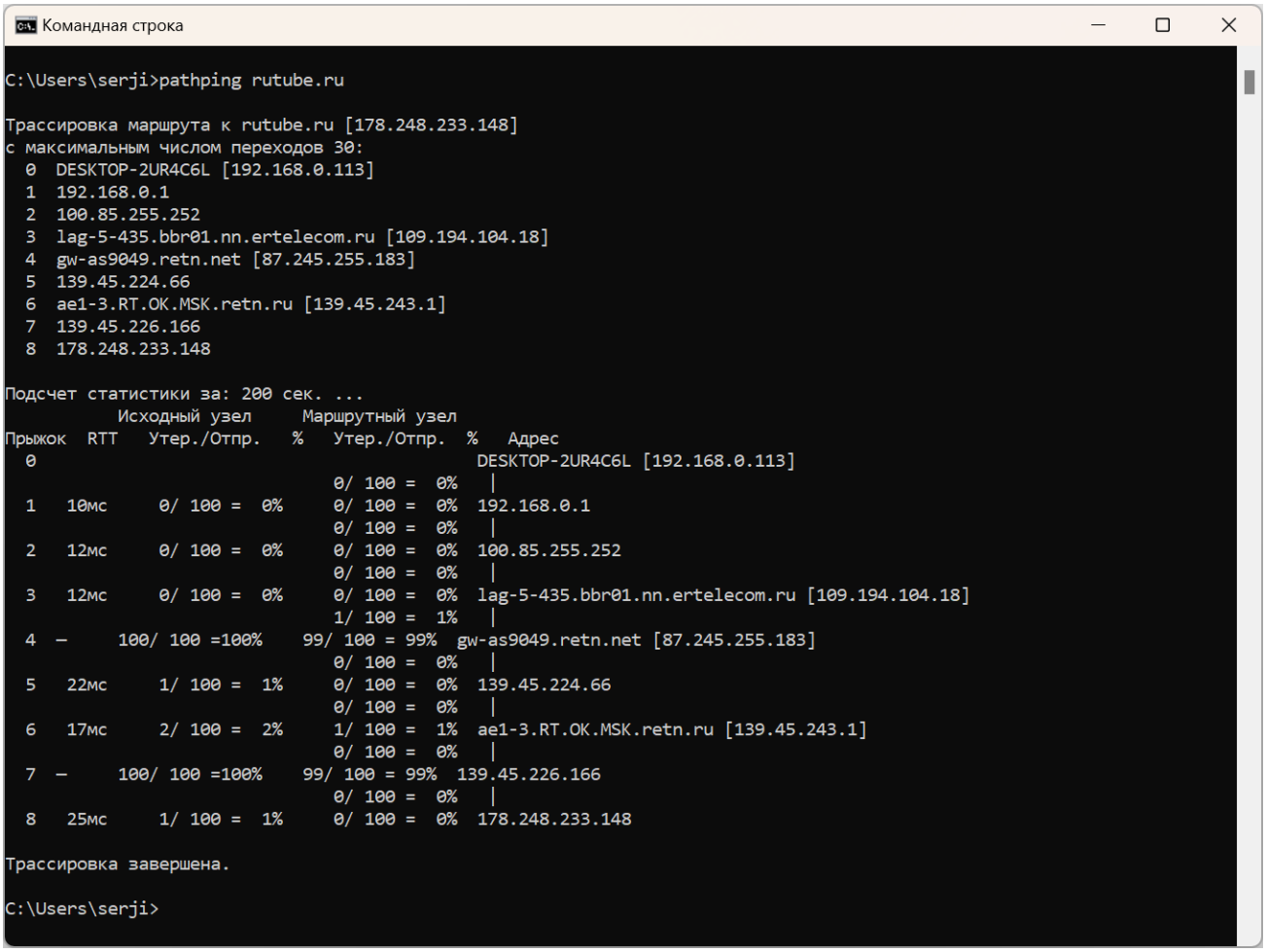


Рис. 1.17. Результат выполнения команды pathping для проверки хоста

Изучение маршрута между сетевыми соединениями с помощью утилиты tracert

Tracert *— это утилита трассировки маршрута. Она использует поле TTL (time – to - live, время жизни) пакета IP и сообщения об ошибках ICMP для определения маршрута от одного хоста до другого.*

Утилита tracert может быть более содержательной и удобной, чем ping, особенно в тех случаях, когда удаленный хост недостижим. С помощью нее можно определить район проблем со связью (у Internet - провайдера, в опорной сети, в сети удаленного хоста) по тому, насколько далеко будет отслежен маршрут. Если возникли проблемы, то утилита выводит на экран звездочки (*), либо сообщения типа «Destination net unreachable», «Destination host unreachable», «Request time out», «Time Exeeded».

Утилита tracert работает следующим образом: посылается по 3 пробных эхо - пакета на каждый хост, через который проходит маршрут до удаленного хоста. На экран при этом выводится время ожидания ответа на каждый пакет (Его можно изменить с помощью параметра - w). Пакеты посылаются с различными величинами времени жизни. Каждый маршрутизатор, встречающийся по пути, перед перенаправлением пакета уменьшает величину TTL на единицу. Таким образом, время жизни является счетчиком точек промежуточной доставки. Когда время жизни пакета достигнет нуля,

предполагается, что маршрутизатор пошлет в компьютер - источник сообщение ICMP “Time Exeeded” (Время истекло). Маршрут определяется путем посылки первого эхо - пакета с TTL=1. Затем TTL увеличивается на 1 в каждом последующем пакете до тех пор, пока пакет не достигнет удаленного хоста, либо будет достигнута максимально возможная величина TTL (по умолчанию 30, задается с помощью параметра - h).

Маршрут определяется путем изучения сообщений ICMP, которые присылаются обратно промежуточными маршрутизаторами. Примечание: некоторые маршрутизаторы просто молча уничтожаю т пакеты с истекшим TTL и не будут видны утилите tracert.

Синтаксис:

```
tracert [-d] [-h maximum_hops] [-j host-list] [-w timeout] имя_целевого_хоста
```

Параметры:

-d - указывает, что не нужно распознавать адреса для имен хостов;
- h maximum_hops - указывает максимальное число ходов для того, чтобы найти цель;
- j host-list - указывает нежесткую статическую маршрутизацию в соответствии с host-list;
- w timeout - указывает, что нужно ожидать ответ на каждый эхо-пакет заданное число мсек.

Пример использования утилиты tracert:

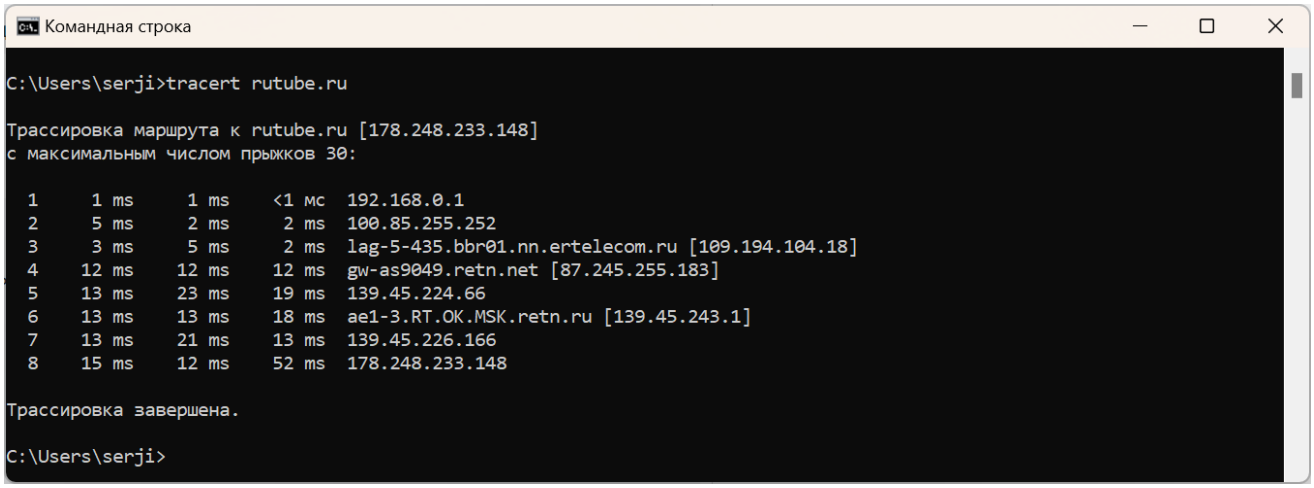


Рис. 1.18. Результат проверки сетевого маршрута

Утилита netstat

Утилита **netstat** позволяет получить статическую информацию по некоторым из **протоколов стека (TCP, UDP, IP, ICMP)**, а также выводит сведения о текущих сетевых соединениях. Особенно она полезна на брандмауэрах, с ее помощью можно обнаружить нарушения безопасности периметра сети.

Синтаксис:

```
netstat [-a] [-e] [-n] [-s] [-p protocol] [-r]
```

Параметры:

-a -выводит перечень всех сетевых соединений и прослушивающихся портов локального компьютера;
-e -выводит статистику для Ethernet-интерфейсов (например, количество полученных и отправленных байт);
-n-выводит информацию по всем текущим соединениям (например, TCP) для всех сетевых интерфейсов локального компьютера. Для каждого соединения выводится информация об IP-адресах локального и удаленного интерфейсов вместе с номерами используемых портов;
-s-выводит статистическую информацию для протоколов UDP, TCP, ICMP, IP. Ключ «/more» позволяет просмотреть информацию постранично;
-r-выводит содержимое таблицы маршрутизации.

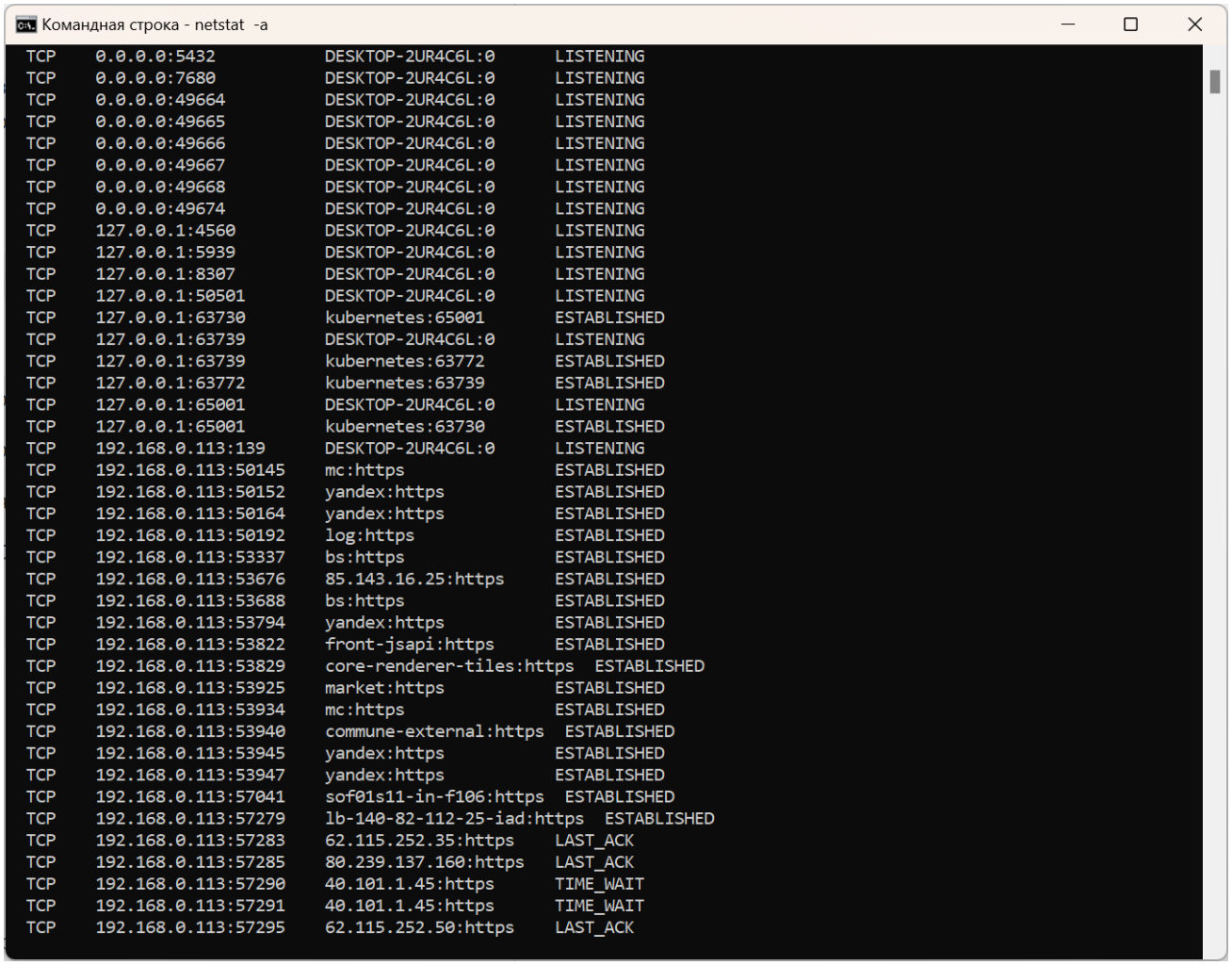


Рис. 1.19. Результат отображения статистической информации

Задания к практической работе

Все задания этой лабораторной работы оформляются в виде отчета в MS. В отчете должен быть выстроен порядок выполнения заданий в соответствии с представленными далее разделами. Результаты выполнения пунктов задания необходимо сопровождать графическими материалами (скриншоты вывода утилит командной строки) там, где это требуется по заданию.

Задание 1. Получение справочной информации по командам

Выведите на экран справочную информацию по утилитам **ipconfig**, **ping**, **tracert**, **hostname**. Команды выполняются на хосте рабочего места. Для этого в командной строке введите имя утилиты **без параметров** или с **/?**. Изучите ключи, используемые при запуске утилит. Результаты с отображением справочной информации и параметров команд/утилит занесите в отчет.

Задание 2. Получение имени хоста

Выведите на экран имя локального хоста с помощью команды **hostname**, на том рабочем месте, где выполняется данная работа. Внесите результат выполнения команды в отчет.

Задание 3. Изучение утилиты ipconfig

Проверьте конфигурацию TCP/IP рабочего места (хоста) с помощью утилиты **ipconfig**. Заполните в отчете таблицу 1.3:

Таблица 1.3. Конфигурация TCP/IP

Имя хоста	
IP-адрес	
Маска подсети	
Основной шлюз	
Используется ли DHCP (адрес DHCP-сервера)	
Описание адаптера	
Физический адрес сетевого адаптера	
Адрес DNS - сервера	
Адрес WINS - сервера	

Сделайте выводы по данному пункту задания:

- Напишите краткое определение каждого из параметров, представленных в таблице.
- Чем полученная информация (занесенная в таблицу) может быть полезна разработчику/администратору сети.

Задание 4. Получение информации о текущих сетевых соединениях и протоколах стека TCP/IP.

С помощью утилиты **netstat** выведите перечень сетевых соединений и статистическую информацию для протоколов UDP, TCP, ICMP, IP. Добавьте результат в отчет и сделайте выводы о том, что иллюстрируют полученные результаты и в чем заключается их практическая ценность.

Задание 5. Выбор ресурса для исследования доступа.

По предоставленному преподавателем варианту подберите сетевой ресурс для дальнейшего исследования согласно таблице 1.4:

Таблица 1.4. Исследование сетевого ресурса

Номер варианта	Класс сетевого ресурса
1	Веб-приложение электронной коммерции
2	Веб-приложение платформы социальной сети
3	Веб-приложение повышения производительности (офисного назначения)
4	Веб-приложение системы управления взаимоотношением с клиентами (CRM)
5	Статичный веб-сайт (сайт-визитка)
6	Динамический веб-сайт (любой направленности)
7	Веб-сайт электронной коммерции
8	Веб-сайт блога

Для выбранного ресурса, в отчет внесите обоснование того, согласно каким критериям, был сделан выбор в пользу сетевого ресурса.

Задание 6. Тестирование связи с помощью утилиты ping

Для выбранного ресурса (в задании 5) с помощью команды **ping** выполните исследования:

a. Проверьте правильность установки и конфигурирования TCP/IP на локальном компьютере.

b. Проверьте, правильно ли добавлен в сеть локальный компьютер и не дублируется ли IP-адрес.
c. Проверьте функционирование шлюза по умолчанию, послав 5 эхо-пакетов длиной 64 байта.
d. Проверьте возможность установления соединения с удаленным хостом.

Добавьте результаты в отчет по каждому из пунктов задания с их объяснением.

Задание 7. Определение пути IP-пакета

С помощью команды **tracert** проверьте для выбранного в задании 5 сетевого ресурса, через какие промежуточные узлы идет сигнал. Отметьте результаты в отчете с объяснением представленной на скриншоте информацией.

Задание 8. Сохранение отчета и отправка на проверку

Создайте на сервисе GitHub публичный репозиторий, в котором организуйте структуру из каталогов, соответствующих перечню практических работ.

Сохраните сформированный отчет по данной практической работе в каталог **lab-1** на GitHub. Прикрепите ссылку на каталог с работой в текущий курс для проверки преподавателем.

Контрольные вопросы

1. Что такое IP-адрес и какую роль он играет в сети?
2. Какое значение имеет маска подсети и как она используется?
3. Что такое DNS и какова его функция в сети?
4. Объясните разницу между статическим и динамическим IP-адресом.
5. Что такое утилита ping и как она работает?
6. Какую информацию можно получить с помощью команды traceroute?
7. Что такое NAT и как он используется в сетях?
8. Какова функция утилиты netstat?
9. Что такое DHCP и как он работает?
10. Каковы основные команды для диагностики сетевых подключений в Windows?
11. Что такое TCP и UDP, и в чем их основные различия?
12. Как использовать утилиту nslookup для проверки DNS?
13. Что такое MAC-адрес и где он используется?
14. Как работает утилита whois и для чего она применяется?
15. Что такое маршрутизация и как она осуществляется в сети?
16. Каковы основные типы сетевых протоколов? Приведите примеры.
17. Что такое "шум" в контексте сетевой передачи данных?
18. Какова роль сетевых интерфейсов в компьютере?
19. Что такое "ping" и как его можно использовать для проверки доступности узла в сети?
20. Какова разница между локальной и глобальной адресацией в сети Интернет?

ПРАКТИЧЕСКАЯ РАБОТА 2. АНАЛИЗ ПРОТОКОЛА HTTP

Цели работы

- **Изучение основ работы протокола HTTP**: понять основные принципы функционирования HTTP, его методы (GET, POST и др.) и структуру запросов и ответов.
- **Овладение навыками работы с Wireshark**: научиться использовать Wireshark для захвата и анализа сетевого трафика.
- **Анализ HTTP-запросов и ответов**: научиться идентифицировать и интерпретировать HTTP-запросы и ответы в захваченном трафике.
- **Изучение кодов состояния HTTP**: ознакомиться с различными кодами состояния ответов HTTP и их значениями.
- **Исследование заголовков HTTP**: понять роль заголовков в HTTP-запросах и ответах, изучить, как они влияют на поведение клиента и сервера.

Теоретическая часть

2.1. Протокол HTTP

Протокол передачи гипертекста (Hypertext Transfer Protocol, HTTP) *— это протокол прикладного уровня для распределенных информационных систем гипермедиа для совместной работы. Протокол передачи гипертекста — это текстовый протокол* ***клиент-сервер запрос-ответ****.*

HTTP-клиент (например, веб-браузер, такой как Mozilla) выполняет HTTP-запрос к HTTP-серверу (например, Apache HTTP-серверу), который в ответ выдает HTTP-ответ. Заголовок протокола HTTP является текстовым, где заголовки записываются в текстовых строках. HTTP — это протокол связи на базе TCP/IP, который используется для доставки данных во всемирную паутину, таких как HTML-файлы, файлы изображений, результаты запросов и так далее. Порт по умолчанию — **TCP 80**, но можно использовать и другие порты. Он обеспечивает стандартизированный способ взаимодействия компьютеров между собой.

Спецификация HTTP определяет, как формируются и отправляются на сервер данные запросов клиентов, а также как серверы реагируют на эти запросы. Версия HTTP/1.1 позволяет организовать конвейеризацию клиент-серверных соединений, благодаря чему можно отправлять несколько запросов (часто в одном пакете), не дожидаясь ответа от сервера. Единственное ограничение — сервер должен возвращать ответы в том же порядке, в котором они были получены. Это обеспечивает большую эффективность, особенно при повторной проверке.

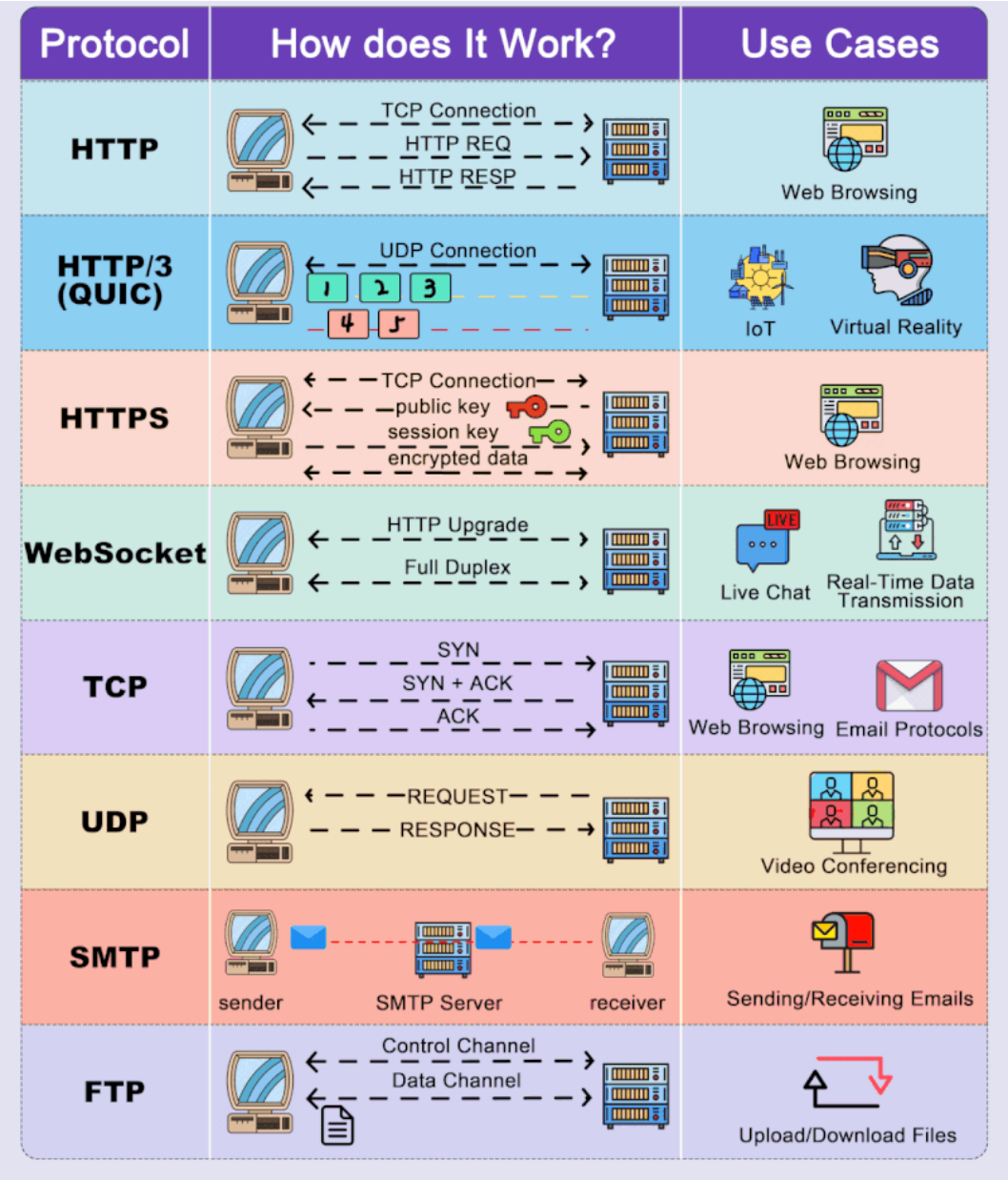


Рис. 2.1. Иллюстрация работы протоколов

Также доступен зашифрованный вариант под названием *HTTPS*. Он часто используется, когда необходима конфиденциальность данных, например, при использовании онлайн-банкинга. Протокол HTTPS на самом деле представляет собой два протокола, работающих друг над другом. Первый протокол — это протокол безопасности, такой как SSL, TLS или PCT. Второй протокол, работающий поверх этого протокола безопасности, — это *HTTP*.

URL-адреса, начинающиеся с https://, на самом деле являются лишь сокращенной записью для конечного пользователя. Веб-браузер считывает схему URI (https://), инициирует протокол безопасности на сервере и после установления этого защищенного соединения отправляет по нему HTTP-запрос с URI, указанным в запросе.

2.2. Функции HTTP протокола

Существуют три основные функции, которые делают HTTP простым, но мощным протоколом:

- **HTTP работает без установления соединения:** HTTP-клиент, т.е. браузер, инициирует HTTP-запрос, и после того, как запрос выполнен, клиент ожидает ответа. Сервер обрабатывает запрос и отправляет ответ обратно, после чего клиент отключает соединение. Таким образом, клиент и сервер знают друг о друге только во время текущего запроса и ответа. Дальнейшие запросы выполняются при новом соединении, как будто клиент и сервер являются новыми друг для друга.
- **HTTP не зависит от носителя:** это означает, что любой тип данных может быть отправлен по HTTP, если и клиент, и сервер знают, как обрабатывать содержимое данных. Как клиенту, так и серверу требуется указать тип контента, используя соответствующий MIME-тип.
- **HTTP не имеет состояния:** как упоминалось выше, HTTP не имеет соединения и является прямым результатом того, что HTTP является протоколом без состояния. Сервер и клиент знают друг о друге только во время текущего запроса. После этого они оба забывают друг о друге. Из-за такой природы протокола ни клиент, ни браузер не могут сохранять информацию между различными запросами на веб-страницах.

2.3. Зависимость от других протоколов и параметры HTTP

- **MIME_multipart**: HTTP использует MIME_multipart для кодирования своих сообщений.
- **TCP**: обычно HTTP использует TCP в качестве транспортного протокола. Известный порт TCP для трафика HTTP — **80**. HTTP-прокси часто использует другой порт; типичные значения — **81**, **3128**, **8000** и **8080**. Однако HTTP может использовать и другие транспортные протоколы.

Рассмотрим основные параметры HTTP:

Версия HTTP. В HTTP для обозначения версий протокола используется схема нумерации вида **<major>.<minor>**. Версия HTTP-сообщения указывается в первой строке в поле **HTTP-Version**. Общий синтаксис указания номера версии в HTTP выглядит следующим образом:

```
HTTP-Version   = "HTTP" "/" 1* DIGIT "."1*DIGIT
```

Например, HTTP/1.0 или HTTP/2.0.

Унифицированные идентификаторы ресурсов (URI). Это просто строка символов, которая не учитывает регистр и содержит имя, местоположение и другую информацию, необходимую для идентификации ресурса, такого как веб-сайт или веб-служба. Общий синтаксис URI, который используется для HTTP, выглядит так:

```
URI = "http:" "//" host [ ":" port ] [ abs_path [ "?" query ]]
```

Здесь, если **порт** пуст или не указан, для HTTP предполагается порт 80, а пустой **abs_path** эквивалентен **abs_path** "/". Символы, отличные от символов в наборах **reserved** и **unsafe**, эквивалентны их кодировке ""%" HEX HEX".

Например, следующие три URI эквивалентны: http://abc.com:80/~smith/home.html, http://ABC.com/%7Esmith/home.html и http://ABC.com:/%7esmith/home.html.

Форматы даты / времени. Все без исключения метки даты и времени HTTP должны быть представлены по среднему времени по Гринвичу (GMT). HTTP-приложениям разрешено использовать любое из следующих трех представлений меток даты и времени:

```
Sun, 06 Nov 1994 08:49:37 GMT; RFC 822, updated by RFC 1123
Sunday, 06-Nov-94 08:49:37 GMT; RFC 850, obsoleted by RFC 1036
Sun Nov 6 08:49:37 1994; ANSI C's asctime() format
```

Наборы символов. Наборы символов используются, чтобы указать, какие символы предпочитает клиент. Если клиент не указал свой набор символов, то по умолчанию используется US-ASCII. Если клиент указал несколько наборов символов, то они перечисляются через запятую. Допустимые наборы символов: US-ASCII, ISO-8859-1 или ISO-8859-7.

Кодировки контента. Кодировка содержимого означает, что перед отправкой по сети содержимое было закодировано с помощью определённого алгоритма. Это позволяет сжать или преобразовать документ без потери его идентичности. Все значения для кодирования содержимого не учитывают регистр символов. В HTTP/1.1 эти значения используются в полях заголовка *Accept-Encoding* и *Content-Encoding*.

В версии HTTP/2, в отличие от своего предшественника HTTP/1.1, внедрено несколько улучшений и изменений для повышения эффективности передачи данных и уменьшения задержек. Одним из ключевых нововведений в HTTP/2 является использование бинарной кодировки вместо текстовой. Вот основные аспекты кодировки в HTTP/2:

1. **Бинарные кадры.** Все данные в HTTP/2 передаются в виде бинарных кадров. Это повышает эффективность передачи, упрощает разбор данных и снижает количество ошибок, связанных с текстовой кодировкой.
2. **Фреймы (кадры).** В HTTP/2 данные разбиваются на фреймы, которые являются минимальными единицами передачи. Существуют различные типы фреймов, такие как:
    - **DATA.** Передает содержимое сообщения.
    - **HEADERS.** Передает метаинформацию, такую как заголовки HTTP.
    - **PRIORITY.** Управляет приоритетом потоков.
    - **RST_STREAM.** Сбрасывает поток.
    - **SETTINGS.** Обмен параметрами конфигурации между клиентом и сервером.
    - **PUSH_PROMISE.** Позволяет серверу отправлять данные, которые клиент может запросить в будущем.

- **PING.** Проверяет состояние соединения.
- **GOAWAY.** Уведомляет о намерении закрыть соединение.

3. **HPACK.** Заголовки в HTTP/2 сжимаются с использованием алгоритма HPACK, что значительно уменьшает их размер. Это особенно полезно для уменьшения накладных расходов при передаче повторяющихся заголовков.
4. **Мультиплексирование.** HTTP/2 позволяет нескольким потокам данных существовать одновременно в одном TCP-соединении. Это устраняет необходимость в установке нескольких соединений для параллельной загрузки ресурсов, как это было в HTTP/1.1, что в свою очередь уменьшает задержки и улучшает производительность.

Типы носителей. HTTP использует типы интернет-медиа в полях заголовка *Content-Type* и *Accept* для обеспечения открытого и расширяемого ввода данных и согласования типов. Все значения типа носителя регистрируются в Управлении по присвоению номеров в Интернете (IANA). Общий синтаксис для указания типа носителя выглядит так:

```
media-type = type "/" subtype *( ";" parameter )
```

Имена атрибутов типа, подтипа и параметра не зависят от регистра. Например, *Accept: image/gif*

Языковые теги. В протоколе HTTP для обозначения языков используются языковые теги в полях *Accept-Language* и *Content-Language*. Эти теги состоят из одной или нескольких частей: основного языкового тега и, при необходимости, пустой серии подтегов.

```
language-tag = primary-tag *( "-" subtag )
```

В теге не допускаются пробелы, и все теги не чувствительны к регистру. Примеры тегов включают: *ru, ru-RU, en, en-US, en-cockney, i-cherokee, x-pig-latin*, где любой двухбуквенный первичный тег представляет собой аббревиатуру языка *ISO-639*, а любой двухбуквенный начальный подтег представляет собой код страны *ISO-3166*.

2.2. HTTP-запросы

HTTP-клиент отправляет HTTP-запрос серверу в виде сообщения-запроса, которое включает следующий формат:

- **Строка запроса**.
- Ноль или более полей заголовка (General | Request | Entity), за которыми следует *CRLF*.
- **Пустая строка** (т.е. строка, перед CRLF которой ничего не стоит), указывающая конец полей заголовка.
- Необязательно *текст сообщения*.

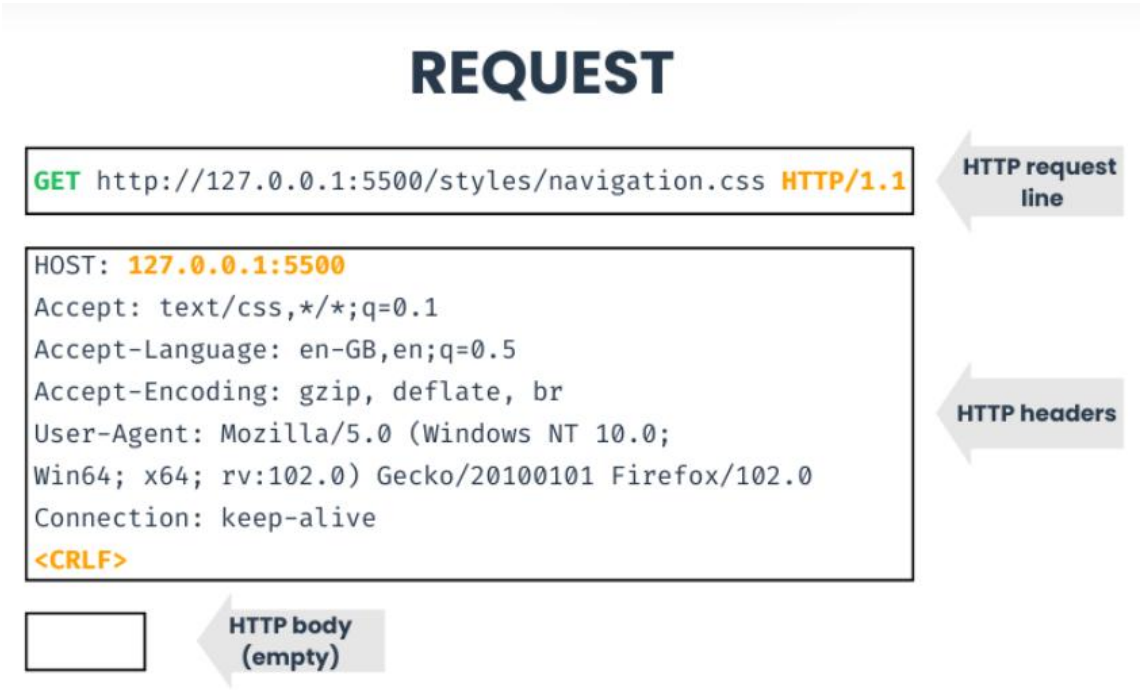


Рис. 2.2. Структура HTTP-запроса

Строка запроса. Строка запроса начинается с маркера метода, за которым следует URI запроса и версия протокола, и заканчивается CRLF. Элементы разделяются пробелом SP.

```
Request-Line = Method SP Request-URI SP HTTP-Version CRLF
```

Метод запроса. Он указывает метод, который должен быть выполнен на ресурсе, идентифицированном данным *запросом-URI*. Метод чувствителен к регистру и всегда должен указываться в верхнем регистре. В следующей таблице перечислены все поддерживаемые методы в HTTP / 1.1.

Методы:

- **GET -** используется для получения информации с данного сервера с использованием данного URI. Запросы, использующие GET, должны извлекать только данные и не должны оказывать никакого другого воздействия на данные.
- **HEAD** - то же, что и GET, но он передает только строку состояния и раздел заголовка.
- **POST -** используется для отправки данных на сервер, например, информации о клиенте, загрузки файла и т.д. С использованием HTML-форм.
- **PUT** - заменяет все текущие представления целевого ресурса загруженным контентом.
- **DELETE** - удаляет все текущие представления целевого ресурса, заданные URI.
- **CONNECT** - устанавливает туннель к серверу, идентифицированному заданным URI.
- **OPTIONS -** описание параметров связи для целевого ресурса.

- **TRACE** - выполняет проверку обратного цикла сообщения вместе с указанием пути к целевому ресурсу.

URI запроса. Request-URI является унифицированным идентификатором ресурса и идентифицирует ресурс, к которому следует применить запрос. Ниже приведены наиболее часто используемые формы для указания URI:

```
Request-URI = "*" | absoluteURI | abs_path | authority
```

Звездочка * используется, когда HTTP-запрос применяется не к определенному ресурсу, а к самому серверу, и допускается только тогда, когда используемый метод не обязательно применим к ресурсу. Например:
OPTIONS * HTTP/1.1. *absoluteURI* используется, когда HTTP-запрос направляется прокси-серверу. Прокси-серверу предлагается переслать запрос или услугу из допустимого кэша и вернуть ответ. Например:
GET http://www.example.org/pub/WWW/Hello.html HTTP/1.1

Наиболее распространенной формой запроса URI является та, которая используется для идентификации ресурса на исходном сервере или шлюзе. Например, клиент, желающий получить ресурс непосредственно с исходного сервера, должен создать TCP-соединение с портом 80 хоста "www.example.org" и отправить следующие строки:

```
GET /pub/WWW/Hello.html HTTP/1.1
Host: www.example.org
```

Обратите внимание, что абсолютный путь не может быть пустым; если в исходном URI его нет, он должен быть указан как "/" (корневой каталог сервера).

Поля заголовка запроса. Они позволяют клиенту передавать дополнительную информацию о запросе и о самом клиенте серверу. Эти поля действуют как модификаторы запроса. Вот список некоторых важных полей заголовка запроса, которые могут быть использованы в зависимости от требований: *Accept-Charset, Accept-Encoding, Accept-Language, Authorization, Expect, From, Host, If-Match, If-Modified-Since, If-None-Match, If-Range, If-Unmodified-Since, Max-Forwards, Proxy-Authorization, Range, Referer, TE, User-Agent*. Допускается ввести свои пользовательские поля на случай, если разрабатывается свой собственный клиент и веб-сервер.

Теперь давайте объединим все это, чтобы сформировать HTTP-запрос для получения страницы *hello.htm* с веб-сервера, работающего на *example.com*.

```
GET /hello.htm HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.example.com
Accept-Language: en-us
Accept-Encoding: gzip, deflate
Connection: Keep-Alive
```

Здесь мы не отправляем никаких данных запроса на сервер, потому что мы извлекаем обычную HTML-страницу с сервера.

Connection *— это общий заголовок, а остальные заголовки являются заголовками запроса.*

В следующем примере показано, как отправить данные формы на сервер, используя тело сообщения запроса:

```
POST /cgi-bin/process.cgi HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: length
Accept-Language: en-us
Accept-Encoding: gzip, deflate
Connection: Keep-Alive
licenseID=string&content=string&/paramsXML=string
```

Здесь указанный URL */cgi-bin/process.cgi* будет использоваться для обработки переданных данных и, соответственно, будет возвращен ответ. Здесь *content-type* сообщает серверу, что переданные данные представляют собой простые данные веб-формы, а *length* будет фактической длиной данных, помещенных в тело сообщения. В следующем примере показано, как можно передавать обычный XML на свой веб-сервер:

```
POST /cgi-bin/process.cgi HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.tutorialspoint.com
Content-Type: text/xml; charset=utf-8
Content-Length: length
Accept-Language: en-us
Accept-Encoding: gzip, deflate
Connection: Keep-Alive

<?xml version="1.0" encoding="utf-8"?>
<string xmlns="http://clearforest.com/">string</string>
```

2.3. HTTP-ответы

После получения и интерпретации сообщения запроса сервер отвечает ответным сообщением HTTP:

- Строка состояния.

- Ноль или более полей заголовка (General | Response | Entity), за которыми следует CRLF.
- Пустая строка (т.е. строка, перед CRLF которой ничего не стоит), указывающая конец полей заголовка.
- Необязательно текст сообщения.

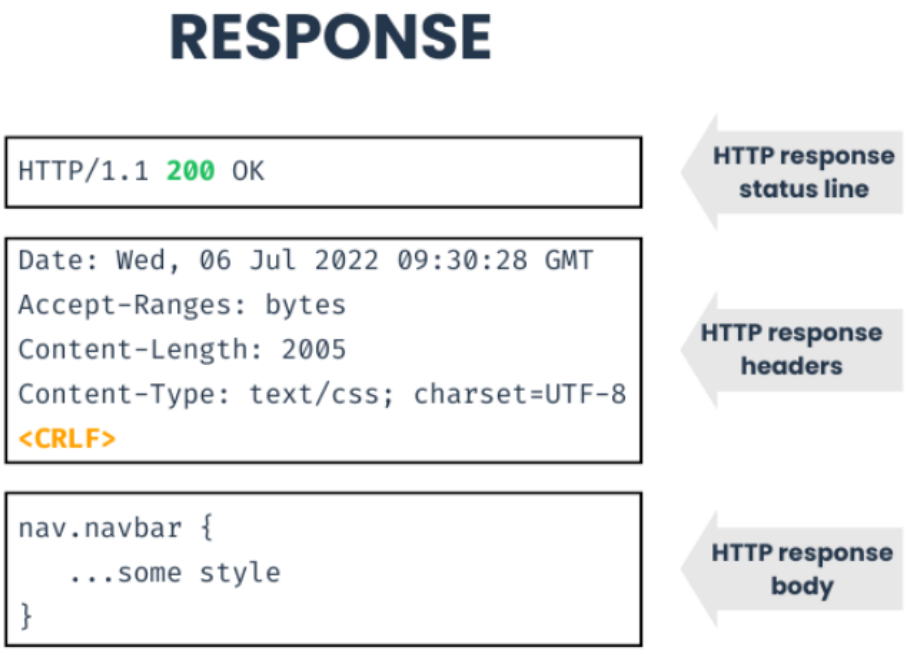


Рис. 2.3. Структура HTTP-ответа

Строка состояния сообщения. Строка состояния состоит из версии протокола, за которой следует цифровой код состояния и связанная с ним текстовая фраза. Элементы разделяются пробелом SP.

```
Status-Line = HTTP-Version SP Status-Code SP Reason-Phrase CRLF
```

Версия HTTP. Сервер, поддерживающий HTTP версии 1.1, вернет следующую информацию о версии:

```
HTTP-Version = HTTP/1.1
```

Код состояния. Элемент *Status-Code* представляет собой трехзначное целое число, где первая цифра *Status-Code* определяет класс ответа, а последние две цифры не играют никакой категоризирующей роли. Для первой цифры имеется 5 значений:

- **Informational:** запрос был получен и процесс продолжается.
- **Success:** действие было успешно получено, понято и принято.
- **Redirection:** необходимо предпринять дальнейшие действия для завершения запроса.
- **Client Error:** запрос содержит неправильный синтаксис или не может быть выполнен.

- **Server Error:** серверу не удалось выполнить явно корректный запрос.

Коды состояния HTTP являются гибкими и могут расширяться. HTTP-приложениям не нужно понимать значения всех зарегистрированных кодов состояния. Список всех кодов состояния вы можете найти в отдельной главе для вашего удобства.

Поля заголовка ответа. Позволяют серверу передавать дополнительную информацию об ответе, которая не может быть помещена в строку состояния. Эти поля заголовка предоставляют информацию о сервере и о дальнейшем доступе к ресурсу, определяемому запросом-URI: *Accept-Ranges, Age, ETag, Location, Proxy-Authenticate, Retry-After, Server, Vary, WWW-Authenticate*. Можно ввести свои пользовательские поля на случай, если разрабатывается свой собственный веб-клиент и сервер.

2.4. Анализ сетевых протоколов с использованием Wireshark

Важным компонентом современных вычислений является сетевой анализ. При таком подходе мы можем понять поток данных, передаваемых по сетям, выявить любые проблемы и предвидеть проблемы безопасности. Wireshark — это мощный инструмент, позволяющий записывать и анализировать сетевые данные на тонком уровне. С его помощью можно собирать информацию о производительности сети и поведении протокола.

***Wireshark** — это анализатор сетевых протоколов с открытым исходным кодом, который широко используется для исследований, коммерческого и некоммерческого использования, а также устранения неполадок. Wireshark используется в государственных учреждениях, а также для обучения и образования.*

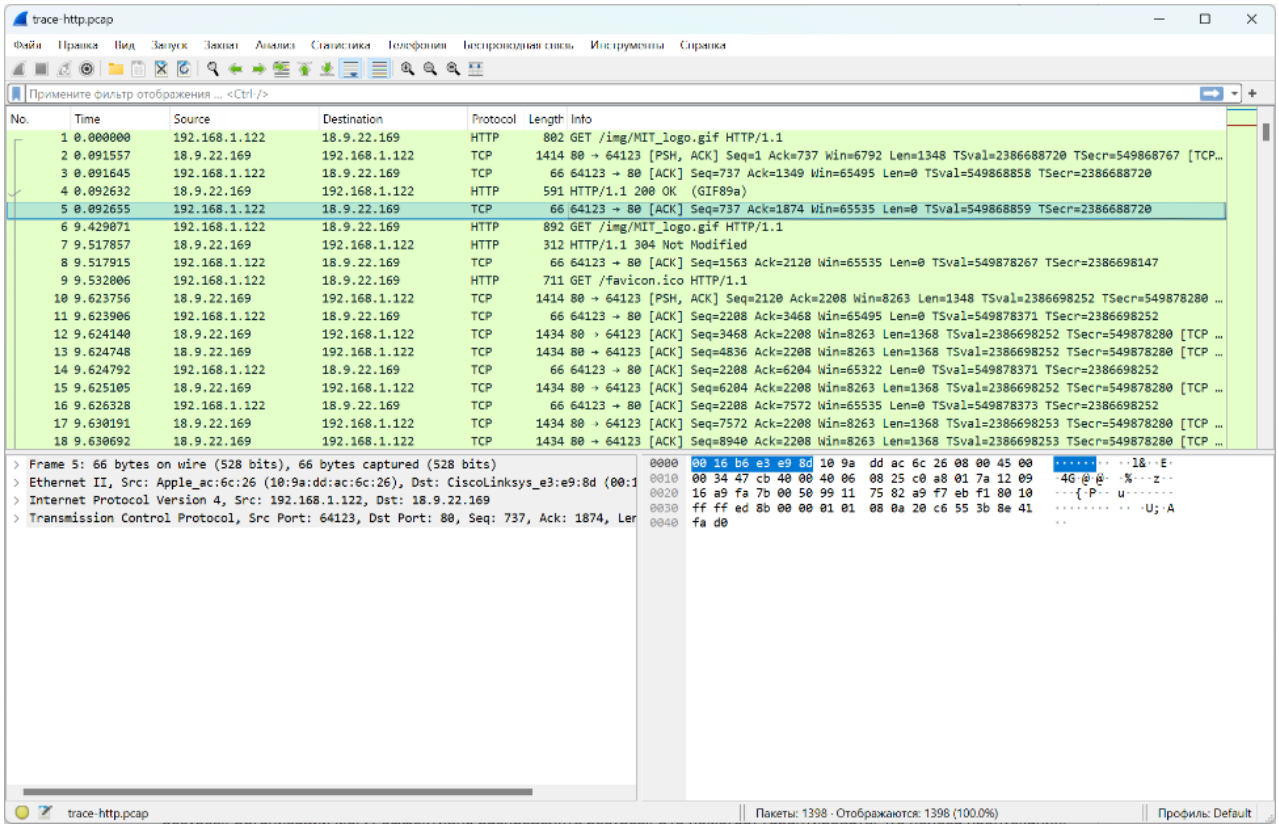


Рис. 2.4. Интерфейс приложения Wireshark

Wireshark *— один из самых надежных и мощных снифферов пакетов с открытым исходным кодом. Это одновременно интерактивный инструмент сниффера пакетов и инструмент анализа.*

Wireshark — самый популярный сниффер пакетов в настоящее время. Одной из причин является его открытая доступность и кроссплатформенная совместимость. Он может работать на Windows, Linux и Mac, удобен для пользователя и прост в захвате и просмотре данных благодаря привлекательному графическому пользовательскому интерфейсу. Wireshark — один из лучших инструментов для перехвата пакетов, проходящих через целевую сеть, а также для чтения содержимого каждого пакета в удобном для восприятия человеком формате, который можно фильтровать и маркировать цветом в соответствии с конкретными требованиями. Он имеет широкий спектр функций, которые в основном используются для устранения неполадок в сети, а также для анализа и тестирования программного обеспечения.

Загрузка и установка

Как уже упоминалось выше, это программное обеспечение с открытым исходным кодом, поэтому оно доступно для бесплатной загрузки. Вы можете загрузить его как для Windows, так и для MacOS с официального сайта Wireshark. Для Linux или других UNIX-подобных систем Wireshark, вероятно, присутствует в репозиториях пакетов. Он предустановлен в ОС Kali Linux.

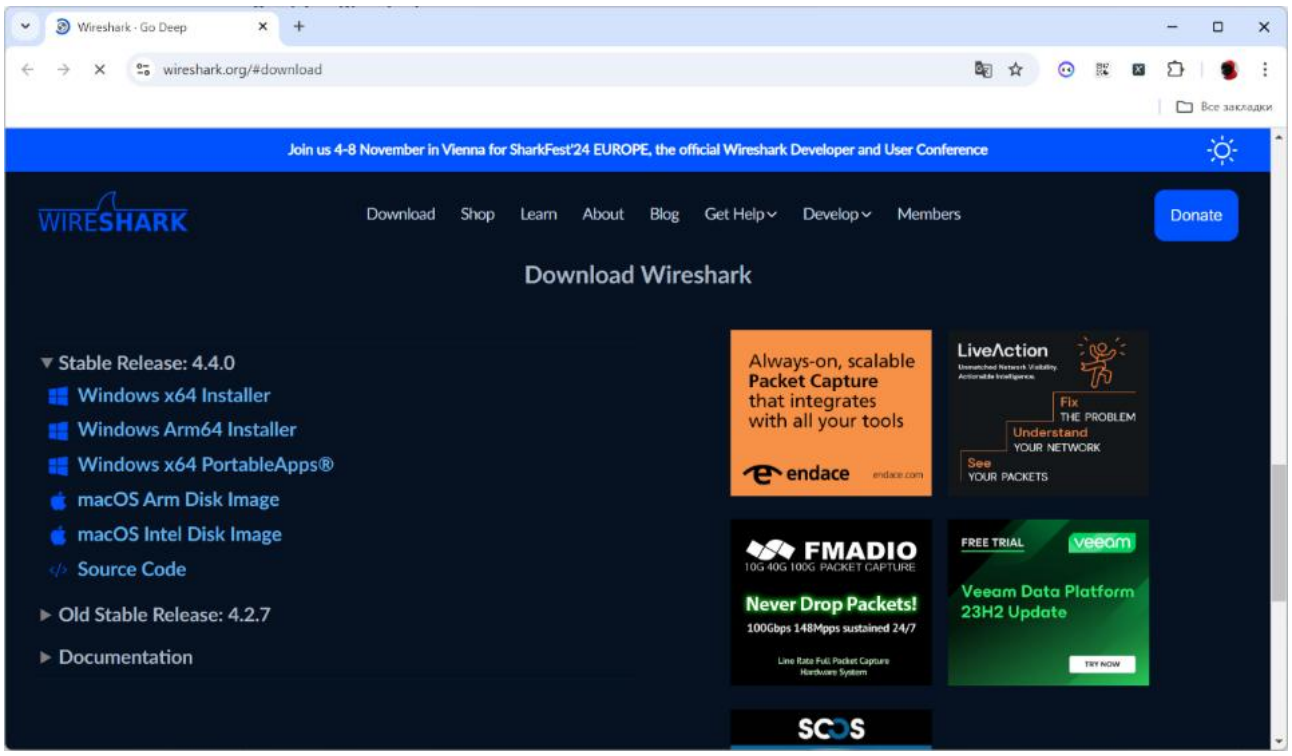


Рис. 2.5. Сайт для скачивания приложения Wireshark

Для захвата трафика (например, при использовании беспроводных сетей) может потребоваться установка драйвера *Npcap* в систему.

Npcap *— это версия библиотеки libpcap для Windows; она включает драйвер для поддержки захвата пакетов. Wireshark использует эту библиотеку для сбора данных сети в реальном времени в Windows.*

Общая информация и материалы для загрузки доступны на веб-сайте Npcap.

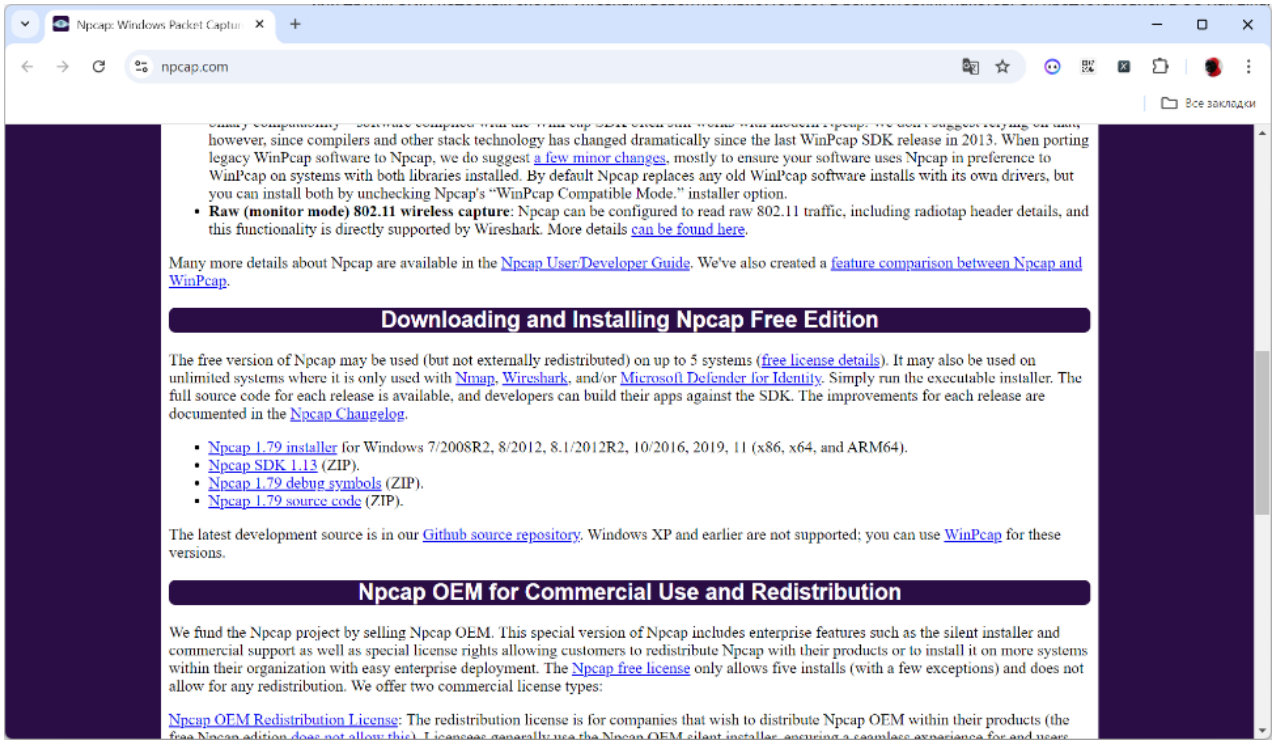


Рис. 2.6. Сайт для скачивания библиотеки Npcap

Графический интерфейс Wireshark

При запуске приложения мы получаем экран приветствия, на котором перечислены различные сетевые интерфейсы, доступные для вашего устройства. В данном случае у нас есть Local Area Connection, Ethernet и Wi-Fi. Маленький график справа от каждого из них отображает поток живого трафика на интерфейсе.

Главное меню в верхней части отображает различные параметры, которыми оснащен Wireshark. Мы рассмотрим, как использовать параметры *«Захват», «Анализ» и «Статистика»*.

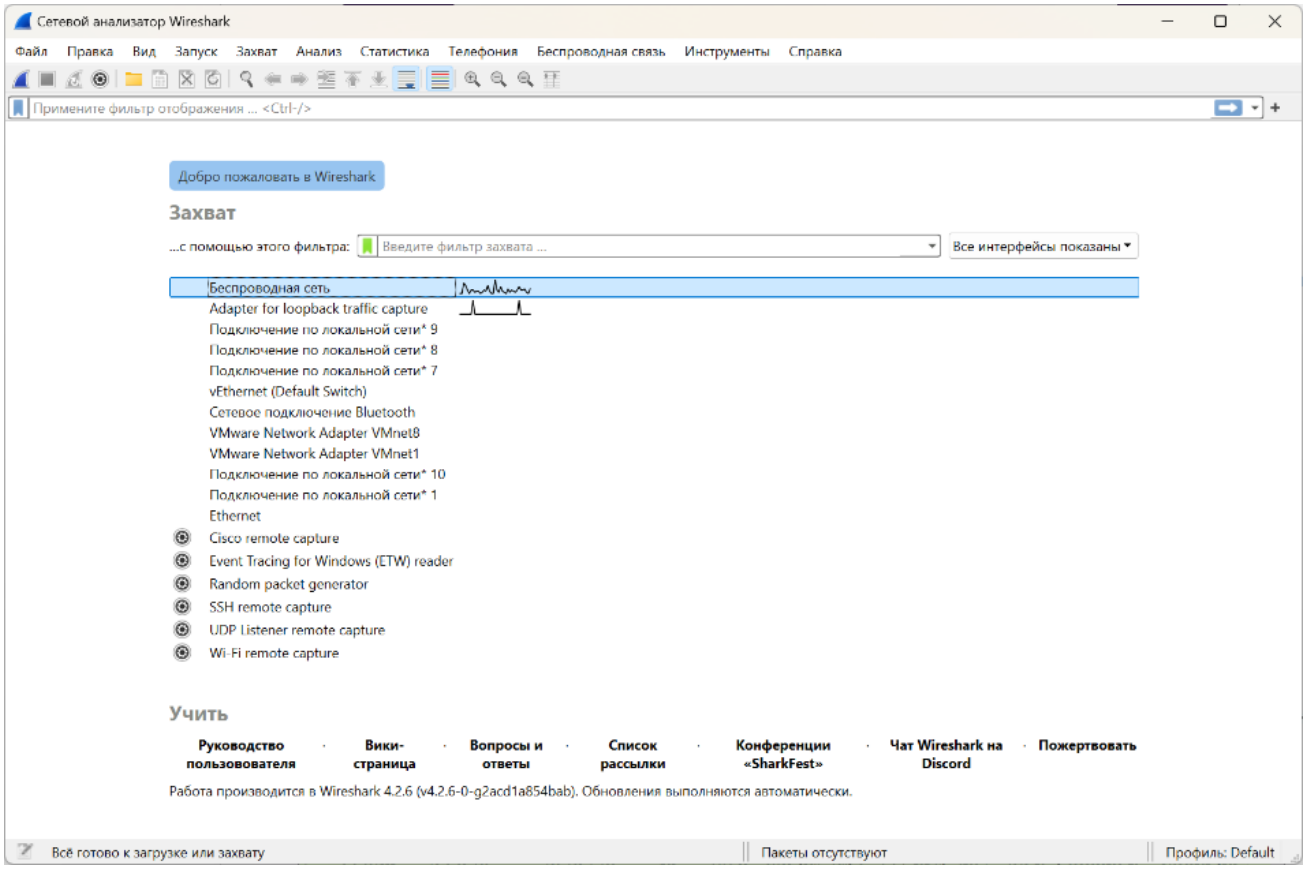


Рис. 2.7. Выбор источника для захвата пакетов в Wireshark

Захват сетевых пакетов

Сетевой сниффинг может использоваться для захвата конфиденциальных данных, таких как учетные данные для входа или для захвата файлов, отправляемых по сети. Различные протоколы, такие как HTTP, Telnet, IMAP, FTP, SMTP и другие, уязвимы для сниффинга.

Чтобы начать захват данных, щелкните по любому сетевому интерфейсу, по которому идет какая-либо прямая передача. При щелчке по захвату в главном меню появится раскрывающееся меню, щелкните по *Пуск***.** Чтобы начать захват с клавиатуры, нажмите *CTRL+E*.

Пакеты низкого уровня, передаваемые по данной конкретной сети, которые Wireshark перехватывает в реальном времени и отображает в цветовой кодировке по умолчанию.

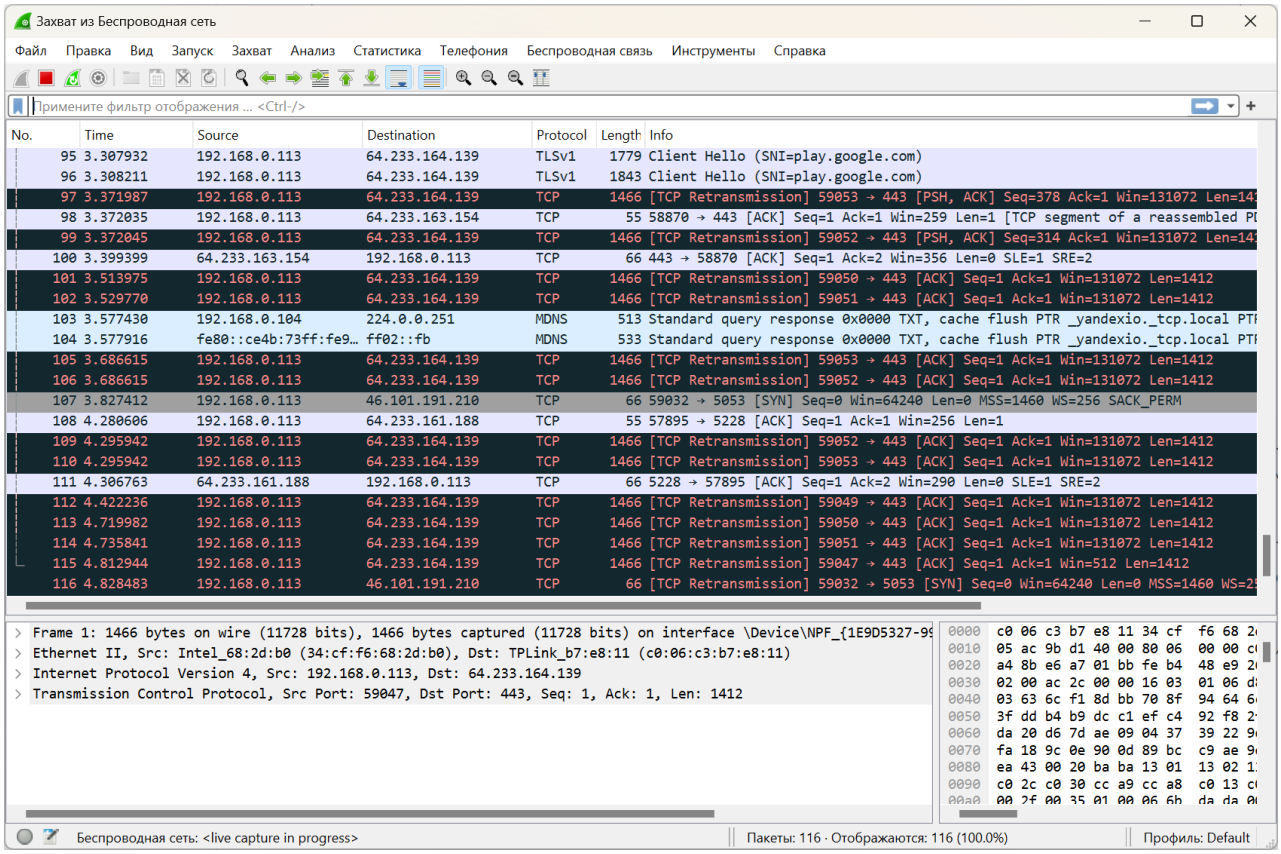


Рис. 2.8. Захват пакетов в Wireshark

Чтобы остановить захват, выберите *Захват, Остановить* или нажмите *CTRL+E*.

Цветовая кодировка

Чтобы узнать, какой протокол подразумевает каждый цвет, перейдите в раздел *Вид, Правила выделения цветом*. Мы также можем изменить код в соответствии со своими предпочтениями или отключить всю цветовую кодировку.

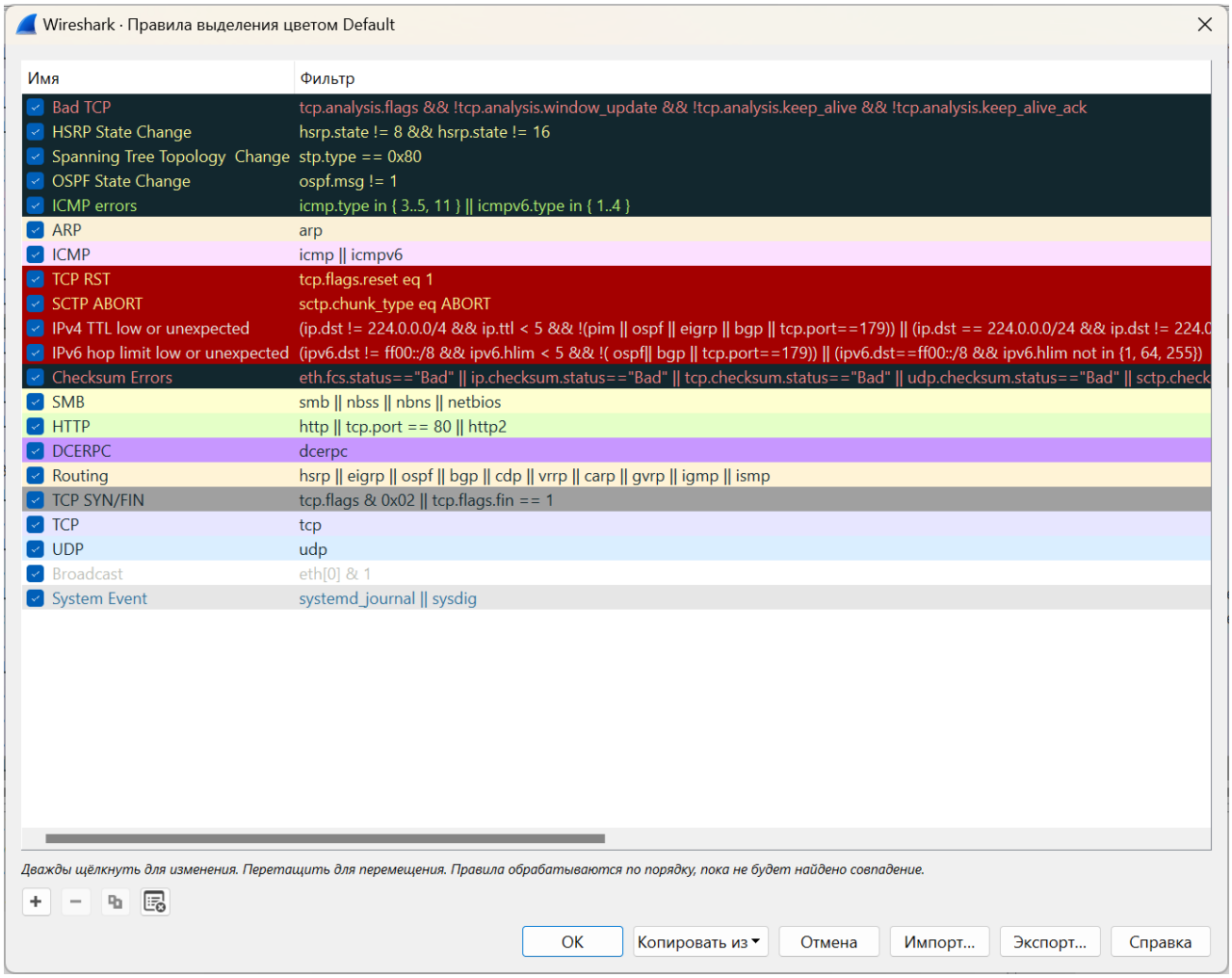


Рис. 2.9. Пример цветовой кодировки

Анализ захваченного контента

При работе с сетью с большим трафиком возможности фильтрации Wireshark оказываются полезными. Мы можем указать в начале, что нужно захватывать только определенные типы пакетов или отфильтровывать их позже. Это можно сделать как с живым захватом, так и с уже сохраненным файлом *.pcapng*.

Фильтры по умолчанию можно увидеть, перейдя в раздел *Анализ, Фильтры отображения*.

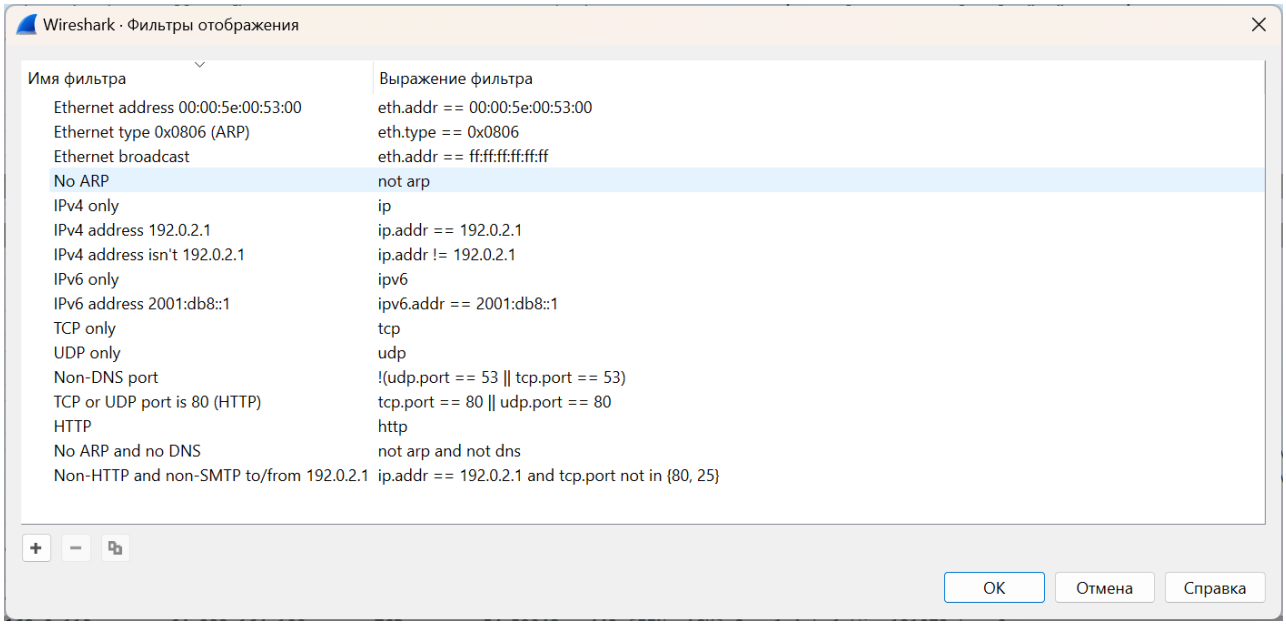


Рис. 2.10. Фильтры отображения в Wireshark

Если мы знаем название фильтра, который хотим применить, нужно начать вводить его в поле «*Применить фильтр отображения*». Оно станет зеленым, если введенные вами данные являются допустимым полем. Мы можем добавить свои фильтры в закладки для будущего или повторного использования, которые можно найти слева от поля ввода.

Например, применение фильтра «icmpv6» к захваченным пакетам.

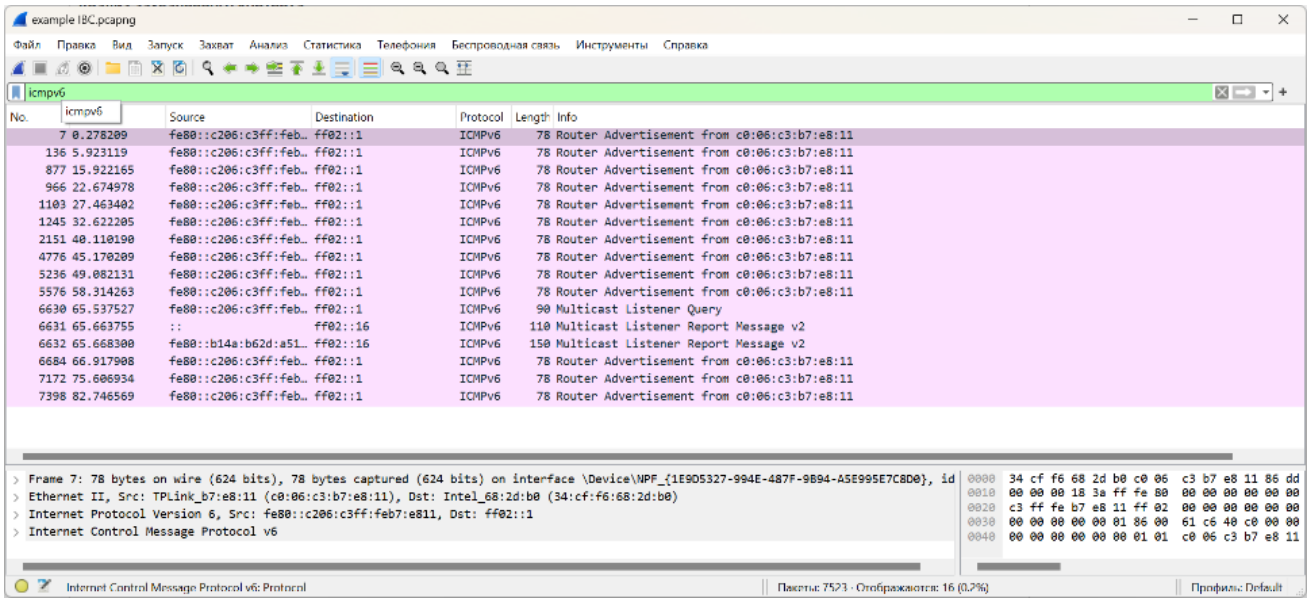


Рис. 2.11. Применение фильтра по icmpv6

Теперь давайте рассмотрим более глубокий анализ отдельных пакетов.

Каждый пакет имеет *временную метку* и *данные о кадре*, которые говорят нам о времени создания пакета и о том, в каком именно кадре он передается. Другие данные

включают адрес источника и назначения. Далее следуют данные отдельного протокола, где пакеты указывают свои заголовки и части полезной нагрузки. Можно получить представление обо всех данных, содержащихся в этом конкретном пакете, и о том, какие другие пакеты связаны с этим пакетом, посмотрев на данные, хранящиеся в заголовке. Справа мы можем увидеть необработанную версию тех же данных в шестнадцатеричном виде.

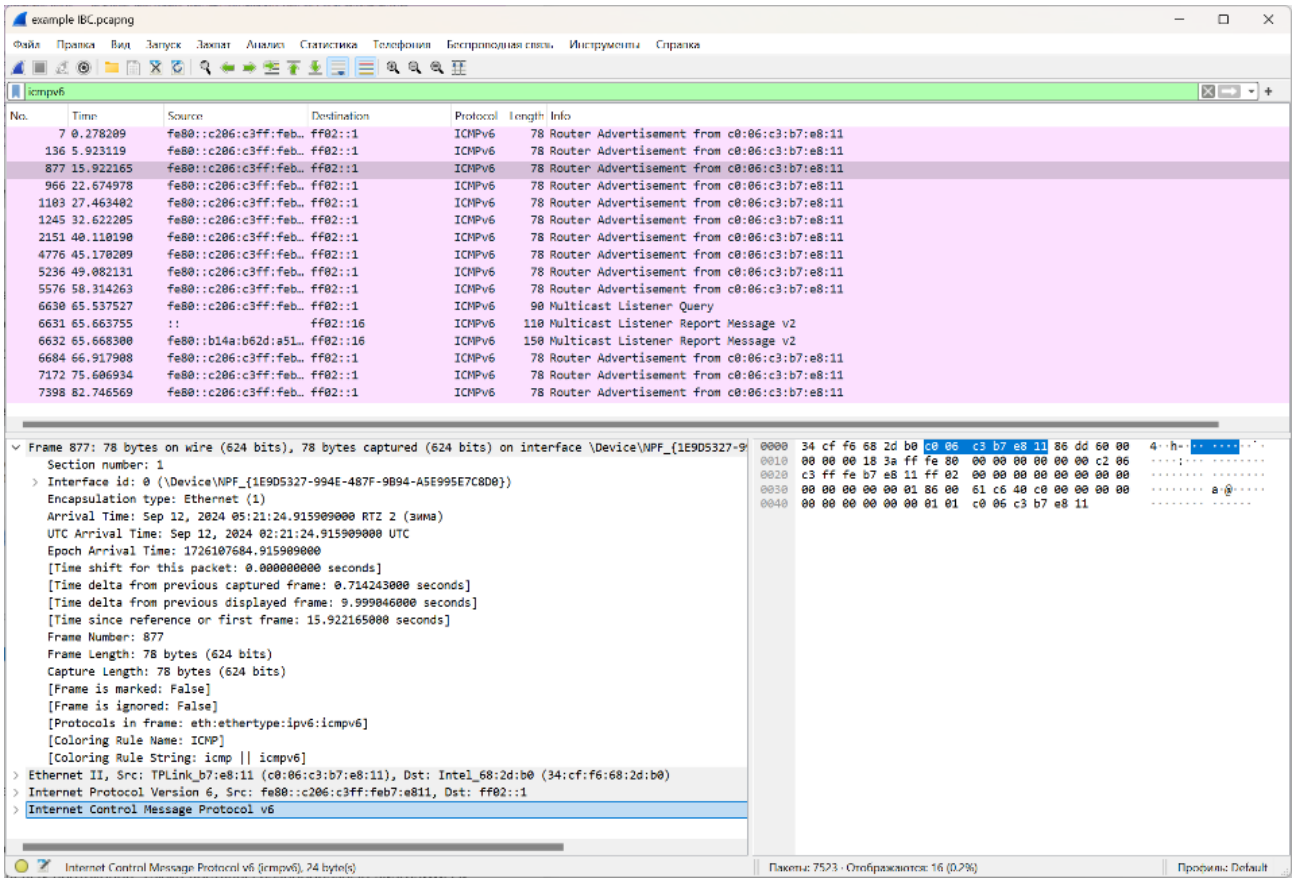


Рис. 2.12. Отображение данных внутри пакетов

Статистика

Ее использование заключается в связывании всех обширных данных и просмотре захваченных пакетов с высоты птичьего полета. Более того, статистика имеет несколько важных инструментов, которые используются для просмотра свойств файлов, иерархии протоколов или другой статистики для отдельных протоколов. Также доступны разнообразные диаграммы и графики для просмотра потоковых графиков транспортных протоколов или графиков данных ввода/вывода.

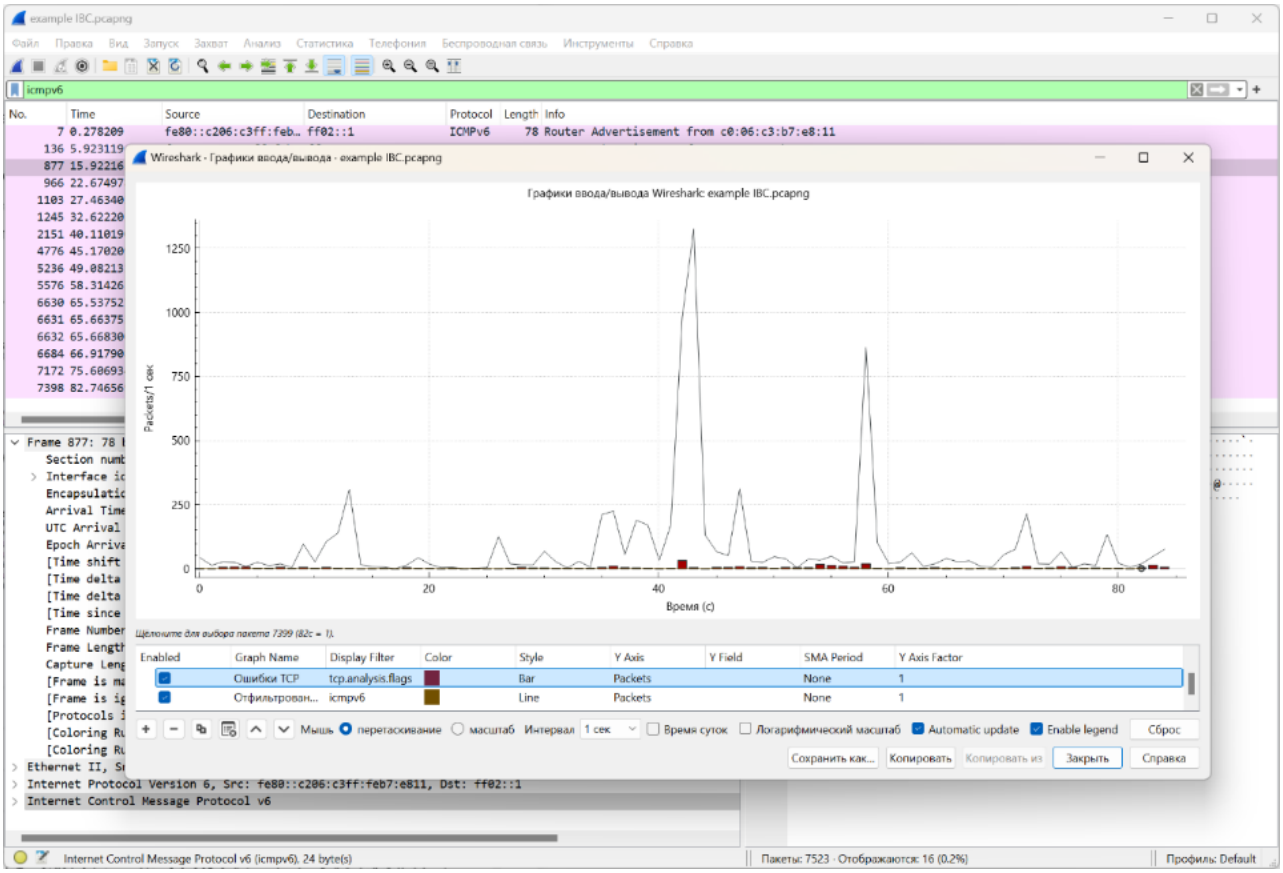


Рис. 2.13. Отображение данных внутри пакетов

Задания к практической работе

Все задания этой лабораторной работы оформляются в виде отчета в MS. В отчете должен быть выстроен порядок выполнения заданий в соответствии с представленными далее разделами. Результаты выполнения пунктов задания необходимо сопровождать графическими материалами (скриншоты вывода утилит командной строки) там, где это требуется по заданию.

Задание 1. Загрузка файла трассировки http

Поведение браузера может быть достаточно сложным, так как там используя больше функций HTTP, чем просто базовый обмен данными. Поэтому в этой работе мы будем использовать предохранённый файл трассировки, который можно скачать отсюда: trace-http.pcap. Откройте этот файл в Wireshark, должен появится экран, как показано ниже.

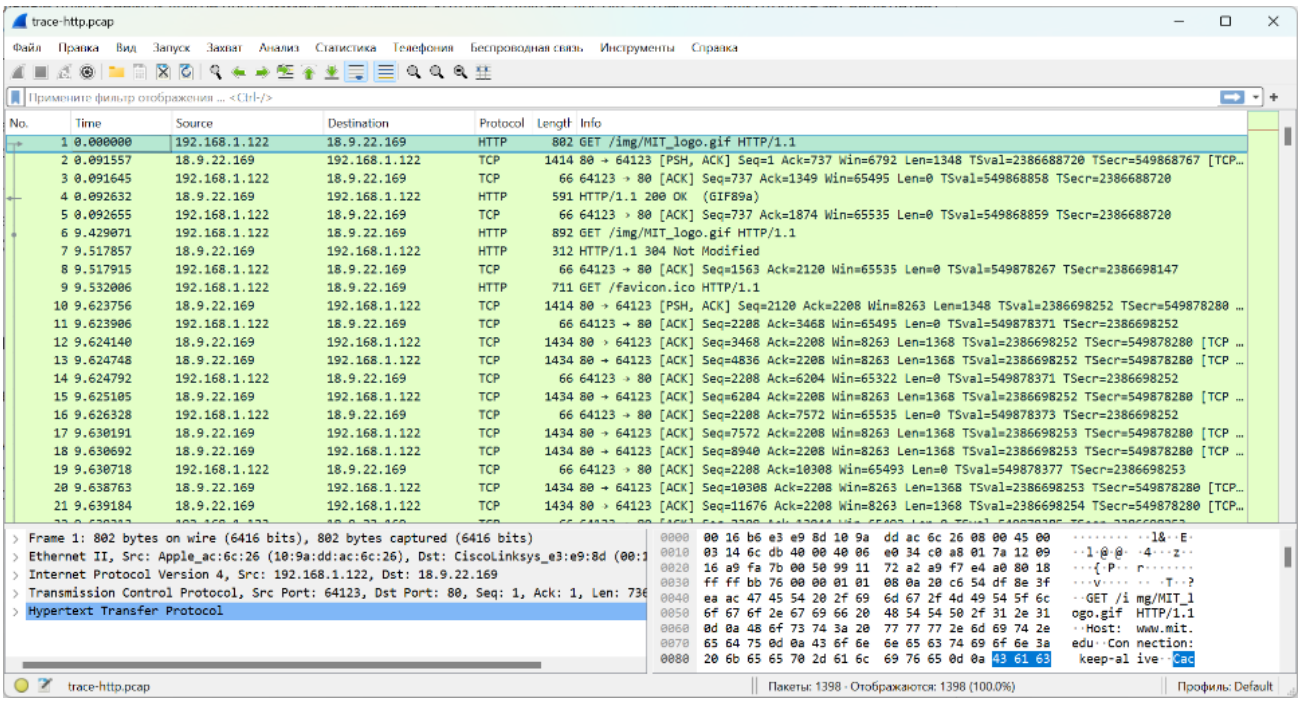


Рис. 2.14. Сохраненный файл трассировки

Используя доступный интерфейс отображения статистики, ответьте на следующие вопросы, подкрепив ответ скриншотом:

- Количество пакетов, который были перехвачены Wireshark и присутствуют в этом файле?
- Какой общий объем перехваченных пакетов (в байтах)?
- Количество пакетов HTTP Request?
- Количество пакетов HTTP Response?

В отчет внести информацию по данному пункту задания: скриншот загрузки файла трассировки и ответы на вопросы, подкрепленные скриншотами.

Задание 2. Изучение трассировки запроса и ответа

Применение фильтра

Чтобы сосредоточиться на трафике HTTP, введем и применим выражение фильтра «http». Этот фильтр отображает запросы и ответы HTTP, но не отдельные пакеты, которые в них участвуют, а весь список. Мы должны увидеть экран, как показано ниже.

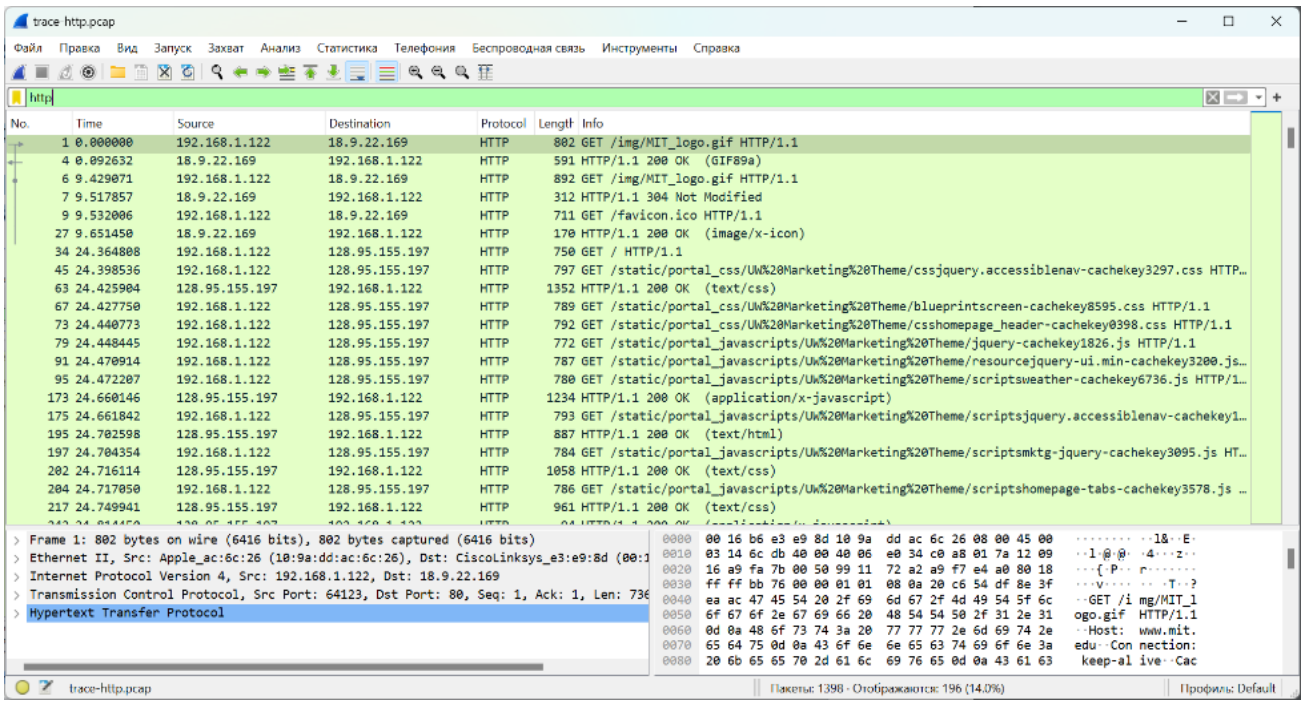


Рис. 2.15. Применение фильтра для HTTP

HTTP-ответ, несущий контент, обычно распределяется по нескольким пакетам. Когда приходит последний пакет в ответе, Wireshark собирает полный ответ и помечает пакет протоколом HTTP. Более ранние пакеты — это просто TCP-сегменты, несущие данные; последний пакет, помеченный HTTP, включает список всех более ранних пакетов, использованных для создания ответа. Аналогичный процесс происходит для запроса, но в этом случае запрос обычно помещается в один пакет.

С помощью выражения фильтра «http» мы скроем промежуточные TCP-пакеты и увидим только HTTP-запросы и ответы.

Проделайте эту операцию и добавьте скриншот результата в свой отчет. Также выполните фильтрацию и оформите результаты по протоколам:

- TCP
- TCP или UDP с использованием порта 80

Просмотр GET запроса и изучение HTTP-заголовка

Выберите первый GET в трассировке и разверните его HTTP-блок (это будет *802 GET /img/MIT_logo.gif HTTP/1.1)*, как показано ниже).

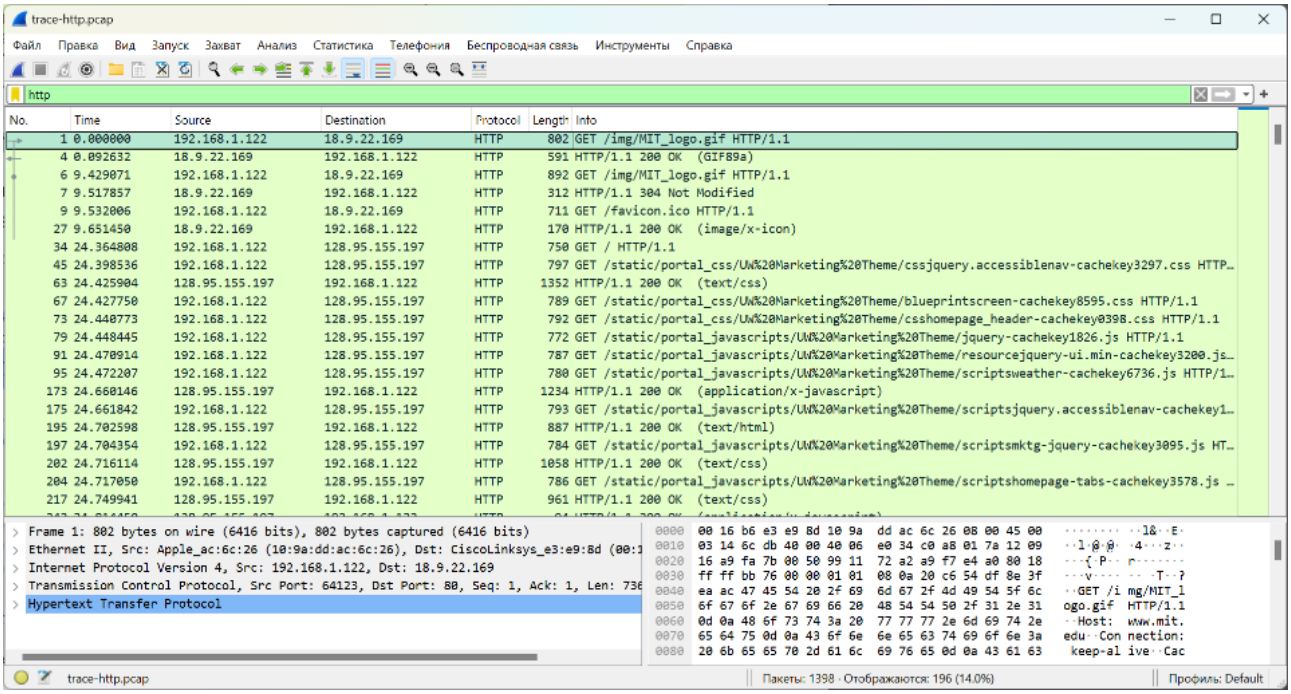


Рис. 2.16. Изучение GET-запроса

Это позволит нам изучить детали HTTP-запроса. Изучите HTTP-заголовок, развернув стрелку вниз рядом с **Протокол передачи гипертекста (Hypertext Transfer Protocol)** в нижней панели.

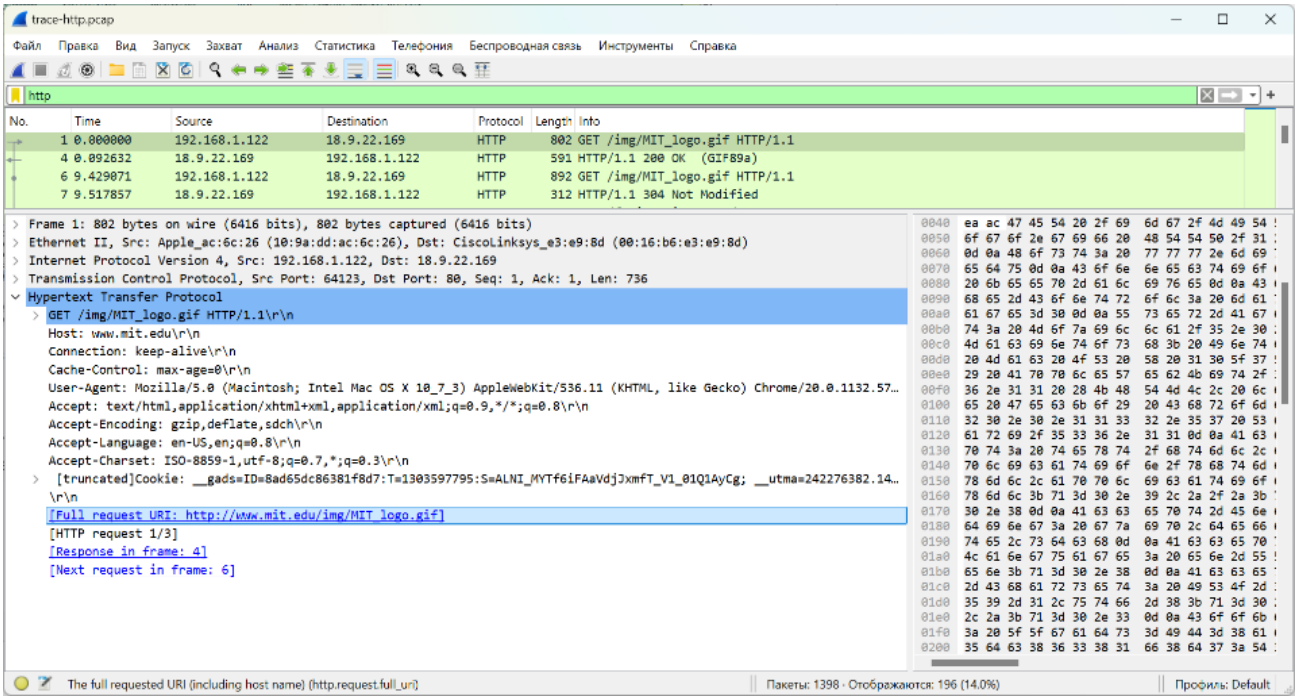


Рис. 2.17. Изучение GET-запроса

Обратите внимание, что заголовок HTTP следует за заголовками TCP и IP, поскольку HTTP — это протокол приложения, который передается с использованием TCP/IP. Чтобы его просмотреть, выберите пакет и найдите блок HTTP на нижней панели; разверните его (этот блок развернут на рисунке выше).

Изучите заголовки, которые отправляются вместе с запросом. Сначала мы увидим метод GET в начале запроса, включая такие сведения, как путь. Затем увидим ряд заголовков в виде помеченных параметров. Заголовков может быть много, и выбор заголовков и их значений варьируется от браузера к браузеру.

Посмотрите имеются ли в файле трассировки какие-либо из общих заголовков:

- **Host:** обязательный заголовок, он идентифицирует имя (и порт) сервера.
- **User-Agent:** вид браузера и его возможности.
- **Accept, Accept-Encoding, Accept-Charset, Accept-Language:** описания форматов, которые будут приняты в ответе, например, text/html, включая его кодировку, например, gzip, и язык.
- **Cookie:** имя и значение cookie-файлов, которые браузер хранит для веб-сайта.
- **Cache-Control:** информация о том, как ответ может быть кэширован.

Добавьте в отчет найденные заголовки с примерами в виде скриншотов.

Просмотр GET ответа

Выберите ответ, который соответствует первому GET в трассировке, и разверните его HTTP-блок (см. ниже).

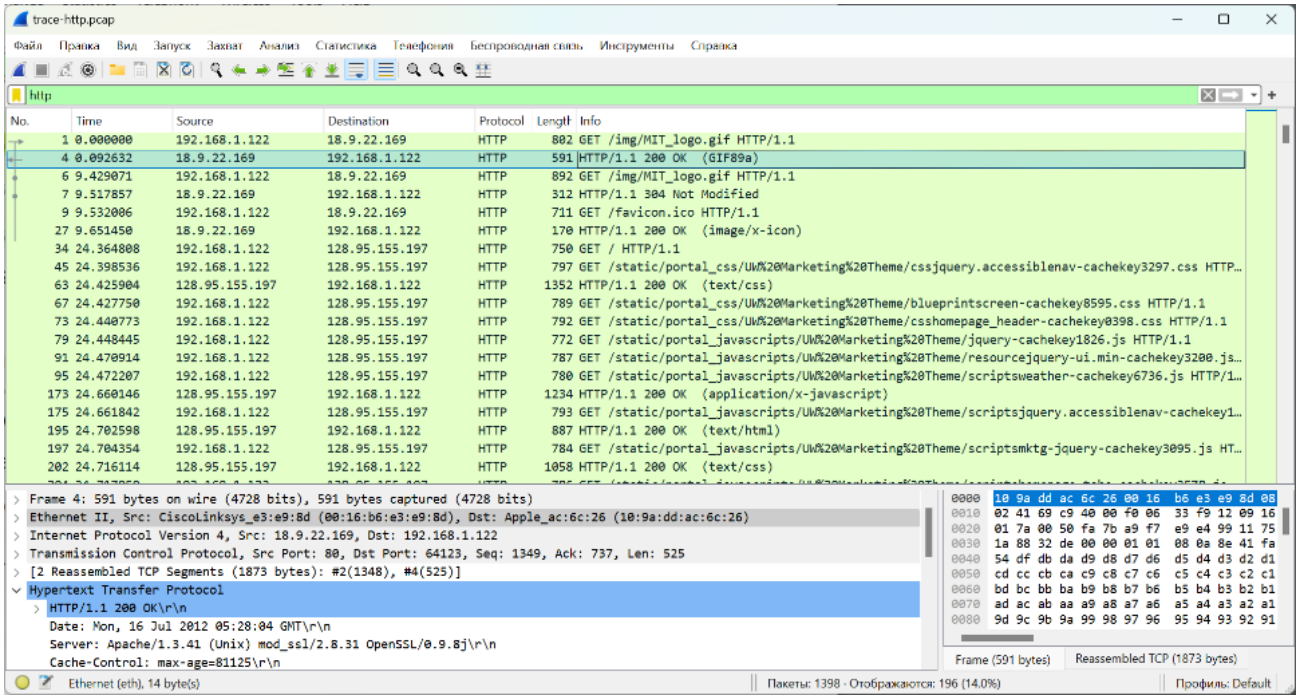


Рис. 2.18. Изучение GET-запроса

Информация для этого пакета будет указывать «200 OK» в случае нормальной, успешной передачи. Вы увидите, что ответ похож на запрос, с серией заголовков, которые следуют за кодом состояния «200 OK». Однако будут использоваться другие заголовки, а за заголовками будет следовать запрошенный контент.

Изучите общие заголовки, такие как:

- **Server.** Тип сервера и его возможности.
- **Date, Last-Modified.** Время ответа и время последнего изменения контента.
- **Cache-Control, Expires, Etag.** Информация о том, как можно кэшировать ответ.

Сделайте в своем отчете по этой работе подробное описание, с примером на основе любого GET запроса в файле трассировки, какая полезная информация содержится в этих заголовках и что она означает.

Задание 3. Кэширование контента

Второй блок записей в трассировке — это повторная выборка по первому URL. Этот блок дает нам возможность рассмотреть кэширование в действии, поскольку весьма вероятно, что изображение или документ не изменились и, следовательно, не нуждаются в повторной загрузке. Механизмы кэширования HTTP должны предоставить такую возможность. Щелкните на третью строку в файле трассировки, т.е. на **892 GET /img/MIT_logo.gif HTTP 1.1**

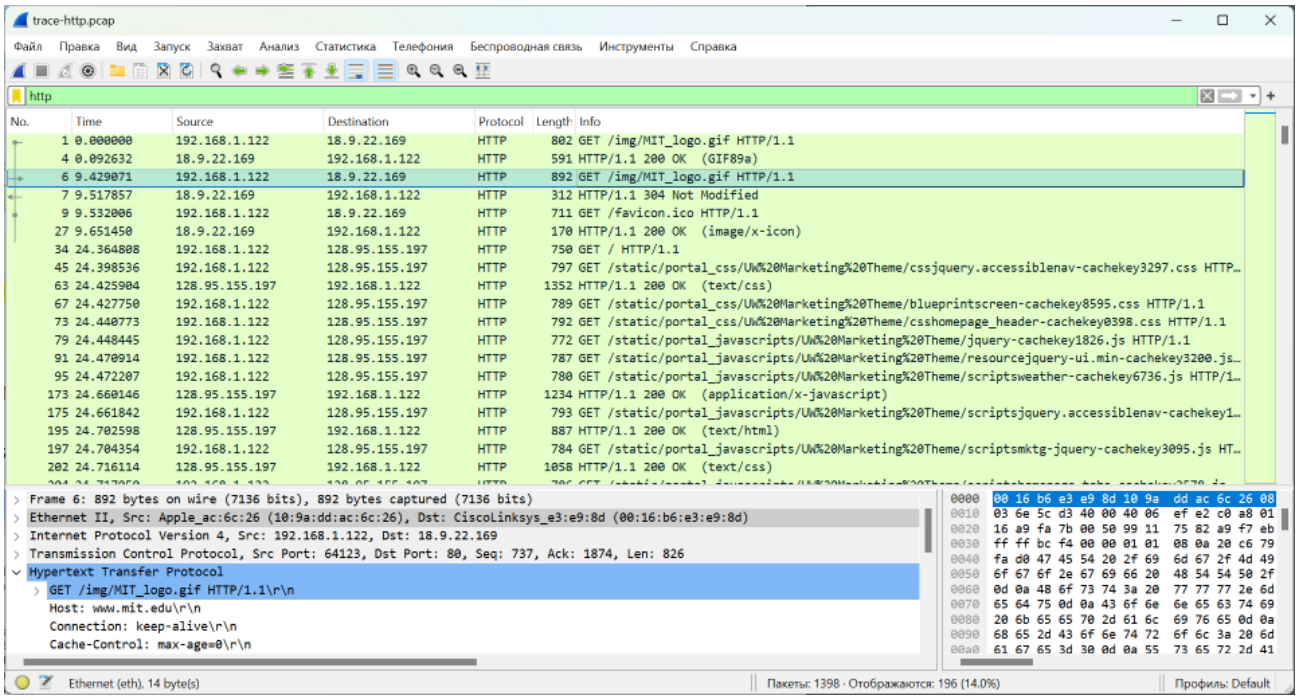


Рис. 2.19. Просмотр кэшированных элементов

Разверните HTTP-блок в нижней панели

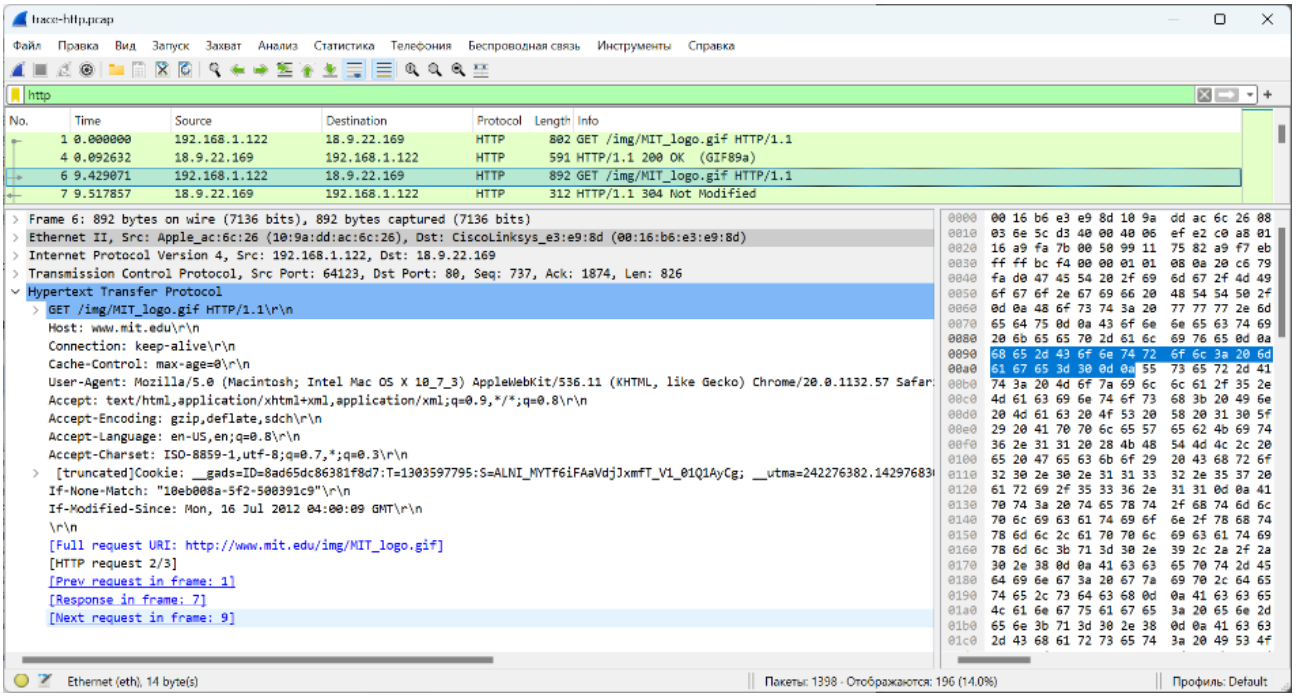


Рис. 2.20. Просмотр кэшированных элементов (продолжение)

Найдите заголовок, который позволит серверу решить, нужно ли отправлять свежий контент. Серверу нужно будет отправлять свежий контент только в том случае, если контент изменился с момента последней загрузки браузером. Чтобы сделать это, браузер включает временную метку, взятую из предыдущей загрузки для контента, который он кэшировал. Этот заголовок отсутствовал в первом GET, поскольку кэш браузера был очищен, поэтому у браузера не было предыдущей загрузки контента, который он мог бы использовать. В большинстве других отношений этот запрос будет таким же, как и первый запрос. Заголовок называется «*If-Modified-Since*», т.е. он просит сервер отправить контент, если он был изменен с определенного времени (см. заголовок ниже).

```
If-Modified-Since: Mon, 16 Jul 2012 04:00:09 GMT\r\n
\r\n
```

Теперь выберем ответ на повторную выборку и развернем его HTTP-блок.

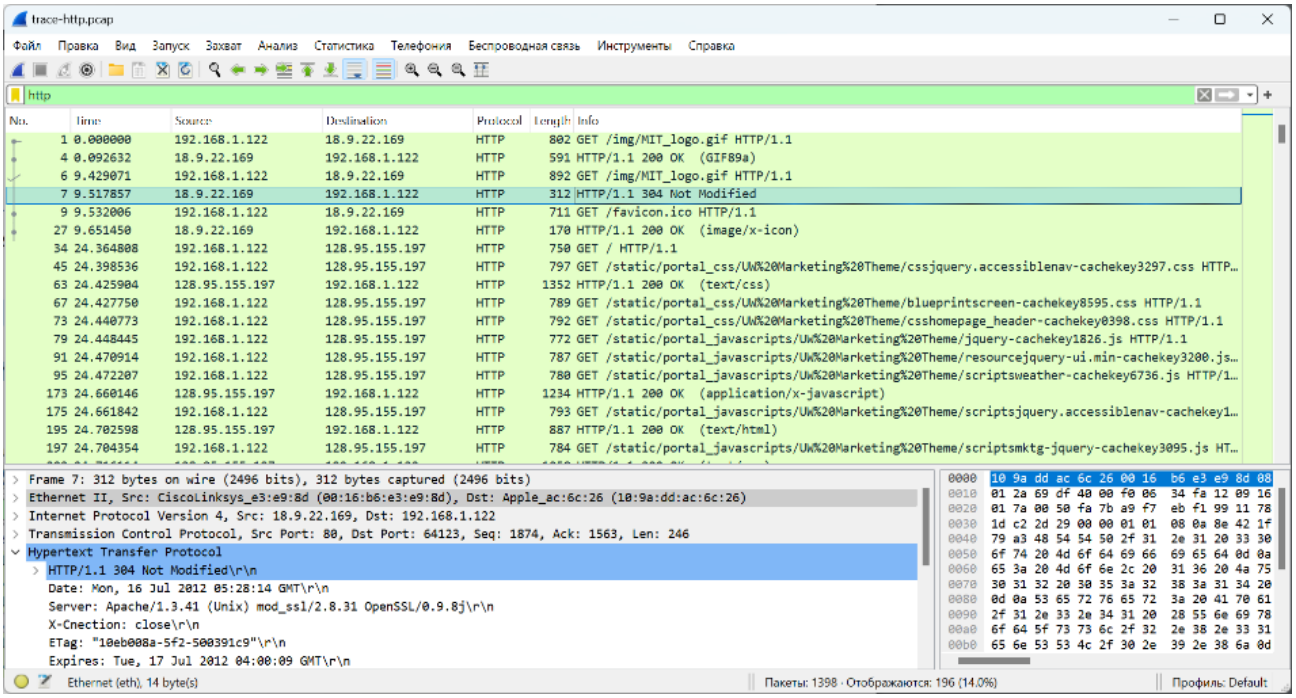


Рис. 2.21. Отображение заголовка «If-Modified-Since»

Мы видим, что кэширование сработало, как и ожидалось, этот ответ не будет содержать контент. Вместо этого код статуса ответа — «*304 Not Modified*». Это сообщает браузеру, что контент не изменился по сравнению с предыдущей копией, и кэшированный контент затем может быть отображен. Обратите внимание, что значение временной метки берется из заголовка «*Last-Modified*» самой последней загрузки контента. Это временная метка сервера для последнего изменения контента, а не временная метка браузера или временная метка времени загрузки.

Добавьте в отчет описание заголовков для данной строки трассировки (строка 7) и добавьте скрин загруженного Gif изображения, по которому обрабатывался запрос.

Задание 4. Сложные страницы

Теперь мы рассмотрим четвертую выборку в трассировке. Эта выборка была для более сложной веб-страницы, которая, скорее всего, будет иметь встроенные ресурсы. Поэтому браузер загрузит исходный HTML плюс все встроенные ресурсы, необходимые для отображения страницы, плюс другие ресурсы, которые запрашиваются во время выполнения сценариев страницы. В этой выборке мы увидим, что одна страница может включать много GET. Щелкните по номеру 34, который является *750 GET / HTTP/1.1* (как показано ниже).

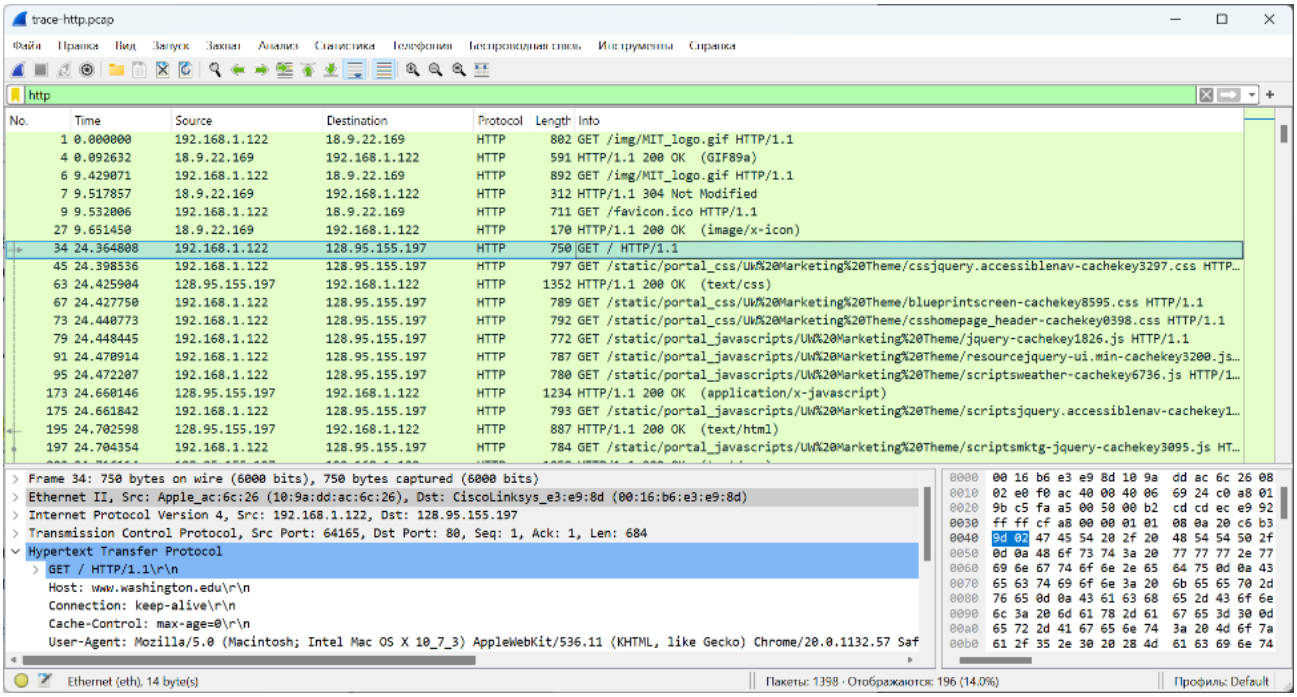


Рис. 2.22. Отображение встроенных ресурсов

Обобщите GET для этой страницы, вызвав панель *HTTP Load Distribution*. Эту панель можно найти в разделах *Статистика* и *HTTP*.

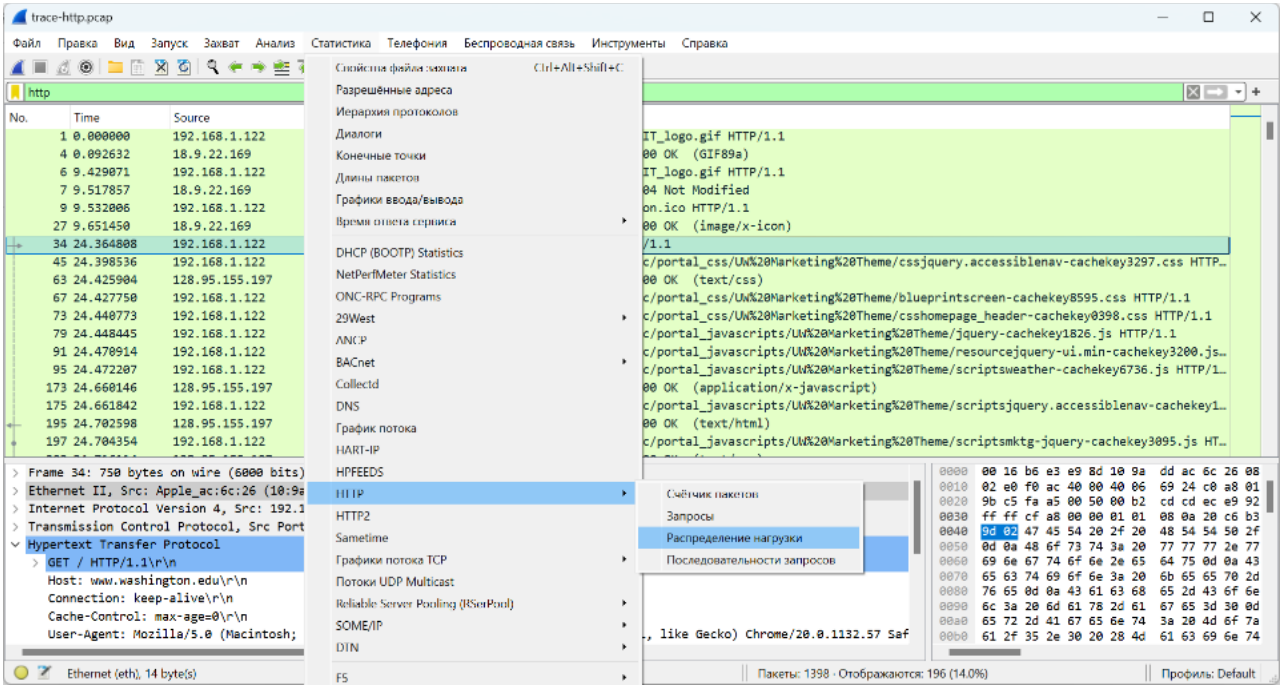


Рис. 2.23. Выбор отчета по статистике

На этой панели показана информация, сколько запросов было сделано на какие серверы. При серфинге есть вероятность, что *fetch* запросит контент с других серверов, о которых, мы могли и не подозревать. Эти другие серверы могут включать третьи стороны, такие как сети распространения контента, рекламные и аналитические сети.

Наша панель показана ниже — при извлечении страницы было сделано 98 запросов на 5 разных серверов.

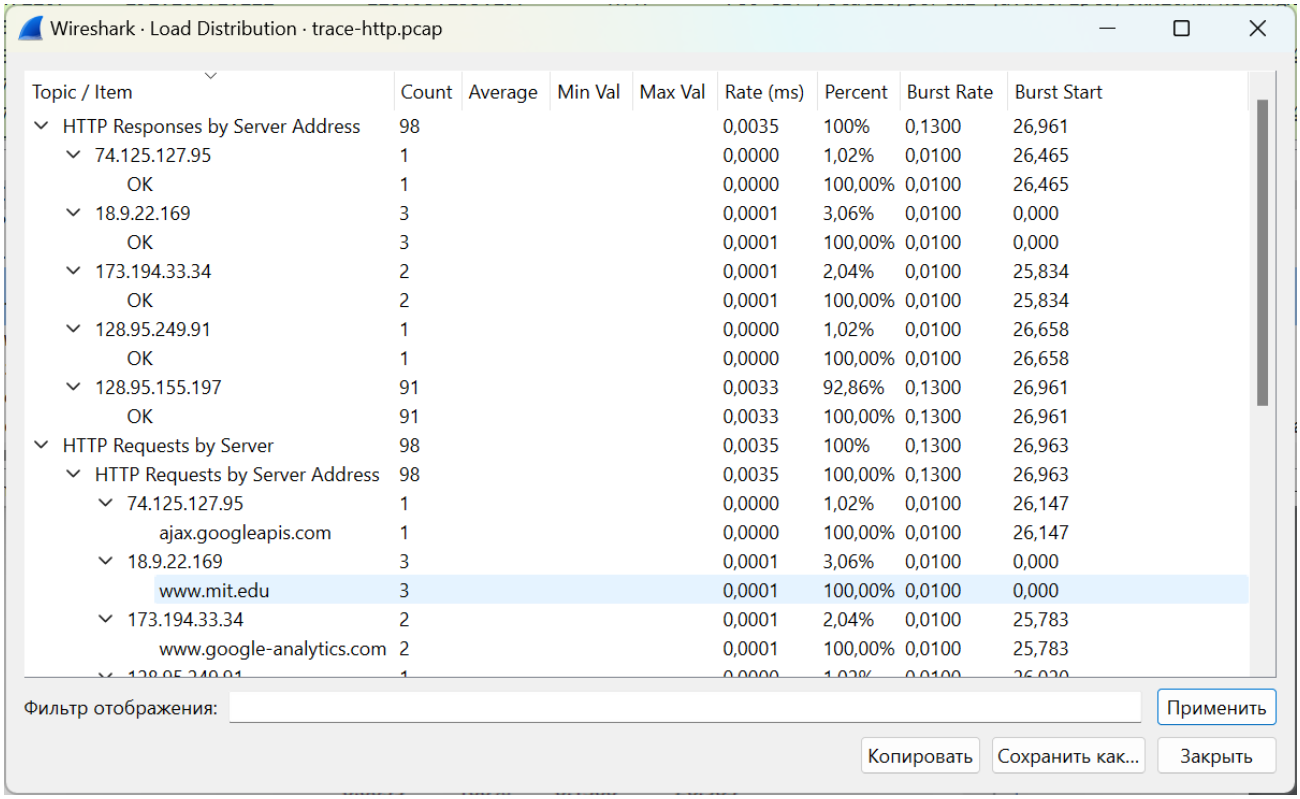


Рис. 2.24. Отображение статистики по HTTP

Для получения другого вида сводки GET-запросов откройте панель счетчика HTTP-пакетов.

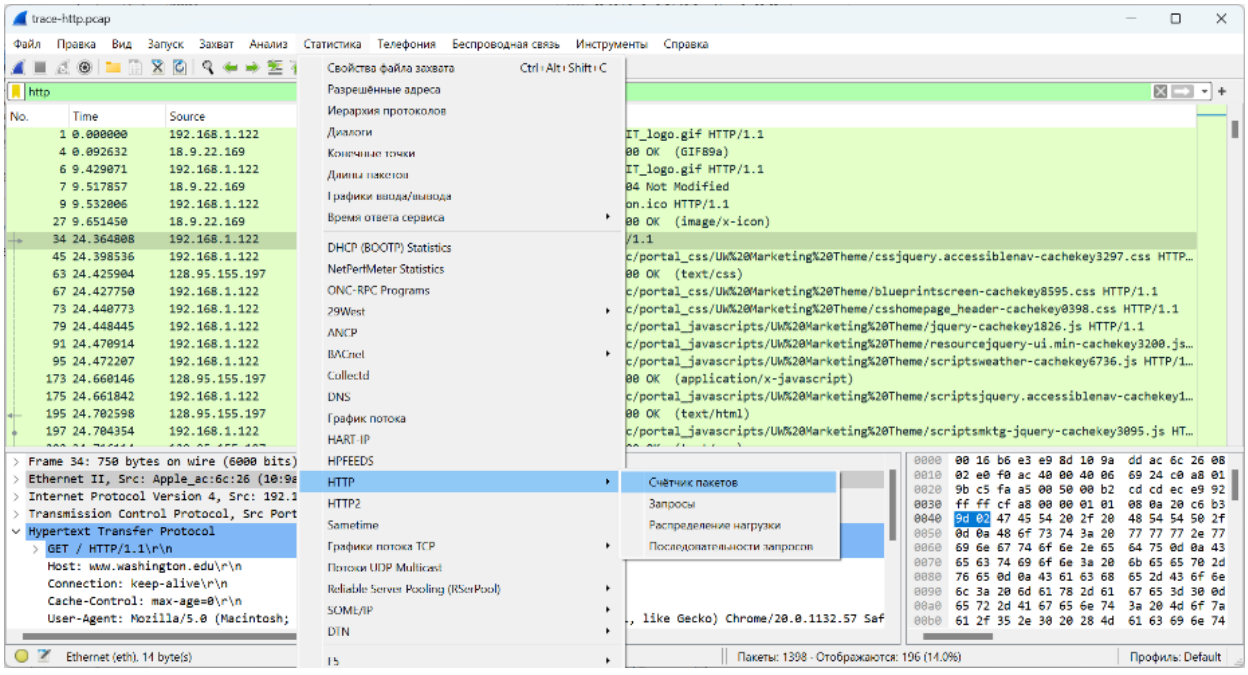


Рис. 2.25. Выбор счетчика пакетов HTTP

На панели отображаются типы запросов и ответов. Она состоит из запросов GET, которые соответствуют ответам **200 OK** и **304 Not Modified**. Однако есть множество других кодов ответов, которые наблюдаются в файле трассировки, например, когда ресурс уже кэширован.

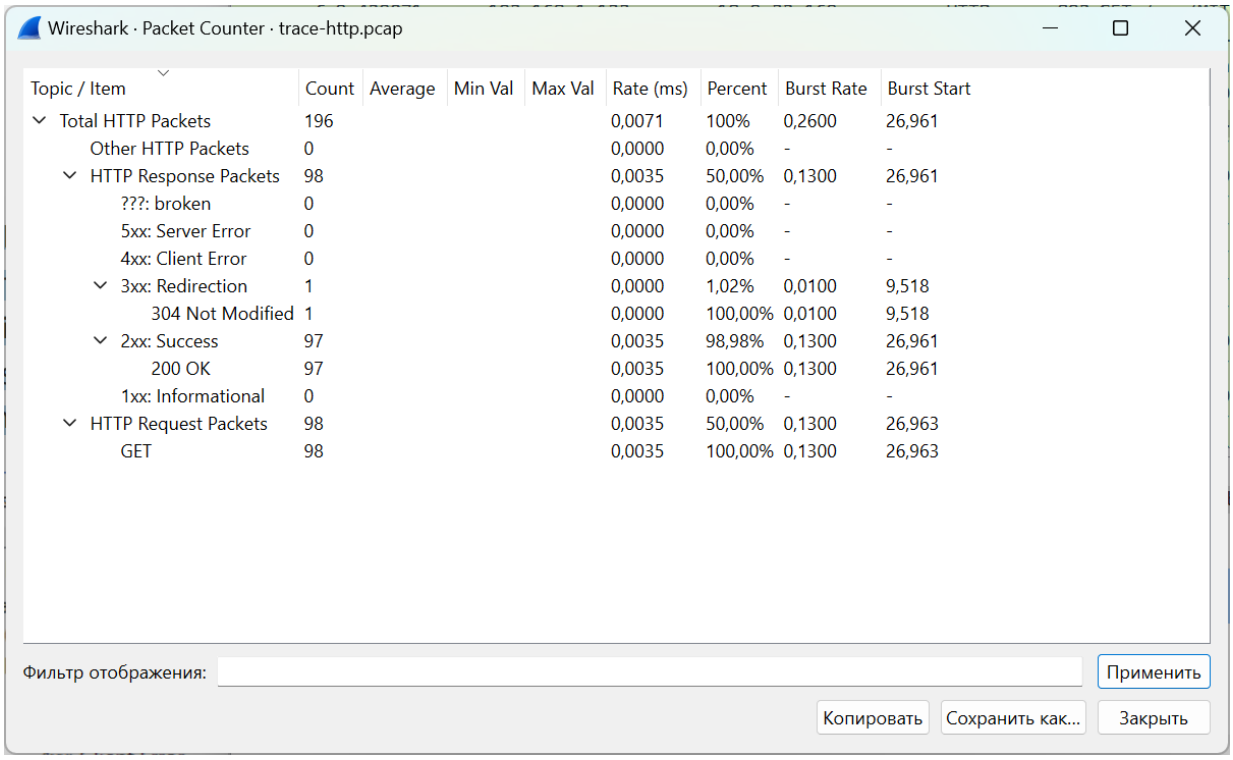


Рис. 2.26. Отображение подсчета пакетов

Любопытно узнать, какой контент загружается всеми этими запросами. Помимо просмотра URL-адресов в столбце **Информация**, мы можем получить сводку URL-адресов на панели HTTP-запросов в разделах **Статистика** и **HTTP**. Каждый из отдельных запросов и ответов имеет ту же форму, которую мы видели на предыдущем шаге. В совокупности они выполняются в процессе извлечения полной страницы с заданным URL-адресом.

Для более подробного обзора общего процесса загрузки страницы используйте такой сайт, как **PageSpeed** от Google или **https://www.webpagetest.org/**. Эти сайты проверят URL-адрес по выбору и сгенерируют отчет об активности загрузки страницы, сообщая, какие запросы были получены в какое время, и давая советы по сокращению общего времени загрузки страницы.
Начало «водопадной» диаграммы для загрузки страницы, соответствующей трассировке на **www.ulster.ac.uk**, показано на следующем рисунке. После извлечения исходного HTML-ресурса происходит множество последующих быстрых извлечений встроенных ресурсов, таких как скрипты JavaScript, таблицы стилей CSS, изображения и т. д.

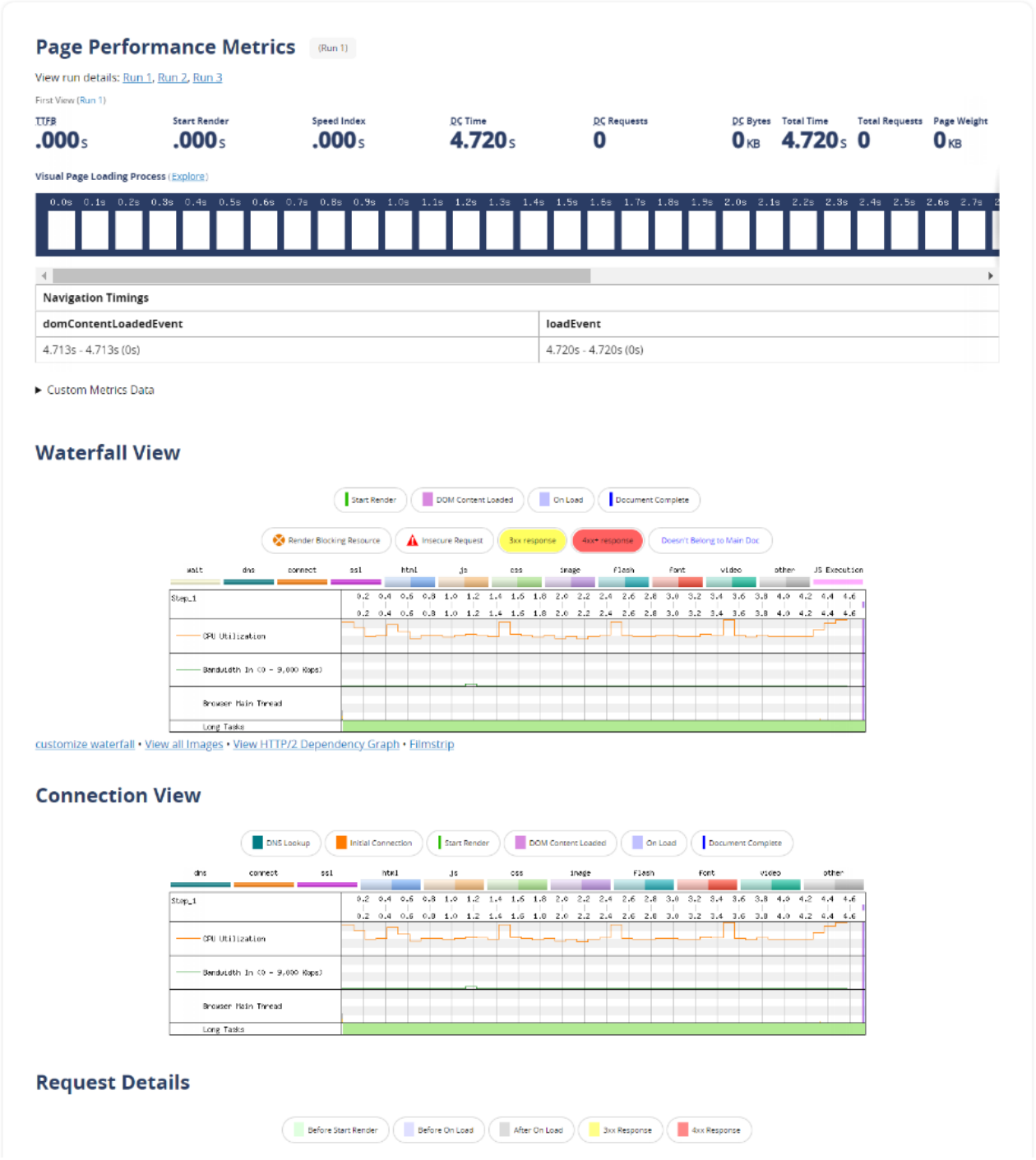


Рис. 2.27. Тестирование веб-ресурса с отображением отчета

Попробуйте выполнить тестирование Интернет-ресурса, с которым мы работали при выполнении первой практической работы (сетевые утилиты) с помощью сервиса **PageSpeed** или **https://www.webpagetest.org/.** Сформируйте для отчета диаграммы демонстрирующие результаты тестирования. Объясните какую ценность имеет эта информация для веб-разработки.

Задание 5. Отправка отчета на проверку

Добавьте созданный отчет MS Word в каталог **lab-2** вашего репозитория на GitHub. Прикрепите ссылку на каталог с работой в текущий курс для проверки преподавателем.

Контрольные вопросы

1. Что такое протокол HTTP и для чего он используется?
2. Каковы основные методы HTTP и какова их функция?
3. Объясните разницу между методами GET и POST.
4. Что такое статус-коды HTTP и какова их роль в протоколе?
5. Перечислите основные статус-коды HTTP и объясните их значение.
6. Что такое заголовки HTTP и какую информацию они содержат?
7. Какова роль заголовка User-Agent в HTTP-запросах?
8. Что такое HTTPS и как он отличается от HTTP?
9. Как работает механизм кэширования в HTTP?
10. Что такое сессия в контексте HTTP и как она реализуется?
11. Как использовать Wireshark для захвата HTTP-трафика?
12. Что такое фильтры в Wireshark и как они помогают в анализе трафика?
13. Как можно просмотреть содержимое HTTP-запроса и ответа в Wireshark?
14. Что такое TCP и как он взаимодействует с HTTP?
15. Каковы особенности работы HTTP/2 по сравнению с HTTP/1.1?
16. Что такое HTTP-заголовок Referer и для чего он используется?
17. Каковы основные принципы работы с куками в HTTP?
18. Что такое Content-Type и как он влияет на обработку данных?
19. Как можно использовать Wireshark для анализа проблем с производительностью HTTP?
20. Что такое REST и как он связан с протоколом HTTP?

ПРАКТИЧЕСКАЯ РАБОТА 3. НАСТРОЙКА СРЕДЫ ВЕБ-РАЗРАБОТКИ

Цель работы

- **Установка IDE:** изучение настройки среды веб-разработки. Установка и настройка **Visual Studio Code** в качестве основного редактора для веб-разработки.
- **Локальный сервер:** использование расширение **Live Server** для запуска локальных веб-серверов из **VS Code**.
- **Организация сетевого тоннеля:** установка и настройка **ngrok**, чтобы предоставить доступ к локальному серверу из Интернета для тестирования и совместного использования.
- **Изучение ПО работы с запросами:** знакомство с основами **Postman** и его ролью в отправке HTTP-запросов и взаимодействии с API.
- **Сервис публикации кода:** создание учетной записи **GitHub** и своего онлайн-портфолио.
- **Публикация своих веб-страниц:** развертывание первого веб-сайта с помощью **GitHub Pages**, чтобы делиться и размещать работы.

Требования программному окружению

- Редактор кода: VS Code . Также установите LiveServer
- Контроль версий: установка git для Windows
- Зарегистрируйтесь для получения бесплатной учетной записи GitHub
- Зарегистрируйтесь для получения бесплатной учетной записи и установите/скопируйте ngrock

Теоретическая часть

3.1. Инструменты Веб-разработчика

Для разработчиков и тестировщиков интерфейсов поиск и исправление ошибок в веб-приложении иногда может оказаться сложной задачей из-за неожиданных данных из вызова API и неопределенных взаимодействий с пользователем. Во всех современных браузерах есть инструмент разработчика для отладки ошибок в коде. Инструменты разработчика позволяют разработчикам и тестировщикам просматривать и изменять HTML/CSS и JavaScript открытой веб-страницы. Также можно изменять элементы DOM напрямую через браузер. Google Chrome, один из самых популярных браузеров, также имеет встроенные инструменты разработчика, называемые **Chrome Developer Tools**.

Chrome Developer Tools *или* ***Chrome DevTools*** *— это набор инструментов веб-разработки, встроенных в браузер Google Chrome, который позволяет нам проверять, редактировать и отлаживать наш код и измерять производительность нашей страницы.*

Эти инструменты позволяют разработчикам видеть изменения на веб-страницах в режиме реального времени, быстрее диагностировать проблемы и создавать лучший веб-сайт. Чтобы получить доступ к инструментам разработчика Chrome, разработчики могут использовать следующие три метода:

- Самый простой способ — использовать сочетания клавиш, например **Ctrl + Shift + J** в Windows и **cmd + Option + J** в Mac.
- Откройте меню браузера, нажав на три точки в правом верхнем углу, прокрутите вниз до пункта **«Дополнительные инструменты»** и выберите пункт **«Инструменты разработчика»**.
- Щелкните правой кнопкой мыши веб-страницу для отладки и выберите опцию **«Посмотреть код»**.

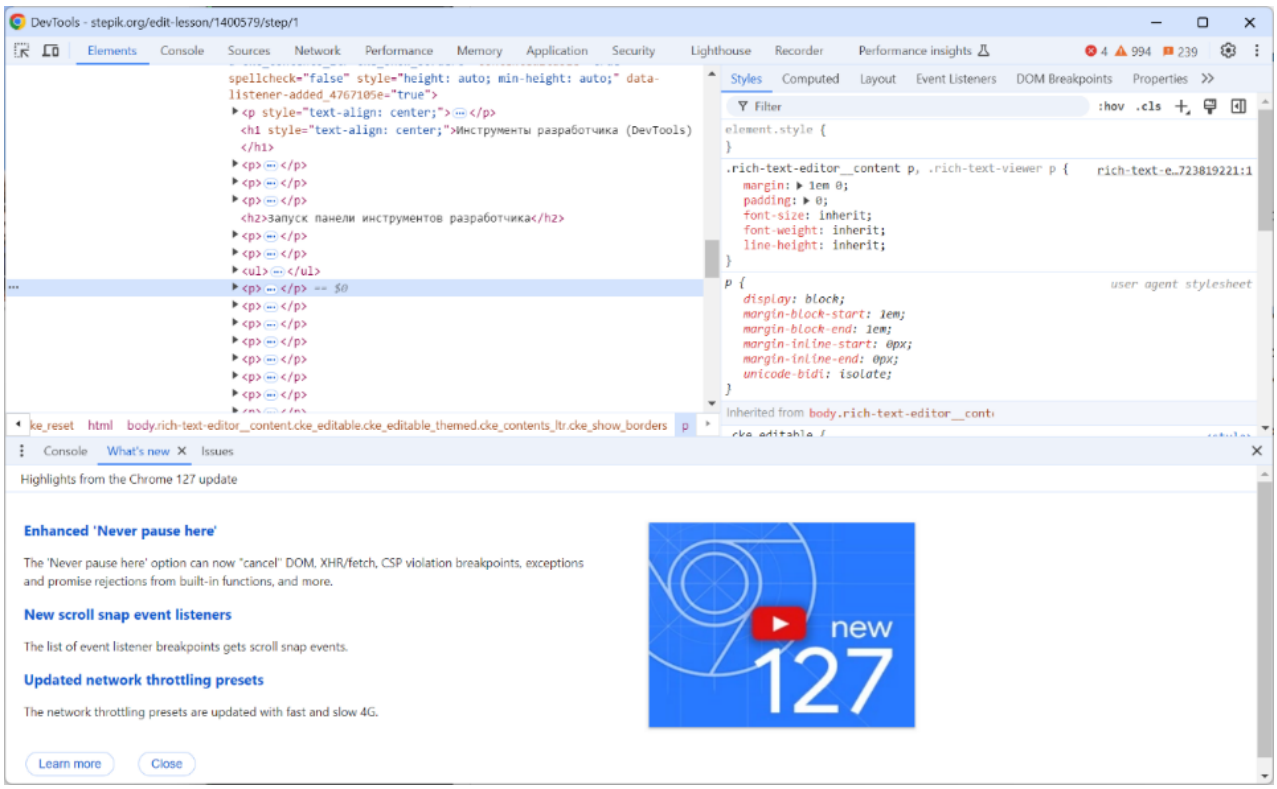


Рис. 3.1. Интерфейс DevTools

3.2. Различные компоненты инструментов разработчика Chrome

Отладчик Chrome имеет девять панелей (их гораздо больше, но мы рассмотрим только самые основные). Все эти панели отвечают за различные задачи отладки.

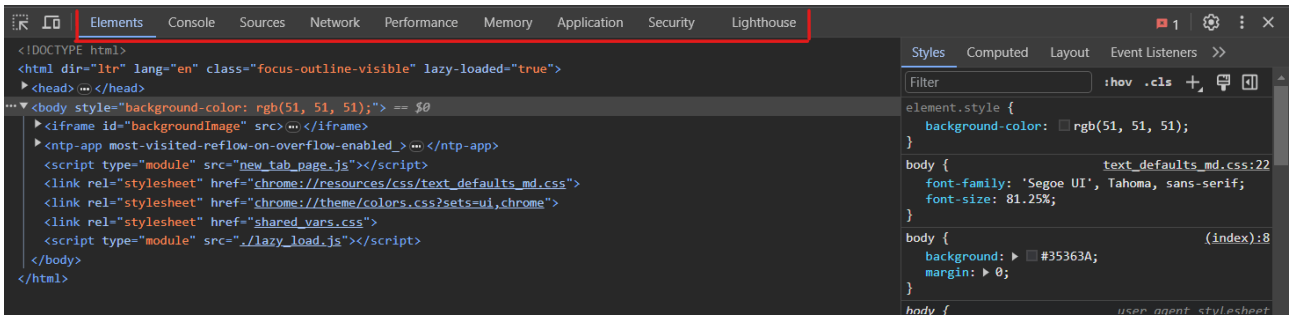


Рис. 3.2. Переключение на панели отладки DevTools

Вот краткий обзор того, за что в основном отвечает каждая панель.

1. **Элементы (Elements):** может проверять и редактировать элементы HTML и атрибуты стилей CSS.
2. **Консоль (Console):** в основном отвечает за просмотр журналов и выполнение кода JavaScript.
3. **Источники (Sources):** используются для отладки JavaScript, редактирования JavaScript, добавления точек останова и т.д.
4. **Сеть (Network):** отвечает за просмотр и отладку сетевых действий.

5. **Производительность (Performance):** используется для анализа скорости загрузки и оптимизации веб-страницы.
6. **Память (Memory):** помогает отслеживать использование памяти и устранять проблемы, связанные с памятью.
7. **Применение (Application):** проверяет файлы cookie, localStorage, sessionStorage, IndexDB и любые другие локальные базы данных.
8. **Безопасность (Security):** используется для отладки сертификатов и других проблем безопасности.
9. **Lighthouse (Lighthouse):** проверяет SEO, производительность, качество, доступность и т.д. веб-приложения/сайта.

Панель элементов: позволяет разработчикам проверять и изменять HTML и CSS веб-страницы. С помощью панели элементов можно изменять код непосредственно в браузере и видеть, как эти изменения мгновенно отражаются на веб-странице.

Разработчики и тестировщики в основном используют панель элементов для экспериментов со структурой HTML или дизайном веб-страницы. Ее функции включают в себя выбор цвета, адаптивные размеры экрана, редактор CSS, анализатор макета, конструктор анимации, прослушиватели событий и линейки.

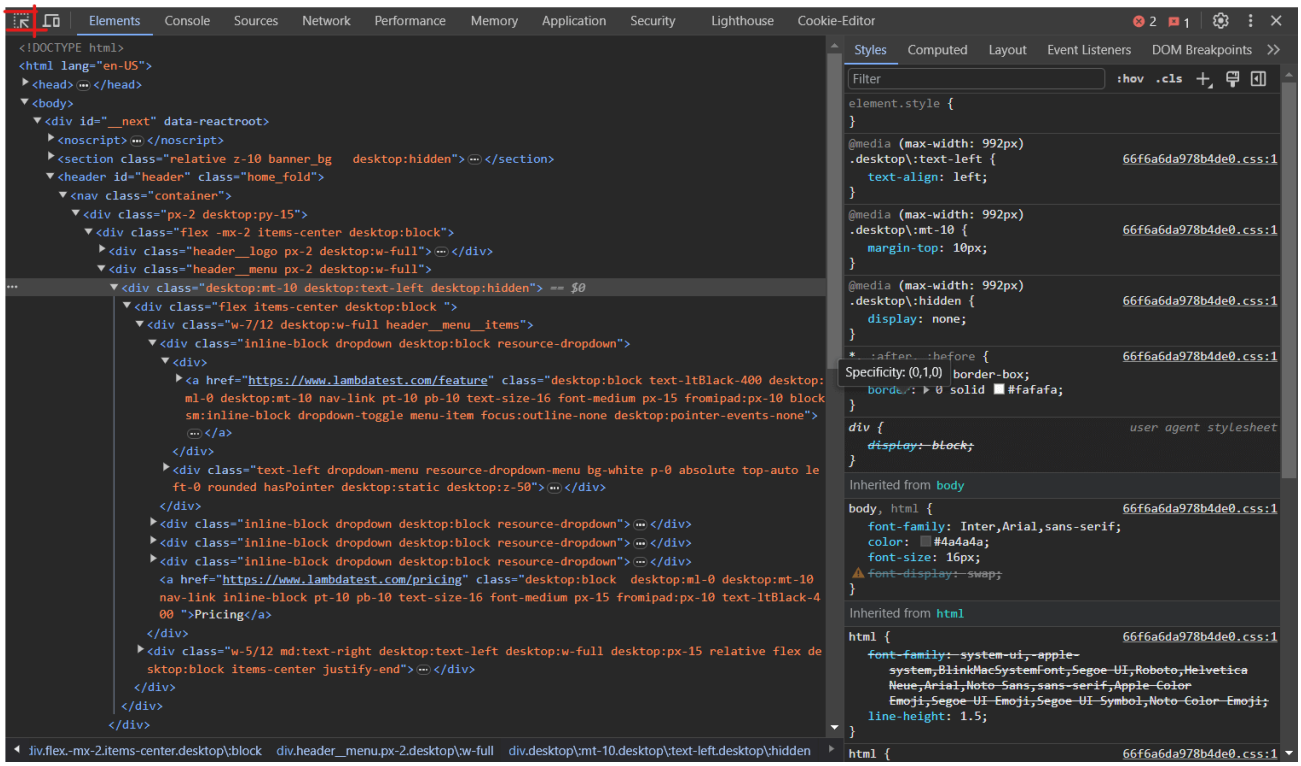


Рис. 3.3. Панель элементов DevTools

Чтобы проверить и изменить любой компонент, нам нужно выбрать его с помощью значка инструмента выбора в правом верхнем углу или нажав *Ctrl + Shift + C* или *Cmd + Shift + C* в качестве сочетания клавиш. Это отобразит *HTML/ Document Object Model (DOM)* и CSS для этого компонента на панели элементов. Далее мы можем

изменить HTML или CSS для этого компонента. Любые внесенные нами изменения будут немедленно отражены на странице и не повлияют на исходный файл напрямую.

Панель управления (консоль): используется для написания и выполнения кодов JavaScript. Мы также можем видеть вывод журнала, созданный нашим веб-сайтом, на этой панели. Чтобы открыть панель консоли, мы можем нажать на вкладку панели консоли в Chrome DevTools.

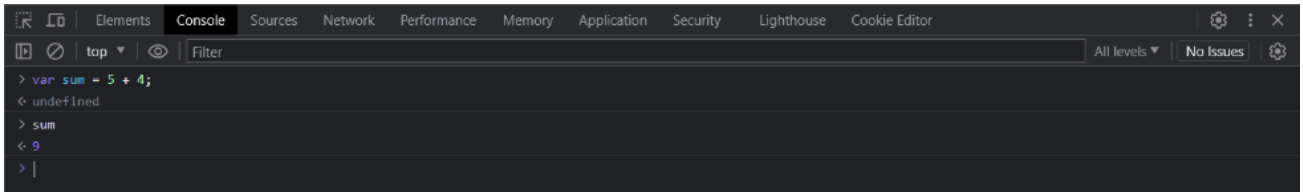


Рис. 3.4. Панель управления DevTools

Это позволяет в реальном времени проверять значения переменных с помощью панели консоли инструментов разработчика Chrome. Разработчики и тестировщики могут вставлять различные операторы *console.log()* в свой код, а затем видеть их вывод на панели консоли, когда пользователь взаимодействует с браузером.

Панель источников: позволяет разработчикам и тестировщикам просматривать исходный код или выходные данные пакета. Эта панель в основном используется для сложной отладки JavaScript. Здесь можно просматривать наши исходные файлы и устанавливать несколько точек останова, следить за переменными и их значениями, а также выполнять большую часть отладки. Она содержит дерево файлов, которое позволяет перемещаться по всем файлам исходного кода. Чтобы открыть панель источников, нужно щелкнуть вкладку панели источников в инструментах разработчика Chrome.

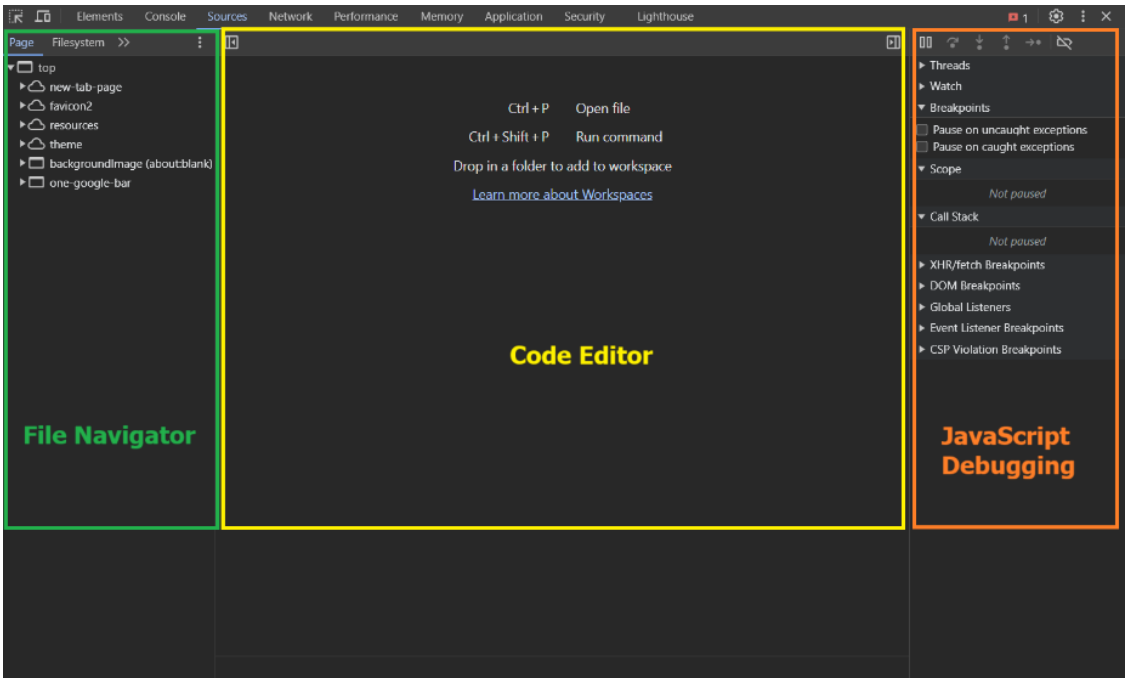


Рис. 3.5. Панель источников DevTools

Как показано на изображении выше, панель источников состоит из трех основных компонентов.

- Слева у нас есть панель *File Navigator*; она отображает дерево файлов веб-приложения, включая HTML, CSS, JavaScript и другие файлы, включая ресурсы, прикрепленные к веб-приложению. Файлы расширений Chrome также могут отображаться здесь.
- В центре панели *редактора кода* отображаются все исходные коды файлов, выбранных в навигаторе файлов.
- Справа расположена панель *отладки JavaScript*, которая используется специально для отладки кода JavaScript.

Панель сети: используется для обнаружения любого запроса на загрузку или скачивание, сделанного веб-страницей с использованием сети. Здесь можно регистрировать и анализировать всю сетевую активность в разделе журнала.

Панель сети в Chrome DevTools имеет каскадный график для всех ресурсов, загруженных в веб-приложение. Этот каскадный график помогает нам измерять индивидуальную производительность каждого актива, ресурса и конечной точки API; следовательно, его можно использовать для улучшения сетевой производительности веб-приложения. Кроме того, мы можем фильтровать время, необходимое для различных типов ресурсов, таких как *Fetch/XHR (для запросов API)*, *JavaScript, CSS, Img, Media, Font, Doc* и других.

Для анализа сетевой производительности веб-приложения с помощью инструмента разработчика Chrome мы можем открыть панель сети, нажав на вкладку *Сеть*. Далее мы можем очистить все предыдущие журналы, нажав на значок очистки. Наконец, мы можем обновить нашу веб-страницу, чтобы получить сетевой журнал и график водопада в реальном времени веб-сайта.

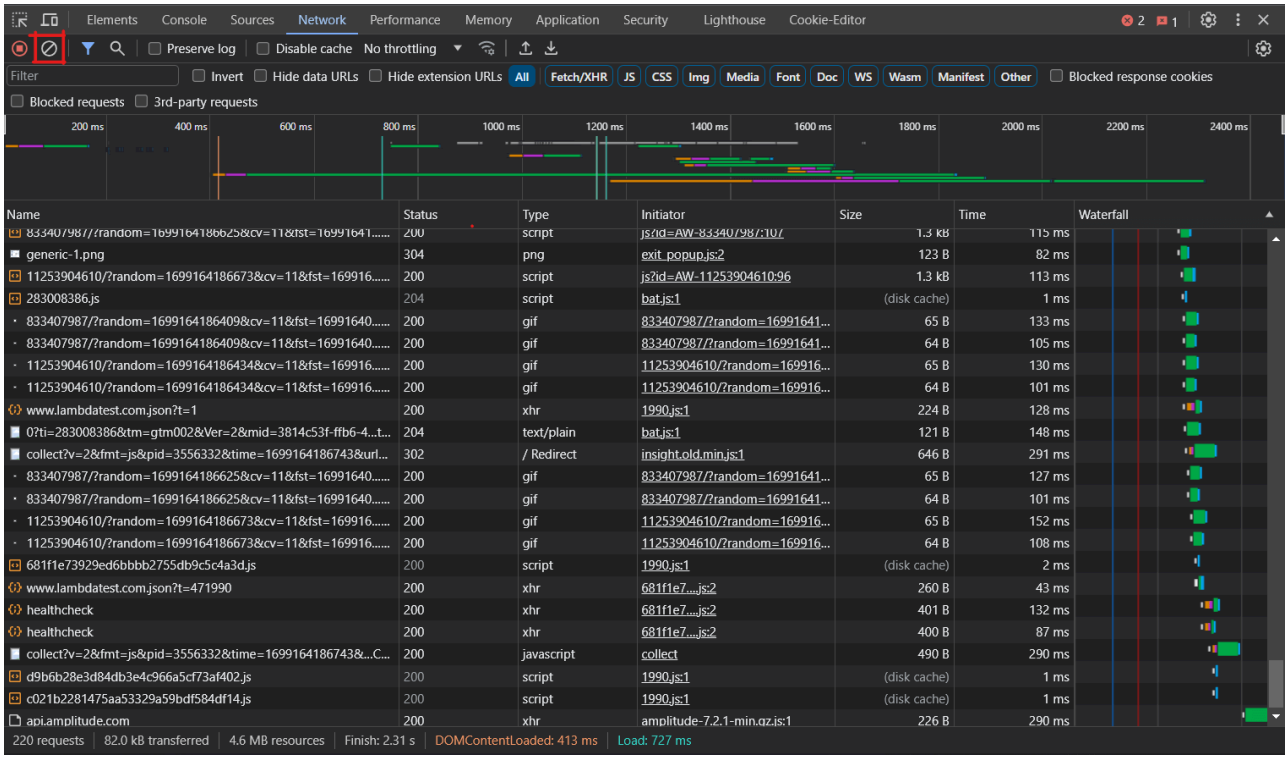


Рис. 3.6. Мониторинг запросов на панели сети DevTools

Сетевая панель используется для мониторинга количества запросов, сделанных веб-страницей, общего объема переданных данных, времени загрузки, загруженных или выгруженных ресурсов, производительности кэширования и других сетевых действий. Эта информация помогает разработчикам и тестировщикам оптимизировать отдельные ресурсы и веб-приложение.

Панель производительности: позволяет разработчикам и тестировщикам измерять производительность загрузки страницы веб-приложения. Это достигается путем запуска анализа производительности на веб-странице и генерации метрик по различным аспектам процесса загрузки страницы, таким как время, необходимое для отображения веб-страницы, загрузка различных сетевых ресурсов и интерактивность веб-страницы.

Чтобы открыть панель производительности, мы можем нажать на вкладку производительности в инструментах разработчика Chrome.

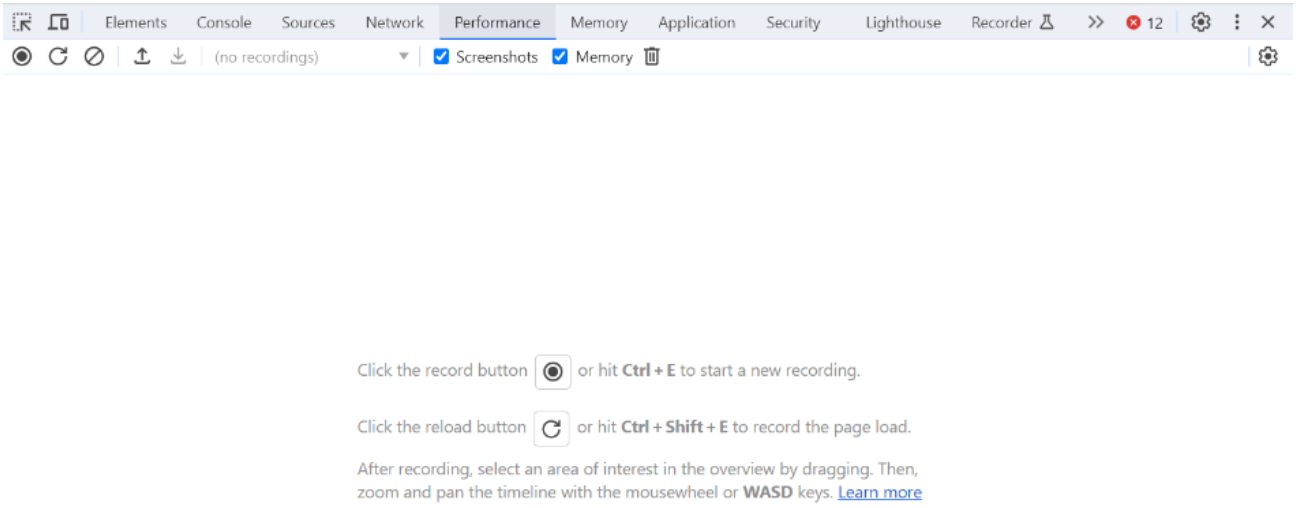


Рис. 3.7. Мониторинг запросов на панели сети DevTools

Кнопка записи и перезагрузки страницы на панели производительности Chrome DevTools позволяет разработчикам и тестировщикам проводить анализ производительности веб-страницы и просматривать подробные показатели того, как страница отображается и загружается. При нажатии кнопки перезагрузки инструмент имитирует пользователя, посещающего и взаимодействующего с веб-страницей, предоставляя информацию о процессе загрузки страницы в виде графиков и снимков экрана.

Подробный анализ производительности на сайте *LambdaTest* представлен ниже.

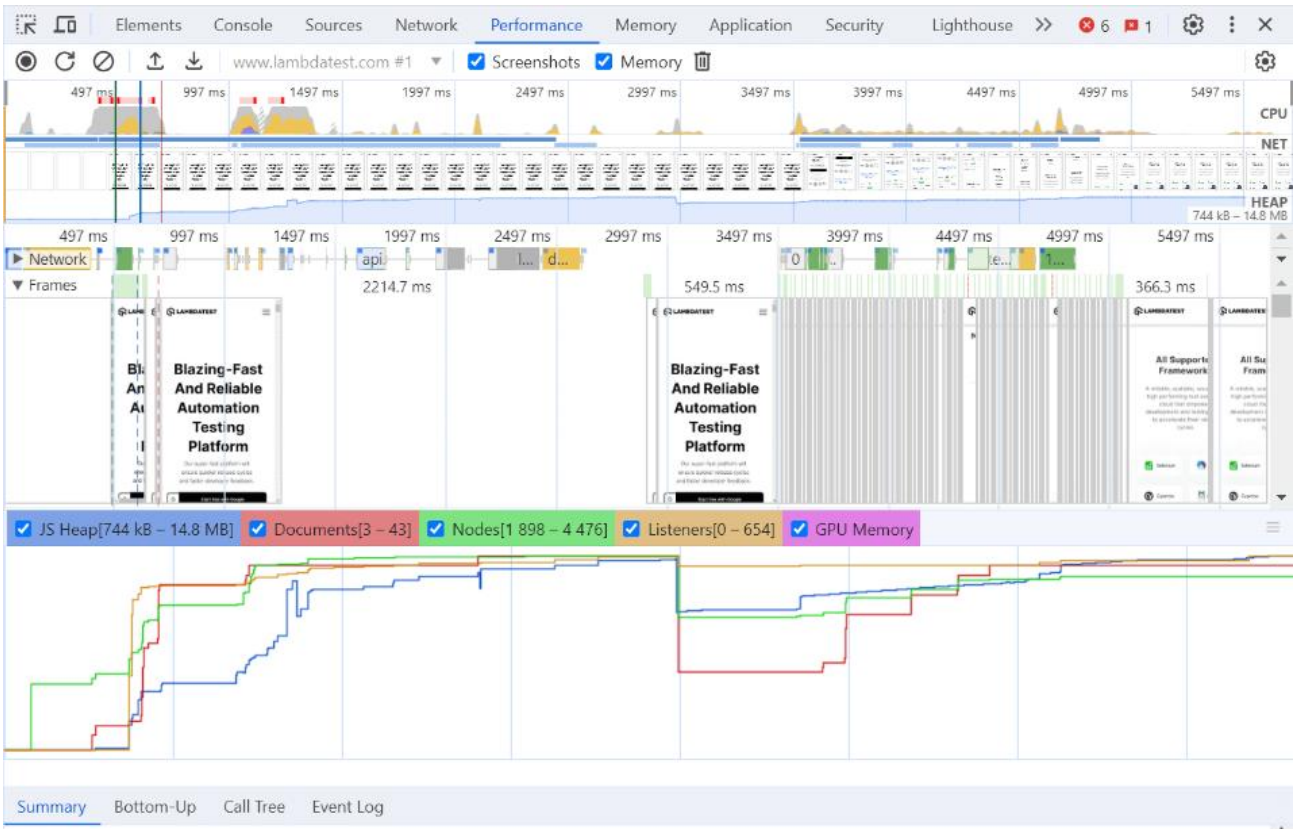


Рис. 3.8. Анализ производительности в DevTools

Мы даже можем взаимодействовать со стеками вызовов и проверять функции, которые могут потреблять больше времени, блокировать основной поток и задерживать метрику *TTI (Time To Interactive)*. Выбор функции предоставляет всевозможную информацию для этой функции. Мы можем проверить, как долго эта функция выполнялась и какие другие функции она вызывала, а также мы можем открыть функцию на *панели источников*.

Панель памяти: обеспечивает расширенное представление о потреблении памяти веб-страницей. Мы можем проверять использование памяти компонентами, выделение кучи моментальных снимков и временные шкалы выделения записей.

Панель памяти обнаруживает и устраняет проблемы, связанные с памятью, такие как утечки, раздувание и сбор мусора. Чтобы открыть панель памяти, мы можем нажать на вкладку *памяти* в инструментах разработчика Chrome.

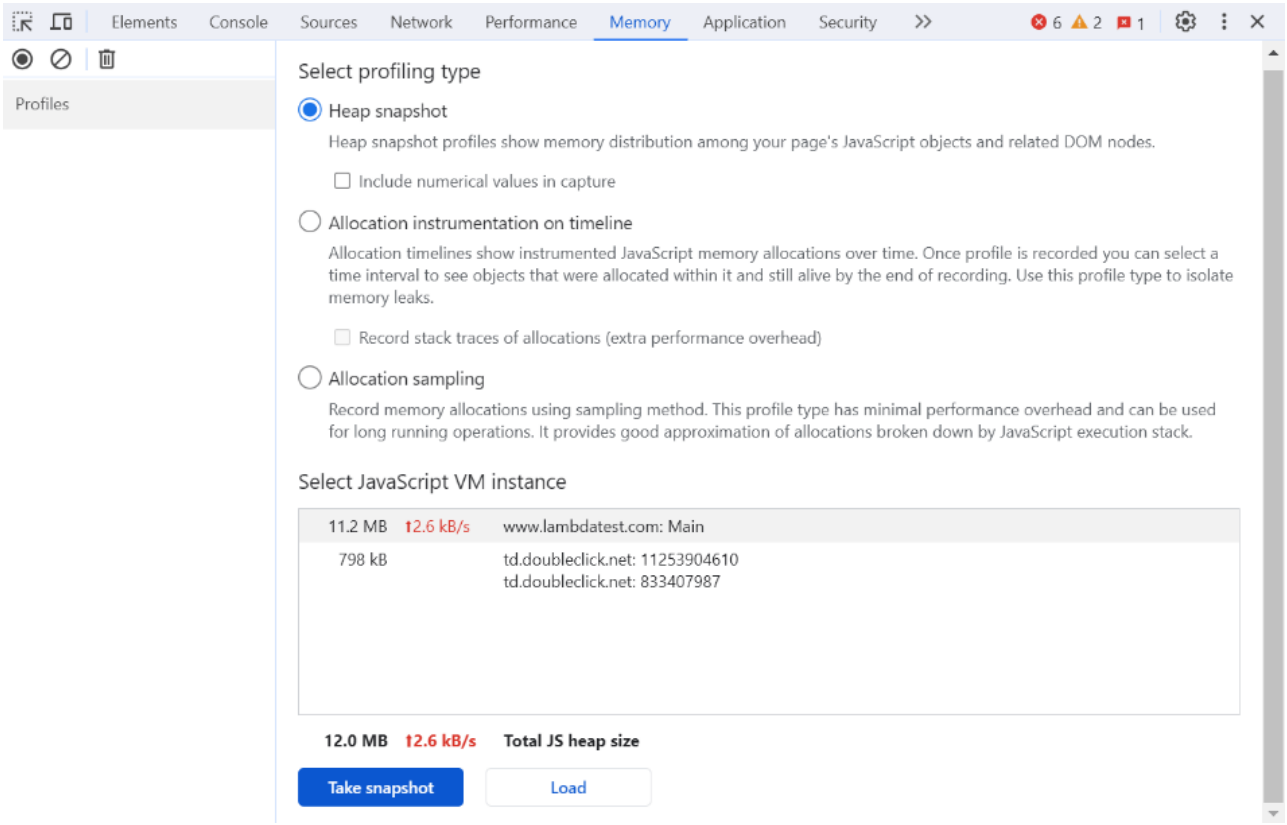


Рис. 3.9. Использование панели памяти в DevTools

Этот экран показывает сводку памяти, используемой веб-страницей, путем анализа общего размера кучи JavaScript веб-страницы. Подробные данные о потреблении памяти веб-страницей можно получить, нажав кнопку *Take Snapshot*. Панель памяти Chrome DevTools позволяет нам выбирать из трех типов профилирования.

Это один из наиболее часто используемых типов профилирования памяти. Он позволяет сделать снимок кучи JavaScript в определенный момент времени. Этот снимок показывает распределение памяти между различными элементами JavaScript и связанными с ними элементами DOM веб-страницы.

Этот тип профилирования предоставляет нам распределение памяти между различными элементами JavaScript и DOM с течением времени при прокрутке и взаимодействии с веб-страницей.

Представление Allocation Sampling отображает потребление памяти для отдельных компонентов страницы, таких как фреймы, документы, веб-работники и графические слои. Этот тип профилирования указывает источник любого высокого потребления памяти.

Панель приложений: используется для проверки и редактирования файла манифеста, *service worker, cookie, localDB, sessionStorage* и других локальных баз данных. Эта панель также используется для просмотра метаданных, протоколов и значков веб-сайта.

Чтобы открыть панель *приложений*, нажмите вкладку *«Приложение»* в Chrome DevTools.

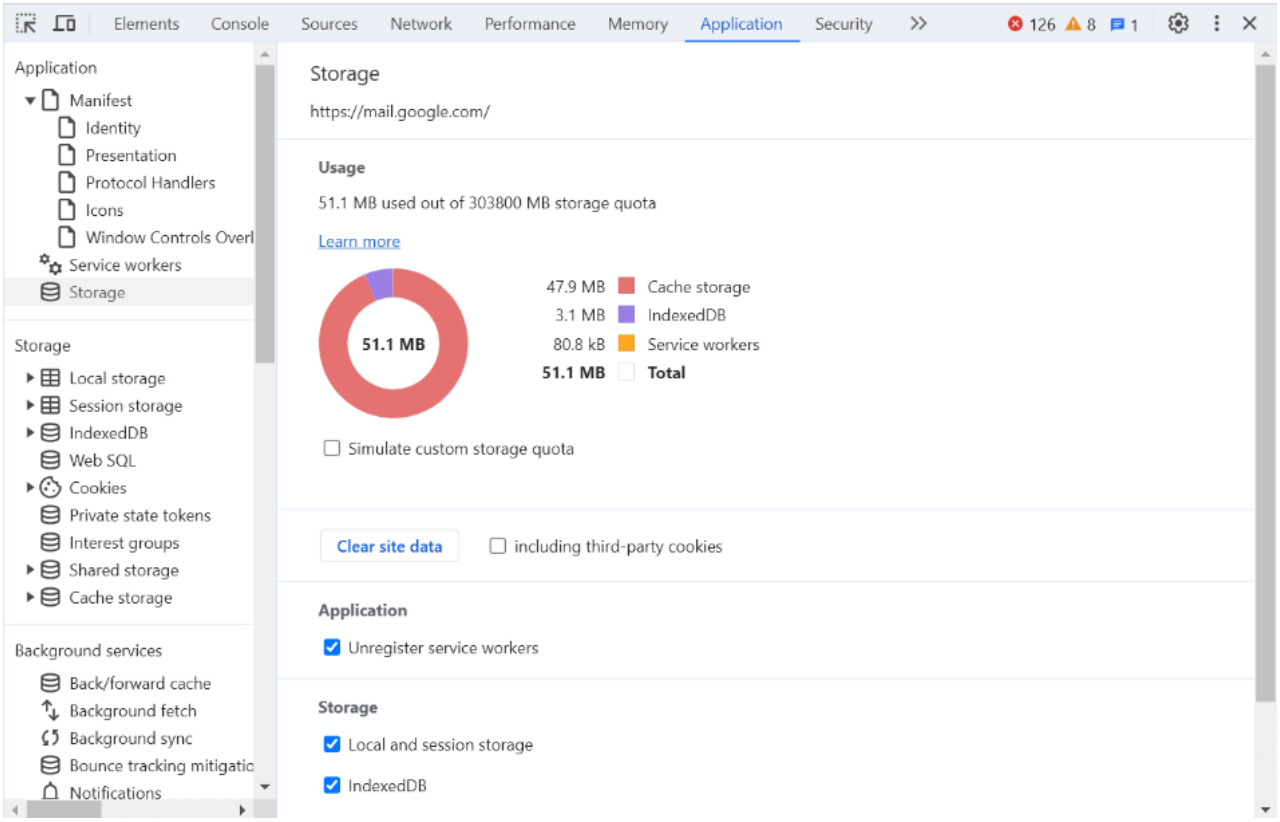


Рис. 3.10. Панель приложений DevTools

Панель *приложений* в основном полезна для проверки прогрессивных веб-приложений, использующих локальное хранилище и API. Это помогает разработчикам и тестировщикам управлять хранилищем, кэшами и базами данных; проверять и очищать файлы cookie; и проверять ресурсы.

Панель безопасности: отображает информацию о безопасности соединения для всех сетевых запросов. Эта панель используется для определения того, защищены ли

данные, собранные через сетевые запросы. Веб-сайт или сетевой запрос будет отображать зеленый значок замка, если он защищен.

Эта панель может использоваться для просмотра SSL-сертификатов и типов шифрования каждого сетевого запроса и всего веб-сайта. Чтобы открыть *панель безопасности*, мы можем нажать на вкладку *Безопасность* в Chrome DevTools.

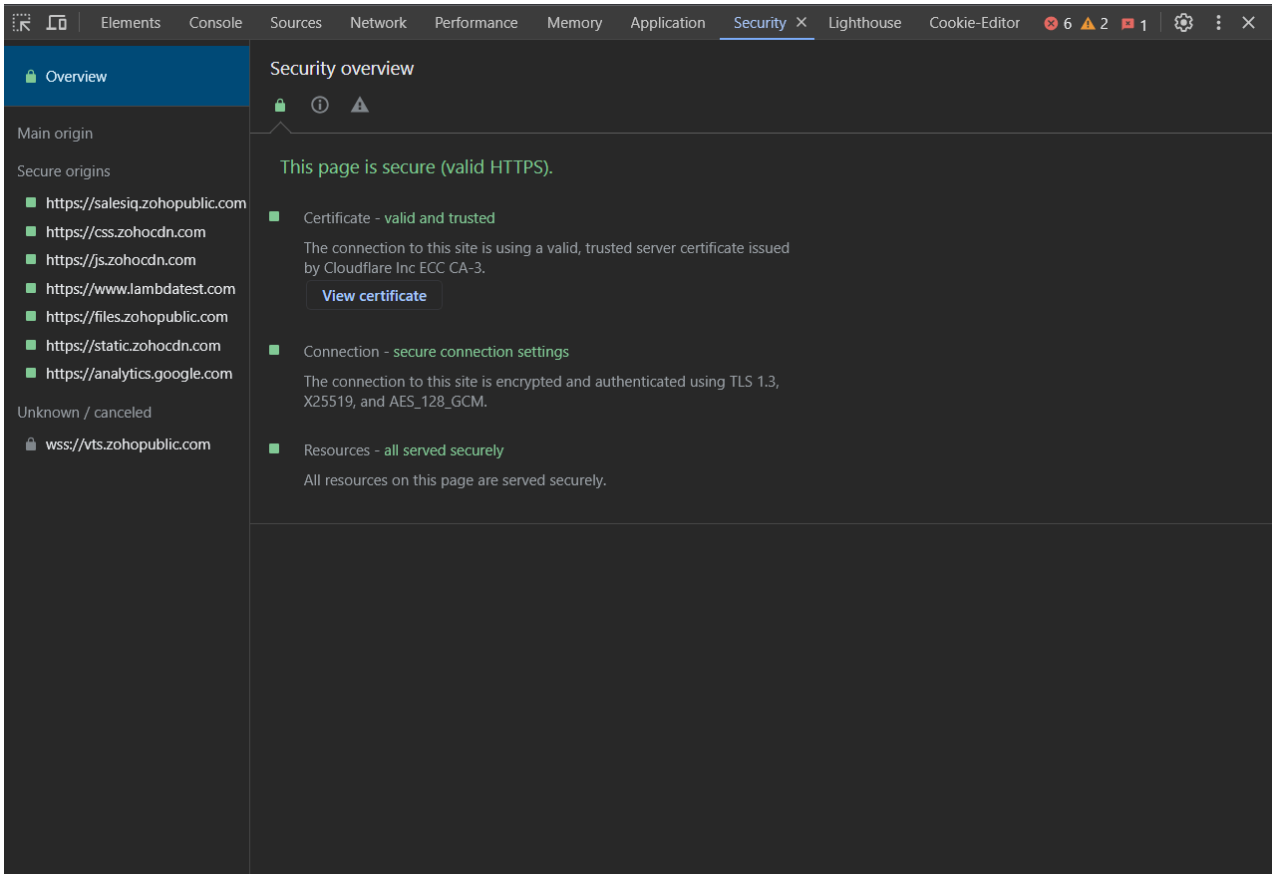


Рис. 3.11. Панель безопасности DevTools

Как показано на изображении выше, панель безопасности предоставляет информацию:

- Проверка сертификата, показывающая, подтвердил ли веб-сайт свою подлинность с помощью сертификата TLS.
- TLS-соединение, показывающее, использует ли веб-сайт современный и безопасный протокол.
- Безопасность подресурсов указывает, загружает ли веб-сайт небезопасные HTTP-подресурсы (смешанный контент).

Панель маяка: это бесплатный инструмент с открытым исходным кодом, который может помочь разработчикам и тестировщикам анализировать и улучшать производительность, скорость и общее впечатление от веб-сайта с помощью создания отчета Lighthouse.

Панель Lighthouse в инструментах разработчика Chrome оснащена Google Lighthouse, который может проводить аудит веб-сайта без дополнительных плагинов или виджетов. Для аудита веб-сайта в Chrome DevTool выполните следующие действия:

1. Перейдите на вкладку *Lighthouse* в инструментах разработчика Chrome.
2. Нажмите кнопку «Создать отчет» на изображении ниже.

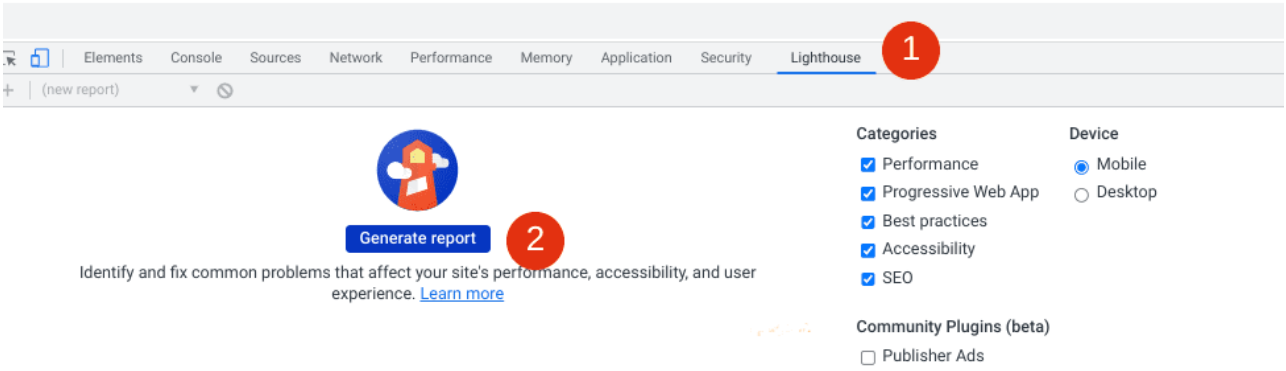


Рис. 3.12. Панель маяка DevTools

Он позволяет оценивать SEO веб-сайта, производительность, доступность, лучшие практики в коде и прогрессивное веб-приложение. Он также обладает высокой степенью настройки, что делает его гибким и полезным для всех типов сайтов, от простых до очень сложных. Эта гибкость включает несколько категорий для проведения аудита как на мобильных, так и на настольных устройствах.

Аудит веб-сайта использует систему оценок от 1 до 100, основанную на тестах, проведенных с использованием имитированного соединения 4G выбранных данных. Сгенерированный отчет для домашней страницы LambdaTest показан ниже.

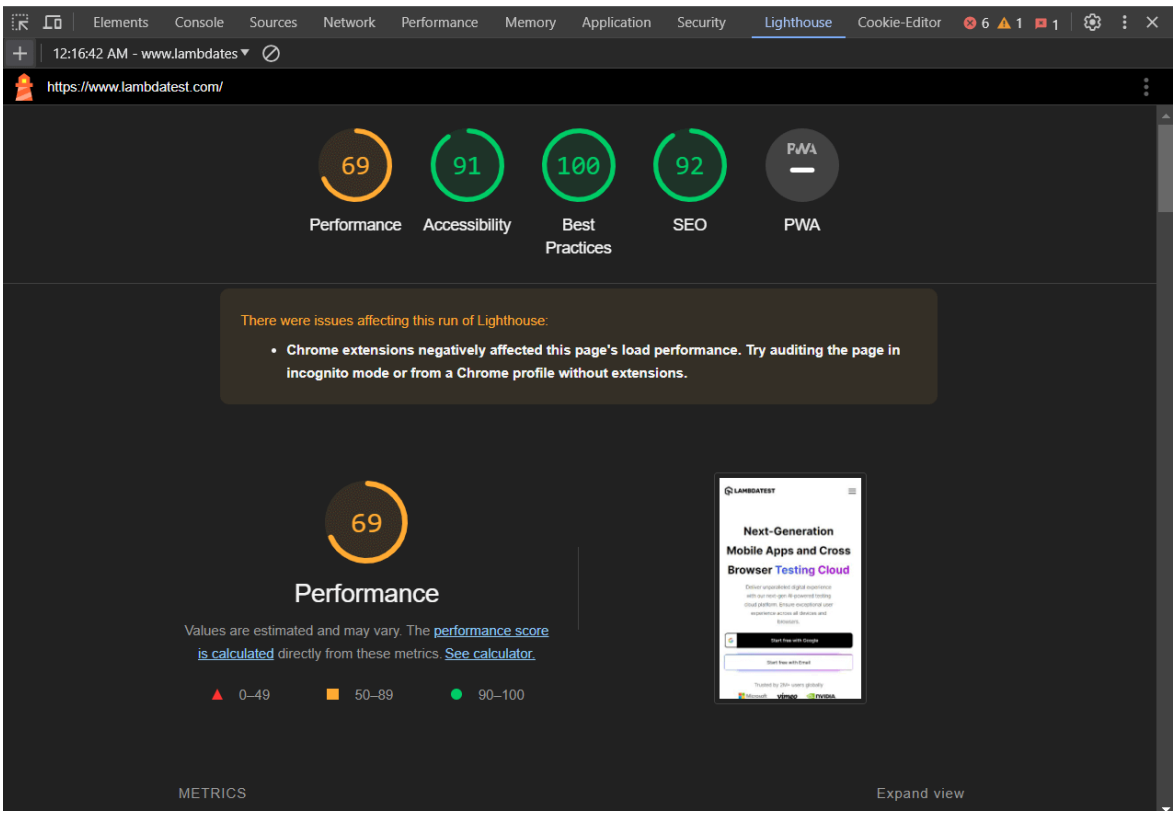


Рис. 3.13. Отчет для домашней страницы сайта

3.3. Облачные инструменты разработчика

Хотя инструменты разработчика Chrome, работающие на наших физических устройствах, могут значительно помочь нам анализировать и улучшать производительность веб-сайта в различных аспектах, мы должны гарантировать, что наш веб-сайт работает одинаково во всех различных версиях браузера, платформах и устройствах. Чтобы добиться этого, мы можем проводить тестирование в реальном времени на облачных инструментах тестирования.

Облачные инструменты автоматизации удобны, поскольку они могут тестировать наше приложение практически во всех популярных браузерах и платформах, обеспечивая лучшую гарантию качества для веб-приложения. Одной из таких популярных облачных платформ тестирования, которой доверяют как крупные предприятия, так и стартапы, является *LambdaTest*.

С помощью LambdaTest вы можете ускорить кроссбраузерное тестирование, выполняя интерактивное и динамическое тестирование в настольных и мобильных браузерах с помощью наших собственных облачных инструментов для разработчиков браузеров.

Чтобы запустить инструменты разработчика на LambdaTest, можно выполнить следующие шаги:

1. Прежде чем продолжить, нам необходимо создать учетную запись LambdaTest, зарегистрировавшись бесплатно, а затем перейти на панель управления.
2. Перейдите на панель управления тестированием в реальном времени, нажав кнопку «*В реальном времени*» в левом навигационном меню.

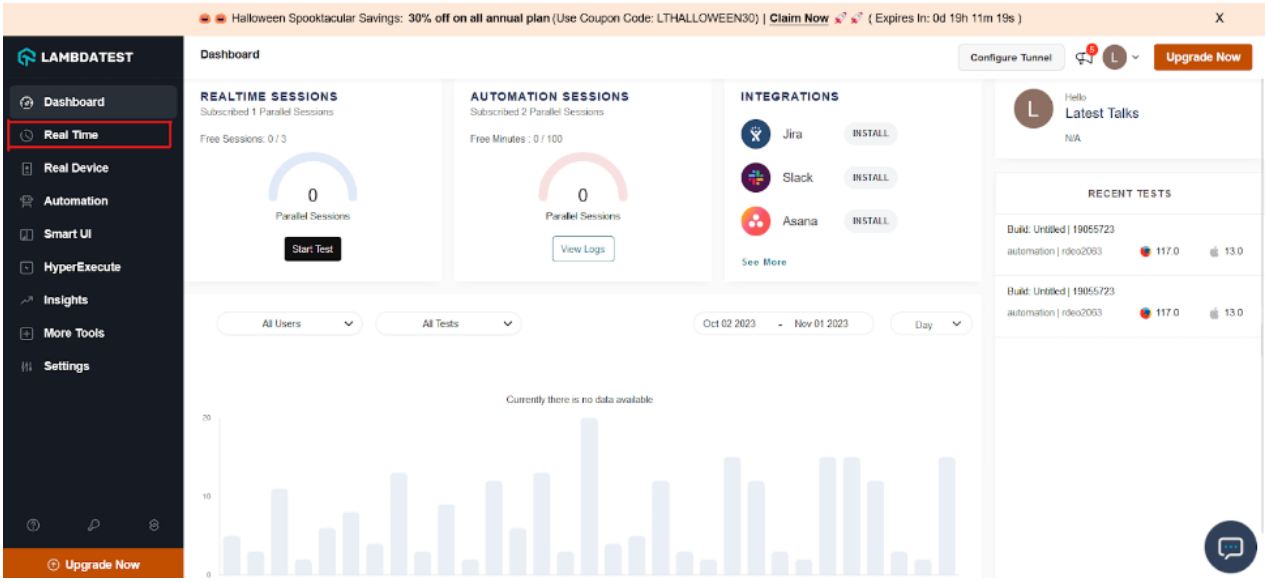


Рис. 3.14. Тестирование в реальном времени

3. Выберите тестовую конфигурацию по вашему выбору. Вы можете выбрать из 8 различных браузеров на настольных компьютерах и телефонах и их сотен версий для проведения сеанса тестирования в реальном времени. Вы также можете выбрать из нескольких операционных систем и разрешений экрана для оценки веб-сайта при отображении на различных устройствах с разными размерами экрана.

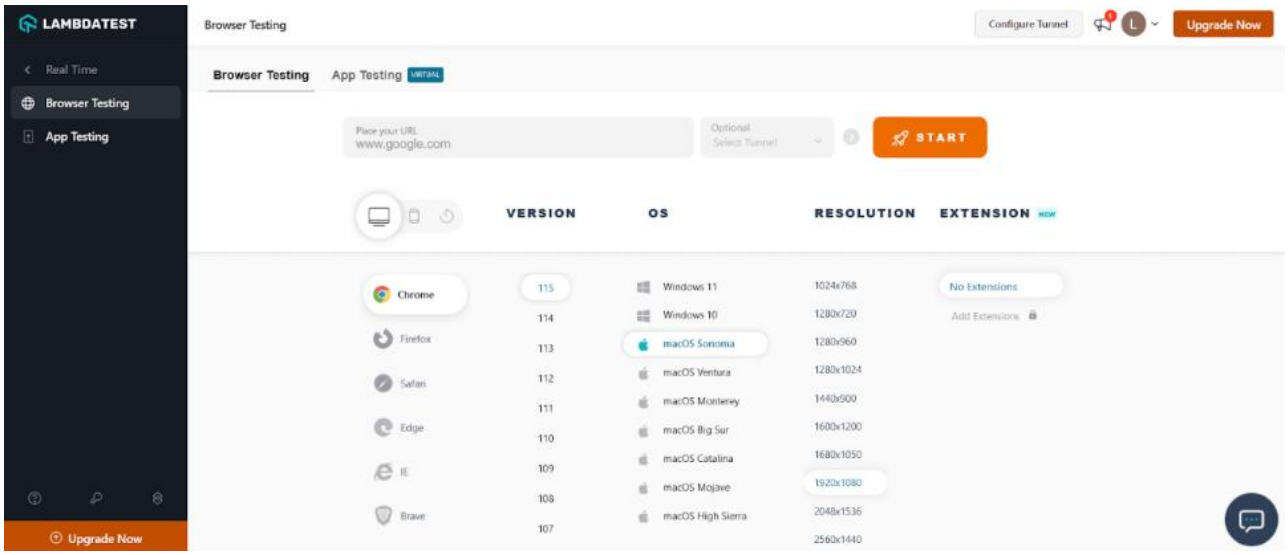


Рис. 3.15. Тестирование в реальном времени

4. После выбора конфигурации браузера введите URL-адрес и нажмите кнопку **«Старт»**, чтобы начать тестирование.

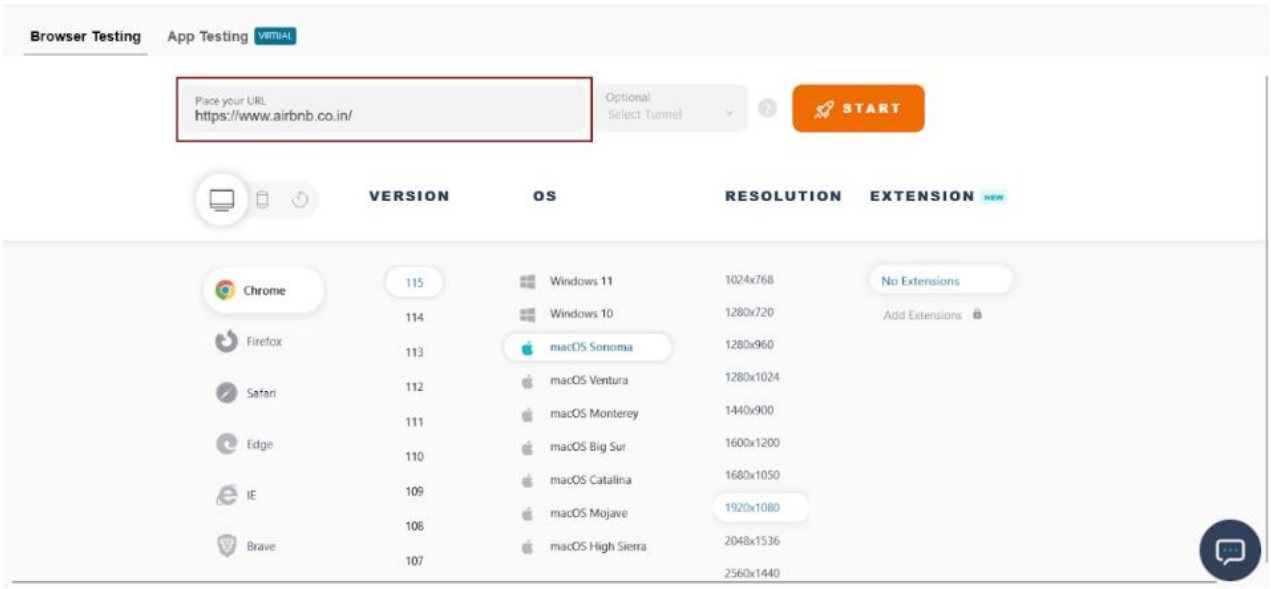


Рис. 3.16. Тестирование в реальном времени

5. **Для настольных устройств**: после того, как нужная виртуальная машина запущена и готова к работе, *щелкните правой кнопкой мыши* в любом месте загруженной веб-страницы и выберите опцию **«Проверить»**.

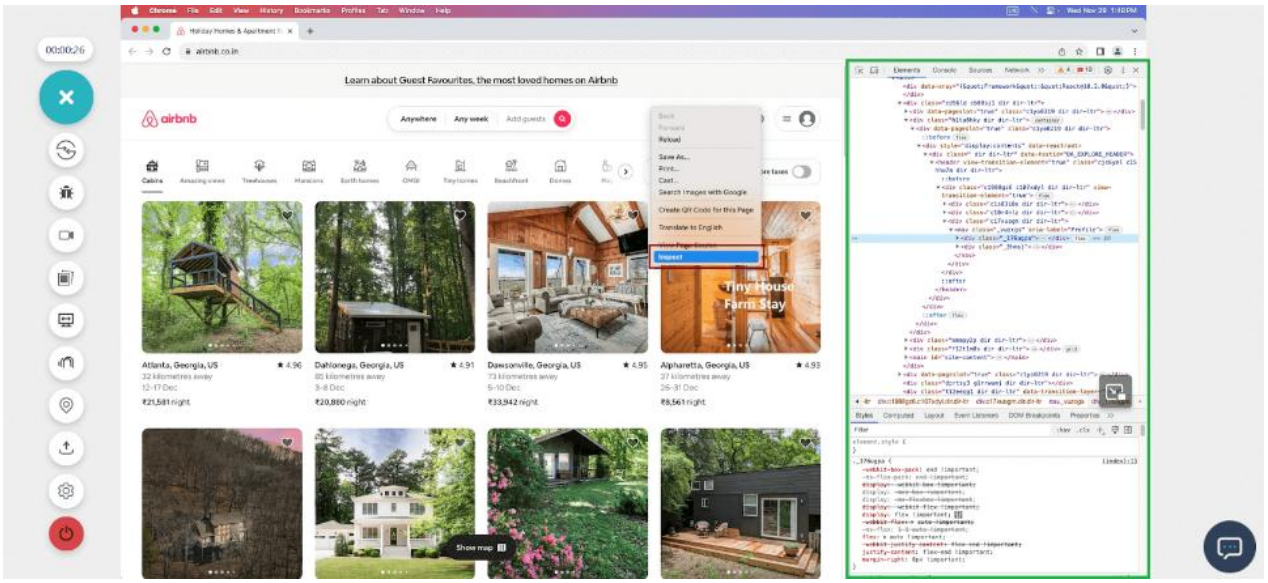


Рис. 3.17. Тестирование в реальном времени

Для мобильных устройств: если выбранная конфигурация виртуальной машины является мобильным устройством, и после ее запуска и работы вы можете заметить кнопки контроллера на левой стороне экрана. Нажмите на кнопку со значком инструментов, чтобы открыть инструменты разработчика.

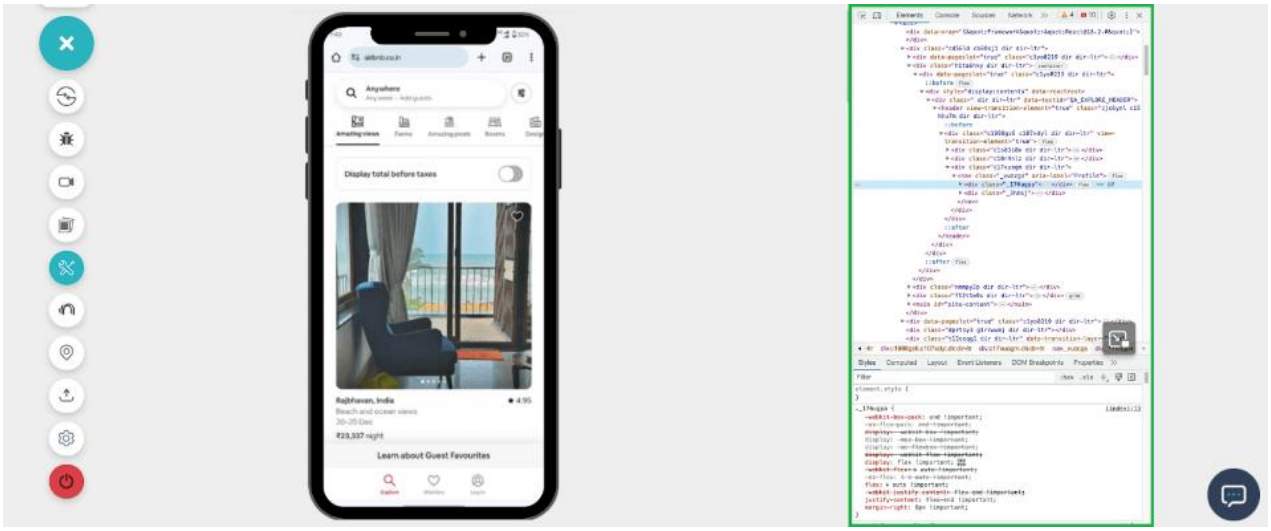


Рис. 3.18. Тестирование в реальном времени

3.4. Установка редактора VS Code

Чтобы установить редактор Visual Studio Code, выполним следующие действия:

- Перейдем на страницу загрузки VS Code.
- Загрузим последнюю версию VS Code, подходящую для ОС (Windows, macOS или Linux).
- Запустим загруженный пакет или установочный файл, чтобы запустить мастер установки. Процесс установки также довольно прост.
- Запустим установленный VS Code.

Мы увидим окно редактора, как показано ниже:

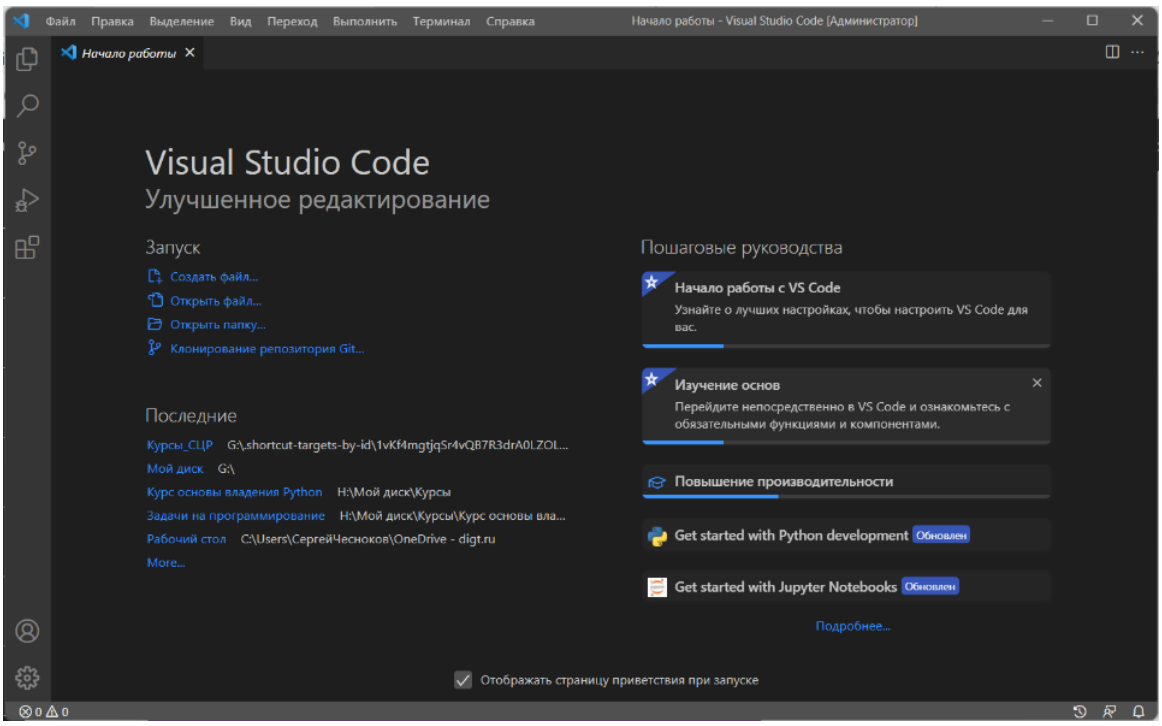


Рис. 3.19. Тестирование в реальном времени

Чтобы лучше ориентироваться в программе вы можете установить языковый пакет для русского языка (он также может погрузится автоматически). Для "ручной" загрузки откройте панель «Расширения» (Ctrl+Shift+X) и введите в ней текст «Russian Language Pack for Visual Studio Code». После этого выберите его в результатах поиска и установите его.

В Visual Studio Code имеется огромное количество различных плагинов кроме языковых пакетов. Предназначены они для расширения функциональности этого редактора. Все плагины для этого редактора размещены на Marketplace.

Добавление необходимых расширений

Установка LiveServer

Чтобы установить расширение Live Server:

- Перейдем на вкладку «**Расширения**», чтобы найти расширения для VS Code.
- Введем название Live Server**,** чтобы найти его.
- Нажмем кнопку «**Установить**».

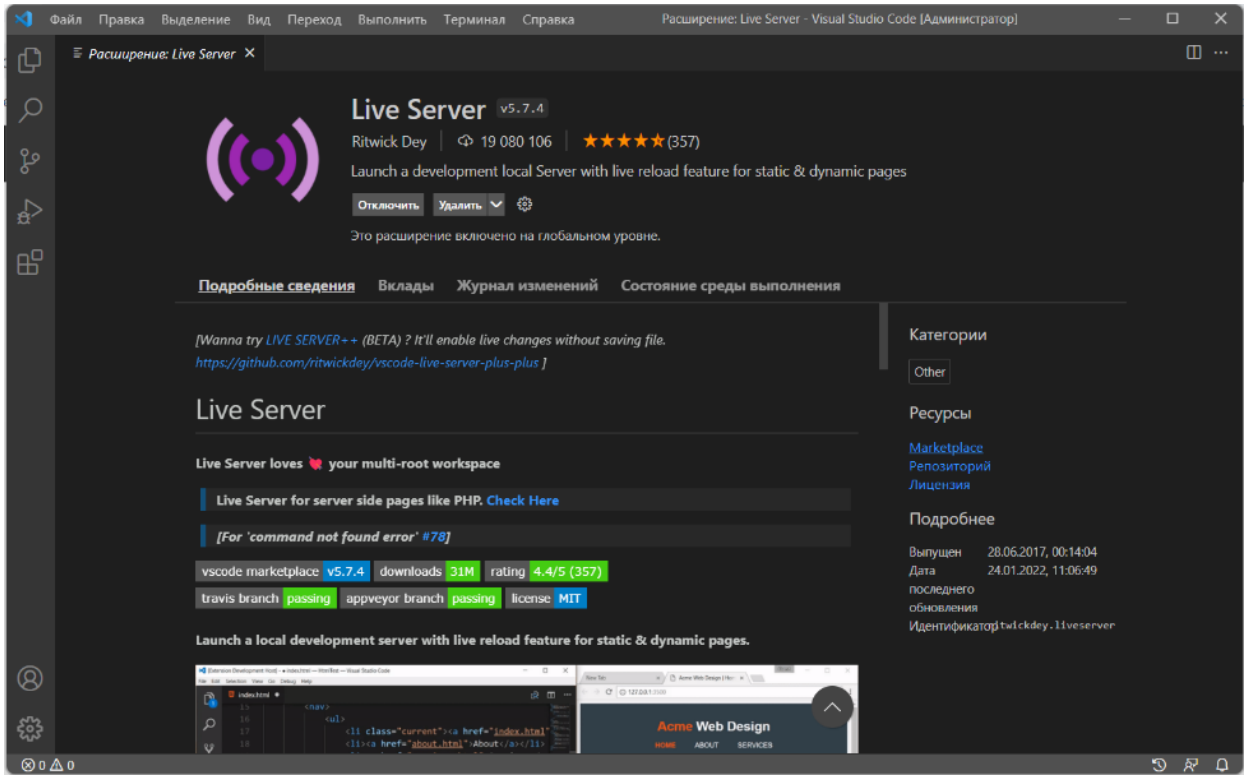


Рис. 3.20. Тестирование в реальном времени

Страница "Hello, World!" в веб-браузере

Рассмотрим процесс создание первой html-страницы с помощью редактора Visual Studio Code. И для этого нам понадобится установленный плагин Live Server.

Создадим новый файл с именем index.html в созданном каталоге helloworld и добавим в него код:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
        <meta   name="viewport"   content="width=device-width,   initial-
scale=1.0">
    <h1>Hello, World!</h1>
</head>
<body>
</body>
</html>
```

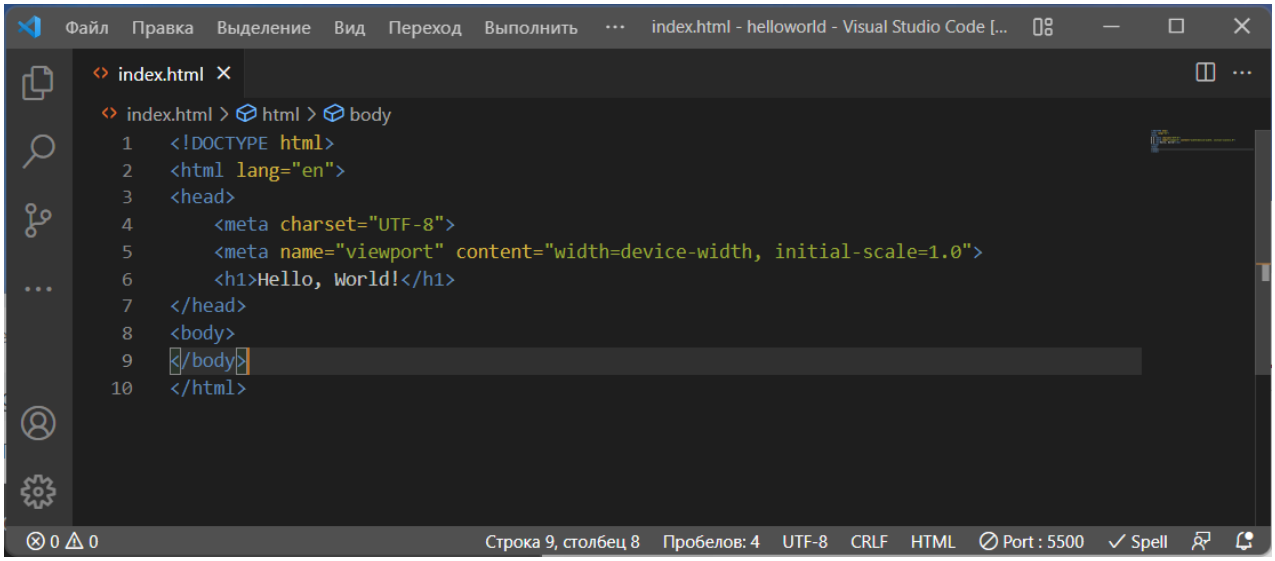


Рис. 3.21. Тестирование в реальном времени

Откроем Live Server из VS Code, щелкнув правой кнопкой мыши на файле index.html и выбрав опцию Open with Live Server. Live Server откроет index.html в новой вкладке браузера по умолчанию.

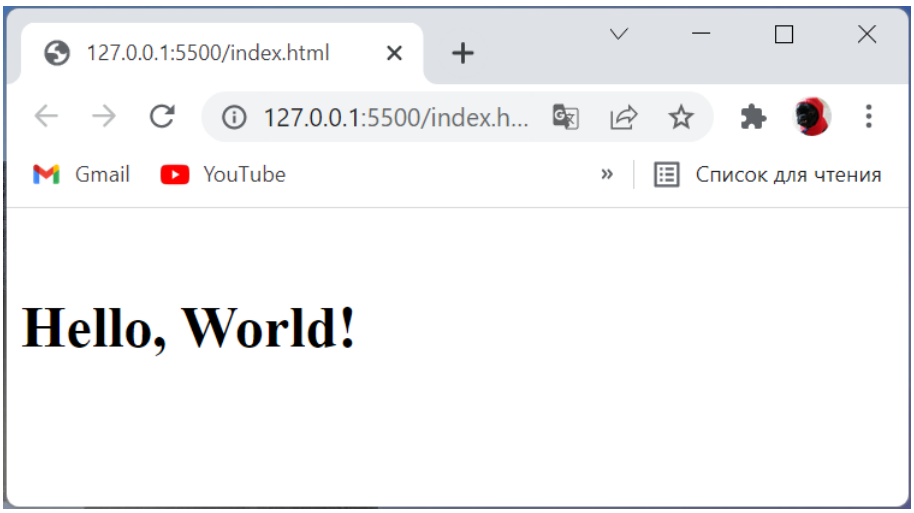


Рис. 3.22. Тестирование в реальном времени

При внесении изменений в файл index.html и сохранении этих изменений, браузер будет автоматически их отображать. Таким образом организуется очень удобный тандем: из редактора и браузера, в котором конечный результат будет всегда перед глазами.

При запуске сервера — Live Server будет считать корнем сервера директорию dist и запускать браузер по умолчанию. Это можно изменить в настройках VS Code — либо через графический интерфейс, либо редактируя файл settings.json.

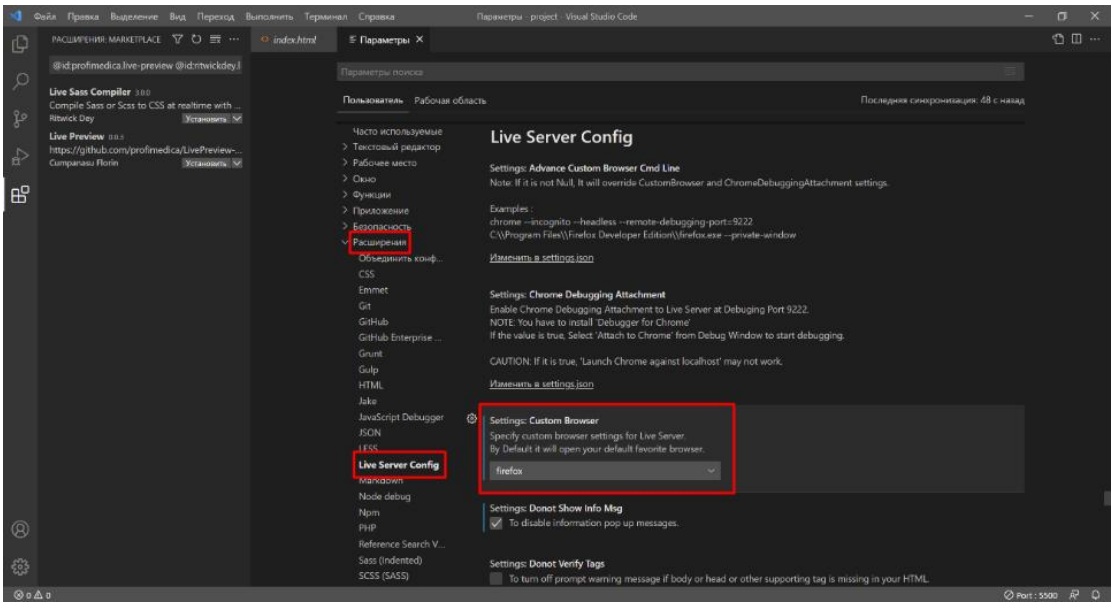


Рис. 3.24. Настройка плагина Live Server

Чтобы открыть на редактирование settings.json нужно кликнуть иконку в правом верхнем углу в разделе настроек.

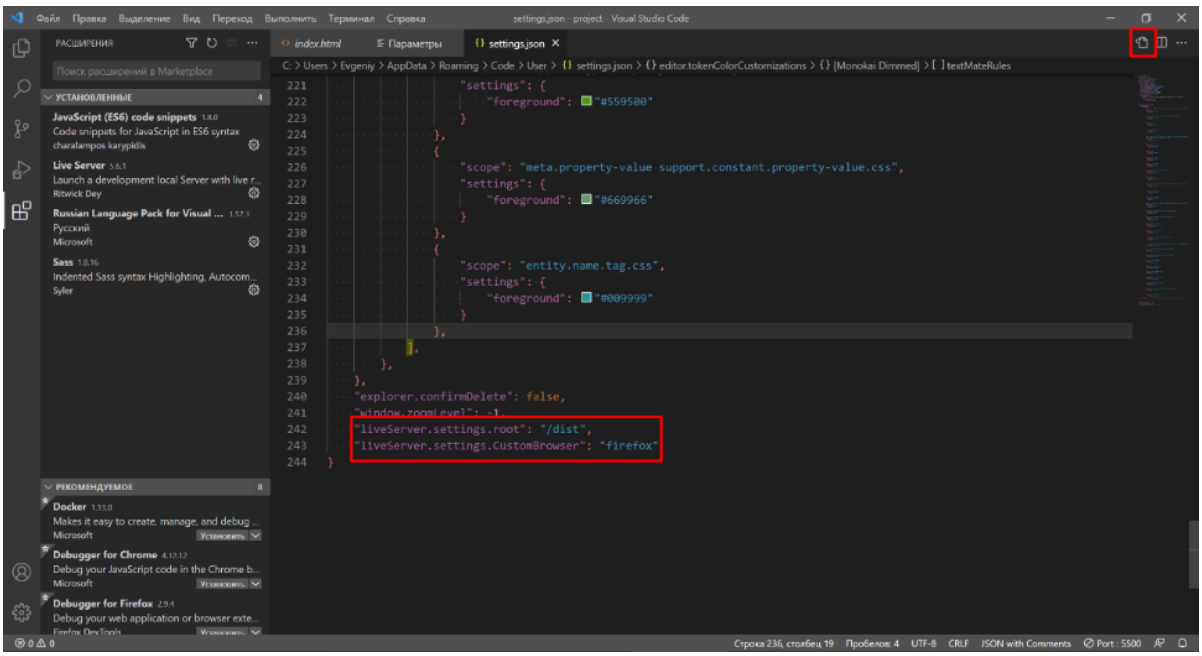


Рис. 3.25. Редактирование settings.json

Некоторые часто используемые настройки плагина:

- liveServer.settings.root — корень веб-сервера
- liveServer.settings.port — порт веб-сервера
- liveServer.settings.CustomBrowser — какой браузер запускать вместо дефолтного
- liveServer.settings.ignoreFiles — игнорировать изменения в этих файлах
- liveServer.settings.host — хост: 127.0.0.1 или localhost
- liveServer.settings.https — работать по протоколу https
- liveServer.settings.wait — задержка в мс перед перезагрузкой браузера
- liveServer.settings.NoBrowser — запускать сервер без открытия браузера
- liveServer.settings.fullReload — перезагружать браузер вместо обновления ссылки на css-файл внутри <head>

В VS Code есть также пользовательские настройки (User) и настройки рабочего пространства (Workspace). Пользовательские настройки применяются всегда, для любого запущенного экземпляра VS Code. Настройки рабочего пространства применяются для директории проекта — в нашем случае это файл project/.vscode/settings.json. Настройки рабочего пространства имеют приоритет над пользовательскими настройками.

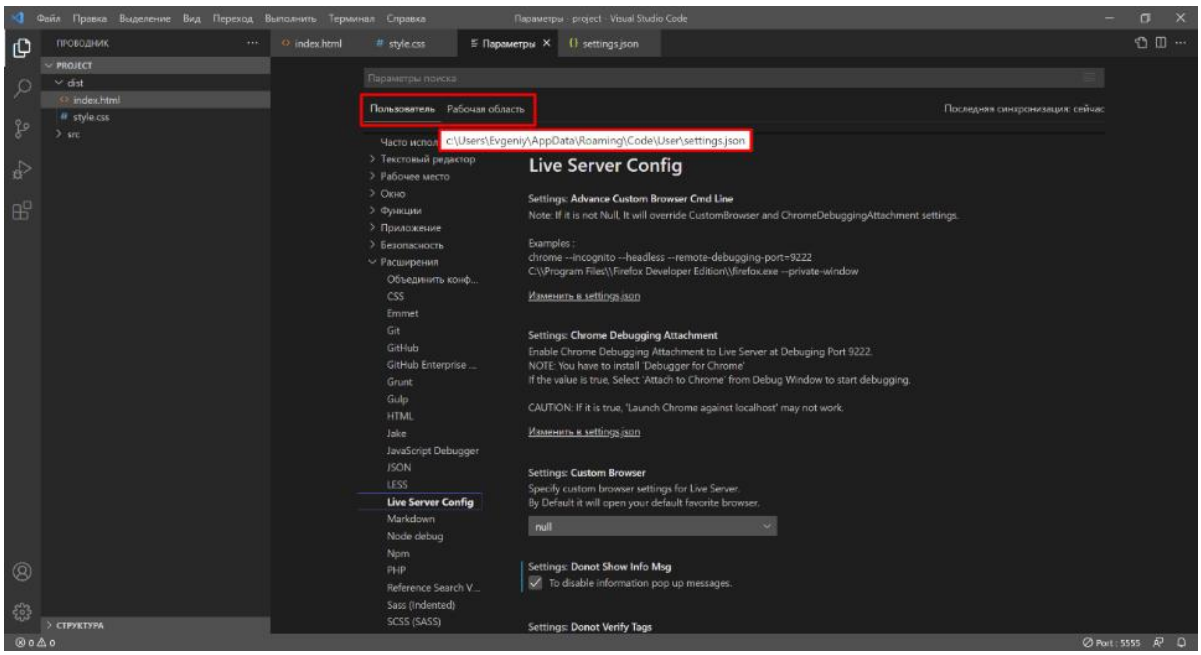


Рис. 3.26. Редактирование settings.json

3.5. Использование ngrok для создания тоннелей

Ngrok *— это кроссплатформенное приложение, которое создает* ***защищенные туннели (пути)*** *к локальной машине. Оно позволяет разработчикам с минимальными усилиями выставлять локальный сервер разработки в Интернет.*

Программное обеспечение делает так, что локально размещенный веб-сервер (например, компьютер, ноутбук, rasbery PI) выглядит размещенным на **поддомене ngrok.com.** Это означает, что не требуется публичный IP или доменное имя на локальной машине.

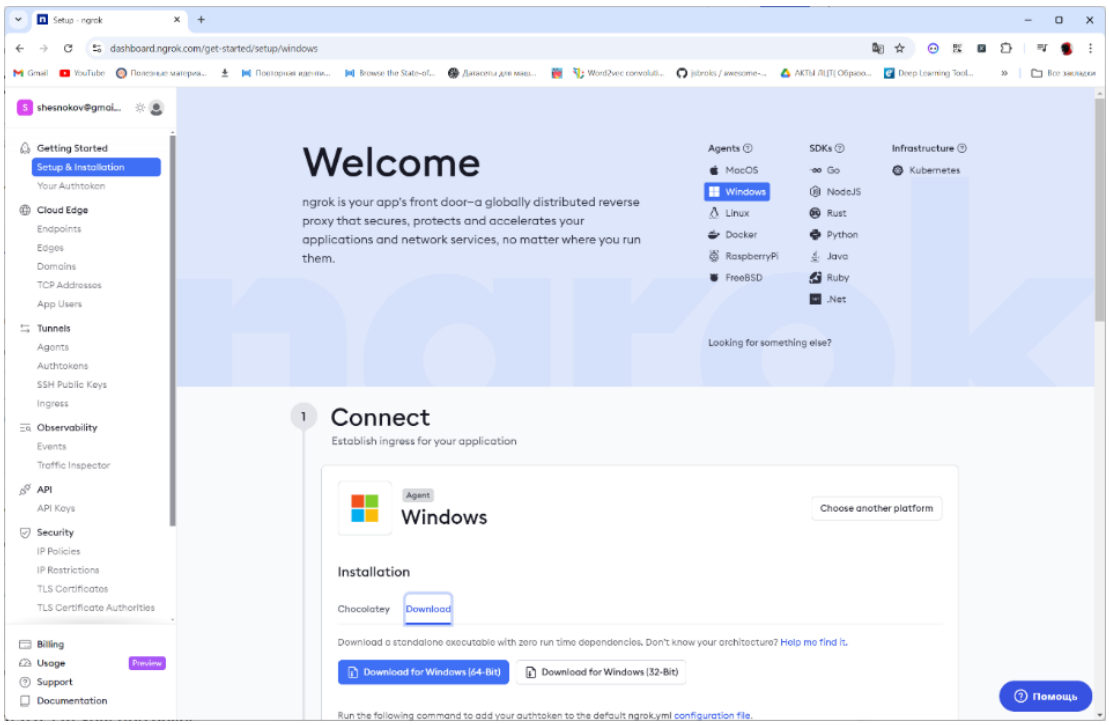


Рис. 3.27. Сайт загрузки приложения для организации тоннеля

Этот инструмент является чрезвычайно популярным решением, которое позволяет разработчикам тестировать свои веб-хуки и другие веб-приложения во время разработки. Скачать приложение для Windows (64-bit) можно по ссылке. Это один исполняемый файл, который можно разместить в любой директории. Для запуска приложения, используем команду:

```
ngrok http 8000
```

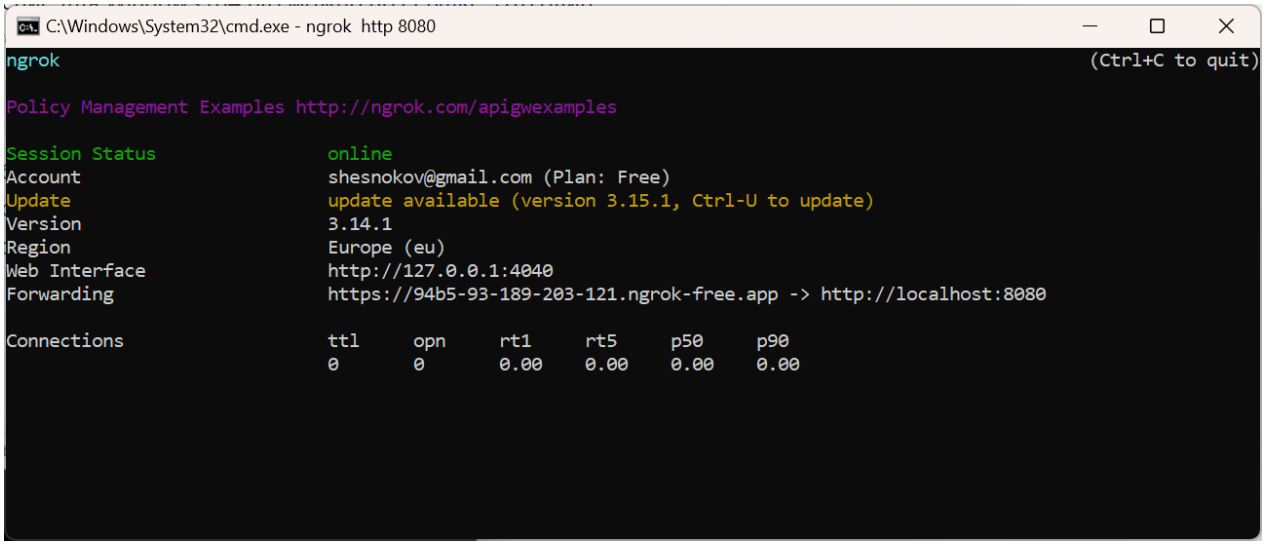


Рис. 3.28. Запуск приложения ngrok

Эта команда создает «туннель» между любым приложением, работающим на вашем компьютере и прослушивающим HTTP-запросы на локальном порту 8000, и

общедоступным URL-адресом на домене *ngrok.io*, который можно предоставить внешним пользователям. Когда кто-то подключается к URL-адресу Ngrok, служба Ngrok автоматически перенаправляет все запросы в приложение.

URL, который вы видите рядом с «Forwarding», — это общедоступный URL, который Ngrok назначил приложению. Этот URL можно использовать в любой точке мира.

Сервис Ngrok имеет бесплатные и платные уровни. Бесплатный уровень можно использовать анонимно или вместе с бесплатной учетной записью Ngrok. Подавляющее большинство учебных пособий, которые вы найдете в сети, используют бесплатный уровень и не требуют **создания учетной записи**. В качестве меры по предотвращению злоупотреблений Ngrok вводит ограничения для анонимных пользователей. Даже с учетом дополнительных ограничений Ngrok — это фантастический сервис, который обеспечивает потрясающую ценность. Проблема в том, что многие руководства были написаны для старых версий Ngrok и теперь содержат устаревшую информацию.

Одним из новых ограничений является **время истечения сеанса в 2 часа**. Когда соединение Ngrok достигает этой длины, оно перестает работать, и единственный способ восстановить его — остановить и перезапустить команду ngrok, что приводит к использованию **нового случайно сгенерированного URL**. Всегда можно проверить вывод Ngrok, чтобы узнать, когда соединение должно истечь.

Еще более серьезная проблема показана на следующем снимке экрана. Здесь отображается сообщение об ошибке, когда пользователь подключается к активному URL-адресу Ngrok:

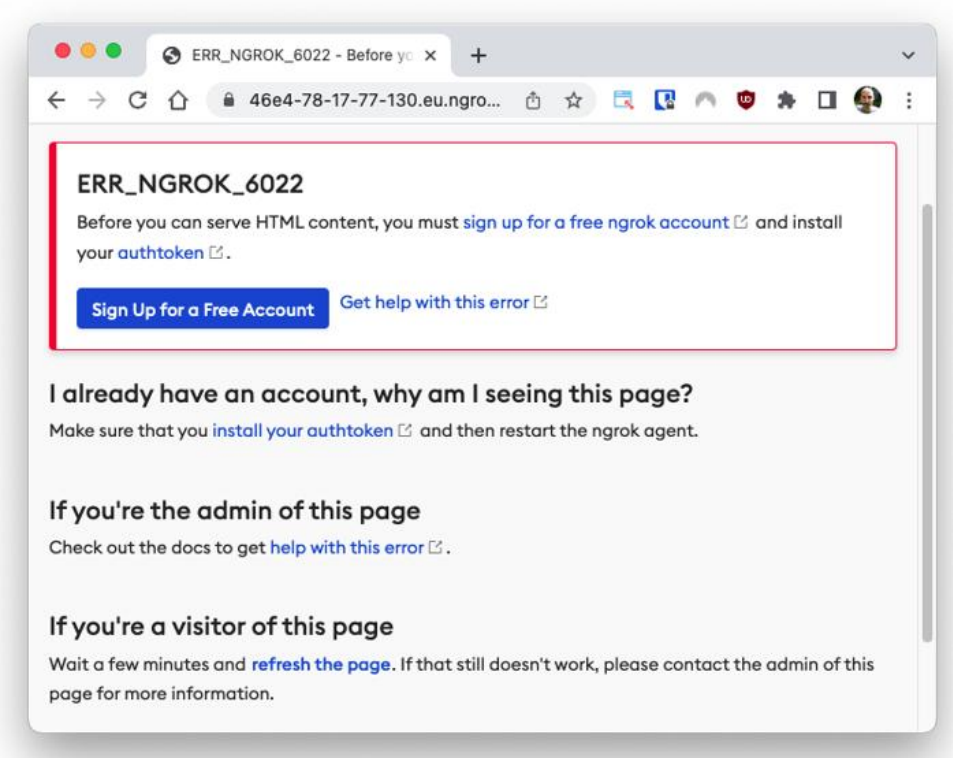


Рис. 3.29. Ошибка, связанная с анонимностью пользователя

Проблема здесь в том, что Ngrok больше **не позволяет анонимным аккаунтам обслуживать HTML-страницы**. Обратите внимание, что в настоящее время все еще возможно обслуживать не-HTML-контент анонимно. Создание бесплатной учетной записи Ngrok снимает эти ограничения, оставаясь на бесплатном уровне сервиса.

Чтобы создать учетную запись Ngrok, посетите их страницу регистрации, задача состоит в том, чтобы добраться до DashBoard. Можно создать свою учетную запись, указав имя пользователя, адрес электронной почты и пароль, или привязать свои учетные записи GitHub или Google. Получив учетную запись, войдите в панель управления Ngrok и нажмите «Y**our Authtoken**» на левой боковой панели.

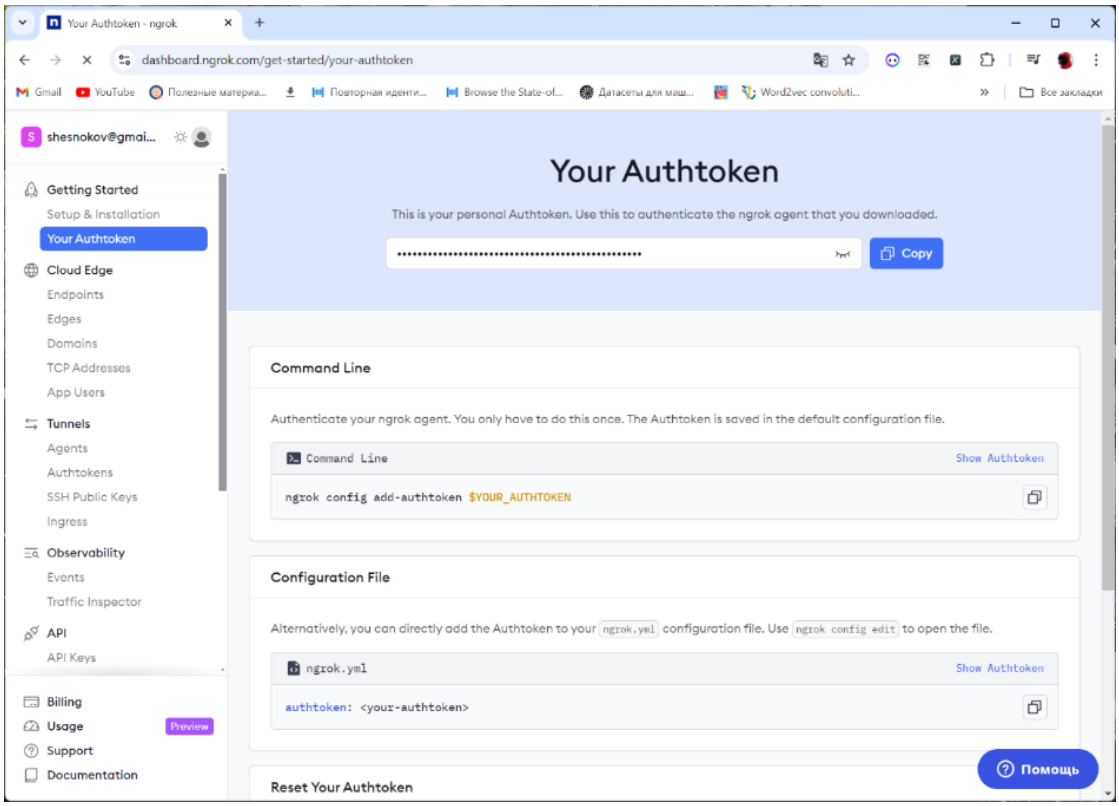


Рис. 3.30. Страница копирования токена авторизации

Здесь вы увидите присвоенный вам уникальный токен аутентификации.

Задания к практической работе

Задание 1. Установите код Visual Studio (VS)

- Установите VS Code и создайте свой первый HTML-код. Создайте новый файл. Назовите его **index.html**
- Установите расширение VS code для LiveServer.
- Добавьте свой личный заголовок. В разделе <body> создайте заголовок, который отображает ваше имя:

```
<h1>Your Name</h1>
```

- Щелкните правой кнопкой мыши и выберите **Открыть на LiveServer**.
- Он должен запустить локальный веб-сервер на порту 5500 или аналогичном, обработать страницу и открыть ее в веб-браузере по умолчанию.

Сделайте снимок экрана и добавьте его в качестве результата выполненных действий в отчет.

Задание 2. Разместите свою первую веб-страницу на GitHub Pages

GitHub Pages — это статический хостинг-сервис, который берет файлы HTML, CSS и JavaScript напрямую из репозитория на GitHub, опционально запускает файлы через процесс сборки и публикует веб-сайт. Мы будем использовать GitHub Pages для размещения веб-сайта напрямую из репозитория GitHub без дополнительных затрат.

Для начала, нам необходимо установить git (установить git для Windows).

- Создайте отдельный репозиторий для публикации веб-страниц, либо используйте уже существующий.
- Загрузите или отправьте туда свой файл ***index.html***, необходимо проделать эти действия как с использованием клиента git, так и с помощью функционала используемого IDE (Visual Studio Code).

Сделайте снимок экрана и добавьте результат выполненных действий в отчет.

Задание 3. Установка ngrok

Иногда мы хотим быстро продемонстрировать веб-сайты без их развертывания или размещения на сервере. Приложение **ngrok** выделяет имя хоста и порт на своем сервере и безопасно туннелирует локальный веб-сервер в Интернет, используя общедоступный домен. Он создает безопасное/зашифрованное соединение между общедоступной конечной точкой и вашей локальной машиной.

- Зарегистрируйте бесплатную учетную запись на ngrok через GitHub.
- Загрузите и установите ngrok. Под **Windows** - автономный исполняемый файл.

- Для настройки следуйте инструкциям по установке (https://dashboard.ngrok.com/get-started/setup/windows):

```
ngrok config add-authtoken YOUR_TOKEN_HERE
```

- Разместите свое веб-приложение в сети, перенаправив его на локальный порт 5500, выполнив команду:

```
ngrok http 5500
```

- Посетите локальную веб-страницу со своего смартфона.

Функционал ngrock доступен в виде плагина для IDE Visual Studio Code.

Сделайте снимок экрана и добавьте результат выполненных действий в отчет. Через определенное время тоннель будет автоматически закрыт (см. теоретический материал), необходимо в отведенное время продемонстрировать работу тоннеля преподавателю, либо включить иллюстративный материал по его работе в отчет.

Задание 4. Установка Postman

Postman — популярный клиент API, который упрощает разработчикам использование, создание, совместное использование, тестирование и документирование API. Мы будем использовать Postman в течение всего семестра для отправки запросов и получения ответов от веб-сервера и API.

- Установить и настроить PostMan
- Отправьте HTTP-запросы на тестовый API httpbin.org
  - Отправить **HTTP GET-** запрос http://httpbin.org/get
  - Отправить **HTTP POST-** запрос http://httpbin.org/post

Сделайте снимок экрана и добавьте результат выполненных действий в отчет.

Задание 5. Отправка отчета на проверку

Добавьте созданный отчет в каталог **lab-3** на GitHub. Прикрепите ссылку на каталог с работой в текущий курс для проверки преподавателем.

Контрольные вопросы

1. Что такое Visual Studio Code и какие его основные функции для веб-разработки?
2. Как установить расширения в Visual Studio Code и для чего они нужны?
3. Что такое LiveServer и как он облегчает процесс разработки веб-приложений?
4. Как запустить LiveServer в Visual Studio Code?
5. Что такое Postman и как он используется для тестирования API?
6. Как создать новый запрос в Postman и какие типы запросов можно использовать?
7. Что такое коллекции в Postman и как они помогают в организации запросов?

8. Как можно использовать переменные в Postman для упрощения работы с запросами?
9. Как настроить заголовки в запросах Postman и зачем это нужно?
10. Что такое GitHub Pages и как он используется для хостинга веб-сайтов?
11. Как опубликовать сайт на GitHub Pages?
12. Что такое index.html и почему он важен для веб-приложений?
13. Как использовать систему контроля версий Git в Visual Studio Code?
14. Что такое коммиты и как они влияют на историю проекта в Git?
15. Как разрешать конфликты при слиянии веток в Git?
16. Как настроить автоматическое обновление страницы при изменении файлов с помощью LiveServer?
17. Что такое REST API и как его тестировать с помощью Postman?
18. Как использовать Markdown в README-файлах на GitHub?
19. Как организовать структуру проекта для веб-разработки в Visual Studio Code?
20. Как использовать инструменты разработчика в браузере для отладки веб-приложений?

ПРАКТИЧЕСКАЯ РАБОТА 4. ЗНАКОМСТВО С БАЗОВЫМИ ТЕГАМИ HTML

Цель работы

- **Ознакомление с основами HTML:** изучить структуру HTML-документа, понять назначение и использование базовых тегов.
- **Создание простой веб-страницы:** научиться создавать HTML-документ с использованием основных тегов, таких как <html>, <head>, <title>, <body>, <h1>, <p>, <a>, <img> и других.
- **Форматирование текста:** изучить теги для форматирования текста, такие как , , <blockquote>, и научиться применять их для улучшения читаемости контента.
- **Работа с гиперссылками:** научиться создавать гиперссылки с помощью тега <a> и использовать атрибуты, такие как href и target.
- **Вставка изображений:** изучить, как вставлять изображения на веб-страницу с помощью тега <img>, а также освоить атрибуты src, alt и title.

Теоретическая часть

4.1. Структура HTML элементов

***HTML** (**HyperText Markup Language** - язык гипертекстовой разметки) не является языком программирования; это* язык разметки*, используемый для определения структуры веб-страниц, посещаемых пользователями.*

Веб-страницы могут иметь сложную или простую структуру, всё зависит от замысла и желания веб-разработчика. HTML состоит из ряда *элементов*, которые вы используете для того, чтобы охватить, обернуть или разметить различные части содержимого, чтобы оно имело определённый вид или срабатывало определённым способом. *Встроенные тэги* могут преобразовать часть содержимого в гиперссылку, по которой можно перейти на другую веб-страницу, выделить курсивом слова и так далее.

Например, рассмотрим следующую строку:

Мой кот очень сердитый

Если необходимо, чтобы строка отобразилась в таком же виде, нужно определить её, как "параграф", заключив её в теги элемента "параграф" (<p>), например:

```
<p>Мой кот очень сердитый</p>
```

Вот так это выглядит на экране

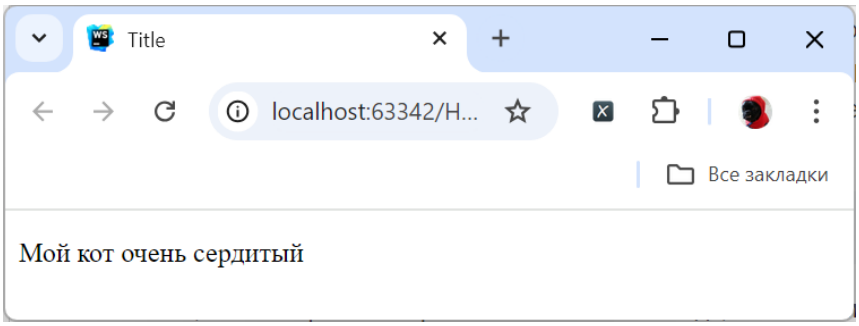


Рис. 4.1. Отображение параграфа

Если мы не поставим тег параграфа, а напишем тест внутри тела HTML документа, ошибки не будет и браузер его отобразит. Но такой подход будет работать до того момента, пока не возникнет необходимость располагать на странице и стилизовать контент. Тогда обычный текст (без приведения к элементу HTML) доставит много проблем.

Теги в HTML нечувствительны к регистру, то есть они могут быть записаны в верхнем или нижнем регистре. Например, тег <title> может быть записан как <title>, <TITLE>, <Title>, <TiTlE>, и т.д., и он будет работать нормально. Лучшей практикой, однако, является запись всех тегов в нижнем регистре для обеспечения согласованности, удобочитаемости и других причин.

Давайте рассмотрим элемент "параграф" чуть подробнее:

Рис. 4.2. Структура HTML элемента «параграф»

Основными частями элемента являются:

1. **Открывающий тег:** он состоит из названия (обозначения) элемента (в нашем случае, *p*), помещённого внутри *угловых скобок*. Данный тег служит признаком

начала элемента, с этого момента тег начинает влиять на следующее после него содержимое.

2. **Закрывающий тег:** выглядит, как и открывающий, но содержит слеш перед названием тега. Он служит признаком конца элемента. Пропуски закрывающих тегов — типичная ошибка новичков, которая может приводить к неопределённым результатам — в лучшем случае всё сработает правильно, в других страница может вовсе не прорисоваться или прорисоваться не как ожидалось.
3. **Содержимое:** как видно, в нашем случае содержимым является простой текст.
4. **Элемент:** открывающий тег + закрывающий тег + содержимое = элемент.

Вложенные элементы

Мы также можем вкладывать элементы внутрь других элементов — это называется **вложенностью**. Если мы хотим подчеркнуть, что наш кот **очень** сердитый, мы можем заключить слово "очень" в элемент , который означает, что это слово крайне важно в данном контексте:

```
<p>Мой кот <strong>очень</strong> сердитый.</p>
```

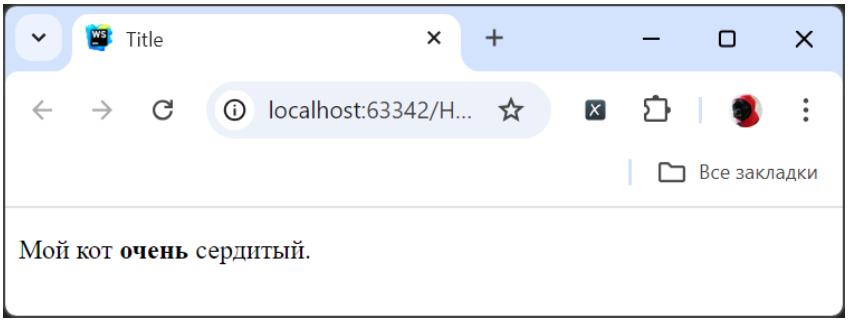


Рис. 4.3. Отображение вложенных элементов

Мы должны удостовериться, что элементы вложены должным образом: в следующем примере мы открываем элемент p первым, затем элемент strong, затем мы закрываем элемент strong первым, затем p. Например, написание подобным образом, будет считаться ошибкой:

```
7    <body>
8    <p>Мой кот <strong>очень сердитый.</p></strong>
9    </body>
```

Рис. 4.4. Ошибка в использовании вложенных элементов

Элементы должны открываться и закрываться правильно таким образом, чтобы явно находиться внутри или снаружи друг друга. Если они перекрываются так, как в примере выше, то ваш браузер попытается «додумать», что имелось в виду, и мы получили непредсказуемый результат. Так что не делайте так!

Написание комментариев в HTML

Комментарии обычно добавляются с целью сделать исходный код более понятным. Это может помочь другому разработчику (или вам в будущем, когда вы будете редактировать исходный код) понять, что было сделано в коде HTML. Комментарии не отображаются в браузере.

Комментарий HTML начинается с <!-- и заканчивается на -->, как показано в примере ниже:

```
<!-- Это HTML-комментарий -->
<!-- Это многострочный HTML-комментарий.
     который занимает более одной строки -->
<p>Это обычный текст.</p>
```

Мы также можем закомментировать часть своего HTML-кода для целей отладки, как показано здесь:

```
<!-- Скрытие этого изображения для тестирования
<img src="images/smiley.png" alt="Smiley">
-->
```

Блочные и строчные элементы

Существует две важные *категории элементов* в HTML, которые стоит знать — элементы *блочного уровня* и *строчные элементы*.

- *Элементы блочного уровня* формируют видимый блок на странице — они окажутся на новой строке после любого контента, который шёл до них, и любой контент после них также окажется на новой строке. Чаще всего элементами блочного уровня бывают структурные элементы страницы, представляющие собой, например, *параграфы (абзацы), списки, меню навигации, футеры, **или** подвалы*, и т.п. Элементы блочного уровня не вкладываются в строчные элементы, но иногда могут вкладываться в другие элементы блочного уровня.
- *Строчные элементы* — это те, которые содержатся в элементах блочного уровня и окружают только малые части содержимого документа, не целые абзацы и группировки контента. Строчные элементы не приводят к появлению новой строки в документе: они обычно встречаются внутри абзаца текста, например, элемент <a> (*ссылка*) или акцентирующие элементы вроде или .

Посмотрите на следующий пример:

```
<em>Первый</em>
<em>второй</em>
<em>третий</em>

<p>четвёртый</p>
```

```
    <p>пятый</p>
    <p>шестой</p>
```

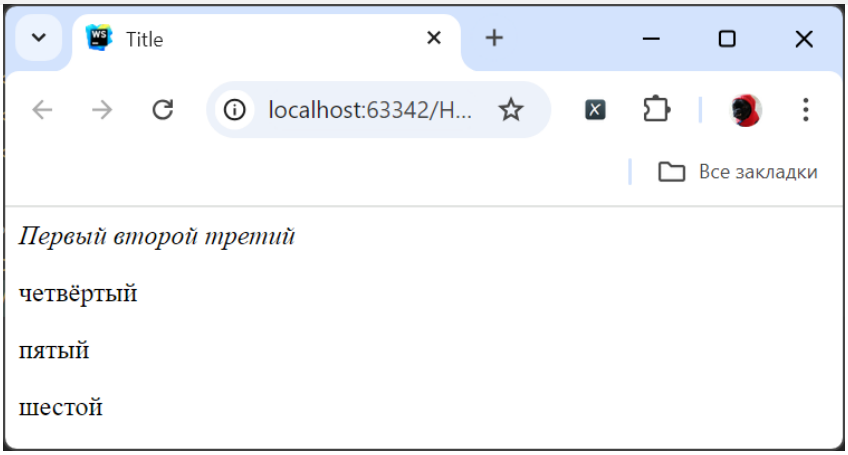


Рис. 4.3. Отображение вложенных элементов

 — это *строчный элемент*, так что, как вы здесь видите, первые три элемента находятся на одной строке друг с другом без пробелов между ними. С другой стороны, <p> — это *элемент блочного уровня*, так что каждый элемент находится на новой строке, с пространством выше и ниже каждого (этот интервал определяется CSS-оформлением по умолчанию, которое браузеры применяют к абзацам).

HTML5 переопределил категории элементов в HTML: смотрите Категории типов содержимого элементов. Хотя эти определения точнее и однозначнее, чем те, которые были раньше, их гораздо сложнее понять, чем «блочный» и «строчный».

Не путайте термины «блочный» и «строчный», используемые здесь, с одноимёнными *типами отображения в CSS*. Хотя по умолчанию они коррелируют, смена типа отображения в CSS не меняет категорию элемента и не влияет на то, во что его можно вкладывать и что можно вкладывать в него. Эта довольно частая путаница — одна из причин, почему HTML5 отказался от этих терминов.

Пустые элементы

Не все элементы соответствуют вышеупомянутому шаблону: открывающий тег, контент, закрывающий тег. Некоторые элементы состоят из одного тега и обычно используются для вставки чего-либо в то место документа, где размещены. Например, элемент <img> вставляет картинку на страницу в том самом месте, где он расположен:

```
    <img
      src="https://github.com/SerjiEvg/web-development-part1/blob/main/Material/angry_cat.jpg?raw=true" />
```

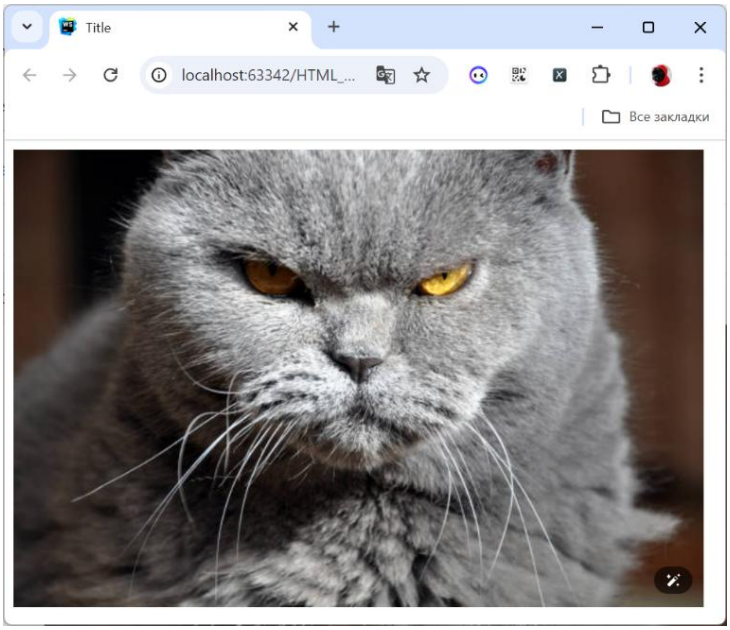


Рис. 4.4. Отображение картинки

Специальные символы в HTML

В HTML символы <, >, ", ' и & являются *специальными*. Они являются частью самого синтаксиса HTML. Так как же включить в текст один из этих специальных символов? Например, если вы хотите использовать амперсанд или знак «меньше» и не интерпретировать его как код.

Мы должны использовать ссылки-мнемоники — специальные коды, которые отображают спецсимволы, и могут быть использованы в необходимых позициях. Каждая *ссылка-мнемоник* начинается с амперсанда (&) и завершается точкой с запятой (;).

Таблица 4.1. Коды для отображения спецсимволов

Буквенный символ	Символьный эквивалент
<	<
>	>
"	"
'	'
&	&

В следующем примере вы видите два абзаца, которые рассказывают о веб-технологиях:

```
<p>В HTML вы определяете параграф элементом <p>.</p>
<p>В HTML вы определяете параграф элементом &lt;p&gt;.</p>
```

В живом выводе ниже вы можете заметить, что первый абзац выводится неправильно, так как браузер считает, что второй элемент <p> является началом нового абзаца!

Второй абзац нашего кода выводится правильно, потому что мы заменили угловые скобки на ссылки-мнемоники.

HTML-атрибуты

Атрибуты определяют дополнительные характеристики или свойства элемента, например, такие как ширина и высота изображения. Атрибуты всегда указываются в начальном (или открывающем теге) теге и обычно состоят из пар имя/значение - name="value". Значения атрибутов всегда должны заключаться в кавычки.

Кроме того, для определенных элементов требуются некоторые атрибуты. Например, тег <img> должен содержать атрибуты src и alt. Давайте посмотрим некоторые примеры использования атрибутов:

```
<img src="images/smiley.png" width="30" height="30" alt="Smiley">
<a href="https://www.yandex.ru/" title="Поисковик">Яндекс</a>
<abbr title="Язык гипертекстовой разметки">HTML</abbr>
<input type="text" value="Николай Васильевич">
```

В приведенном выше примере src внутри тега <img> является именем атрибута, а путь к изображению - его значением. Точно так же href внутри тега <a> является атрибутом, а предоставленная ссылка - его значением и так далее.

Для обозначения значений атрибутов можно использовать как одинарные, так и двойные кавычки. Однако наиболее распространены **двойные кавычки**. В ситуациях, когда само значение атрибута содержит двойные кавычки, необходимо заключить значение в одинарные кавычки, например, value='John "Williams" Jr.'

В HTML5 есть несколько атрибутов, которые не состоят из пар имя/значение, а состоят только из имени. Такие атрибуты называются **логическими атрибутами**. Примерами некоторых часто используемых логических атрибутов являются checked, disabled, readonly, required и т.д.

```
<input type="email" required>
<input type="submit" value="Submit" disabled>
<input type="checkbox" checked>
<input type="text" value="Read only text" readonly>
```

Значения атрибутов, как правило, нечувствительны к регистру, за исключением некоторых значений атрибутов, таких как атрибуты id и class. Однако Консорциум **World**

Wide Web (W3C) в своей спецификации рекомендует использовать строчные буквы для значений атрибутов.

Атрибуты общего назначения

Есть некоторые атрибуты, такие как id, title, class, style и т.д., которые можно использовать для большинства элементов HTML. В следующем разделе описывается их использование.

Атрибут идентификатора

Атрибут id используется для присвоения **уникального имени** или **идентификатора** элементу в документе. Это упрощает выбор элемента с помощью CSS или JavaScript.

```
<input type="text" id="firstName">
<div id="container">Некоторый контент</div>
<p id="infoText">Это параграф.</p>
```

Элемент id должен быть уникальным в пределах одного документа. Никакие два элемента в одном и том же документе не могут иметь одинаковые имена id. Каждый элемент может иметь только один id.

Атрибут класса

Подобно атрибуту id, атрибут class используется для идентификации элементов. Но в отличие от id, атрибут class не обязательно должен быть уникальным в документе. Это означает, что один и тот же класс можно применить к нескольким элементам в документе, как показано в следующем примере:

```
<input type="text" class="highlight">
<div class="box highlight">Некоторый контент</div>
<p class="highlight">Это параграф.</p>
```

Поскольку class может применяться к нескольким элементам, то любые правила стиля, написанные для него, будут применяться ко всем элементам, имеющим этот же class.

Атрибут заголовка

Атрибут title используется для предоставления **справочного текста** об элементе или его содержимом. Рассмотрим следующий пример, чтобы понять, как он работает.

```
<abbr title="Консорциум World Wide Web">W3C</abbr>
<a href="images/kites.jpg" title="Нажмите, чтобы посмотреть увеличенное изображение">
```

```
    <img src="images/kites-thumb.jpg" alt="воздушные змеи">
</a>
```

Значение атрибута title (т.е. текст заголовка) отображается в виде всплывающей подсказки веб-браузерами, когда пользователь наводит курсор мыши на элемент.

Атрибут стиля

Атрибут style позволяет указать **правила стиля CSS**, такие как цвет, шрифт, граница и т.д., непосредственно внутри элемента. Давайте рассмотрим пример, чтобы увидеть, как это работает:

```
<p style="color: blue;">Это параграф.</p>
<img src="images/sky.jpg" style="width: 300px;" alt="Cloudy Sky">
<div style="border: 1px solid red;">Некоторый контент</div>
```

Булевы атрибуты

Иногда встречаются атрибуты, написанные без значения — и это совершенно допустимо. Такие атрибуты называются **булевыми**, они могут иметь только одно значение, которое совпадает с его именем.

В качестве примера возьмём атрибут disabled, который можно назначить для формирования элементов ввода. Он используется если понадобиться отключить элемент (сделать неактивным), чтобы пользователь не мог вводить какие-либо данные.

```
<input type="text" disabled="disabled">
```

Для краткости совершенно допустимо записывать их следующим образом (мы также для справки разместили не деактивированный элемент input, чтобы дать вам большее понимание происходящего):

```
<input type="text" disabled>
<input type="text">
```

Опускание кавычек вокруг значений атрибутов

В различных вариантах кода HTML будут попадаться незнакомые способы написания разметки, включая написание значений атрибутов без кавычек. Это допустимо при определённых условиях, но такого разрушающего фактора нужно избегать.

Например, возвращаясь, к примеру с гиперссылкой, мы можем написать основной вариант только с атрибутом href так:

```
<a href=https://www.yandex.ru/>любимый веб-сайт</a>
```

Однако, как только мы добавим атрибут title в таком же стиле, мы поступим неверно:

```
<a href=https://www.yandex.ru/ title=Домашняя страница Яндекс>любимый веб-сайт</a>
```

В этом месте браузер неверно истолкует разметку, думая, что атрибут title — это на самом деле три разных атрибута — атрибут title со значением "Домашняя" и два булевых атрибута: страница и Яндекс.

Это, очевидно, не то, что имелось в виду, и приведёт к ошибке или неожиданному поведению кода, как это показано в примере ниже. Попробуйте навести курсор на ссылку, чтобы увидеть, на что похож текст title!

Всегда используйте кавычки в атрибутах — это позволит избежать подобных проблем, и, следовательно, код будет более читабельным.

Одинарные или двойные кавычки?

Мы все атрибуты заключаем в двойные кавычки. Однако, в коде можно иногда встретить одинарные кавычки. Это исключительно дело вкуса, и мы можем свободно выбирать, какие из них использовать.

Обе следующие строки эквивалентны:

```
<a href="http://www.example.com">Ссылка к моему примеру.</a>
<a href='http://www.example.com'>Ссылка к моему примеру.</a>
```

Однако мы должны убедиться, что не смешиваем их вместе. Следующее будет неверным!

```
<a href="http://www.example.com'>Ссылка к моему примеру.</a>
```

Если мы используем один тип кавычек в своём HTML, то можем поместить внутрь их кавычки другого типа, не вызывая никаких проблем:

```
<a href="http://www.example.com" title="Разве это не самый 'крутой' дизайн?">Ссылка на мой пример.</a>
```

Если мы хотим вставить кавычки того же типа, то должны использовать объекты HTML. Например, это работать не будет:

```
<a href='http://www.example.com' title='Разве это не самый 'крутой' дизайн?'>Ссылка на мой пример.</a>
```

Поэтому вам нужно сделать так:

```
<a href='http://www.example.com' title='Разве это не самый &#39крутой&#39 дизайн?'>Ссылка на мой пример.</a>
```

Организация контента с помощью заголовков

Заголовки помогают определить иерархию и структуру содержимого веб-страницы.

HTML предлагает шесть уровней тегов заголовков, от <h1> до <h6>; чем ниже номер уровня заголовка, тем выше его важность. Поэтому тег <h1> определяет самый важный заголовок, тогда как тег <h6> определяет наименее важный заголовок в документе.

По умолчанию браузеры отображают заголовки более крупным и жирным шрифтом, чем обычный текст. Кроме того, заголовки <h1> отображаются самым крупным шрифтом, а заголовки <h6> отображаются самым мелким шрифтом.

```
<h1>Heading level 1</h1>
<h2>Heading level 2</h2>
<h3>Heading level 3</h3>
<h4>Heading level 4</h4>
<h5>Heading level 5</h5>
<h6>Heading level 6</h6>
```

Вывод приведенного выше примера будет выглядеть примерно так:

Heading level 1

Heading level 2

Heading level 3

Heading level 4

Heading level 5

Heading level 6

Каждый раз, когда тег заголовка размещается на веб-странице, встроенные таблицы стилей веб-браузера автоматически создают некоторое пустое пространство (называемое полем) до и после каждого заголовка. Мы можем использовать свойство CSS margin, чтобы переопределить таблицу стилей браузера по умолчанию.

С помощью CSS очень легко настроить внешний вид тегов заголовков HTML, например размер шрифта, жирность, шрифт и т. д.

Важность заголовков

- Заголовки HTML предоставляют ценную информацию, выделяя важные темы и структуру документа, поэтому тщательно оптимизируйте их, чтобы улучшить взаимодействие с пользователем.
- Не используйте заголовки, чтобы текст выглядел БОЛЬШИМ или жирным. Используйте их только для выделения заголовка документа и отображения структуры документа.
- Поскольку поисковые системы, такие как Google, используют заголовки для индексации структуры и содержания веб-страниц, используйте их очень разумно на своей веб-странице.
- Используйте заголовки <h1> в качестве основных заголовков вашей веб-страницы, затем следуют заголовки <h2>, затем менее важные заголовки <h3> и так далее.

Используйте тег <h1>, чтобы отметить самый важный заголовок, который обычно находится вверху страницы. HTML-документ обычно должен иметь только один заголовок <h1>, за которым следуют заголовки более низкого уровня, такие как <h2>, <h3>, <h4>, и так далее.

Создание абзацев

Элемент абзаца используется для публикации текста на веб-страницах. Абзацы определяются тегом <p>. Тег абзаца является очень простым и, как правило, первым тегом, который нам понадобится для публикации текста. Вот пример:

```
<p>Это параграф.</p>
<p>Это другой параграф.</p>
```

Встроенные таблицы стилей браузеров автоматически создают некоторое пространство над и под содержимым абзаца (называемое **отступом**), но его можно переопределить с помощью правил CSS.

Закрытие элемента абзаца

В HTML4 и более ранних версиях достаточно было инициировать новый абзац с помощью открывающего тега. Большинство браузеров будут правильно отображать HTML, даже если будет пропущен конечный тег. Например:

```
<p>Это параграф.
<p>Это другой параграф.
```

Код HTML в приведенном выше примере будет работать в большинстве веб-браузеров, но не нужно особо полагаться на это. Пропуск конечного тега может привести к неожиданным результатам или ошибкам.

В целях совместимости с последующими версиями и хорошей практики кодирования рекомендуется использовать как открывающий, так и закрывающий теги для абзацев.

Создание разрывов строк

Тег
 используется для вставки разрыва строки на веб-странице. Поскольку
 - это пустой элемент, соответствующий тег </br> не нужен.

```
<p>Это абзац <br> с разрывом строки.</p>
<p>Это <br>еще один абзац <br> с разрывами строк.</p>
```

Нет необходимости в использовании пустого абзаца, например , для добавления дополнительного места на веб-страницы. Браузер может игнорировать пустые абзацы, поскольку это логический тег. Вместо этого нужно использовать свойство CSS margin, чтобы освободить пространство вокруг элементов.

Создание горизонтальных линий

Для создания горизонтальных разделителей (линий) можно использовать тег <hr>, он подходит для визуального разделения разделов контента на веб-странице. Как и <br>, тег <hr> также является пустым элементом. Вот пример:

```
<p>Это параграф.</p>
<hr>
<p>Это другой параграф.</p>
```

Управление пробелами

Обычно браузер отображает несколько пробелов, созданных внутри кода HTML, нажатием клавиши пробела или клавиши табуляции на клавиатуре, в виде одного пробела. Множественные разрывы строк, созданные внутри HTML-кода при нажатии клавиши ввода, также отображаются как один пробел.

Вставка   для создания дополнительных последовательных пробелов и вставка тега
 для создания разрывов строк на веб-страницах, как показано в следующем примере:

```
<p>В исходном коде этот абзац содержит несколько   пробелов.</p>
<p>Этот параграф содержит несколько<br><br>line<br><br><br>разрывов строк.</p>
```

Определение предварительно отформатированного текста

Иногда использование ,
и т.д. для управления пробелами не очень удобно. Для этих целей можно использовать тег <pre>. Он служит для отображения пробелов, табуляции, разрывов строк и т.д. Это очень полезно при представлении текста, где важны пробелы и разрывы строк, например стихотворение или ступенчатый код (например, код Python).

В следующем примере текст будет отображаться в браузере так же, как и в исходном коде:

```
<pre>
    Мерцай, мерцай, звездочка,
    Как же я потрясен тем, кто ты!
    Так высоко над миром,
    Как алмаз на небе.
</pre>
```

Текст внутри элемента <pre> обычно отображается браузерами моноширинным шрифтом или шрифтом фиксированной ширины, например Courier, но его можно переопределить это с помощью свойства CSS font.

4.2. Организация HTML страницы

Понимание типа документа HTML5

Ранее мы рассматривали объявление *типа документа*, или сокращенно DOCTYPE. Оно представляет собой инструкцию для веб-браузера о версии языка разметки, на котором написана веб-страница. Объявление DOCTYPE появляется в верхней части веб-страницы перед всеми остальными элементами. В соответствии со спецификацией или стандартами HTML каждый HTML-документ требует действительного объявления типа документа, чтобы гарантировать, что ваши веб-страницы отображаются так, как они должны отображаться.

Объявление типа документа обычно является самым первым, что определяется в документе HTML (даже до открывающего тега `<html>`); однако само объявление типа документа не является тегом HTML. DOCTYPE для HTML5 очень короткий, лаконичный и нечувствительный к регистру.

```
<!DOCTYPE HTML>
```

Типы документов для более ранних версий HTML были длиннее, потому что язык HTML был основан на SGML и, следовательно, требовал ссылки на DTD, но сейчас они устарели. В HTML5 объявление *doctype* требуется только для включения стандартного режима для документов, написанных с использованием синтаксиса HTML.

Мы можем использовать следующую разметку в качестве шаблона для создания нового документа HTML5.

```
<!DOCTYPE html> <html lang="ru">
<head>
  <meta charset="utf-8">
  <title>
    <!-- Вставить заголовок здесь -->
  </title>
</head>
<body>
  <!-- Вставить контекст сюда -->
</body>
</html>
```

*Объявление doctype относится к определению **типа документа (DTD)**. Это инструкция для веб-браузера о том, на какой версии языка разметки написана страница. Консорциум **World Wide Web (W3C)** предоставляет DTD для всех версий HTML.*

Необходимо добавить объявление типа документа при создании HTML-документа. Кроме того, используется валидатор W3C, чтобы проверить HTML код на наличие ошибок разметки или синтаксиса перед публикацией в Интернете.

Структура HTML страницы

Ниже дан пример оборачивания основных, самостоятельных HTML-элементов, которые сами по себе не очень полезны. Давайте посмотрим, как самостоятельные элементы объединяются для формирования всей HTML страницы:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Тестовая страница</title>
  </head>
  <body>
    <p>Это — моя страница</p>
  </body>
</html>
```

Вот что мы имеем:

1. `<!DOCTYPE html>` - объявление типа документа. Очень давно, ещё когда HTML только развивался (1991), типы документов использовались в качестве ссылок на набор правил, которым HTML-страница должна была следовать. Объявление типа документа выглядело примерно вот так:

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"

"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

Однако в наши дни никто особо не думает о этом и типы документа стали историческим артефактом. *<!DOCTYPE html>* — это самый короткий вид типа документа, который считается действующим. На самом деле это всё, что нужно вам знать о типах документов в HTML5.

2. `<html></html>` - элемент `<html>` содержит в себе всё содержимое на всей странице, и иногда его называют "**корневой элемент**".
3. `<head></head>` - элемент `<head>`. Данный элемент выступает в качестве контейнера для всего содержимого, которое нужно включить в HTML документ, но не отображать посетителям страницы. Он включает ключевые слова и описание страницы (для поисковых запросов), CSS для стилизации контента, объявление поддерживаемого набора символов и др.
4. `<meta charset="utf-8">` - этот элемент устанавливает в качестве символьной кодировки для документа utf-8 , который включает большинство символов из всех известных человечеству языков. По существу, теперь страница сможет отобразить любой текстовый контент. Нет причин не устанавливать эту кодировку, так как это также позволит избежать многих проблем.
5. `<title></title>` - элемент `<title>`. Этот элемент устанавливает заголовок страницы, который появляется во вкладке браузера, загружающей эту страницу. Также заголовок используется при описании страницы, когда она сохраняется в закладках или избранном.
6. `<body></body>` - элемент `<body>`. Он содержит весь контент, который нужно показывать посетителям страницы, — текст, изображения, видео, игры, проигрываемые аудио дорожки или что-то ещё.

Рис. 4.5. Структура HTML-страницы

Создание макетов веб-сайтов

Создание **макета веб-сайта** — это деятельность по позиционированию различных элементов, которые делают веб-страницу хорошо структурированной и придают веб-сайту привлекательный вид.

Большинство веб-сайтов в Интернете обычно отображают свой контент в несколько строк и столбцов, отформатированных как журнал или газета, чтобы предоставить пользователям лучшую среду для чтения. Этого можно легко добиться, используя теги HTML: `<table>`, `<div>`, `<header>`, `<footer>`, `<section>` и т.д., и добавив к ним стили CSS.

Макет HTML-таблицы

Table предоставляет самый простой способ создания макетов в HTML. Как правило, он включает в себя процесс размещения содержимого, такого как текст, изображения и т.д., в строки и столбцы.

Следующий макет создан с использованием таблицы HTML с 3 строками и 2 столбцами — первая и последняя строки охватывают оба столбца с помощью атрибута `colspan` таблицы:

Метод, используемый для создания макета в приведенном выше примере, не является неправильным, но не рекомендуется. Желательно избегать таблиц и встроенных стилей для создания макетов. Макеты, созданные с использованием таблиц, часто отображаются очень медленно. Таблицы следует использовать только для отображения табличных данных.

Макет на основе HTML Div

Использование элементов `<div>` — наиболее распространенный метод создания макетов в HTML. Элемент `<div>` (обозначает разделение) используется для выделения блока контента или набора других элементов внутри HTML-документа. При необходимости он может содержать другие элементы div.

В следующем примере элементы `div` используются для создания макета с несколькими столбцами. Это даст тот же результат, что и в предыдущем примере, где использовался элемент таблицы.

Мы создали этот макет, используя *методы CSS float*, так как большинство веб-браузеров поддерживают его. Кроме того, можно также использовать *flexbox CSS3* для создания современных и более гибких макетов. Подробнее об этом можно узнать на курсе, посвященном изучению CSS3.

Улучшенные макеты веб-страниц можно создать с помощью элементов DIV и CSS. Таким образом, появляется возможность изменить макет всех страниц сайта, отредактировав всего один файл CSS.

Использование структурных элементов HTML5

HTML5 представил новые структурные элементы, такие как *<header>*, *<footer>*, *<nav>*, *<section>* и т.д., для семантического определения различных частей веб-страницы. Эти элементы рассматриваются как замена обычно используемым классам, таким как *.header*, *.footer*, *.nav*, *.section* и т.д.

В следующем примере используются новые структурные элементы HTML5 для создания того же макета, который мы создавали в предыдущих разделах. В следующей таблице представлен краткий обзор новых структурных элементов HTML5.

Таблица 4.2. Новые структурные элементы HTML5

Ярлык	Описание
<header>	Представляет заголовок документа или раздела.
<footer>	Представляет нижний колонтитул документа или раздела.
<nav>	Представляет собой раздел навигационных ссылок.
<section>	Представляет раздел документа, например заголовок, нижний колонтитул и т. д.
<article>	Представляет статью, сообщение в блоге или другую автономную единицу информации.
<aside>	Представляет некоторое содержимое, слабо связанное с содержимым страницы.

Пожалуйста, ознакомьтесь со справкой по тегам HTML5, чтобы узнать о недавно введенных тегах.

Элемент заголовка HTML

Элемент <head> в первую очередь является контейнером для всех элементов заголовка, которые предоставляют дополнительную информацию о документе (метаданные) или ссылки на другие ресурсы, необходимые для правильного отображения или поведения документа в веб-браузере.

Элементы head совместно описывают свойства документа, такие как заголовок, предоставляют метаинформацию, такую как набор символов, указывают браузеру, где

найти таблицы стилей или сценарии, которые позволяют расширять HTML-документ очень активными и интерактивными способами.

Элементы HTML, которые можно использовать внутри `<head>` элемента: `<title>`, `<base>`, `<link>`, `<style>`, `<meta>`, `<script>` и элемент `<noscript>`.

Элемент `<title>` определяет заголовок документа. Элемент необходим во всех документах `HTML/XHTML` для создания действительного документа. В документе допускается только один элемент заголовка, и он должен быть помещен внутри элемента `<head>`. Элемент `title` содержит простой текст и объекты - он не может содержать другие теги разметки.

Название документа может использоваться для разных целей. Например:

- Чтобы отобразить заголовок в строке заголовка браузера и на панели задач.
- Для предоставления заголовка страницы, когда она добавляется в избранное или добавляется в закладки.
- Отображать заголовок страницы в результатах поиска.

В следующем примере показано, как разместить заголовок в HTML-документе.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <title>Это простой HTML документ</title>
</head>
<body>
   <p>Hello World!</p>
</body>
</html>
```

Хороший заголовок должен быть кратким и соответствовать содержанию документа, поскольку поисковые роботы обращают особое внимание на слова, используемые в заголовке. Длина заголовка в идеале не должна превышать 65 символов.

Базовый элемент HTML

Элемент HTML `<base>` используется для определения базового URL-адреса для всех относительных ссылок, содержащихся в документе. Например, мы можем установить базовый URL-адрес один раз в верхней части страницы, а затем все последующие относительные ссылки будут использовать этот URL-адрес в качестве отправной точки:

```
<!DOCTYPE html>
<html lang="ru">
<head>
```

```
    <title>Определение базового URL</title>
    <base href="https://www.digtlab.ru/">
</head>
<body>
    <p><a href="html-tutorial/html-head.php">Заголокок HTML</a>.</p>
</body>
</html>
```

Гиперссылка в приведенном выше примере фактически соответствует `https://www.digtlab.ru/html-tutorial/html-head.php` независимо от URL-адреса текущей страницы. Это связано с тем, что относительный URL-адрес, указанный в ссылке `html-tutorial/html-head.php`, добавляется в конец базового URL-адреса: `https://www.digtlab.ru/`.

HTML- элемент `<base>` должен стоять перед любым элементом, ссылающимся на внешний ресурс. HTML допускает только один базовый элемент для каждого документа.

HTML-элемент ссылки

Элемент `<link>` определяет связь между текущим документом и внешними документами или ресурсами. Обычно элемент link используется для ссылки на внешние таблицы стилей.

```
<head>
    <title>Linking Style Sheets</title>
    <link rel="stylesheet" href="style.css">
</head>
```

Элемент HTML-документа `<head>` может содержать любое количество элементов `<link>`. У элемента `<link>` есть атрибуты, но нет содержимого.

Элемент стиля HTML

Элемент `<style>` используется для определения *встроенной информации о стиле* для HTML-документа. Правила стиля внутри элемента `<style>` определяют, как HTML-элементы отображаются в браузере.

```
<head>
    <title>Встраивание таблиц стилей</title>
    <style>
        body { background-color: YellowGreen; }
        h1 { color: red; }
```

```
        p { color: green; }
    </style>
</head>
```

Встроенную таблицу стилей следует использовать, если отдельный документ имеет уникальный стиль. Если одна и та же таблица стилей используется в нескольких документах, наиболее подходящей будет внешняя таблица стилей.

Мета-элемент HTML

Элемент `<meta>` предоставляет метаданные о HTML-документе. **Метаданные** — это набор данных, который описывает и предоставляет информацию о других данных. Вот пример:

```
<head>
    <title>Указание метаданных</title>
    <meta charset="utf-8">
    <meta name="author" content="Сюн Цзы">
</head>
```

Элемент HTML-скрипта

Элемент `<script>` используется для определения скрипта на стороне клиента, такого как `JavaScript` в документах HTML. В следующем примере в браузере отобразится приветственное сообщение:

```
<head>
    <title>Добавление JavaScript</title>
    <script>
        document.write("<h1>Hello World!</h1>")
    </script>
</head>
```

Стилизация HTML-элементов

Разметка HTML очень ограничена по возможностям, когда дело доходит до представления веб-страницы. Дело в том, что HTML был разработан как способ представления информации, но без акцента на красоту представления или дизайн.

CSS (каскадные таблицы стилей) были представлены в декабре 1996 года *Консорциумом World Wide Web (W3C)* для обеспечения лучшего способа стилизации HTML-элементов. С помощью CSS очень легко задать такие параметры, как размер и шрифт для текста, цвета для текста и фона, сделать выравнивание текста и изображений, установить расстояние между элементами, границы и контуры для элементов и многое другое.

Добавление стилей к элементам HTML

Информация о стиле может быть прикреплена как отдельный документ (содержащий правила CSS) или встроена в сам HTML-документ (то есть к коду HTML подмешиваются блоки с правилами CSS). Выделяют три способа подключения стилей CSS в HTML-документ:

- **Встроенные стили** — использование атрибута `style` в открывающем теге HTML.
- **Встроенный стиль** — использование элемента `<style>` в разделе заголовка документа.
- **Внешняя таблица стилей** — использование элемента `<link>`, указывающего на внешние файлы CSS.

Встроенные стили имеют самый высокий приоритет, а внешние таблицы стилей — самый низкий. Это означает, что, если указать стили для абзацев как во встроенных, так и во внешних таблицах - конфликтующие правила стилей во встроенной таблице стилей переопределят внешнюю таблицу стилей.

Встроенные стили

Встроенные стили используются для применения уникальных правил стиля к элементу путем их помещения в начальный тег. Их можно прикрепить к элементу с помощью атрибута `style`.

Атрибут стиля включает пары свойств и значений CSS. Каждая пара `property: value` отделяется точкой с запятой (`;`), как если бы мы писали во встроенную или внешнюю таблицу стилей. Но это должно быть написано в одной строке (то есть без разрыва строки после точки с запятой). В следующем примере показано, как установить `color` и `font-size` для текста:

```
<h1 style="color:red; font-size:30px;">Это заголовок</h1>
<p style="color:green; font-size:18px;">Это параграф.</p>
<div style="color:green; font-size:18px;">Это некоторый текст.</div>
```

Использование встроенных стилей обычно считается плохой практикой. Поскольку правила стиля встроены непосредственно в тег html, то представление смешивается с содержимым документа, что очень затрудняет обновление или поддержку веб-сайта.

В новых версиях стало невозможно стилизовать псевдоэлементы и псевдоклассы с помощью встроенных стилей. Поэтому следует избегать использования атрибутов `style` в разметке. Использование внешней таблицы стилей является предпочтительным способом добавления информации о стиле в HTML-документ.

Встроенные таблицы стилей

Встроенные или внутренние таблицы стилей влияют только на документ, в который они встроены. Встроенные таблицы стилей определяются в разделе *<head>* HTML-документа с помощью тега *<style>*. Мы можем определить любое количество элементов *<style>* внутри раздела *<head>*. В следующем примере показано, как правила стиля внедряются в веб-страницу.

```
<head>
  <style>
    body { background-color: YellowGreen; }
    h1 { color: blue; }
    p { color: red; }
  </style>
</head>
```

Внешние таблицы стилей

Внешняя таблица стилей просто необходима, когда стиль применяется ко многим страницам. Внешняя таблица стилей содержит все правила стиля в отдельном документе, на который можно ссылаться из любого HTML-документа на сайте. Внешние таблицы стилей являются более гибкими, потому что с помощью внешней таблицы стилей мы можем изменить внешний вид всего веб-сайта, обновив всего один файл.

Прикрепить внешние таблицы стилей допускается двумя способами — *связыванием* и *импортом.*

Связывание внешних таблиц стилей

Внешняя таблица стилей может быть связана с документом HTML с помощью тега *<link>*. Тег *<link>* находится внутри раздела *<head>*, как показано здесь:

```
<head> <link rel="stylesheet" href="css/style.css"> </head>
```

Импорт внешних таблиц стилей

Правило *@import* — это еще один способ загрузки внешней таблицы стилей. Оператор *@import* указывает браузеру загрузить внешнюю таблицу стилей и использовать ее стили. Его можно использовать двумя способами. Самый простой способ — вызвать оператор внутри элемента *<style>* в вашем разделе *<head>*. Обратите внимание, что в элемент *<style>* могут быть включены также и другие правила CSS.

```
<style> @import url("css/style.css"); p { color: blue; font-size: 16px; } </style>
```

Точно так же можно использовать правило `@import` для импорта таблицы стилей в другую таблицу стилей.

```
@import url("css/layout.css");
@import url("css/color.css");
body { color: blue; font-size: 14px; }
```

Все правила `@import` должны располагаться в начале таблицы стилей. Любое правило стиля, определенное в самой таблице стилей, переопределяет конфликтующее правило в импортированных таблицах стилей.

Правило `@import` может привести к проблемам с производительностью, так что следует избегать импорта таблиц стилей.

4.3. Метаданные в HTML

*Теги `<meta>` обычно используются для предоставления **структурированных метаданных**, таких как ключевые слова документа,* описание*,* имя автора*, кодировка символов и т.д. Любое количество метатегов может быть помещено в **раздел заголовка документа** HTML или XHTML.*

Метаданные не будут отображаться на веб-странице, но будут обрабатываться машиной и используются браузерами, поисковыми системами (например, Yandex, Google), или другими веб-службами.

Объявление кодировки символов в HTML

Метатег очень часто используется для объявления кодировки символов внутри HTML-документа.

```
<!DOCTYPE html>
<html lang="ru">
<head>
   <title>Обявление кодировки символов</title>
   <meta charset="utf-8">
</head>
<body>
   <h1>Привет мир!</h1>
</body>
</html>
```

Для установки кодировки символов внутри документа CSS используется правило @charset.

UTF-8 — очень универсальная и рекомендуемая кодировка символов. Если кодировка не указана, то используется кодировка платформы по умолчанию.

Добавление автора документа

Метатег применяется, чтобы четко определить, кто является автором или создателем веб-страницы. Автором может быть физическое лицо, компания в целом или третье лицо.

```
<head>
  <title>Определение автора документа</title>
  <meta name="author" content="Сергей Чесноков">
</head>
```

Атрибут метатега `name` определяет имя фрагмента метаданных уровня документа, а атрибут `content` дает соответствующее значение. Значение атрибута `content` содержит текст и сущности, но не должно включать теги HTML.

Ключевые слова и описание для поисковых систем

Некоторые поисковые системы используют метаданные, особенно ключевые слова и описания, для индексации веб-страниц. Ключевые слова и описание придают дополнительный вес документу и создают краткий обзор страницы. Вот пример:

```
<head>
  <title>Определение ключевых слов и описания</title>
  <meta name="keywords" content="HTML, CSS, javaScript">
  <meta name="description" content="Простые для понимания руководства и справочники по HTML, CSS, javaScript и т.д.">
</head>
```

Поисковые системы часто используют метаописание страницы для создания краткого описания, когда страница появляется в результатах поиска.

Настройка области просмотра для мобильных устройств

При разработке для мобильных устройств обязательно потребуется метатег viewport - для корректного отображения веб-страниц на мобильных устройствах. Без метатега области просмотра мобильные браузеры отображают веб-страницы с типовой шириной экрана рабочего стола, а затем уменьшают ее, чтобы она соответствовала экрану мобильного устройства. В результате для правильного просмотра веб-страницы на мобильных устройствах пользователю требуется выполнять масштабирование, что очень неудобно.

Следующая демонстрация показывает две веб-страницы — одну *с* **метатегом области просмотра** и другую **без набора метатегов области просмотра**.

С метатегом видового экрана | Без метатега видового экрана

Рис. 4.6. Иллюстрация применения метатега

Метатег области просмотра устанавливает наилучший размер и пределы масштабирования для просмотра веб-страниц на мобильных устройствах. Типичное определение **метатега области просмотра** будет выглядеть следующим образом:

```
<head>
  <title>Настройка Viewport</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">
</head>
```

Пара ключ-значение `width=device-width` внутри атрибута `content` устанавливает ширину области просмотра равной ширине экрана устройства, тогда как `initial-scale=1` устанавливает начальный масштаб или уровень масштабирования на 100% при первой загрузке страницы браузером.

Всегда нужно использовать тег области просмотра `<meta>` на веб-страницах. Это сделает сайт удобным для пользователя и более доступным на мобильных устройствах.

Назначение <head>

Давайте снова посмотрим на минимальный HTML-документ:

```
<!DOCTYPE html>
<html>
```

```
  <head>
    <meta charset="utf-8">
    <title>Моя тестовая страница</title>
  </head>
  <body>
    <p>Это — моя страница</p>
  </body>
</html>
```

Содержимое `<head>`, в отличие от содержимого элемента `<body>`, не отображается на странице. Основная задача `<head>` — хранить **метаданные документа**. В приведённом выше примере `<head>` совсем небольшой:

```
<head>
  <meta charset="utf-8">
  <title>Моя тестовая страница</title>
</head>
```

Однако на больших страницах блок `<head>` может быть довольно объёмным. Попробуйте зайти на какие-нибудь из сайтов и посмотреть содержимое `<head>` с помощью **инструментов разработчика**.

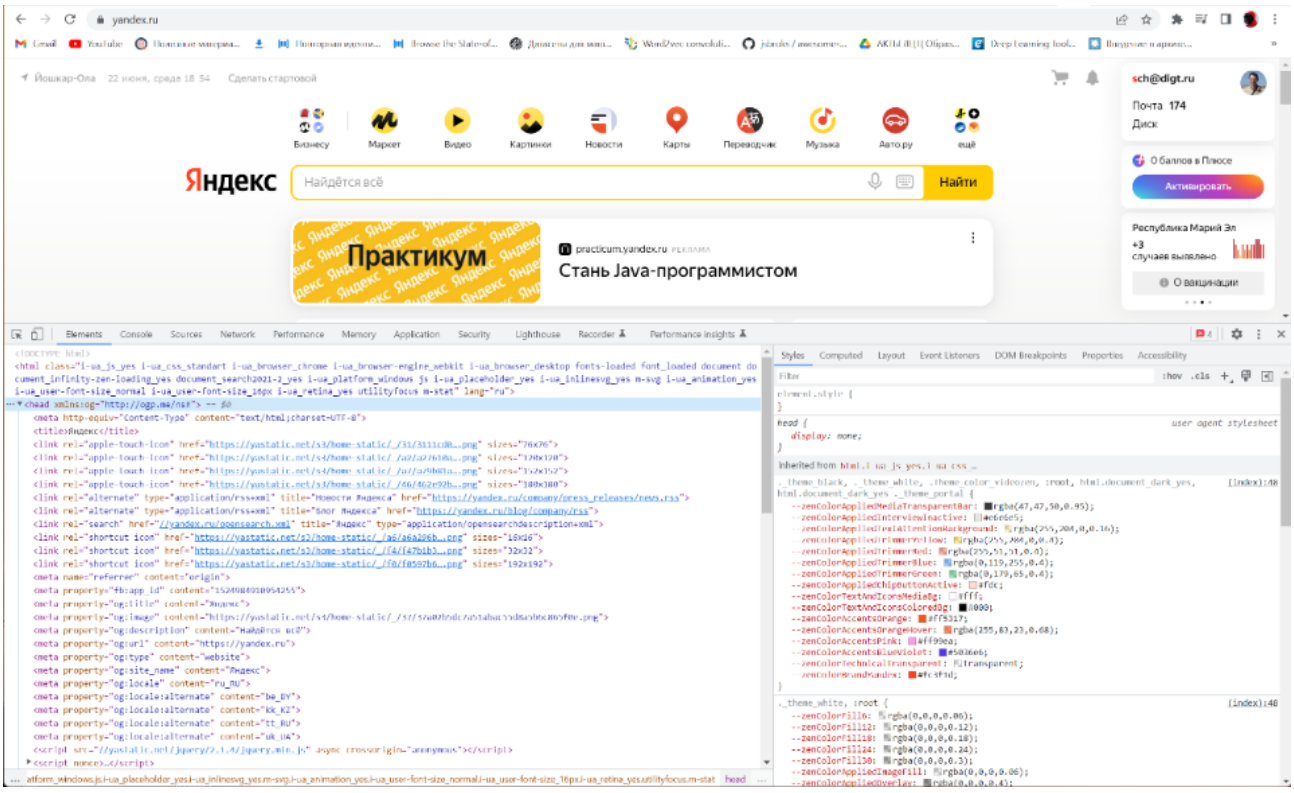


Рис. 4.7. Просмотр блока head

Название страницы (title)

Мы уже видели, как работает элемент `<title>`: его используют для добавления **заголовка (названия страницы)** в документ. Элемент `<h1>` тоже иногда называют **заголовком страницы**. Но это разные вещи!

- Элемент `<h1>` виден на странице, открытой в браузере, — его используют один раз на странице, чтобы выделить название содержимого. Это может быть название истории, заголовок новости или что-то в этом роде.
- Элемент `<title>` — метаданные, **название всего HTML-документа**, а не заголовок внутри его содержимого.

Другие виды метаданных

В сети мы можем встретить другие типы метаданных, которые еще не рассматривали. Многие из них — это собственные форматы, созданные для предоставления определённым сайтам (например, социальным сетям) специальной информации, которую они могут использовать.

Добавление иконок

Чтобы добавить узнаваемости сайту, можно добавить в метаданные разные иконки. **Favicon** - один из старожилов интернета, стал первой из таких иконок. Браузеры показывают её в заголовке вкладки и в списке избранных страниц.

Чтобы добавить на страницу `favicon`:

1. Сохраните изображение в формате `.ico` (многие браузеры поддерживают и в более привычных форматах, таких как `.gif` или `.png`) в папку со своим документом. Старые браузеры, поддерживают только формат `.ico`. Сконвертировать изображение можно с помощью online-сервисов.
2. Добавьте ссылку на иконку в `<head>` документа:

```
<link rel="shortcut icon" href="favicon.ico" type="image/x-icon">
```

Для разных устройств можно указывать разные иконки. Например:

```
<!-- Для iPad 3 с Retina-экраном высокого разрешения: -->
<link rel="apple-touch-icon-precomposed" sizes="144x144" href="https://developer.mysite.ru/static/img/favicon144.png">
<!-- Для iPhone с Retina-экраном высокого разрешения: -->
<link rel="apple-touch-icon-precomposed" sizes="114x114" href="https://developer.mysite.ru/static/img/favicon114.png">
<!-- Для iPad первого и второго поколения: -->
```

```
    <link rel="apple-touch-icon-precomposed" sizes="72x72" href="https://developer.my-site.ru/static/img/favicon72.png">
    <!-- Для iPhone, iPod Touch без Retina и устройств с Android 2.1+: -->
    <link rel="apple-touch-icon-precomposed" href="https://developer.my-site.ru/static/img/favicon57.png">
    <!-- Для других случаев - обычный favicon -->
    <link rel="shortcut icon" href="https://developer.mysite.ru/static/img/favicon32.png">
```

В комментариях указано, для чего используется каждая иконка — например, при добавлении страницы на домашний экран iPad будет использована иконка в высоком разрешении.

Подключение CSS и JavaScript

Современные сайты используют CSS, чтобы выглядеть привлекательнее, и добавляют интерактивные функции через JavaScript: видеоплееры, интерактивные карты, эффекты. Обычно связанные стили добавляют на страницу через элемент `<link>`, а скрипты — через элемент `<script>`.

- Элемент `<link>` помещают в заголовок документа. У него есть два атрибута: `rel="stylesheet"` показывает, что мы указываем **стиль документа**, а в `href` указан **путь к файлу**:

```
    <link rel="stylesheet" href="my-css-file.css">
```

- Элемент `<script>` не обязательно находится в заголовке — на самом деле лучше поместить его в самом конце страницы, прямо перед закрывающем тегом `</body>`. Так браузер сначала отобразит саму страницу, а уже затем загрузит и запустит скрипт — иначе скрипт может обратиться к ещё не созданному элементу страницы и сломаться.

```
    <script src="my-js-file.js"></script>
```

Элемент `<script>` кажется пустым, но это не всегда так и указывать закрывающий тег обязательно. Вместо того чтобы ссылаться на внешний скрипт, код можно писать прямо внутри этого элемента — так можно не тратить время на загрузку отдельного скрипта, но тогда не выйдет сослаться на один js-файл с нескольких страниц.

Основной язык HTML страницы

Наконец, стоит отметить, как установить язык по умолчанию для страницы. Делается это добавлением атрибута `lang` в открывающий HTML-тег (как показано ниже):

```
<html lang="en-US">
<html lang="ru">
```

Наибольший эффект от этого связан с тем, что HTML-документ будет более эффективно индексироваться поисковыми системами.

Допускается указывать язык для части документа. Например, установить язык для части страницы на японском:

```
<p>Пример на японском: <span lang="jp">ご飯が熱い。</span>.</p>
```

Коды языков (их абревиатуры) определены в стандарте ISO 639-1.

Работа с клиентским скриптом

Сценарии на стороне клиента относятся к типу компьютерных программ, которые выполняются веб-браузером пользователя. **JavaScript (JS)** — самый популярный клиентский язык сценариев в Интернет.

Элемент `<script>` используется для встраивания или ссылки на JavaScript в HTML-документе, чтобы добавить интерактивности веб-страницам и значительно улучшить взаимодействие с пользователем.

Некоторыми из наиболее распространенных применений JavaScript являются проверка формы, создание предупреждающих сообщений, создание галереи изображений, отображение скрытого содержимого, манипулирование DOM и многое другое.

Добавление JavaScript на HTML-страницы

JavaScript можно либо встроить непосредственно в HTML-страницу, либо поместить во **внешний файл сценария**, на который можно ссылаться внутри HTML-страницы. Оба метода используют элемент `<script>`.

Встраивание JavaScript. Чтобы встроить JavaScript в файл HTML, добавляем его код в качестве содержимого элемента `<script>`. JavaScript в следующем примере записывает строку текста на веб-страницу.

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="utf-8">
    <title>Встраивание JavaScript</title>
</head>
<body>
    <div id="greet"></div>
    <script>
        document.getElementById("greet").innerHTML = "Привет мир!";
```

```
    </script>
</body>
</html>
```

В идеале элементы скрипта следует размещать внизу страницы перед закрывающим тегом `</body>`*. Дело в том, когда браузер встречает скрипт, он приостанавливает отрисовку остальной части страницы до тех пор, пока не проанализирует скрипт. Это значительно влияет на производительность сайта.*

Вызов внешнего файла JavaScript. Код JavaScript также помещают в отдельный файл (с `.js` расширением) и вызывают этот файл в HTML-документе через атрибут `src` тега `<script>`. Делается это для того, чтобы один и тот же сценарий был доступен для нескольких документов. Данный способ избавляет от повторения одной и той же задачи снова и снова и значительно упрощает обслуживание сайта.

В следующем примере показано, как сделать ссылку на внешний JS-файл в HTML.

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="utf-8">
    <title>Ссылка на внешний JavaScript</title>
</head>
<body>
    <div id="greet"></div>
    <script src="hello.js"></script>
</body>
</html>
```

Когда атрибут `src` *указан, элемент* `<script>` *должен быть пустым. Это означает, что нельзя использовать один и тот же элемент* `<script>` *как для встраивания JavaScript, так и для ссылки на внешний файл JavaScript в документе HTML.*

Файлы JavaScript — это обычные текстовые файлы с расширением `.js`*, например «hello.js». Кроме того, внешний файл JavaScript содержит только операторы JavaScript; он не содержит такого элемента* `<script>...</script>`*, как для встроенного JavaScript.*

HTML-элемент noscript

Элемент `<noscript>` используется для предоставления альтернативного содержимого пользователям, которые либо отключили JavaScript в своем браузере, либо имеют браузер, не поддерживающий сценарии на стороне клиента.

Этот элемент может содержать любые HTML-элементы, кроме `<script>`. Эти элементы включаются в элемент `<body>` обычной HTML-страницы. Давайте посмотрим пример:

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="utf-8">
    <title>Демонстрация Noscript</title>
</head>
<body>
    <div id="greet"></div>
    <script>
        document.getElementById("greet").innerHTML = "Привет мир!";
    </script>
    <noscript>
        <p>Ой! Для этого веб-сайта требуется браузер с поддержкой JavaScript.</p>
    </noscript>
</body>
</html>
```

Содержимое внутри элемента `noscript` будет отображаться только в том случае, если браузер пользователя не поддерживает сценарии или в браузере отключены сценарии.

Объект HTML

Некоторые символы зарезервированы в HTML, например, знаки меньше (`<`) или больше (`>`). Поэтому нельзя добавлять угловые скобки в текст, потому что браузер может принять их за разметку. В то же время некоторые нужные нам символы отсутствуют на клавиатуре, например символ авторского права ©.

Чтобы отобразить эти специальные символы, их необходимо заменить **объектами символов (символьные сущности)**. Ссылки на символьные сущности, позволяют использовать символы, которые не могут быть выражены в кодировке символов документа или которые нельзя ввести с клавиатуры.

Таблица 4.3. Часто используемые объекты символов HTML

Результат	Описание	Имя сущности	Цифровая ссылка
	неразрывный пробел		
<	меньше, чем	<	<
>	лучше чем	>	>
&	амперсанд	&	&
"	кавычка	"	"
'	апостроф	'	'
¢	цент	¢	¡
£	фунт	£	¢
¥	иена	¥	¥
€	евро	€	€
©	Авторские права	©	©
®	зарегистрированная торговая марка	®	®
™	товарный знак	™	™

Числовые ссылки на символы используются как альтернатива именам объектов. Ключевым преимуществом использования числовых ссылок на символы является то, что они имеют более надежную поддержку браузера и могут использоваться для указания любого символа Unicode, в то время как объекты ограничены его подмножеством.

Имена объектов HTML чувствительны к регистру!

Неразрывный пробел (` `) используется для создания пробела между двумя элементами, которые не могут быть разделены разрывом строки. Они также используются для отображения нескольких пробелов, поскольку веб-браузеры отображают только один пробел, если несколько пробелов создаются с помощью клавиши пробела.

4.4. Работа с SVG графикой

Векторная графика широко применяется при создании HTML документов. Существует так называемая SVG графика, которая согласно официальной спецификации на w3.org является *языком для описания двумерной графики в XML. SVG включает в себя три типа объектов: фигуры, изображения и текст.* SVG существует с 1999

года, а с 16 августа 2011 включена в рекомендации W3C. SVG сильно недооценена веб-разработчиками, хотя имеет несколько важных преимуществ.

- **Масштабирование**: в отличие от растровой графики, SVG не теряет в качестве при масштабировании, поэтому ее удобно использовать для разработки под retina.
- **Уменьшение HTTP-запросов**: при использовании SVG сокращается количество обращений к серверу, соответственно увеличивается скорость загрузки сайта.
- **Стайлинг** и **скриптинг**: при помощи CSS можно менять параметры графики на сайте, например фон, прозрачность или границы.
- **Анимация и редактирование**: при помощи javascript можно анимировать SVG, а также редактировать в текстовом или графическом редакторе (InkScape или Adobe Illustrator).
- **Малый размер**: объекты SVG весят намного меньше растровых изображений.

SVG создают как с помощью графических редакторов (и сохраняют в формате **.svg**), так и описывают посредством кода. Для этого **существует специальный парный тег <svg>**.

Для создания SVG-изображений **используются редакторы, которые имеют возможность работать с вектором,** например:

- Adobe Illustrator;
- Adobe InDesign;
- CorelDRAW;
- Inkscape и прочее.

После создания изображения желательно его **оптимизировать для быстрой загрузки и правильного внедрения на сайт**. Для этого делают следующее:

- уменьшают количество точек;
- если используется текст, его переводят в кривые;
- удаляют ненужные перекрытия, если используют несколько изображений.

Можно обойтись без графических редакторов и создать SVG с помощью кода. Внедрить SVG на веб-сайт можно различными способами. Рассмотрим самые используемые из них.

Inline (<svg>)

Этот тег подходит для простых фигур и картинок. Изображение рисуется прямо в HTML коде. Можно применять CSS-стили (немного отличаются от стандартных, для применения уточняйте наличие в спецификации) и анимацию. Сам тег характеризуется шириной, высотой и уникальным идентификатором **id**. Плюсы данного метода - не нужно подгружать дополнительные файлы, можно многократно использовать один и тот же элемент, менять нарисованную графику с помощью стилей и анимировать.

Как изображение

Для более сложных фигур и иллюстраций используют тег **<img>**. Для этого нужно сохранить файл в формате **.svg** через специальный графический редактор или конвертер. Пример вставки тега в HTML:

```
<img src="img/circle.svg" alt="Circle in IdBi" height="300" width="250">
```

В папке **img** мы храним изображение круга в формате **.svg**, задаем изображению высоту и ширину, а также вспомогательный атрибут **alt** для тех пользователей, у кого не загружаются изображения в браузере. Также можно загрузить SVG-файл как фоновое изображение. В HTML картинка вставляется как обычный блок:

```
<div class="imageSvg"></div>
```

А потом меняются стили:

```
.imageSvg {
background-image: url("img/circle.svg");
width: 250px;
height: 300px;
}
```

К сожалению, при использовании данного варианта нельзя изменять свойства SVG с помощью стилей и применять интерактивные анимации.

Как объект

Универсальный способ вставки мультимедийного контента в код HTML - тег **<object>**. SVG формат вставляется также, как и любой другой:

```
<object type="image/svg+xml" data="img/circle.svg">Image circle</object>
```

В **type** указывают принадлежность к типу «изображение» и формат **SVG**. Объекты можно менять с помощью стилей CSS, хорошо поддерживаются браузерами.

Анимировать SVG также можно несколькими способами. Первый - специальный тег **<animate>**, который предназначен для работы с <svg>. Он вкладывается внутрь анимируемого объекта, обозначается тип анимации и соответствующие координаты. Второй способ - возможности языка JavaScript. Кроме стандартных приемов существуют специальные библиотеки для работы с эти форматом - **Vivus.js**, **Snap.svg** и прочее. Также анимацию можно создать с помощью стилей CSS.

SVG поддерживается почти всеми современными браузерами за исключением Internet Explorer 8 и ниже. Но и это может быть исправлено использованием javascript-библиотеки Raphael.js. Конвертировать SVG-файл в формат этой библиотеки можно на сайте ReadySetRaphael.com.

4.5. Страница 404 ошибки на сайте

404 Error (ошибка) или **Not Found** открывается, когда пользователь переходит на несуществующую страницу. Причины могут быть разные:

- Он ввел некорректный адрес;
- Страница переехала, удалена, и такой ссылки больше нет.

Важно при разработке сайта не оставлять страницу 404 без внимания!
Во-первых, 404 страница играет большую роль в конверсии сайта. При грамотной разработке, посетитель останется на сайте — перейдет в нужный раздел, или просто подвиснет на интересной и нестандартной 404-ой.
Во-вторых, не каждый пользователь понимает значение этой ошибки и, увидев стандартное сообщение, испугается, решит, что сайт сломался, и покинет его.

Разработка странички 404 может решить ряд проблем:

- Объяснить пользователю, что произошло;
- Перенаправить его на главную или основные разделы сайта;
- Поднять настроение пользователя;
- Показать заботу о нем;
- Произвести впечатление и продемонстрировать, на что вы способны;
- Упростить поиск, например, разместив навигационное меню;
- Повысить продажи, разместив блок популярное или акции в ошибке интернет-магазина;
- Запомниться!

Есть разные способы оформления данной страницы. Можно создать 404-ю в юмористическом стиле, или в корпоративном, сделать ее максимально простой или креативной, а можете создать полезную страничку с навигацией. Рассмотрим несколько примеров.

Рис. 4.6. Пример оформления страницы 404

Рис. 4.7. Пример оформления страницы 404 (продолжение)

Пользователь закрывает сайт, когда попадает на страницу ошибки, поскольку часто просто не понятно, где ты оказался: «А я точно ещё на этом сайте?», «А может это вирус?», «Сайт не работает?».
Важно успокоить и дать понять, что ничего страшного не случилось, и вы по-прежнему находитесь на сайте. Не пишите стандартные фразы «Error», «Not Found», «Ошибка 404». Переведите фразы на человеческий понятный язык:

- Извините, страница не найдена;
- Эта страница удалена, но на нашем сайте еще много интересного;
- Упс! Что-то пошло не так, этой страницы не существует;

- Вы попали на несуществующую страницу;
- Любой ваш вариант, понятный пользователю.

Обязательно предложите решение, что делать дальше: вернуться на главную, в каталог, разместите соответствующие кнопки.

Рис. 4.8. Пример оформления страницы 404 с кнопкой возврата

Задания к практической работе

Задание 1. Создание HTML и размещение текстового контента

Создайте веб-страницу **welcome.html**, на которой будет отображаться контент.

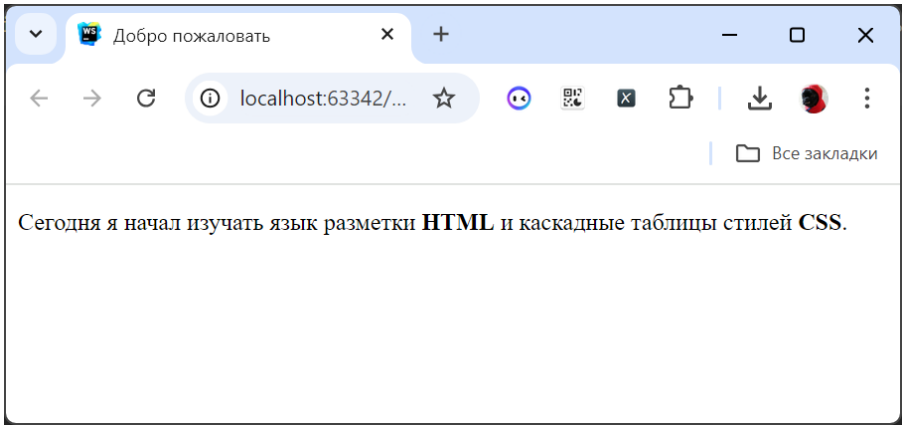


Рис. 4.9. Результат размещения текстового контента

При реализации контента учтите следующие требования:

- Страница должна иметь заголовок **Добро пожаловать**

- Текст должен быть размечен, как **параграф**
- Для оформления полужирного текста, используйте тег ****
- Слово **Сегодня** должно быть помечено как текущая дата с помощью тега **<time>**

Задание 2. Размещение графики

Создайте веб-страницу **signs.html**, на которой будут располагаться изображения дорожных знаков. Разместите эти изображения, используя тег <img> внутри параграфов (в каждом параграфе содержится по пять изображений).

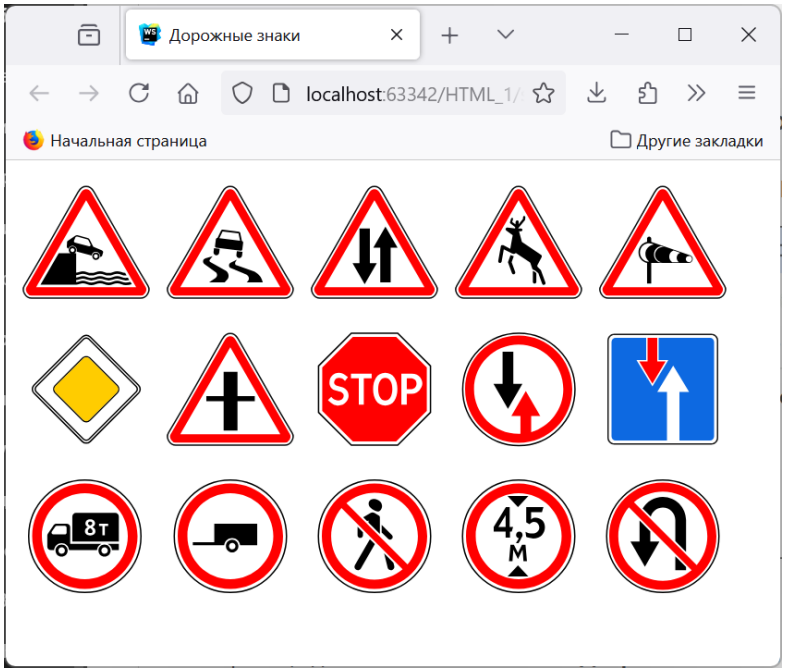


Рис. 4.10. Результат размещения графики

Для этого в проекте создайте каталог **img/signs** и распакуйте в него изображения из архива. Помимо вывода изображений, необходимо настроить к каждому изображению вывод элемента **Tooltip** (подсказки) для того, чтобы идентифицировать каждый дорожный знак. Вывод **tooltip** нужно реализовать с использованием библиотеки **jQuery**. Для этого в коде разметки нужно выполнить следующее:

- Подключить необходимые библиотеки **jQuery**.

```
    <link rel="stylesheet" href="https://code.jquery.com/ui/1.14.1/themes/base/jquery-ui.css">
    <script src="https://code.jquery.com/jquery-3.7.1.js"></script>
    <script src="https://code.jquery.com/ui/1.14.1/jquery-ui.js"></script>
```

- Внедрить в код необходимый набор стилей для подсказок

```
        <style>
            .tooltip {
                position: relative;
```

```
    }

    .tooltip .ui-tooltip {
        position: absolute;
        background-color: #fff;
        border: 1px solid #ddd;
        padding: 5px;
        z-index: 1;
    }
</style>
```

- Внедрить в код разметки код JScript для поиска необходимых элементов и отображения подсказок

```
    <script>
    $(function () {
        $(document).tooltip({
            items: "img",
            content: function () {
                var $element = $(this);
                if ($element.is("img")) {
                    return $element.data('hint');
                }
            }
        });
    });
</script>
```

- Для каждого из изображений нужно добавить дополнительные атрибуты: **data-hint**, содержащий сам текст подсказки и **class**, содержащий значение **tooltip**.

В результате для каждого из изображений по наведению на него указателя мыши, должны появляться всплывающие подсказки, содержащие текст

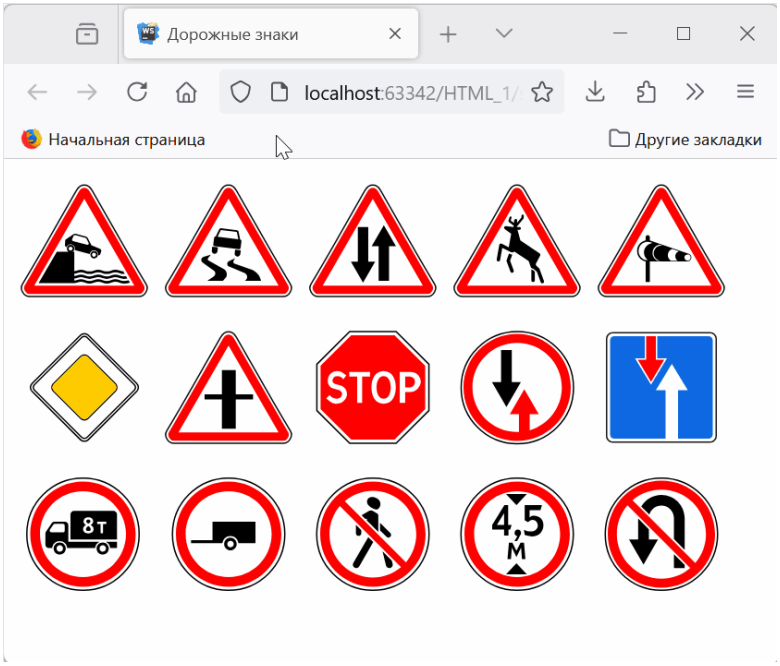


Рис. 4.11. Иллюстрация появления подсказок

Задание 3. Разработка страницы 404 для сайта

Создайте веб-страницу **404_page.html** на которой необходимо разместить всю необходимую информацию, необходимую для пользователей Вашего учебного заведения. Страница обязательно должна содержать фоновую графику. Графика и дизайн страницы выбирается на усмотрение автора страницы.

Задание 4. Создание списка дел ToDo

Списки дел являются основой производительности. Рассмотрим процесс создания одного из них с нуля, используя только HTML, CSS и JavaScript. Этот минималистский подход обеспечивает доступность и простоту, что делает его идеальным для новичков и опытных разработчиков. Чтобы создать простой список дел, нам необходимо выполнить следующие четыре шага:

- Создайте веб-страницу *todo_list.html*, где создается контейнер div, содержащий поле для ввода задачи и кнопку ее добавления. Фрагмент кода контейнера выглядит следующим образом

```
    <div class="container">
    <div id="newtask">
        <input type="text" placeholder="Введите описание задачи">
        <button id="push">Добавить</button>
    </div>
    <div id="tasks"></div>
</div>
```

- Создайте каталог **css** и в нем файл стилей **styles.css**. Добавьте в него следующие стили

```
*,
*:before,
*:after{
    padding: 0;
    margin: 0;
    box-sizing: border-box;
}
body{
    height: 100vh;
    background: #066acd;
}
.container{
    width: 40%;
    top: 50%;
    left: 50%;
    background: white;
    border-radius: 10px;
    min-width: 450px;
    position: absolute;
    min-height: 100px;
    transform: translate(-50%,-50%);
}
#newtask{
    position: relative;
    padding: 30px 20px;
}
#newtask input{
    width: 75%;
    height: 45px;
    padding: 12px;
    color: #111111;
    font-weight: 500;
    position: relative;
    border-radius: 5px;
    font-family: 'Poppins',sans-serif;
    font-size: 15px;
    border: 2px solid #d1d3d4;
}

#newtask input:focus{
    outline: none;
    border-color: #0d75ec;
```

```
}
#newtask button{
    position: relative;
    float: right;
    font-weight: 500;
    font-size: 16px;
    background-color: #0d75ec;
    border: none;
    color: #ffffff;
    cursor: pointer;
    outline: none;
    width: 20%;
    height: 45px;
    border-radius: 5px;
    font-family: 'Poppins',sans-serif;
}
#tasks{
    border-radius: 10px;
    width: 100%;
    position: relative;
    background-color: #ffffff;
    padding: 30px 20px;
    margin-top: 10px;
}

.task{
    border-radius: 5px;
    align-items: center;
    justify-content: space-between;
    border: 1px solid #939697;
    cursor: pointer;
    background-color: #c5e1e6;
    height: 50px;
    margin-bottom: 8px;
    padding: 5px 10px;
    display: flex;
}
.task span{
    font-family: 'Poppins',sans-serif;
    font-size: 15px;
    font-weight: 400;
}
.task button{
    background-color: #6583e5;
    color: #ffffff;
```

```
    border: none;
    cursor: pointer;
    outline: none;
    height: 100%;
    width: 40px;
    border-radius: 5px;
}
```

Стили подключите к **todo_list.html** по относительному пути.

- Создайте каталог **js** и включите в него файл **todo.js** со следующим кодом

```
window.onload = function(){
    document.querySelector('#push').onclick = function(){
        if(document.querySelector('#newtask input').value.length == 0){
            alert("Kindly Enter Task Name!!!!")
        }

        else{
            document.querySelector('#tasks').innerHTML += `
                <div class="task">
                    <span id="taskname">
                        ${document.querySelector('#newtask input').value}
                    </span>
                    <button class="delete">
                        <i class="far fa-trash-alt"></i>
                    </button>
                </div>
            `;

            var current_tasks = document.querySelectorAll(".delete");
            for(var i=0; i<current_tasks.length; i++){
                current_tasks[i].onclick = function(){
                    this.parentNode.remove();
                }
            }
        }
    }
}
```

Не забываем подключить **todo.js** по относительному пути. Тестируем созданный список задач.

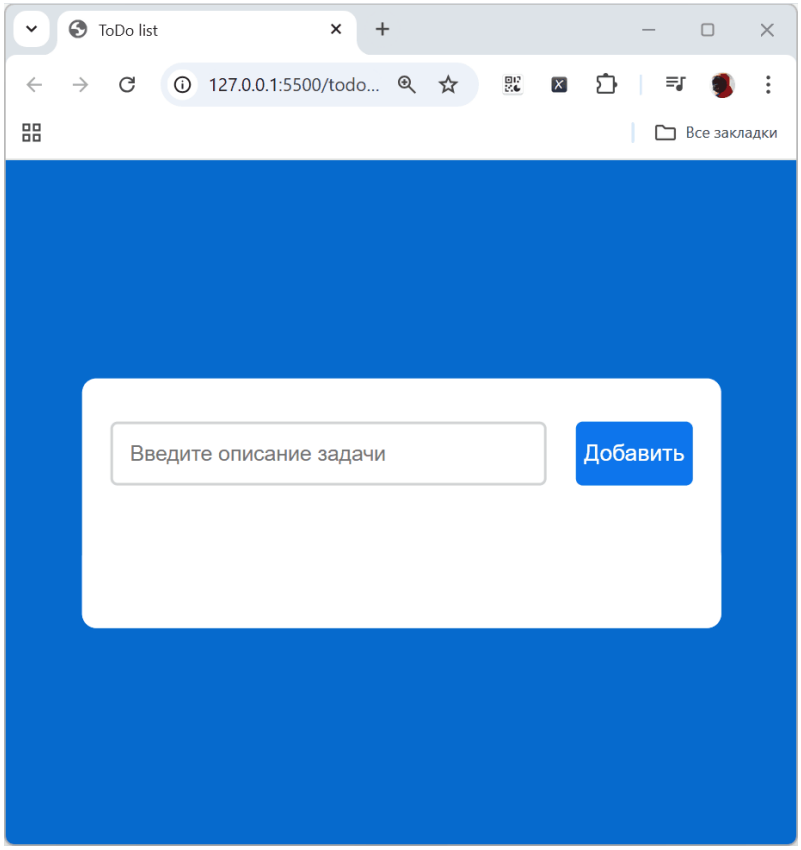


Рис. 4.12. Интерфейс создания списка задач

Это очень простая реализация списка, предлагается улучшить его представление за счет использования элементов графического дизайна.

- Добавьте вместо синего фона страницы изображение.
- Добавьте на кнопку вместо надписи *Добавить* иконку SVG, что позволит уменьшить ее размеры (не забудьте увеличить размеры поля для ввода).
- Вместо фиолетового прямоугольника справа от формулировки задачи добавьте SVG иконку, обозначающую удаление этой задачи.
- Подберите цветовую гамму элементов в соответствии с вашими представлениями о дизайне этого мини-проекта.

Задание 5. Представление практической работы

Добавьте созданные файлы и каталоги в каталог **lab-4** на GitHub. Прикрепите ссылку на этот каталог с работой в текущий курс для проверки преподавателем.

Контрольные вопросы

1. Что такое HTML и какова его основная функция в веб-разработке?
2. Какова структура базового HTML-документа? Приведите пример.
3. Что такое тег <head> и какую информацию он содержит?
4. Какой тег используется для заголовка страницы и как его правильно использовать?
5. Что такое тег <body> и какую роль он играет в HTML-документе?
6. Как создать заголовки разных уровней с помощью тегов HTML?

7. Что такое тег <p> и как он используется для создания абзацев?
8. Как создать гиперссылку с помощью тега <a>? Какие атрибуты он поддерживает?
9. Как вставить изображение на веб-страницу с использованием тега <img>?
10. Как создать неупорядоченный и упорядоченный список в HTML? Приведите примеры.
11. Что такое таблицы в HTML и как они создаются с помощью тегов <table>, <tr>, <th>, и <td>?
12. Как создать форму для ввода данных с использованием тегов <form> и <input>?
13. Что такое атрибут alt в теге <img> и почему он важен?
14. Каковы основные семантические теги в HTML и их назначение?
15. Что такое страница 404 и когда она отображается?
16. Как оформить страницу 404 с помощью HTML? Какие элементы стоит включить?
17. Как использовать тег <meta> для настройки параметров страницы, таких как кодировка?
18. Что такое атрибут href в теге <a> и как он влияет на навигацию?
19. Как создать кнопку с помощью тега <button> и как она может взаимодействовать с формами?
20. Какие рекомендации по дизайну и содержанию следует учитывать при создании страницы 404?

ПРАКТИЧЕСКАЯ РАБОТА 5. СТРУКТУРНЫЕ ЭЛЕМЕНТЫ HTML5

Цель работы

- **Ознакомление с HTML5:** изучить основные нововведения и преимущества HTML5 по сравнению с предыдущими версиями.
- **Изучение структурных элементов:** познакомиться с основными структурными элементами HTML5, такими как <header>, <footer>, <nav>, <article>, <section>, <aside> и <main>.
- **Создание семантически правильной разметки:** научиться правильно использовать структурные элементы для создания семантически корректной разметки веб-страницы.
- **Понимание роли семантики:** понять, как семантические теги улучшают доступность, SEO и читаемость кода.
- **Организация контента:** научиться организовывать контент на странице с использованием структурных элементов для улучшения его восприятия пользователями.
- **Создание многоуровневой структуры:** изучить, как создавать сложные страницы с многоуровневыми структурами, используя различные семантические элементы.

Теоретическая часть

5.1. Структурные элементы HTML5

Прежде что-то делать, зайдите и посмотрите на некоторые из любимых веб-сайтов (мы уже умеем использовать инструменты анализа разметки). Они содержат совершенно разный контент, функциональность, внешний вид и ощущения, но у всех будут общие структурные элементы. Существует очень мало сайтов, которые хотя бы приблизительно не следуют этому шаблону:

- **Заголовок**: обычно это большая полоса сверху с большим заголовком, логотипом и, возможно, слоганом. Он остается неизменным от одной веб-страницы к другой.
- **Панель навигации**: ссылки на основные разделы сайта; представлены кнопками меню, ссылками или вкладками. Как и заголовок, этот контент остается неизменным от одной веб-страницы к другой — непоследовательная навигация на сайте приведет только к путанице и разочарованию пользователей. Многие веб-дизайнеры считают панель навигации частью заголовка, а не отдельным компонентом, но это не обязательно; на самом деле, некоторые также утверждают, что наличие двух отдельных элементов лучше для доступности, поскольку программы чтения с экрана могут лучше читать эти две функции, если они разделены.
- **Основное содержание**: большая область в центре, которая содержит большую часть уникального контента данной веб-страницы, например, видео, основную статью, карту или заголовки новостей и т.д. Это та часть веб-сайта, которая определенно будет меняться от страницы к странице!
- **Боковая панель**: некоторая периферийная информация, ссылки, цитаты, реклама и т.д. Это контекстно по отношению к основному контенту (например, на странице новостной статьи боковая панель может содержать биографию автора или ссылки на связанные статьи), но бывают и случаи, когда найдется некоторые повторяющиеся элементы, такие как вторичная навигационная система.
- **Нижний колонтитул**: полоса в нижней части страницы, которая содержит мелкий шрифт, уведомления об авторских правах или контактную информацию. Это место для размещения общей информации (например, заголовка), но эта информация не является критической или второстепенной по отношению к самому веб-сайту. Нижний колонтитул также иногда используется в целях SEO, предоставляя ссылки для быстрого доступа к популярному контенту.

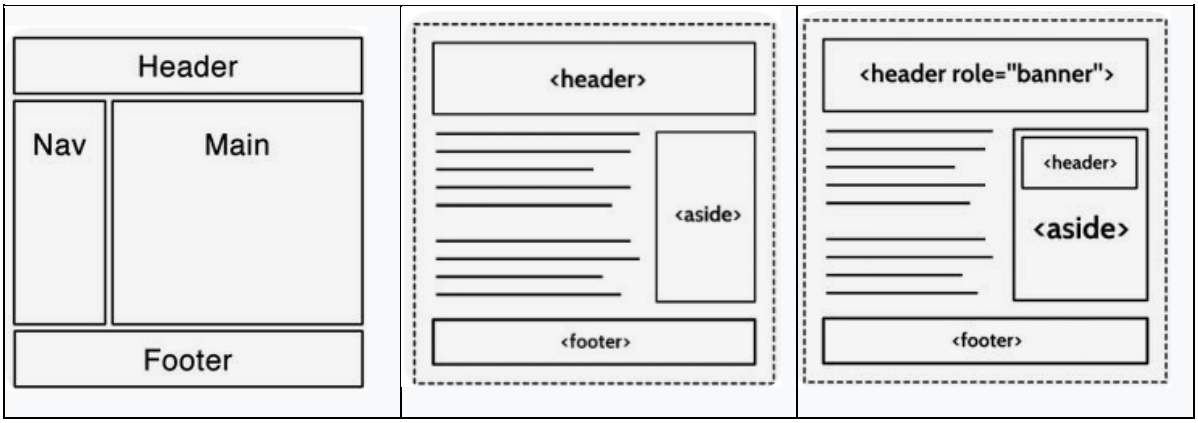


Рис. 5.1. Структурные элементы

Эти разделы могут содержать любое количество различных вложенных элементов; например, нижний колонтитул и может включать список ссылок, пару абзацев и изображение.

Структурирование с помощью HTML 4

HTML4 содержит два общих элемента-контейнера:

- Это блочный контейнер , который можно использовать для группировки разделов контента на уровне блока.
- Это встроенный контейнер , который можно использовать для группировки фрагментов встроенного контента.

Обратите внимание, что элементы и не имеют собственного содержания или размеров, только их содержимое, пока они не будут стилизованы через CSS или не будут обработаны JavaScript. У них также нет семантики, и единственный способ их идентифицировать — это присвоить им атрибут class или id.

Эти элементы следует использовать только в том случае, если нет более подходящего семантического элемента для разметки контента. Разметка абзацев и заголовков с помощью — плохая практика; хотя на первый взгляд это может выглядеть нормально, страница будет недоступна для тех, кто использует программы чтения с экрана (программам чтения с экрана нужна хорошая структура заголовков для навигации), и она будет более раздутой, трудной для чтения и менее эффективной для стилизации с помощью CSS. Постарайтесь сократить количество используемых и , когда это возможно: злоупотребление этими элементами приводит к тому, что часто называют «**div-itis**».

Итак, чтобы разметить пример сайта с помощью HTML4, вы можете использовать следующие элементы. Разметка будет выглядеть примерно так:

```
<body>

  <div id="header">
    <!-- header content goes in here -->
  </div>

  <div id="nav">
    <!-- navigation menu goes in here -->
  </div>

  <div id="sidebar1">
    <!-- sidebar content goes in here -->
  </div>

  <div id="main">
    <!-- main page content goes in here -->
  </div>

  <div id="sidebar2">
```

```
    <!-- sidebar content goes in here -->
  </div>

  <div id="footer">
    <!-- footer content goes in here -->
  </div>

</body>
```

Структурирование в HTML5

Метод HTML4 хорош и эффективен, но семантически он мог бы быть намного лучше.

Люди могут различать разный контент, но машины не могут — браузер не видит разные div как заголовок, нижний колонтитул и т. д. Он видит их как разные div. Разве не было бы полезнее, если бы браузеры и программы чтения с экрана могли явно идентифицировать, скажем, навигационное меню, чтобы слабовидящий пользователь мог легче его найти, или разные новостные статьи в куче блогов, чтобы их можно было легко синдицировать в RSS-канале без дополнительного программирования? Даже если используется дополнительный код для решения некоторых из этих проблем, все равно можно сделать решение надежным для своих веб-сайтов, поскольку разные веб-разработчики будут использовать разные имена классов и идентификаторов, особенно если учесть международную аудиторию — разные веб-разработчики в разных странах будут использовать разные языки для написания своих имен классов и идентификаторов.

Имеет смысл определить единый набор элементов, который все могли бы использовать для общих структурных блоков, появляющихся на многих веб-сайтах, и именно это определено в HTML5. Для реализации семантической разметки HTML5 предоставляет специальные теги, которые можно использовать для представления таких разделов, например:

- **заголовок:** <header>.
- **панель навигации:** <nav>.
- **основное содержимое:**, с различными подразделами содержимого , <main>представленными элементами, <article>, <section>, .
- **боковая панель:** <aside>; часто размещается внутри <main>.
- **нижний колонтитул:** <footer>.

На данный момент мы рассматриваем и работаем со следующими тегами:

- **<main>**для контента, *уникального для этой страницы.* Используйте <main> только *один раз* на странице и поместите его непосредственно внутри <body>. В идеале это не должно быть вложено в другие элементы.

- **<article>**включает в себя блок связанного контента, который имеет смысл сам по себе, без остальной части страницы (например, отдельная запись в блоге).
- **<section>**похож на <article>, но он больше предназначен для группировки одной части страницы, которая составляет одну единую часть функциональности (например, мини-карта или набор заголовков и резюме статей), или темы. Считается лучшей практикой начинать каждый раздел с заголовка; также обратите внимание, что вы можете разбить <article> на разные <section> или <section> на разные <article>, в зависимости от контекста.
- **<aside>**содержит контент, который не связан напрямую с основным контентом, но может предоставить дополнительную информацию, косвенно связанную с ним (словарные статьи, биография автора, ссылки по теме и т. д.).
- **<header>**представляет собой группу вводного контента. Если он является потомком , <body> то определяет глобальный заголовок веб-страницы, но если он является потомком <article> или <section>, то определяет конкретный заголовок для этого раздела (старайтесь не путать это с заголовками и заголовками).
- **<nav>**содержит основную навигационную функциональность для страницы. Вторичные ссылки и т. д. не попадут в навигацию.
- **<footer>**представляет собой группу конечного контента для страницы.

5.2. Метаданные в HTML

Форматирование текста с помощью HTML

HTML имеет в арсенале несколько тегов для представления текста на веб-страницах, чтобы он отличался от обычного текста. Например, использовать тег `<b>`, чтобы сделать текст полужирным, или тег `<i>`, чтобы сделать текст курсивом, тег `<mark>`, чтобы выделить текст, тег `<code>` - для отображения фрагмента компьютерного кода, теги `<ins>` и `<del>` для пометки редакционных вставок и удалений и многое другое.

В следующем примере показаны наиболее часто используемые теги форматирования в действии. Давайте попробуем разобраться в том, как работают эти теги:

```
<p>This is <b>bold text</b>.</p>
<p>This is <strong>strongly important text</strong>.</p>
<p>This is <i>italic text</i>.</p>
<p>This is <em>emphasized text</em>.</p>
<p>This is <mark>highlighted text</mark>.</p>
<p>This is <code>computer code</code>.</p>
<p>This is <small>smaller text</small>.</p>
<p>This is <sub>subscript</sub> and <sup>superscript</sup> text.</p>
<p>This is <del>deleted text</del>.</p>
<p>This is <ins>inserted text</ins>.</p>
```

По умолчанию тег обычно отображается в браузере как <b>, тогда как сам тег отображается как <i>. Однако есть разница в значении этих тегов.

Разница между тегами и <b>

Оба тега и <b>, по умолчанию отображают заключенный текст полужирным шрифтом, но тег указывает на то, что его содержимое имеет большую важность. Тег <b> просто используется для привлечения внимания читателя, не сообщая какой-либо особой важности.

```
<p><strong>ВНИМАНИЕ!</strong> Пожалуйста, действуйте с осторожностью.</p>
<p>Концерт пройдет в <b>парке Сокольников</b> в Москве.</p>
```

Разница между тегами и <i>

Точно так же оба тега и <i> отображают заключенный текст курсивом. Тег указывает, что его содержимое имеет подчеркнутое выделение по сравнению с окружающим текстом, а тег <i> используется для разметки текста, который отделен от обычного текста по соображениям удобочитаемости (например, технический термин, идиоматическая фраза из другого языка, мысль и т.д.)

```
<p>Кошки — <em>милые</em> животные.</p>
<p><i>Королевский круиз</i> отплыл прошлой ночью.</p>
```

Используйте теги и , когда содержание страницы требует, чтобы определенные слова или фразы имели сильный акцент или были важны. Также в HTML5 переопределены теги <b> и <i>, ранее не имевшие ***семантического значения****.*

Форматирование цитат

Форматировать блоки цитат из других источников в HTML можно легко с помощью тега <blockquote>. Цитаты обычно отображаются с отступом влево и вправо, а также с добавлением небольшого дополнительного пространства сверху и снизу. Попробуем увидеть, как это работает:

```
<blockquote>
  <p>Учись у вчера, живи сегодня, надейся на завтра. Главное не переставать задавать вопросы.</p>
  <cite>— Альберт Эйнштейн</cite>
</blockquote>
```

Тег `cite` используется для описания ссылки на творческую работу. Он должен включать название этой работы или имя автора (людей или организации) или ссылку на URL.

Для коротких встроенных цитат используется HTML тег `<q>`. Большинство браузеров отображают встроенные кавычки, заключая текст в кавычки. Вот пример:

```
<p>По данным Всемирной организации здравоохранения (ВОЗ): <q>Здоровье — это состояние полного физического, психического и социального благополучия.</q></p>
```

Отображение сокращений

Аббревиатура *– это сокращенная форма слова, фразы или имени.*

Для обозначения аббревиатуры, используется тег `<abbr>`. Атрибут `title` используется внутри этого тега, чтобы обеспечить полное раскрытие аббревиатуры. Она отображается браузерами в виде всплывающей подсказки при наведении курсора мыши на элемент. Давайте рассмотрим пример:

```
<p>The <abbr title="World Wide Web Consortium">W3C</abbr> является основной международной организацией по стандартизации для <abbr title="World Wide Web">WWW или W3</abbr>. Ее основал Тим Бернерс-Ли.</p>
```

Маркировка контактных адресов

Веб-страницы часто содержат уличные или почтовые адреса. HTML предоставляет специальный тег `<address>` для представления **контактной информации (физической и/или цифровой) человека**, людей или организации.

Этот тег, в идеале, следует использовать для отображения контактной информации, связанной с самим документом, например, автора статьи. Большинство браузеров отображают адресный блок курсивом. Вот пример:

```
<address>
ООО СТУДИЯ ЦИФРОВЫХ РЕШЕНИЙ<br>
Россия, 424000, г. Йошкар-Ола,<br>
ул. Карла Маркса, д. 109Б, ПОЗИЦИЯ 9
</address>
```

Редактирование текста

Большинство структурированных текстов состоят из **параграфов** и **заголовков**, независимо от того, читаете ли мы рассказ, газету, учебник или журнал. Упорядоченный контент делает чтение более лёгким и приятным.

В HTML каждый абзац заключён в элемент `<p>`, подобно:

```
<p>Я параграф. Да, это я.</p>
```

Каждый заголовок заключён в элемент заголовка `<h1>`:

```
<h1>Я заголовок истории.</h1>
```

Имеется шесть элементов заголовка: `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>` и `<h6>`. Каждый элемент представляет разный уровень контента в документе; `<h1>` - представляет главный заголовок, `<h2>` - представляет подзаголовки, `<h3>` - представляет под-подзаголовки и т.д.

Создание иерархической структуры

Например, в рассказе `<h1>` будет представлять заглавие рассказа, `<h2>` обозначит название каждой главы, `<h3>` будет обозначать подзаголовки в каждой главе и так далее.

```
<h1> Сокрушительная скука </ h1>
<p> Крис Миллс </ p>
<h2> Глава 1: Тёмная ночь </ h2>
<p> Это была тёмная ночь. Где-то кричала сова. Дождь обрушился на ... </ p>
<h2> Глава 2: Вечное молчание </ h2>
<p> Наш главный герой ничего не мог, когда шёпот из тёмной фигуры ... </ p>
<h3> Призрак говорит </ h3>
<p> Прошло ещё несколько часов, когда внезапно призрак выпрямился и воскликнул: «Пожалуйста, помилуй мою душу!» </ p>
```

Всё это действительно зависит от нас — что именно будут представлять собой элементы, пока существует иерархия. Нам просто нужно иметь в виду несколько хороших правил при создании таких структур.

- Предпочтительнее использовать `<h1>` только один раз на странице — это заголовок самого верхнего уровня, и все остальные заголовки располагаются ниже его в иерархии.
- Нужно убедиться, что мы используем заголовки в правильном порядке в иерархии. Не используем `<h3>` для создания подзаголовков при одновременном

использовании `<h2>` для представления под-подзаголовков — это не имеет смысла и приведёт к странным результатам.

- Из шести доступных уровней заголовка мы должны стремиться использовать не более трёх на странице, если только в этом нет острой необходимости. Документы с большим количеством уровней (то есть с глубокой иерархией) становятся громоздкими и трудными для навигации. В таких случаях рекомендуется распределять контент по нескольким страницам, если это возможно.

Зачем нам необходима структура?

Это связано с тем, что нет элементов для создания структуры контента, поэтому браузер не знает, где в тексте заголовок, а где абзац. Более того:

- Пользователи, просматривающие веб-страницу, быстро сканируют её в поиске подходящего контента, часто просто просматривая только заголовки. Если они не смогут увидеть ничего полезного в течение нескольких секунд, они, скорее всего, расстроятся и отправятся куда-нибудь ещё.
- Поисковые системы, индексирующие страницу, считают содержание заголовков важными ключевыми словами для влияния на ранжирование поиска страницы. Без заголовков страница будет плохо работать с точки зрения **SEO (Search Engine Optimization — поисковая оптимизация)**.
- Сильно слабовидящие люди часто не читают веб-страницы — они их слушают. Это делается с помощью программного обеспечения, называемого программой чтения с экрана. Это программное обеспечение предоставляет способы быстрого доступа к данному текстовому контенту. Среди различных используемых методов они предоставляют схему документа, считывая заголовки, позволяя своим пользователям быстро находить нужную им информацию. Если заголовки недоступны, они будут вынуждены слушать весь документ вслух.
- Чтобы стилизовать контент с помощью CSS или сделать его интересным с помощью JavaScript, нужно, чтобы элементы обёртывали соответствующий контент, тогда CSS и JavaScript будут работать эффективно.

Поэтому нужно дать структурную разметку нашему контенту.

Почему мы нуждаемся в семантике?

Семантика проявляется всюду вокруг нас — мы полагаемся на опыт, который рассказывает нам о функциях бытовых предметов; когда мы что-то видим, мы знаем, какова функция предмета. Например, мы ожидаем, что красный свет на светофоре означает «стоп», а зелёный свет означает «идти». Жизнь станет очень сложной, если применяется неправильная семантика. В подобном ключе нужно убедиться, что мы используем правильные элементы, придавая контенту осмысленность, функциональность или внешний вид.

В этом контексте элемент `<h1>` также является **семантическим элементом**, который даёт тексту роль (или значение) «заголовка верхнего уровня на вашей странице».

```
<h1>Это заголовок верхнего уровня</h1>
```

По умолчанию, браузер придаст ему большой размер шрифта, чтобы он выглядел как заголовок (хотя вы можете стилизовать его, как угодно, используя CSS). И что ещё более важно, его **семантическое значение** будет использоваться несколькими способами, например, поисковыми системами и программами чтения с экрана (как упоминалось выше).

С другой стороны, любой элемент можно сделать похожим на заголовок верхнего уровня. Рассмотрим пример:

```
<span style="font-size: 32px; margin: 21px 0;">Это заголовок верхнего уровня?</span>
```

Это элемент `<span>`. У него, кстати, нет семантики. Мы используем его, когда хотим применить к контенту CSS (или сделать что-то с ним с помощью JavaScript), не придавая ему никакого дополнительного значения (об этом мы поговорим позже).

Определяем CSS, чтобы он выглядел как заголовок верхнего уровня, но поскольку он не имеет семантического значения, он не получит никаких дополнительных преимуществ, описанных выше. Рекомендуется использовать соответствующий элемент HTML на практике.

Акцент и важность

Когда мы хотим добавить акцент в разговорный язык, то подчёркиваем определённые слова, тонко изменяя смысл того, что говорим. Точно так же на письменном языке мы склонны подчёркивать слова, выделяя их *курсивом*. Например, следующие два предложения имеют разные значения.

Я рад, что ты не опоздал.

Я *рад,* что ты не *опоздал*.

В первом предложении звучит искреннее облегчение, что человек не опоздал. Во втором, напротив, звучит **сарказм** или **пассивная агрессия**: так выражена досада от того, что человек немного опоздал. В таких случаях в HTML используется элемент `<em>` (выделение).

Браузеры стилизуют это по умолчанию курсивом, но мы можем не использовать этот тег, чтобы получить курсив. Для выделения курсивом вы можете использовать элемент `<span>` и CSS, или, возможно, элемент `<i>`.

```
<p>Я <em>рад</em>, что ты не <em>опоздал</em>.</p>
```

Важное значение

Чтобы подчеркнуть важные слова, мы склонны подчёркивать их в устной речи и **выделять жирным** в письменном изложении. Например:

Эта жидкость **очень токсична**.
Я рассчитываю на вас. **Не опаздывай**!

В таких случаях в HTML используется элемент `<strong>` (важное значение). Помимо того, что документ становится более полезным, они распознаются программами, считывающими с экрана, и проговариваются другим тоном. Браузеры стилизуют это как полужирный текст по умолчанию, но мы можем не использовать этот тег, чтобы получить жирный шрифт. Для получения жирного шрифта обычно используется элемент `<span>` и CSS, или элемент `<b>`.

```
<p>Эта жидкость <strong>очень токсична</strong>.</p>

<p>Я рассчитываю на тебя. <strong>Не </strong>опаздывай!</p>
```

При желании можно вложить важные и акцентированные слова друг в друга:

```
<p>Эта жидкость <strong>очень токсична</strong> —
если ты выпьешь её, <strong>то можешь<em>умереть</em></strong>.</p>
```

Аббревиатуры

Другой элемент, который часто встречается в Интернет — это `<abbr>`. Используемый для обёртывания **аббревиатур** или **акронимов** и обеспечивающий полную расшифровку сокращения (с помощью атрибута `title`).

Давайте рассмотрим несколько примеров:

```
<p>Мы используем <abbr title="Hypertext Markup Language">HTML</abbr> для структурирования наших веб-документов.</p>

<p>Я думаю, <abbr title="Почтенный">Почт.</abbr> Грин сделал это на кухне.</p>
```

Они будут выглядеть примерно так (расшифровка появится в подсказке при наведении курсора мыши на сокращение):

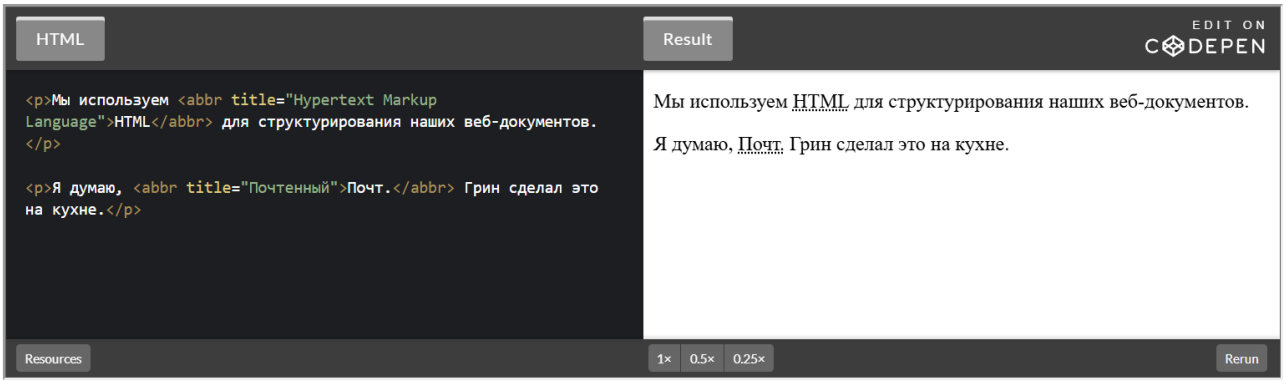


Рис. 5.1. Результат разметки аббревиатур

Существует ещё один элемент `<acronym>`*, который в основном делает то же самое, что и* `<abbr>`*, но предназначен специально для* ***акронимов*** *(тип аббревиатур). Он не поддерживается в браузерах на том же уровне, что* `<abbr>`*, но имеет такую же функциональность. Считается бессмысленным применять оба элемента разметки в коде. Просто используйте* `<abbr>`*.*

Разметка контактных данных

HTML имеет элемент для разметки **контактных данных** — `<address>`. Он просто обёртывает контактные данные, например:

```
<address>
  <p>Борис Канорский, Норильск, Жестокий Север, РФ</p>
</address>
```

Однако следует помнить, что элемент `<address>` предназначен для разметки контактных данных человека, который написал HTML-документ, а не любого адреса.

Таким образом, написанное выше было бы корректным только в том случае, если бы Борис написал документ, на котором появляется разметка. Обратите внимание, что следующее также подойдёт:

```
<address>
  <p>Автор страницы — <a href="../authors/boris/">Борис Канорский</a>.</p>
</address>
```

Верхний и нижний индекс

Иногда нам понадобится использовать **надстрочный** или **подстрочный индекс** при разметке таких вещей, как даты, химические формулы и математические

уравнения, чтобы они имели правильное представление. Элементы `<sup>` и `<sub>` созданы для таких ситуаций. Приведём пример:

```
<p>Я просыпаюсь в 6<sup>35</sup> часов утра.</p>
<p>Химическая формула кофеина:
C<sub>8</sub>H<sub>10</sub>N<sub>4</sub>O<sub>2</sub>.</p>
<p>Если x<sup>2</sup> равно 9, x должен равняться 3 или -3.</p>
```

Результат этого кода выглядит следующим образом:

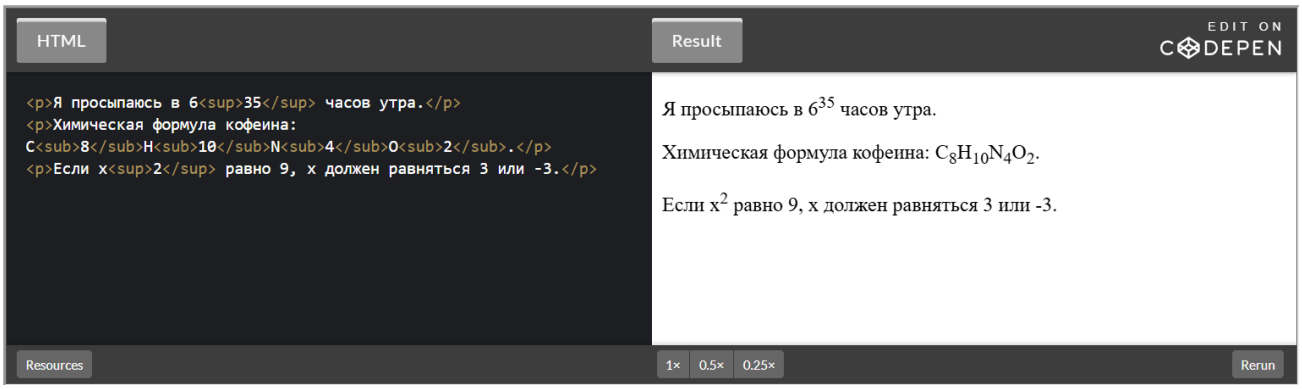


Рис. 5.2. Результат разметки верхних и нижних индексов

Представление кода

Существует несколько элементов для маркировки программного кода с использованием HTML:

- `<code>`: для разметки общих частей программного кода.
- `<pre>`: для сохранения пробелов (как правило, кодовых блоков) — если мы используем отступы или лишние пробелы внутри текста, браузеры будут игнорировать его, и мы не увидим его на отображаемой странице; если мы поместим текст в теги `<pre></pre>`, то пробелы будут отображаться идентично тому, как мы видим текст в редакторе.
- `<var>`: для конкретной маркировки имён переменных.
- `<kbd>`: для маркировки ввода с клавиатуры (и других типов).
- `<samp>`: для маркировки вывода программы.

Давайте рассмотрим несколько примеров:

```
<pre><code>var para = document.querySelector('p');

para.onclick = function() {
  alert('Owww, stop poking me!');
```

```
}</code></pre>

<p>Вы не должны использовать презентационные элементы, такие как <code>&lt;font&gt;</code> и <code>&lt;center&gt;</code>.</p>

<p>В приведенном выше примере JavaScript <var>para</var> представляет элемент абзаца.</p>

<p>Выделите весь текст с помощью <kbd>Ctrl</kbd>/<kbd>Cmd</kbd> + <kbd>A</kbd>.</p>

<pre>$ <kbd>ping mozilla.org</kbd>
<samp>PING mozilla.org (63.245.215.20): 56 data bytes
64 bytes from 63.245.215.20: icmp_seq=0 ttl=40 time=158.233 ms</samp></pre>
```

Вышеприведённый код будет выглядеть так:

Рис. 5.3. Результат разметки программного кода

Разметка времени и даты

HTML также содержит элемент `<time>` для отметки времени и дат в машиночитаемом формате. Например:

```
<time datetime="2020-01-20">20 Января 2020</time>
```

Почему это полезно? Есть много разных способов, которыми люди записывают даты. Вышеуказанная дата может быть записана как:

- 20 Января 2020
- 20-ое Января 2020
- Янв 20 2020
- 20/01/20
- 01/20/20
- 20e Janvier 2020

- 2020年1月20日
- и т.д...

Но эти разные формы не могут быть легко распознаны — так что, если мы хотим автоматически захватить даты всех событий на странице и вставить их в календарь?

Элемент `<time>` позволяет прикрепить к этой цели однозначное машиночитаемое время / дату. В приведённом выше базовом примере приведена простая машиносчитываемая дата, но есть много других возможных вариантов, например:

```
<!-- Стандартная дата -->
<time datetime="2020-01-20">20 Января 2020</time>
<!-- Только год и месяц -->
<time datetime="2020-01">Январь 2020</time>
<!-- Только месяц и день -->
<time datetime="01-20">20 Января</time>
<!-- Только время, часы и минуты -->
<time datetime="19:30">19:30</time>
<!-- Также вы можете отобразить секунды и миллисекунды! -->
<time datetime="19:30:01.856">19:30:01.856</time>
<!-- Дата и время -->
<time datetime="2020-01-20T19:30">7.30pm, 20 Января 2020</time>
<!-- Дата и время со смещением по часовому поясу -->
<time datetime="2020-01-20T19:30+01:00">7.30pm, 20 Января 2020, — это 8.30pm во Франции.</time>
<!-- Вызов номера недели -->
<time datetime="2020-W04">Четвёртая неделя 2020</time>
```

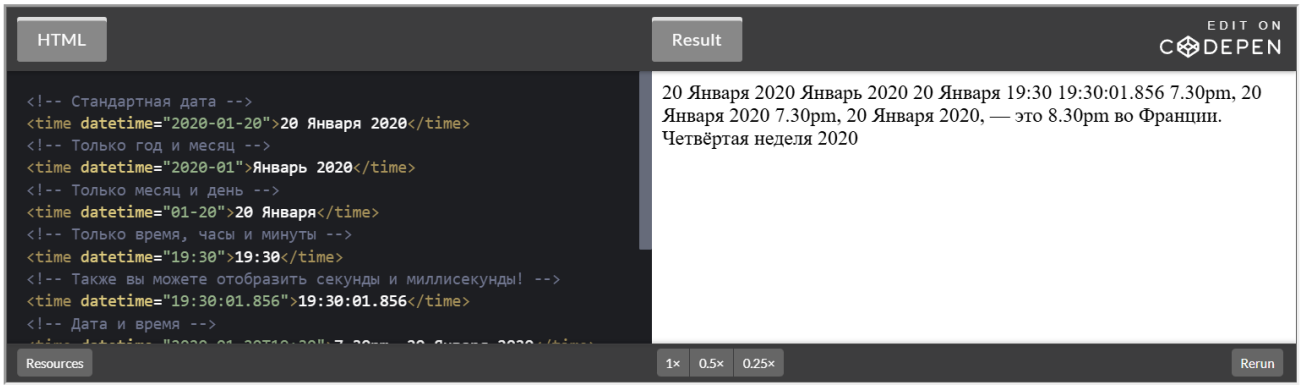


Рис. 5.4. Результат разметки дат

5.3. Практика написания хороших ссылок

При написании ссылок рекомендуется следовать некоторым правилам. Давайте рассмотрим их.

- **Используйте чёткие формулировки описания ссылок**

На странице легко добавить ссылки, но этого недостаточно. Мы должны сделать наши ссылки доступными для всех читателей, независимо от их возможностей и инструментов просмотра страницы, которые они предпочитают.

Например:

- Пользователям программ, читающих с экрана, нравится переходить по ссылкам на странице, читая адрес ссылки в тексте.
- Поисковые системы используют текст ссылки для индексирования файлов, поэтому рекомендуется включать ключевые слова в текст ссылки, чтобы эффективно описывать, куда ведёт ссылка.
- Пользователи часто бегло просматривают страницу, не читая каждое слово, и их глаза будут привлечены к тексту, который выделяется, например, ссылки. Им будет полезно описание того, куда ведёт ссылка.

Взгляните на этот пример:

***Хороший** текст ссылки:* Скачать Яндекс

```
<p><a href="https://yandex.ru/">
  Скачать Яндекс
</a></p>
```

Плохой *текст ссылки:* Нажми сюда, чтобы скачать Яндекс

```
<p><a href="https://yandex.ru/">
  Нажми сюда
</a>
чтобы скачать Яндекс</p>
```

Советы:

- **Не пишите URL-адрес как часть текста ссылки**. URL-адреса выглядят сложными, а звучат ещё сложнее, когда программы чтения с экрана читают их по буквам.
- **Не пишите «ссылка» или «ссылки на» в тексте ссылки — это лишнее**. Программы чтения с экрана сами проговаривают, что есть ссылка. На экране пользователи также видят, что есть ссылка, потому что ссылки, как правило, оформлены в другом цвете и подчёркнуты (подчёркивая ссылки, вы соблюдаете правила хорошего тона, поскольку пользователи привыкли к этому).

- **Следите за тем, чтобы текст ссылки был как можно короче**. Длинный текст ссылки особенно раздражает пользователей программ чтения с экрана, которым придётся услышать всё, что написано.
- **Минимизируйте случаи, когда несколько копий одного и того же текста ссылок указывает на разные страницы**. Это может вызвать проблемы для пользователей программ чтения с экрана, которые часто вызывают список ссылок — несколько ссылок, помеченных как «нажмите здесь», «нажмите здесь», «нажмите здесь», будут путать.

- **Используйте относительные ссылки, где это возможно**

Из прочитанного выше, можно подумать, что всё время использовать **абсолютные ссылки** — хорошая идея; в конце концов, они не ломаются, когда страница перемещается. Тем не менее, лучше использовать **относительные ссылки** везде, где это возможно, **в пределах одного сайта** (при ссылке на другие сайты необходимо использовать абсолютную ссылку):

- Во-первых, гораздо проще прописать в коде относительные URL-адреса. Как правило, они намного короче абсолютных URL-адресов, что значительно упрощает чтение кода.
- Во-вторых, использование относительных URL-адресов эффективней по следующей причине. Когда используется абсолютный URL-адрес, браузер начинает поиск реального местоположения сервера запрашивая адрес у **Domain Name System (DNS)**, затем он переходит на этот сервер и находит файл, который запрашивается. С относительным URL-адресом проще: браузер просто ищет файл, который запрашивается на том же сервере. Использование абсолютных URL-адресов вместо относительных, происходит постоянная нагрузка браузер дополнительной работой.

- **Создавая ссылки на не HTML ресурсы — добавляйте описание**

Когда создается ссылка на файл, нажав на который можно загрузить документ PDF или Word или открыть просмотр видео, прослушивание аудио файла или перейти на страницу с другим, неожиданным для пользователя результатом (всплывающее окно), следует добавлять чёткую формулировку, чтобы уменьшить путаницу. Отсутствие описания может раздражать пользователя.

Приведём пример:

- Если мы используем соединение с низкой пропускной способностью и вдруг нажмём на ссылку без описания, начнётся загрузка большого файла.
- Если у нас нет установленного плеера и мы нажмём ссылку, то внезапно произойдет переход на страницу с контентом.

Рассмотрим примеры, чтобы увидеть, как добавить описание:

```
<p><a href="http://www.example.com/large-report.pdf">
  Скачать отчёт о продажах (PDF, 10MB)
</a></p>

<p><a href="http://www.example.com/video-stream/">
  Посмотреть видео (видео откроется в отдельном окне, HD качество)
</a></p>

<p><a href="http://www.example.com/car-game">
  Играть в гонки (необходим Flash)
</a></p>
```

- **Используйте атрибут download, когда создаётся ссылка**

Когда создается ссылка на файл, который должен быть загружен, а не открыт в браузере, можно использовать атрибут `download`. Указывается имя файла, по умолчанию, для его сохранения.

Приведём пример ссылки для загрузки браузера Firefox 39:

```
<a href="https://download.mozilla.org/?product=firefox-39.0-SSL&os=win&lang=en-US"
   download="firefox-39-installer.exe">
  Скачать Firefox 39 для Windows
</a>
```

- **Ссылки электронной почты**

Можно создавать ссылки или кнопки, которые при нажатии открывают новое исходящее сообщение электронной почты, а не ссылку на ресурс или страницу. Для этого используется элемент `<a>` и `mailto:` — **адрес почты**. Самыми простыми и часто используемыми формами `mailto:` являются **subject**, **cc**, **bcc** и **body**; дальше прописываем адрес электронной почты.

Например:

```
<a href="mailto:nowhere@yandex.ru">Отправить письмо для nowhere</a>
```

В результате получим ссылку вида:

Сам адрес электронной почты не является обязательным для заполнения. Если оставить это поле пустым (в поле `href` оставить только "mailto:"), откроется новое исходящее сообщение почтовой программой, в поле получателя будет пусто. Это можно использовать для кнопки "**Поделиться**".

- **Особенности и детали**

Помимо адреса электронной почты, мы можем предоставить другую информацию. Фактически, любые стандартные поля для отправки почты могут быть добавлены к указанному адресу `mailto`.

Часто используемыми из них являются «subject», «cc» и «body» (которые не являются истинным полем заголовка, но позволяют указать дополнительную информацию для нового сообщения электронной почты). Каждое поле и его значение задаются в качестве условия запроса.

Вот пример, который включает **cc** (кому отправить копию сообщения, все получатели письма видят список тех кто это письмо получит), **bcc** (скрытый адрес получателя, никто из получателей не будет видеть полный список получателей письма), **subject** (тема письма) и **body** (текст сообщения):

```
    <a href="mailto:nowhere@yan-
dex.ru?cc=name2@rapidtables.com&bcc=name3@rapidtables.com&amp;sub-
ject=The%20subject%20of%20the%20email
&amp;body=The%20body%20of%20the%20email">
      Отправить письмо с cc, bcc, subject и body
    </a>
```

Значение каждого поля должно быть написано в URL-кодировке (то есть с непечатаемыми символами и пробелами percent-escaped). Обратите внимание на знак вопроса (?) для разделения основного адреса и дополнительных полей, амперсанд (&) для разделения каждого поля `mailto:` URL. Для этого используется стандартное описание URL запроса.

Вот несколько примеров использования `mailto` URLs:

```
    mailto:
    mailto:nowhere@yandex.ru
    mailto:nowhere@yandex.ru,nobody@yandex.ru
    mailto:nowhere@yandex.ru?cc=nobody@yandex.ru
    mailto:nowhere@yandex.ru?cc=nobody@yandex.ru&subject=This%20is%20the%20sub
ject
```

5.4. Гиперссылки

Гиперссылки *— одно из самых интересных нововведений Интернета. Они были особенностью Сети с самого начала, но именно они превращают Интернет в Интернет. Они позволяют нам связывать документы с любым другим документом (или ресурсом), с которым мы хотим.*

Ссылка или гиперссылка — это соединение одного веб-ресурса с другим. Ссылки позволяют пользователям беспрепятственно переходить с одной страницы на другую на любом сервере в любой точке мира. С их помощью мы также можем связывать документы с их конкретными частями, с также делать приложения доступными на простом веб-адресе. Почти любой веб-контент может быть преобразован в ссылку, при клике по ней (или активации иным образом), она заставляет веб-браузер перейти на другой **веб-адрес (URL)**.

URL-адрес может указывать на файлы HTML, текстовые файлы, изображения, текстовые документы, видео и аудиофайлы и все остальное, что может присутствовать в Интернет. Если веб-браузер не знает, как отображать или обрабатывать файл, он запросит у пользователя разрешения на открытие файла (в этом случае обязанность открытия или обработки файла передаётся в соответствующее локальное приложение на устройстве) или загрузки файла (в этом случае вы можете попытаться разобраться с ним позже).

У ссылки есть два конца, называемые **анкорами (якорь ссылки)**. Ссылка начинается с исходной привязки и указывает на целевую привязку, которая может быть любым веб-ресурсом, например, изображением, аудио- или видеоклипом, файлом PDF, документом HTML или элементом в самом документе и т.д.

По умолчанию в большинстве браузеров ссылки отображаются следующим образом:

- Не посещённая ссылка подчеркнута и окрашена в синий цвет.
- Посещенная ссылка подчеркнута и окрашена в фиолетовый цвет.
- Активная ссылка подчеркнута и окрашена в красный цвет.

Однако эти состояния и вид переопределяются с помощью CSS.

Анатомия ссылки (синтаксис)

Ссылки указываются в HTML с помощью тега `<a>`. Ссылка или гиперссылка может быть словом, группой слов или изображением.

```
<a href="url"> Текст ссылки </a>
```

Все, что находится между открывающим `<a>` и закрывающим `</a>` тегами, становится частью ссылки — это пользователь видит и нажимает в браузере. Вот несколько примеров ссылок:

```
<a href="https://www.yandex.ru/">Поисковая система</a>
<a href="https://digtlab.ru">Сайт Студии цифровых решений</a>
<a href="images/kites.jpg"> <img src="kites-thumb.jpg" alt="kites"> </a>
```

Атрибут `href` указывает **цель ссылки**. Его значение может быть **абсолютным** или **относительным** URL-адресом. Простая ссылка создаётся путём обёртывания текста (или другого содержимого), который мы хотим превратить в ссылку в элемент `<a>`. Этому элементу назначается атрибут `href` (который также известен как **гипертекстовая ссылка, якорь**), который будет содержать веб-адрес для перехода. Конечно, гиперссылка может обертываться и внедряться внутрь других элементов, например:

```
<p>Я создал ссылку на
  <a href="https://www.yandex.ru/">домашнюю страницу Яндекс</a>.
</p>
```

Это дает нам следующий результат:

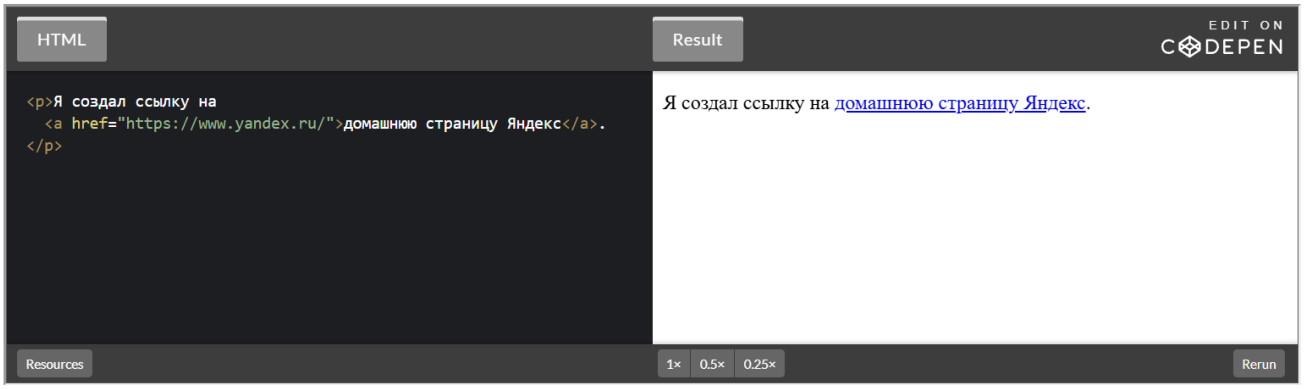


Рис. 5.5. Результат оформления гиперссылки

Установка целей для ссылок

Атрибут `target` сообщает браузеру, где открыть связанный документ. Существует четыре определенных цели, и имя каждой цели начинается с символа подчеркивания (`_`):

- `_blank` — открывает связанный документ в новом окне или вкладке.
- `_parent` — открывает связанный документ в родительском окне.
- `_self` — открывает связанный документ в том же окне или вкладке, что и исходный документ. Это значение по умолчанию, поэтому нет необходимости явно указывать это значение.
- `_top` — открывает связанный документ в полном окне браузера.

Посмотрим следующий пример, чтобы понять, как работает цель ссылки:

```
<a href="/about-us.php" target="_top">О нас</a>
```

```
<a href="https://www.yandex.ru/" target="_blank">Yandex</a>
<a href="images/sky.jpg" target="_parent"> <img src="sky-thumb.jpg" alt="Облачное небо"> </a>
```

Если веб-страница размещена внутри iframe, мы можем использовать `target="_top"` ссылки, чтобы выйти из `iframe` и отобразить целевую страницу в новом окне браузера.

Создание якорей закладок

Чтобы у пользователей была возможность переходить к определенному разделу веб-страницы, создаются привязки закладок. Закладки особенно полезны, если веб-страница очень длинная.

Создание закладок — это двухэтапный процесс:

- сначала добавляется атрибут `id` к элементу на который нужно перейти.
- затем, используется значение `id` этого атрибута, которому предшествует знак решетки (`#`), в качестве значения атрибута `href` тега `<a>`. Как показано в этом примере:

```
<a href="#sectionA">Переход к разделу A</a>
<h2 id="sectionA">Раздел A</h2>
<p>Лед тронулся, господа присяжные заседатели! Лед тронулся.</p>
```

Допускается выполнять переход к разделу другой веб-страницы, указав URL-адрес этой страницы вместе с якорем (т.е. `#elementId`) в атрибуте `href`, например, `<a href="mypage.html#topicA">Переход к разделу A</a>`.

Создание ссылок для скачивания

Создать **ссылку для скачивания файла** так же просто, как и текстовую ссылку. Указываем целевой URL-адрес файла, который мы хотим сделать доступным для загрузки. В следующем примере мы создали ссылки для загрузки файлов ZIP, PDF и JPG.

```
<a href="downloads/test.zip">Скачать Zip файл</a>
<a href="downloads/masters.pdf">Скачать PDF файл</a>
<a href="downloads/sample.jpg">Скачать файл изображения</a>
```

Когда происходит щелчок по ссылке, указывающей на файл PDF или файл изображения, файл не загружается напрямую на жесткий диск. Файл открывается в веб-браузере, а, в дальнейшем, его можно сохранить или скачать на жесткий диск.

Добавляем информацию через атрибут title

Атрибутом, который добавляется к ссылкам, является `title`. Он предназначен для хранения полезной информации о ссылке. Например, какую информацию содержит страница или пояснение о содержимом ресурса. Например:

```
<p>Я создал ссылку на
  <a href="https://www.yandex.ru/"
    title="Лучшее место для поиска дополнительной информации
    о сервисах Яндекс и о том, с чем можно работать">домашнюю страницу Яндекс
  </a>.
</p>
```

Вот что получилось (описание появится, если навести курсор на ссылку):

Описание из атрибута `title` отображается только при наведении курсора. Это означает, что полагание на клавиатурные элементы управления для навигации по веб-страницам, будут ограничивать полезность атрибута. Если информация заголовка действительно важна, то ее нужно представить соответствующим образом, например, поместив её в обычный текст.

Ссылки-блоки

Как упоминалось ранее, мы можем превратить любой элемент в ссылку, и даже блочный элемент. Если у нас есть изображение, которое нужно превратить в ссылку, то нужно просто поместить изображение между тегами `<a></a>`.

```
<a href="https://www.yandex.ru/">
  <img src="yandex-image.png" alt="логотип Яндекс со ссылкой на их домашнюю страницу">
</a>
```

Фрагменты документа

Можно ссылаться на определённую часть документа HTML (известную как **фрагмент документа**), а не только на верхнюю часть. Для этого нам сначала нужно назначить атрибут `id` элементу, с которым мы хотим связаться. Обычно имеет смысл ссылаться на определённый заголовок, поэтому это выглядит примерно так:

```
<h2 id="Почтовый_адрес">Почтовый адрес</h2>
```

Затем, чтобы связаться с этим конкретным `id`, мы должны включить его в конец URL-адреса, которому предшествует знак решётки, например:

```
<p>Хотите написать мне письмо? Используйте наш
```

```
    <a href="contacts.html#Почтовый_адрес">почтовый адрес</a>.
  </p>
```

Мы даже можете использовать ссылку на фрагмент документа отдельно для ссылки на **другую часть того же документа**:

```
  <p>
    <a href="#Почтовый_адрес">Почтовый адрес компании</a>
    можно найти в нижней части этой страницы.
  </p>
```

Абсолютные и относительные URL-адреса

Два понятия, с которыми вы столкнётесь в Интернете, — это **абсолютный URL** и **относительный URL:**

Абсолютный URL - *указывает на местоположение, определяемое его абсолютным местоположением в Интернете, включая протокол и доменное имя.*

Например, если страница `index.html` загружается в каталог, называемый `projects`, который находится внутри корня веб-сервера, а домен веб-сайта — `http://www.example.com`, страница будет доступна по адресу `http://www.example.com/projects/index.html` или даже просто `http://www.example.com/projects/`), так как большинство веб-серверов просто ищет целевую страницу, такую как `index.html`, для загрузки, если он не указан в URL-адресе.).

Абсолютный URL всегда будет указывать на одно и то же местоположение, независимо от того, где он используется.

Относительный URL - *указывает расположение относительно файла, с которого мы связываемся, и это больше похоже на случай, который мы рассматривали в предыдущем разделе.*

Для примера, если мы хотим указать со страницы `http://www.example.com/projects/index.html` на PDF файл, находящийся в той же директории, наш URL может быть просто названием файла — `project-brief.pdf` — никакой дополнительной информации не требуется. Если PDF расположен в поддиректории `pdfs` внутри каталога `projects`, относительная ссылка будет `pdfs/project-brief.pdf` (аналогичный абсолютный URL был бы `http://www.example.com/projects/pdfs/project-brief.pdf`.).

Относительный URL будет указывать на различные места, в зависимости от того, где находится файл, в котором он используется, — например, если мы переместим наш файл `index.html` из каталога `projects` в корневой каталог веб-сервера (верхний уровень, не в директорию), то относительный URL `pdfs/project-brief.pdf` будет вести на `http://www.example.com/pdfs/project-brief.pdf`, а не на `http://www.example.com/projects/pdfs/project-brief.pdf`.

5.5. Списки

Работа со списками HTML

Списки HTML используются для представления информации в виде перечислений в правильном, семантическом виде. В HTML существует три различных типа списков, и каждый из них имеет определенное назначение.

- **Неупорядоченный список** — используется для создания списка связанных элементов в произвольном порядке.
- **Упорядоченный список** — используется для создания списка связанных элементов в определенном порядке.
- **Список описаний** — используется для создания списка терминов и их описаний.

Внутри элемента списка размещается текст, изображения, ссылки, разрывы строк и т.д. Также весь список помещается внутри элемента списка, чтобы создать вложенный список.

1. Ненумерованные списки

Неупорядоченный список создается с использованием элемента `<ul>`, и каждый элемент списка начинается с элемента `<li>`. Элементы списка в ненумерованных списках отмечены маркерами. Вот пример:

```
<ul>
   <li>Шоколадный торт</li>
   <li>Торт Черный  лес</li>
   <li>Ананасовый торт</li>
</ul>
```

Вывод приведенного выше примера будет выглядеть примерно так:

- Шоколадный торт
- Торт Черный лес
- Ананасовый торт

Тип маркера изменяется через свойство CSS `list-style-type`. Следующее правило стиля изменяет тип маркера по умолчанию на квадрат:

```
ul {
    list-style-type: square;
}
```

2. Упорядоченные списки

Упорядоченный список создается с использованием элемента `<ol>`, и каждый элемент списка начинается с элемента `<li>`. Упорядоченные списки используются, когда важен порядок элементов списка. Элементы списка в упорядоченном списке отмечены цифрами. Например:

```
<ol>
    <li>Пристегнуть ремень безопасности</li>
    <li>Запустить двигатель</li>
    <li>Включить поворотник</li>
</ol>
```

Вывод приведенного выше примера будет выглядеть примерно так:

```
1. Пристегнуть ремень безопасности
2. Запустить двигатель
3. Включить поворотник
```

Нумерация элементов в упорядоченном списке обычно начинается с 1. Однако, если нужно изменить нумерацию, то используем атрибут `start`, как показано в следующем примере:

```
<ol start="10">
    <li>Смешать ингридиенты</li>
    <li>Запекать в духовке час</li>
    <li>Дать постоять десять минут</li>
</ol>
```

Вывод приведенного выше примера будет выглядеть примерно так:

```
1. Смешать ингредиенты
2. Запекать в духовке час
3. Дать постоять десять минут
```

Как и в случае с неупорядоченным списком, для изменения типа нумерации в упорядоченном списке используется свойство CSS `list-style-type`. Следующее правило стиля изменяет тип маркера на римские цифры.

```
ol {
    list-style-type: upper-roman;
}
```

Для изменения типа нумерации допускается использовать атрибут `type`, например type="I". Однако лучше избегать этого атрибута и применять свойство CSS `list-style-type`.

4. Список определений

На предыдущем шаге мы рассмотрели, как пометить привычные типы списков в HTML, поговорим подробнее о третьем типе списка, с которым мы можем столкнуться, — **список определения**. Цель этих списков состоит в том, чтобы пометить набор элементов и их связанных описаний, таких как термины и определения или вопросы и ответы.

Давайте рассмотрим пример набора терминов и определений:

***Солилоквий** - драматическая речь, в которой персонаж разговаривает сам с собой, передавая свои ощущения и мысли публике (но не другим персонажам).*
***Монолог** - драматическая речь, в которой персонаж передаёт свои мысли публике и некоторым персонажам.*
***Ремарка** - в драме: речь персонажа, при которой делается замечание с юмористическим или драматическим эффектом;*
чаще всего это чувства, мысли или предпосылки к чему-либо.

В списках описания используется иная оболочка, чем в других типах списков — `<dl>` (от *description list*); кроме того, каждый термин завёрнут в элемент `<dt>` (*description term* — термин для описания) и каждое определение завёрнуто в элемент `<dd>` (*description definition* — описание определения). Сделаем разметку примера:

```
<dl>
  <dt>Солилоквий</dt>
  <dd>Драматическая речь, в которой персонаж разговаривает сам с собой, передавая свои ощущения и мысли публике (но не другим персонажам).</dd>
  <dt>Монолог</dt>
  <dd>Драматическая речь, в которой персонаж передаёт свои мысли публике и некоторым персонажам.</dd>
  <dt>Ремарка</dt>
  <dd>В драме: речь персонажа, при которой делается замечание с юмористическим или драматическим эффектом; чаще всего это чувства, мысли или предпосылки к чему-либо.</dd>
</dl>
```

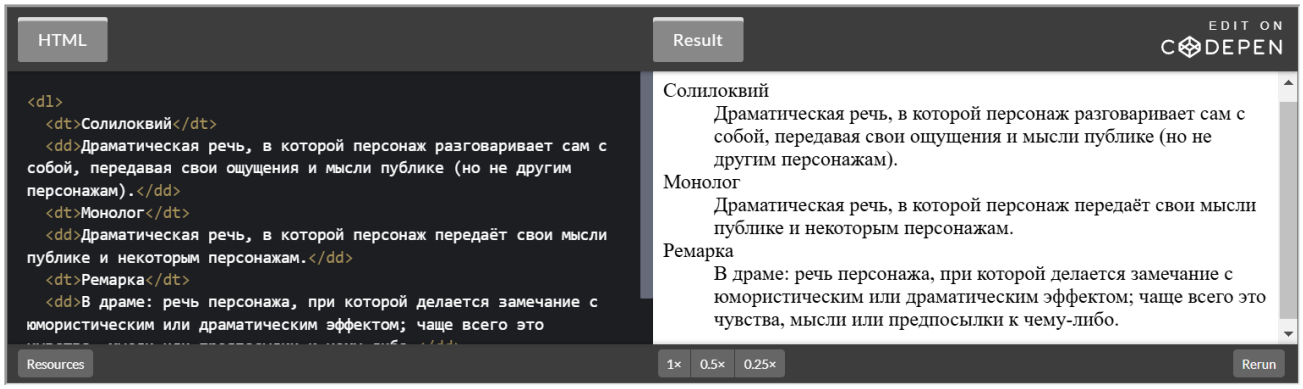


Рис. 5.6. Список определений

В стилях браузера по умолчанию будут отображаться списки описания с некоторыми отступами от терминов.

Цитаты

HTML также имеет функции, доступные для маркировки цитат; какой элемент мы используем, зависит от того, маркируем ли мы **блочную** или **строчную цитату**.

1. Блочные цитаты

Если часть содержимого уровня блока (будь, то абзац, несколько абзацев, список и т. д.) цитируется из другого источника, то это нужно обернуть элементом `<blockquote>`. Дополнительно, потребуется указать URL-адрес, указывающий на источник цитаты, внутри атрибута `cite`. Например, следующая разметка берётся из страницы элемента `<blockquote>`:

```
<p><strong>HTML-элемент<code>&amp;lt;blockquote&amp;gt;</code></strong> (от англ. <em>HTML Block
Quotation Element</em>) указывает на то, что заключённый в нём текст является развёрнутой цитатой.</p>
```

Чтобы превратить её в блочную цитату, мы просто делаем следующее:

```
<blockquote cite="https://www.yandex.ru/docs/Element/blockquote">
  <p><strong>HTML-элемент<code>&amp;lt;blockquote&amp;gt;</code></strong> (от англ. <em>HTML Block Quotation Element</em>) указывает на то, что заключённый в нём текст является развёрнутой цитатой.</p>
</blockquote>
```

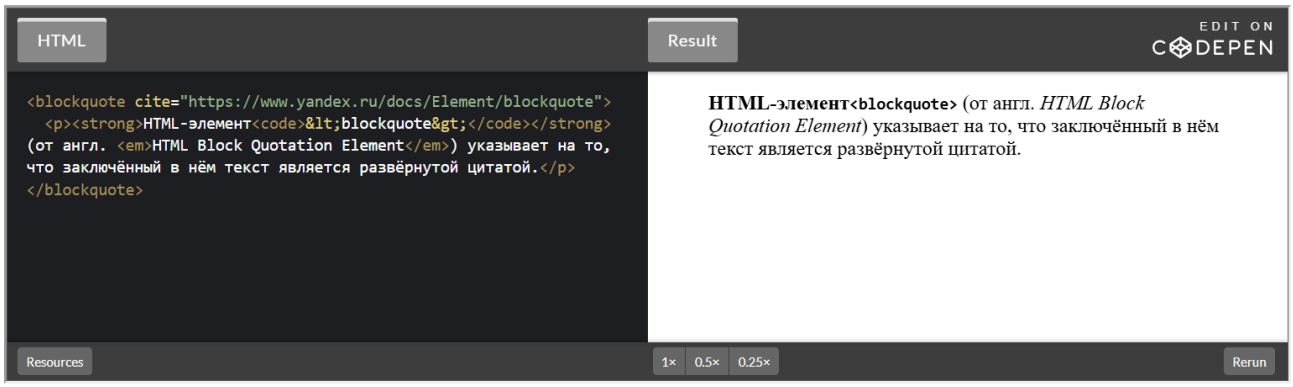


Рис. 5.7. Блочные цитаты

Стиль браузера по умолчанию будет отображать это как **абзац с отступом**, как индикатор того, что это цитата; абзац над цитатой призван продемонстрировать это.

2. Строчные цитаты

Строчные цитаты работают точно так же, за исключением того, что они используют элемент `<q>`. Например, следующий кусочек разметки содержит цитату из страницы `<q>`:

```
<p>Элемент цитирования — <code>&lt;q&gt;</code> — <q
cite="https://www.yandex.ru/docs/Element/q">предназначен
для коротких цитат, не требующих прерывания абзаца</q>.</p>
```

Стиль браузера по умолчанию будет отображать это как **обычный текст**, заключённый в кавычки для обозначения цитаты, например:

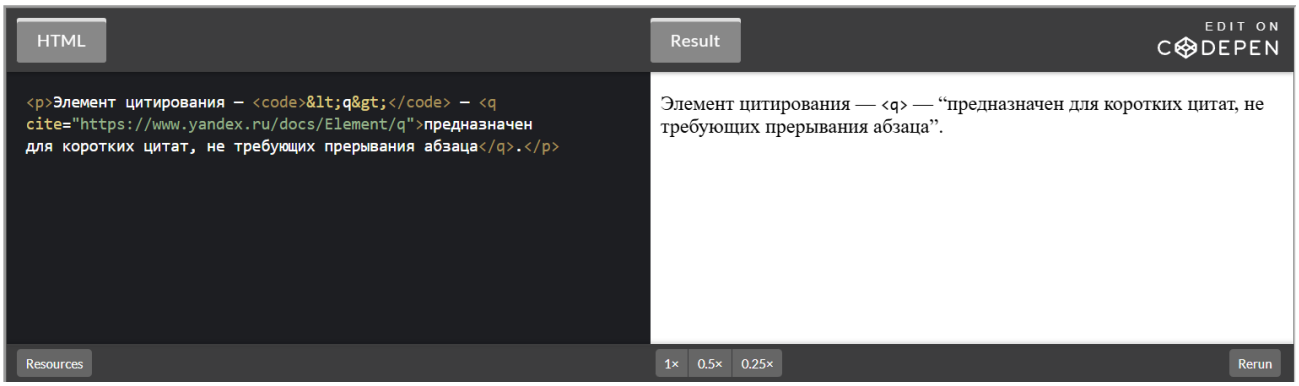


Рис. 5.8. Строчные цитаты

3. Цитирование

Невозможно заставить браузер отображать содержимое атрибута `<cite>` без написания собственного решения с использованием JavaScript или CSS.

Чтобы источник цитирования был доступен на странице, лучший способ его разметки - поместить элемент `<cite>` рядом с элементом цитаты (или внутри него). Это действительно будет означать то, что имя источника цитаты — то есть имя книги или имя человека, которое произвело цитату, — будет включено в текст.

Нет причин, по которым мы не могли бы связать текст внутри `<cite>` с источником цитаты:

```
<p>Как указано в статье о <a href="https://www.yandex.ru/docs/Element/blockquote">
<cite>блочных цитатах</cite></a>:
</p>

<blockquote cite="https://www.yandex.ru/docs/Element/blockquote">
 <p><strong>HTML-элемент<code>&lt;blockquote&gt;</code></strong> (от англ.
<em>HTML Block
 Quotation Element</em>) указывает на то, что заключённый в нем текст является
развёрнутой цитатой.</p>
</blockquote>

<p>Элемент цитирования — <code>&lt;q&gt;</code> — <q
cite="https://www.yandex.ru/docs/Element/q">предназначен
для коротких цитат, не требующих прерывания абзаца</q>. -- <a
href="https://www.yandex.ru/docs/Element/q">
<cite>Строчные цитаты</cite></a>.</p>
```

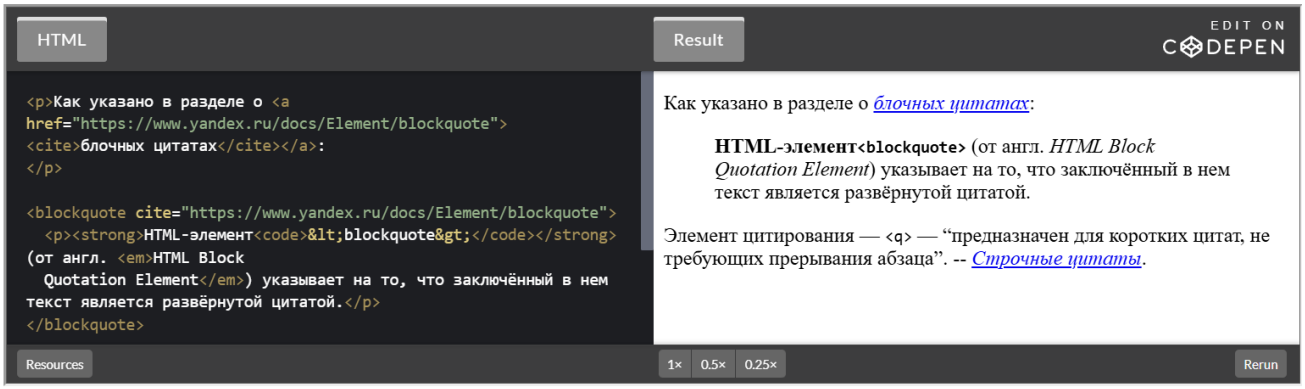


Рис. 5.9. Цитирование

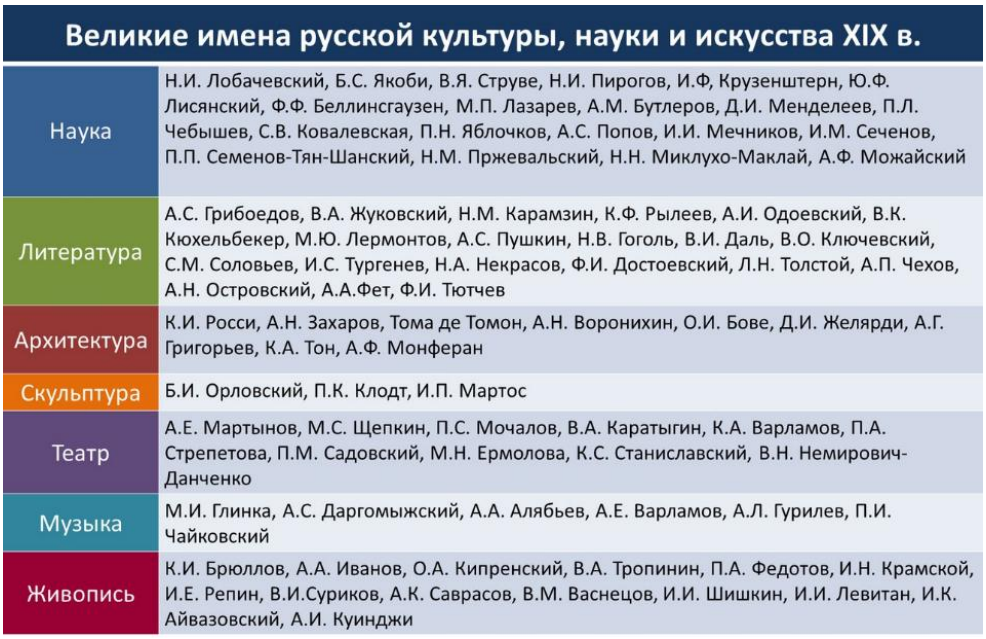
По умолчанию цитаты стилизованы **курсивом**.

5.6. Таблицы

Основы формирования таблиц

Таблица *— это структурированный набор данных, состоящий из строк и столбцов.*

Таблицы позволяют быстро и легко посмотреть значения, показывающие некоторую взаимосвязь между различными типами данных, например - человек и его возраст, расписание, перечень покупок и их стоимость и т.п. Люди постоянно используют таблицы, причём уже давно.



Великие имена русской культуры, науки и искусства XIX в.	
Наука	Н.И. Лобачевский, Б.С. Якоби, В.Я. Струве, Н.И. Пирогов, И.Ф. Крузенштерн, Ю.Ф. Лисянский, Ф.Ф. Беллинсгаузен, М.П. Лазарев, А.М. Бутлеров, Д.И. Менделеев, П.Л. Чебышев, С.В. Ковалевская, П.Н. Яблочков, А.С. Попов, И.И. Мечников, И.М. Сеченов, П.П. Семенов-Тян-Шанский, Н.М. Пржевальский, Н.Н. Миклухо-Маклай, А.Ф. Можайский
Литература	А.С. Грибоедов, В.А. Жуковский, Н.М. Карамзин, К.Ф. Рылеев, А.И. Одоевский, В.К. Кюхельбекер, М.Ю. Лермонтов, А.С. Пушкин, Н.В. Гоголь, В.И. Даль, В.О. Ключевский, С.М. Соловьев, И.С. Тургенев, Н.А. Некрасов, Ф.И. Достоевский, Л.Н. Толстой, А.П. Чехов, А.Н. Островский, А.А.Фет, Ф.И. Тютчев
Архитектура	К.И. Росси, А.Н. Захаров, Тома де Томон, А.Н. Воронихин, О.И. Бове, Д.И. Желярди, А.Г. Григорьев, К.А. Тон, А.Ф. Монферан
Скульптура	Б.И. Орловский, П.К. Клодт, И.П. Мартос
Театр	А.Е. Мартынов, М.С. Щепкин, П.С. Мочалов, В.А. Каратыгин, К.А. Варламов, П.А. Стрепетова, П.М. Садовский, М.Н. Ермолова, К.С. Станиславский, В.Н. Немирович-Данченко
Музыка	М.И. Глинка, А.С. Даргомыжский, А.А. Алябьев, А.Е. Варламов, А.Л. Гурилев, П.И. Чайковский
Живопись	К.И. Брюллов, А.А. Иванов, О.А. Кипренский, В.А. Тропинин, П.А. Федотов, И.Н. Крамской, И.Е. Репин, В.И.Суриков, А.К. Саврасов, В.М. Васнецов, И.И. Шишкин, И.И. Левитан, И.К. Айвазовский, А.И. Куинджи

Рис. 5.10. Представление таблицы

Так что не удивительно, что создатели HTML включили в него средства для структурирования и представления табличных данных в сети.

В чем преимущества таблицы?

Смысл таблицы в том, что она **жёсткая**. Информацию легко интерпретировать, визуально сопоставляя заголовки строк и столбцов. Например, посмотрите на приведённую ниже таблицу и найдите единственное **личное местоимение**, используемое **в третьем лице**, с полом ♀, выступающее в качестве **объекта** в предложении. Ответ можно найти, сопоставив соответствующие заголовки столбцов и строк.

Таблица 5.1. Часто используемые местоимения

		Субъект	Объект
Единствен. числ.	**1-е лицо**	Я	меня
	2-е лицо	ты	тебя

	3-е лицо	♂	он	его
		♀	она	её
		о	оно	его
Множ.числ.	**1-е лицо**		мы	нас
	2-е лицо		вы	вас
	2-е лицо		они	их

Если правильно представить таблицу HTML, интерпретировать её данные смогут даже люди, имеющие проблемы со зрением.

Оформление таблиц

Чтобы эффективно представлять таблицы в веб, необходимо придать им хорошую структуру в HTML и применить к ним **таблицы стилей** (**CSS**). HTML-таблицы следует использовать для **табличных данных** — это то, для чего они предназначены. К сожалению, многие используют таблицы HTML для оформления веб-страниц, а это неправильно.

Есть много ситуаций, когда таблицы могут пригодиться при разработке проектов:

- Когда мы хотим сравнить и сопоставить данные с общими характеристиками, такими как различия между A и B или результаты команды X с результатами команды Y.
- Если мы хотим дать обзор числовых данных. Хорошим примером этого является, попытка представить оценки студентов или даже баллы команд.
- Таблица может помочь читателям быстро найти конкретную информацию, изложенную в понятной форме. Например, поиск в длинном списке имен может облегчаться использованием таблицы с разделением списка на подпункты.

Чтобы создать таблицу в HTML, вам нужно будет использовать теги. Наиболее важным из них является тег `<table>`, который является основным **контейнером таблицы**. Он показывает, где начинается и где заканчивается таблица.

Другие теги описания элементов таблиц:

- `tr>` - представляет строки.
- `<td>` - используется для создания ячеек данных.
- `<th>` - используется для добавления заголовков таблицы.
- `<caption>` - используется для вставки титров.

- `<thead>` - добавляет в таблицу отдельный заголовок.
- `<tbody>` - показывает основную часть таблицы.
- `<tfoot>` - создает отдельный нижний колонтитул для таблицы.

Создание простейшей таблицы

Для создания таблицы нам нужно реализовать следующий программный код:

```
<table>
 <tr>
  <td>Ячейка 1</td>
  <td>Ячейка 2</td>
  <td>Ячейка 3</td>
 </tr>
 <tr>
  <td>Ячейка 4</td>
  <td>Ячейка 5</td>
  <td>Ячейка 6</td>
 </tr>
</table>
```

Вот как это выглядит в браузере:

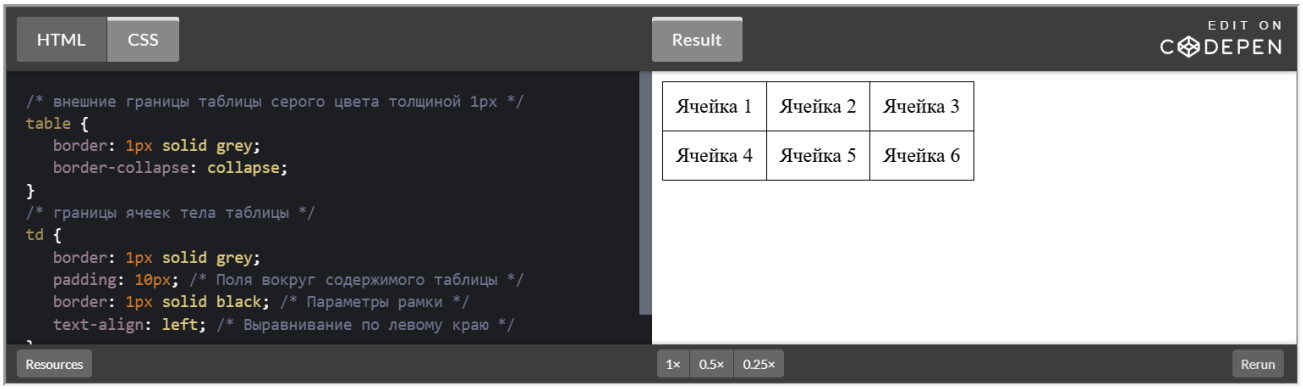


Рис. 5.11. Простейшая таблица

Теперь, когда вы понимаете, что такое таблица HTML и как ее создать, давайте продолжим и посмотрим, как мы можем использовать дополнительные теги для создания отдельных элементов таблиц.

Добавление заголовков <th>

Теперь обратимся к **табличным заголовкам** — особым ячейкам, которые идут вначале строки или столбца и определяют тип данных, которые содержит данная строка или столбец (как "Имя", "Возраст" и т.д.).

Чтобы показать, для чего они нужны, возьмём следующий пример. Сначала исходный код:

```
<table>
 <tr>
  <th>Имя</th>
  <th>Фамилия</th>
  <th>Email</th>
 </tr>
 <tr>
  <td>Семен</td>
  <td>Коршенков</td>
  <td>korshenkov@yandex.ru</td>
 </tr>
 <tr>
  <td>Дмитрий</td>
  <td>Вашерский</td>
  <td>vashersky@yandex.ru</td>
 </tr>
</table>
```

Теперь как выглядит таблица:

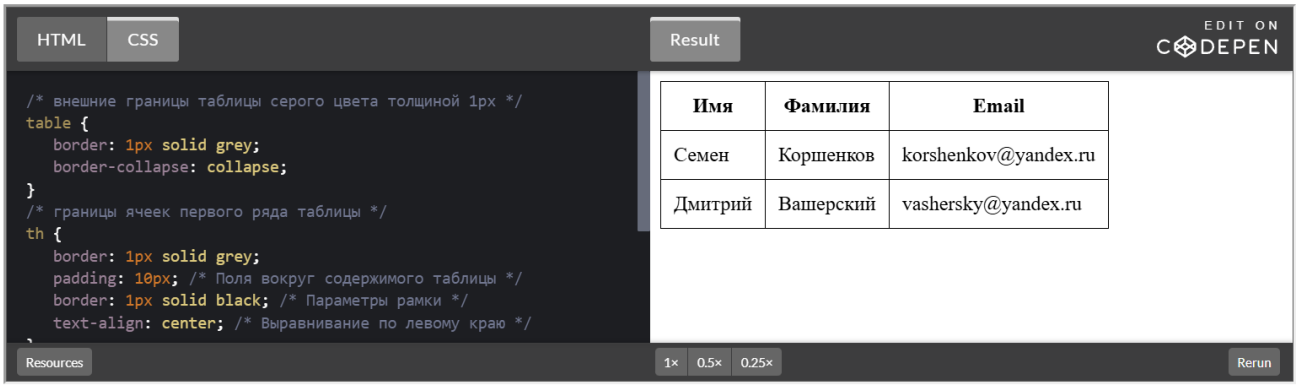


Рис. 5.12. Таблица с заголовками

Проблема в том, что, хотя мы и можем представить, о чем идёт речь, ссылаться на эти данные не так легко, как хотелось бы. Лучше, чтобы строка и столбец с заголовками как-то выделялись.

Мы уже частично ответили на этот вопрос — когда заголовки выделяются, легче искать данные и таблица выглядит лучше.

*По умолчанию к **заголовкам таблицы** применяется определённый стиль — они выделены жирным шрифтом и выровнены по центру, даже если вы не задавали для них стиль специально.*

Заголовки дают дополнительное преимущество — вместе с атрибутом `scope`, они помогают улучшить связь каждого заголовка со всеми данными строки или столбца одновременно, что довольно полезно.

Добавление заголовка к таблице

Основное назначение добавления подписи к таблице — предоставить описание данных, представленных в таблице. Заголовок можно разместить вверху или внизу таблицы, и по умолчанию он всегда будет по центру.

Чтобы вставить подпись в таблицу, используйте тег `<caption>`.

```
<table>
 <caption>Адресная книга</caption>
 <tr>
  <th>Имя</th>
  <th>Фамилия</th>
  <th>Email</th>
 </tr>
 <tr>
  <td>Семен</td>
  <td>Коршенков</td>
  <td>korshenkov@yandex.ru</td>
 </tr>
 <tr>
  <td>Дмитрий</td>
  <td>Вашерский</td>
  <td>vashersky@yandex.ru</td>
 </tr>
</table>
```

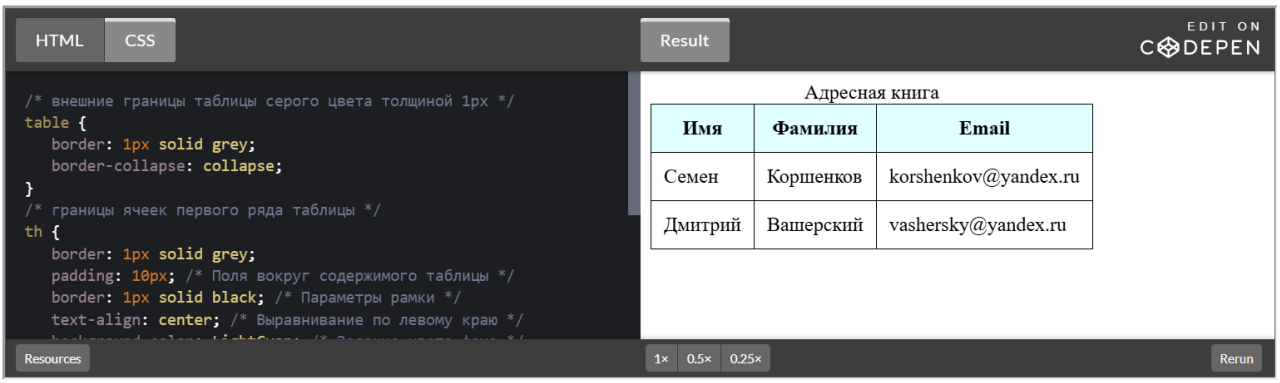


Рис. 5.12. Таблица с заголовками (продолжение)

Группирование строк и столбцов таблицы

Элемент `<colgroup>` создает структурную группу столбцов, выделяя логически однородные ячейки. Группирует один или более столбцов для единого форматирования, позволяя применить стили к столбцам вместо того, чтобы повторять стили для каждой ячейки и для каждой строки.

Добавляется непосредственно после тегов `<table>` и/или `<caption>`. Элемент `<col>` формирует группы столбцов, которые делят таблицу на разделы, не относящиеся к общей структуре, т.е. не содержащие информацию одного типа. Позволяет задавать свойства столбцов для каждого столбца в пределах элемента `<colgroup>`.

С помощью атрибута `style` можно изменить основной цвет фона ячеек. Для элемента `<col>` доступен атрибут `span`, задающий количество столбцов для объединения. Рассмотрим пример кода:

```
<table>
 <colgroup>
  <col span="2" style="background:Khaki"><!-- С помощью этой конструкции задаем цвет фона для первых двух столбцов таблицы-->
  <col style="background-color:LightCyan"><!-- Задаем цвет фона для следующего (одного) столбца таблицы-->
 </colgroup>
 <tr>
  <th>№ п/п</th>
  <th>Наименование</th>
  <th>Цена, руб.</th>
 </tr>
 <tr>
  <td>1</td>
  <td>Карандаш цветной</td>
  <td>20,00</td>
 </tr>
 <tr>
  <td>2</td>
  <td>Линейка 20 см</td>
  <td>30,00</td>
 </tr>
</table>
```

Рис. 5.13. Группировка строк и столбцов в таблице

Группировка разделов таблицы

Элемент <thead> создает группу заголовков для строк таблицы с целью задания единого оформления. Используется в сочетании с элементами <tbody> и <tfoot> для указания каждой части таблицы.

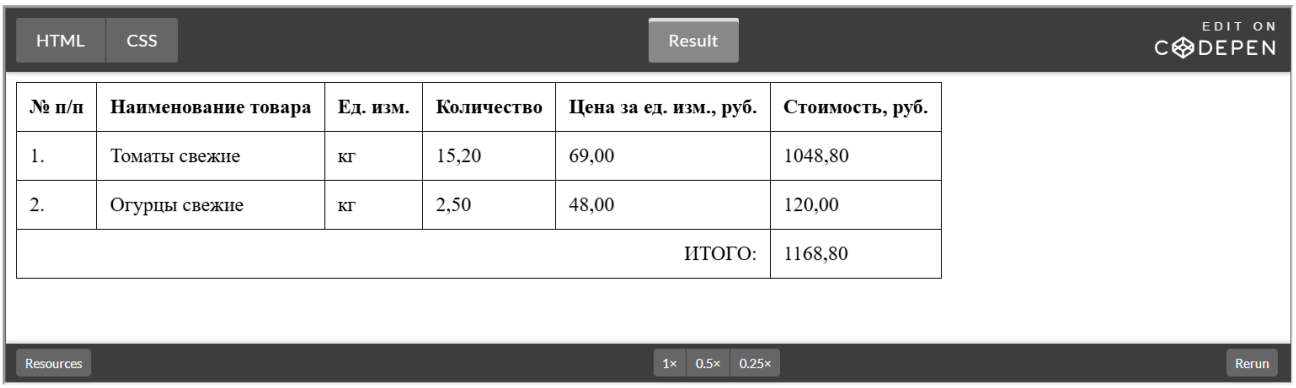
Элемент должен быть использован в следующем порядке: как дочерний элемент <table>, после <caption> и <colgroup>, и перед <tbody>, <tfoot> и <tr> элементами. В пределах одной таблицы можно использовать один раз.

Элемент <tbody> группирует основное содержимое таблицы. Используется в сочетании с элементами <thead> и <tfoot>.

Элемент <tfoot> создает группу строк для представления информации о суммах или итогах, расположенную в нижней части таблицы. Используется в таблице один раз. Располагается после элемента <thead>, перед элементами <tbody> и <tr>.

```
<table>
 <thead>
  <tr>
    <th>№ п/п</th>
    <th>Наименование товара</th>
     <th>Ед. изм.</th>
     <th>Количество</th>
     <th>Цена за ед. изм., руб.</th>
     <th>Стоимость, руб.</th>
  </tr>
 </thead>
<tfoot>
 <tr>
   <td colspan="5" style="text-align:right">ИТОГО:</td><td>1168,80</td>
```

```
 </tr>
</tfoot>
<tbody>
 <tr>
  <td>1.</td>
  <td>Томаты свежие</td><td>кг</td><td>15,20</td><td>69,00</td><td>1048,80</td>
 </tr>
 <tr>
  <td>2.</td>
  <td>Огурцы свежие</td><td>кг</td><td>2,50</td><td>48,00</td><td>120,00</td>
 </tr>
</tbody>
</table>
```



№ п/п	Наименование товара	Ед. изм.	Количество	Цена за ед. изм., руб.	Стоимость, руб.
1.	Томаты свежие	кг	15,20	69,00	1048,80
2.	Огурцы свежие	кг	2,50	48,00	120,00
				ИТОГО:	1168,80

Рис. 5.14. Группировка разделов таблицы

Такая группировка строк была заложена в стандарте в расчете на то, что обозреватели при отображении длинных таблиц обеспечат прокрутку строк данных при сохранении надзаголовка и подзаголовка неподвижными, а при их выводе на принтер смогут использовать надзаголовок и подзаголовок в качестве колонтитулов страницы. Однако, современные обозреватели этого не делают и либо просто отображают <thead> и <tfoot> как строки данных, либо, в лучшем случае, просто помещают соответствующие строки в начало и конец таблицы.

Объединение ячеек таблицы

Атрибуты `colspan` и `rowspan` объединяют ячейки таблицы. Атрибут `colspan` задает количество ячеек, объединенных по горизонтали, а `rowspan` — по вертикали.

```
<table>
 <tr>
  <th>№ п/п</th>
  <th>Наименование товара</th>
   <th>Ед. изм.</th>
```

```
    <th>Количество</th>
    <th>Цена за ед. изм., руб.</th>
    <th>Стоимость, руб.</th>
  </tr>
  <tr>
    <td>1.</td>
    <td>Томаты                                                                свежие</td><td>кг</td><td>15,20</td><td>69,00</td><td>1048,80</td>
  </tr>
  <tr>
    <td>2.</td>
    <td>Огурцы свежие</td><td>кг</td><td>2,50</td><td>48,00</td><td>120,00</td>
  </tr>
  <tr>
    <td colspan="5" style="text-align:right">ИТОГО:</td><td>1168,80</td><!-- Задаем количество ячеек по горизонтали для объединения-->
  </tr>
</table>
```

Таблица 5.2 Атрибуты элементов таблицы

Атрибут	Описание, принимаемое значение
colspan	Количество ячеек в строке для объединения по горизонтали. `<td colspan="3">` Возможные значения: число от 1 до 999.

headers	Задает список ячеек заголовка, содержащих информацию о заголовке текущей ячейки данных. Предназначен для речевых браузеров.
	`<th id="идентификатор">...</th>`
	`<th headers="идентификатор">...</th>`
	Принимаемые значения: список имен ячеек, разделенных пробелами; эти имена должны быть присвоены ячейкам через их атрибут id.

rowspan	Количество ячеек в столбце для объединения по вертикали. `<td rowspan="2">` Возможные значения: число от 1 до 999.
span	Количество колонок, объединяемых для задания единого стиля, по умолчанию равно 1. `<col span="2">` Принимаемые значения: любое целое положительное число.

Добавление структуры с помощью <thead>, <tfoot> и <tbody>

Когда таблицы становятся более сложными по структуре полезно дать им более **структурированное определение**. Отличный способ сделать это используя `<thead>`, `<tfoot>` и `<tbody>`, которые позволяют вам разметить **header**, **footer** и **body** секции таблицы.

Эти элементы не создают дополнительной доступности для пользователей и не приводят к какому-то визуальному улучшению при их использовании. Зато они очень полезны при стилизации и добавления разметки CSS к таблице.

В случае длинной таблицы можно сделать **header** и **footer** таблицы, повторяемые на каждой печатной странице, или сделать **body** таблицы отображаемым на одной странице с доступом ко всему содержимому контенту (прокручивая вверх и вниз).

Использование:

- Элементом `<thead>` нужно обернуть часть таблицы, которая относится к заголовку — обычно это первая строка, содержащая заголовки колонок, но это не обязательно всегда такой случай. Если мы используем элемент `<col>` / `<colgroup>`, тогда заголовок должен находиться ниже его.
- Элементом `<tfoot>` нужно обернуть ту часть, которая относится к footer таблицы — например, это может быть последняя строка, в которой отражаются суммы по столбцам таблицы. Мы можем включить сюда footer таблицы, как и следовало ожидать, или чуть ниже заголовка таблицы (браузер всё равно отобразит его внизу таблицы).
- Элементом `<tbody>` необходимо обернуть остальную часть содержимого таблицы, которая не находится в header или footer таблицы. Этот блок располагают ниже заголовка таблицы или иногда footer таблицы, зависит от того какую структуру решено использовать.

`<tbody>` всегда включён в каждой таблице, неявно если не укажете его в коде.

Вложенные таблицы

В одну таблицу вкладывать другую таблицу возможно, если вы используете полную структуру включая элемент `<table>`. Это как правило не рекомендуется, так как делает разметку более запутанной и менее доступной, так в большинстве случаев мы можем просто вставить дополнительные ячейки/строки/столбцы в существующую таблицу.

Однако, иногда это необходимо, например, если мы хотим легко импортировать контент из других источников. Разметка простой вложенной таблицы:

```
<table>
```

```
  <tbody>
    <tr>
      <th>Заголовок_1</th>
      <th>&amp;nbsp;Заголовок_2&amp;nbsp;</th>
      <th>&amp;nbsp;Заголовок _3&amp;nbsp;</th>
    </tr>
    <tr>
      <td>
      <table>
        <tbody>
          <tr>
            <td>ячейка_1</td>
            <td>ячейка_2</td>
            <td>ячейка_3</td>
          </tr>
        </tbody>
      </table>
      </td>
      <td>ячейка_2</td>
      <td>ячейка_3</td>
    </tr>
    <tr>
      <td>ячейка_4</td>
      <td>ячейка_5</td>
      <td>ячейка_6</td>
    </tr>
  </tbody>
</table>
```

Результат которого выглядит примерно так:

Заголовок_1	**Заголовок_2**	**Заголовок _3**
ячейка_1 | ячейка_2 | ячейка_3	ячейка_2	ячейка_3
ячейка_4	ячейка_5	ячейка_6

Атрибут scope

Атрибут `scope` может быть добавлен к элементу `<th>` и он сообщает какие ячейки точно являются заголовками — например, заголовок строки, в которой он находится, или столбца. Например:

```
<thead>
 <tr>
  <th scope="col">Purchase</th>
  <th scope="col">Location</th>
  <th scope="col">Date</th>
  <th scope="col">Evaluation</th>
  <th scope="col">Cost (€)</th>
 </tr>
</thead>
```

И у каждой строки может быть определён заголовок, как здесь (если мы добавили заголовки строк и заголовки столбцов):

```
<tr>
 <th scope="row">Haircut</th>
 <td>Hairdresser</td>
 <td>12/09</td>
 <td>Great idea</td>
 <td>30</td>
</tr>
```

Это дает возможность распознать структурированную разметку и дать пользователям прочитать весь столбец или строку целиком. Атрибут `scope` имеет ещё два возможных значения — `colgroup` и `rowgroup`. Они используются для заголовков, которые располагаются вверху ваших столбцов или строк.

Атрибуты id и headers

Альтернатива атрибута `scope` это использование атрибутов `id` и `headers` для создания ассоциации между заголовками и ячейками. Этот выглядит следующим образом:

1. Мы устанавливаем уникальный `id` для каждого элемента `<th>`.
2. Мы устанавливаем атрибут `headers` для каждого элемента `<td>`. Каждый атрибут `headers` должен содержать список всех `id` , разделённый пробелами, ко всем элементам `<th>`, которые действуют как заголовок для этой ячейки.

Это обеспечивает явное определение позиции для каждой ячейки вашей HTML таблицы, определяет заголовки столбцов и строк таблицы. Для того чтобы это работало реально хорошо таблице нужно определить и заголовки столбцов, и заголовки строк. Вернёмся к нашему примеру с расчётом затрат, его можно переписать следующим образом:

```
<thead>
 <tr>
  <th id="purchase">Purchase</th>
  <th id="location">Location</th>
  <th id="date">Date</th>
  <th id="evaluation">Evaluation</th>
  <th id="cost">Cost (€)</th>
 </tr>
</thead>
<tbody>
<tr>
 <th id="haircut">Haircut</th>
 <td headers="location haircut">Hairdresser</td>
 <td headers="date haircut">12/09</td>
 <td headers="evaluation haircut">Great idea</td>
 <td headers="cost haircut">30</td>
</tr>

 ...

</tbody>
```

Этот метод создания очень точного определения ассоциаций между заголовками и данными в ячейках, но использует гораздо больше разметки и оставляет обширное пространство для ошибок. Атрибута `scope` *обычно достаточно для большинства таблиц.*

Задания к практической работе

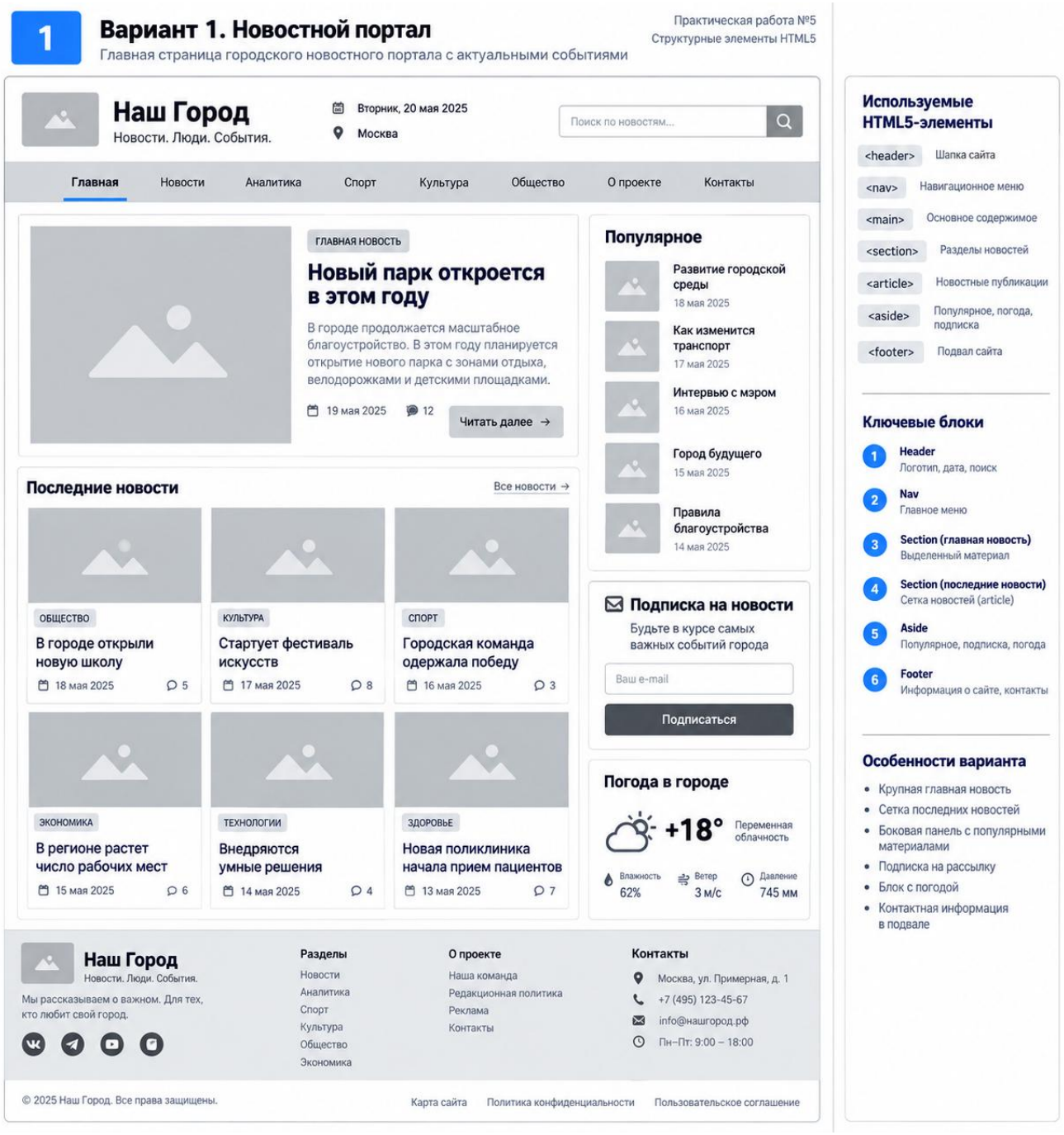
Задание 1. Определиться с номером варианта

Выбрать один вариант из 15 предложенных. Изучить соответствующий визуальный концепт. Определить смысловые области страницы и составить логическую HTML5-структуру.

Варианты индивидуальных заданий

Каждый вариант содержит предметную область, обязательные смысловые блоки, рекомендуемые HTML5-элементы и визуальные требования.

Вариант 1. Новостной портал



Разработать веб-страницу новостного портала с использованием **HTML5 и CSS3**.

Основная структура страницы:

- **Header** — логотип/название новостного портала.
- **Nav** — меню с категориями новостей.
- **Main** — основная область страницы.

- **Главная новость** — крупный блок с изображением, заголовком и кратким текстом.
- **Лента публикаций** — несколько новостей в виде отдельных блоков article, каждая с заголовком, изображением и кратким описанием.
- **Популярное** — список популярных новостей в боковой области aside.
- **Подписка** — блок для подписки на новости.
- **Footer** — нижняя часть страницы с дополнительной информацией.

Требования:

- Использовать семантические элементы header, nav, main, section, article, aside, footer.
- Оформить страницу с помощью CSS3.
- Для новостей использовать изображения.
- Воспроизвести расположение основных блоков новостного портала согласно варианту.
- **Без адаптивности, эффектов и JavaScript.**

Вариант 2. Афиша мероприятий

Разработать веб-страницу афиши мероприятий с использованием **HTML5 и CSS3**.

Основная структура страницы:

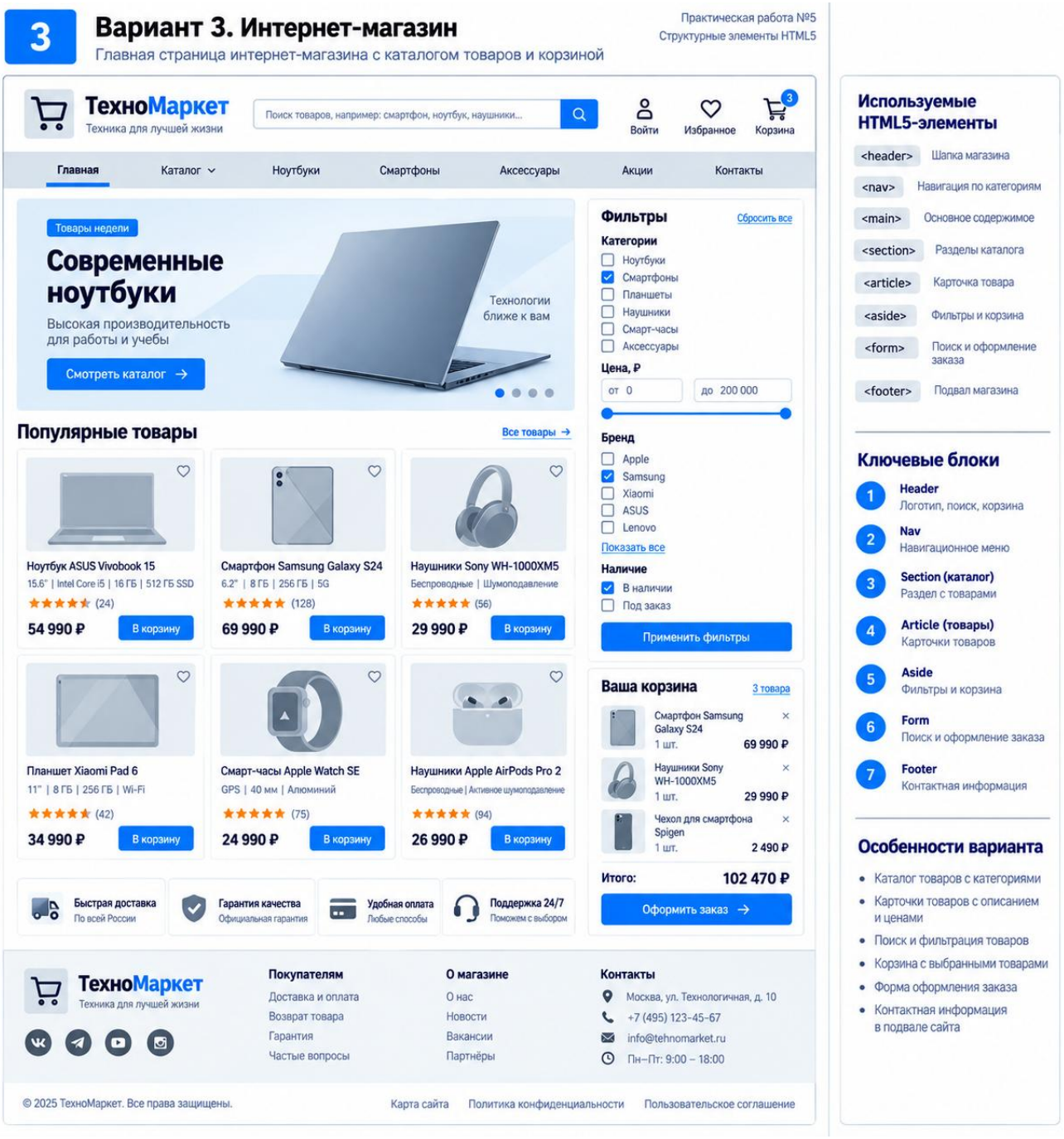
- **Header** — название афиши и основная информация.
- **Nav** — меню с разделами или категориями мероприятий.
- **Main** — основная область страницы.
- **Мероприятия** — список мероприятий с названием, датой, временем, местом и изображением.
- **Описание мероприятия** — краткая информация о выбранных мероприятиях.

- **Aside** — дополнительная информация или категории мероприятий.
- **Footer** — контактная и служебная информация.

Требования:

- Использовать семантические элементы header, nav, main, section, article, aside, footer.
- Оформить страницу с помощью CSS3.
- Для мероприятий использовать изображения.
- Указать **название, дату, время и место проведения**.
- Воспроизвести расположение основных блоков согласно варианту.
- **Без адаптивности, эффектов и JavaScript.**

Вариант 3. Интернет-магазин



Разработать веб-страницу интернет-магазина с использованием **HTML5 и CSS3**.

Основная структура страницы:

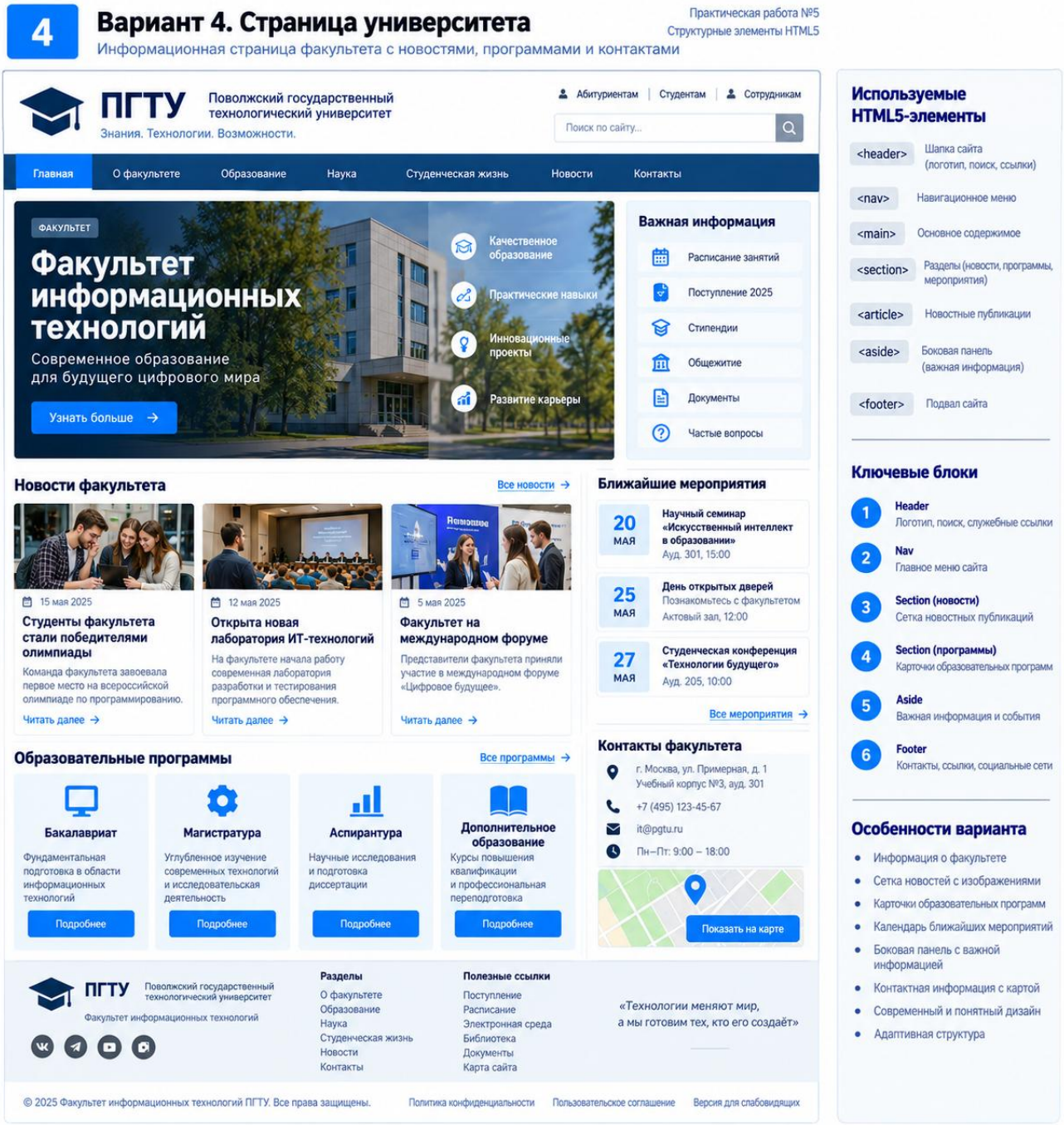
- **Header** — название магазина и основная информация.
- **Nav** — меню с категориями товаров.
- **Main** — основная область страницы.
- **Каталог товаров** — список товаров в виде отдельных карточек.
- **Карточка товара** — изображение, название, краткое описание и цена.
- **Корзина** — информация о выбранных товарах.

- **Aside** — дополнительные категории или фильтры товаров.
- **Footer** — контактная и служебная информация.

Требования:

- Использовать семантические элементы header, nav, main, section, article, aside, footer.
- Оформить страницу с помощью CSS3.
- Для товаров использовать изображения.
- Для каждого товара указать **название и цену**.
- Воспроизвести расположение основных блоков согласно варианту.
- **Без адаптивности, эффектов и JavaScript.**

Вариант 4. Страница университета



Разработать веб-страницу университета с использованием **HTML5 и CSS3**.

Основная структура страницы:

- **Header** — название и логотип университета.
- **Nav** — основное меню сайта.
- **Main** — основное содержимое страницы.
- **Информация об университете** — краткая информация и основные направления.
- **Факультеты / направления** — список основных подразделений.

- **Дополнительная информация** — новости или полезные ссылки.
- **Footer** — контактная информация и служебные ссылки.

Требования:

- Использовать семантические элементы header, nav, main, section, article, aside, footer.
- Оформить страницу с помощью **CSS3**.
- Использовать изображения и текстовый контент.
- Воспроизвести основную композицию макета.
- **Без адаптивности, эффектов и JavaScript.**

Вариант 5. Страница курса

Разработать веб-страницу курса с использованием **HTML5 и CSS3**.

Основная структура страницы:

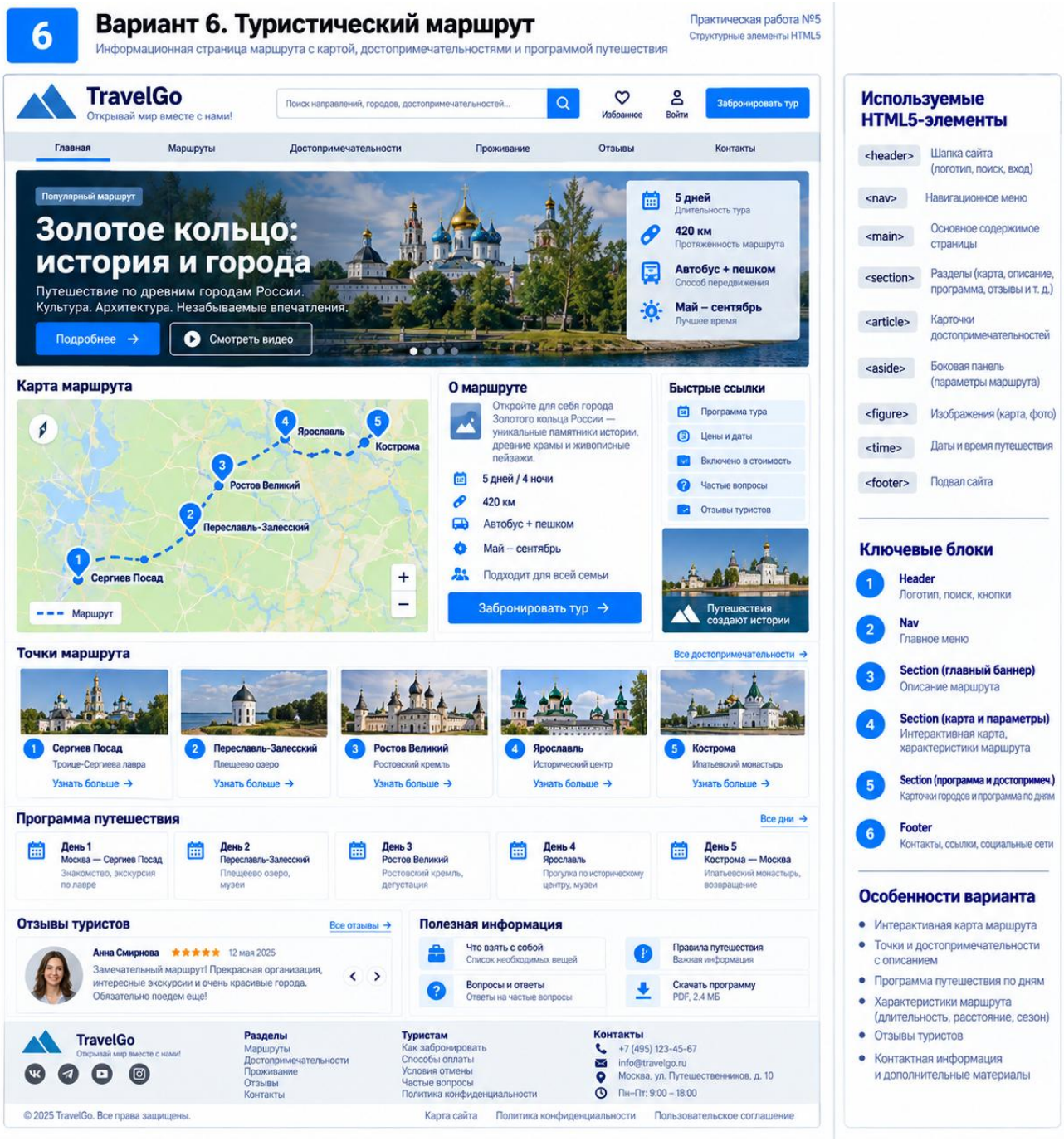
- **Header** — название курса и основная информация.
- **Nav** — навигация по разделам курса.
- **Main** — основное содержимое страницы.
- **Описание курса** — краткое описание и основные сведения.
- **Программа курса** — темы или разделы курса.

- **Дополнительная информация** — преподаватель, длительность или другие сведения.
- **Footer** — контактная и служебная информация.

Требования:

- Использовать семантические элементы header, nav, main, section, article, aside, footer.
- Оформить страницу с помощью **CSS3**.
- Использовать текстовый контент и изображения.
- Воспроизвести основную композицию макета.
- **Без адаптивности, эффектов и JavaScript.**

Вариант 6. Туристический маршрут



Разработать веб-страницу туристического маршрута с использованием **HTML5 и CSS3**.

Основная структура страницы:

- **Header** — название маршрута и основная информация.
- **Nav** — навигация по разделам маршрута.
- **Main** — основное содержимое страницы.
- **Описание маршрута** — краткое описание и основные сведения.
- **Маршрут** — основные точки или этапы путешествия.

- **Дополнительная информация** — полезные сведения о маршруте.
- **Footer** — контактная и служебная информация.

Требования:

- Использовать семантические элементы header, nav, main, section, article, aside, footer.
- Оформить страницу с помощью **CSS3**.
- Использовать текстовый контент и изображения.
- Воспроизвести основную композицию макета.
- **Без адаптивности, эффектов и JavaScript.**

Вариант 7. Рецепт блюда

Разработать веб-страницу рецепта блюда с использованием **HTML5 и CSS3**.

Основная структура страницы:

- **Header** — название блюда и краткая информация.
- **Nav** — навигация по разделам страницы.
- **Main** — основное содержимое рецепта.
- **Описание блюда** — краткое описание и изображение блюда.
- **Ингредиенты** — список необходимых продуктов.
- **Приготовление** — последовательность шагов приготовления.

- **Aside** — дополнительная информация о блюде.
- **Footer** — дополнительная/контактная информация.

Требования:

- Использовать семантические элементы header, nav, main, section, article, aside, footer.
- Оформить страницу с помощью **CSS3**.
- Добавить изображение блюда.
- Ингредиенты оформить списком, этапы приготовления — отдельными блоками или списком.
- Воспроизвести расположение основных блоков согласно варианту.
- **Без адаптивности, эффектов и JavaScript.**

Вариант 8. Спортивный клуб

Разработать веб-страницу спортивного клуба с использованием **HTML5 и CSS3**.

Основная структура страницы:

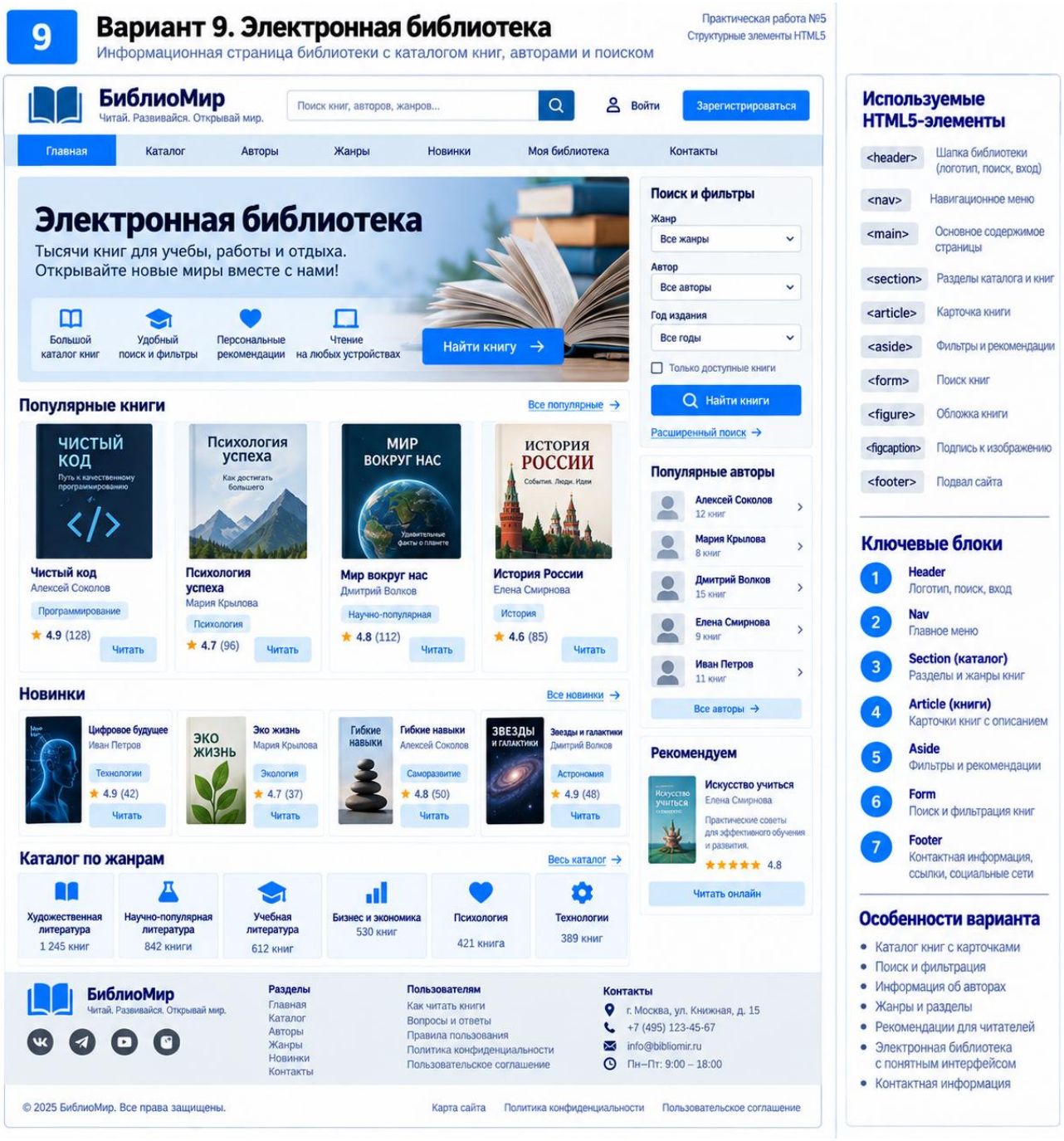
- **Header** — название и логотип спортивного клуба.
- **Nav** — основная навигация по странице.
- **Main** — основное содержимое.
- **О клубе** — краткая информация о клубе.
- **Направления** — список спортивных направлений или секций.
- **Тренеры** — информация о тренерах клуба.

- **Aside** — дополнительная информация.
- **Footer** — контакты и дополнительная информация.

Требования:

- Использовать семантические элементы header, nav, main, section, article, aside, footer.
- Оформить страницу с помощью **CSS3**.
- Добавить необходимые изображения.
- Информацию о направлениях и тренерах оформить отдельными блоками.
- Воспроизвести расположение основных блоков согласно варианту.
- **Без адаптивности, эффектов и JavaScript.**

Вариант 9. Электронная библиотека



Разработать веб-страницу электронной библиотеки с использованием **HTML5 и CSS3**.

Основная структура страницы:

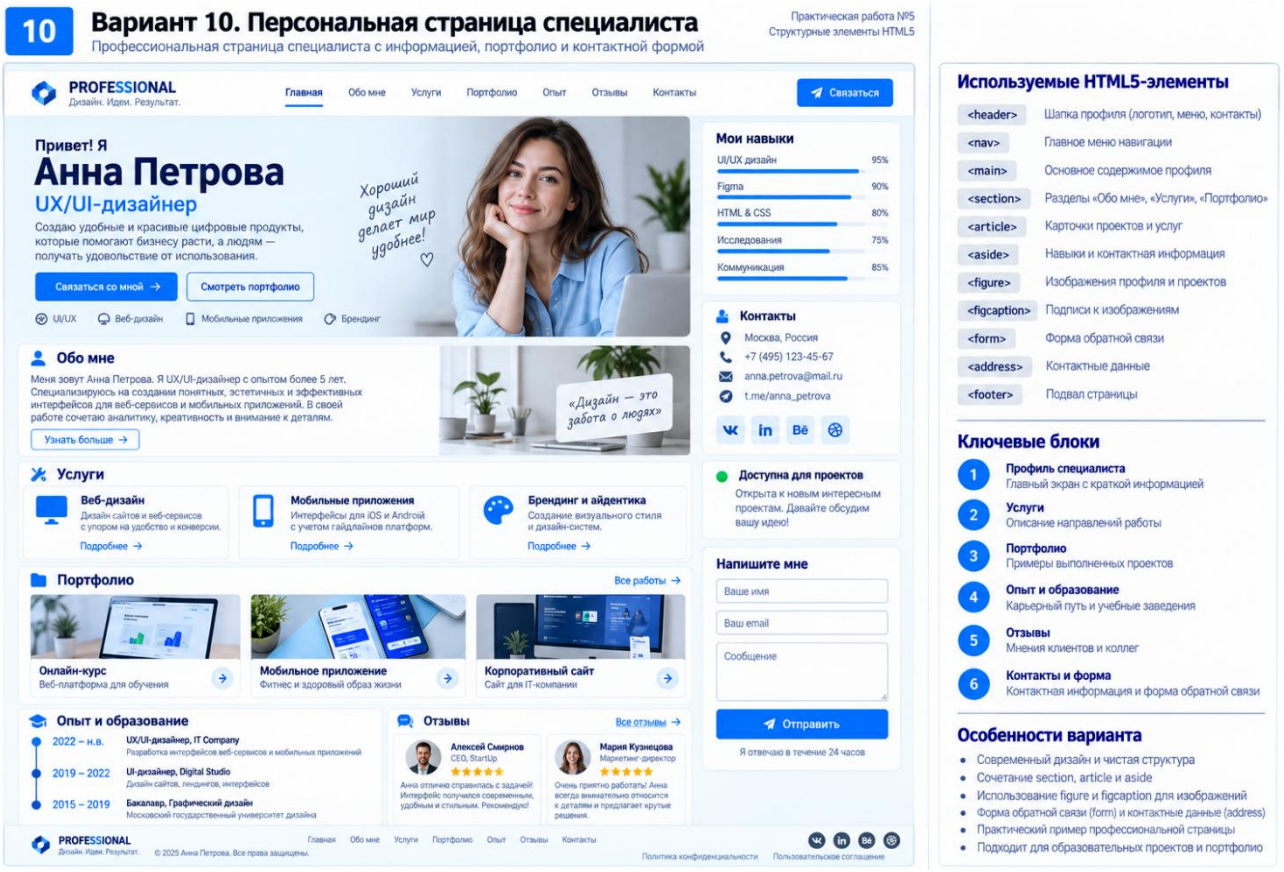
- **Header** — название и логотип библиотеки.
- **Nav** — основная навигация.
- **Main** — основное содержимое.
- **Каталог книг** — список доступных книг.
- **Карточка книги** — обложка, название, автор и краткая информация.
- **Категории** — разделы библиотеки.

- **Aside** — дополнительная информация или ссылки.
- **Footer** — контактная и дополнительная информация.

Требования:

- Использовать семантические элементы header, nav, main, section, article, aside, footer.
- Оформить страницу с помощью **CSS3**.
- Добавить изображения обложек книг.
- Книги оформить отдельными блоками.
- Воспроизвести расположение основных блоков согласно варианту.
- **Без адаптивности, эффектов и JavaScript.**

Вариант 10. Персональная страница специалиста



Разработать веб-страницу персонального специалиста с использованием **HTML5 и CSS3**.

Основная структура страницы:

- **Header** — имя специалиста и краткая информация.
- **Nav** — навигация по разделам страницы.
- **Main** — основное содержимое.
- **О специалисте** — краткая информация о специалисте.
- **Навыки и опыт** — основные компетенции и опыт работы.
- **Портфолио** — примеры выполненных работ.
- **Aside** — дополнительная информация.
- **Footer** — контакты и дополнительная информация.

Требования:

- Использовать семантические элементы header, nav, main, section, article, aside, footer.
- Оформить страницу с помощью **CSS3**.
- Добавить фотографию/изображения и материалы портфолио.
- Информацию представить отдельными логическими блоками.
- Воспроизвести расположение основных элементов страницы.

- **Без адаптивности, эффектов и JavaScript.**

Вариант 11. Вакансия

Разработать веб-страницу вакансии с использованием **HTML5 и CSS3**.

Основная структура страницы:

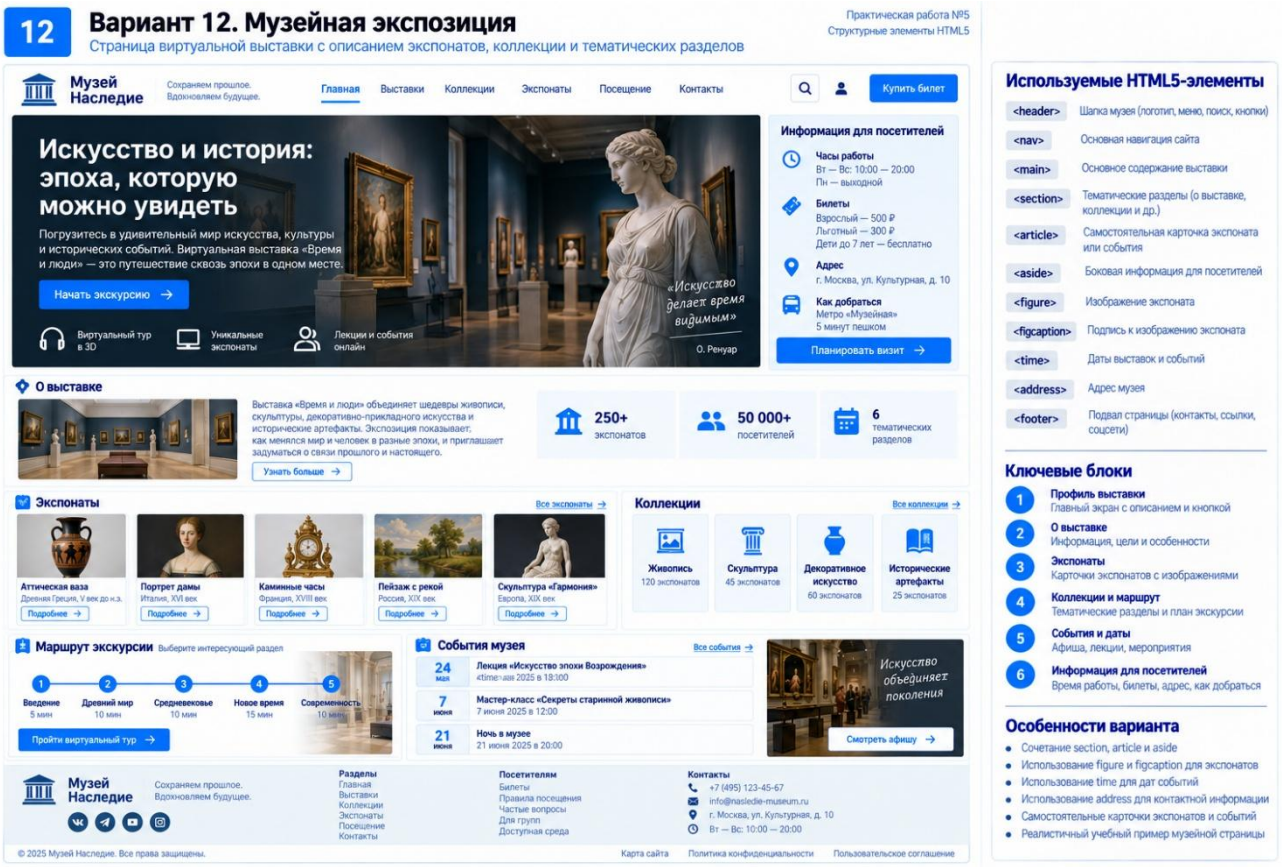
- **Header** — название вакансии и краткая информация.
- **Nav** — навигация по разделам страницы.
- **Main** — основное содержимое.
- **Описание вакансии** — должность и основные обязанности.
- **Требования** — требования к кандидату.
- **Условия работы** — условия и дополнительная информация.

- **Aside** — дополнительная информация.
- **Footer** — контакты и дополнительная информация.

Требования:

- Использовать семантические элементы header, nav, main, section, article, aside, footer.
- Оформить страницу с помощью **CSS3**.
- Информацию разделить на отдельные логические блоки.
- Воспроизвести расположение основных элементов страницы.
- **Без адаптивности, эффектов и JavaScript.**

Вариант 12. Музейная экспозиция



Разработать веб-страницу музейной экспозиции с использованием **HTML5 и CSS3**.

Основная структура страницы:

- **Header** — название музея и экспозиции.
- **Nav** — навигация по разделам.
- **Main** — основное содержимое.
- **Экспозиция** — описание выставки и её основные материалы.
- **Экспонаты** — отдельные блоки с изображением, названием и кратким описанием.
- **Aside** — дополнительная информация о выставке.
- **Footer** — дополнительная информация и контакты.

Требования:

- Использовать семантические элементы header, nav, main, section, article, aside, footer.
- Оформить страницу с помощью **CSS3**.
- Добавить изображения экспонатов.
- Каждый экспонат представить отдельным логическим блоком.
- Воспроизвести расположение основных элементов страницы.
- **Без адаптивности, эффектов и JavaScript.**

Вариант 13. Научно-популярная статья

Разработать веб-страницу научно-популярной статьи с использованием **HTML5 и CSS3**.

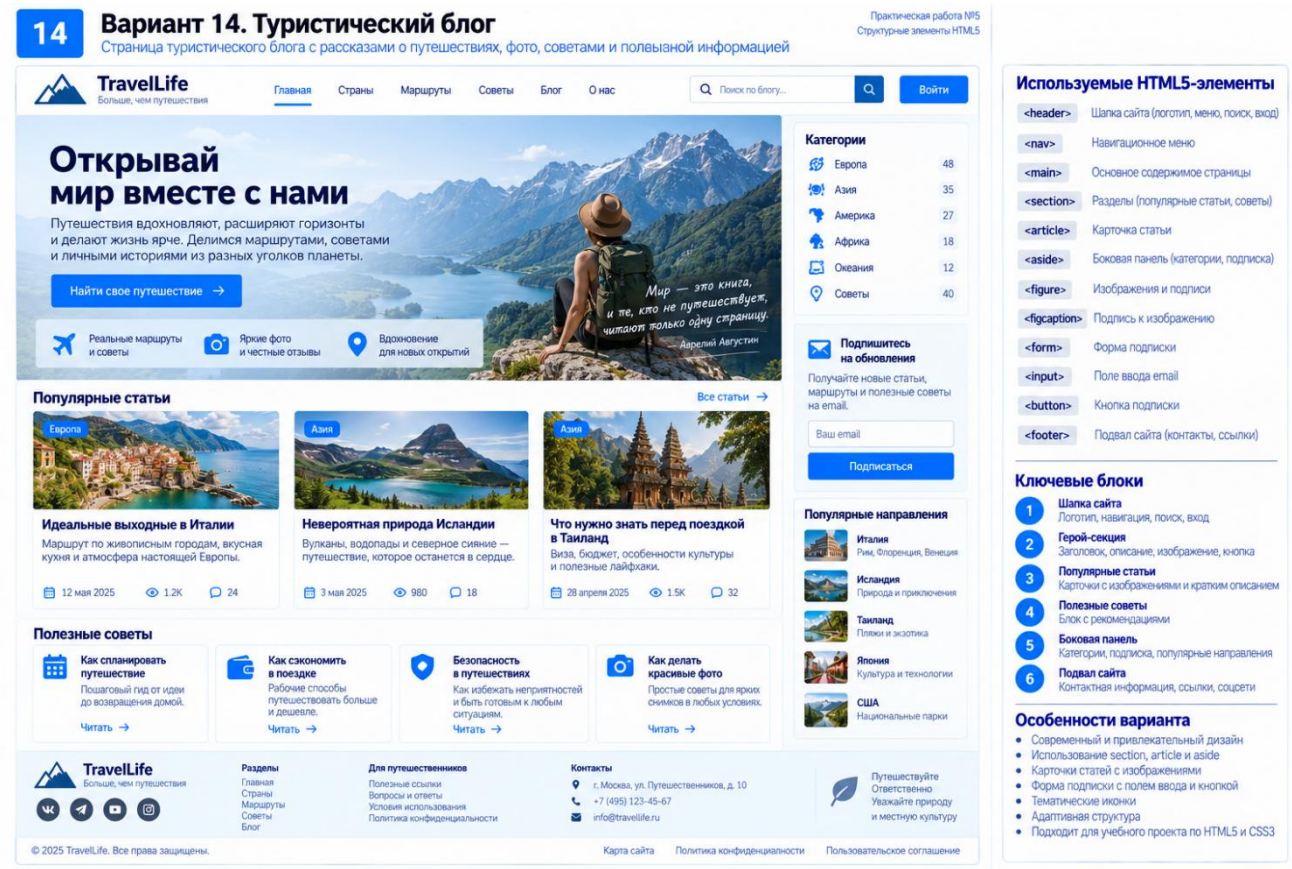
Основная структура страницы:

- **Header** — название статьи.
- **Nav** — навигация по разделам статьи.
- **Main** — основное содержимое.
- **Введение** — краткое описание темы.
- **Основная часть** — текст статьи, разделённый на тематические блоки.
- **Иллюстрации** — изображения или схемы по теме статьи.
- **Aside** — дополнительная информация.
- **Footer** — дополнительная информация и контакты.

Требования:

- Использовать семантические элементы header, nav, main, section, article, aside, footer.
- Оформить страницу с помощью **CSS3**.
- Добавить необходимые изображения или иллюстрации.
- Текст статьи разделить на отдельные логические блоки.
- Воспроизвести расположение основных элементов страницы.
- **Без адаптивности, эффектов и JavaScript.**

Вариант 14. Туристический блог



Разработать веб-страницу туристического блога с использованием **HTML5 и CSS3**.

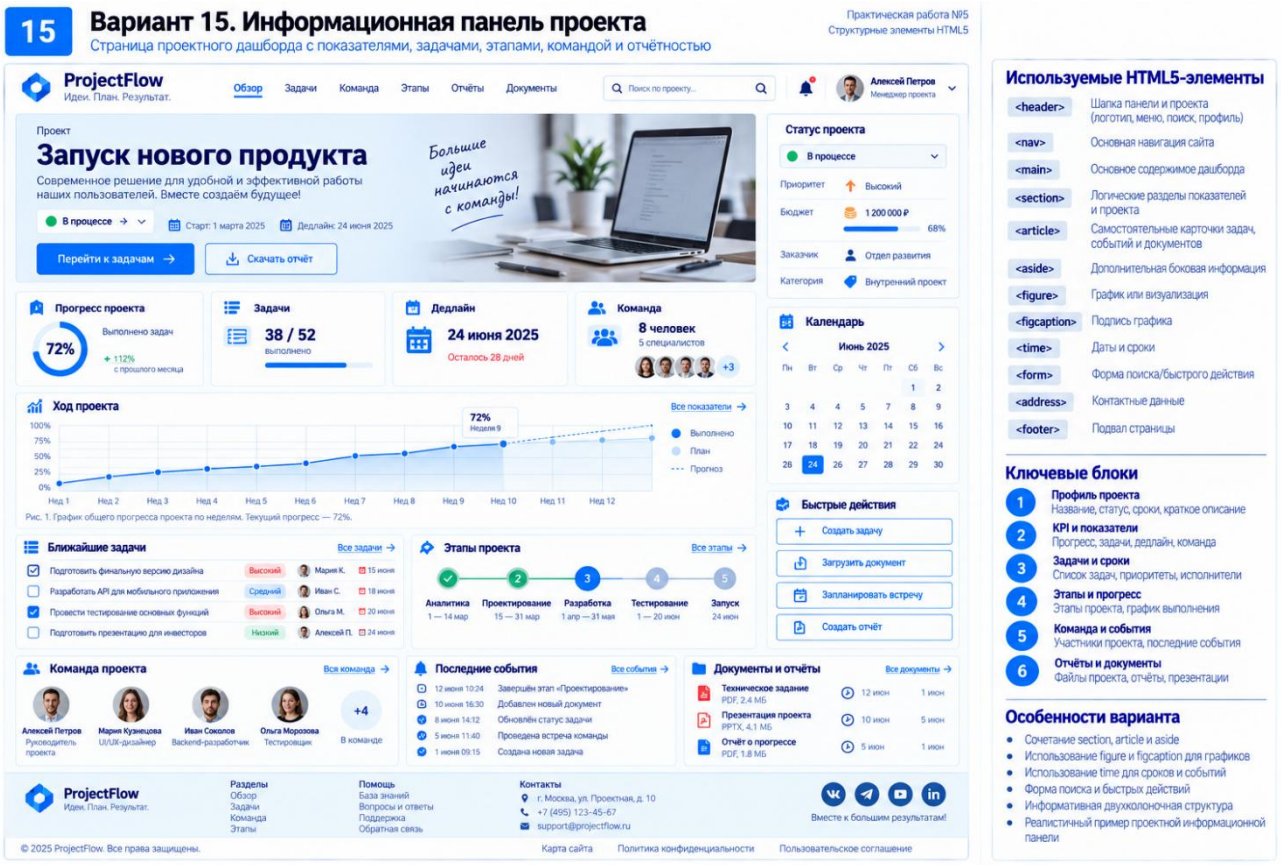
Основная структура страницы:

- **Header** — название блога.
- **Nav** — навигация по разделам блога.
- **Main** — основное содержимое.
- **Статьи** — публикации о путешествиях с заголовком, изображением и текстом.
- **Популярные материалы** — дополнительные записи блога.
- **Aside** — дополнительная информация или категории.
- **Footer** — дополнительная информация и контакты.

Требования:

- Использовать семантические элементы header, nav, main, section, article, aside, footer.
- Оформить страницу с помощью **CSS3**.
- Добавить изображения к материалам блога.
- Каждую публикацию представить отдельным логическим блоком.
- Воспроизвести расположение основных элементов страницы.
- **Без адаптивности, эффектов и JavaScript.**

Вариант 15. Информационная панель проекта



Разработать веб-страницу информационной панели проекта с использованием **HTML5 и CSS3**.

Основная структура страницы:

- **Header** — название проекта и основная информация.
- **Nav** — навигация по разделам панели.
- **Main** — основное содержимое.
- **Информация о проекте** — основные сведения о проекте.
- **Статус проекта** — текущая информация о ходе проекта.
- **Основные показатели** — ключевые данные проекта.
- **Aside** — дополнительная информация.
- **Footer** — дополнительная информация и контакты.

Требования:

- Использовать семантические элементы header, nav, main, section, article, aside, footer.
- Оформить страницу с помощью **CSS3**.
- Информацию представить отдельными логическими блоками.
- Воспроизвести расположение основных элементов страницы.
- **Без адаптивности, эффектов и JavaScript.**

Задание 2. Создать страницу

Создать страницу самостоятельно; макет нельзя использовать как единое фоновое изображение. Подключить внешний CSS-файл и воспроизвести визуальную композицию. Реализовать указанные для варианта интерактивные состояния и базовые CSS3-эффекты. Проверить страницу на desktop и мобильной ширине.

При разработке руководствоваться спецификацией.

Общая мини-спецификация HTML5. Обязательная семантическая структура

Элемент	Назначение	Требование
<header>	Шапка документа или раздела	Использовать для верхней области страницы/раздела.
<nav>	Навигационные ссылки	Основное меню и/или отдельный блок навигации.
<main>	Основное содержимое	Один основной main для содержимого страницы.
<section>	Тематический раздел	Использовать для самостоятельных смысловых областей.
<article>	Самостоятельный материал	Например, новость, книга, вакансия, карточка товара.
<aside>	Дополнительное содержимое	Фильтры, рекомендации, контакты, sidebar и т. п.
<footer>	Нижняя область	Контакты, ссылки и служебная информация.

Дополнительные элементы

- <figure> + <figcaption> — самостоятельная иллюстрация с подписью;
- <time> — дата, время события или публикации;
- <address> — контактная информация;
- <form>, <label>, <input>, <button> — поиск, подписка, отклик и обратная связь;
- <table> — только для табличных данных;
- , <ol>, <li> — списки;
- <h1>–<h6> — логическая иерархия заголовков;
- alt у изображений — обязательный текстовый эквивалент.

Общие ограничения

- Основная реализация выполняется средствами HTML5 и CSS3.
- Не допускается собирать страницу из одного большого изображения.
- CSS рекомендуется хранить во внешнем файле.
- JavaScript для этой работы не является обязательным и не должен подменять семантическую HTML-разметку.
- Сторонние изображения допускаются только при наличии права на использование; предпочтительны собственные или учебные ресурсы.

Базовые CSS3-требования оформления

Хотя основная тема работы — семантическая HTML5-структура, визуальный результат должен быть аккуратно оформлен CSS3. В каждом варианте необходимо продемонстрировать осознанное применение нескольких базовых возможностей CSS.

Техника	Минимальное требование	Пример применения
Внешняя таблица стилей	style.css	Все основные стили вынесены из HTML.
Box Model	width/height/padding/border/margin	Карточки, секции, кнопки.
Селекторы	не менее 5 типов	тег, класс, id/атрибут, псевдокласс, составной селектор.
:hover / :focus	для интерактивных элементов	Ссылки, кнопки, карточки.
::before / ::after	хотя бы один осмысленный элемент	Декоративная линия, маркер, overlay.
transition	не менее 2 элементов	Плавное изменение тени/цвета/transform.
transform	минимум один эффект	scale(), translateY().
box-shadow / border-radius	использовать в композиции	Карточки и панели.
Фоны/градиенты	по назначению макета	Hero, CTA, информационные блоки.
Адаптивность	без горизонтального переполнения	Media queries для узких экранов.

Базовый шаблон интерактивной карточки

```
   .card {
  border-radius: 16px;
  box-shadow: 0 8px 24px rgba(15, 23, 42, .12);
  transition: transform .3s ease, box-shadow .3s ease;
}

.card:hover {
  transform: translateY(-3px);
  box-shadow: 0 14px 32px rgba(15, 23, 42, .18);
}
```

Псевдоэлемент для декоративного слоя

```
   .hero {
  position: relative;
  overflow: hidden;
}

.hero::after {
  content: "";
  position: absolute;
  inset: 0;
  background: linear-gradient(
    to bottom,
    rgba(15, 23, 42, .05),
    rgba(15, 23, 42, .65)
```

```
  );
}
```

Адаптивность

```
  @media (max-width: 768px) {
  .layout {
    grid-template-columns: 1fr;
  }

  .sidebar {
    order: 2;
  }
}
```

Требования к структуре проекта

```
  lab-5/
├── index.html
├── css/
│   └── style.css
├── images/
│   └── ...
└── README.md
```

- index.html — точка входа страницы;
- css/style.css — основная таблица стилей;
- images/ — локальные изображения проекта;
- README.md — ФИО, группа, номер варианта, краткое описание и ссылка на опубликованную страницу.

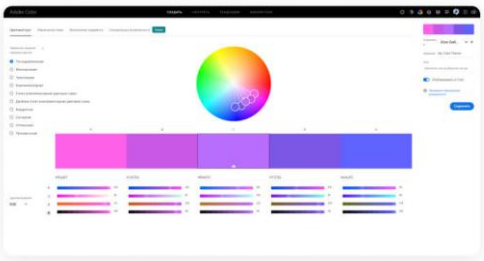
Если используется несколько страниц, допускается каталог **pages/**.

Сделайте редизайн элементов страницы после их реализации и размещения на странице. Подбирать цветовую палитру, ориентируясь исключительно на зрительное восприятие, неудобно. Сегодня веб-дизайнеры используют программы и онлайн-сервисы для подбора гаммы. Среди наиболее популярных:

1. Adobe Color CC. Подходит как для профи, так и для рядовых разработчиков сайтов, которые решили поэкспериментировать с оформлением сайта самостоятельно. Сервис позволяет сформировать гамму с нуля или подобрать контекстные цвета

к уже имеющимся базовым. К преимуществам можно отнести простоту и гибкость

Adobe Color CC



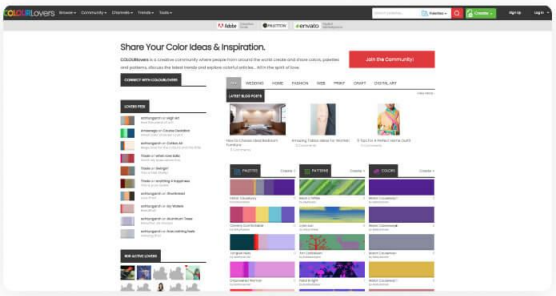
настроек.

2. paletton.com. Узкоспециализированный ресурс для подбора цветовой гаммы для сайта. Выбранное сочетание можно сразу «примерить» на макет.

paletton.com

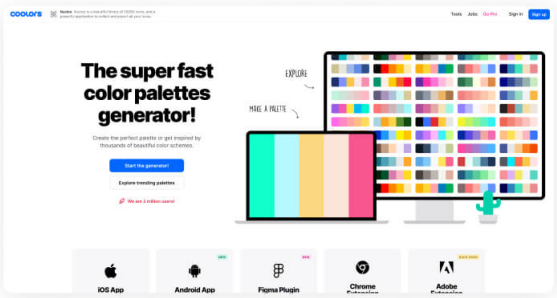
3. colourlovers.com. Сайт с готовыми цветовыми схемами, ими делятся рядовые пользователи. Ознакомиться и отобрать схемы можно с помощью фильтра категорий.

colourlovers.com



4. Coolors. Генератор цветовых палитр, отличный выбор для новичков. Учиться можно, не только экспериментируя, но и, используя схемы, созданные опытными дизайнерами.

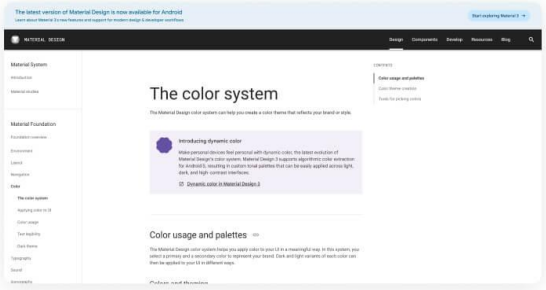
Coolors



5. Material Color Tool. Узкоспециализированный инструмент для визуального оформления пользовательского интерфейса. С его помощью можно не только

сгенерировать идеи, но и сразу посмотреть, как они могут быть реализованы в реальных проектах.

Material Color Tool



Задание 3. Представление практической работы

Созданную страницу необходимо опубликовать на сервисе **GitHub.io** и предоставить ссылку (вида https://username.github.io) на законченный и рабочий вариант для проверки преподавателем.

Последовательность действий для публикации:

1. Зарегистрируйтесь на https://github.com/, если ещё не сделали этого.
2. Создайте новый репозиторий с именем username.github.io, где username — ваш ник на GitHub.
3. Создайте новый файл. Когда репозиторий не пуст, то в верхнем правом углу есть кнопка "Create New File", но сейчас, когда ещё нет ни одного файла, создать первый можно по прямой ссылке https://github.com/username/username.github.io/new/master, где username — ваш ник.
4. Введите имя index.html и скопируйте туда содержание файла index.html.
5. Нажмите кнопку "Commit new file".
6. Повторите то же самое для файла style.css.
7. Подтвердите публикацию на GitHub Pages. В настройках репозитория: Settings -> Раздел Options -> Секция GitHub Pages.
8. Через несколько мгновений страница станет доступна по вашему адресу https://username.github.io

Примечание. *Страница должна оставаться доступной по ссылке до момента проверки работы в курсе. Затем опубликованные материалы можно удалить или заместить следующими работами.*

Дополнительно, все созданные элементы (исходные материалы) нужно поместить в каталог **lab-5** на GitHub.

Чек-лист самопроверки

HTML-документ

- ☐ Используется <!DOCTYPE html>.

- ☐ Указан lang="ru".
- ☐ Задана кодировка UTF-8.
- ☐ Задан meta viewport.
- ☐ Есть понятный title.
- ☐ Есть один логически обоснованный main.
- ☐ Использованы header и footer.
- ☐ Навигация оформлена через nav.
- ☐ Разделы организованы через section там, где это уместно.
- ☐ Самостоятельные материалы оформлены через article там, где это уместно.
- ☐ Дополнительная информация оформлена через aside там, где это уместно.
- ☐ Заголовки имеют логичную иерархию.

Семантика и доступность

- ☐ HTML-элементы выбраны по смыслу, а не только ради визуального расположения.
- ☐ Все изображения имеют alt.
- ☐ Формы имеют label для полей.
- ☐ Даты/время размечены через time, если предусмотрено вариантом.
- ☐ Контакты размечены через address, если предусмотрено вариантом.
- ☐ Таблица используется только для табличных данных.
- ☐ figure/figcaption применяются для иллюстраций с подписями, если они нужны.

CSS и визуальная часть

- ☐ Основная композиция соответствует макету.
- ☐ Основные стили находятся во внешнем CSS.
- ☐ Использованы width/height/padding/border/margin.
- ☐ Есть минимум пять типов селекторов.
- ☐ Есть :hover.
- ☐ Есть :focus у интерактивных элементов.
- ☐ Использован ::before и/или ::after.
- ☐ Есть transition.
- ☐ Есть transform.
- ☐ Есть box-shadow и border-radius.
- ☐ Использованы фон/градиент по назначению.
- ☐ Нет горизонтального переполнения.
- ☐ Изображения не искажаются.

GitHub и GitHub Pages

- ☐ Исходный код находится в репозитории.
- ☐ В репозитории есть index.html.
- ☐ В репозитории есть CSS и все необходимые ресурсы.
- ☐ README содержит ФИО, группу, номер варианта и ссылки.

- ☐ Выполнен commit/push последней версии.
- ☐ В репозитории нет паролей, токенов и других секретных данных.
- ☐ GitHub Pages включён.
- ☐ Страница открывается по публичному URL.
- ☐ CSS и изображения загружаются после публикации.
- ☐ URL репозитория и URL страницы переданы преподавателю.

Контрольные вопросы

1. Что такое семантические теги в HTML5 и какова их основная функция?
2. Назовите основные семантические теги HTML5 и опишите их назначения.
3. Какой тег используется для обозначения заголовка страницы и его структуры?
4. Какой тег в HTML5 предназначен для создания навигационного меню?
5. Что такое тег <header> и какие элементы обычно включаются в него?
6. Какой тег используется для обозначения основного содержимого страницы, и какова его роль?
7. Какой тег HTML5 используется для выделения независимого содержимого, например, статьи?
8. Что такое тег <aside> и в каком контексте он обычно используется?
9. Какой тег HTML5 используется для создания подвала страницы и что обычно в него включается?
10. Как правильно использовать тег <section> для организации контента?
11. Как семантические теги улучшают доступность веб-страниц для людей с ограниченными возможностями?
12. Почему важно использовать семантические теги для SEO?
13. Какой тег используется для создания списка ссылок на другие страницы или разделы сайта?
14. Что такое тег <main> и как он отличается от других семантических тегов?
15. Как семантические теги влияют на структуру документа и его читаемость?
16. Как правильно использовать тег <article> для организации контента на странице?
17. Как можно комбинировать семантические теги для создания сложных макетов?
18. Что такое "ария" (ARIA) и как она связана с семантическими тегами?
19. Как можно использовать семантические теги для улучшения пользовательского опыта на сайте?
20. Как проверить валидность семантической разметки на веб-странице?

ПРАКТИЧЕСКАЯ РАБОТА 6. ВВЕДЕНИЕ В CSS3

Цель работы

- **Понимание основ CSS3:** изучить основные понятия и принципы работы CSS3, ознакомиться с синтаксисом и структурой CSS.
- **Применение стилей к HTML-документам:** научиться подключать CSS к HTML-документам (внутренние и внешние стили), изучить различные способы применения стилей (по классу, идентификатору, тегу).
- **Работа с селекторами и свойствами:** освоить различные типы селекторов (теговые, классные, идентификаторы, псевдоклассы и псевдоэлементы), изучить основные свойства CSS (цвет, фон, шрифты, размеры и отступы).
- **Создание макетов с использованием CSS3:** научиться использовать Flexbox и Grid для создания адаптивных макетов, понять принципы позиционирования элементов (static, relative, absolute, fixed), стилизовать различные HTML-элементы.

Теоретическая часть

6.1. Модель коробки в CSS

CSS *– это формальный язык, служащий для описания оформления внешнего вида документа, созданного с использованием языка разметки (HTML, XHTML, XML). Название происходит от английского Cascading Style Sheets, что означает «каскадные таблицы стилей».*

Этот язык позволяет:

- добавлять макет или дизайн на веб страницы
- делиться стилями от элемента к элементу и от страницы к странице.

Важно точно знать, как визуализируются стили. В частности, нужно знать, как работают разные типы селекторов и как определяется порядок этих селекторов, т.к. они могут влиять на отображение стилей. Нужно понимать значения общих свойств,

которые постоянно появляются в CSS, особенно те, которые имеют дело с цветом и длиной элементов.

Модель коробки

Все, что отображается с помощью CSS, представляет собой прямоугольник. Понимание того, как работает ***блочная (коробочная) модель CSS*** *- является основой CSS.*

В HTML-документе каждому элементу на странице соответствует *прямоугольная область* (*бокс* или *блок*). Каждый элемент, который может отображаться на веб-странице, состоит из одного или нескольких прямоугольных блоков. Блочная модель CSS обычно описывает, как эти прямоугольные блоки расположены на веб-странице. Эти блоки могут иметь разные свойства и могут по-разному взаимодействовать друг с другом, но каждый блок имеет *область содержимого* и необязательные окружающие области *padding*, *border* и *margin*.

На следующем рисунке показано, как CSS-свойства `width`, `height`, `padding`, `border` и `margin` определяют, сколько места элемент может занимать на веб-странице.

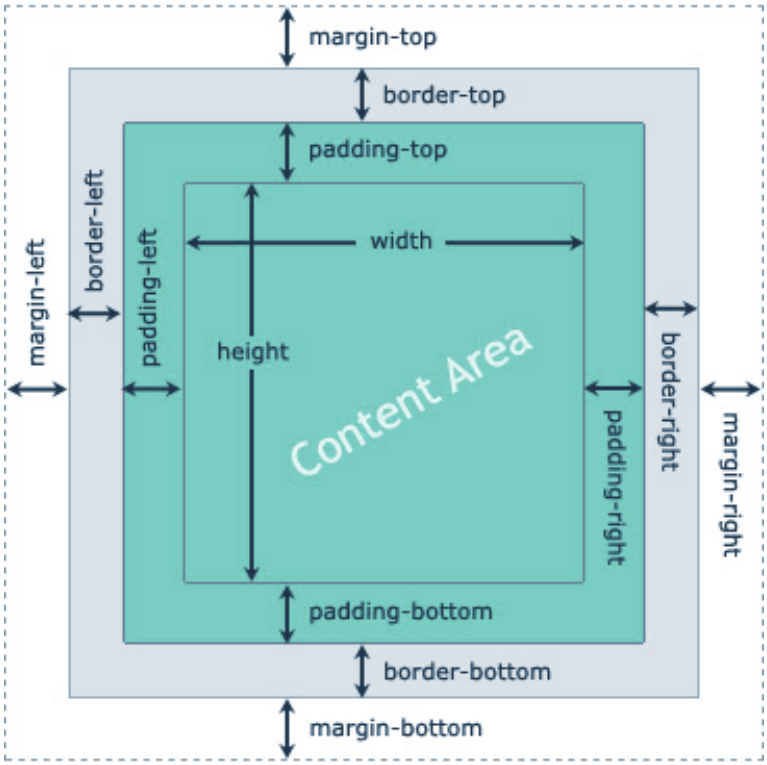


Рис. 6.1. Модель коробки

`Padding` — это прозрачное пространство между содержимым элемента и его границей (или краем блока, если у него нет границы), а `margin` — это прозрачное пространство вокруг границы. Кроме того, если элемент имеет цвет фона, он будет виден через

область заполнения. Область поля всегда остается прозрачной, на нее не влияет цвет фона элемента, однако из-за нее сквозь нее виден цвет фона родительского элемента.

Ширина и высота элементов

Обычно, когда устанавливается ширина и высота элемента с помощью CSS свойств *width* и *height*, на самом деле выставляется только ширина и высота области содержимого этого элемента. Фактическая ширина и высота блока элемента зависит от нескольких факторов. Фактическое пространство, которое поле элемента может занимать на веб-странице, рассчитывается следующим образом:

Таблица 6.1. Расчет занимаемого пространства элемента

Размер коробки	CSS-свойства
Общая ширина	width + padding-left + padding-right + border-left + border-right + margin-left + margin-right
Общая высота	height + padding-top + padding-bottom + border-top + border-bottom + margin-top + margin-bottom

Рендеринг

Движок рендеринга в браузере определяет размеры и положение боксов на странице, а также их свойства в виде цвета или фоновой картинки для того, чтобы отобразить их на экране. Предположим, у нас есть такой фрагмент HTML:

```
<p>Я абзац текста, в котором есть несколько слов.</p>
```

Затем мы добавляем для него этот CSS:

```
p {
  width: 100px;
  height: 50px;
  padding: 20px;
  border: 1px solid;
}
```

И получаем следующее ...

Рис. 6.2. Результат рендеринга

Контент выйдет за пределы элемента и будет иметь ширину 142 пикселя, а не 100 пикселей.

Отладка коробочной модели

Практически в каждый современный браузер (и не только) встроены средства для просмотра исходного кода HTML + CSS + JScript и его отладки. Браузерные средства, называемые DevTools, обеспечивают визуализацию вычислений блочной модели для выбранного элемента HTML. Это позволяет понять, как работает блочная модель, и как она влияет на элементы веб-сайта в динамике.

Для открытия инструментов разработчика можно воспользоваться сочетанием *Ctrl + Shift + I* в Windows и Linux, *Command + Option + I* в macOS. Рассмотрим на примере инструментов разработчика браузеров Chrome и Firefox.

На панели разработки мы сразу увидим правую область, где будет отображаться эквивалент (или фрагмент) модель коробки с числовыми характеристиками для выбранного элемента html. Эти характеристики можно изменять и сразу видеть результат этих изменений. Данный инструментарий просто незаменим для разработчиков.

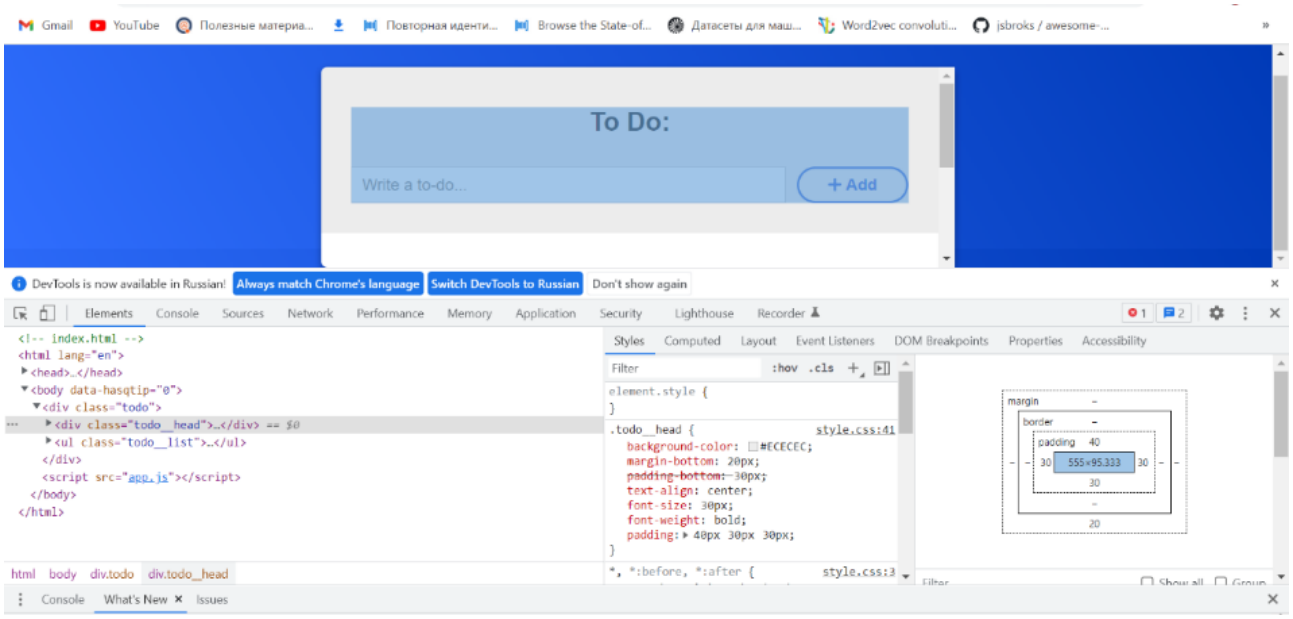


Рис. 6.3. Отладка коробочной модели в DevTools

Еще одно применение инструментария разработки - отладка программного кода (JScript). Это выполняется в специальном окне Отладчик (на примере Firefox). Окно отладки разделено на три части:

- Слева список всех JavaScript-файлов открытой страницы.
- По центру область предпросмотра.
- Справа панель инструментов.

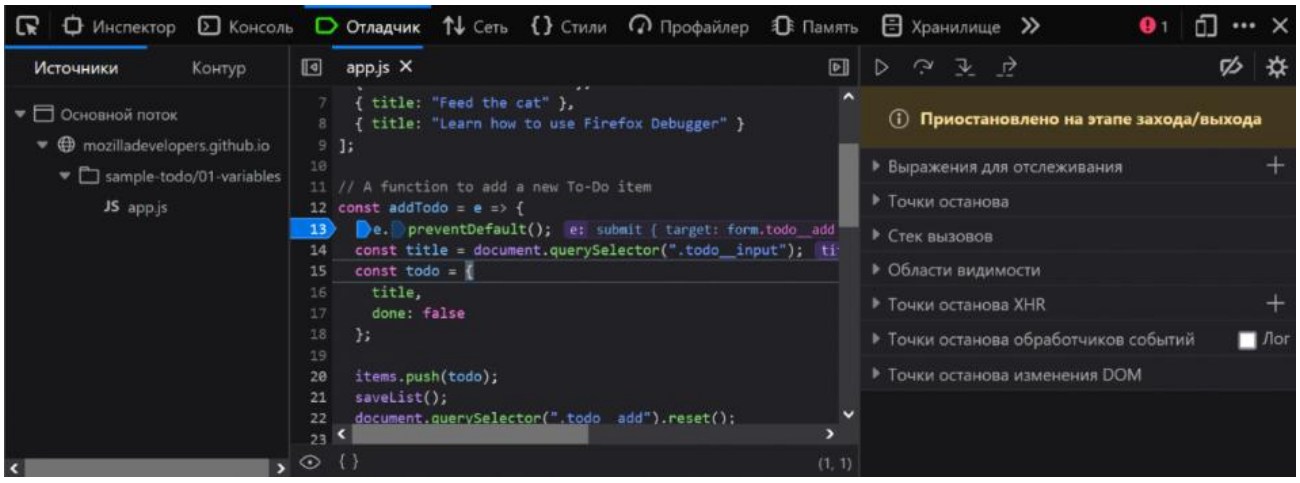


Рис. 6.4. Окно отладки DevTools

На панели инструментов можно выделить:

- В самом верху ***кнопки для навигации*** по коду во время отладки.
- **Выражения для отслеживания** — это набор значений, изменение которых мы хотим отследить во время отладки.
- **Точки останова** — список всех указанных в отладчике точек останова с возможностью включить-выключить каждую. Для удобства разработчика отдельно выделены точки останова в HTTP-запросах (XHR, fetch), обработчиках событий и на изменения DOM.
- **Стек вызовов** — список всех вызванных функций до момента остановки во время отладки. Справа от названия функции указаны номер строки и имя файла, где она была вызвана.
- **Области видимости** со списком всех доступных значений на момент остановки во время отладки.

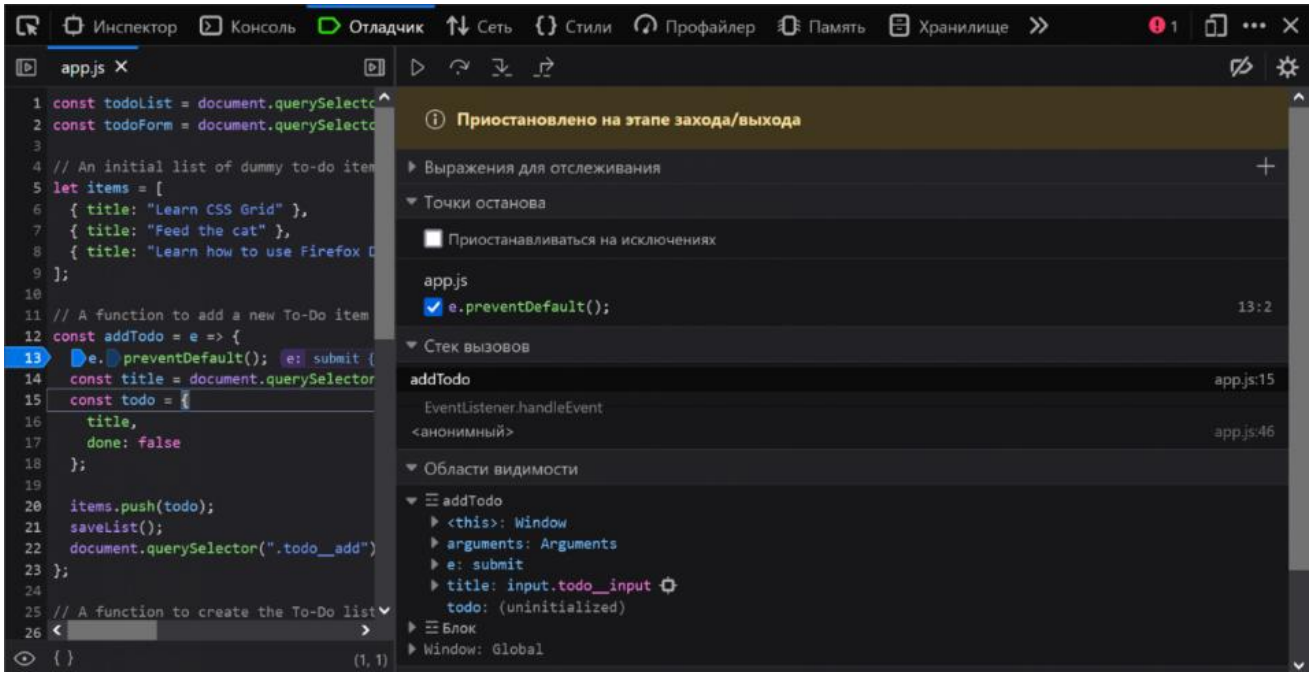


Рис. 6.4. Области панели инструментов DevTools

Содержание и размер

Модели коробки имеют разное поведение в зависимости от их *ценности (display)*, *установленных размеров* и *содержимого*.

Под *содержимым* понимается множество других блоков (коробок), созданных дочерними элементами, или содержимое текстового поля. В любом случае это содержимое, по умолчанию, влияет на размер поля.

В CSS есть два способа изменения размера:

- *внутренний (Intrinsic sizing)*.
- *внешний (Extrinsic sizing)*.

Последний является более распространенным, и он означает использование фиксированных значений ширины или высоты элемента. Первый способ основан на том, что размер элемента зависит от размера его содержимого. Мы можем контролировать представление элемента, используя *внешний размер*, или позволить браузеру принимать решения за вас в зависимости от размера контента, выставляя *внутренний размер*. Давайте рассмотрим на разницу этих ситуаций, используя эту демонстрацию.

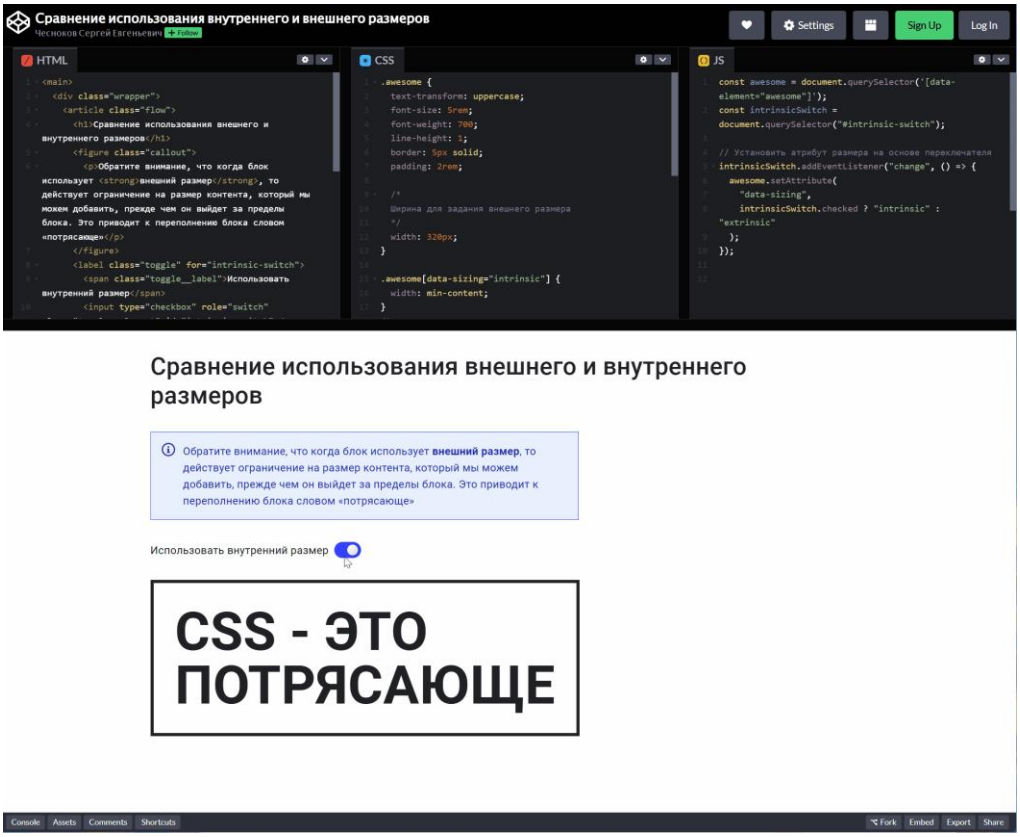


Рис. 6.5. Демонстрация отличий внешнего и внутреннего размеров

В примере есть слова *«CSS - это потрясающе»* в поле с фиксированными размерами и толстой рамкой. Блок имеет определенную ширину 320 пикселей, которая задана в CSS и поэтому имеет *внешний размер*. Он контролирует размер своего *дочернего содержимого*. Проблема заключается в том, что слово «потрясающий» слишком велико для блока, поэтому оно выходит за пределы рамки границы родительского блока, что вызывает **переполнение**. Можно сказать, что это слово переполняет блок.

Один из вариантов предотвратить это переполнение состоит в том, чтобы сбросить фиксированную ширину блока, установив в *width* значение *min-content*. Значение представляет собой **предпочтительную ширину**, и говорит, что элемент должен быть равен ширине содержимого. Ключевое слово min-content указывает полю стать равным ширине внутренней минимальной ширины его содержания (слово «потрясающе»). Это позволяет рамке идеально вписаться в блок с *«CSS — это потрясающе»*.

Давайте посмотрим на что-то более сложное, чтобы увидеть влияние различных размеров на реальный контент. Чтобы дополнительно увидеть эффект, который оказывает внутреннее и внешнее изменение размеров, добавьте на карточку несколько предложений содержания. Когда этот элемент использует внешний размер, существуют пределы по добавляемому содержимому, прежде чем контент выйдет за границы элемента. Это происходит в случае, когда включен внутренний размер.

По умолчанию у этого элемента есть набор `width` и `height` - оба по `400px`.

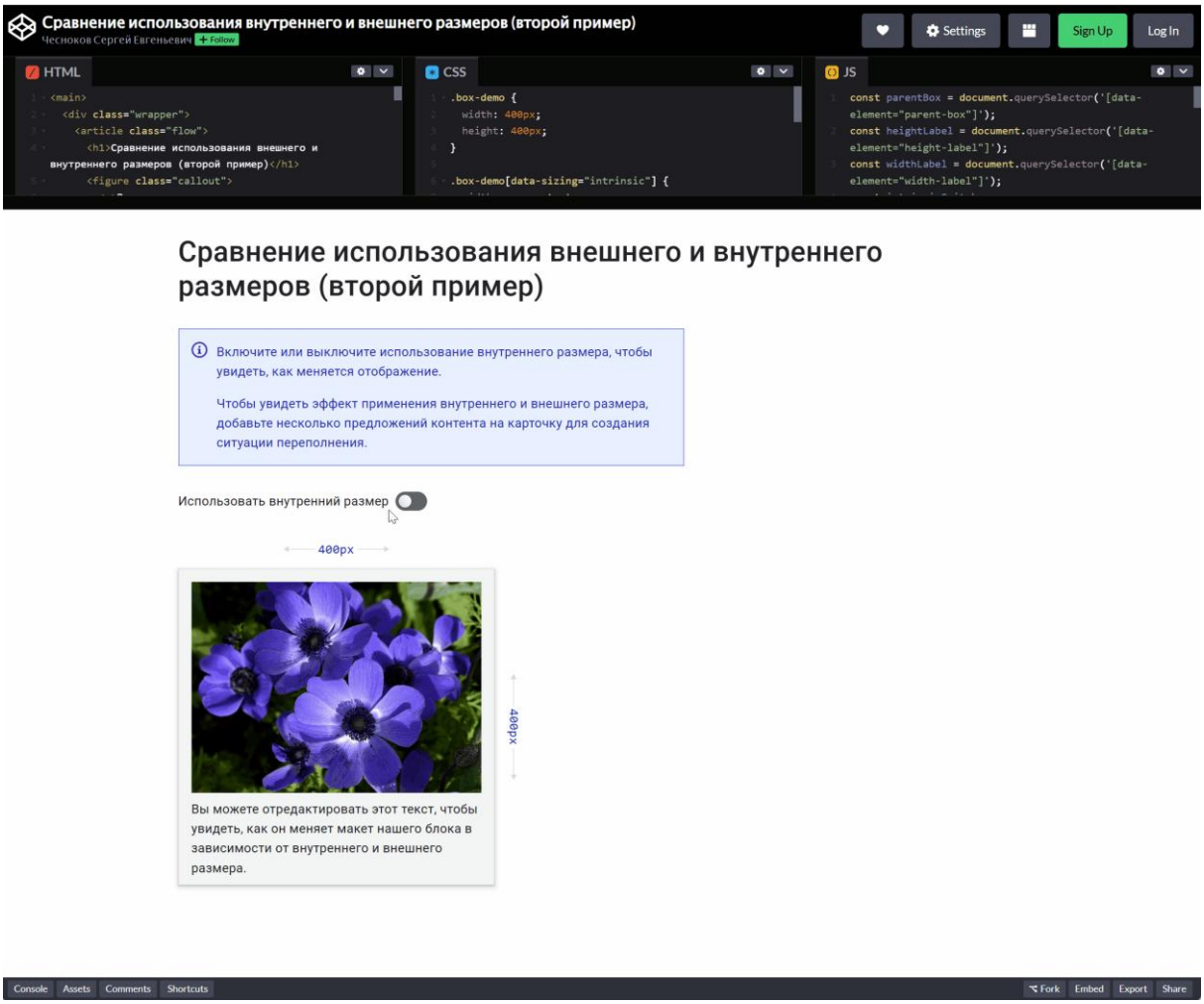


Рис. 6.6. Пример возникновения переполнения

Эти размеры дают строгие границы всему внутри элемента, которые будут соблюдаться, если содержимое слишком велико для блока, и в этом случае произойдет видимое переполнение.

Когда содержимое слишком велико для коробки, в которой оно находится, это называют переполнением. Вы можете управлять тем, как элемент обрабатывает содержимое переполнения, используя свойство overflow.

Когда происходит переключение на собственное (внутреннее) определение размеров, то браузеру принимает решения за нас в зависимости от размера содержимого

окна. Очень сложно получить переполнение с вариантом внутреннего определения размеров, потому что размер нашего блока будет изменяться вместе с его содержимым. И это более предпочтительный вариант, чем пытаться изменить размер содержимого. Важно помнить, что это гибкое поведение браузера работает по умолчанию.

Области коробочной модели

Коробки состоят из отдельных областей блочной модели, каждая из которых выполняет определенную работу и имеет собственные параметры. Выделяют четыре основных области блочной модели:

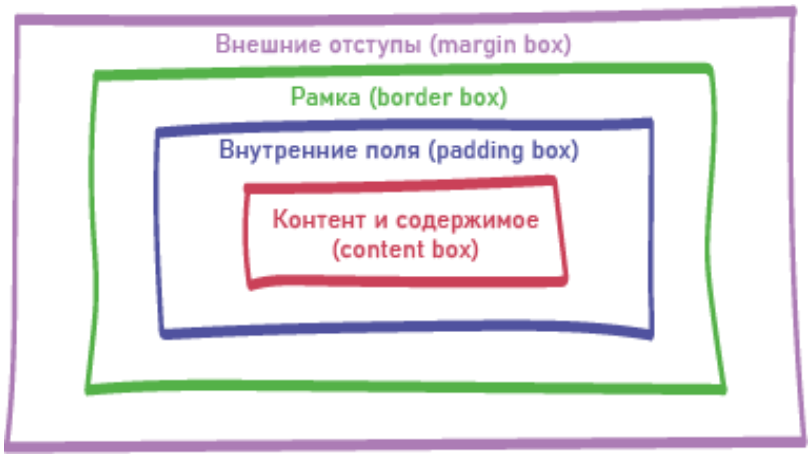


Рис. 6.7. Области блочной (коробочной) модели

Все начинается с *блока содержимого (content box)* - область с содержимым или контентом. Это содержимое может управлять размером своего родителя, поэтому обычно это область с изменяемым размером. У блока содержимого часто задается фон, цвет или изображение (в таком порядке: фоновый цвет скрывается под непрозрачным изображением) и он находится внутри `content edge`. Размеры обозначаются как `ширина контента (content width или content-box width)` и `высота контента (content height или content-box height)`. Иногда ещё говорят **«** `внутренняя ширина/высота элемента` **»**.

Поля элемента (padding box) — это пустая область, окружающая контент. Она может быть залита каким-то цветом, покрыта фоновый картинкой, а её границы называются `края полей (padding edge)`. Размеры полей задаются по отдельности с разных сторон свойствами `padding-top, padding-right, padding-bottom, padding-left` или общим свойством `padding`. Поскольку отступ находится внутри блока, фон блока будет виден в создаваемом им пространстве. Если в нашем поле установлены правила переполнения, такие как `overflow: auto` или `overflow: scroll`, полосы прокрутки тоже займут это пространство.

Область рамки (border box) окружает поля элемента, а её граница называется `края рамки (border edge)`. Ширина рамки задаётся отдельным свойством `border-width` или в составе свойства `border`. Размеры элемента с учётом полей и рамки иногда называют `внешней шириной/высотой элемента`.

Отступы (margin box) добавляют пустое пространство вокруг элемента и определяют расстояние до соседних элементов. Величина отступов задаётся по отдельности в разных направлениях свойствами `margin-top`, `margin-right`, `margin-bottom`, `margin-left` или общим свойством `margin`. Отступы двух соседних элементов, расположенных друг над другом или вложенных друг в друга, могут накладываться. Это называется **схлопыванием границ (margin collapsing)**. Схлопываются только вертикальные отступы.

Такие свойства, как `outline` и `box-shadow` занимают это пространство, потому что они нарисованы сверху, поэтому они не влияют на размер нашего блока. Можно иметь `outline-width` равный `200px` в коробке и все внутри, включая границу окна будет с точно такого же размера.

6.2. Селекторы CSS

Простые селекторы

Каким способом можно увеличить шрифт текста первого абзаца статьи сайта и сделать его красным?

```
<article>
  <p>Шрифт этого абзаца должен быть красным и более крупным, чем в остальном тексте.</p>
  <p>Шрифт этого абзаца должен иметь нормальный размер и цвет, используемый по умолчанию.</p>
</article>
```

Конечно же это можно сделать при помощи CSS. С помощью селектора CSS найти нужный элемент и применить к нему правило CSS, например, так:

```
article p:first-of-type {
  color: red;
  font-size: 1.5em;
}
```

Для подобных ситуаций в CSS имеется множество способов выбора элементов и применения правил к ним — от очень простых до очень сложных. Чтобы понять, как работают селекторы и какова их роль в CSS, важно знать части правила CSS.

Правило CSS *— это блок кода, в котором содержится один или несколько селекторов и одно или несколько объявлений.*

Рис. 6.8. Иллюстрация правила CSS

В этом правиле CSS **селектором** является элемент `.my-css-rule`, который находит все элементы с классом `my-css-rule` на странице. В фигурных скобках содержатся три объявления.

Объявление *— это пара, состоящая из свойства и значения, благодаря которой можно применять стили к элементам, соответствующим селекторам.*

Правило CSS может иметь неограниченное количество объявлений и селекторов.

Разновидности простых селекторов

Группа самых простых селекторов предназначена для элементов HTML, а также классов, идентификаторов и других атрибутов, которые можно добавить в тег HTML.

1. Универсальный селектор

`Универсальный селектор`, также называемый `подстановочным знаком`, соответствует любому элементу.

```
* {
  color: orangered;
}
```

Благодаря этому правилу текст каждого элемента HTML на странице будет иметь оранжевый ("Orangered") цвет.

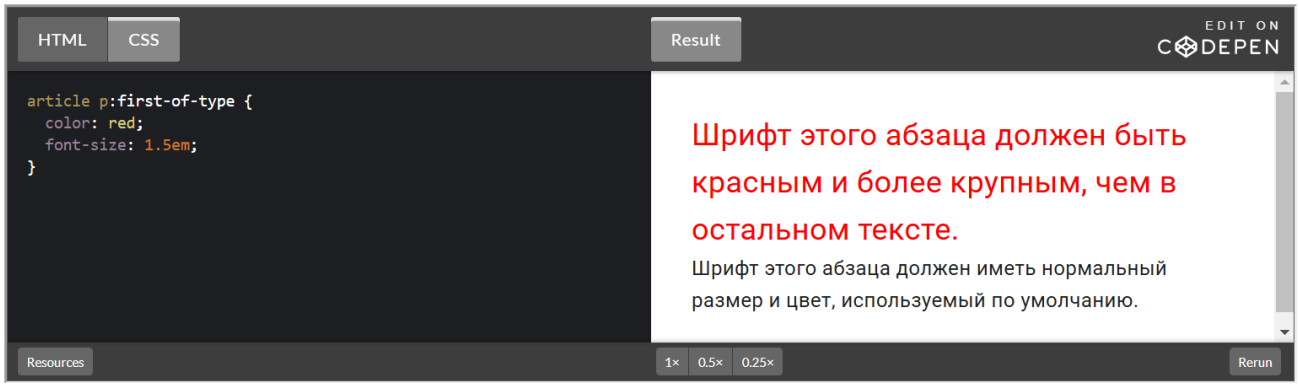


Рис. 6.9. Применение простого селектора

Цвета и их устоявшееся имена можно посмотреть на сервисе https://hashtagcolor.com/

2. Селектор типа

`Селектор типа` напрямую соответствует элементу HTML.

```
section {
  padding: 2em;
}
```

Благодаря этому правилу для всех сторон каждого элемента `<section>` атрибут отступа `padding` будет иметь значение `2em`.

3. Селектор класса

Элемент HTML может содержать один или несколько элементов, определенных в его атрибуте `class`. Селектор класса соответствует любому элементу, к которому применен этот класс.

```
<div class="my-class"></div>
<button class="my-class"></button>
<p class="my-class"></p>
```

Любой элемент, к которому применен этот класс, будет окрашен в красный цвет:

```
.my-class {
  color: red;
}
```

Обратите внимание на то, что символ. имеется только в CSS, но не в HTML. Это связано с тем, что символ. указывает языку CSS, что необходимо сопоставить члены атрибутов класса.

Это распространенный шаблон в CSS, когда специальный символ или набор символов используется для определения типов селекторов.

Элемент HTML, имеющий класс `.my-class`, по-прежнему будет соответствовать указанному выше правилу CSS, даже если у него есть несколько других классов. Пример:

```
<div class="my-class another-class some-other-class"></div>
```

Это связано с тем, что CSS ищет атрибут `class`, который *содержит* определенный класс, а не точно соответствует этому классу.

Значение атрибута класса может быть практически любым. Единственное, что может ввести в заблуждение, — то, что класс (или идентификатор) не должен начинаться с числа, например так: `.1element`.

4. Селектор идентификатора

Элемент HTML с атрибутом `id` должен быть единственным элементом на странице с нужным значением идентификатора. Выбирайте элементы с помощью селектора идентификатора следующим образом:

```
#rad {
  border: 1px solid blue;
}
```

Этот код CSS задает синюю границу для элемента HTML, у которого атрибут `id` имеет значение `rad`. Например:

```
<div id="rad"></div>
```

Как и в случае с селектором класса, используйте символ `#`, чтобы указать CSS, что нужно искать элемент, соответствующий следующему за ним атрибуту `id`.

Если браузер найдет несколько экземпляров атрибута `id`, он все равно будет применять все правила CSS, соответствующие селектору этого атрибута. Тем не менее у любого элемента с атрибутом `id` должно быть уникальное значение этого атрибута, поэтому если мы не пишем очень специфический код CSS для одного элемента, лучше не применять стили с использованием селектора `id`, так как мы не сможем повторно использовать эти стили в другом месте.

5. Селектор атрибута

С помощью селектора атрибута можно выполнять поиск элементов, которые имеют определенный атрибут HTML или определенное значение атрибута HTML. Чтобы указать CSS, что нужно найти атрибуты, заключите селектор в квадратные скобки (`[ ]`).

```
[data-type='primary'] {
  color: red;
}
```

Этот код CSS ищет все элементы, у которых есть атрибут `data-type` со значением `primary`. Пример:

```
<div data-type="primary"></div>
```

Вместо того, чтобы искать конкретное значение атрибута `data-type`, можно также искать элементы, у которых имеется этот атрибут независимо от его значения.

```
[data-type] {
  color: red;
}
<div data-type="primary"></div>
<div data-type="secondary"></div>
```

Текст в обоих этих элементах `<div>` будет красным. Допустимо использовать селекторы атрибутов с учетом регистра, добавляя в них оператор `s`.

```
[data-type='primary' s] {
  color: red;
}
```

Это означает, что, если бы у элемента HTML атрибут `data-type` имел значение `Primary`, а не `primary`, текст в нем не был бы красным. Можно сделать наоборот — отключить функцию учета регистра — с помощью оператора `i`. Наряду с операторами case можно использовать операторы, которые сопоставляют части строк внутри значений атрибутов.

```
/* Элемент href, содержащий адрес "example.com" */
[href*='example.com'] {
  color: red;
}
/* Элемент href, начинающийся с https */
[href^='https'] {
  color: green;
}
```

```
/* Элемент href, оканчивающийся на .com */
[href$='.com'] {
  color: blue;
}
```

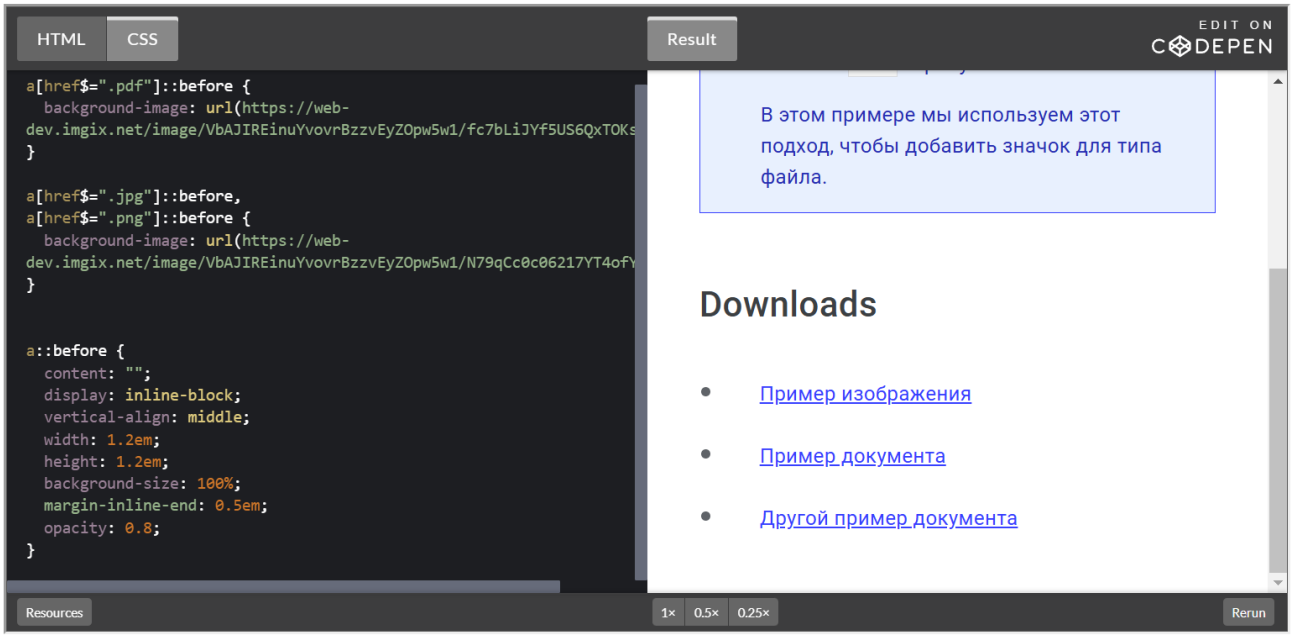


Рис. 6.10. Применение селекторов атрибута

В этой демонстрации оператор `$` в селекторе атрибута получает значение типа файла из атрибута `href`. Это позволяет добавить префикс к подписи на основе этого типа файла с помощью `псевдоэлемента`.

Группирование селекторов

Селектор необязательно должен соответствовать только одному элементу. Можно сгруппировать несколько селекторов, разделив их запятыми:

```
strong,
em,
.my-class,
[lang] {
  color: red;
}
```

В этом примере изменяется цвет как элементов `<strong>`, так и элементов `<em>`. Он также расширен и включает класс `.my-class` и элемент с атрибутом `lang`.

Псевдоклассы и псевдоэлементы

В CSS имеются полезные типы селекторов, которые ориентированы на определенное состояние платформы, например на структуры *внутри* элемента или на части элемента, когда на элемент наведен указатель мыши.

Псевдоклассы

Элементы HTML могут находиться в разных состояниях, потому что с ними взаимодействует пользователь либо потому что один из их дочерних элементов находится в том или ином состоянии. Например, пользователь может навести указатель мыши как на сам элемент HTML, так и на его дочерний элемент. В таких ситуациях используйте псевдокласс `:hover`.

```
/* На нашу ссылку навели указатель мыши */
a:hover {
  outline: 1px dotted green;
}

/* Настройка другого цвета фона для всех четных абзацев */
p:nth-child(even) {
  background: floralwhite;
}
```

Псевдоэлемент

Псевдоэлементы отличаются от псевдоклассов тем, что они не реагируют на состояние платформы, а действуют так, как если бы они вставляли новый элемент с помощью CSS. Кроме того, синтаксис псевдоэлементов отличается от синтаксиса псевдоклассов, потому что вместо одинарного двоеточия (`:`) в них используется двойное двоеточие (`::`). Двойное двоеточие (`::`) отличает псевдоэлемент от псевдокласса, но так как этого различия не было в старых версиях спецификаций CSS, браузеры поддерживают одинарное двоеточие для исходных псевдоэлементов, например `:before` и `:after`. Это обеспечивает обратную совместимость со старыми браузерами, например IE8.

```
.my-element::before {
  content: 'Prefix - ';
}
```

Как и в приведенном выше примере, в котором вы указали тип файла в качестве префикса подписи ссылки, с помощью псевдоэлемента `::before` можно вставить содержимое **в начало элемента**, а с помощью псевдоэлемента `::after` — в **конец элемента**. Перечень функций псевдоэлементов не ограничен вставкой содержимого. Их

также можно использовать для нацеливания на определенные части элемента. Предположим, у вас есть список. С помощью псевдоэлемента `::marker` можно применить стиль к каждому маркеру (или номеру) в списке.

```
/* Теперь в списке будут либо красные точки, либо красные номера */
li::marker {
  color: red;
}
```

Кроме того, с помощью псевдоэлемента `::selection` можно применять стили к содержимому, выделенному пользователем.

```
::selection {
  background: black;
  color: white;
}
```

Сложные селекторы

Мы уже познакомились с большим количеством селекторов, но иногда требуется более детальный контроль с помощью CSS. В этом случае на помощь приходят **сложные селекторы**. Следует сказать, что хотя сложные селекторы предоставляют больше возможностей, но применяя их мы можем выполнять только **каскадирование вниз**, выбирая дочерние элементы. Нацеливаться вверх и выбирать родительские элементы не получится. Что такое каскад и как он работает мы рассмотрим в следующей теме.

Комбинаторы

***Комбинатор** — это то, что находится между двумя селекторами. Например, если используется селектор `p > strong`, то комбинатором будет символ `>`.*

Селекторы, в которых используются такие комбинаторы, позволяют выбирать элементы в зависимости от их расположения в документе.

1. **Комбинатор потомков**

Чтобы понять, что такое **комбинаторы потомков**, нужно сначала разобраться, что представляют собой **родительские** и **дочерние** элементы. Рассмотрим следующий пример.

```
<p>Абзац текста, часть которого <strong>выделена полужирным шрифтом</strong>.</p>
```

Родительский элемент *— это элемент* `<p>`*, который содержит текст. Внутри этого элемента* `<p>` *находится элемент* `<strong>`*, благодаря которому шрифт заключенного в нем текста становится полужирным. Так как он находится внутри элемента* `<p>`*, он представляет собой* ***дочерний элемент*** *этого элемента.*

Комбинатор потомков позволяет нам нацеливаться на дочерний элемент. В нем используется пробел ( ), указывающий браузеру искать дочерние элементы:

```
p strong {
  color: blue;
}
```

Этот фрагмент кода выбирает все элементы `<strong>`, которые являются дочерними элементами только для элементов `<p>` и рекурсивно делает их синими.

Так как комбинатор потомков является **рекурсивным**, то к каждому дочернему элементу будет добавлен отступ, что приведет к эффекту «ступенек». Этот эффект визуализирован в примере выше, в котором используется селектор комбинаторов `.top div`. Это правило CSS добавляет левый отступ к этим элементам `<div>`. К всем элементам `<div>`, находящимся в элементе `.top`, будет применен одинаковый отступ.

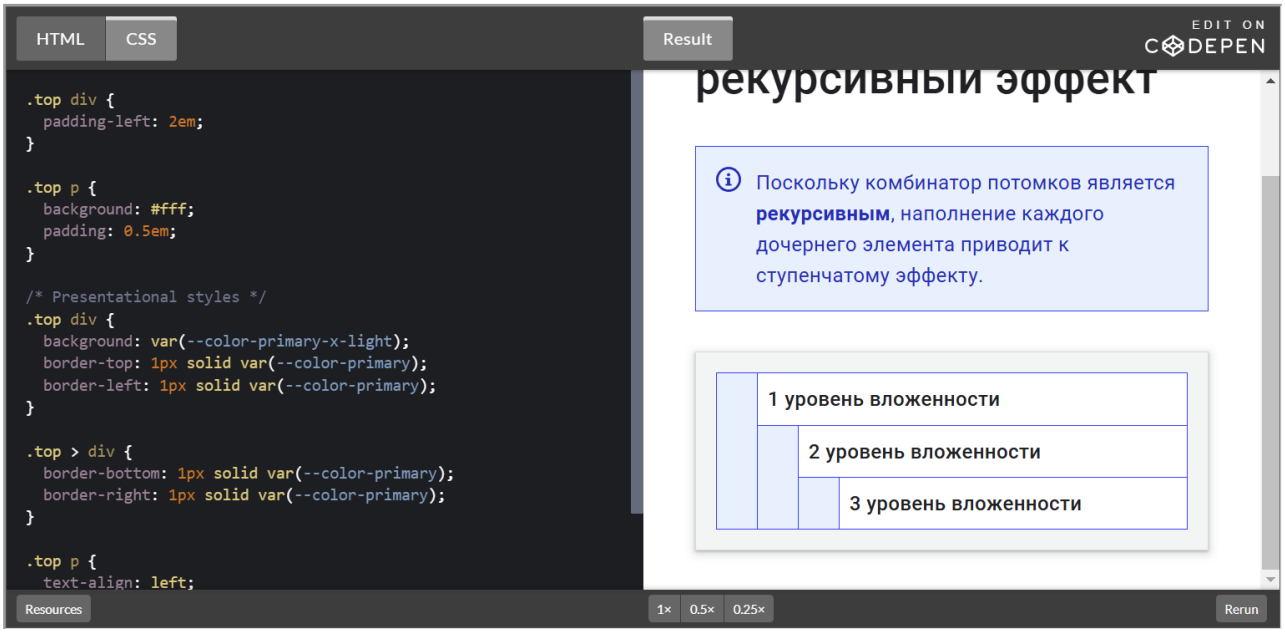


Рис. 6.11. Комбинатор потомков и рекурсивный эффект

Посмотрите на панель HTML в этой демонстрации, и вы увидите, что элемент `.top` имеет несколько дочерних элементов `<div>`, у которых, в свою очередь, есть дочерние элементы `<div>`.

2. Комбинатор следующего элемента одного уровня

С помощью символа `+` в селекторе можно выполнить поиск элемента, который следует непосредственно за другим элементом.

Чтобы добавить пробел между элементами в стопке, используется комбинатор следующего элемента одного уровня, только если элемент является **следующим элементом одного уровня** для дочернего элемента `.top`.

С помощью следующего селектора можно добавить поле ко всем дочерним элементам `.top`:

```
.top * {
  margin-top: 1em;
}
```

Проблема заключается в том, что при выборе каждого дочернего элемента `.top`, правило может создать дополнительное ненужное пространство.

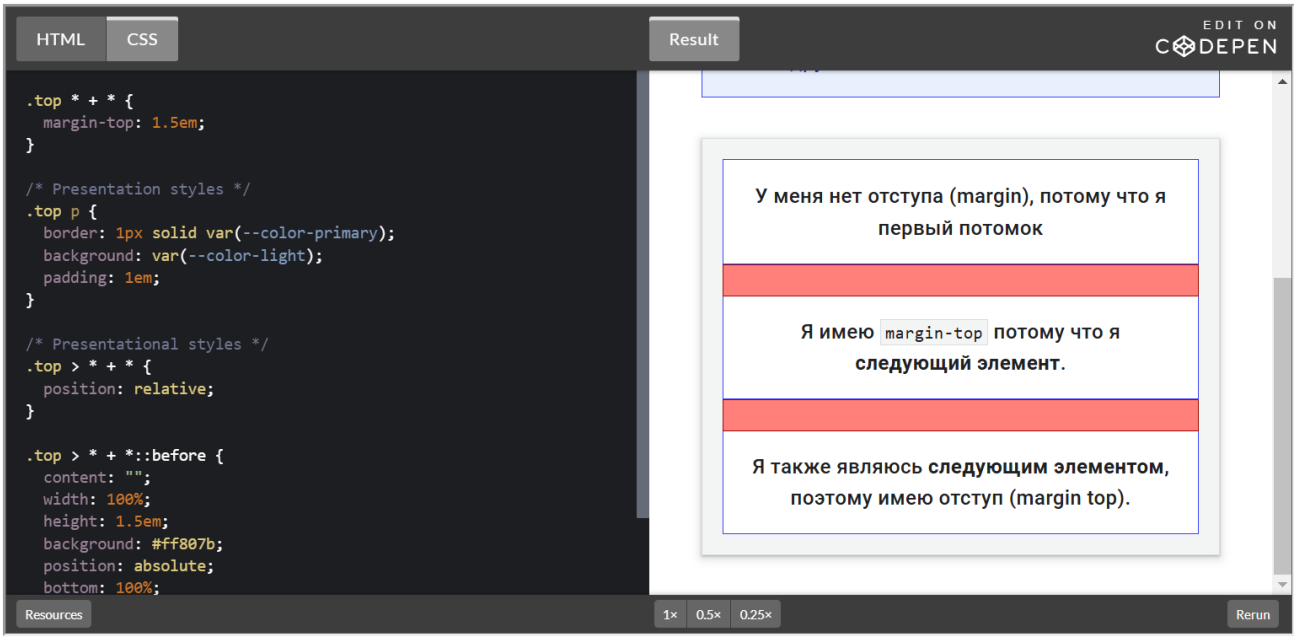


Рис. 6.12. Комбинатор следующего элемента

Комбинатор следующего элемента одного уровня в сочетании с **универсальным селектором** позволяет не только управлять тем, для каких элементов будет добавлено пространство, но и добавлять пространство к **любому элементу**. Это дает некоторую гибкость в долгосрочной перспективе независимо от того, какие элементы HTML будут встречаться в элементе `.top`.

3. Комбинатор последующего элемента

Комбинатор последующего элемента очень похож на селектор следующего элемента одного уровня. Однако вместо символа `+` здесь используется символ `~`. Отличие заключается в том, что элемент просто должен следовать за другим элементом с одинаковым родительским элементом, а не быть следующим элементом с тем же родителем. С помощью селектора последующего элемента и псевдокласса `:checked` можно создать «чистый» элемент переключателя CSS.

Этот комбинатор последующего элемента обеспечивает немного меньшую жесткость, что полезно в контекстах, аналогичных тому, который приведен в примере выше. В этом контексте мы меняем цвет настраиваемого переключателя, если связанный с ним флажок находится в состоянии `:checked`.

4. Комбинатор дочернего элемента

Комбинатор дочернего элемента (также называемый **прямым потомком**) предоставляет дополнительные возможности управления рекурсией, которая применяется в селекторах комбинаторов. С помощью символа `>` можно использовать селектор комбинатора **только** для непосредственных дочерних элементов.

Рассмотрим предыдущий пример с комбинатором следующего элемента одного уровня. Код добавляет пространство к каждому **следующему элементу одного уровня**, но если у одного из этих элементов также есть **следующие элементы одного уровня** в качестве дочерних элементов, это может привести к добавлению нежелательного дополнительного пространства.

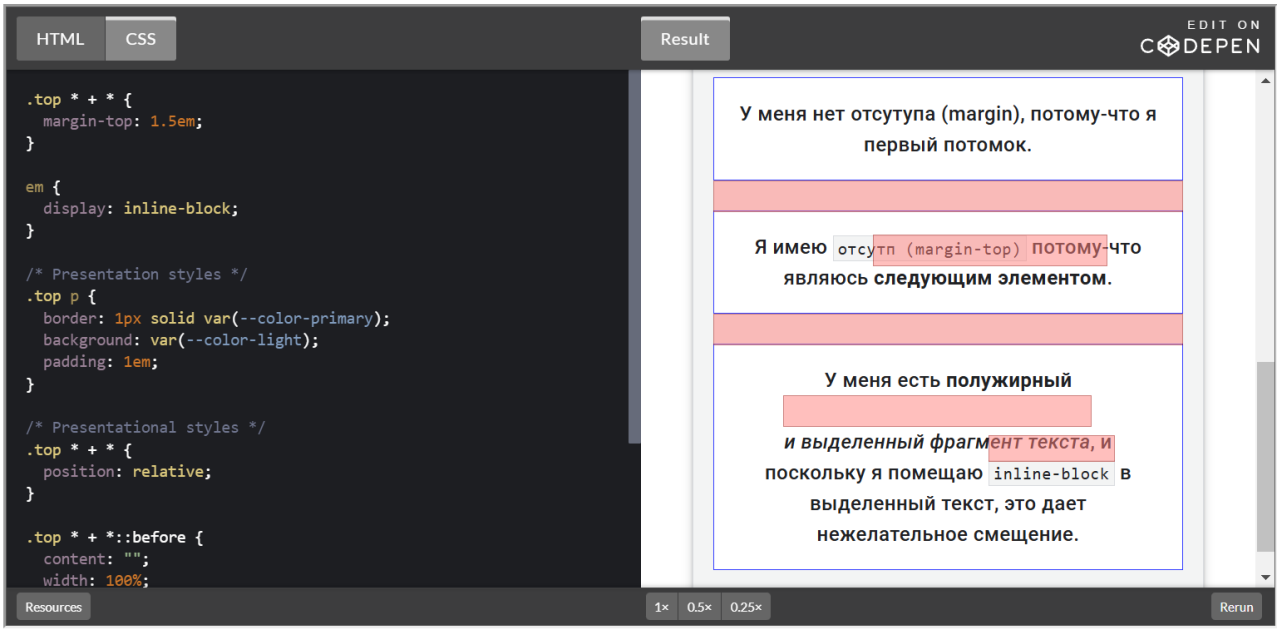


Рис. 6.13. Пример добавления пространства

Чтобы смягчить последствия этой проблемы, нужно изменить **селектор следующего элемента одного уровня**, включив в него комбинатор дочернего элемента `> *` `+ *`. Теперь это правило будет применяться **только** к непосредственным дочерним элементам элемента `.top`.

Рис. 6.14. Применение селектора к дочерним элементам

Составные селекторы

На практике можно сочетать селекторы, чтобы повысить уровень конкретности и удобочитаемость. Например, чтобы выполнить нацеливание на элементы `<a>`, которые также имеют класс `.my-class`, используется следующий код:

```
a.my-class {
  color: red;
}
```

Этот код не будет применять красный цвет ко всем ссылкам; он будет применять красный цвет только к элементу `.my-class`, если он находится в элементе `<a>`.

6.3. Каскады

Появление каскадов

Мы оформляем веб-сайты с помощью CSS, что означает использование **каскадной таблицы стилей**. Но что на самом деле означает **каскадирование**?

Каскад CSS — одна из самых мощных частей CSS. Любой, кто работал над достаточно большим веб-сайтом, жаловался: **«Почему это свойство CSS не работает ?!»** И у них возникало искушение поставить `!important` туда, где это случалось.

***Каскад CSS** — это то, как браузеры разрешают конкурирующие объявления CSS.*

Каждый раз, когда мы пишем объявление CSS (или правило), оно входит в каскад CSS, который определяет, станет ли он **окончательным стилем** или нет. Чем ниже по каскаду падает декларация, тем меньше вероятность того, что она окажется окончательным стилем.

Порядок, в котором появляются правила CSS, и то, как они появляются, учитывается каскадом при разрешении конфликтов. Например, если имеются два правила с селекторами с одинаковой специфичностью - победил тот, который был объявлен последним.

Основные уровни каскада:

1. **Важность**

2. **Источник**

3. **Специфика**

4. **Позиция**

Использование DevTools для просмотра каскада

Браузерные DevTools обычно показывают все CSS, которые могут соответствовать элементу, а те, которые не используются, зачеркиваются.

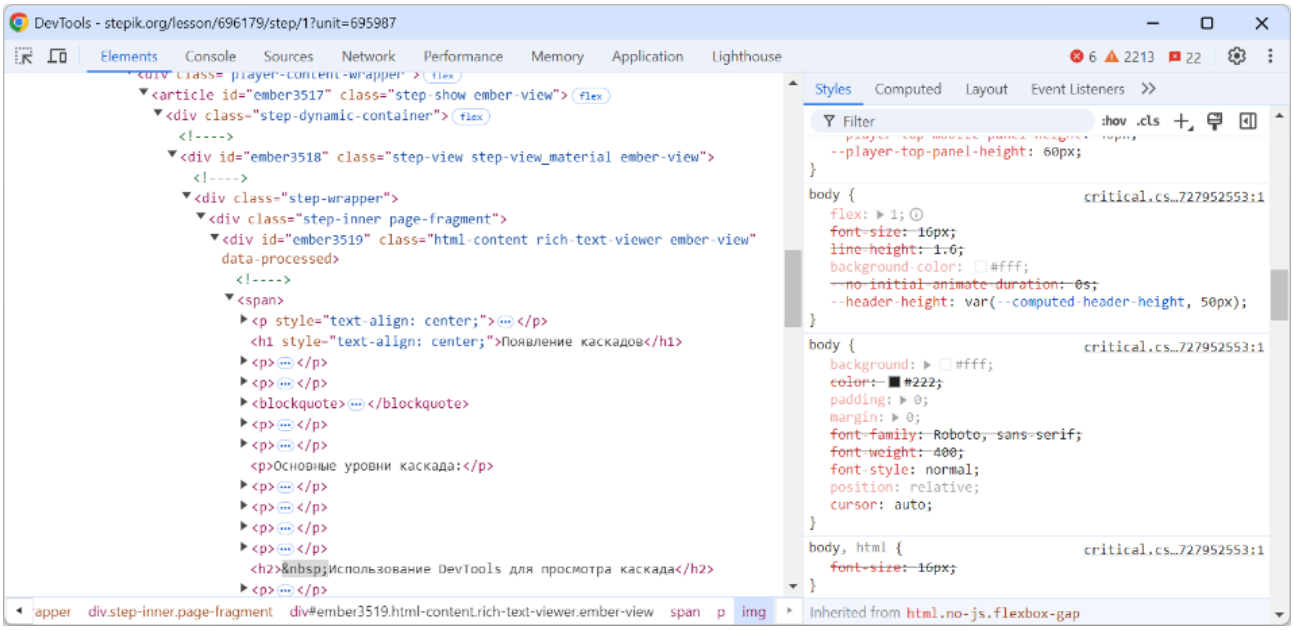


Рис. 6.15. Просмотр каскада

Если CSS, который вы ожидали применить, вообще не отображается, значит, он не соответствует элементу. В этом случае вам нужно искать в другом месте, возможно, опечатку в имени класса или элемента или какой-то неверный CSS.

Важность

Первый уровень каскада относится к типу рассматриваемого правила. Существует четыре основных **типа правил**:

1. **Правила перехода (`transition`)**, применяемые к активному переходу *и* имеющие первостепенное значение.
2. **!important** - когда мы добавляем `!important` в конец нашего объявления, оно переходит на этот уровень каскада. В идеале этот уровень резервируется для переопределения стилей из сторонних библиотек.
3. **Правила анимации (`animation`)**, применяемые к активной анимации, прыгающие вверх на уровень в каскаде.
4. **Нормальный** - на этом уровне живет основная часть правил, например, `font-size`, `background` или `color`.

Как видите, этот верхний уровень в основном зарезервирован для обеспечения правильной анимации наших элементов и для помощи отчаявшимся разработчикам (`!important`). Типы правил активной анимации и перехода имеют большее значение, чем обычные правила. Это связано с тем, что, когда анимация или переход становятся активными, их ожидаемое поведение заключается в изменении визуального состояния.

Если два правила равны на одном уровне каскада, разрешение конфликта переходит на другой следующий уровень для поиска различий.

Источник (происхождение)

Второй уровень каскада смотрит на то, где было определено правило.

CSS, который мы пишем — не единственный CSS, применяемый к странице. Каскад учитывает происхождение CSS. Этот источник включает в себя внутреннюю таблицу стилей браузера, стили, добавленные расширениями браузера или операционной системой, и наш авторский CSS.

Порядок **специфичности этих источников** от наименее специфичного к наиболее специфичному следующий:

1. **Базовые стили агента пользователя**. Это стили, которые ваш браузер применяет к элементам HTML по умолчанию.

2. **Локальные пользовательские стили**. Они могут исходить от уровня операционной системы, например, от базового размера шрифта или предпочтения уменьшения движения. Они также могут исходить из расширений браузера, таких как расширение браузера, которое позволяет пользователю писать свой собственный CSS для веб-страницы.
3. **Автор CSS** . CSS, который мы создаем.
4. **Автор** `!important` . Все `!important`, что мы добавляем к своим авторским объявлениям.
5. **Локальные пользовательские стили** `!important`. Любые `!important`, которые исходят от уровня операционной системы или уровня расширения браузера CSS.
6. **Пользовательский агент** `!important`. Любые `!important`, которые определены в CSS по умолчанию, предоставляемом браузером.

Например, вот стандартный набор стилей Chromium .

Интересный факт! Иерархия здесь фактически *перевернута* для правил `!important`. Это означает, что правило браузера по умолчанию `!important` имеет преимущество перед правилом `!important` веб-сайта (тогда как правило веб-сайта обычно имеет преимущество перед правилом браузера по умолчанию).

Специфика

Третий уровень каскада рассматривает **специфику** правила. Существует пять уровней селекторов:

1. **встроенные -** стили, объявленные в свойстве HTML `style`, являются наиболее специфичными.

2. **слой** - мы сможем определять явные «слои» стилей для преднамеренной обработки специфики. Слои «выиграют», если будут определены позже, например:

```
@layers one, two; // defining our layers
@layer one {
  body { color: red; }
}
@layer two {
  body { color: green; }
}
```

В этом примере наш основной текст будет зеленым, так как `layer two` определен после первого слоя.

3. **id -** мы можем ориентироваться на элементы на основе их `id`, используя синтаксис `#id`

4. **класс | атрибут | псевдокласс -** мы можем нацеливаться на элементы на основе их синтаксиса `class`
   Этот уровень также включает **селекторы атрибутов,** которые нацелены на атрибуты HTML, такие как `[checked]` и `[href="https://wattenberger.com"]`. Этот уровень также включает **псевдоселекторы** , такие как `:hover` и `:first-of-type`

5. **тип | псевдоэлемент - м**ы можем ориентироваться на элементы на основе их **типа тега**, используя синтаксис `type`
   Этот уровень также включает **псевдоэлементы**, такие как `:before` и `:selection`

Интересный факт! Универсальный селектор (`*`*) и комбинаторы (* `+`, `>`, `~`, `_`, `||`*) не влияют на специфичность! Больше информации в MDN.*

***Специфика (специфичность)** — это алгоритм, который определяет, какой селектор CSS является наиболее конкретным, используя систему взвешивания или оценки для выполнения этих расчетов. Делая правило более конкретным, мы можем заставить его применяться, даже если какой-либо другой CSS, соответствующий селектору, появляется позже в CSS.*

CSS, ориентированный на класс элемента, сделает это правило более конкретным и, следовательно, более важным для применения, чем CSS, ориентированный только на элемент. Это означает, что с помощью следующего CSS `h1` будет окрашен в красный цвет, даже если оба правила совпадают, а правило для `h1` селектора находится позже в таблице стилей.

```
<h1 class="my-element">Heading</h1>
.my-element {
  color: red;
}

h1 {
  color: blue;
}
```

A `id` делает CSS еще более конкретным, поэтому стили, примененные к идентификатору, переопределяют те, которые применяются многими другими способами. Это одна из причин, по которой обычно не рекомендуется присоединять стили к файлу `id`. Это может затруднить перезапись этого стиля чем-то другим.

Специфичность накапливается

Каждому типу селектора присуждаются **баллы**, которые показывают, насколько он специфичен. Баллы за все селекторы, которые мы использовали для нацеливания на элемент, складываются вместе.

Это означает, что если мы нацелитесь на элемент со списком селекторов, например, `a.my-class.another-class[href]:hover` то мы получим что-то, что довольно сложно перезаписать другим CSS. По этой причине, а также для того, чтобы сделать CSS более удобным для повторного использования, лучше делать селекторы как можно более простыми.

Позиция (положение)

Стили могут поступать из различных источников на HTML-странице, таких как тег `<link>`, встроенный тег `<style>` или встроенный CSS - через атрибут элемента `style`.

Рассмотрим пример, когда у нас есть элемент `<link>`, который включает CSS в верхней части HTML-страницы. И другой `<link>`, который включает CSS в нижней части страницы. Нижняя часть `<link>` будет иметь **наибольшую специфичность**. То же самое происходит и со встроенными элементами `<style>`. Они становятся более **конкретными**, чем дальше по странице они находятся.

Синий фон кнопки определен в CSS, который включается элементом `<link />`. Правило CSS темного фона кнопки содержится во второй связанной таблице стилей и применяется из-за его более позднего размещения.

Подобный порядок применяется также к встроенным элементам `<style>`. Если они объявлены перед `<link>`, то таблицы стилей будут иметь наибольшую **специфичность**.

Элемент `<style>` объявлен в `<head>` файле. Это означает, что он получает больше специфичности, чем элемент `<link /><body><style>` Встроенный атрибут `style` с объявленным в нем CSS переопределит все остальные CSS, независимо от его положения, если только не определено `!important`.

Позиция также применяется в порядке нашего правила CSS. В этом примере элемент будет иметь фиолетовый фон, потому что `background: purple` был объявлен последним. Поскольку зеленый фон был объявлен перед фиолетовым фоном, теперь он игнорируется браузером.

```
.my-element {
  background: green;
  background: purple;
}
```

Возможность указать два значения для одного и того же свойства может быть простым способом создания запасных вариантов для браузеров, которые не поддерживают конкретное значение.

В следующем примере `font-size` объявляется дважды. Если `clamp()` поддерживается в браузере, то предыдущее объявление `font-size` будет отброшено. Если `clamp()` не поддерживается браузером, исходное объявление будет учтено, а размер шрифта будет равен 1,5 rem.

```
.my-element {
  font-size: 1.5rem;
  font-size: clamp(1.5rem, 1rem + 3vw, 2rem);
}
```

Этот подход с двойным объявлением одного и того же свойства работает, потому что браузеры игнорируют значения, которые они не понимают.

В отличие от некоторых других языков программирования, CSS не выдаст ошибку и не сломает программу, когда обнаружит строку, которую не может проанализировать — значение, которое он не может проанализировать, является недопустимым и поэтому игнорируется. Затем браузер продолжает обрабатывать остальную часть CSS, не нарушая то, что он уже понимает.

Оценка специфичности

Предположим, мы работаем со следующими HTML и CSS:

```
<button class="branding">Привет, специфичность!</button>
button {
  color: red;
}

.branding {
  color: blue;
}
```

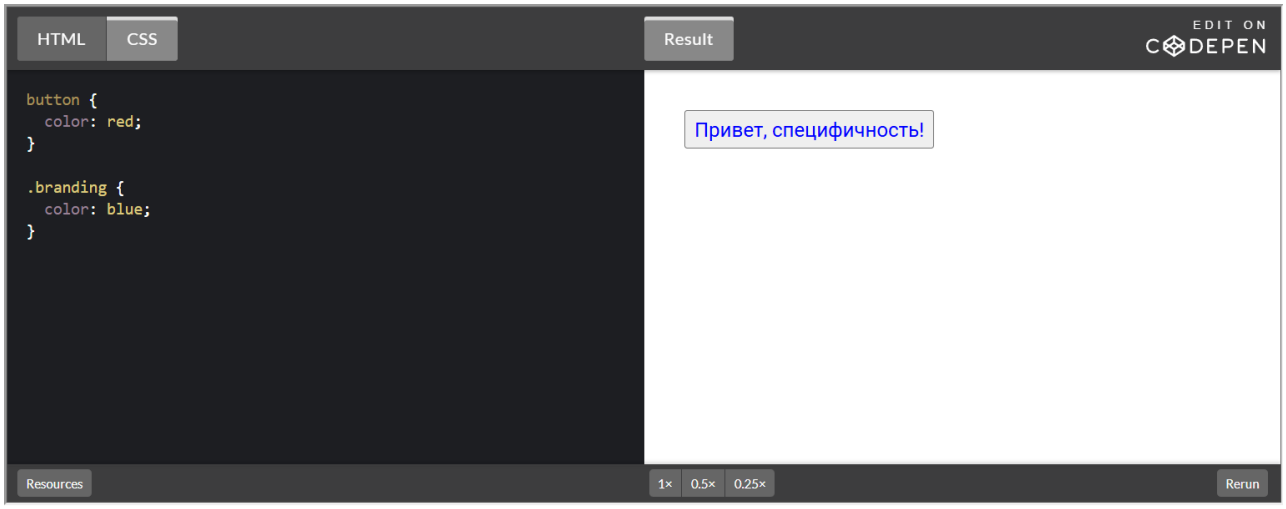


Рис. 6.16. Конкурирующие правила

Здесь есть два конкурирующих правила. Один окрасит кнопку в красный цвет, а другой — в синий. Какое правило применяется к элементу? Понимание алгоритма специфичности CSS является ключом к пониманию того, как CSS выбирает между конкурирующими правилами.

Оценка специфичности

Каждое правило селектора получает оценку. Вы можете думать о специфичности как об общем балле, и каждый тип селектора зарабатывает баллы для этого балла. Выигрывает селектор с наибольшим количеством очков.

Со спецификой реального проекта балансировка заключается в том, чтобы убедиться, что правила CSS, которые вы ожидаете применить, действительно *применяются,* при этом, как правило, сохраняя низкие оценки, чтобы предотвратить сложность. Оценка должна быть настолько высокой, насколько нам нужно, а не стремиться к максимально возможной оценке. В будущем, возможно, потребуется применить некоторые действительно более важные CSS. Если вы стремитесь к наивысшему баллу, вы усложните эту работу.

Оценка каждого типа селектора

Каждый тип селектора вносит свои баллы (веса). Эти баллы суммируются и рассчитывается общая специфичность селектора.

1. **Универсальный селектор**

Универсальный селектор (`*`) не имеет специфичности и получает **0 баллов** . Это означает, что любое правило с 1 или более баллами переопределит его.

```
* {
  color: red;
}
```

2. **Селектор элемента или псевдоэлемента**

Селектор элемента (типа) или псевдоэлемента получает **1 балл специфичности**.

Селектор типа

```
div {
  color: red;
}
```

Селектор псевдоэлементов

```
::selection {
  color: red;
}
```

3. **Селектор класса, псевдокласса или атрибута**

Селектор класса, псевдокласса или атрибута получает **10 баллов специфичности**.

Селектор класса

```
.my-class {
  color: red;
}
```

Селектор псевдокласса

```
:hover {
  color: red;
}
```

Селектор атрибутов

```
[href='#'] {
  color: red;
}
```

Сам псевдокласс `:not()` ничего не добавляет к расчету специфичности. Однако селекторы, переданные в качестве аргументов, добавляются к расчету специфичности.

```
div:not(.my-class) {
  color: red;
}
```

В этом примере общий селектор будет иметь **11 пунктов** специфичности, потому что он имеет один селектор типа и один класс `div` внутри `:not()`

4. **Селектор идентификатора**

Селектор ID получает **100 баллов специфичности** , если мы используем селектор ID (`#myID`), а не селектор атрибутов (`[id="myID"]`).

```
#myID {
  color: red;
}
```

5. **Атрибут встроенного стиля**

CSS, примененный непосредственно к атрибуту `style` HTML-элемента, получает **оценку специфичности 1000 баллов**. Это означает, что для того, чтобы переопределить его в CSS, мы должны написать очень специфичный селектор.

```
<div style="color: red"></div>
```

6. **Правило !important**

Наконец, элемент `!important` в конце значения CSS получает показатель специфичности в **10 000 баллов**. Это самая высокая специфичность, которую может получить один отдельный элемент.

Правило `!important` применяется к свойству CSS, поэтому все в общем правиле (селектор и свойства) не получают одинаковой оценки специфичности.

```
.my-class {
  color: red !important; /* 10,000 баллов */
  background: white; /* 10 баллов */
}
```

Специфичность в контексте

Специфика каждого селектора, который соответствует элементу, суммируется. Рассмотрим этот пример HTML:

```
<a class="my-class another-class" href="#">Это ссылка</a>
```

Эта ссылка имеет два класса. Добавляем следующий CSS, и он получит **1 балл специфичности**:

```
a {
```

```
  color: red;
}
```

Сошлемся на один из классов в этом правиле, теперь он имеет **11 баллов специфичности**:

```
a.my-class {
  color: green;
}
```

Добавим другой класс в селектор, теперь он имеет **21 балл специфичности**:

```
a.my-class.another-class {
  color: rebeccapurple;
}
```

Добавим `href` атрибут в селектор, теперь он имеет **31 балл специфичности**:

```
a.my-class.another-class[href] {
  color: goldenrod;
}
```

Наконец, добавим ко всему этому псевдокласс `:hover`, и селектор получит **41 балл специфичности**:

```
a.my-class.another-class[href]:hover {
  color: lightgrey;
}
```

Визуализация специфичности

В диаграммах и калькуляторах специфичности часто специфичность визуализируют следующим образом:

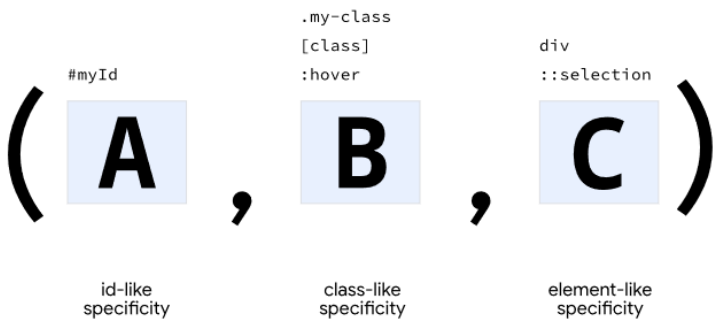


Рис. 6.17. Визуализация специфичности

- Левая группа — селекторы `id`.
- Вторая группа — это селекторы классов, атрибутов и псевдоклассов.
- Последняя группа — это селекторы элементов и псевдоэлементов.

Для справки, следующий селектор `0-4-1`:

```
a.my-class.another-class[href]:hover {
  color: lightgrey;
}
```

Увеличение специфичности

Допустим, у нас есть некоторый CSS, который выглядит так:

```
.my-button {
  background: blue;
}
button[onclick] {
  background: grey;
}
```

С HTML это выглядит так:

```
<button class="my-button" onclick="alert('Привет')">Нажми меня</button>
```

Кнопка имеет серый фон, потому что второй селектор получает **11 баллов специфичности** (`0-1-1`). Это связано с тем, что у него есть один селектор типа (`button`), который равен **1 баллу** , и селектор атрибута (`[onclick]`), который равен **10 баллам**. Предыдущее правило — `.my-button` получает **10 баллов** (`0-1-0`), потому что оно имеет один селектор класса. Если мы хотим усилить это правило, нужно повторить выбор класса следующим образом:

```
.my-button.my-button {
  background: blue;
}

button[onclick] {
  background: grey;
}
```

Теперь у кнопки будет синий фон, потому что новый селектор получает **20 баллов** за специфичность (`0-2-0`).

Если в процессе кодирования приходится слишком часто повышать специфичность, это указывает на то, что мы пишем слишком специфичные селекторы. В этом случае нужно реорганизовать CSS, чтобы уменьшить специфичность других селекторов и избежать этой проблемы.

При совпадающем показателе специфичности выигрывает самый новый экземпляр. Давайте пока остановимся на примере кнопки и посмотрим следующее:

```
.my-button {
  background: blue;
}

[onclick] {
  background: grey;
}
```

Кнопка имеет серый фон, поскольку **оба селектора имеют одинаковую оценку специфичности** (`0-1-0`). Если мы поменяем местами правила, то кнопка станет синей.

```
[onclick] {
  background: grey;
}

.my-button {
  background: blue;
}
```

Это единственный случай, когда побеждает более новый CSS. Для этого он должен соответствовать специфичности другого селектора, нацеленного на тот же элемент.

6.4. Наследование в CSS

Наследование свойств CSS

Рассмотрим пример CSS, которая создает эффект элементов в виде кнопок:

```
<a href="http://example.com" class="my-button">Я кнопка со ссылкой</a>
.my-button {
  display: inline-block;
  padding: 1rem 2rem;
  text-decoration: none;
  background: pink;
  font: inherit;
  text-align: center;
}
```

Затем добавляется элемент ссылки в статью контента со значением `.my-button`. Однако есть проблема: текст не того цвета, которого мы ожидали. Как это произошло? Некоторые свойства CSS наследуются, если мы не укажем для них значение. В случае с кнопкой цвет был унаследовал от `color` этого CSS:

```
article a {
  color: maroon;
}
```

Поток наследования

Давайте посмотрим, как работает наследование, используя этот фрагмент HTML:

```
<html>
  <body>
    <article>
      <p>Lorem ipsum dolor sit amet.</p>
    </article>
  </body>
</html>
```

Корневой элемент (`<html>`) ничего не наследует, потому что это первый элемент в документе. Добавим немного правил CSS к элементу HTML, и он начнет каскадно перемещаться по документу.

```
html {
  color: lightslategray;
}
```

Свойство `color` наследуется другими элементами. Элемент `html` имеет `color: lightslategray`, поэтому все элементы, которые могут наследовать цвет, теперь будут иметь цвет `lightslategray`.

```
body {
  font-size: 1.2em;
}
```

Поскольку здесь устанавливается размер шрифта для элемента `body`, элемент `html` по-прежнему будет иметь исходный размер шрифта, установленный браузером (таблица стилей пользовательского агента), но `article` и `p` унаследует размер

шрифта, объявленный в файле `body`*. Это связано с тем, что наследование распространяется только каскадом вниз.*

```
p {
  font-style: italic;
}
```

Только текст `<p>` будет выделен курсивом, потому что это самый глубокий вложенный элемент. Наследование происходит только вниз, а не обратно к родительским элементам.

Каждый элемент HTML имеет свойство CSS, определенное по умолчанию с начальным значением. **Начальное значение (default)** — это свойство, которое не наследуется и отображается по умолчанию, если каскаду не удается вычислить значение для этого элемента.

Свойства, которые могут быть унаследованы каскадом вниз, и дочерние элементы получат вычисленное значение, которое представляет значение его родителя. Это означает, что если родитель `font-weight` установил `bold`, то все дочерние элементы, они будут выделены полужирным шрифтом, если только для них не установлено другое значение `font-weight` или таблица стилей пользовательского агента не имеет значения `font-weight` для этого элемента.

Наследование может влиять на элементы неожиданным образом, поэтому в CSS есть инструменты, которые помогут в этом.

Ключевое слово inherit

Мы можем заставить любое свойство наследовать вычисленное значение своего родителя с помощью ключевого слова `inherit`. Полезным способом использования этого ключевого слова является создание исключений.

```
strong {
  font-weight: 900;
}
```

Этот фрагмент кода CSS задает для всех элементов `<strong>` значение `font-weight`, равное `900`, а не значение по умолчанию `bold`, которое было бы эквивалентно `font-weight: 700`.

```
.my-component {
  font-weight: 500;
}
```

Класс `.my-component` устанавливает `font-weight` вместо `500`. Чтобы сделать `<strong>` элементы внутри, `.my-component` также `font-weight: 500` добавьте:

```
.my-component strong {
  font-weight: inherit;
}
```

Теперь `<strong>` элементы внутри `.my-component` будут иметь `font-weight` размер `500`.

Вы можете явно установить это значение, но если вы будете использовать `inherit` CSS `.my-component` изменений в будущем, вы можете гарантировать, что вы `<strong>` будете автоматически обновляться с ним.

Ключевое слово initial

Наследование может вызвать проблемы с элементами и `initial` предоставляет мощный вариант сброса. Ранее мы узнали, что каждое свойство имеет значение по умолчанию в CSS. Ключевое слово `initial` возвращает свойству исходное значение по умолчанию.

```
aside strong {
  font-weight: initial;
}
```

Этот фрагмент удалит полужирный жирный шрифт у всех `<strong>` элементов внутри `<aside>` элемента и вместо этого сделает их нормальными, которые являются начальным значением.

Ключевое слово unset

Свойство `unset` ведет себя по-разному - в зависимости от того наследуется свойство или нет. Если свойство наследуется, `unset` ключевое слово будет таким же, как `inherit`. Если свойство не наследуется, `unset` ключевое слово равно `initial`.

Например, `color` наследуется, а вот `margin` - нет. Поэтому мы можем написать следующее:

```
/* Глобальные цветовые стили для абзаца в авторском CSS */
p {
  margin-top: 2em;
  color: goldenrod;
```

```
}

/* p необходимо оставить в стороне, потому-что мы можем использовать unset */
aside p {
  margin: unset;
  color: unset;
}
```

Теперь `margin` удаляется и `color` снова становится унаследованным вычисляемым значением.

Мы также можем использовать значение `unset` со свойством `all`. Возвращаясь к приведенному выше примеру, что произойдет, если глобальные стили `p` получат несколько дополнительных свойств? Только правило, которое было установлено `margin` и `color` будет применяться.

```
/* Глобальные цветовые стили для абзаца в авторском CSS */
p {
	margin-top: 2em;
	color: goldenrod;
	padding: 2em;
	border: 1px solid;
}

/* Не все свойства учитываются больше */
aside p {
	margin: unset;
	color: unset;
}
```

Если вместо этого мы изменим правило `aside p` на `all: unset`, не имеет значения, к каким глобальным стилям будут применены `p` в будущем, они всегда будут отключены.

```
aside p {
	margin: unset;
	color: unset;
	all: unset;
}
```

Горизонтальная линия средствами CSS

Для формирования горизонтальной линии можно пойти разными путями. Давайте рассмотрим некоторые из них:

Вариант №1. Можно определить обычный div элемент и покрасить у него одну из сторон с помощью свойства border.

```
    <div class="create-line"></div>
    .create-line {
      width: 200px;
       border-top: 3px solid #32a1ce;
      }
```

Вариант №2. Используем готовый тег который формирует горизонтальную линию - **hr**.

Вариант №3. С помощью псевдоэлементов **:before** и **:after**. Этот вариант немного сложнее, но за то мы обойдемся без правки HTML-кода. Итак, если нам нужно разместить вертикальную или горизонтальную линию сверху или с левой стороны элемента, то нам нужно использовать псевдоэлемент **:before**. Если же линия должна быть справа или снизу, то мы должны использовать псевдоэлемент **:after**.

```
    <div class="create-line-left">
      Добавим линию слева и справа
</div>
```

Теперь рассмотрим стили.

```
    .create-line-left {
      padding-left: 20px;
  /* необходим для формирования позиций линий */
  position: relative;
  /* весь блок будет занимать ширину по размеру контента в нём */
  display: inline-block;
}
    .create-line-left:before{
  content: " ";
  background: #33CCFF;
  width: 50px;
  height: 1px;
  /* располагаем линию сверху от текста */
  top: 15px;
  /* включаем абсолютное позиционирование относительно блока с relative */
  position: absolute;
}
    .create-line-left:after{
  content: " ";
  background: #333333;
  width: 50px;
```

```
  height: 1px;
  /* размещаем линию снизу и справа*/
  bottom: 15px;
  right: 20px;
  /* включаем абсолютное позиционирование относительно блока с relative */
  position: absolute;
}
```

Задания к практической работе

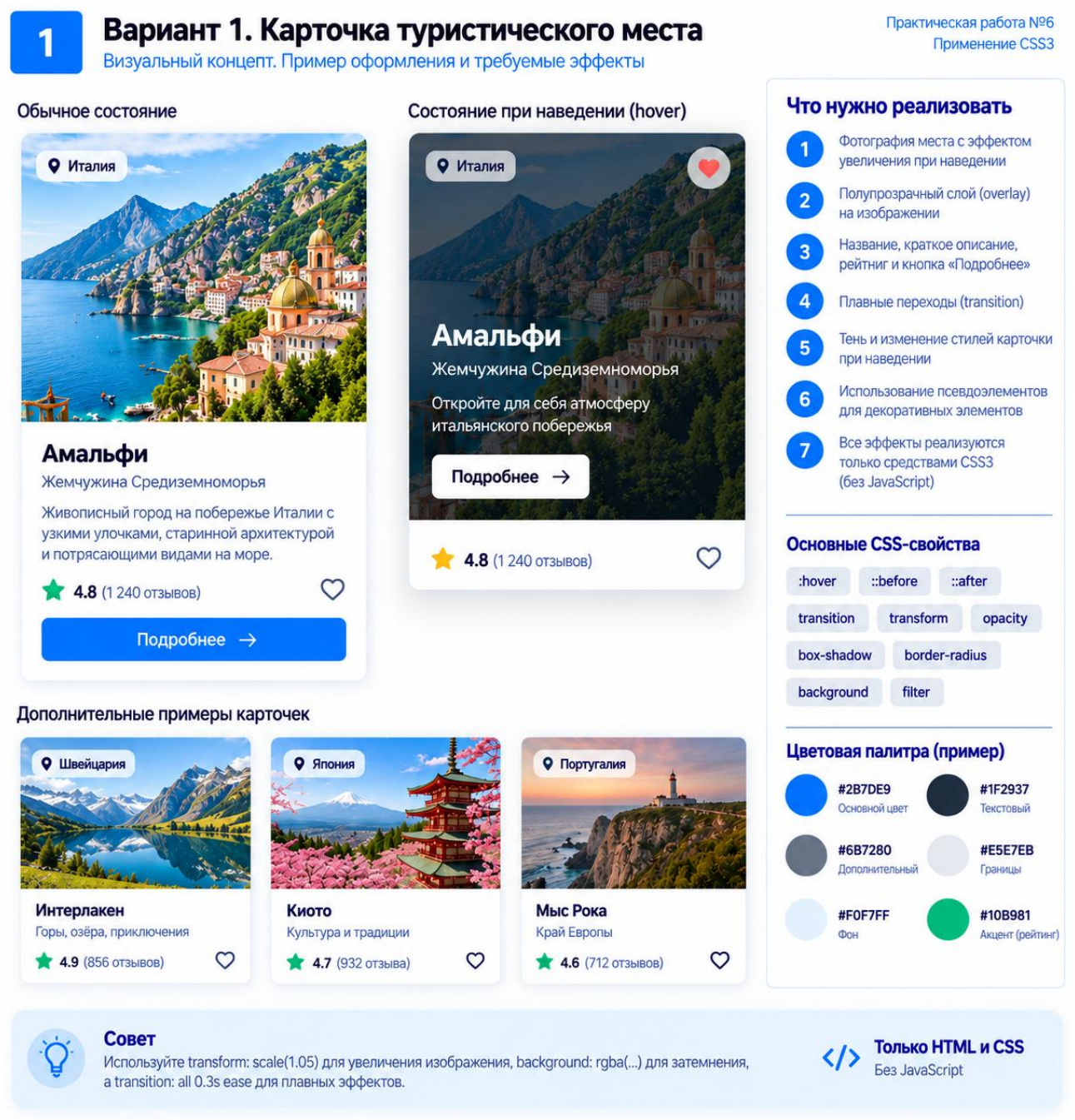
Задание 1. Определиться с номером варианта

Выбрать один вариант из 15 предложенных. Изучить соответствующий визуальный концепт. Определить смысловые области страницы и составить логическую HTML5-структуру.

Варианты индивидуальных заданий

Каждый вариант содержит предметную область, обязательные смысловые блоки, рекомендуемые HTML5-элементы и визуальные требования.

Вариант 1. Карточка туристического места



Построить карточку туристического места с фотографией, названием, кратким описанием, рейтингом и кнопкой. При наведении карточка должна переходить в интерактивное состояние: изображение немного увеличивается, поверх появляется затемняющий слой, проявляются дополнительные сведения и меняется оформление карточки.

Требуемый эффект / приём	Что требуется реализовать
Масштабирование / transform	При наведении элемент должен визуально изменять размер без разрушения сетки.
Переход / transition	Изменение состояния выполняется плавно, без резкого скачка.
Псевдоклассы	Использовать :hover и :focus там, где пользователь взаимодействует с элементом.

Псевдоэлементы	Добавить минимум один декоративный элемент через ::before или ::after.

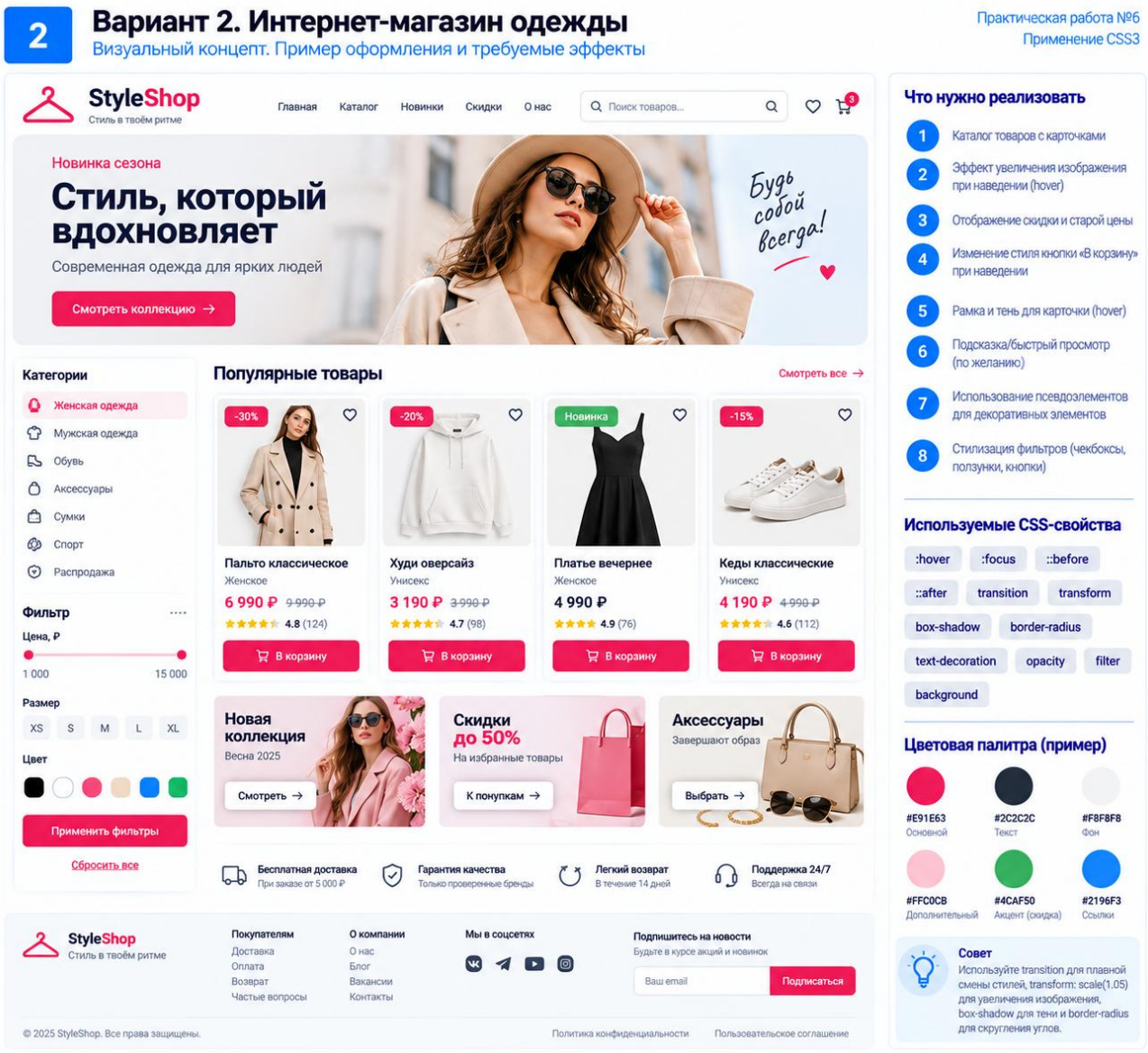
Обязательные техники варианта: transform: scale(), background: rgba() / opacity, transition, box-shadow, border-radius, ::before / ::after

Как реализовать сложный эффект. *Вложенное изображение должно увеличиваться без изменения размеров самой карточки. Затемняющий слой следует реализовать отдельным псевдоэлементом, а текстовый overlay — поверх изображения.*

Контрольный результат варианта

- Обычное состояние страницы соответствует назначению макета.
- Hover-эффекты видимы и не ломают размеры соседних блоков.
- Интерактивные элементы имеют :focus.
- Псевдоэлемент используется не формально, а для реального визуального элемента.
- CSS-код остаётся читаемым: классы имеют понятные имена, а повторяющиеся значения по возможности вынесены в переменные.

Вариант 2. Интернет-магазин одежды



Создать страницу каталога одежды с карточками товаров, ценами, скидками, фильтрами и кнопками. При наведении на товар требуется показать интерактивное состояние: увеличить изображение, выделить карточку, изменить вид кнопки и проявить декоративный элемент.

Требуемый эффект / приём	**Что требуется реализовать**
Масштабирование / transform	При наведении элемент должен визуально изменять размер без разрушения сетки.
Переход / transition	Изменение состояния выполняется плавно, без резкого скачка.
Псевдоклассы	Использовать :hover и :focus там, где пользователь взаимодействует с элементом.
Псевдоэлементы	Добавить минимум один декоративный элемент через ::before или ::after.

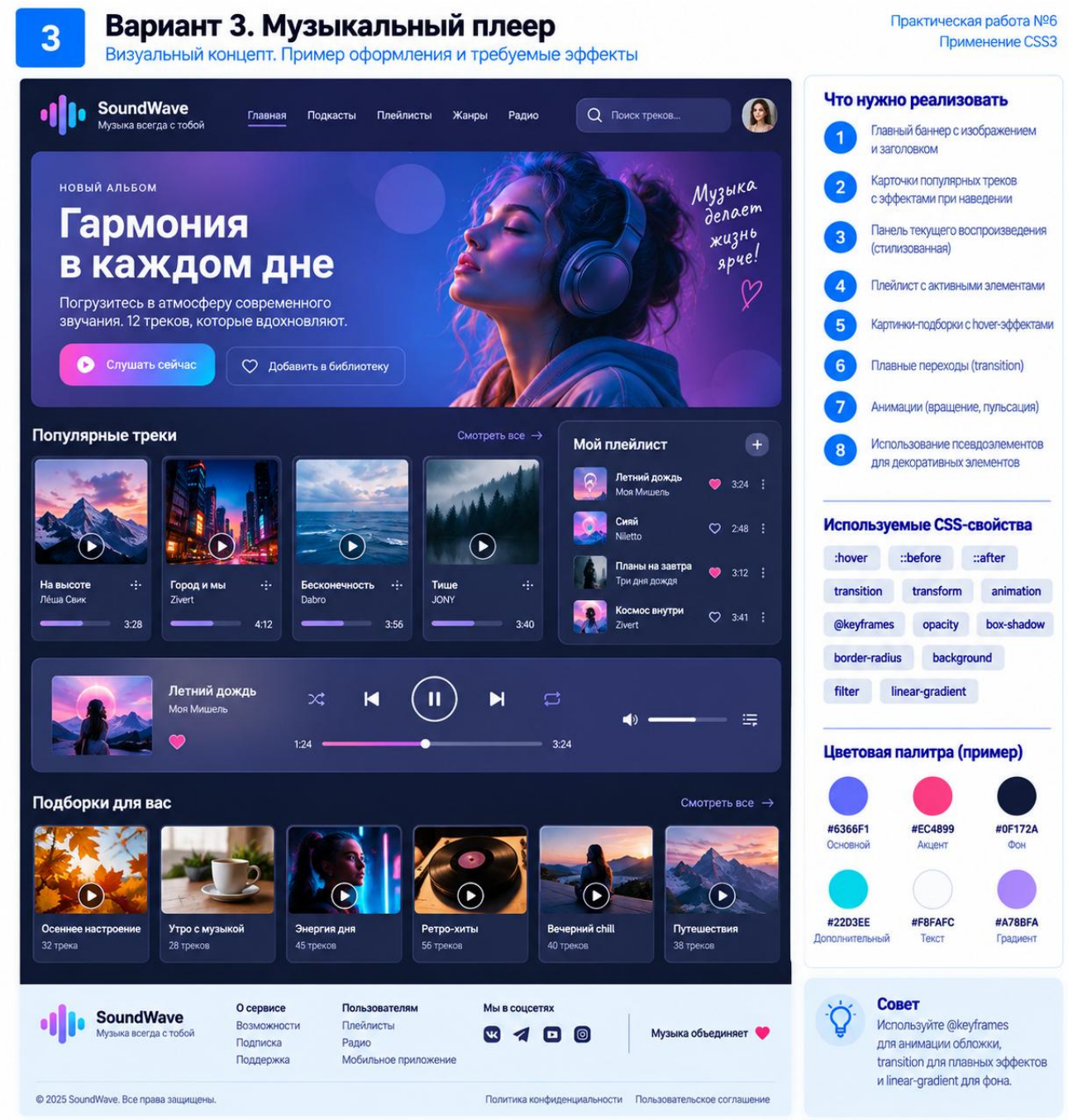
Обязательные техники варианта: transform: scale(), transition, box-shadow, opacity, :hover / :focus, ::before / ::after

Как реализовать сложный эффект *Сделать эффект «поднятия» карточки без изменения потока документа: transform: translateY(-...) и box-shadow. Для скидки использовать псевдоэлемент или отдельный бейдж.*

Контрольный результат варианта

- Обычное состояние страницы соответствует назначению макета.
- Hover-эффекты видимы и не ломают размеры соседних блоков.
- Интерактивные элементы имеют :focus.
- Псевдоэлемент используется не формально, а для реального визуального элемента.
- CSS-код остаётся читаемым: классы имеют понятные имена, а повторяющиеся значения по возможности вынесены в переменные.

Вариант 3. Музыкальный плеер



Оформить интерфейс музыкального сервиса: обложка, название композиции, исполнитель, панель управления, прогресс и подборка треков. Использовать визуальную анимацию для прогресс-бара и активного элемента, не изменяя логику воспроизведения.

Требуемый эффект / приём	Что требуется реализовать
Масштабирование / transform	При наведении элемент должен визуально изменять размер без разрушения сетки.
Переход / transition	Изменение состояния выполняется плавно, без резкого скачка.

Псевдоклассы	Использовать :hover и :focus там, где пользователь взаимодействует с элементом.
Псевдоэлементы	Добавить минимум один декоративный элемент через ::before или ::after.

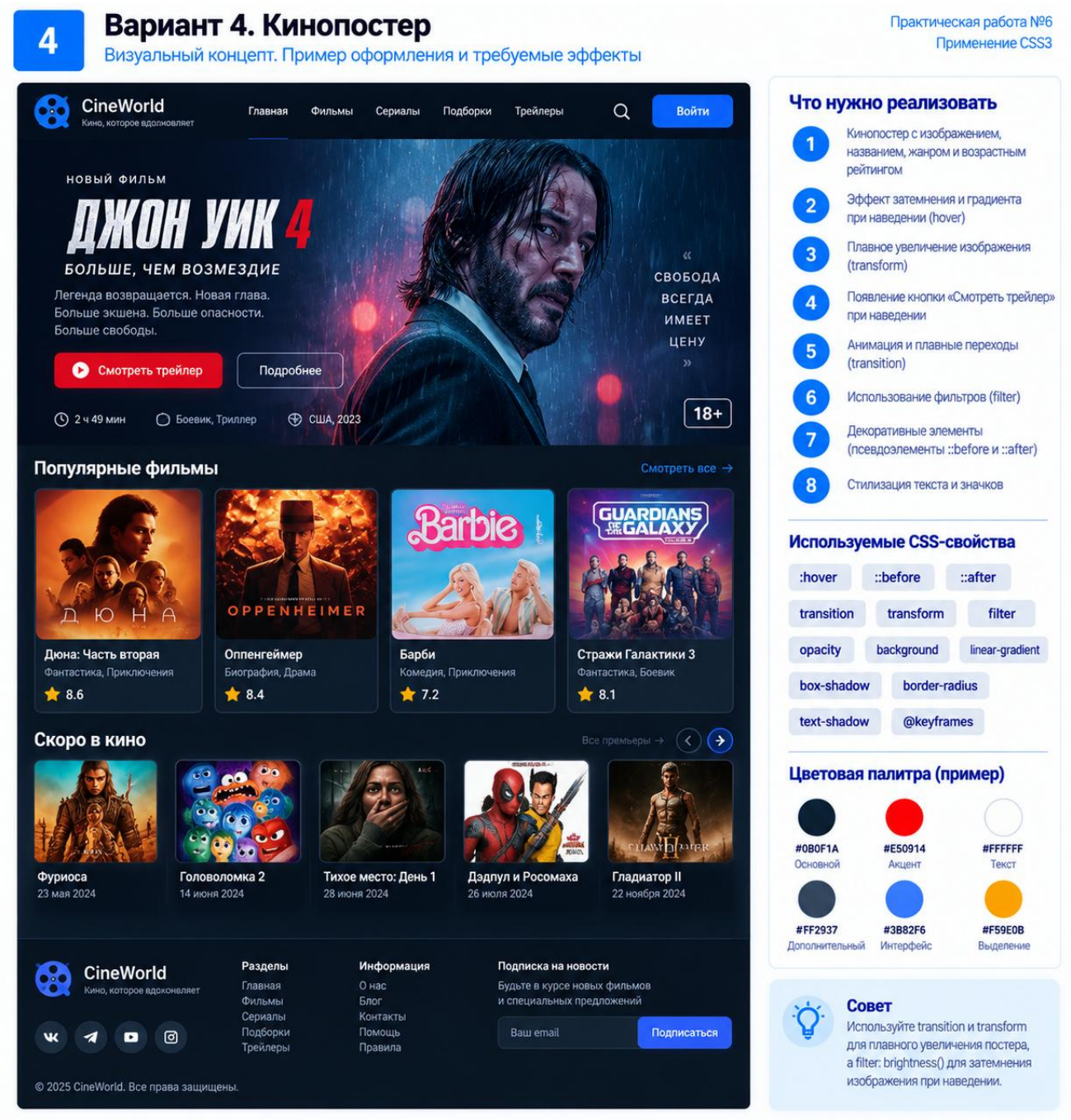
Обязательные техники варианта: @keyframes, transition, transform, linear-gradient, box-shadow, ::before / ::after

Как реализовать сложный эффект *Для сложного эффекта использовать @keyframes только для визуальной индикации. Прогресс можно оформить градиентом, а активный трек — псевдоэлементом-индикатором.*

Контрольный результат варианта

- Обычное состояние страницы соответствует назначению макета.
- Hover-эффекты видимы и не ломают размеры соседних блоков.
- Интерактивные элементы имеют :focus.
- Псевдоэлемент используется не формально, а для реального визуального элемента.
- CSS-код остаётся читаемым: классы имеют понятные имена, а повторяющиеся значения по возможности вынесены в переменные.

Вариант 4. Кинопостер



Создать страницу-подборку фильмов с крупным постером, жанром, рейтингом, описанием и кнопкой. При наведении постер должен плавно увеличиваться, появляться цветовой overlay, кнопка и дополнительные сведения.

Требуемый эффект / приём	Что требуется реализовать
Масштабирование / transform	При наведении элемент должен визуально изменять размер без разрушения сетки.
Переход / transition	Изменение состояния выполняется плавно, без резкого скачка.
Псевдоклассы	Использовать :hover и :focus там, где пользователь взаимодействует с элементом.

Псевдоэлементы	Добавить минимум один декоративный элемент через ::before или ::after.

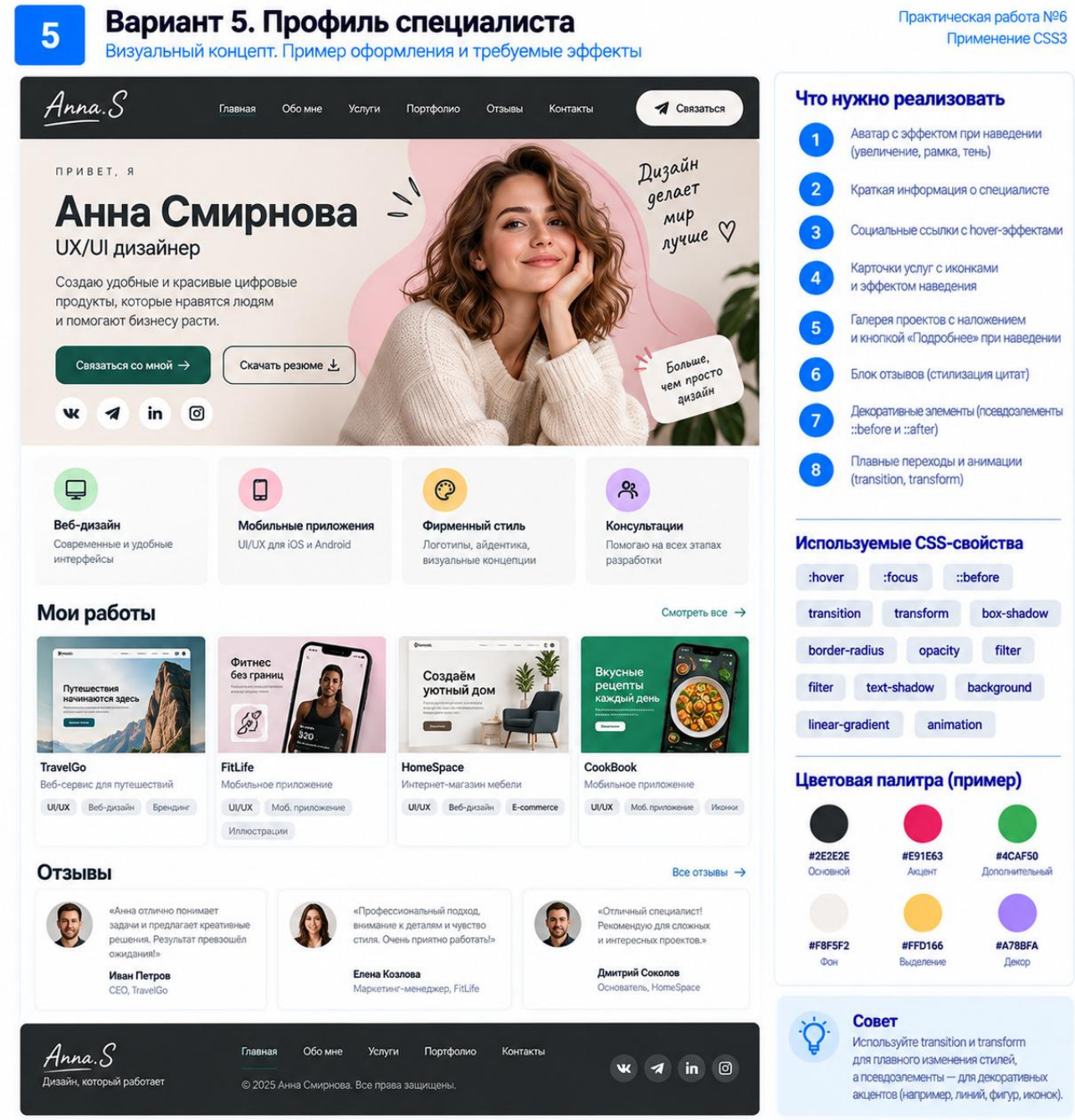
Обязательные техники варианта: filter, transform: scale(), opacity, transition, background: linear-gradient(), ::before / ::after

Как реализовать сложный эффект *Фоновый overlay лучше делать через ::before. Для изображения допускается filter: brightness(...) или grayscale(...), а восстановление эффекта — в состоянии :hover.*

Контрольный результат варианта

- Обычное состояние страницы соответствует назначению макета.
- Hover-эффекты видимы и не ломают размеры соседних блоков.
- Интерактивные элементы имеют :focus.
- Псевдоэлемент используется не формально, а для реального визуального элемента.
- CSS-код остаётся читаемым: классы имеют понятные имена, а повторяющиеся значения по возможности вынесены в переменные.

Вариант 5. Профиль специалиста



Разработать персональную карточку специалиста с фотографией, должностью, краткой биографией, навыками и кнопкой связи. Предусмотреть изменение изображения/рамки/тени при наведении и стилизованный индикатор статуса.

Требуемый эффект / приём	**Что требуется реализовать**
Масштабирование / transform	При наведении элемент должен визуально изменять размер без разрушения сетки.
Переход / transition	Изменение состояния выполняется плавно, без резкого скачка.

Псевдоклассы	Использовать :hover и :focus там, где пользователь взаимодействует с элементом.
Псевдоэлементы	Добавить минимум один декоративный элемент через ::before или ::after.

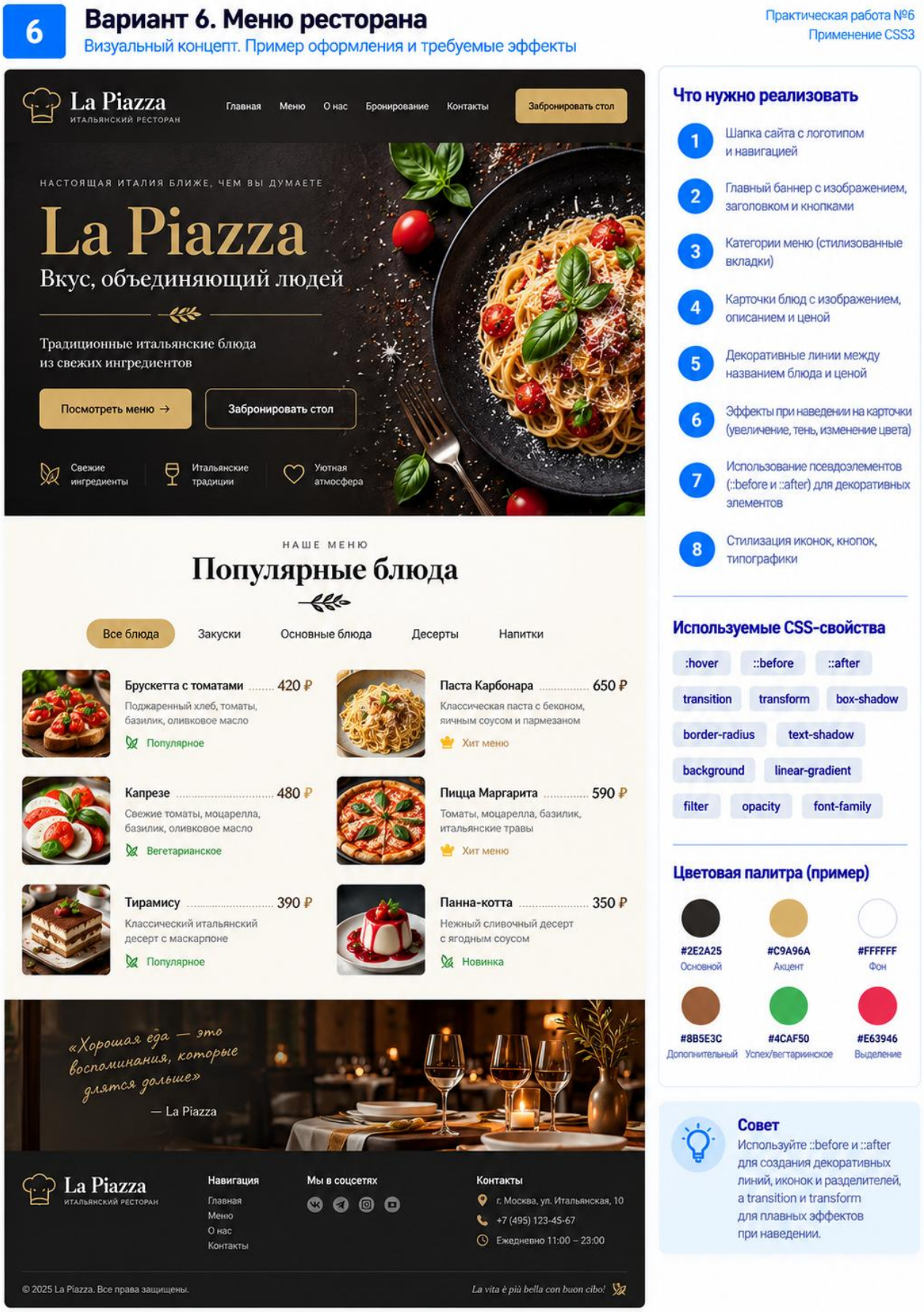
Обязательные техники варианта: border / outline, box-shadow, transform, transition, ::before / ::after, :hover / :focus

Как реализовать сложный эффект *Статус можно сделать псевдоэлементом с круглым индикатором. Для фотографии использовать overflow: hidden и transform: scale() на img при наведении.*

Контрольный результат варианта

- Обычное состояние страницы соответствует назначению макета.
- Hover-эффекты видимы и не ломают размеры соседних блоков.
- Интерактивные элементы имеют :focus.
- Псевдоэлемент используется не формально, а для реального визуального элемента.
- CSS-код остаётся читаемым: классы имеют понятные имена, а повторяющиеся значения по возможности вынесены в переменные.

Вариант 6. Меню ресторана



Создать страницу меню ресторана с шапкой, hero-блоком, категориями блюд и карточками позиций. При наведении карточка должна плавно изменяться: изображение увеличивается, цена или метка выделяется, появляется декоративная линия.

Требуемый эффект / приём	**Что требуется реализовать**
Масштабирование / transform	При наведении элемент должен визуально изменять размер без разрушения сетки.
Переход / transition	Изменение состояния выполняется плавно, без резкого скачка.
Псевдоклассы	Использовать :hover и :focus там, где пользователь взаимодействует с элементом.
Псевдоэлементы	Добавить минимум один декоративный элемент через ::before или ::after.

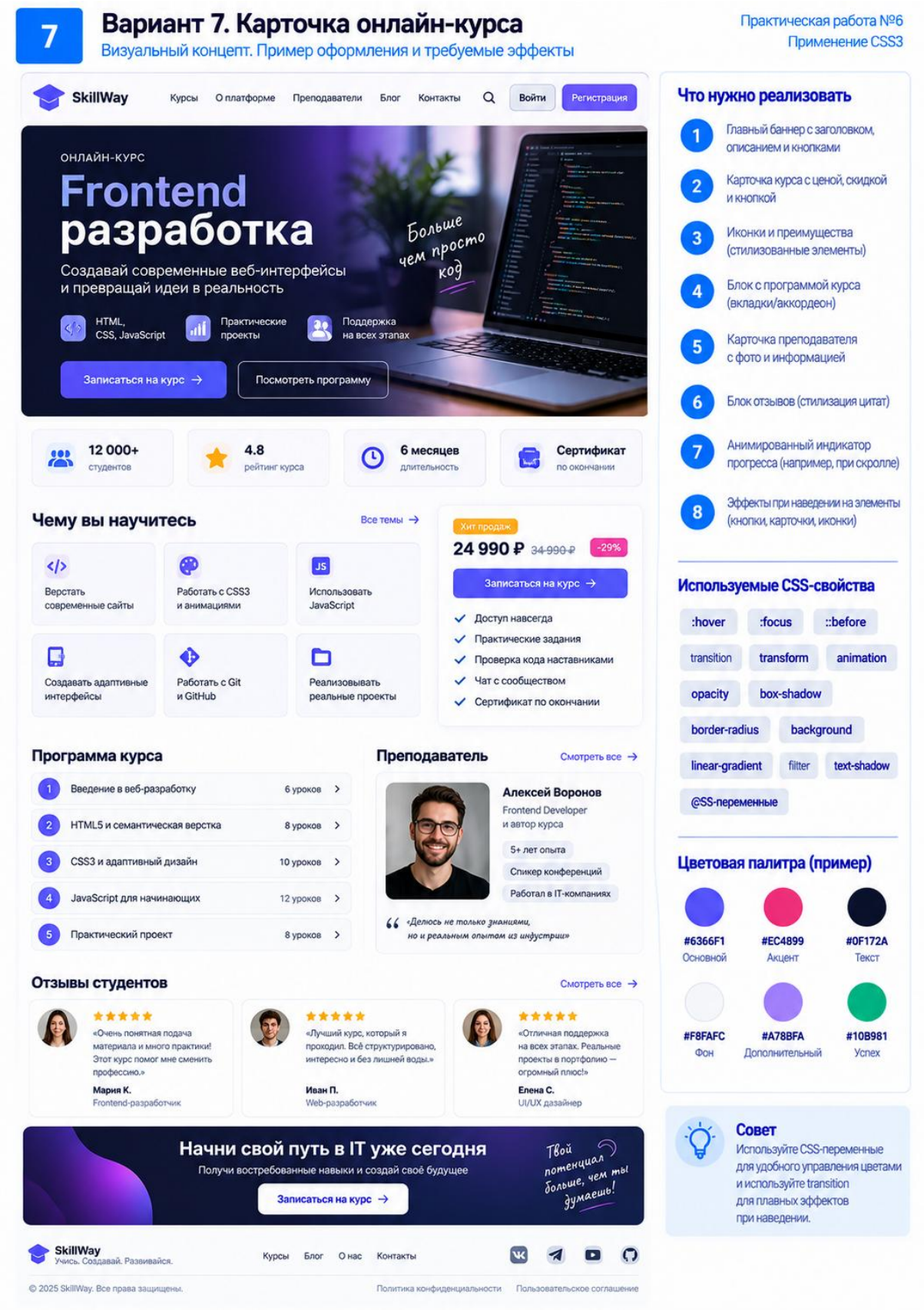
Обязательные техники варианта: background / linear-gradient, transform, transition, box-shadow, ::before / ::after, text-shadow

Как реализовать сложный эффект *Декоративные линии реализовать псевдоэлементами без добавления лишних HTML-элементов. Для hero использовать background-image + background-size: cover.*

Контрольный результат варианта

- Обычное состояние страницы соответствует назначению макета.
- Hover-эффекты видимы и не ломают размеры соседних блоков.
- Интерактивные элементы имеют :focus.
- Псевдоэлемент используется не формально, а для реального визуального элемента.
- CSS-код остаётся читаемым: классы имеют понятные имена, а повторяющиеся значения по возможности вынесены в переменные.

Вариант 7. Карточка онлайн-курса



Оформить страницу онлайн-курса: hero, преимущества, карточка покупки, программа, преподаватель, отзывы и CTA. Предусмотреть интерактивный progress-индикатор и состояния кнопок/карточек.

Требуемый эффект / приём	**Что требуется реализовать**
Масштабирование / transform	При наведении элемент должен визуально изменять размер без разрушения сетки.
Переход / transition	Изменение состояния выполняется плавно, без резкого скачка.
Псевдоклассы	Использовать :hover и :focus там, где пользователь взаимодействует с элементом.
Псевдоэлементы	Добавить минимум один декоративный элемент через ::before или ::after.

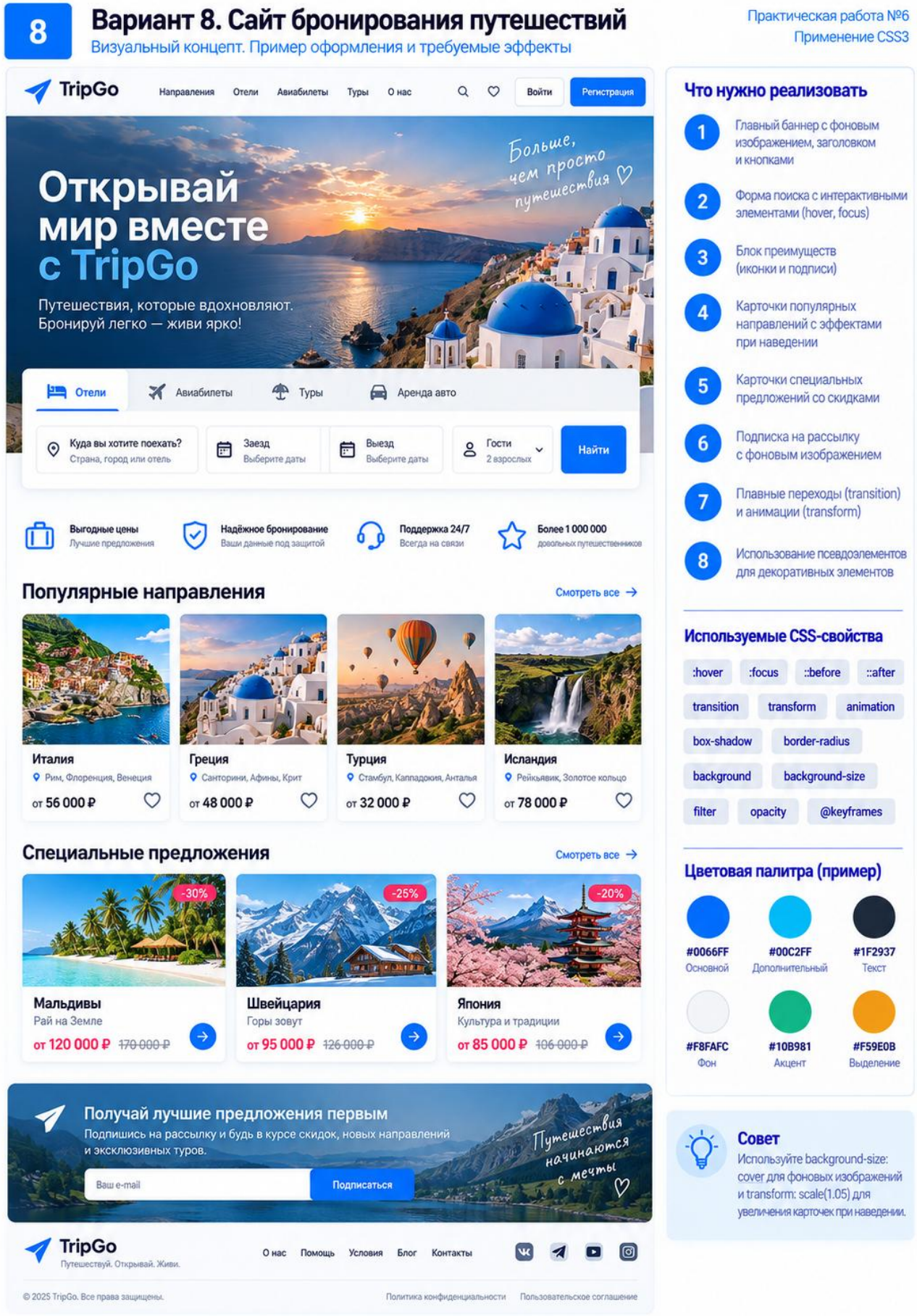
Обязательные техники варианта: @keyframes, transition, transform, linear-gradient, box-shadow, CSS custom properties

Как реализовать сложный эффект *Progress можно сделать элементом с внутренней шкалой и CSS-переменной --progress. Анимация появления/заполнения — через @keyframes.*

Контрольный результат варианта

- Обычное состояние страницы соответствует назначению макета.
- Hover-эффекты видимы и не ломают размеры соседних блоков.
- Интерактивные элементы имеют :focus.
- Псевдоэлемент используется не формально, а для реального визуального элемента.
- CSS-код остаётся читаемым: классы имеют понятные имена, а повторяющиеся значения по возможности вынесены в переменные.

Вариант 8. Сайт бронирования путешествий



Создать интерфейс сервиса бронирования путешествий с hero, формой поиска, популярными направлениями и специальными предложениями. При наведении карточки направлений должны увеличиваться, менять тень и показывать декоративный акцент.

Требуемый эффект / приём	**Что требуется реализовать**
Масштабирование / transform	При наведении элемент должен визуально изменять размер без разрушения сетки.
Переход / transition	Изменение состояния выполняется плавно, без резкого скачка.
Псевдоклассы	Использовать :hover и :focus там, где пользователь взаимодействует с элементом.
Псевдоэлементы	Добавить минимум один декоративный элемент через ::before или ::after.

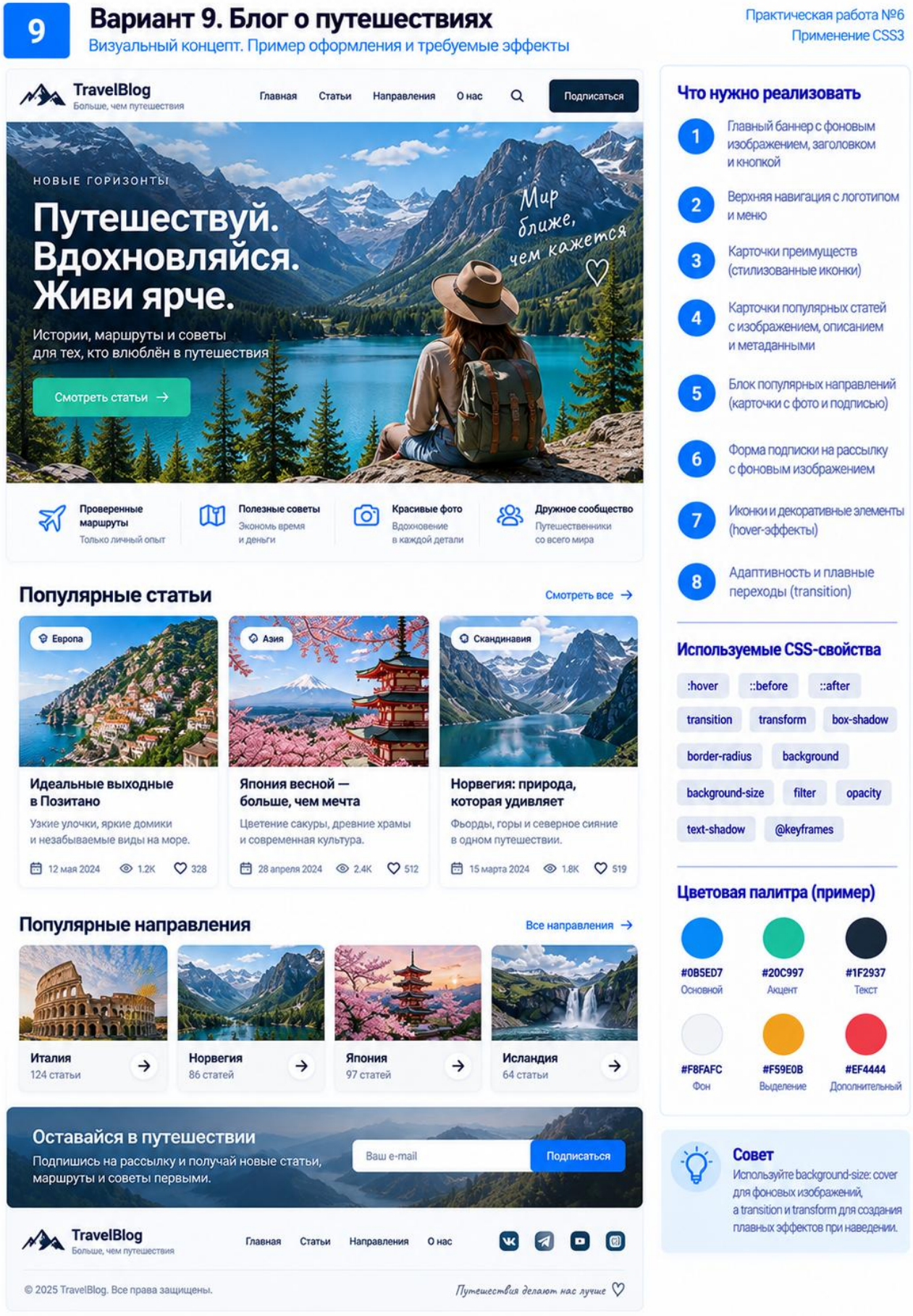
Обязательные техники варианта: background-size: cover, transform, transition, box-shadow, opacity, :focus / :hover

Как реализовать сложный эффект *Фоновое изображение hero оформить через background-size: cover u background-position: center. Для формы обязательно предусмотреть :focus y input u button.*

Контрольный результат варианта

- Обычное состояние страницы соответствует назначению макета.
- Hover-эффекты видимы и не ломают размеры соседних блоков.
- Интерактивные элементы имеют :focus.
- Псевдоэлемент используется не формально, а для реального визуального элемента.
- CSS-код остаётся читаемым: классы имеют понятные имена, а повторяющиеся значения по возможности вынесены в переменные.

Вариант 9. Блог о путешествиях



Создать блог с крупным баннером, популярными статьями, направлениями и формой подписки. Карточки статей должны иметь эффект увеличения изображения, изменение тени и появление декоративного элемента при наведении.

Требуемый эффект / приём	**Что требуется реализовать**
Масштабирование / transform	При наведении элемент должен визуально изменять размер без разрушения сетки.
Переход / transition	Изменение состояния выполняется плавно, без резкого скачка.
Псевдоклассы	Использовать :hover и :focus там, где пользователь взаимодействует с элементом.
Псевдоэлементы	Добавить минимум один декоративный элемент через ::before или ::after.

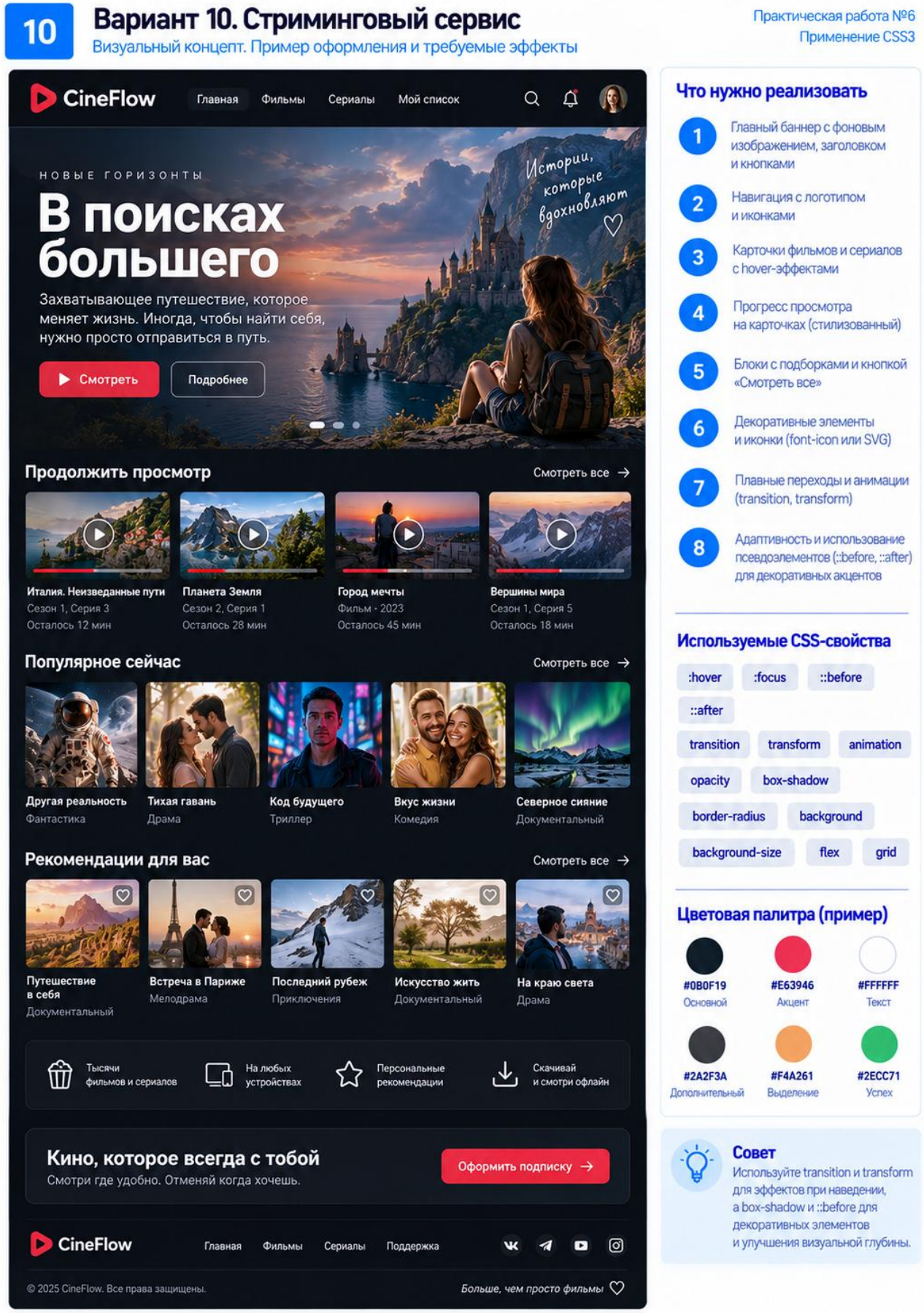
Обязательные техники варианта: object-fit, transform, transition, opacity, ::before / ::after, background-size: cover

Как реализовать сложный эффект *Изображения карточек не должны искажаться: использовать object-fit: cover. Overlay можно анимировать через opacity.*

Контрольный результат варианта

- Обычное состояние страницы соответствует назначению макета.
- Hover-эффекты видимы и не ломают размеры соседних блоков.
- Интерактивные элементы имеют :focus.
- Псевдоэлемент используется не формально, а для реального визуального элемента.
- CSS-код остаётся читаемым: классы имеют понятные имена, а повторяющиеся значения по возможности вынесены в переменные.

Вариант 10. Стриминговый сервис



Создать интерфейс видеосервиса с hero, навигацией, карточками «продолжить просмотр», популярными материалами и рекомендациями. Реализовать полосы прогресса просмотра, hover-состояния и мягкие анимации.

Требуемый эффект / приём	**Что требуется реализовать**
Масштабирование / transform	При наведении элемент должен визуально изменять размер без разрушения сетки.
Переход / transition	Изменение состояния выполняется плавно, без резкого скачка.
Псевдоклассы	Использовать :hover и :focus там, где пользователь взаимодействует с элементом.
Псевдоэлементы	Добавить минимум один декоративный элемент через ::before или ::after.

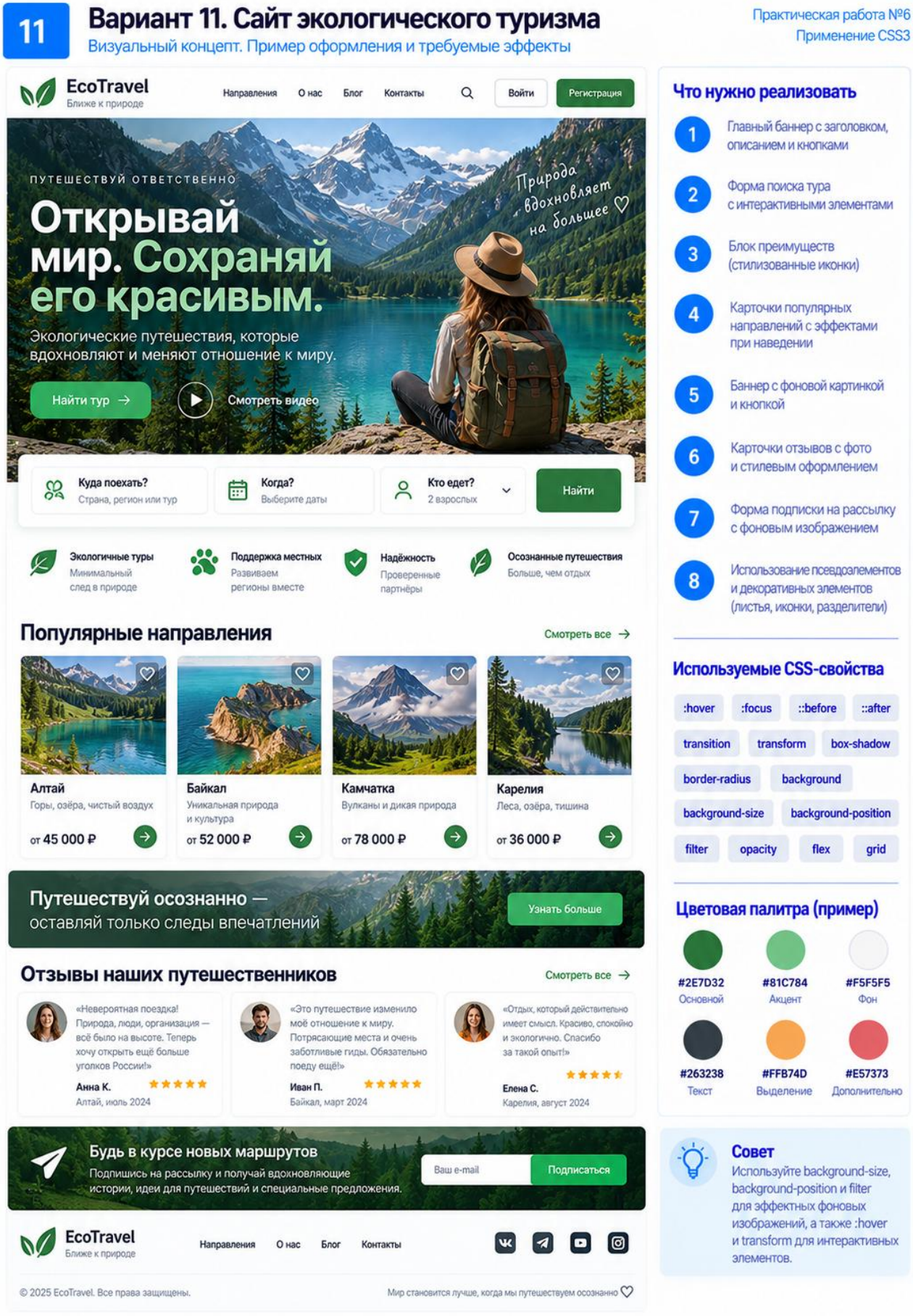
Обязательные техники варианта: linear-gradient, transform, transition, @keyframes, box-shadow, opacity

Как реализовать сложный эффект *Полосу прогресса можно сделать двумя вложенными блоками или через linear-gradient. Анимация допускается только как визуальный эффект и не должна требовать JavaScript.*

Контрольный результат варианта

- Обычное состояние страницы соответствует назначению макета.
- Hover-эффекты видимы и не ломают размеры соседних блоков.
- Интерактивные элементы имеют :focus.
- Псевдоэлемент используется не формально, а для реального визуального элемента.
- CSS-код остаётся читаемым: классы имеют понятные имена, а повторяющиеся значения по возможности вынесены в переменные.

Вариант 11. Сайт экологического туризма



Создать страницу экологического туризма с поиском тура, направлениями, преимуществами, отзывами и подпиской. Использовать зелёную палитру, фоновые фотографии и интерактивные карточки.

Требуемый эффект / приём	Что требуется реализовать
Масштабирование / transform	При наведении элемент должен визуально изменять размер без разрушения сетки.
Переход / transition	Изменение состояния выполняется плавно, без резкого скачка.
Псевдоклассы	Использовать :hover и :focus там, где пользователь взаимодействует с элементом.
Псевдоэлементы	Добавить минимум один декоративный элемент через ::before или ::after.

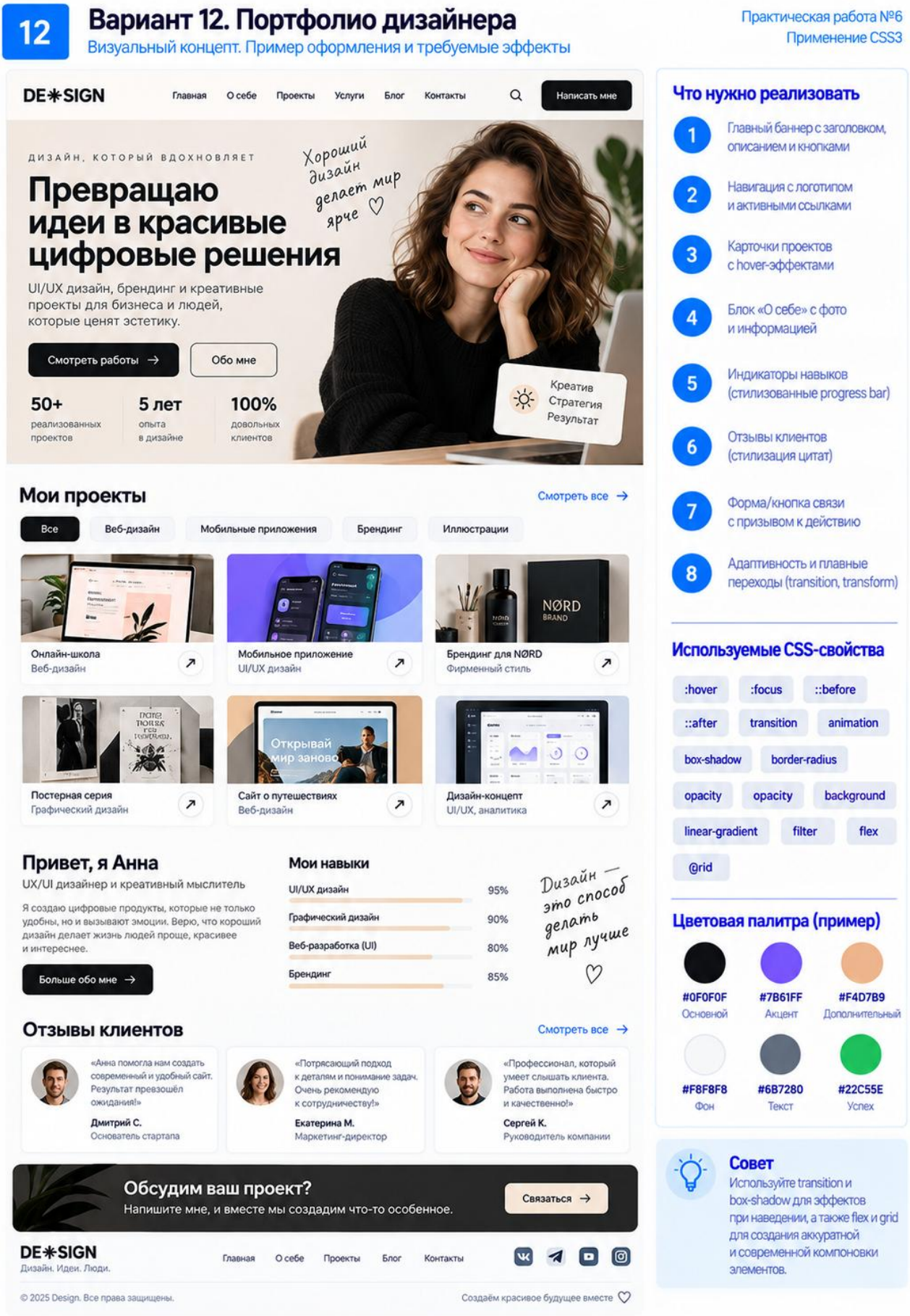
Обязательные техники варианта: background-position / background-size, filter, transform, transition, box-shadow, ::before / ::after

Как реализовать сложный эффект *Для фото при наведении использовать filter или лёгкое увеличение. Декоративные листы, линии и маркеры выполнить псевдоэлементами.*

Контрольный результат варианта

- Обычное состояние страницы соответствует назначению макета.
- Hover-эффекты видимы и не ломают размеры соседних блоков.
- Интерактивные элементы имеют :focus.
- Псевдоэлемент используется не формально, а для реального визуального элемента.
- CSS-код остаётся читаемым: классы имеют понятные имена, а повторяющиеся значения по возможности вынесены в переменные.

Вариант 12. Портфолио дизайнера



Создать портфолио специалиста: hero, проекты, навыки, отзывы и CTA. Карточки проектов должны реагировать на наведение, а индикаторы навыков — визуально отображать уровень.

Требуемый эффект / приём	Что требуется реализовать
Масштабирование / transform	При наведении элемент должен визуально изменять размер без разрушения сетки.
Переход / transition	Изменение состояния выполняется плавно, без резкого скачка.
Псевдоклассы	Использовать :hover и :focus там, где пользователь взаимодействует с элементом.
Псевдоэлементы	Добавить минимум один декоративный элемент через ::before или ::after.

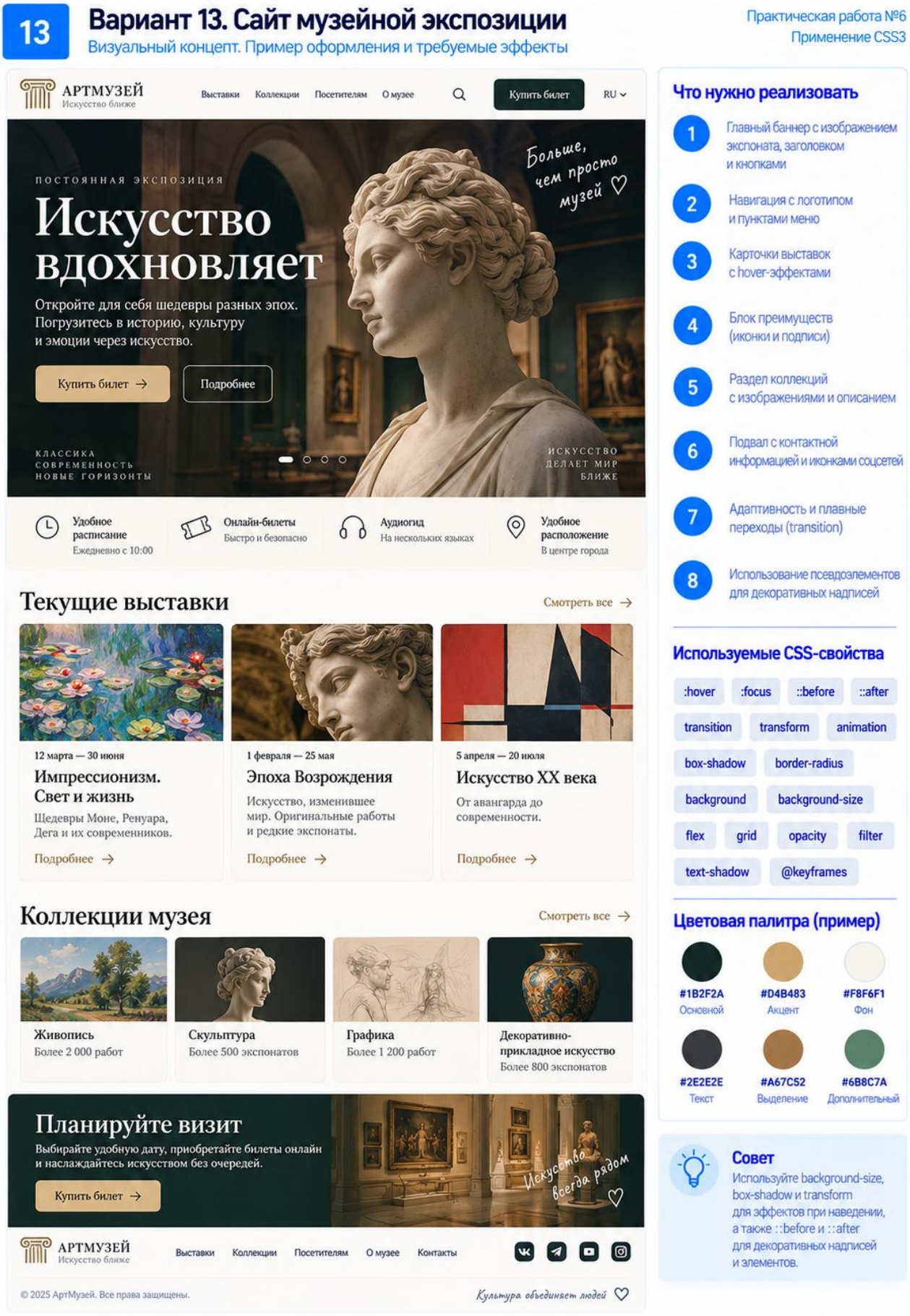
Обязательные техники варианта: transform, transition, box-shadow, linear-gradient, CSS variables, @keyframes

Как реализовать сложный эффект *Для progress использовать CSS-переменную --level. Проекты оформить grid/flex-контейнером; hover не должен менять размеры соседних элементов.*

Контрольный результат варианта

- Обычное состояние страницы соответствует назначению макета.
- Hover-эффекты видимы и не ломают размеры соседних блоков.
- Интерактивные элементы имеют :focus.
- Псевдоэлемент используется не формально, а для реального визуального элемента.
- CSS-код остаётся читаемым: классы имеют понятные имена, а повторяющиеся значения по возможности вынесены в переменные.

Вариант 13. Сайт музейной экспозиции



Создать сайт выставки с hero-экспонатом, текущими выставками, коллекциями, информационным блоком и CTA покупки билета. Использовать спокойную музейную палитру и эффект появления деталей на карточках.

Требуемый эффект / приём	Что требуется реализовать
Масштабирование / transform	При наведении элемент должен визуально изменять размер без разрушения сетки.
Переход / transition	Изменение состояния выполняется плавно, без резкого скачка.
Псевдоклассы	Использовать :hover и :focus там, где пользователь взаимодействует с элементом.
Псевдоэлементы	Добавить минимум один декоративный элемент через ::before или ::after.

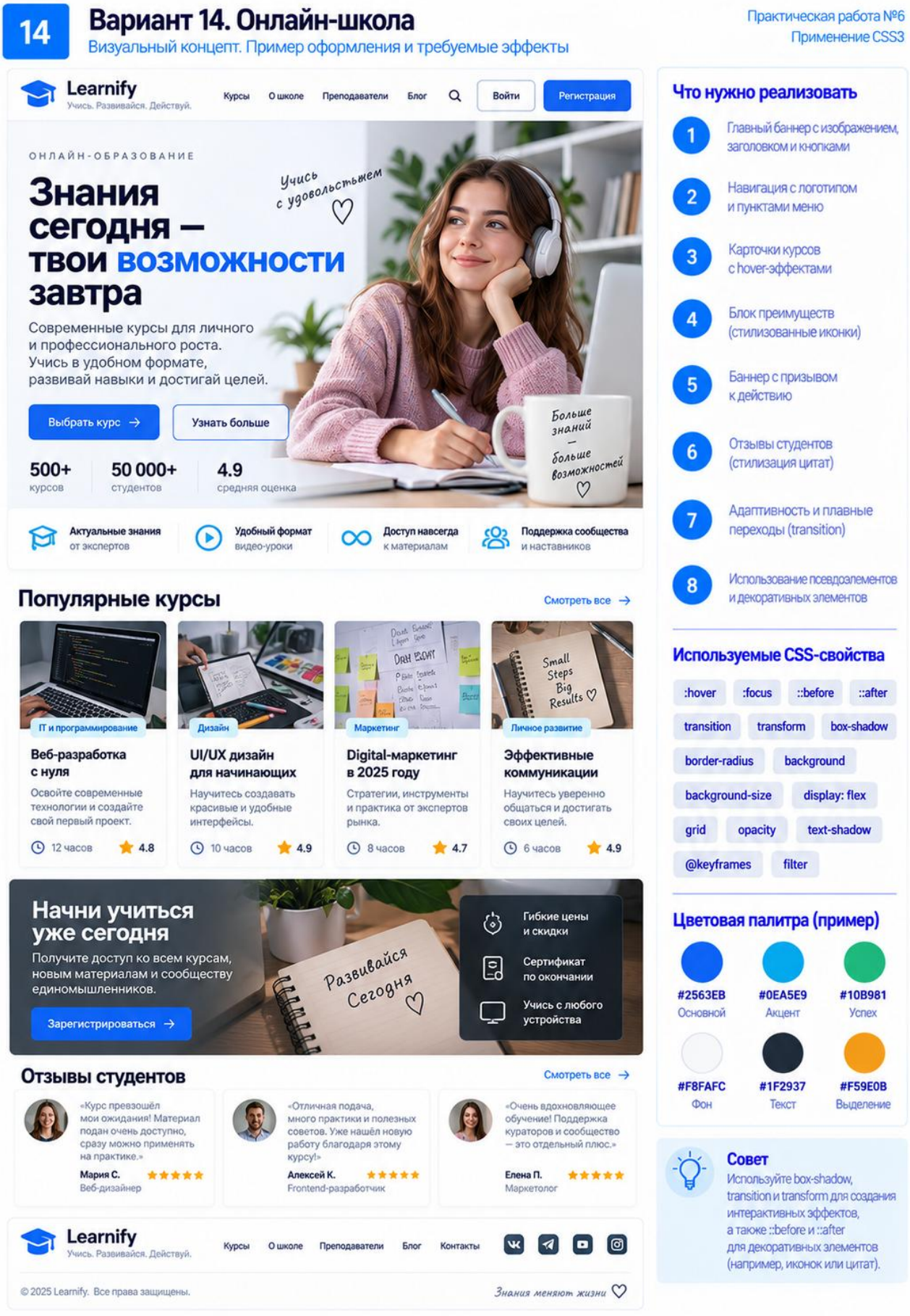
Обязательные техники варианта: background-size: cover, filter, transition, transform, box-shadow, ::before / ::after

Как реализовать сложный эффект *Overlay карточки сделать через псевдоэлемент. При наведении можно слегка изменить насыщенность изображения и показать кнопку «Подробнее».*

Контрольный результат варианта

- Обычное состояние страницы соответствует назначению макета.
- Hover-эффекты видимы и не ломают размеры соседних блоков.
- Интерактивные элементы имеют :focus.
- Псевдоэлемент используется не формально, а для реального визуального элемента.
- CSS-код остаётся читаемым: классы имеют понятные имена, а повторяющиеся значения по возможности вынесены в переменные.

Вариант 14. Сайт музейной экспозиции



Создать страницу онлайн-школы с hero, популярными курсами, преимуществами, промо-блоком и отзывами. Предусмотреть hover-эффекты карточек, интерактивные кнопки и визуальные состояния.

Требуемый эффект / приём	**Что требуется реализовать**
Масштабирование / transform	При наведении элемент должен визуально изменять размер без разрушения сетки.
Переход / transition	Изменение состояния выполняется плавно, без резкого скачка.
Псевдоклассы	Использовать :hover и :focus там, где пользователь взаимодействует с элементом.
Псевдоэлементы	Добавить минимум один декоративный элемент через ::before или ::after.

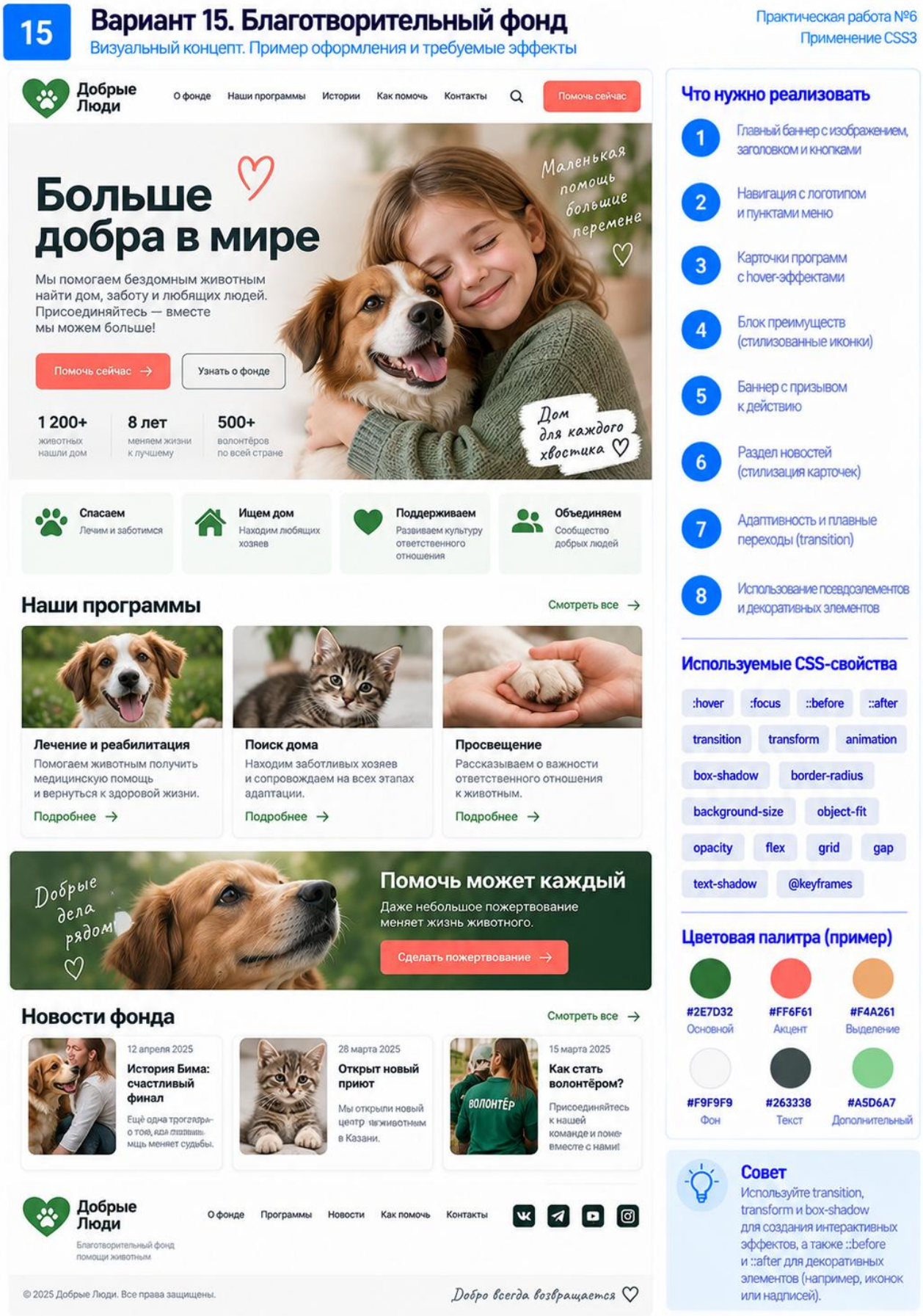
Обязательные техники варианта: transition, transform, box-shadow, @keyframes, ::before / ::after, background

Как реализовать сложный эффект *Кнопка должна иметь минимум три состояния: обычное, hover и focus. Для декоративного свечения/подложки использовать псевдоэлемент.*

Контрольный результат варианта

- Обычное состояние страницы соответствует назначению макета.
- Hover-эффекты видимы и не ломают размеры соседних блоков.
- Интерактивные элементы имеют :focus.
- Псевдоэлемент используется не формально, а для реального визуального элемента.
- CSS-код остаётся читаемым: классы имеют понятные имена, а повторяющиеся значения по возможности вынесены в переменные.

Вариант 15. Сайт музейной экспозиции



Создать страницу фонда с hero, программами помощи, статистикой, новостями и блоком пожертвования. Визуальный стиль должен быть дружелюбным; карточки и CTA — интерактивными.

Требуемый эффект / приём	Что требуется реализовать
Масштабирование / transform	При наведении элемент должен визуально изменять размер без разрушения сетки.
Переход / transition	Изменение состояния выполняется плавно, без резкого скачка.
Псевдоклассы	Использовать :hover и :focus там, где пользователь взаимодействует с элементом.
Псевдоэлементы	Добавить минимум один декоративный элемент через ::before или ::after.

Обязательные техники варианта: transition, transform, box-shadow, opacity, linear-gradient, ::before / ::after

Как реализовать сложный эффект *Для CTA использовать плавное изменение тени и масштаба. Декоративные сердечки/линии можно реализовать псевдоэлементами, не добавляя лишнюю разметку.*

Контрольный результат варианта

- Обычное состояние страницы соответствует назначению макета.
- Hover-эффекты видимы и не ломают размеры соседних блоков.
- Интерактивные элементы имеют :focus.
- Псевдоэлемент используется не формально, а для реального визуального элемента.
- CSS-код остаётся читаемым: классы имеют понятные имена, а повторяющиеся значения по возможности вынесены в переменные.

Задание 2. Создать страницу

Создать HTML-структуру страницы без вставки макета как единого изображения. Подключить внешний файл CSS и выполнить оформление страницы. Реализовать все эффекты, перечисленные в задании варианта. Проверить состояния элементов при обычном отображении, наведении и фокусе. Проверить страницу при изменении ширины окна браузера.

При разработке руководствоваться спецификацией.

Общие требования к HTML и CSS

HTML

- Использовать <!DOCTYPE html>, lang="ru" и meta charset="UTF-8".
- Задать meta viewport для корректного отображения на мобильных экранах.
- Использовать логичную структуру header, nav, main, section/article и footer там, где это уместно.

- Изображения должны иметь alt.
- Кнопки и ссылки должны быть различимы по назначению.
- Формы, если они есть в макете, должны иметь понятные label и доступные состояния focus.

CSS

- Основное оформление выполнять во внешнем файле style.css.
- Использовать box-sizing: border-box для предсказуемого расчёта размеров блоков.
- Продемонстрировать работу минимум пяти типов селекторов: тег, класс, id/атрибут, псевдокласс, составной/комбинаторный селектор.
- Использовать минимум два псевдокласса, среди которых обязательно :hover; :focus рекомендуется сделать обязательным для интерактивных элементов.
- Использовать ::before и/или ::after хотя бы для одного осмысленного декоративного элемента.
- Использовать transition минимум для двух интерактивных элементов.
- Использовать transform минимум в одном hover-эффекте.
- Использовать box-shadow, border-radius и работу с цветом/фоном.
- Для сложных вариантов использовать @keyframes или CSS-переменные.
- Не использовать inline-style как основной способ оформления.

Базовые CSS3-эффекты, которые должны быть освоены

Эффект	Что должен понять студент
Модель коробки	Показывает, как content, padding, border и margin формируют фактический размер элемента. Для карточек рекомендуется box-sizing: border-box.
Hover	Изменяет визуальное состояние элемента при наведении указателя. Изменения должны быть плавными и предсказуемыми.
Focus	Показывает состояние интерактивного элемента при клавиатурной навигации. Нельзя полностью удалять outline без создания альтернативного визуального признака.
Transition	Задаёт плавное изменение свойств между состояниями. Рекомендуется ограничивать переход конкретными свойствами, например transform, opacity, box-shadow.
Transform	Позволяет визуально смещать, масштабировать или поворачивать элемент без перестроения обычного потока страницы.
Opacity	Используется для появления/исчезновения overlay, текста и декоративных элементов.
Box-shadow	Создаёт визуальную глубину карточки и может усиливаться при наведении.
Gradient	linear-gradient() и radial-gradient() позволяют создать цветовой переход или затемнение без отдельного изображения.
Pseudo-elements	::before и ::after позволяют добавлять декоративные линии, бейджи, overlay и маркеры без изменения HTML.
Filter	brightness(), grayscale(), blur() и другие фильтры позволяют менять визуальное восприятие изображения.
Animation	@keyframes используется для повторяющейся или последовательной анимации: progress, pulse, floating, shimmer и т.п.

Шаблон карточки с hover

```
    .card {
  position: relative;
  overflow: hidden;
  border-radius: 18px;
  box-shadow: 0 8px 24px rgba(15, 23, 42, .12);
  transition: transform .3s ease, box-shadow .3s ease;
}

.card:hover {
  transform: translateY(-6px);
  box-shadow: 0 16px 36px rgba(15, 23, 42, .20);
}

.card::before {
  content: "";
  position: absolute;
  inset: 0;
  background: rgba(15, 23, 42, .45);
  opacity: 0;
  transition: opacity .3s ease;
}

.card:hover::before { opacity: 1; }
```

Шаблон изображения с увеличением

```
    .card__image { overflow: hidden; }
.card__image img {
  display: block;
  width: 100%;
  transition: transform .35s ease, filter .35s ease;
}
.card:hover .card__image img {
  transform: scale(1.05);
  filter: brightness(.82);
}
```

Шаблон progress через CSS-переменную

```
    .progress {
  --progress: 72%;
  height: 8px;
  border-radius: 999px;
  background: #e5e7eb;
  overflow: hidden;
}
```

```
.progress::before {
  content: "";
  display: block;
  width: var(--progress);
  height: 100%;
  background: linear-gradient(90deg, #2563eb, #7c3aed);
}
```

Шаблон @keyframes

```
   @keyframes pulse {
  0%, 100% { transform: scale(1); opacity: .75; }
  50% { transform: scale(1.06); opacity: 1; }
}
.badge--animated {
  animation: pulse 1.8s ease-in-out infinite;
}
```

Важно
Анимация должна иметь учебный смысл: показывать прогресс, активность, появление элемента или акцентировать действие. Не следует анимировать всё подряд.

Требования к структуре проекта

```
  lab-6/
├── index.html
├── css/
│   └── style.css
├── images/
│   └── ...
└── README.md
```

- index.html — точка входа страницы.
- css/style.css — основная таблица стилей.
- images/ — локальные изображения проекта.
- README.md — ФИО, группа, вариант, краткое описание, ссылка на опубликованную страницу.

Дополнительные CSS-файлы допускаются, если их назначение понятно. В репозитории не должно быть паролей, токенов и иных секретных данных.

Задание 3. Представление практической работы

Созданную страницу необходимо опубликовать на сервисе **GitHub.io** и предоставить ссылку (вида https://username.github.io) на законченный и рабочий вариант для проверки преподавателем.

Последовательность действий для публикации:

1. Зарегистрируйтесь на https://github.com/, если ещё не сделали этого.
2. Создайте новый репозиторий с именем username.github.io, где username — ваш ник на GitHub.
3. Создайте новый файл. Когда репозиторий не пуст, то в верхнем правом углу есть кнопка "Create New File", но сейчас, когда ещё нет ни одного файла, создать первый можно по прямой ссылке https://github.com/username/username.github.io/new/master, где username — ваш ник.
4. Введите имя index.html и скопируйте туда содержание файла index.html.
5. Нажмите кнопку "Commit new file".
6. Повторите то же самое для файла style.css.
7. Подтвердите публикацию на GitHub Pages. В настройках репозитория: Settings -> Раздел Options -> Секция GitHub Pages.
8. Через несколько мгновений страница станет доступна по вашему адресу https://username.github.io

Примечание. *Страница должна оставаться доступной по ссылке до момента проверки работы в курсе. Затем опубликованные материалы можно удалить или заместить следующими работами.*

Все созданные элементы (исходные материалы) также нужно поместить в каталог **lab-6** на GitHub.

Чек-лист самопроверки

- ☐ Используется <!DOCTYPE html>.
- ☐ Указан lang="ru".
- ☐ Есть UTF-8 и meta viewport.
- ☐ CSS подключён внешним файлом.
- ☐ Основные блоки страницы соответствуют выбранному макету.
- ☐ Использована модель коробки: width/height/padding/border/margin.
- ☐ Использованы минимум пять типов селекторов.
- ☐ Есть :hover.
- ☐ Есть :focus у интерактивных элементов.
- ☐ Есть ::before и/или ::after.
- ☐ Есть transition.
- ☐ Есть transform.
- ☐ Есть box-shadow и border-radius.
- ☐ Есть работа с фоном/градиентом.
- ☐ Реализован специфический эффект выбранного варианта.
- ☐ Для сложного варианта есть @keyframes или CSS-переменная, если это предусмотрено заданием.

- ☐ Нет горизонтального переполнения на типовой ширине.
- ☐ Изображения не искажаются.
- ☐ В Console нет критических ошибок.
- ☐ В Network нет ошибок загрузки основных ресурсов.
- ☐ Репозиторий содержит весь исходный код.
- ☐ README содержит ФИО, группу, вариант и ссылки.
- ☐ GitHub Pages открывается без авторизации.
- ☐ CSS и изображения загружаются после публикации.
- ☐ URL репозитория и URL страницы переданы преподавателю.

Контрольные вопросы

1. Что такое CSS и какова его основная функция в веб-разработке?
2. Каковы основные различия между CSS1, CSS2 и CSS3?
3. Что такое селекторы в CSS и какие их типы вы знаете?
4. Как подключить CSS к HTML-документу? Приведите примеры различных методов.
5. Что такое каскадность в CSS и как она влияет на применение стилей?
6. Как задать цвет фона для элемента с помощью CSS?
7. Что такое блочные и строчные элементы в CSS? Приведите примеры.
8. Как изменить размер шрифта элемента с помощью CSS?
9. Что такое модель коробки (box model) в CSS и какие ее основные компоненты?
10. Как задать отступы (margin) и внутренние отступы (padding) для элемента?
11. Что такое псевдоклассы и как они используются в CSS? Приведите примеры.
12. Как создать границы (border) для элементов с помощью CSS?
13. Что такое медиа-запросы и как они помогают в адаптивной верстке?
14. Как использовать CSS для создания фоновых изображений и их настройки?
15. Что такое Flexbox и как он облегчает работу с макетом в CSS?
16. Как задать стили для различных состояний элемента (например, при наведении)?
17. Что такое CSS Grid и в чем его преимущества по сравнению с Flexbox?
18. Как использовать анимации и переходы в CSS3? Приведите примеры.
19. Что такое переменные в CSS и как они могут быть полезны?
20. Как проверить совместимость CSS3 с различными браузерами?

ПРАКТИЧЕСКАЯ РАБОТА 7. РАЗМЕЩЕНИЕ И МЕТОДЫ КОМПОНОВКИ

Цель работы

- **Изучение основ компоновки:** ознакомиться с понятиями компоновки и ее значением в веб-дизайне, понять различия между традиционными и современными методами компоновки.
- **Работа с Flexbox:** научиться использовать Flexbox для создания гибких и адаптивных макетов, изучить свойства контейнера flex (например, display: flex, flex-direction, justify-content, align-items), применить Flexbox для выравнивания и распределения пространства между элементами.
- **Использование CSS Grid:** освоить основы CSS Grid для построения сложных сеток, изучить основные свойства сетки (например, display: grid, grid-template-columns, grid-template-rows, grid-area), создать макеты с использованием различных комбинаций строк и колонок.
- **Адаптивный дизайн:** изучить принципы адаптивного дизайна и медиа-запросов, применить методы компоновки для создания адаптивных интерфейсов, которые хорошо выглядят на различных устройствах.

7.1. Цветовые схемы и единицы измерения

Цвет в CSS

Цвет является важной частью любого веб-сайта, поэтому в CSS существует множество вариантов цветовых типов, функций и обработок. CSS имеет различные типы данных, такие как строки и числа. Цвет является одним из этих типов и использует другие типы, например, числа, для своих собственных определений.

Свойство `color` определяет цвет текста (цвет переднего плана в целом) элемента. Например, свойство `color`, указанное в селекторе `body`, определяет цвет текста по умолчанию для всей страницы. Давайте попробуем следующий пример, чтобы увидеть, как это работает:

```
body {
    color: #ff5722;
}
```

Свойство `color` обычно наследует значение цвета от своего родительского элемента, за исключением случая элементов привязки. Например, если мы укажем `color` для элемента `body`, он будет автоматически передан в заголовки, абзацы и т. д.

Определение значений цвета

1. **Числовые цвета**

Очень вероятно, что первое знакомство с цветами в CSS происходит через числовые значения цвета. Мы можем работать с числовыми значениями цвета в нескольких различных формах.

Цвета в CSS чаще всего указываются в следующих форматах:

- ключевое слово цвета — например, "red", "green", "blue", "transparent" и т.д.
- шестнадцатеричное значение — например, «#ff0000», «#00ff00» и т.д.
- значение RGB - например, "rgb(255, 0, 0)"

CSS3 представил несколько других цветовых форматов, таких как HSL, HSLA и RGBA, которые также поддерживают альфа-прозрачность (альфа-канал).

2. **Шестнадцатеричные цвета**

Шестнадцатеричная запись (часто сокращается до шестнадцатеричной) — это сокращенный синтаксис для RGB, который присваивает числовое значение красному, зеленому и синему, которые являются тремя **основными цветами**. Согласно веб-альманаху, hex — самый популярный тип синтаксиса цвета.

```
h1 {
  color: #b71540;
}
```

Шестнадцатеричные диапазоны: **0-9** и **AF**. При использовании в виде последовательности из шести цифр они преобразуются в числовые диапазоны RGB от 0 до 255, которые соответствуют каналам красного, зеленого и синего цветов соответственно.

Альфа-значение можно определить с любым числовым цветом. Альфа-значение представляет собой процент прозрачности. В шестнадцатеричном коде мы добавляем еще две цифры к шестизначной последовательности, создавая восьмизначную последовательность. Например, чтобы задать черный цвет в шестнадцатеричном коде, напишите `#000000`. Чтобы добавить прозрачность 50%, измените его на `#00000080`.

Поскольку шестнадцатеричная шкала **0-9** и **AF**, значения прозрачности, вероятно, не совсем такие, как ожидаются. Вот некоторые ключевые, общие значения, добавленные к черному шестнадцатеричному коду `#000000`:

- 0% alpha — полностью прозрачный — равен **00**: `#00000000`
- 50% альфа равно **80**: `#00000080`
- 75% альфа - **BF**: `#000000BF`

Чтобы преобразовать двузначное шестнадцатеричное число в десятичное, нужно взять первую цифру и умножить ее на 16 (поскольку шестнадцатеричное число является основанием 16), а затем добавить вторую цифру. Используя **BF** в качестве примера для 75% альфа:

1. B равно 11, что при умножении на 16 равно 176.
2. F равно 15
3. 176 + 15 = 191
4. Альфа-значение составляет 191—75% от 255.

Шестнадцатеричные коды также записываются в сокращении из трех цифр. Трехзначный шестнадцатеричный код — это сокращение от эквивалентной шестизначной последовательности. Например, `#a4e` идентичен `#aa44ee`. Чтобы добавить альфу, затем `#a4e8` расширится до `#aa44ee88`.

3. RGB (красный, зеленый, синий)

Цвета могут быть определены в модели **RGB (красный, зеленый и синий)**, используя функциональную нотацию `rgb()`. Функция `rgb()` принимает три значения, разделенные запятыми, которые определяют количество красного, зеленого и синего компонентов цвета. Эти значения обычно указываются как целые числа от 0 до 255, где 0 означает отсутствие цвета, а 255 — полный или максимальный цвет.

В следующем примере указывается тот же цвет, что и в примере выше, но в нотации RGB.

```
h1 {
  color: rgb(183, 21, 64);
}
```

Цвета RGB определяются с помощью функции цвета `rgb()` не только с использованием чисел, но и процентов в качестве параметров. Числа должны находиться в диапазоне от 0 до **255**, а проценты — от **0% до 100%** . RGB работает в масштабе 0-255, поэтому 255 будет эквивалентно 100%, а 0 — 0%.

Чтобы установить черный цвет в RGB, определите его как `rgb(0 0 0)`, то есть ноль красного, ноль зеленого и ноль синего. Черный также может быть определен как `rgb(0%, 0%, 0%)`. Белый - полная противоположность: `rgb(255, 255, 255)` или `rgb(100%, 100%, 100%)`.

Альфа устанавливается `rgb()` одним из двух способов. Либо добавляется `/` после красных, зеленых и синих параметров, либо используется функция `rgba()`. Альфа может быть определена в процентах или десятичной дробью от 0 до 1.

Например, чтобы установить 50% альфа-черного цвета в современных браузерах, нужно добавить: `rgb(0 0 0 / 50%)` или `rgb(0 0 0 / 0.5)`. Для более широкой поддержки, используя функцию `rgba()`, добавляем: `rgba(0, 0, 0, 50%)` или `rgba(0, 0, 0, 0.5)`.

Запятые были удалены из обозначения `rgb()` или `hsl()`, потому что новые цветовые функции, такие как `lab()` и `lch()` используют пробелы вместо запятых в качестве разделителя. Это изменение обеспечивает большую согласованность не только с новыми функциями цвета, но и с CSS в целом. Для лучшей обратной совместимости по-прежнему можно использовать запятые для определения `rgb()` и `hsl()`.

4. Ключевые слова цвета

В CSS есть 148 именованных цветов. Это простые английские названия, такие как `red`, `blue`, `black` или `lightseagreen`. Некоторые из самых популярных имен, согласно веб-альманаху, — черный, белый, красный, синий и серый. Нашими фаворитами являются золотарник, aliceblue и ярко-розовый.

Помимо стандартных цветов, также доступны специальные ключевые слова:

- `transparent` - полностью прозрачный цвет. Это также начальное значение `background-color`
- `currentColor` - контекстно-вычисленное динамическое значение свойства `color`. Если у вас есть цвет текста `red`, а затем устанавливается значение `border-color`, то `currentColor` также будет красным. Если элемент, который определяет `currentColor`, не имеет значения для определения `color`, то `currentColor` вместо этого будет вычисляться каскадом.

Системные ключевые слова *— это цвета, определенные темой операционной системы. Некоторые примеры этих цветов:* `Background`*, который является*

цветом фона рабочего стола, или `Highlight`*, который является цветом выделения выбранных элементов. Это только два из множества вариантов.*

Все цветовые ключевые слова не чувствительны к регистру, однако часто встречаются системные цвета с заглавными буквами, чтобы отличить их от стандартных цветовых ключевых слов.

Где использовать цвет в правилах CSS

Если свойство CSS принимает тип данных `<color>` в качестве значения, оно будет принимать любой из вышеперечисленных методов выражения цвета. Для фона можно установить цвет в качестве значения для `background` или `background-color`. Цвета также можно использовать в градиентах, например `linear-gradient`.

***Градиенты** — это тип изображения, который можно программно определить в CSS. Градиенты принимают два или более цветов в любой комбинации цветового формата, например, hex, rgb или hsl.*

Наконец, и установите цвет границ `border-color` и контуров блоков `outline-color`. Свойство `box-shadow` также принимает цвет в качестве одного из значений.

Влияние свойств цвета на границы и контуры

Свойство `color` предназначено не только для текстового содержимого, но и для всего, что находится на переднем плане и принимает значение цвета.

Например, если значение `border-color` или `outline-color` не было определено явно для элемента, то вместо него будет использоваться значение цвета.

Определение цветов в CSS3

Мы узнали, как определять цвета, используя ключевые слова color и обозначения RGB. В дополнение к этому CSS3 добавляет несколько новых функциональных обозначений для установки значений цвета для элементов, таких как `rgba()`, `hsl()` и `hsla()`.

Цветовые значения RGBA

Цвета могут быть определены в **модели RGBA (красный-зеленый-синий-альфа)**, используя функциональную нотацию `rgba()`. Цветовая модель RGBA является расширением цветовой модели RGB с альфа-каналом, который определяет непрозрачность цвета. Параметр альфа принимает значения от `0.0` (полностью прозрачный) до `1.0` (полностью непрозрачный).

```
h1 {
   color: rgba(0,0,255,0.5);
}
p {
   background-color: rgba(0%,0%,100%,0.3);
}
```

Цветовые значения HSL

Цвета также могут быть определены **моделью HSL (оттенок-насыщенность-яркость)** с использованием функциональной нотации `hsl()`. Оттенок представлен как угол (от `0` до `360`) цветового круга или круга (т.е. радуга, представленная в круге). Этот угол задается как число без единицы измерения, потому что угол обычно измеряется в градусах, и эта единица неявно используется в CSS.

Насыщенность и яркость представлены в процентах. Насыщенность `100%` означает полный цвет и `0%` - является оттенком серого. Посмотрите пример ниже:

```
h1 {
  color: hsl(344, 79%, 40%);
}
```

HSL означает оттенок, насыщенность и легкость. Оттенок описывает значение на цветовом круге от 0 до 360 градусов, начиная с красного (от 0 до 360). Оттенок 180 или 50% будет в синем диапазоне. Это происхождение цвета, который мы видим.

Насыщенность показывает, насколько ярким является выбранный оттенок. Полностью обесцвеченный цвет (с насыщенностью `0%`) будет отображаться в градациях серого. И, наконец, светлота — это параметр, описывающий шкалу добавленного света от белого до черного. Легкость `100%` всегда будет придавать вам белый цвет.

Используя функцию цвета `hsl()`, мы определяем настоящий черный, написав `hsl(0 0% 0%)`, или даже `hsl(0deg 0% 0%)`. Это связано с тем, что параметр `hue` определяет степень на цветовом круге, которая, если вы используете числовой тип, составляет **0-**

360. Мы также можете использовать тип угла, например (`0deg`) или `(0turn)`. И насыщенность, и яркость определяются в процентах.

Тип угла в CSS отлично подходит для определения оттенка, потому что он действительно хорошо представляет угол цветового круга. Этот тип принимает градусы, обороты, радианы.

Альфа определяется в `hsl()`, так же, как в `rgb()` путем добавления `/` после параметров оттенка, насыщенности и яркости или с помощью функции `hsla()`. Альфа может быть определена в процентах или десятичной дробью от 0 до 1. Например, чтобы установить 50% альфа-черного, используйте: `hsl(0 0% 0% / 50%)` или `hsl(0 0% 0% / 0.5)`. Используя `hsla()` функцию, напишите: `hsla(0, 0%, 0%, 50%)` или `hsla(0, 0%, 0%, 0.5)`.

В CSS появилось несколько новых цветовых типов. К ним относятся `lab()` и `lch()` , которые позволяют указывать гораздо более широкий диапазон цветов, чем это возможно в `RGB`.

Цветовые значения HSLA

Цвета могут быть определены в модели **HSLA (оттенок-насыщенность-яркость-альфа)**, используя функциональную нотацию `hsla()`. Цветовая модель HSLA является расширением цветовой модели HSL с альфа-каналом, который определяет непрозрачность цвета.

Параметр альфа принимает значения от `0.0` (полностью прозрачный) до `1.0` (полностью непрозрачный).

```
h1 {
    color: hsla(360,80%,50%,0.5);
}
p {
    background-color: hsla(480,60%,30%,0.3);
}
```

Работа с цветовыми кодами

Выражение цветов с помощью шестнадцатеричных значений очень популярно, потому что с его помощью мы можем представить огромное количество цветовых комбинаций (более 16,7 миллиона). Однако работать с ними несколько сложно. В большинстве случаев вам понадобится палитра цветов. Если вы используете `Visual Studio`

`Code` в качестве редактора, мы можем открыть палитру цветов прямо в файле CSS:

наведите указатель мыши на код цвета, и должны увидеть палитру цветов.

Если у нас нет встроенного средства выбора цвета в ваш редактор, мы можем обратиться к многочисленным онлайн-селекторам цветов:

- HTML Color Chart
- Color-Picker

В большинстве случаев вам понадобится не один цвет для веб-сайта. Нам понадобится палитра цветов. С помощью простого сервиса палитры цветов довольно сложно выбрать цвета, которые хорошо сочетаются друг с другом.

Есть несколько инструментов, которые упрощают поиск цветовой палитры. Вот несколько примеров:

Предопределенные цветовые палитры

- Coolors
- Colormind
- Material Design (Google)

Генераторы цветовой палитры

- Adobe Color
- Paletton
- Colourco.de

Единицы измерения

CSS имеет несколько свойств размеров, таких как `width`, `height`, `max-width`, `min-width`, `max-height`, `min-height` что позволяет контролировать ширину и высоту элемента. В следующих разделах описано, как использовать эти свойства для создания лучшего макета веб-страницы.

Настройка ширины и высоты

Свойства `width` и `height` определяет ширину и высоту области содержимого элемента. Эта ширина и высота не включают отступы, границы или поля.

Давайте попробуем следующий пример и посмотрим, как это на самом деле работает:

```
div {
    width: 300px;
```

```
    height: 200px;
}
```

Приведенные выше правила стиля назначают элементу `<div>` фиксированную ширину 300 пикселей и высоту 200 пикселей.

Свойства `width` и `height` могут принимать следующие значения:

- *длина* - указывает ширину в px, em, rem, pt, cm и т.д.
- *%* - указывает ширину в процентах (%) от ширины содержащего элемента.
- auto — браузер вычисляет подходящую ширину элемента.
- Initial — устанавливает ширину и высоту в значение по умолчанию, которое равно `auto`.
- inherit — указывает, что ширина должна быть унаследована от родительского элемента.

Нельзя указывать отрицательные значения для свойств ширины и высоты.

Обычно, когда вы создаете блочный элемент, такой как `<div>`, `<p>`, и т. д., браузер автоматически устанавливает их ширину на 100% доступной ширины, а высоту — на то, что необходимо для отображения всего содержимого. Следует избегать установки фиксированной ширины и высоты, если в этом нет необходимости.

Установка максимальной ширины и высоты

Чтобы указать максимальную ширину и высоту области содержимого, можно использовать свойство `max-width` и `max-height`. Эта максимальная ширина и высота не включают отступы, границы или поля.

Элемент не может быть шире значения `max-width`, даже если значение свойства `width` установлено больше. Например, если для параметра `width` установлено значение 300 пикселей, а для параметра `max-width` установлено значение 200 пикселей, фактическая ширина элемента будет равна 200 пикселей. Давайте посмотрим пример:

```
div {
    width: 300px;
    max-width: 200px;
}
```

Если свойство `min-width` указано со значением, превышающим значение свойства `max-width`, в этом случае значение `min-width` будет фактически примененным.

Точно так же примененный элемент `max-height` никогда не будет выше указанного значения, даже если для свойства `height` задано большее значение. Например, если для `height` установлено значение 200 пикселей и для `max-height` установлено значение 100 пикселей, фактическая высота элемента будет равна 100 пикселей.

```
div {
    height: 200px;
    max-height: 100px;
}
```

Если свойство `min-height` указано со значением, превышающим значение свойства `max-height`, в этом случае значение `min-height` будет фактически примененным.

Установка минимальной ширины и высоты

Для указания минимальной ширины и высоты области содержимого, можно использовать свойство `min-width` и `min-height`. Эта минимальная ширина и высота не включают отступы, границы или поля.

Элемент не может быть уже, чем значение `min-width`, даже если значение свойства `width` установлено на что-то меньшее. Например, если для параметра `width` установлено значение 300 пикселей, а для параметра `min-width` установлено значение 400 пикселей, фактическая ширина элемента будет равна 400 пикселей. Давайте посмотрим, как это работает на самом деле:

```
div {
    width: 200px;
    min-width: 300px;
}
```

Свойство `min-width` обычно используется, чтобы гарантировать, что элемент имеет по крайней мере минимальную ширину, даже если содержимое отсутствует. Однако элементу будет разрешено нормально расти, если его содержимое превышает установленную минимальную ширину.

Точно так же элемент, к которому применяется `min-height`, никогда не будет меньше указанного значения, даже если свойство `height` установлено на что-то меньшее. Например, если для параметра установлено значение 200 пикселей, `height` а для параметра `min-height` установлено значение 300 пикселей, фактическая высота элемента будет равна 300 пикселей.

```
div {
    height: 100px;
    min-height: 200px;
}
```

Свойство `min-height` обычно используется, чтобы гарантировать, что элемент имеет по крайней мере минимальную высоту, даже если содержимое отсутствует. Однако элементу будет разрешено нормально расти, если содержимое превышает установленную минимальную высоту.

Настройка диапазона ширины и высоты

Свойства `min-width` и `min-height` часто используются в сочетании со свойствами `max-width` и `max-height` для создания диапазона ширины и высоты элемента.

Это может быть очень полезно для создания гибкого дизайна. В следующем примере минимальная ширина элемента `<div>` будет 300 пикселей, и он может растягиваться по горизонтали до максимум 500 пикселей.

```
div {
    min-width: 300px;
    max-width: 500px;
}
```

Точно так же мы можем определить диапазон высоты для элемента. В приведенном ниже примере минимальная высота элемента `<div>` будет 300 пикселей, и он может растягиваться по вертикали максимум до 500 пикселей.

```
div {
    min-height: 300px;
    max-height: 500px;
}
```

Интернет — это адаптивная среда, но иногда вы хотите контролировать ее размеры, чтобы улучшить общее качество интерфейса. Хорошим примером этого является ограничение длины строки для улучшения читаемости. Как бы вы сделали это в такой гибкой среде, как Интернет?

Для этого применяется единица `ch`, равную ширине символа «0» в отображаемом шрифте при его вычисленном размере. Этот модуль позволяет ограничить ширину текста с помощью модуля, предназначенного для изменения размера текста, что, в свою очередь, обеспечивает предсказуемое управление независимо от размера этого текста.

Единица `ch` — одна из нескольких единиц, полезных для определенных контекстов, таких как этот пример.

Числа

Числа используются для определения `opacity`, `line-height` и даже для значений цветовых каналов в файлах `rgb`. Числа представляют собой безразмерные целые (1, 2, 3, 100) и десятичные дроби (.1, .2, .3).

Числа имеют значение в зависимости от их контекста. Например, при определении `line-height` число представляет отношение, если мы определяем его без единицы измерения:

```
p {
  font-size: 24px;
  line-height: 1.5;
}
```

В этом примере `1.5` равно **150%** вычисленного размера шрифта элемента `p` в пикселях. Это означает, что, если `p` имеет значение `font-size`, высота строки `24px` будет вычисляться как `36px`.

Рекомендуется использовать безразмерное значение для `line-height`, а не указывать единицу измерения. Как нам уже известно из темы наследования, `font-size` можно наследовать. Определение безразмерного значения `line-height` сохраняет высоту строки относительно размера шрифта. Это обеспечивает лучший опыт, чем, скажем, `line-height: 15px`, который не изменится и может выглядеть странно при определенных размерах шрифта.

Цифры также можно использовать в следующих местах:

- При установке значений для фильтров: `filter: sepia(0.5)` применяет `50%` к элементу фильтра.
- При установке непрозрачности: `opacity: 0.5` применяет `50%` непрозрачности.
- В цветовых каналах: `rgb(50, 50, 50)`, где допустимы значения 0-255 для установки значения цвета.
- Чтобы преобразовать элемент: `transform: scale(1.2)` масштабирует элемент на 120% от его начального размера.

Проценты

При использовании процента в CSS нам нужно знать, как рассчитывается процент. Например, `width` вычисляется в процентах от доступной ширины в родительском элементе.

```
div {
  width: 300px;
  height: 100px;
}

div p {
  width: 50%;
}
```

В приведенном выше примере ширина `div p` равна `150px`, при условии, что в макете используется значение по умолчанию `box-sizing: content-box`.

Если мы установим `margin` или `padding` в процентах, они будут частью ширины родительского элемента, независимо от направления.

```
div {
  width: 300px;
  height: 100px;
}

div p {
  margin-top: 50%; /* calculated: 150px */
  padding-left: 50%; /* calculated: 150px */
}
```

В приведенном выше фрагменте `margin-top` и `padding-left` будут вычисляться до `150px`.

```
div {
  width: 300px;
  height: 100px;
}

div p {
  width: 50%; /* calculated: 150px */
  transform: translateX(10%); /* calculated: 15px */
}
```

Если мы установим значение `transform` в процентах, оно основывается на элементе с установленным преобразованием. В этом примере `p` имеет значение `translateX`, равное `10%` и значение `width` - `50%`. Во-первых, рассчитывается, какой будет ширина: `150px`, потому что она составляет 50% от ширины родителя. Затем берется `10%` от `150px` - то есть `15px`.

Размеры и длина

Если единица добавляется к числу, оно становится **измерением**. Например, `1rem` — это размер. В этом контексте единица, присоединенная к числу, в спецификациях называется **токеном измерения**. **Длины** — это измерения, которые относятся к расстоянию**,** и они могут быть как **абсолютными**, так и **относительными**.

1. Абсолютные длины

Все абсолютные длины соотносятся с одной и той же базой, что делает их предсказуемыми, где бы они ни использовались в CSS. Например:

```
div {
  width: 10cm;
  height: 5cm;
  background: black;
}
```

Если напечатать эту страницу, она будет напечатана в виде черного прямоугольника `div` размером 10x5 см. Имейте в виду, что CSS используется не только для цифрового контента, но и для оформления печатного контента. Абсолютная длина действительно может пригодиться при разработке дизайна для печати.

Таблица 7.1. Единицы измерения контента

Ед. изм	Имя	Эквивалентно
cm	Сантиметры	1 см = 96 пикселей/2,54
mm	Миллиметры	1 мм = 1/10 часть 1 см
Q	Четверть миллиметра	1Q = 1/40 часть 1 см
in	Дюймы	1 дюйм = 2,54 см = 96 пикселей
pc	Пикас	1 шт = 1/6 дюйма
pt	Точки	1 пт = 1/72 от 1 дюйма

Ед. изм	Имя	Эквивалентно
px	Пиксели	1 пиксель = 1/96 часть 1 дюйма

2. Относительные длины

Относительная длина вычисляется относительно базового значения, как в процентах. Разница между ними и процентами заключается в том, что имеется возможность контекстно изменять размер элементов.

Это означает, что в CSS есть такие единицы `ch`, которые используют размер текста в качестве основы и ориентируются на ширину области просмотра (окна вашего браузера). Относительная длина особенно полезна в Интернете из-за ее адаптивного характера.

3. Относительные единицы измерения размера шрифта

CSS предоставляет полезные единицы измерения, связанные с размером отображаемых элементов типографики, например, размер самого текста (единицы `em`) или ширину символов гарнитуры (единицы `ch`).

Таблица 7.2. Единицы измерения размера шрифта

Ед. изм	относительно:
em	Относительно размера шрифта, т.е. 1.5em будет на 50% больше, чем базовый вычисленный размер шрифта его родителя. (Исторически сложилось так, что высота заглавной буквы «M»).
ex	Эвристика для определения того, следует ли использовать высоту x, букву "x" или `.5em` в текущем вычисленном размере шрифта элемента.
cap	Высота заглавных букв в текущем вычисленном размере шрифта элемента.
ch	Среднее продвижение символа узкого глифа в шрифте элемента (представленного глифом «0»).
ic	Среднее продвижение символа глифа полной ширины в шрифте элемента, представленного глифом «水» (идеограф воды CJK, U+6C34).
rem	Размер шрифта корневого элемента (по умолчанию 16 пикселей).
lh	Высота строки элемента.
rlh	Высота строки корневого элемента.

4. Единицы относительно окна просмотра

Размеры области просмотра (окна браузера) задействуются в качестве относительной основы. Эти единицы занимают доступное пространство окна просмотра.

Таблица 7.3. Единицы измерения области просмотра

Ед. изм	относительно
vw	1% ширины области просмотра. Люди используют этот блок, чтобы делать крутые трюки со шрифтами, например, изменять размер шрифта заголовка в зависимости от ширины страницы, чтобы при изменении пользователем размера шрифт также изменялся.
vh	1% высоты области просмотра. Мы можем использовать это для упорядочивания элементов в пользовательском интерфейсе, например, если есть панель инструментов нижнего колонтитула.
vi	1% размера области просмотра по встроенной оси корневого элемента. Ось относится к режимам записи. В режимах горизонтального письма, таких как английский, встроенная ось горизонтальна. В режимах вертикального письма, таких как некоторые японские шрифты, встроенная ось проходит сверху вниз.
vb	1% размера окна просмотра по оси блока корневого элемента . Для оси блока это будет направленность языка. Языки LTR, такие как английский, будут иметь вертикальную ось блоков, поскольку англоязычные читатели анализируют страницу сверху вниз. Вертикальный режим записи имеет горизонтальную ось блока.
vmin	1% меньшего размера области просмотра.
vmax	1% от большего размера области просмотра.

```
div {
  width: 10vw;
}

p {
  max-width: 60ch;
}
```

В этом примере это `div` будет 10% ширины окна просмотра, потому что `1vw` — это 1% ширины окна просмотра. Элемент `p` имеет значение `max-width` равное `60ch`, то есть его ширина не может превышать 60 символов "0" в рассчитанном шрифте и размере.

При изменении размера текста в относительных единицах, таких как `em` или `rem`, а не в абсолютных единицах, например `px`, размер вашего текста может

соответствовать предпочтениям пользователя. Это может включать размер системного шрифта или размер шрифта родительского элемента, например, `<body>`.

Базовый размер `em` — это родительский элемент, а базовый размер `rem` — это базовый размер шрифта документа. Если `font-size` не определен для своего элемента `html`, этот предпочитаемый пользователем системный размер шрифта будет учитываться при использовании относительной длинны `em` или `rem`.

5. Угловые единицы

При рассмотрении моделей цвета было упоминание об угловые единицах, которые полезны для определения значений, таких как оттенок в `hsl`. Они также используются для задания вращения элементов внутри функций преобразования.

```
div {
  width: 150px;
  height: 150px;
  transform: rotate(60deg);
}
```

Используя угловую единицу `deg`, можно повернуть `div` на 90° вокруг своей центральной оси.

```
div {
  background-image: url('a-low-resolution-image.jpg');
}

@media (-webkit-min-device-pixel-ratio: 2), (min-resolution: 192dpi) {
  div {
    background-image: url('a-high-resolution-image.jpg');
  }
}
```

Другие угловые единицы - это `rad` (радианы), `grad` (градианы) и единицы `turn`, которые представляют собой часть угла, где `1turn` = `360deg` и `0.5turn` = `180deg`.

6. Единицы разрешения

В предыдущем примере значение `min-resolution` равно `192dpi`. Единица `dpi` означает **количество точек на дюйм**. Полезным контекстом для этого является обнаружение экранов с очень высоким разрешением, таких как дисплеи **Retina**, в мультимедийном запросе и предоставление изображения с более высоким разрешением.

7.2. Методы компоновки

Макет

Представьте, что вы работаете разработчиком, и коллега-дизайнер передает вам дизайн совершенно нового веб-сайта. Дизайн имеет всевозможные интересные макеты и композиции: двухмерные макеты, учитывающие ширину и высоту области просмотра, а также макеты, которые должны быть плавными и гибкими. Как решить с помощью средств CSS их лучше стилизовать?

CSS предоставляет нам различные способы решения проблем компоновки по горизонтальной оси, вертикальной оси или даже по обоим направлениям. Выбор правильного метода макета для контекста является трудной задачей, и на практике может понадобиться более одного метода макета.

Современный CSS обладает исключительно мощными инструментами компоновки. Имеются специальные системы для компоновки. Мы рассмотрим их в общих чертах, прежде чем углубиться в детали `Flexbox` и `Grid`.

Понимание свойства display

Свойство `display` делает две вещи. Во-первых, определяет, действует ли блок, к которому он применяется, как **встроенный** или **блочный**.

```
.my-element {
  display: inline;
}
```

Строчные элементы ведут себя как слова в предложении. Они расположены рядом друг с другом в линейном направлении. Элементы `<span>` и `<strong>` по умолчанию являются **встроенными**, если они используются для стилизации фрагментов текста внутри таких элементов, как `<p>` (абзац). Они также сохраняют окружающие пробелы.

Мы не можем установить явную ширину и высоту для встроенных элементов. Любые поля и отступы на уровне блока будут игнорироваться окружающими элементами.

```
.my-element {
        display: block;
}
```

Блочные элементы не располагаются рядом друг с другом. Они создают новую линию для себя. Блочный элемент будет расширяться до размера встроенного размера, охватывая всю ширину по горизонтали. Поля со всех сторон блочного элемента будут соблюдаться.

```
.my-element {
```

```
        display: flex;
}
```

Во-вторых, свойство `display` определяет, как должны вести себя дочерние элементы элемента. Например, установка свойства `display: flex` делает блок блоком уровня блока, а также преобразует его дочерние элементы в гибкие элементы. Это включает **свойства гибкости**, которые управляют **выравниванием**, **порядком** и **потоком**.

Флексбокс и сетка

Существует два основных механизма компоновки, которые создают правила компоновки для нескольких элементов: `flexbox` и `grid`. Они имеют сходство, но предназначены для решения различных проблем компоновки.

Флексбокс

Flexbox — это механизм оформления одномерных макетов. Расположение идет по одной оси - горизонтально или вертикально.

```
.my-element {
        display: flex;
}
```

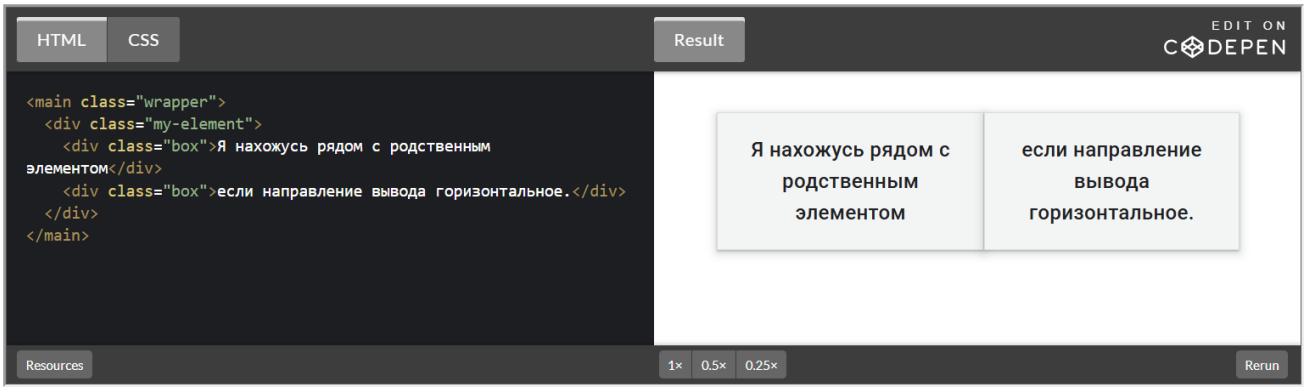


Рис. 7.1. Выравнивание элементов по умолчанию

По умолчанию `flexbox` выравнивает дочерние элементы элемента, расположенные рядом друг с другом в линейном направлении, и растягивает их в направлении блока, поэтому они все имеют одинаковую высоту.

Элементы останутся на той же оси и не будут перенесены, даже когда им не хватит места. Вместо этого они будут пытаться попасть на одну линию друг с другом. Это поведение можно изменить с помощью свойств `align-items`, `justify-content` и `flex-wrap`.

Flexbox также преобразует дочерние элементы во flex-элементы, то есть можно задавать правила поведения внутри flex-контейнера. Например, изменять порядок и выравнивание для отдельного элемента, а также изменять способ его уменьшения или увеличения с помощью свойства `flex`.

```
.my-element div {
        flex: 1 0 auto;
}
```

Свойство `flex` является сокращением для `flex-grow`, `flex-shrink` и `flex-basis`. Приведенный выше пример дополняется следующим образом:

```
.my-element div {
 flex-grow: 1;
 flex-shrink: 0;
 flex-basis: auto;
}
```

Разработчики формируют эти низкоуровневые правила, чтобы подсказать браузеру, как макет должен вести себя, когда на него влияет содержимое и размеры области просмотра. Также это полезный механизмом для создания адаптивного веб-дизайна.

Сетка

Механизм **Grid** во многом похож на **flexbox**, но он предназначен для управления многоосевыми макетами вместо одноосевых (вертикальное или горизонтальное пространство).

```
.my-element {
        display: grid;
}
```

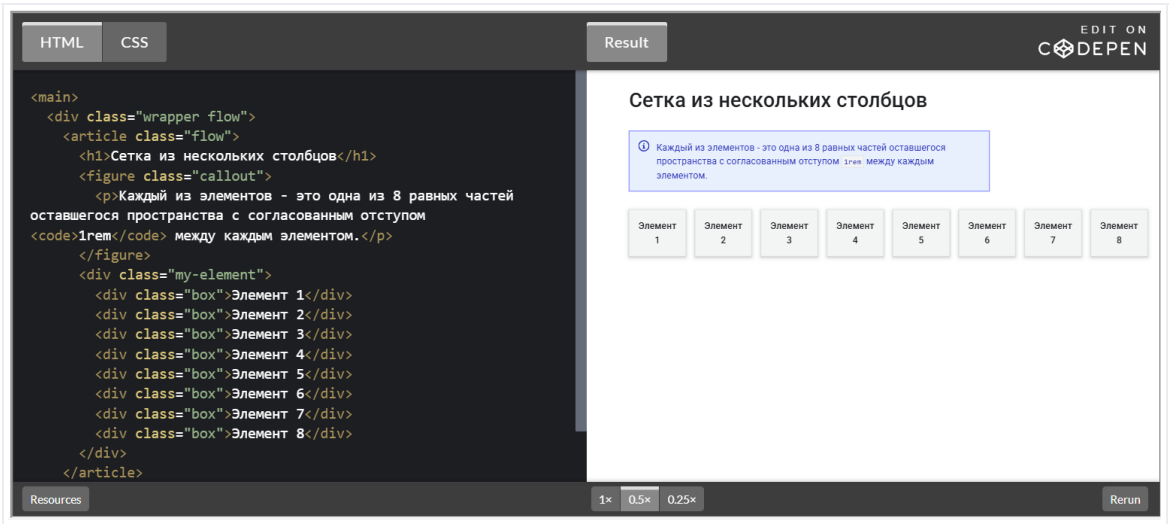


Рис. 7.2. Выравнивание элементов по сетке (по умолчанию)

`Grid` позволяет писать правила макета для элемента, который имеет `display: grid` и вводит несколько новых примитивов для стиля макета, таких как функции `repeat()` и `minmax()`. Одна полезная единица сетки — это единица `fr`, представляющая собой часть оставшегося пространства. Например, можно создавать традиционные сетки из 12 столбцов с промежутком между каждым элементом с помощью 3-х свойств CSS:

```
.my-element {
  display: grid;
  grid-template-columns: repeat(12, 1fr);
  gap: 1rem;
}
```

В приведенном выше примере показан макет с одной осью. Если `flexbox` в основном обрабатывает элементы как группу, сетка дает точный контроль над их размещением в двух измерениях. Мы можем наблюдать, что первый элемент в этой сетке занимает 2 строки и 3 столбца:

```
.my-element :first-child {
  grid-row: 1/3;
  grid-column: 1/4;
}
```

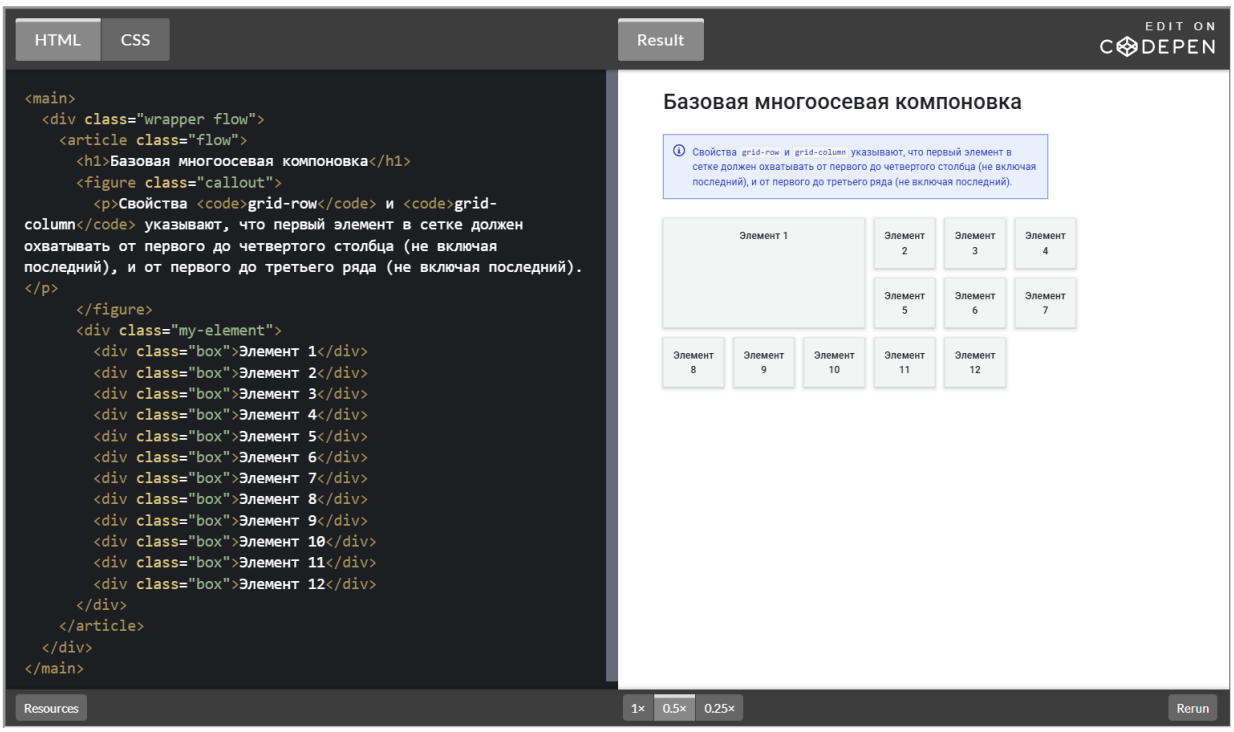


Рис. 7.3. Пример реализации многоосевой компоновки

Свойства `grid-row` и `grid-column` указывают, что первый элемент в сетке должен охватывать начало четвертого столбца из первого столбца, а затем охватывать третью строку из первой строки.

Схема потока

Если не используется сетка или флексбокс, элементы отображаются в обычном режиме. Существует ряд методов компоновки, которые можно использовать для настройки поведения и положения элементов в обычном потоке.

Встроенный блок

Помните, как окружающие элементы не учитывают поля блока и отступы встроенного элемента? Это проявляется с `inline-block`.

```
p span {
        display: inline-block;
}
```

Использование `inline-block` создает блок, имеющий характеристики блочного элемента, но по-прежнему встроен в текст.

```
p span {
        margin-top: 0.5rem;
}
```

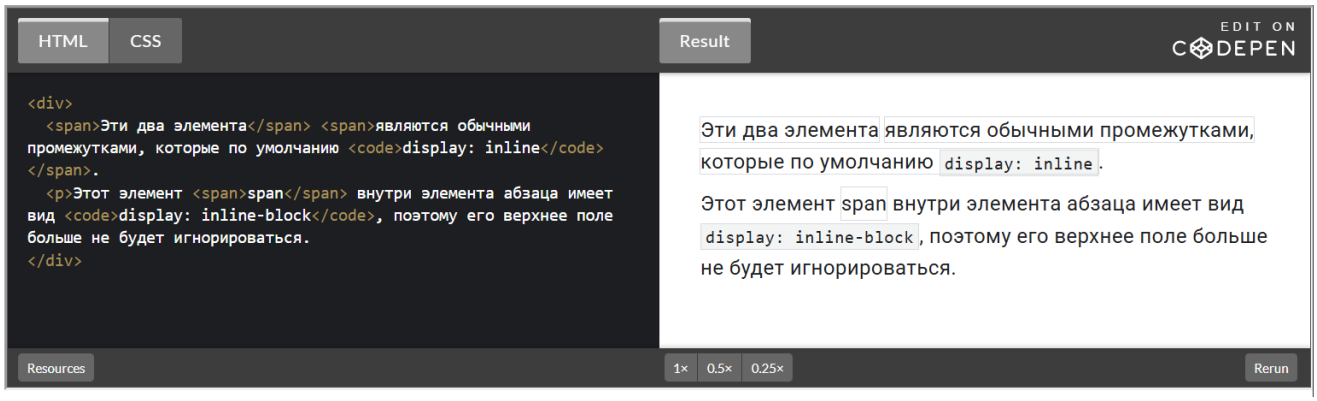


Рис. 7.4. Пример встроенного блока

Поплавки

Если у вас есть изображение, которое находится внутри абзаца текста, не было бы удобно, если бы этот текст обтекал это изображение, как вы могли бы видеть в газете? Это можно сделать с помощью **поплавков**.

```
img {
        float: left;
        margin-right: 1em;
}
```

Свойство `float` указывает элементу «плавать» в указанном направлении. Изображение в этом примере получает команду перемещаться влево, что позволяет элементам-близнецам «оборачиваться» вокруг него. Также допускается указать элементу `float left` или `rightinherit`.

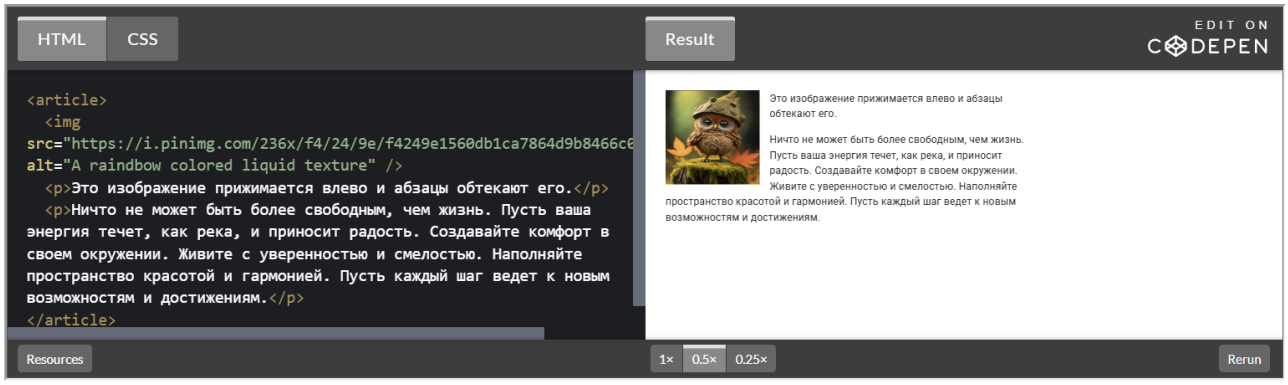


Рис. 7.5. Пример «плавающего» элемента

Когда используется `float`, то макет любых элементов, следующих за плавающим элементом, может быть скорректирован. Для предотвращения этого, нужно очистить поплавок либо с помощью элемента `clear: both`, который следует за плавающим элементом, либо с помощью родительского элемента `display: flow-root` плавающих элементов.

Многоколоночный макет

В CSS3 появился модуль макета с несколькими столбцами для простого и эффективного создания макетов из нескольких столбцов. Теперь имеется возможность создавать макеты, подобные тем, которые встречаются в журналах и газетах, без использования плавающих блоков.

Вот простой пример разделения некоторого текста на три столбца с использованием функции компоновки нескольких столбцов CSS3.

```
p {
    -webkit-column-count: 3; /* Chrome, Safari, Opera */
      -moz-column-count: 3; /* Firefox */
          column-count: 3; /* Standard syntax */
}
```

Если имеется очень длинный список элементов, например, список всех стран мира, это может привести к тому, что пользователь будет много прокручивать и тратить время. Это также может создать лишние пробелы на странице. Многоколоночный CSS делит его на несколько столбцов, чтобы решить обе эти проблемы.

```
<h1>All countries</h1>
<ul class="countries">
  <li>Argentina</li>
  <li>Aland Islands</li>
  <li>Albania</li>
  <li>Algeria</li>
  <li>American Samoa</li>
  <li>Andorra</li>
  …
</ul>
.countries {
        column-count: 2;
        column-gap: 1em;
}
```

Это автоматически разбивает этот длинный список на два столбца и добавляет промежуток между двумя столбцами.

```
.countries {
width: 100%;
column-width: 260px;
column-gap: 1em;
}
```

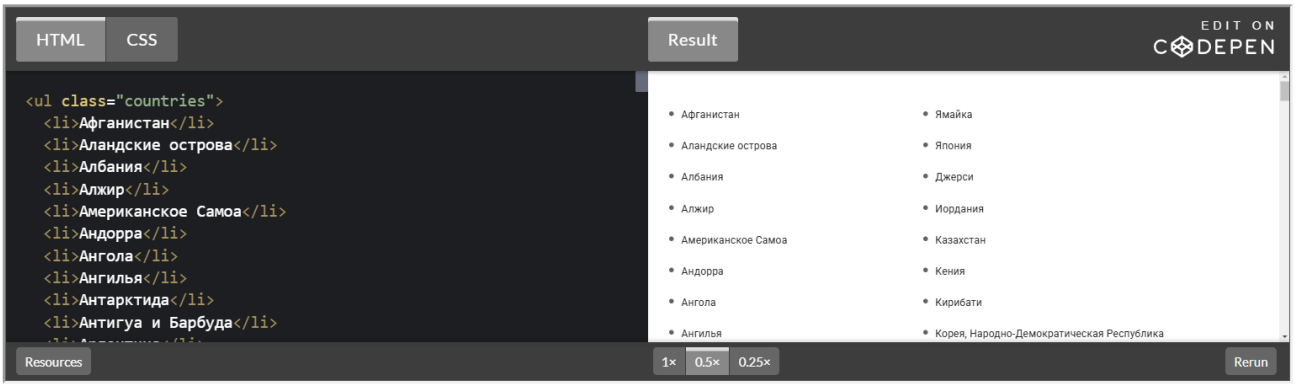


Рис. 7.6. Пример многоколоночного макета

Вместо установки количества столбцов, на которые разбивается содержимое, также определяется минимальная желаемая ширина, используя `column-width`. По мере того, как в окне просмотра становится доступным больше места, автоматически создается больше столбцов, а по мере уменьшения пространства количество столбцов также уменьшается. Это очень полезно в контекстах адаптивного веб-дизайна.

Позиционирование

Последним в этом обзоре механизмов компоновки является позиционирование. Свойство `position` изменяет то, как элемент ведет себя в обычном потоке документа и как он связан с другими элементами. Доступны варианты `relative`, `absolute` и `fixed` со `sticky` значением по умолчанию `static`.

```
.my-element {
  position: relative;
  top: 10px;
}
```

Этот элемент смещается на 10 пикселей вниз в зависимости от его текущей позиции в документе, поскольку он расположен относительно самого себя. Добавление `position: relative` к элементу также делает его содержащим блок любых дочерних элементов с `position: absolute`. Это означает, что его дочерний элемент теперь будет перемещен к этому конкретному элементу, а не к самому верхнему относительному родителю, когда к нему применяется абсолютная позиция.

```
.my-element {
  position: relative;
  width: 100px;
  height: 100px;
}

.another-element {
        position: absolute;
        bottom: 0;
        right: 0;
        width: 50px;
        height: 50px;
}
```

Когда вы устанавливаете `position` значение `absolute`, он вырывает элемент из текущего потока документа. Это означает две вещи:

1. Вы можете расположить этот элемент где угодно, используя `top`, `right` и `bottom` в `left` его ближайшем относительном родительском элементе.
2. Все содержимое, окружающее абсолютный элемент, перекомпоновывается, чтобы заполнить оставшееся пространство, оставленное этим элементом.

Элемент со `position` значением `fixed` ведет себя так же, как и `absolute`, при этом его родителем является корневой `<html>` элемент. Элементы с фиксированным положением остаются привязанными к левому верхнему углу в зависимости от установленных значений, `top` и . `rightbottomleft`

С помощью. `fixed` _ С этим значением, когда область просмотра прокручивается мимо элемента, она остается привязанной к установленным вами значениям, и . `relativestickytoprightbottomleft`

Флексбокс

Шаблон дизайна, который может быть сложным в адаптивном дизайне, — это боковая панель, встроенная в некоторый контент. Там, где есть пространство для окна просмотра, этот шаблон отлично работает, но там, где пространство сжато, такая жесткая компоновка может стать проблематичной.

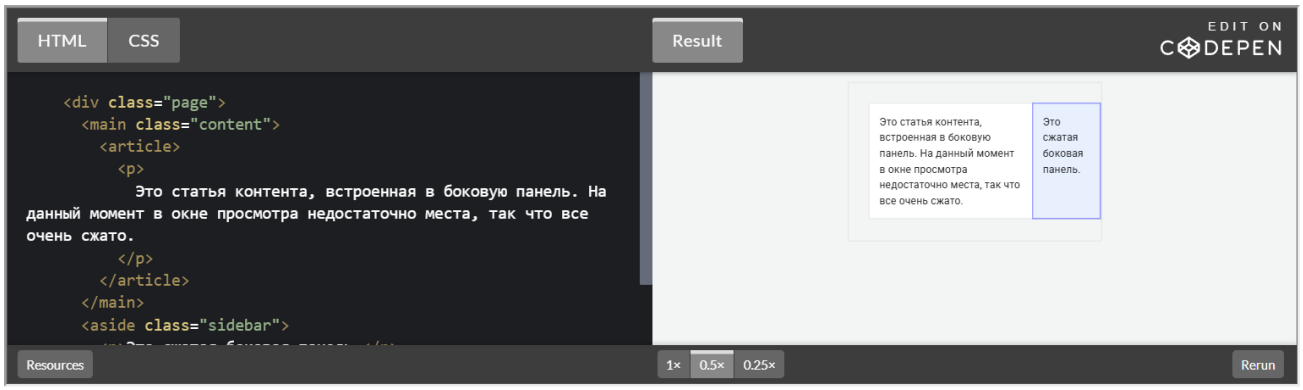


Рис. 7.7. Пример реализации элементов флексбокс

Модель макета гибкого блока (flexbox) *— это модель макета, предназначенная для одномерного контента. Он отлично справляется с набором элементов разного размера и возвращает лучший макет для этих элементов.*

Это идеальная модель макета для этого шаблона боковой панели. Flexbox не только помогает размещать боковую панель и содержимое внутри строки, но и там, где не хватает места, боковая панель переносится на новую строку. Вместо того, чтобы устанавливать жесткие размеры для браузера, с помощью flexbox вы можете вместо этого предоставить гибкие границы, чтобы намекнуть, как может отображаться контент.

Макеты Flex имеют следующие функции:

- Они могут отображаться в виде строки или столбца.
- Они уважают режим написания документа.
- По умолчанию они однострочные, но их можно попросить перенести на несколько строк.
- Элементы в макете можно визуально изменить в порядке, отличном от их порядка в DOM.
- Пространство внутри элементов может быть распределено, поэтому они становятся больше и меньше в соответствии с пространством, доступным в их родителях.
- Пространство можно распределить вокруг элементов и гибких линий в обернутом макете с помощью свойств выравнивания поля.
- Сами элементы можно выравнивать по поперечной оси.

Главная ось и поперечная ось

Ключом к пониманию flexbox является понимание **концепции главной оси** и **поперечной оси**. Основная ось – это та, которая задана вашим ресурсом `flex-direction`. Flex-элементы перемещаются группой по главной оси.

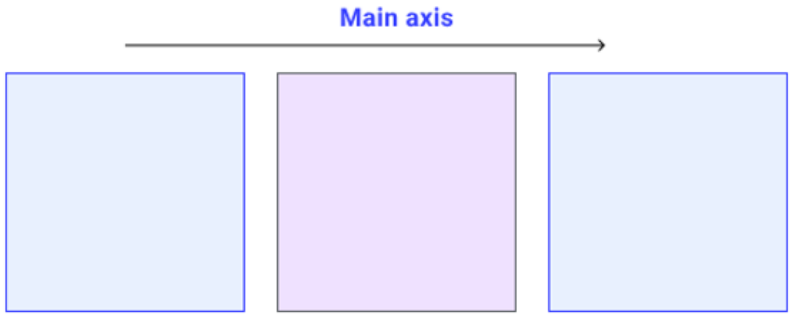


Рис. 7.8. Иллюстрация главной оси

Поперечная ось проходит в направлении, противоположном главной оси, поэтому поперечная ось проходит вдоль колонны `flex-direction`. `row`

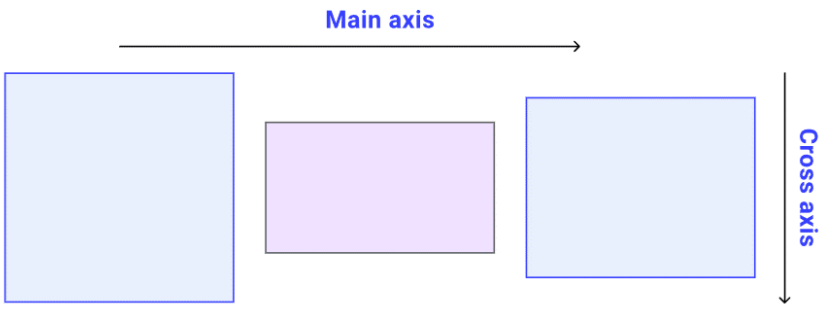


Рис. 7.9. Иллюстрация поперечной оси

Вы можете сделать две вещи на поперечной оси: можете перемещать элементы по отдельности или группой, чтобы они выровнялись относительно друг друга и гибкого контейнера; кроме того, если у вас есть обернутые гибкие строки, вы можете рассматривать эти строки как группу, чтобы контролировать, как пространство назначается этим строкам.

Создание гибкого контейнера

Давайте посмотрим, как ведет себя flexbox, взяв группу элементов разного размера и используя flexbox для их расположения.

```
<div class="container" id="container">
  <div>One</div>
  <div>Item two</div>
  <div>The item we will refer to as three</div>
</div>
```

Чтобы использовать flexbox, вам нужно объявить, что вы хотите использовать гибкий контекст форматирования, а не обычный блок и встроенный макет. Сделайте это, изменив значение свойства `display` на `flex`.

```
.container {
  display: flex;
}
```

Флекс-элементы немедленно начинают вести себя как флексбоксы, используя свои **начальные значения**.

Все свойства CSS имеют начальные значения, которые управляют их поведением «из коробки», когда вы не применили какой-либо CSS для изменения этого начального поведения. Дочерние элементы нашего контейнера flex становятся элементами flex, как только их родитель получает `display: flex`, поэтому эти начальные значения означают, что мы начинаем видеть некоторое поведение flexbox.

Начальные значения означают, что:

- Элементы отображаются в виде строки.
- Они не заворачиваются.
- Они не растут, чтобы заполнить контейнер.
- Они выстраиваются в начале контейнера.

Управление направлением элементов

Даже если вы еще не добавили свойство `flex-direction`, элементы отображаются в виде строки, поскольку начальное значение `flex-direction` равно `row`. Если вам нужна

строка, вам не нужно добавлять свойство. Чтобы изменить направление, добавьте свойство и одно из четырех значений:

- `row` : элементы располагаются в ряд.
- `row-reverse:` элементы располагаются в виде строки от конца гибкого контейнера.
- `column` : элементы располагаются в виде столбца.
- `column-reverse` : элементы располагаются в виде столбца с конца гибкого контейнера.

Реверсирование потока элементов и доступности

Вы должны быть осторожны при использовании любых свойств, которые изменяют порядок визуального отображения в отличие от того, как вещи упорядочиваются в документе HTML, поскольку это может негативно повлиять на доступность. Значения `row-reverse` и `column-reverse` являются хорошим примером этого. Изменение порядка происходит только для визуального, а не логического порядка. Это важно понимать, поскольку логический порядок — это порядок, в котором средство чтения с экрана будет читать содержимое, и ему будут следовать все, кто перемещается с помощью клавиатуры.

Все, что может изменить порядок элементов во flexbox или сетке, может вызвать эту проблему. Поэтому любое изменение порядка должно включать тщательное тестирование, чтобы убедиться, что это не затруднит использование вашего сайта для некоторых людей.

Режимы записи и направление

Элементы Flex по умолчанию располагаются в виде строки. Строка проходит в том же направлении, что и предложения в вашем режиме письма и направлении сценария. Это означает, что, если вы работаете на арабском языке, который имеет направление письма справа налево (rtl), элементы будут выстраиваться справа. Порядок табуляции также будет начинаться справа, так как именно так предложения читаются на арабском языке.

Если вы работаете с вертикальным режимом письма, как некоторые японские шрифты, тогда строка будет располагаться вертикально сверху вниз.

Поэтому поведение гибких элементов по умолчанию связано с режимом записи документа. Большинство руководств написаны с использованием английского языка или другого горизонтального режима письма слева направо. Это позволяет легко предположить, что гибкие элементы **располагаются слева** и располагаются **горизонтально**.

Учитывая основную и поперечную оси, а также режим записи, тот факт, что мы говорим о **начале** и **конце**, а не о верхней, нижней, левой и правой частях флексбокса, может быть легче понять. Каждая ось имеет начало и конец. Начало главной оси

называется **main-start**. Таким образом, наши гибкие элементы изначально выстраиваются в линию от main-start. Конец этой оси **main-end**. Начало поперечной оси — **cross-start**, а конец **— cross-end**.

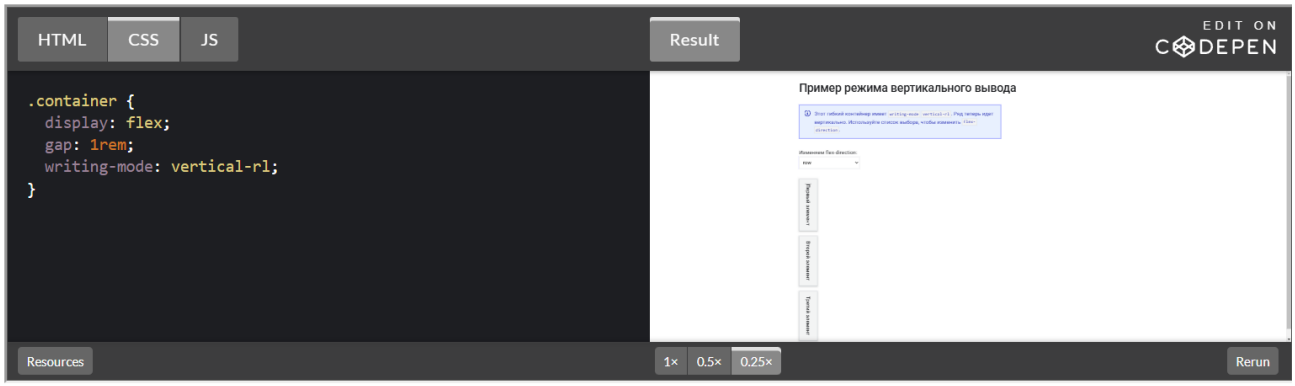


Рис. 7.10. Пример вертикального вывода элементов

Обертывание гибких элементов

Начальное значение свойства `flex-wrap` равно `nowrap`. Это означает, что, если в контейнере недостаточно места, элементы будут переполнены. Как только они достигнут минимального размера содержимого, гибкие элементы начнут переполнять свой контейнер.

Элементы, отображаемые с использованием начальных значений, будут уменьшаться настолько, насколько это возможно, вплоть до `min-content` размера, предшествующего переполнению.

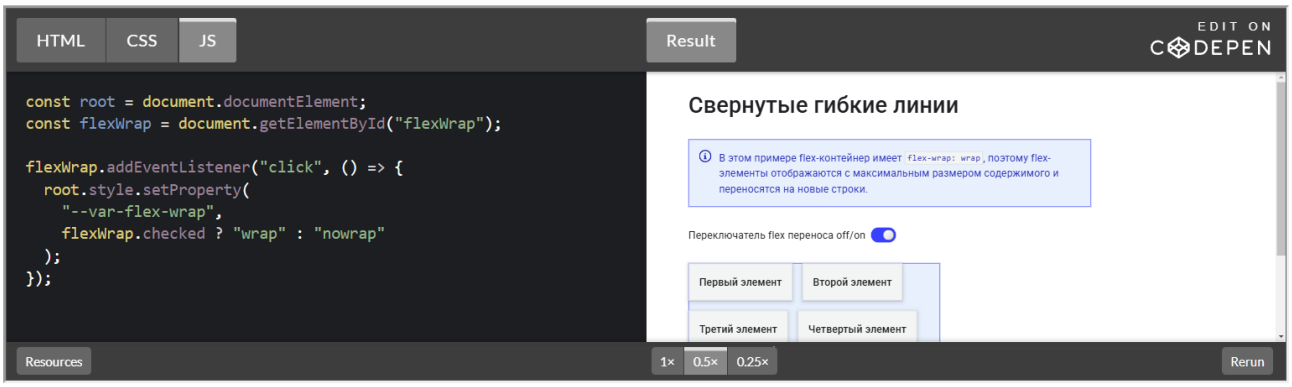


Рис. 7.11. Поведение элементов при переполнении

Чтобы элементы переносились, добавьте `flex-wrap: wrap` в контейнер flex.

```
.container {
  display: flex;
  flex-wrap: wrap;
```

```
}
```

Когда flex-контейнер сворачивает, он создает несколько **flex-линий**. С точки зрения распределения пространства каждая строка действует как новый гибкий контейнер. Поэтому, если вы переносите строки, невозможно заставить что-то в строке 2 выровняться с чем-то над ним в строке 1. Это то, что подразумевается под одномерностью flexbox. Вы можете управлять выравниванием по одной оси, строке или столбцу, а не по обоим вместе, как мы можем сделать в сетке.

Сокращение гибкого потока

Вы можете установить свойства `flex-direction` и `flex-wrap` с помощью сокращения `flex-flow`. Например, чтобы установить `flex-direction` и `column` разрешить перенос элементов:

```
.container {
  display: flex;
  flex-flow: column wrap;
}
```

Управление пространством внутри гибких элементов

Предполагая, что в нашем контейнере больше места, чем необходимо для отображения элементов, элементы выстраиваются в линию в начале и не увеличиваются, чтобы заполнить пространство. Они перестают расти при максимальном размере контента. Это связано с тем, что начальное значение `flex-` свойств:

- `flex-grow: 0`: предметы не растут.
- `flex-shrink: 1`: элементы могут уменьшаться меньше, чем их `flex-basis`.
- `flex-basis: auto`: элементы имеют базовый размер `auto`.

Это может быть представлено значением ключевого слова `flex: initial`. Сокращенное свойство `flex` или полное и `flex-grow` применяется к дочерним элементам гибкого контейнера . `flex-shrinkflex-basis`

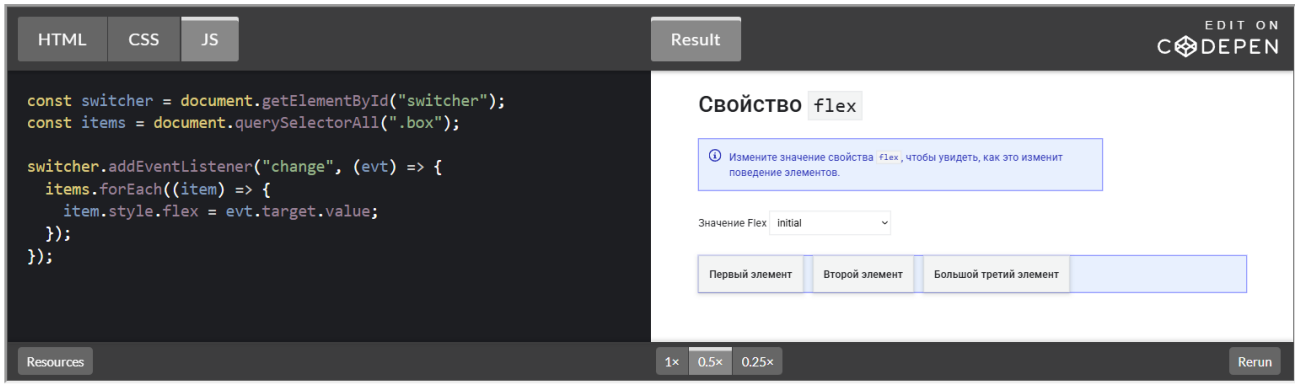


Рис. 7.12. Пример заполнения пространства

Чтобы элементы увеличивались, в то же время позволяя большим элементам занимать больше места, чем маленьким, использовать `flex:auto`. Этим управляют свойства:

- `flex-grow: 1`: элементы могут вырасти больше, чем их `flex-basis`.
- `flex-shrink: 1`: элементы могут уменьшаться меньше, чем их `flex-basis`.
- `flex-basis: auto`: элементы имеют базовый размер `auto`.

Использование `flex: auto` будет означать, что элементы в конечном итоге будут иметь разные размеры, поскольку пространство, которое используется совместно между элементами, распределяется после того, как каждый элемент выложен как максимальный размер содержимого. Так крупный предмет получит больше места. Чтобы заставить все элементы иметь одинаковый размер и игнорировать размер содержимого, измените `flex:auto` его на `flex: 1`.

Это распаковывается в:

- `flex-grow: 1`: элементы могут вырасти больше, чем их `flex-basis`.
- `flex-shrink: 1`: элементы могут уменьшаться меньше, чем их `flex-basis`.
- `flex-basis: 0`: элементы имеют базовый размер `0`.

Использование `flex: 1` говорит о том, что все элементы имеют нулевой размер, поэтому все пространство в гибком контейнере доступно для распределения. Поскольку у всех предметов есть `flex-grow` фактор `1`, все они растут одинаково, и пространство распределяется поровну.

Существует также значение `flex: none`, которое даст вам негибкие гибкие элементы, которые не увеличиваются и не уменьшаются. Это может быть полезно, если вы используете только flexbox для доступа к свойствам выравнивания, но не хотите никакого гибкого поведения.

Разрешение элементам расти с разной скоростью

Вам не нужно давать всем элементам `flex-grow` коэффициент `1`. Вы можете присвоить своим гибким элементам различные факторы `flex-grow`. В демо ниже первый элемент имеет `flex: 1`, второй `flex: 2` и третий `flex: 3`. По мере роста этих элементов `0` доступное пространство в гибком контейнере делится на шесть. На первый предмет отдается одна часть, на второй – две части, на третий – три части.

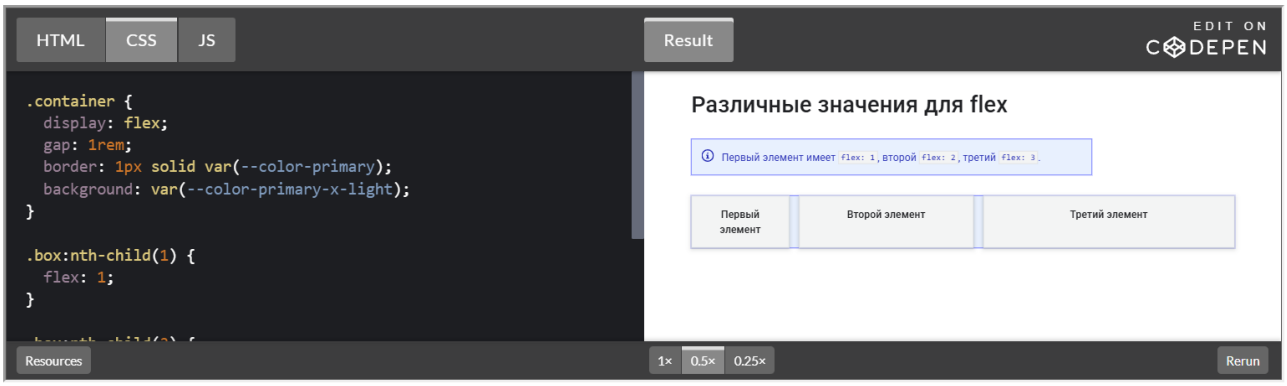


Рис. 7.13. Пример различной скорости роста элементов

Вы можете сделать то же самое из a `flex-basis` из `auto`, хотя вам нужно будет указать три значения. Первое значение равно `flex-grow`, второе `flex-shrink` и третье `flex-basis`.

```
.item1 {
  flex: 1 1 auto;
}

.item2 {
  flex: 2 1 auto;
}
```

Это менее распространенный вариант использования, так как причина использования `flex-basis` of `auto` заключается в том, чтобы позволить браузеру определить распределение пространства. Если вы хотите, чтобы элемент рос немного больше, чем решает алгоритм, это может быть полезно.

Изменение порядка гибких элементов

Элементы в вашем гибком контейнере можно переупорядочивать с помощью `order` свойства. Это свойство позволяет упорядочивать элементы в **порядковых группах**. Элементы располагаются в направлении, заданном `flex-direction`, сначала

самые низкие значения. Если несколько элементов имеют одинаковое значение, оно будет отображаться вместе с другими элементами с таким же значением.

Предупреждение. *Использование* `order` *имеет те же проблемы, что* `row-reverse` *и* `column-reverse` *значения* `flex-direction`*. Было бы очень легко создать автономный опыт для некоторых пользователей. Не используйте* `order`*, потому что вы исправляете то, что не в порядке в документе. Если элементы логически должны быть в другом порядке, измените свой HTML!*

Обзор выравнивания Flexbox

Flexbox принес с собой набор свойств для выравнивания элементов и распределения пространства между элементами. Эти свойства были настолько полезны, что с тех пор они были перемещены в собственную спецификацию, и вы встретите их и в Grid Layout. Здесь вы можете узнать, как они работают, когда вы используете flexbox.

Набор свойств можно разделить на две группы. Свойства распределения пространства и свойства выравнивания. Свойства, которые распределяют пространство:

- `justify-content`: распределение пространства по главной оси.
- `align-content`: пространственное распределение по поперечной оси.
- `place-content`: сокращение для установки обоих вышеуказанных свойств.

Свойства, используемые для выравнивания во flexbox:

- `align-self`: выравнивает один элемент по поперечной оси
- `align-items`: выравнивает все элементы как группу по поперечной оси

Если вы работаете с главной осью, то свойства начинаются с `justify-`. На поперечной оси они начинаются с `align-`.

Распределение пространства по главной оси

При использовании HTML, который использовался ранее, flex-элементы расположены в виде строки, на главной оси есть пространство. Элементы недостаточно велики, чтобы полностью заполнить гибкий контейнер. Элементы располагаются в начале контейнера flex, потому что начальное значение `justify-content` равно `flex-start`. Элементы выстраиваются в линию в начале, а любое дополнительное пространство в конце.

Добавьте `justify-content` свойство во гибкий контейнер, присвойте ему значение `flex-end`, и элементы выстроятся в конец контейнера, а свободное пространство будет помещено в начало.

```
.container {
  display: flex;
  justify-content: flex-end;
}
```

Вы также можете распределить пространство между элементами с помощью `justify-content: space-between`.

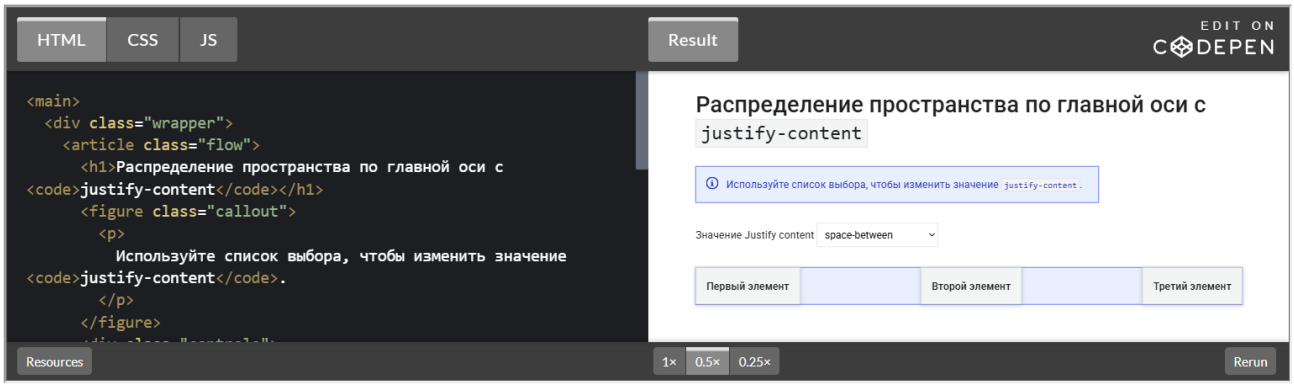


Рис. 7.14. Распределение элементов

Чтобы свойство `justify-content` что-либо делало, у вас должно быть свободное место в вашем контейнере на главной оси. Если ваши элементы заполняют ось, то нет места для совместного использования, поэтому свойство ничего не делает.

С flex-direction: column

Если вы изменили свой `flex-direction` на `column`, то `justify-content` будет работать на столбце. Чтобы иметь свободное место в вашем контейнере при работе в качестве столбца, вам нужно указать контейнеру а `height` или `block-size`. В противном случае у вас не будет свободного места для распространения.

Распределение пространства между гибкими строками

С обернутым гибким контейнером у вас может быть место для распределения по поперечной оси. В этом случае вы можете использовать свойство `align-content` с теми же значениями, что и `justify-content`. В отличие от того `justify-content`, что выравнивает элементы `flex-start` по умолчанию, начальное значение `align-content` равно `stretch`. Добавьте свойство `align-content` в контейнер flex, чтобы изменить это поведение по умолчанию.

```
.container {
  align-content: center;
}
```

Попробуйте это в демо. В примере есть обернутые строки гибких элементов, а в контейнере есть `block-size`, чтобы у нас было немного свободного места.

Чтобы установить оба `justify-content` и `align-content` вы можете использовать `place-content` с одним или двумя значениями. Для обеих осей будет использоваться одно значение, если вы укажете, что первая используется для `align-content`, а вторая — для `justify-content`.

```
.container {
  place-content: space-between;
  /* sets both to space-between */
}

.container {
  place-content: center flex-end;
  /* wrapped lines on the cross axis are centered,
  on the main axis items are aligned to the end of the flex container */
}
```

Выравнивание элементов по поперечной оси

На поперечной оси вы также можете выровнять свои элементы в пределах гибкой линии, используя `align-items` и `align-self`. Пространство, доступное для этого выравнивания, будет зависеть от высоты гибкого контейнера или гибкой линии в случае обернутого набора элементов.

Начальное значение `align-self равно` `stretch`, поэтому гибкие элементы в строке по умолчанию растягиваются до высоты самого высокого элемента. Чтобы изменить это, добавьте свойство `align-self` к любому из ваших flex-элементов.

```
.container {
  display: flex;
}

.item1 {
  align-self: flex-start;
}
```

Используйте любое из следующих значений для выравнивания элемента:

- `flex-start`
- `flex-end`
- `center`
- `stretch`
- `baseline`

В следующем примере есть одна строка гибких элементов с расширением `flex-direction: row`. Последний элемент определяет высоту гибкого контейнера. Первый элемент имеет свойство `align-self` со значением `flex-start`. Попробуйте изменить значение этого свойства, чтобы увидеть, как оно перемещается в пределах своего пространства на поперечной оси.

Свойство `align-self` применяется к отдельным элементам. Это свойство `align-items` можно применить к гибкому контейнеру, чтобы установить все отдельные свойства `align-self` как группу.

```
.container {
  display: flex;
  align-items: flex-start;
}
```

Попробуйте изменить значение, `align-items` чтобы выровнять все элементы на поперечной оси как группу.

Элементы Flex действуют как группа на главной оси. Таким образом, нет концепции разделения отдельного элемента из этой группы.

В макете сетки свойства `justify-self` и `justify-items` работают на встроенной оси, чтобы выполнить выравнивание элементов на этой оси в пределах их области сетки. Из-за того, что гибкие макеты обрабатывают элементы как группу, эти свойства не реализованы в гибком контексте.

Стоит знать, что flexbox очень хорошо работает с автоматическими полями. Если вы столкнулись с необходимостью отделить один элемент от группы или разделить группу на две группы, вы можете применить для этого отступ. В приведенном ниже примере последний элемент имеет левое поле `auto`. Автоматическое поле поглощает все пространство в том направлении, в котором оно применяется. Это означает, что он сдвигает элемент вправо, тем самым разделяя группы.

Центрирование элемента по вертикали и горизонтали

Свойства выравнивания можно использовать для центрирования элемента внутри другого блока. Свойство `justify-content` выравнивает элемент по главной оси, то есть по строке. Свойство `align-items` на поперечной оси.

```
.container {
  width: 400px;
  height: 300px;
  display: flex;
  justify-content: center;
  align-items: center;
}
```

В будущем мы сможем сделать это выравнивание без необходимости делать родителя гибким контейнером. Свойства выравнивания указаны для блочного и встроенного макета. В настоящее время ни один браузер не реализовал их. Однако переключение в контекст форматирования flex дает вам доступ к свойствам. Если вам нужно что-то выровнять, это отличный способ сделать это.

Сетка

Очень распространенный макет в веб-дизайне — это макет заголовка, боковой панели, тела и нижнего колонтитула.

За прошедшие годы было предложено много способов выполнения такой компоновки, но с сеткой CSS это не только относительно просто, но и ничто не ограничивает в выборе различных вариантов. Сетка является идеальным инструментом для такого рода макетов. Это связано с тем, что сетка — это метод компоновки, разработанный для двумерного контента. Таким образом, есть возможность раскладывать объекты по строкам и столбцам одновременно. При создании макета сетки мы определяем сетку со строками и столбцами. Затем размещаем элементы в этой сетке или браузер автоматически размещает их в созданных ячейках.

Макеты сетки имеют следующие особенности:

1. Сетка может быть определена строками и столбцами. Вы можете выбрать размер этих дорожек строк и столбцов, или они могут реагировать на размер содержимого.
2. Прямые дочерние элементы контейнера сетки будут автоматически помещены в эту сетку.
3. Или вы можете разместить элементы в точном месте, которое вы хотите.
4. Линии и области на сетке могут быть названы, чтобы упростить размещение.
5. Свободное пространство в контейнере сетки можно распределить между дорожками.
6. Элементы сетки можно выравнивать в пределах своей области.

Grid поставляется с кучей новой терминологии, так как это первый раз, когда CSS имеет настоящую систему компоновки.

Линии сетки

Сетка состоит из линий, которые проходят горизонтально и вертикально. Если ваша сетка состоит из четырех столбцов, она будет состоять из пяти строк столбцов, включая один после последнего столбца.

Строки нумеруются, начиная с 1, с нумерацией в соответствии с режимом записи и направлением скрипта компонента. Это означает, что строка столбца 1 будет слева на языке с письмом слева направо, таком как английский, и справа на языке с письмом справа налево, таком как арабский.

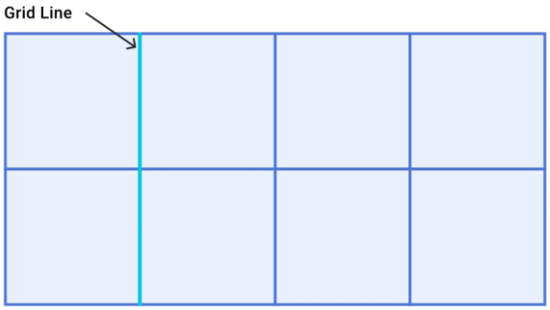


Рис. 7.15. Иллюстрация линий сетки

Треки сетки

Трек *— это пространство между двумя линиями сетки.*

Дорожка строк находится между двумя строками строк, а дорожка столбцов — между двумя линиями столбцов. Когда мы создаем нашу сетку, мы создаем эти дорожки, назначая им размер.

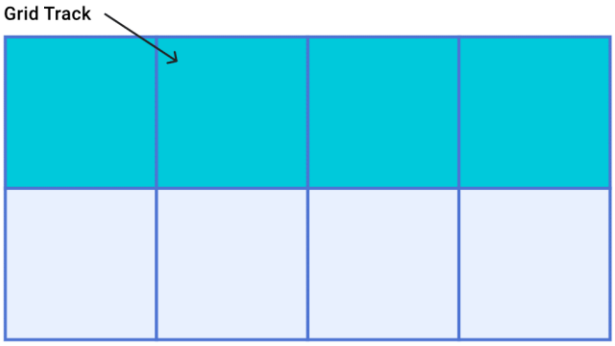


Рис. 7.16. Иллюстрация трека

Ячейка сетки

Ячейка сетки *— это наименьшее пространство в сетке, определяемое пересечением дорожек строк и столбцов.*

Это как ячейка таблицы или ячейка в электронной таблице. Если вы определите сетку и не поместите какие-либо элементы, они будут автоматически размещены по одному элементу в каждой определенной ячейке сетки.

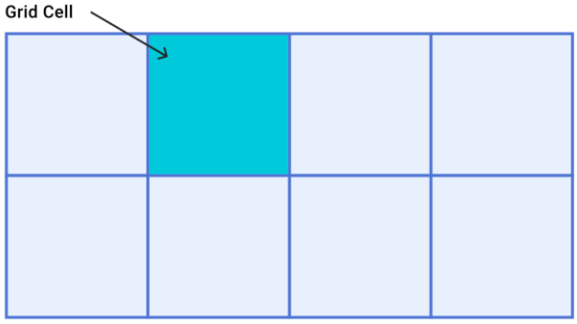


Рис. 7.17. Иллюстрация ячеек сетки

Площадь сетки

Несколько ячеек сетки вместе. Области сетки создаются путем размещения элемента на нескольких дорожках.

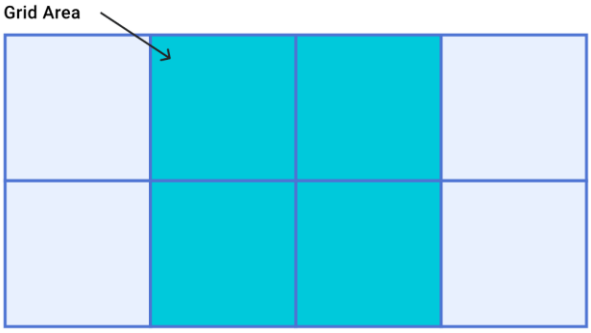


Рис. 7.18. Иллюстрация площади сетки

Пробелы

Желоб или переулок между путями. В целях определения размера они действуют как обычные гусеницы. Вы не можете разместить контент в промежутке, но вы можете распределить элементы сетки по нему.

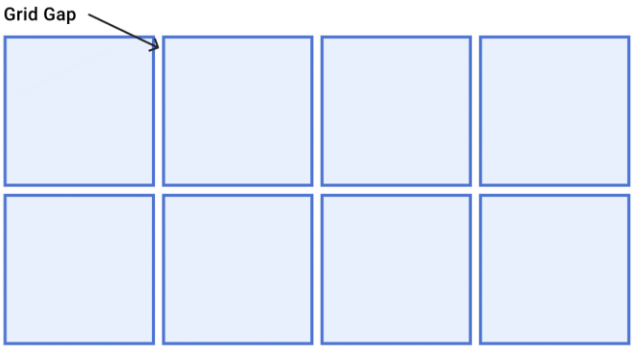


Рис. 7.19. Иллюстрация пробелов сетки

Контейнер сетки

Элемент HTML `display: grid` создает новый контекст форматирования сетки для прямых дочерних элементов.

```
.container {
  display: grid;
}
```

Элемент сетки

Элемент сетки — это элемент, который является прямым дочерним элементом контейнера сетки.

```
<div class="container">
  <div class="item"></div>
  <div class="item"></div>
  <div class="item"></div>
</div>
```

Строки и столбцы

Чтобы создать базовую сетку, вы можете определить сетку с тремя дорожками столбцов, двумя дорожками строк и промежутком в 10 пикселей между дорожками следующим образом.

```
.container {
    display: grid;
    grid-template-columns: 5em 100px 30%;
    grid-template-rows: 200px auto;
    gap: 10px;
}
```

Эта сетка демонстрирует многие вещи, описанные в разделе терминологии. Он имеет три колоночные дорожки. Каждая дорожка использует разные единицы длины. Он имеет две дорожки, одна из которых использует единицу измерения длины, а другая — автоматическую. При использовании в качестве размера дорожки auto можно рассматривать как такой же большой, как и содержимое. Треки имеют автоматический размер по умолчанию.

Если у элемента с классом `.container` есть дочерние элементы, они сразу разложатся по этой сетке. Вы можете увидеть это в действии в демо ниже.

Ключевые слова внутреннего размера

В дополнение к размерам длины и процентам, как описано в разделе о единицах измерения , треки сетки могут использовать внутренние ключевые слова размера. Эти ключевые слова определены в спецификации Box Sizing и добавляют дополнительные методы изменения размеров блоков в CSS, а не только дорожки сетки.

- `min-content`
- `max-content`
- `fit-content()`

Ключевое слово `min-content` сделает дорожку настолько маленькой, насколько это возможно, без переполнения содержимого дорожки. Изменение макета сетки в примере на три дорожки столбца одинакового `min-content` размера будет означать, что они станут такими же узкими, как самое длинное слово в дорожке.

Ключевое слово `max-content` имеет противоположный эффект. Трек станет достаточно широким, чтобы весь контент отображался одной длинной непрерывной строкой. Это может привести к переполнению, так как строка не будет перенесена.

Функция `fit-content()` действует как `max-content` в первый раз. Однако, как только дорожка достигает размера, который вы передаете в функцию, содержимое начинает переноситься. Таким образом `fit-content(10em)` , будет создана дорожка размером менее 10em, если размер `max-content` меньше 10em, но никогда не больше 10em.

Единица fr _

У нас есть существующие размеры длины, проценты, а также эти новые ключевые слова. Существует также специальный метод изменения размера, который работает только в сетке. Это единица `fr` , гибкая длина, которая описывает долю доступного пространства в грид-контейнере.

Модуль `fr` работает аналогично использованию `flex: auto` во flexbox. Он распределяет пространство после того, как предметы были выложены. Поэтому иметь три столбца, которые получают одинаковую долю доступного пространства:

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
}
```

Поскольку блок fr разделяет доступное пространство, его можно комбинировать с зазором фиксированного размера или дорожками фиксированного размера. Чтобы иметь компонент с элементом фиксированного размера и второй дорожкой, занимающей все оставшееся место, вы можете использовать в качестве списка дорожек файлы `grid-template-columns: 200px 1fr`.

Использование разных значений для единицы fr пропорционально разделит пространство. Большие значения получают больше свободного места. В демо ниже измените значение третьей дорожки.

Функция minmax() _

Эта функция означает, что вы можете установить минимальный и максимальный размер дорожки. Это может быть очень полезно. Если мы возьмем приведенный выше пример модуля, который распределяет оставшееся пространство, его можно записать, используя `minmax()` как `minmax(auto, 1fr)`. Grid оценивает внутренний размер содержимого, а затем распределяет доступное пространство после того, как для содержимого будет достаточно места. Это означает, что вы можете не получить дорожки, каждая из которых занимает равную долю всего пространства, доступного в контейнере сетки.

Чтобы заставить дорожку занимать равную долю пространства в контейнере сетки за вычетом пробелов, используйте minmax. Замените `1fr` в качестве размера дорожки на `minmax(0, 1fr)`. Это делает минимальный размер дорожки равным 0, а не размер минимального содержимого. Затем сетка возьмет весь доступный размер в контейнере, вычтет размер, необходимый для любых промежутков, и разделит оставшуюся часть в соответствии с вашими единицами fr.

repeat()

Если вы хотите создать сетку дорожек из 12 столбцов с одинаковыми столбцами, вы можете использовать следующий CSS.

```
.container {
    display: grid;
    grid-template-columns:
```

```
        minmax(0,1fr),
        minmax(0,1fr),
        minmax(0,1fr),
        minmax(0,1fr),
        minmax(0,1fr),
        minmax(0,1fr),
        minmax(0,1fr),
        minmax(0,1fr),
        minmax(0,1fr),
        minmax(0,1fr),
        minmax(0,1fr),
        minmax(0,1fr);
    }
```

Или вы можете написать это, используя `repeat()`:

```
    .container {
        display: grid;
        grid-template-columns: repeat(12, minmax(0,1fr));
    }
```

Эту `repeat()` функцию можно использовать для повтора любого раздела вашего трек-листа. Например, вы можете повторить шаблон дорожек. Вы также можете иметь несколько обычных треков и повторяющийся раздел.

```
    .container {
        display: grid;
        grid-template-columns: 200px repeat(2, 1fr 2fr) 200px; /*creates 6 tracks*/
    }
```

auto-fill и auto-fit

Вы можете комбинировать все, что вы узнали о размерах дорожек `minmax()`, и повторять, чтобы создать полезный шаблон с макетом сетки. Возможно, вы не хотите указывать количество дорожек столбцов, а вместо этого хотите создать столько, сколько поместится в вашем контейнере.

Вы можете добиться этого с помощью ключевых слов `repeat()` и `auto-fill` или `auto-fit`.

Автоматическое размещение

Элементы помещаются в сетку по одному на ячейку в том порядке, в котором они появляются в источнике. Для многих макетов это может быть все, что вам нужно. Если

вам нужно больше контроля, есть несколько вещей, которые вы можете сделать. Во-первых, настроить макет автоматического размещения.

Размещение элементов в столбцах

Поведение сетки по умолчанию заключается в размещении элементов вдоль строк. Вместо этого вы можете разместить элементы в столбцах, используя `grid-auto-flow: column`. Вам необходимо определить дорожки строк, иначе элементы будут создавать внутренние дорожки столбцов и располагаться в одной длинной строке.

Эти значения относятся к режиму записи документа. Строка всегда идет в направлении, в котором идет предложение в режиме записи документа или компонента.

Объединение дорожек

Вы можете сделать так, чтобы некоторые или все элементы в макете с автоматическим размещением занимали более одной дорожки. Используйте ключевое слово `span` плюс количество строк в качестве значения для `grid-column-end` или `grid-row-end`.

```
.item {
    grid-column-end: span 2; /* will span two lines, therefore covering two tracks */
}
```

Поскольку вы не указали a `grid-column-start`, он использует начальное значение `auto` и размещается в соответствии с правилами автоматического размещения. Вы также можете указать то же самое, используя сокращение `grid-column`:

```
.item {
    grid-column: auto / span 2;
}
```

Заполнение пробелов

Автоматически размещенный макет с некоторыми элементами, охватывающими несколько дорожек, может привести к сетке с некоторыми незаполненными ячейками. По умолчанию макет сетки с полностью автоматически размещаемым макетом всегда продвигается вперед. Элементы будут размещены в соответствии с порядком, в котором они находятся в источнике, или в соответствии с любой модификацией `order` свойства. Если для размещения элемента недостаточно места, сетка оставит зазор и перейдет к следующей дорожке.

Размещение предметов

У вас уже есть много функций из CSS Grid. Давайте теперь посмотрим, как мы размещаем элементы в созданной нами сетке.

Первое, что нужно помнить, это то, что CSS Grid Layout основан на сетке из пронумерованных строк. Самый простой способ разместить объекты на сетке — разместить их с одной строки на другую.

Свойства, которые можно использовать для размещения элементов по номеру строки:

- `grid-column-start`
- `grid-column-end`
- `grid-row-start`
- `grid-row-end`

У них есть сокращения, которые позволяют вам установить как начальную, так и конечную линии одновременно:

- `grid-column`
- `grid-row`

Чтобы разместить ваш элемент, установите начальную и конечную линии области сетки, в которую он должен быть помещен.

```
.container {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    grid-template-rows: repeat(2, 200px 100px);
}

.item {
    grid-column-start: 1; /* start at column line 1 */
    grid-column-end: 4; /* end at column line 4 */
    grid-row-start: 2; /*start at row line 2 */
    grid-row-end: 4; /* end at row line 4 */
}
```

Chrome DevTools может дать вам визуальное руководство по линиям, чтобы проверить, где находится ваш элемент.

Нумерация строк соответствует способу написания и направлению компонента.

Укладка предметов

Используя линейное позиционирование, вы можете размещать элементы в одной и той же ячейке сетки. Это означает, что вы можете складывать элементы друг на друга или делать так, чтобы один элемент частично перекрывал другой. Элементы, которые появляются позже в источнике, будут отображаться поверх элементов, которые появляются раньше. Вы можете изменить этот порядок укладки с помощью `z-index` так же, как и с позиционированными элементами.

Отрицательные номера строк

Когда вы создаете сетку с помощью `grid-template-rows` и `grid-template-columns` вы создаете так называемую **явную сетку**. Это сетка, которую вы определили и задали размер дорожкам.

Иногда у вас будут элементы, которые отображаются за пределами этой явной сетки. Например, вы можете определить дорожки столбцов, а затем добавить несколько строк элементов сетки, даже не определяя дорожки строк. Треки будут иметь автоматический размер по умолчанию. Вы также можете разместить элемент `grid-column-end`, который находится за пределами явно заданной сетки. В обоих этих случаях сетка будет создавать дорожки, чтобы макет работал, и эти дорожки называются **неявной сеткой**.

В большинстве случаев не имеет значения, работаете ли вы с неявной или явной сеткой. Однако при линейном размещении вы можете столкнуться с основным различием между ними.

Используя отрицательные номера строк, вы можете размещать элементы из конечной строки явной сетки. Это может быть полезно, если вы хотите, чтобы элемент располагался от первой до последней строки столбца. В таком случае вы можете использовать `grid-column: 1 / -1`. Элемент будет растягиваться прямо по явной сетке.

Однако это работает только для явной сетки. Возьмите макет из трех рядов автоматически размещенных элементов, где вы хотели бы, чтобы самый первый элемент простирался до конечной линии сетки.

Вы можете подумать, что можете дать этот предмет `grid-row: 1 / -1`. В демо ниже вы можете увидеть, что это не работает. Дорожки создаются в неявной сетке, нет возможности добраться до конца сетки с помощью `-1`.

Размер неявных дорожек

Треки, созданные в неявной сетке, будут иметь автоматический размер по умолчанию. Однако, если вы хотите контролировать размер строк, используйте свойство `grid-auto-rows`, а для колонок - `grid-auto-columns`.

Чтобы создать все неявные строки с минимальным размером `10em` и максимальным размером `auto`:

```
.container {
    display: grid;
    grid-auto-rows: minmax(10em, auto);
}
```

Чтобы создать неявные столбцы с шаблоном дорожек шириной 100 и 200 пикселей. В этом случае первый неявный столбец будет иметь размер 100 пикселей, второй — 200 пикселей, третий — 100 пикселей и так далее.

```
.container {
    display: grid;
    grid-auto-columns: 100px 200px;
}
```

Именованные линии сетки

Размещение элементов в макете может быть проще, если строки имеют имя, а не номер. Вы можете назвать любую линию в вашей сетке, добавив имя по вашему выбору в квадратных скобках. Можно добавить несколько имен, разделенных пробелом в одних и тех же скобках. После того, как вы назвали линии, их можно использовать вместо номеров.

```
.container {
    display: grid;
    grid-template-columns:
      [main-start aside-start] 1fr
      [aside-end content-start] 2fr
      [content-end main-end]; /* a two column layout */
}

.sidebar {
    grid-column: aside-start / aside-end;
    /* placed between line 1 and 2*/
}

footer {
```

```
    grid-column: main-start / main-end;
    /* right across the layout from line 1 to line 3*/
}
```

Области шаблона сетки

Вы также можете называть области сетки и размещать элементы в этих именованных областях. Это прекрасная техника, поскольку она позволяет вам увидеть, как выглядит ваш компонент прямо в CSS.

Для начала дайте имена прямым дочерним элементам контейнера сетки, используя свойство `grid-area`:

```
header {
    grid-area: header;
}

.sidebar {
    grid-area: sidebar;
}

.content {
    grid-area: content;
}

footer {
    grid-area: footer;
}
```

Имя может быть любым, кроме ключевых слов `auto` и `span`. Как только все ваши элементы будут названы, используйте свойство `grid-template-areas`, чтобы определить, какие ячейки сетки будут охватывать каждый элемент. Каждая строка определяется в кавычках.

```
.container {
    display: grid;
    grid-template-columns: repeat(4,1fr);
    grid-template-areas:
        "header header header header"
        "sidebar content content content"
        "sidebar footer footer footer";
}
```

При использовании есть несколько правил `grid-template-areas`.

- Значение должно быть полной сеткой без пустых ячеек.
- Для охвата дорожек повторяйте название.
- Области, создаваемые повторением имени, должны быть прямоугольными и не могут быть разъединены.

Если вы нарушаете какое-либо из вышеперечисленных правил, значение считается недействительным и выбрасывается.

Чтобы оставить пустое пространство в сетке, используйте множители `.` или без пробелов между ними. Например, чтобы оставить самую первую ячейку в сетке пустой, я мог бы добавить ряд `.` символов:

```
.container {
    display: grid;
    grid-template-columns: repeat(4,1fr);
    grid-template-areas:
        "..... header header header"
        "sidebar content content content"
        "sidebar footer footer footer";
}
```

Поскольку весь макет определяется в одном месте, его можно легко переопределить с помощью медиа-запросов. В следующем примере я создал макет из двух столбцов, который переходит к трем столбцам путем переопределения значения `grid-template-columns` и `grid-template-areas`. Откройте пример в новом окне, чтобы поиграть с размером области просмотра и увидеть изменение макета.

Вы также можете увидеть, как свойство `grid-template-areas` связано с направлением `writing-mode`, как и в других методах сетки.

Сокращенные свойства

Есть два сокращенных свойства, которые позволяют вам установить многие свойства сетки за один раз. Это может показаться немного запутанным, пока вы не разберетесь, как именно они сочетаются. Хотите ли вы использовать их или предпочитаете использовать длинные руки, зависит от вас.

grid-template

Свойство `grid-template` является сокращением для `grid-template-rows`, `grid-template-columns` и `grid-template-areas`. Сначала определяются строки вместе со значением `grid-template-areas`. Размер столбца добавляется после файла `/`.

```
.container {
```

```
    display: grid;
    grid-template:
      "head head head" minmax(150px, auto)
      "sidebar content content" auto
      "sidebar footer footer" auto / 1fr 1fr 1fr;
}
```

свойство grid

Сокращение `grid` можно использовать точно так же, как и `grid-template`. При таком использовании он сбрасывает другие свойства сетки, которые он принимает, к их начальным значениям. Полный набор:

- `grid-template-rows`
- `grid-template-columns`
- `grid-template-areas`
- `grid-auto-rows`
- `grid-auto-columns`
- `grid-auto-flow`

Вы можете поочередно использовать это сокращение, чтобы определить, как ведет себя неявная сетка, например:

```
.container {
    display: grid;
    grid: repeat(2, 80px) / auto-flow  120px;
}
```

Выравнивание

В сетке свойства, которые начинаются с `justify-`, всегда используются на встроенной оси, направлении, в котором идут предложения в вашем режиме письма.

Свойства, которые начинаются с `align-`, используются на оси блока, направлении, в котором блоки располагаются в вашем режиме письма.

- `justify-content` и `align-content`: распределить дополнительное пространство в контейнере сетки вокруг дорожек или между ними.
- `justify-self` и `align-self`: применяются к элементу сетки, чтобы перемещать его внутри области сетки, в которой он находится.

- `justify-items` и `align-items`: применяются к контейнеру сетки для установки всех свойств элементов `justify-self`.

Задания к практической работе

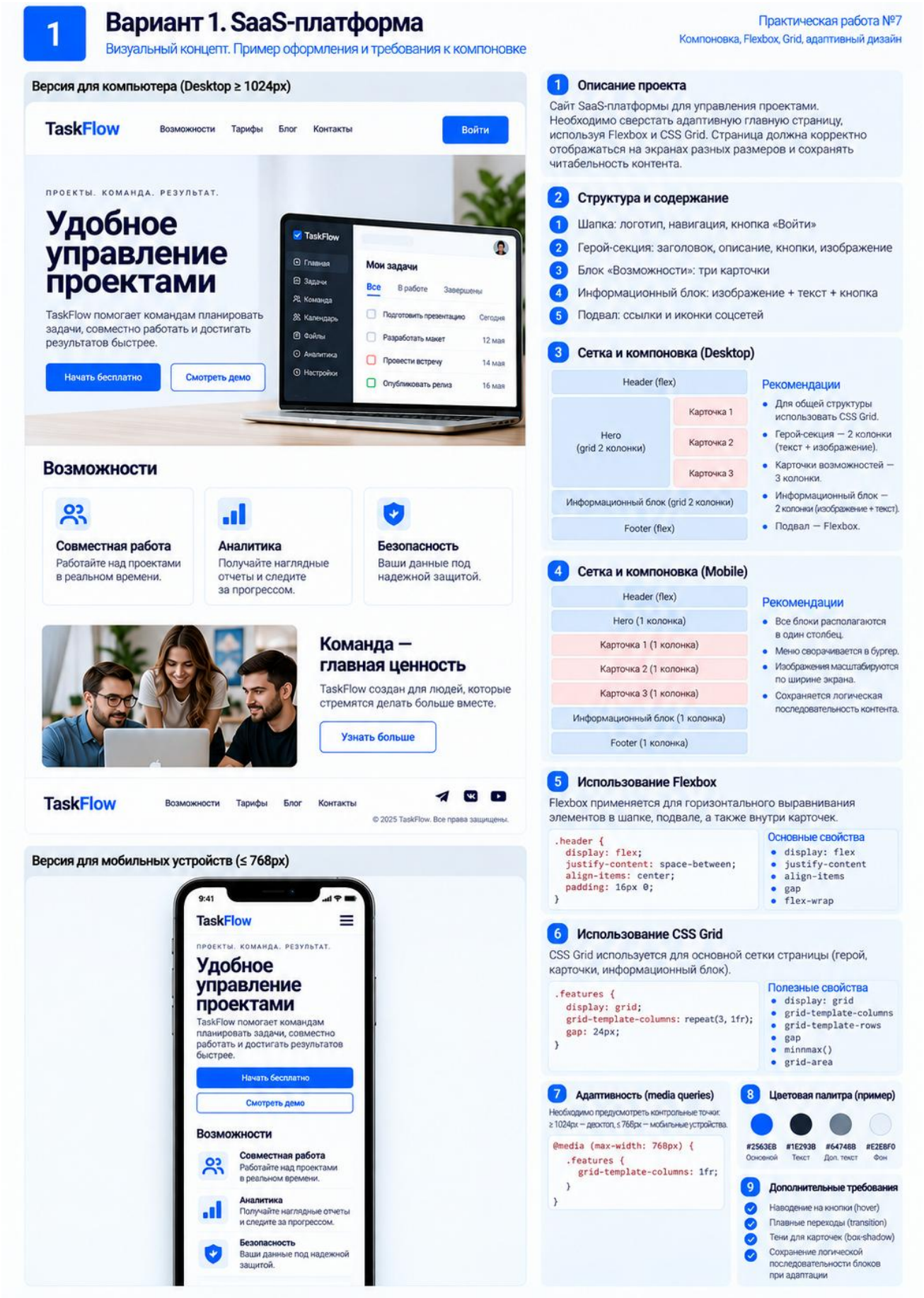
Задание 1. Определиться с номером варианта

Выбрать один вариант из 15 предложенных. Изучить соответствующий визуальный концепт, который реализует механизм изменения представления и страницу тарифного плана в Desktop и Mobile версиях.

Варианты индивидуальных заданий

Каждый вариант содержит предметную область, обязательные смысловые блоки, рекомендуемые HTML5-элементы и визуальные требования.

Вариант 1. Тарифный план SaaS



Разработать адаптивную веб-страницу SaaS-сервиса **TaskFlow**, предназначенного для управления проектами и совместной работы команды. Страница должна воспроизводить структуру и визуальную логику представленного макета, отдельно реализуя Desktop- и Mobile-представления.

В верхней части страницы разместить **Header** с логотипом TaskFlow, пунктами навигации «Возможности», «Тарифы», «Блог», «Контакты» и кнопкой «Войти».

Далее реализовать **Hero-блок** с небольшим информационным заголовком «ПРОЕКТЫ. КОМАНДА. РЕЗУЛЬТАТ.», основным заголовком **«Удобное управление проектами»**, кратким описанием сервиса, кнопками **«Начать бесплатно»** и **«Смотреть демо»**, а также изображением интерфейса TaskFlow на ноутбуке.

После Hero разместить секцию **«Возможности»**, содержащую три карточки:

1. **Совместная работа** — работа над проектами в реальном времени;
2. **Аналитика** — получение наглядных отчётов и отслеживание прогресса;
3. **Безопасность** — защита данных.

Ниже разместить информационный блок **«Команда — главная ценность»** с фотографией команды, описанием и кнопкой **«Узнать больше»**.

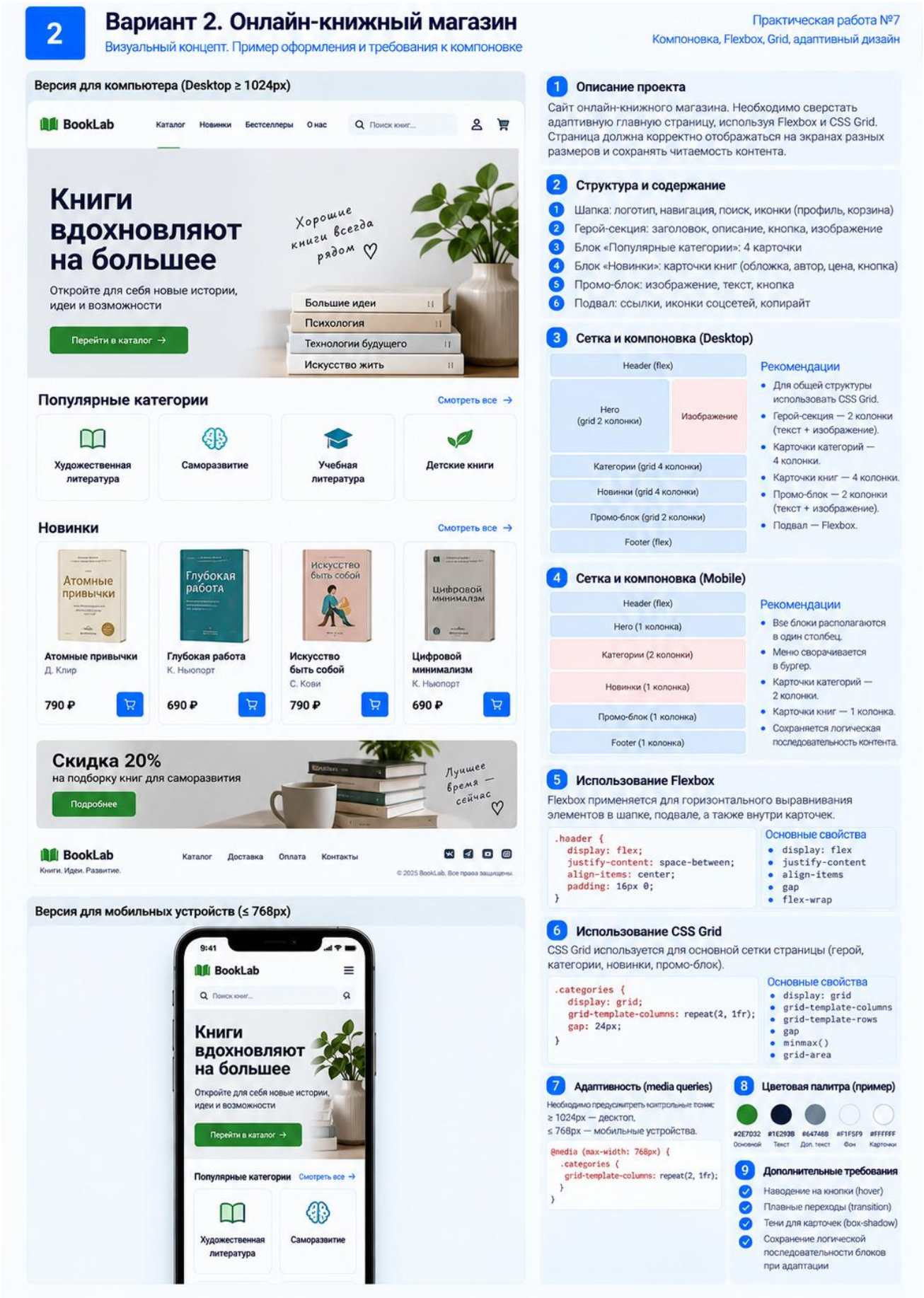
В нижней части страницы предусмотреть **Footer** с логотипом, повторной навигацией, иконками социальных сетей и информацией об авторских правах.

Требования к компоновке:

- на Desktop Hero должен состоять из текстовой и визуальной областей;
- три карточки секции «Возможности» должны располагаться в одну строку с помощью **CSS Grid**;
- Header, группы кнопок и элементы Footer выровнять с помощью **Flexbox**;
- на Mobile все основные блоки должны переходить в последовательный вертикальный поток;
- карточки «Возможности» должны перестраиваться из трёх колонок в одну;
- изображения не должны выходить за границы контейнера и должны сохранять пропорции;
- предусмотреть ручное переключение **Desktop / Mobile** в Header или Nav;
- при изменении ширины окна страница должна автоматически перестраиваться с использованием **media queries**;
- в качестве контрольной границы адаптации использовать **768 px**, как указано в общем задании практической работы.

При реализации нельзя использовать весь макет как одно фоновое изображение. Все элементы страницы должны быть созданы средствами **HTML и CSS**, а изображения подключены отдельно.

Вариант 2. Онлайн-книжный магазин



Разработать адаптивную веб-страницу **онлайн-кинотеатра** с подборками фильмов и сериалов. Страница должна воспроизводить структуру и визуальную логику представленного макета, отдельно реализуя Desktop- и Mobile-представления. В задании для Варианта 2 указана композиция **Header + поиск + Hero-фильм + категории + сетка постеров + боковой блок**.

В верхней части страницы разместить **Header** с логотипом онлайн-кинотеатра, основной навигацией по разделам каталога, элементами управления и переключателем представления **Desktop / Mobile**.

Ниже разместить строку **поиска**, предназначенную для поиска фильмов и сериалов. Поле поиска и управляющие элементы должны быть расположены компактно и не выходить за пределы контейнера.

Основную часть страницы начать с **Hero-блока**, посвящённого выбранному фильму. В блоке разместить крупное изображение или постер фильма, название, краткое описание, дополнительную информацию и кнопку действия, например **«Смотреть»** или **«Подробнее»**. Hero должен быть главным визуальным элементом страницы.

После Hero разместить панель **«Категории»** с элементами вроде **«Фильмы»**, **«Сериалы»**, **«Новинки»**, **«Популярное»**, **«Комедии»**, **«Боевики»** и другими категориями. Элементы категории должны располагаться в одну строку на Desktop и перестраиваться при уменьшении ширины экрана.

Ниже реализовать основной каталог **постеров фильмов и сериалов**. Каждая карточка должна содержать:

1. **Постер** фильма или сериала;
2. **Название**;
3. Дополнительную информацию, например год выпуска или жанр;
4. При необходимости — рейтинг или небольшой информационный бейдж.

Постеры должны располагаться в гибкой **CSS Grid-сетке**. На Desktop карточки должны формировать несколько колонок, а на Mobile переходить в **1–2 колонки** в зависимости от доступной ширины.

Рядом с основным каталогом на Desktop разместить **боковой информационный блок** с дополнительными подборками или управляющими элементами. Например, в нём можно разместить блок **«Сейчас популярно»**, список рекомендуемых фильмов или дополнительные фильтры.

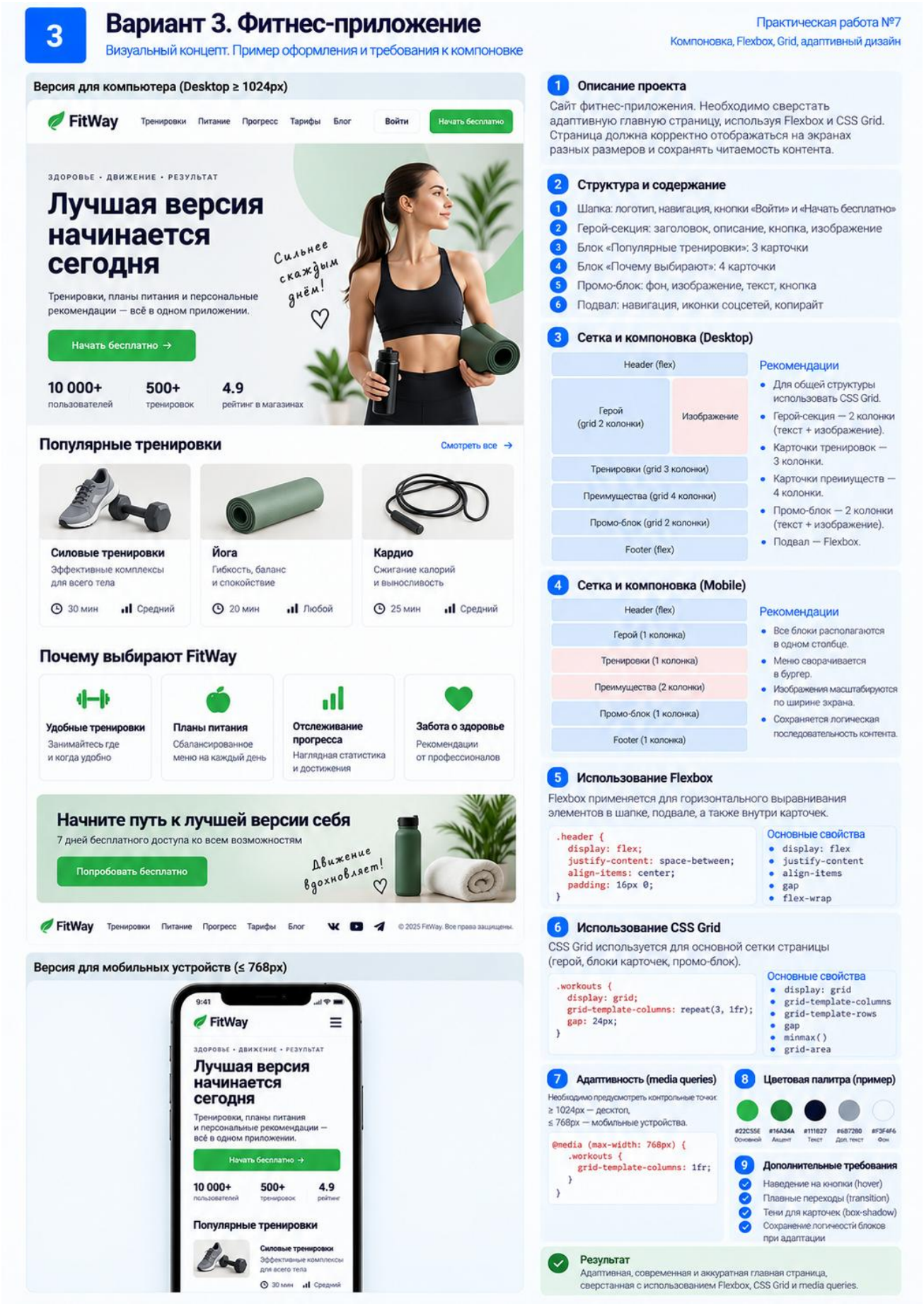
На Mobile боковой блок должен **перемещаться ниже основного каталога**, чтобы основное содержимое оставалось главным и сохранялась последовательность **Hero → категории → каталог → дополнительная информация**.

Требования к компоновке:

- на Desktop основная область страницы должна сочетать Hero-блок, каталог постеров и боковой блок;
- постеры разместить с помощью **CSS Grid**;
- панель категорий, поиск и управляющие элементы выровнять с помощью **Flexbox**;
- на Mobile все основные блоки должны переходить в последовательный вертикальный поток;
- сетка постеров должна перестраиваться из нескольких колонок в **1–2 колонки**;
- боковой блок на Mobile должен располагаться **после основного каталога**;
- ширина колонок каталога должна быть гибкой;
- изображения постеров не должны выходить за границы контейнера и должны сохранять пропорции;
- предусмотреть ручное переключение **Desktop / Mobile** в Header или Nav;
- при изменении ширины окна страница должна автоматически перестраиваться с использованием **media queries**;
- в качестве контрольной границы адаптации использовать **768 px**, предусмотренную общим заданием практической работы.
- при сужении экрана необходимо сохранить визуальную иерархию **Hero → категории → каталог**.

При реализации нельзя использовать весь макет как одно фоновое изображение. Все элементы страницы должны быть созданы средствами **HTML и CSS**, а изображения постеров, Hero-фильма и другие графические материалы подключены отдельно.

Вариант 3. Фитнес-приложение



Разработать адаптивную веб-страницу **персонального фитнес-приложения**, предназначенного для отслеживания физической активности, тренировок и прогресса пользователя. Страница должна воспроизводить структуру и визуальную логику представленного макета, отдельно реализуя Desktop- и Mobile-представления. В исходном задании для Варианта 3 предусмотрена композиция **профиль + статистика + текущая тренировка + расписание + рекомендации**.

В верхней части страницы разместить **Header** с логотипом фитнес-приложения, навигацией по основным разделам и элементами управления профилем пользователя. В Header также предусмотреть ручное переключение представления **Desktop / Mobile**.

Основную часть страницы представить в виде **Dashboard** пользователя.

В верхней области Dashboard разместить **блок профиля** с фотографией или аватаром пользователя, его именем, краткой информацией и основным показателем текущего прогресса.

Рядом или ниже разместить секцию **«Статистика»**, содержащую несколько информационных карточек:

1. **Тренировки** — количество выполненных тренировок;
2. **Калории** — количество сожжённых калорий;
3. **Активность** — показатель физической активности;
4. **Прогресс** — текущий результат или процент достижения цели.

Показатели внутри карточек должны быть выровнены с помощью **Flexbox**, а сами карточки должны быть частью общей Grid-композиции Dashboard.

Следующим блоком разместить секцию **«Текущая тренировка»**. В ней указать название тренировки, её продолжительность, основные упражнения или этапы, а также кнопку действия, например **«Начать тренировку»** или **«Продолжить»**. Информацию о тренировке расположить таким образом, чтобы название, показатели и кнопка оставались визуально связанными.

Ниже разместить блок **«Расписание»**, содержащий запланированные тренировки на ближайшие дни. Для каждого элемента расписания указать день, время и название тренировки. На Desktop расписание может занимать отдельную область Dashboard, а на Mobile должно переходить в последовательный вертикальный блок.

После расписания разместить секцию **«Рекомендации»** с несколькими карточками или информационными блоками, содержащими рекомендации по тренировкам и активности пользователя.

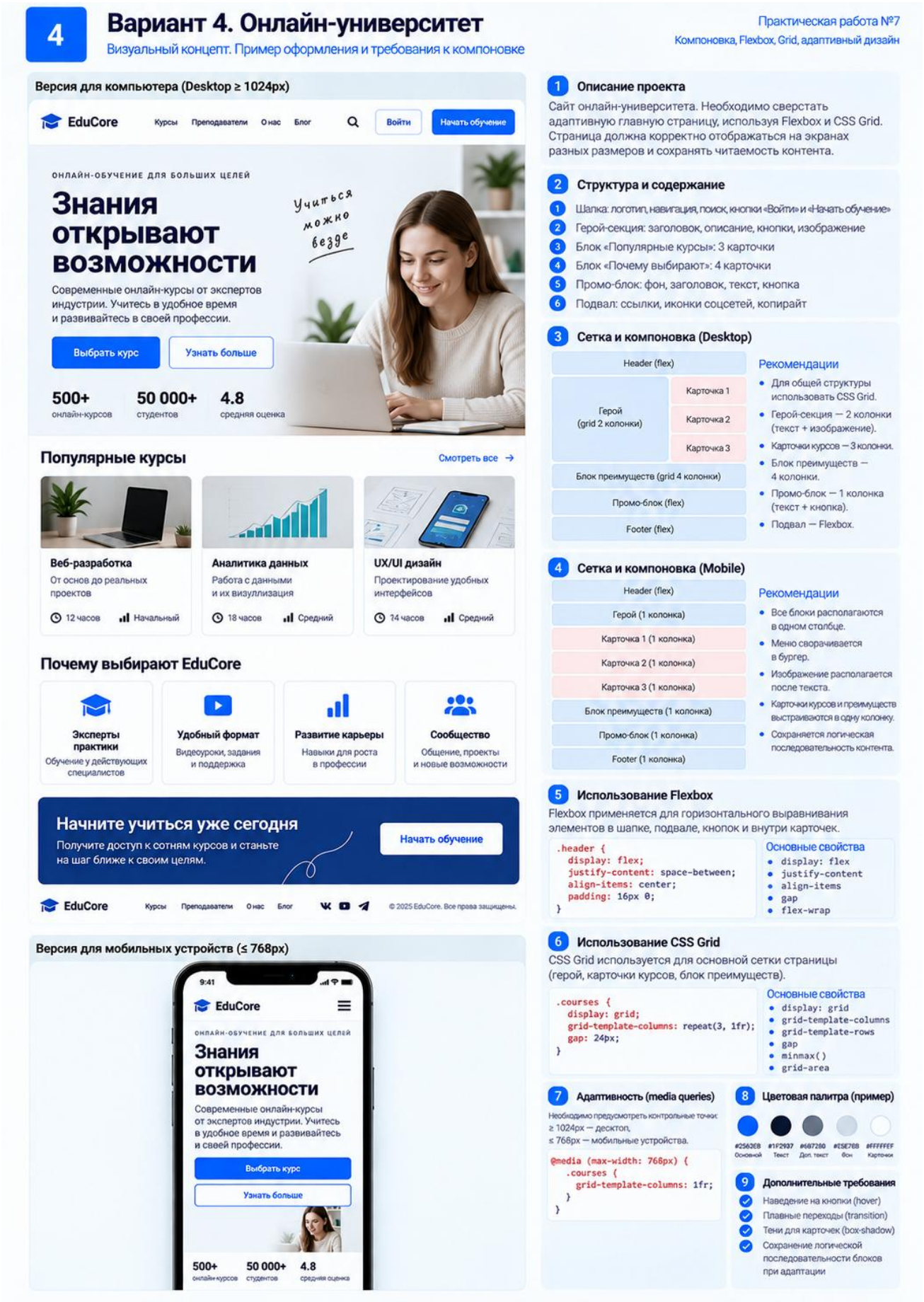
Требования к компоновке:

- на Desktop Dashboard должен строиться с использованием **CSS Grid Areas**;

- профиль, статистика, текущая тренировка, расписание и рекомендации должны формировать единую Dashboard-композицию;
- показатели внутри карточек статистики выровнять с помощью **Flexbox**;
- на Mobile Dashboard должен переходить в **последовательный вертикальный поток**;
- блоки статистики и тренировок на Mobile должны становиться отдельными последовательными секциями;
- карточки показателей не должны сжиматься до нечитаемого состояния;
- элементы управления и группы информации должны сохранять аккуратное выравнивание после перестройки;
- предусмотреть ручное переключение **Desktop / Mobile** в Header или Nav;
- при изменении ширины окна страница должна автоматически перестраиваться с использованием **media queries**;
- в качестве контрольной границы адаптации использовать **768 px**;
- расписание и рекомендации должны сохранять читаемую иерархию на мобильной ширине.

При реализации нельзя использовать весь макет как одно фоновое изображение. Все элементы страницы должны быть созданы средствами **HTML и CSS**, а фотографии, аватары и другие изображения подключены отдельно. Технология компоновки должна соответствовать требованиям варианта: **CSS Grid для основной Dashboard-композиции и Flexbox для самостоятельных блоков интерфейса**.

Вариант 4. Онлайн-университет



Разработать адаптивную веб-страницу **главной страницы онлайн-университета**, предназначенную для отображения каталога учебных курсов, прогресса обучения, преподавателей и расписания. Страница должна иметь отдельные Desktop- и Mobile-представления и сохранять образовательную иерархию при изменении ширины экрана. В исходном задании для Варианта 4 предусмотрена композиция **Header + навигация + каталог курсов + прогресс + преподаватели + расписание**.

В верхней части страницы разместить **Header** с логотипом онлайн-университета, основной навигацией и элементами управления. В Header или Nav также предусмотреть ручное переключение представления **Desktop / Mobile**.

Основную область страницы разделить на две части: **Sidebar** и **основной контент**.

В **Sidebar** разместить навигацию по образовательным разделам, например:

- «Главная»;
- «Мои курсы»;
- «Все курсы»;
- «Прогресс обучения»;
- «Расписание»;
- «Преподаватели».

На Desktop Sidebar должен располагаться сбоку от основного содержимого и участвовать в общей **Grid-композиции** страницы.

В основной части разместить заголовок страницы, например **«Обучение»** или **«Мои курсы»**, после чего создать секцию **«Каталог курсов»**.

Каталог представить в виде набора карточек курсов. Каждая карточка может содержать:

- название курса;
- краткое описание;
- имя преподавателя;
- продолжительность или количество занятий;
- текущий прогресс прохождения;
- кнопку действия, например **«Продолжить обучение»** или **«Открыть курс»**.

Элементы внутри карточек организовать с помощью **Flexbox**, чтобы название, информация о курсе, прогресс и кнопка сохраняли аккуратное выравнивание. Сетка самого каталога должна строиться с помощью **CSS Grid**. На Desktop предусмотреть, например, **3 колонки**, а при уменьшении ширины экрана перейти к меньшему числу колонок и в итоге к **1 колонке**.

Отдельно разместить блок **«Прогресс обучения»**. В нём показать несколько показателей текущего обучения — например, количество завершённых курсов, процент прохождения или прогресс отдельных курсов. Конкретное содержание показателей можно подобрать самостоятельно, сохраняя назначение блока.

Ниже или рядом разместить секцию **«Преподаватели»** с несколькими карточками преподавателей. В карточке можно предусмотреть изображение, имя, дисциплину и краткую информацию.

После этого разместить блок **«Расписание»**, содержащий ближайшие учебные занятия. Для каждого элемента расписания указать дату, время и название курса или занятия. На мобильной версии информация должна оставаться читаемой и визуально структурированной.

Требования к компоновке

- на Desktop **Sidebar и основной контент должны формироваться средствами CSS Grid**;
- карточки курсов и другие элементы внутри блоков организовать с помощью **Flexbox**;
- каталог курсов должен использовать гибкую Grid-сетку;
- на Desktop карточки могут располагаться в **3 колонки**;
- при сужении экрана сетка должна перестраиваться, например, **3 → 2 → 1 колонка**;
- ширина колонок должна быть гибкой и не приводить к горизонтальному переполнению;
- на Mobile **Sidebar должен превращаться в компактный верхний блок**;
- после Sidebar основной контент должен занимать всю доступную ширину;
- карточки курсов на Mobile должны переходить в **одну колонку**;
- порядок блоков при перестройке должен оставаться логичным и не нарушать **образовательную иерархию**;
- предусмотреть ручное переключение **Desktop / Mobile** в Header или Nav;
- при изменении ширины окна страница должна автоматически перестраиваться;
- для автоматической адаптации использовать **media queries**, с контрольной границей **768 px**;
- карточки и их сетка должны сохранять гибкую ширину при изменении размера экрана.

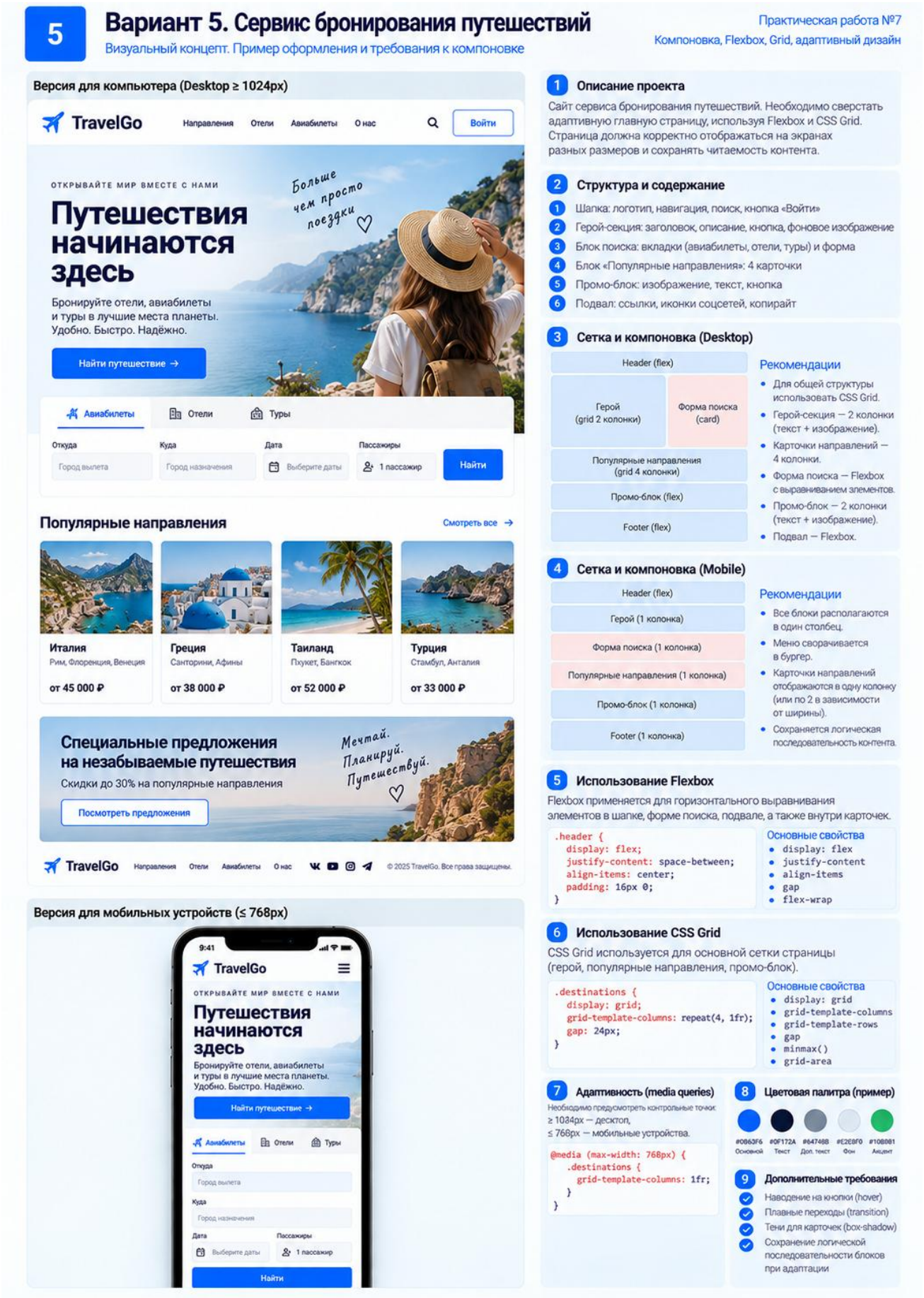
При реализации нельзя использовать весь макет как единое фоновое изображение. Все блоки страницы необходимо создать средствами **HTML и CSS**, а изображения преподавателей и другие графические материалы подключать как отдельные ресурсы.

Таким образом, итоговая структура страницы должна отражать последовательность:

Header → навигация → Sidebar → каталог курсов → прогресс → преподаватели → расписание,

при этом на Desktop Sidebar располагается сбоку, а на Mobile перемещается наверх, после чего каталог курсов и остальные блоки выстраиваются в последовательный поток. Технология компоновки остаётся общей для варианта: **CSS Grid для основной структуры и Flexbox для самостоятельных блоков и элементов карточек**.

Вариант 5. Сервис бронирования путешествий



Разработать адаптивную веб-страницу **сервиса планирования и бронирования путешествий**, предназначенного для поиска направлений и выбора туристических предложений. Страница должна иметь Desktop- и Mobile-представления и сохранять визуальную иерархию основных блоков при изменении ширины экрана. В исходном задании для Варианта 5 предусмотрена композиция **Hero + форма поиска + направления + туристические пакеты + карта + отзывы**.

В верхней части страницы разместить **Header** с логотипом сервиса, основной навигацией и элементами управления. В Header или Nav также предусмотреть ручное переключение представления **Desktop / Mobile**.

После Header разместить **Hero-блок**, который должен выступать основным визуальным элементом страницы. В нём разместить крупный заголовок, краткое описание сервиса и призыв воспользоваться поиском путешествия.

Ниже Hero разместить **форму поиска путешествия**. Форма может содержать следующие элементы:

- поле **«Откуда»**;
- поле **«Куда»**;
- выбор даты поездки;
- выбор количества путешественников;
- кнопку **«Найти»**.

Группу полей и кнопку поиска организовать с помощью **Flexbox**. На Desktop элементы формы могут располагаться в одной горизонтальной строке, а на Mobile должны перестраиваться в вертикальную последовательность.

Следующим блоком разместить секцию **«Популярные направления»**. Представить несколько направлений в виде карточек. Каждая карточка может содержать изображение направления, название места, краткую информацию и дополнительные сведения о поездке.

Карточки направлений разместить с помощью **CSS Grid**. На Desktop они должны формировать гибкую сетку из нескольких колонок, а на Mobile переходить в **одноколоночный список**.

После направлений разместить секцию **«Туристические пакеты»**. В ней представить несколько готовых предложений для путешествий. Каждый пакет может содержать название, направление, продолжительность, краткое описание и кнопку действия, например **«Подробнее»** или **«Забронировать»**.

Далее разместить блок **«Карта»**, предназначенный для визуального представления направлений путешествий. На странице можно использовать стилизованную область карты с отмеченными точками или отдельный графический элемент, соответствующий макету.

После карты разместить секцию **«Отзывы»** с несколькими отзывами путешественников. Каждый отзыв оформить отдельным блоком с именем пользователя, текстом отзыва и, при необходимости, дополнительной информацией.

Требования к компоновке

- на Desktop карточки направлений должны строиться с помощью **CSS Grid**;
- форму поиска и группы фильтров организовать с помощью **Flexbox**;
- Flexbox применить минимум в одном самостоятельном блоке интерфейса;
- CSS Grid применить минимум в одной основной сетке или композиции;
- на Desktop карточки направлений должны располагаться в несколько колонок;
- на Mobile карточки направлений должны переходить в **одну колонку**;
- форма поиска на Mobile должна становиться **вертикальной**;
- все поля формы должны оставаться доступными и не выходить за пределы ширины экрана;
- предусмотреть ручное переключение **Desktop / Mobile** в Header или Nav;
- при изменении ширины окна страница должна автоматически перестраиваться с использованием **media queries**;
- в качестве контрольной границы адаптации использовать **768 px**;
- после перестройки должна сохраняться визуальная иерархия **Header → Nav → основной контент → дополнительные блоки**;
- изображения направлений и другие графические элементы не должны искажаться при изменении размеров карточек.

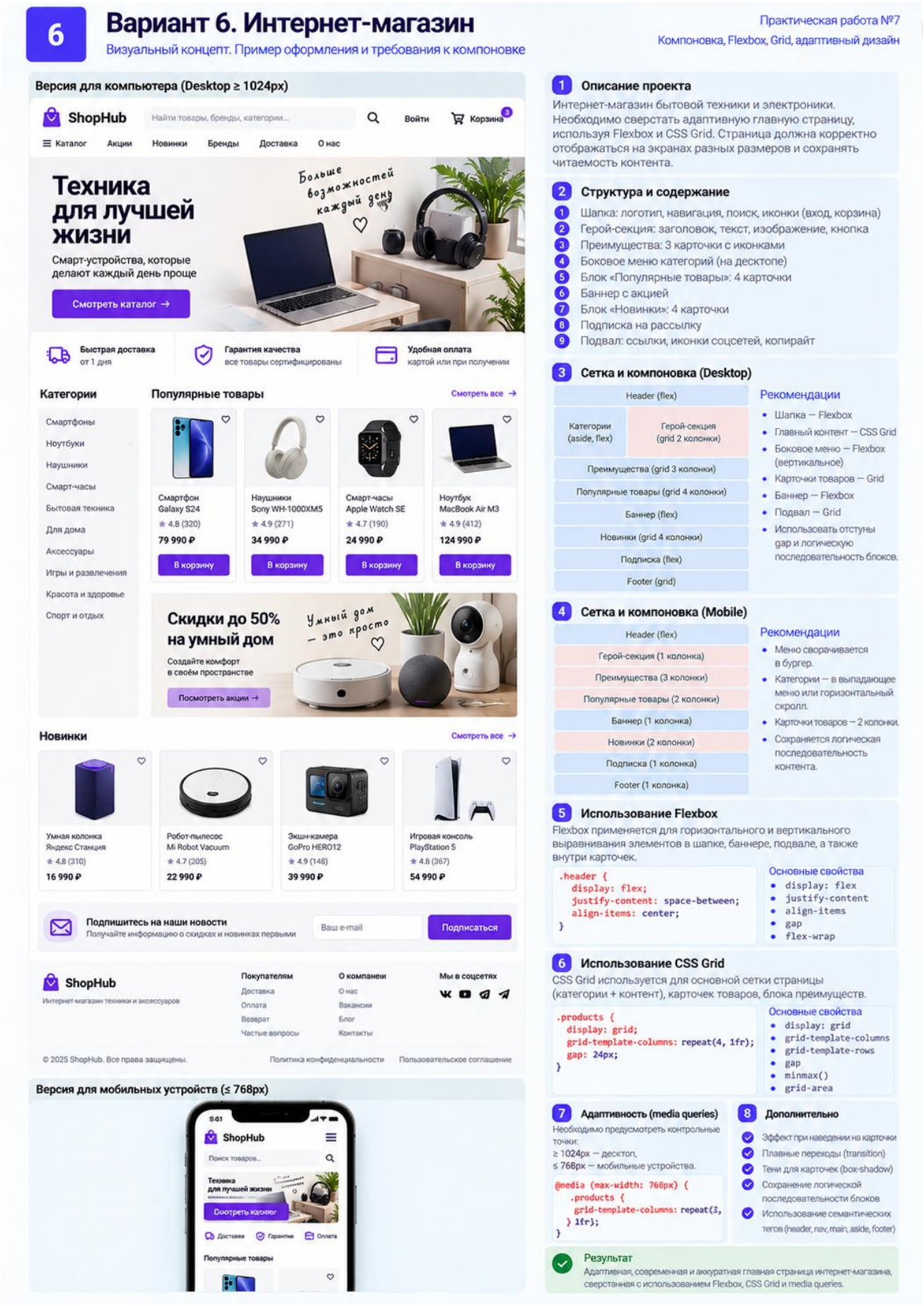
При реализации нельзя использовать весь макет как единое фоновое изображение. Все элементы необходимо создать средствами **HTML и CSS**, а изображения направлений, карты и другие графические материалы подключать отдельно.

Итоговая структура страницы должна отражать последовательность:

Header → Hero → форма поиска → направления → туристические пакеты → карта → отзывы.

При этом на Desktop основной акцент делается на **Grid-сетке карточек и горизонтальной форме поиска**, а на Mobile форма поиска перестраивается вертикально, карточки направлений переходят в одну колонку, а вся страница сохраняет последовательную и понятную структуру. Технология компоновки остаётся общей для варианта: **CSS Grid для основной сетки и Flexbox для формы поиска и самостоятельных групп элементов**.

Вариант 6. Интернет-магазин



Разработать адаптивную веб-страницу **каталога интернет-магазина**, предназначенную для просмотра товаров, поиска нужного товара, применения фильтров и добавления товаров в корзину. Страница должна иметь Desktop- и Mobile-представления и сохранять понятную визуальную иерархию при изменении ширины экрана.

Основная структура страницы согласно заданию:

- **Header → поиск → фильтры + каталог товаров → карточки товаров → корзина**.
- В верхней части страницы разместить **Header**. В нём предусмотреть название или логотип интернет-магазина, навигацию и элемент управления корзиной. В Header или Nav также необходимо разместить ручное переключение представления **Desktop / Mobile**.
- Ниже Header разместить **поисковую область**. Она должна визуально выделяться как основной инструмент работы с каталогом. В качестве конкретного содержимого можно использовать строку поиска с кнопкой поиска, например «Найти». Это является примером реализации, поскольку методичка конкретные поля поиска не задаёт.
- Основную часть страницы организовать как блок из двух зон:
- **панель фильтров**;
- **каталог товаров**.

На Desktop эти две зоны должны формировать **двухзонную CSS Grid-композицию**. Панель фильтров располагается сбоку, а каталог занимает основную часть доступной ширины.

Панель фильтров

В панели фильтров разместить элементы, позволяющие ограничивать отображаемые товары. Например, можно предусмотреть категории, диапазон цены, производителя или другие характеристики товаров.

Важно, чтобы фильтры были отдельной самостоятельной областью интерфейса и не смешивались визуально с карточками каталога.

На Mobile панель фильтров должна **перемещаться над каталогом**, чтобы пользователь сначала мог настроить параметры отображения, а затем перейти к товарам.

Каталог товаров

В основной области разместить **сетку карточек товаров**.

Каждая карточка может содержать:

- изображение товара;
- название;
- краткую информацию;
- цену;
- кнопку действия, например «В корзину».

Это конкретизация содержимого карточки; обязательным по заданию является наличие **карточек товаров**, а не перечисленный набор отдельных полей.

Карточки необходимо расположить с помощью **CSS Grid**. На Desktop каталог должен поддерживать гибкую сетку примерно из **3–4 колонок**, а при переходе на Mobile — перестраиваться в **одну колонку**.

Внутреннее содержимое каждой карточки товара организовать с помощью **Flexbox**. Например, название и цену можно расположить в одной группе, а кнопку действия — выровнять относительно нижней части карточки. Главное требование — Flexbox должен использоваться минимум в одном самостоятельном блоке интерфейса и непосредственно в элементах карточки товара согласно варианту.

Корзина

В интерфейсе необходимо предусмотреть **корзину** как отдельный функциональный элемент. Например, в Header можно разместить иконку или кнопку корзины с количеством выбранных товаров.

При изменении ширины экрана корзина должна оставаться доступной и не нарушать структуру Header.

Требования к компоновке

- на Desktop панель фильтров и каталог должны образовывать **двухзонную CSS Grid-композицию**;
- каталог товаров реализовать с помощью **CSS Grid**;
- элементы карточки товара организовать с помощью **Flexbox**;
- Flexbox применить минимум в одном самостоятельном блоке интерфейса;
- CSS Grid применить минимум в одной основной сетке или композиции;
- на Desktop сетка каталога должна гибко формировать примерно **3–4 колонки**;
- на Mobile каталог должен переходить в **одну колонку**;
- на Mobile панель фильтров должна перемещаться **над каталогом**;
- предусмотреть ручное переключение **Desktop / Mobile** в Header или Nav;
- реализовать автоматическую адаптацию при изменении ширины окна;
- при перестройке должна сохраняться визуальная иерархия **Header → Nav → основной контент → дополнительные блоки**;
- карточки товаров и сетка должны иметь гибкую ширину;
- при уменьшении ширины экрана элементы не должны выходить за границы страницы.
- В качестве контрольной границы для автоматического переключения можно использовать **768 px**, что соответствует общей спецификации практической работы.

При реализации нельзя использовать весь макет как единое фоновое изображение. Структуру страницы необходимо создать средствами **HTML и CSS**, а изображения товаров подключать отдельно.

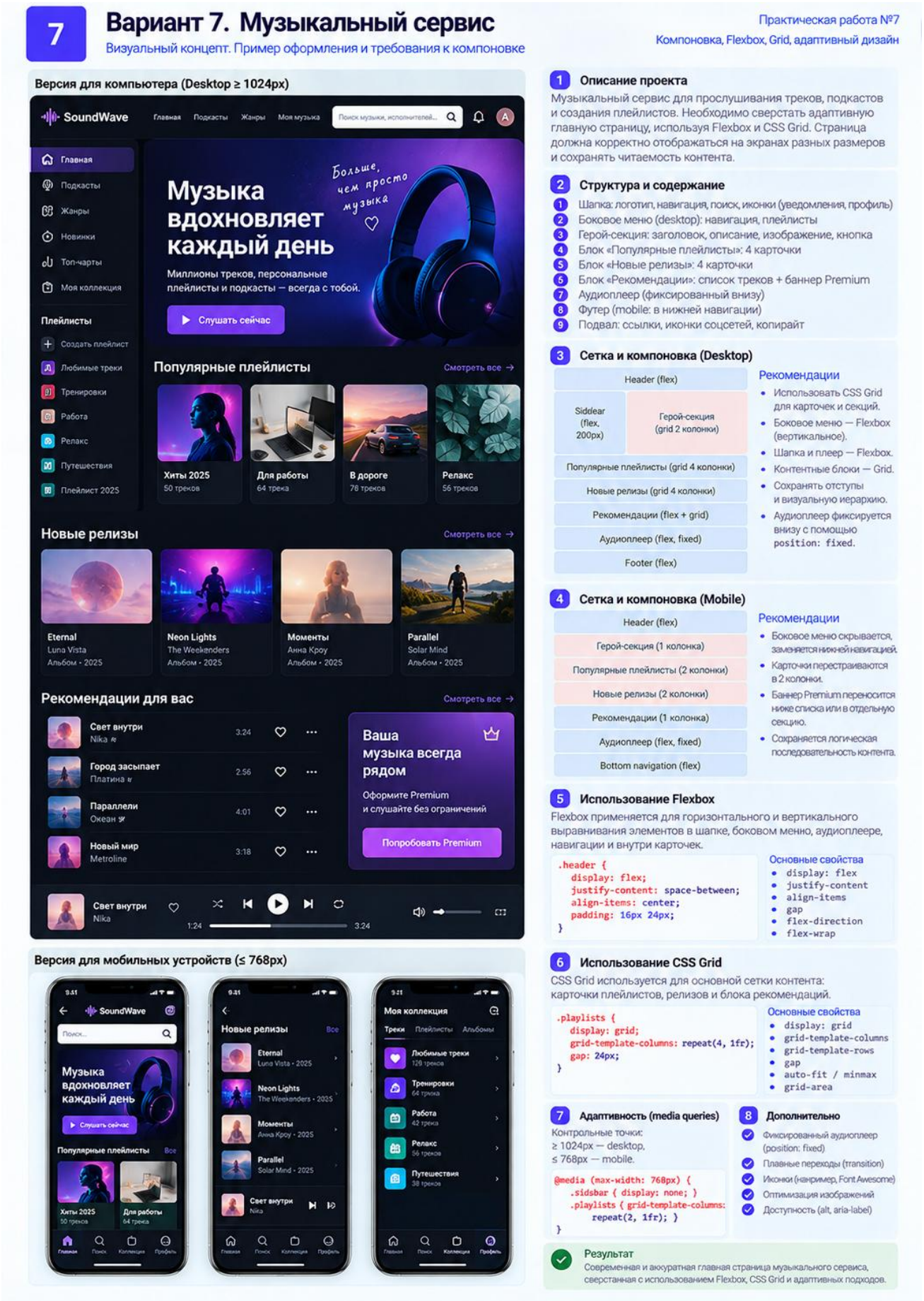
Итоговая схема страницы:

Header → поиск → двухзонная область «фильтры + каталог» → карточки товаров → корзина

На **Desktop** основной акцент сделать на двухзонной Grid-композиции: фильтры слева, каталог справа. Каталог должен формировать гибкую сетку из 3–4 колонок, а содержимое карточек организовываться с помощью Flexbox.

На **Mobile** фильтры перемещаются над каталогом, а карточки товаров выстраиваются **в одну колонку**. При этом основные элементы интернет-магазина должны оставаться доступными, а визуальная иерархия страницы — сохраняться.

Вариант 7. Музыкальный сервис



Разработать адаптивную веб-страницу **музыкального сервиса**, предназначенную для просмотра музыкальных подборок, альбомов и плейлистов с возможностью управления воспроизведением музыки. Страница должна иметь Desktop- и Mobile-представления и сохранять удобную структуру интерфейса при изменении ширины экрана.

Основная структура страницы согласно заданию:

Sidebar → Header → подборки альбомов → плейлисты → нижний музыкальный плеер.

Header

В верхней части основного содержимого разместить **Header**. В нём можно предусмотреть название или логотип музыкального сервиса, элементы навигации и дополнительные элементы управления.

В Header или Nav обязательно добавить ручное переключение представления **Desktop / Mobile**.

Header должен оставаться отдельной верхней областью страницы и не смешиваться с каталогом музыкального контента.

Sidebar

Слева разместить **Sidebar**, содержащий основную навигацию музыкального сервиса.

В качестве содержимого можно использовать, например:

- «Главная»;
- «Моя музыка»;
- «Плейлисты»;
- «Альбомы»;
- «Исполнители».

Это примеры наполнения — в методичке конкретные названия пунктов не заданы. Обязательным является сам блок **Sidebar** и его роль в основной композиции страницы.

На Desktop Sidebar должен занимать отдельную навигационную зону рядом с основным контентом. Основной экран при этом необходимо построить с помощью **CSS Grid**.

На Mobile Sidebar должен превращаться в **компактную навигацию**, чтобы не занимать значительную часть вертикального пространства страницы.

Подборки альбомов

В основной области разместить секцию с **подборками альбомов**.

Альбомы представить в виде карточек. Карточка может содержать:

- обложку альбома;
- название;
- имя исполнителя;
- дополнительную информацию.

Конкретный состав текста внутри карточки можно определить самостоятельно, поскольку методичка требует именно наличие **карточек альбомов**, но не задаёт их точное содержимое.

Карточки расположить в гибкой сетке. На Desktop можно использовать несколько колонок, а на Mobile карточки должны перестраиваться в **одну или две колонки**.

При уменьшении ширины экрана карточки не должны становиться слишком узкими или выходить за пределы страницы.

Плейлисты

После подборок разместить отдельную секцию **«Плейлисты»**.

В ней представить несколько пользовательских или тематических плейлистов. Каждый элемент может содержать название плейлиста, небольшое изображение или обложку и дополнительную информацию.

Плейлисты должны оставаться визуально отделёнными от подборок альбомов, чтобы пользователь мог легко различать эти два типа музыкального контента.

Нижний музыкальный плеер

В нижней части страницы разместить **музыкальный плеер**.

Это один из ключевых элементов варианта. Плеер может содержать:

- обложку текущего трека;
- название композиции;
- исполнителя;
- кнопки управления воспроизведением;
- индикатор прогресса;
- дополнительные элементы управления.

Точный набор элементов в методичке не указан, поэтому перечисленные элементы являются примером конкретизации интерфейса.

Внутреннюю структуру плеера организовать с помощью **Flexbox**. Например, информацию о текущем треке можно расположить с одной стороны, а управляющие элементы — в отдельной группе.

Особое внимание уделить тому, чтобы нижняя панель плеера сохраняла **читаемую структуру на малой ширине экрана**.

Требования к компоновке

- основной Desktop-экран построить с помощью **CSS Grid**;
- Grid должен разделять навигационную зону Sidebar и основную область контента;
- элементы нижнего музыкального плеера организовать с помощью **Flexbox**;
- Flexbox применить минимум в одном самостоятельном блоке интерфейса;
- CSS Grid применить минимум в одной основной сетке или композиции;

- на Desktop Sidebar должен быть отдельной навигационной зоной;
- на Mobile Sidebar должен превращаться в **компактную навигацию**;
- карточки альбомов на Mobile должны перестраиваться в **одну или две колонки**;
- ширина колонок должна быть гибкой;
- предусмотреть ручное переключение **Desktop / Mobile** в Header или Nav;
- реализовать автоматическую адаптацию при изменении ширины окна;
- нижний музыкальный плеер должен сохранять читаемую структуру на малой ширине;
- после перестройки основные элементы страницы должны оставаться доступными и не выходить за границы экрана.
- Для автоматической адаптации можно использовать контрольную границу **768 px**, предусмотренную общей спецификацией практической работы. При изменении ширины окна интерфейс должен автоматически переходить между соответствующими представлениями.

При реализации нельзя использовать весь макет как единое фоновое изображение. Все структурные элементы необходимо создать средствами **HTML и CSS**, а обложки альбомов и другие изображения подключать отдельно.

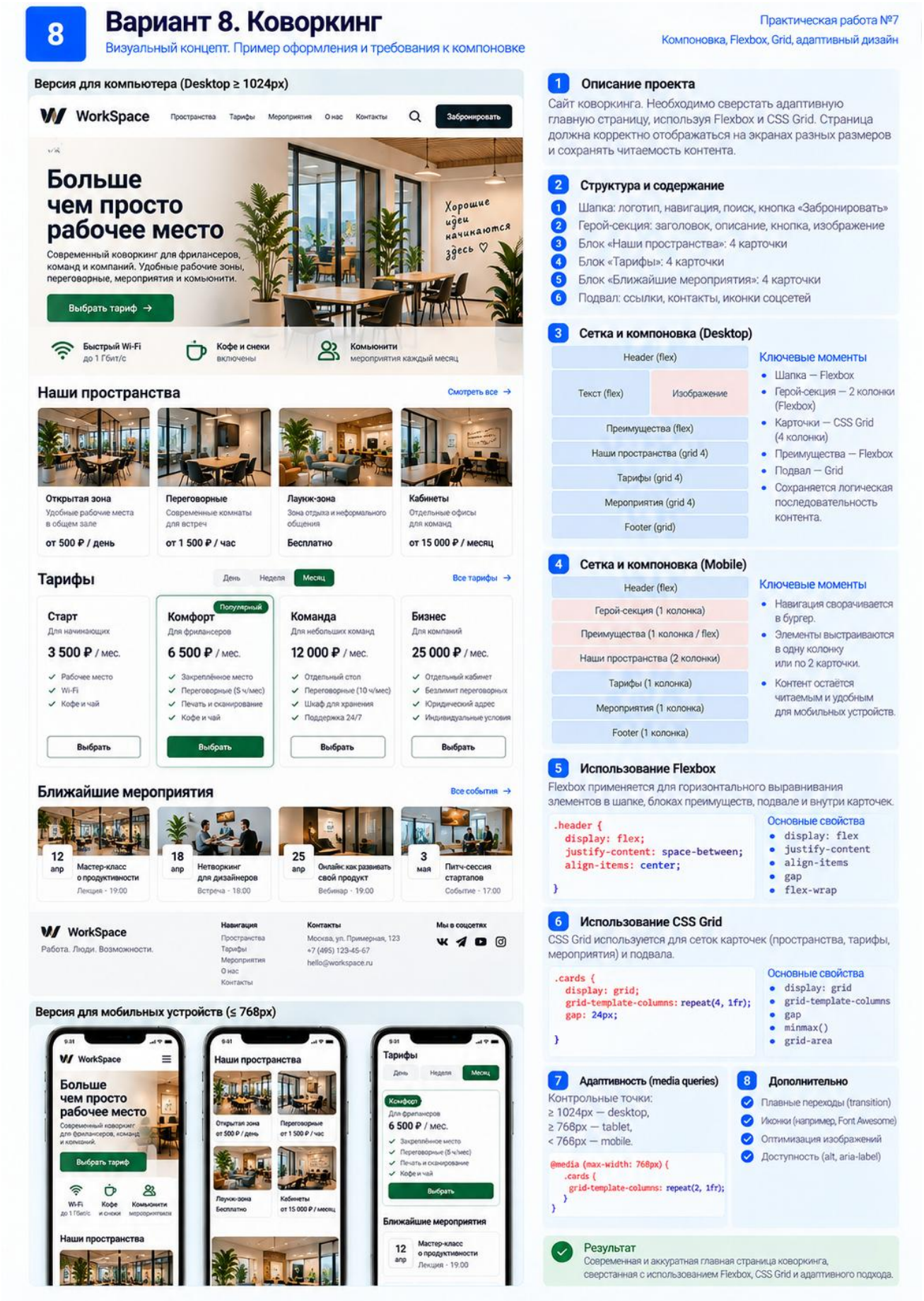
Итоговая структура страницы:

Sidebar → Header → подборки альбомов → плейлисты → нижний музыкальный плеер

На **Desktop** основная композиция должна строиться как Grid с отдельной навигационной зоной и областью контента. Карточки альбомов располагаются в гибкой сетке, а внутренние элементы музыкального плеера организуются с помощью Flexbox.

На **Mobile** Sidebar превращается в компактную навигацию, карточки альбомов перестраиваются в **одну или две колонки**, а нижний плеер должен сохранять удобную и читаемую структуру.

Вариант 8. Коворкинг



Разработать адаптивную веб-страницу **современного коворкинга**, предназначенную для просмотра тарифов, рабочих зон, переговорных помещений и расписания с возможностью получения контактной информации. Страница должна иметь Desktop- и Mobile-представления и сохранять удобную структуру интерфейса при изменении ширины экрана.

Основная структура страницы согласно заданию:

Hero → тарифы → рабочие зоны → переговорные → расписание → контакты.

Header

В верхней части страницы разместить **Header**. В нём можно предусмотреть название или логотип коворкинга, элементы навигации и дополнительные элементы управления.

В Header или Nav обязательно добавить ручное переключение представления **Desktop / Mobile**.

Header должен оставаться отдельной верхней областью страницы и не смешиваться с основным содержимым коворкинга.

Hero

В верхней части основного содержимого разместить **Hero-блок**, представляющий коворкинг.

В нём можно предусмотреть:

- название коворкинга;
- краткое описание пространства;
- основное изображение;
- дополнительную информацию;
- кнопку или другое основное действие.

Конкретное содержимое Hero в методичке не задано, поэтому его наполнение можно определить самостоятельно. Обязательным является наличие самого блока **Hero** как основной вводной части страницы.

Тарифы

После Hero разместить отдельную секцию **«Тарифы»**.

В ней представить несколько вариантов тарифов коворкинга. Каждый тариф можно оформить отдельной карточкой.

Карточка тарифа может содержать:

- название тарифа;
- стоимость;
- краткое описание;
- список доступных возможностей;
- кнопку выбора или бронирования.

Конкретные названия, цены и состав тарифов в методичке не заданы, поэтому их можно определить самостоятельно.

На Desktop карточки тарифов должны располагаться с использованием **CSS Grid**.

При уменьшении ширины экрана тарифы должны переходить из многоколоночного представления в вертикальное расположение.

Рабочие зоны

После тарифов разместить секцию **«Рабочие зоны»**.

В ней представить несколько вариантов рабочих пространств коворкинга. Каждую рабочую зону можно оформить отдельной карточкой.

Карточка может содержать:

- изображение рабочей зоны;
- название;
- краткое описание;
- дополнительную информацию.

Конкретное наполнение карточек методичка не задаёт, поэтому оно определяется самостоятельно.

Рабочие зоны на Desktop необходимо расположить с помощью **CSS Grid**. Сетка должна быть гибкой, а карточки должны иметь согласованные отступы и визуальную структуру.

На Mobile рабочие зоны должны становиться **вертикальными**, чтобы карточки последовательно располагались по ширине экрана.

Переговорные

После рабочих зон разместить отдельную секцию **«Переговорные»**.

В ней представить несколько переговорных помещений. Каждый элемент может содержать:

- название переговорной;
- изображение;
- вместимость;
- краткое описание;
- дополнительную информацию.

Точный состав содержимого в методичке не указан, поэтому его можно определить самостоятельно.

Переговорные должны оставаться визуально отделёнными от рабочих зон, чтобы пользователь мог различать разные типы пространств коворкинга.

Расписание

После переговорных разместить блок **«Расписание»**.

В нём представить информацию о времени, доступности или запланированных активностях коворкинга.

Элементы расписания могут содержать:

- время;
- название активности;
- дополнительную информацию.

На Desktop расписание может быть представлено в более подробном виде.

На Mobile расписание должно получать **компактное представление**, чтобы информация сохраняла читаемую структуру и не выходила за границы экрана.

Особое внимание необходимо уделить тому, чтобы расписание сохраняло понятную иерархию информации после перестройки страницы.

Контакты

В нижней части страницы разместить секцию **«Контакты»**.

В ней можно предусмотреть:

- адрес коворкинга;
- контактные данные;
- время работы;
- дополнительные ссылки или информацию.

Конкретное содержание контактного блока в методичке не задано, поэтому его можно определить самостоятельно.

Требования к компоновке

- основные сетки тарифов и рабочих зон построить с помощью **CSS Grid**;
- CSS Grid применить минимум в одной основной сетке или композиции;
- использовать **Flexbox** минимум в одном самостоятельном блоке интерфейса;
- Flexbox можно применить для строк информации, кнопок, элементов карточек или других самостоятельных блоков;
- на Desktop тарифы должны располагаться в сетке;
- на Desktop рабочие зоны должны располагаться в сетке;
- на Mobile рабочие зоны и тарифы должны становиться **вертикальными**;
- расписание на Mobile должно получать **компактное представление**;
- разные типы карточек должны иметь согласованные отступы и визуальную сетку;
- предусмотреть ручное переключение **Desktop / Mobile** в Header или Nav;
- реализовать автоматическую адаптацию при изменении ширины окна;
- расписание, тарифы и другие информационные блоки должны сохранять читаемую иерархию на мобильной ширине;
- после перестройки основные элементы страницы должны оставаться доступными и не выходить за границы экрана.

- Для автоматической адаптации можно использовать контрольную границу **768 px**, предусмотренную общей спецификацией практической работы. При изменении ширины окна интерфейс должен автоматически переходить между соответствующими представлениями.

При реализации нельзя использовать весь макет как единое фоновое изображение. Все структурные элементы необходимо создать средствами **HTML и CSS**, а изображения рабочих зон, переговорных и другие изображения подключать отдельно.

Итоговая структура страницы:

Hero → тарифы → рабочие зоны → переговорные → расписание → контакты

На **Desktop** тарифы и рабочие зоны должны располагаться с помощью **CSS Grid**, а отдельные строки информации и элементы интерфейса — с помощью **Flexbox**.

На **Mobile** тарифы и рабочие зоны становятся вертикальными, расписание получает компактное представление, а все типы карточек должны сохранять согласованные отступы и визуальную сетку.

Вариант 9. Ресторан

Разработать адаптивную веб-страницу **электронной библиотеки**, предназначенную для поиска и просмотра книг, работы с категориями, популярными изданиями, новинками, авторами и личной библиотекой пользователя. Страница должна иметь Desktop- и Mobile-представления и сохранять удобную структуру интерфейса при изменении ширины экрана.

Основная структура страницы согласно заданию: **поиск → категории → популярные книги → новинки → авторы → личная библиотека**.

Header

В верхней части страницы разместить название или логотип электронной библиотеки, навигацию и элементы управления. В Header или Nav необходимо разместить ручное переключение между режимами **Desktop / Mobile**.

В Header можно расположить:

- название или логотип библиотеки;
- ссылки на основные разделы;
- кнопку или поле перехода к поиску;
- переключатель Desktop/Mobile;
- дополнительные действия пользователя.

Шапка должна оставаться отдельным визуальным блоком и не смешиваться с каталогом книг.

Поиск

Основной блок поиска предназначен для поиска книг в электронной библиотеке.

В блоке можно разместить:

- строку поиска;
- кнопку поиска;
- дополнительные фильтры;
- выбор категории или типа литературы.

Точное содержание полей поиска методический материал не задаёт, поэтому конкретные названия фильтров можно определить самостоятельно.

На Desktop строка поиска и связанные с ней элементы могут располагаться в одну строку с использованием **Flexbox**.

На Mobile блок поиска и фильтры должны перемещаться вверх страницы и перестраиваться так, чтобы элементы не выходили за ширину экрана.

Категории

В данном блоке разместить основные категории книг библиотеки.

Категории могут быть представлены:

- кнопками;

- ссылками;
- небольшими карточками;
- списком тематических разделов.

Конкретный набор категорий методический материал не определяет.

На Desktop категории могут располагаться горизонтально или в несколько колонок. На Mobile блок должен сохранять удобную навигацию и не создавать горизонтального переполнения.

Популярные книги

Основной каталог популярных книг представить в виде карточек.

Каждая карточка может содержать:

- обложку книги;
- название;
- автора;
- краткие сведения;
- дополнительную информацию;
- кнопку действия, например для открытия или добавления в личную библиотеку.

Точные данные книг в задании не указаны, поэтому их можно выбрать самостоятельно.

На Desktop каталог книг должен строиться с использованием **CSS Grid**. Сетка должна иметь гибкую ширину колонок и, например, переходить с 3 колонок на 1 при уменьшении ширины экрана.

Особое внимание необходимо уделить обложкам: они **не должны искажаться при изменении размеров карточек**.

Новинки

Отдельный блок предназначен для недавно добавленных книг.

Карточки новинок могут иметь тот же принцип построения, что и популярные книги:

- обложка;
- название;
- автор;
- краткая информация;
- действие.

При этом блок должен визуально отличаться от популярных книг за счёт расположения или оформления, сохраняя общую визуальную сетку страницы.

На Desktop элементы размещаются в Grid, а на Mobile книжные карточки переходят в одну колонку.

Авторы

В этом разделе разместить список или карточки авторов.

В качестве содержимого можно использовать:

- фотографию или аватар;
- имя автора;
- краткую информацию;
- количество книг;
- ссылку на произведения автора.

Конкретный состав информации об авторах в методическом материале не указан, поэтому его можно определить самостоятельно.

Блок должен сохранять аккуратную структуру при переходе на мобильную ширину.

Личная библиотека

Отдельный раздел предназначен для книг, сохранённых пользователем.

В нём можно представить:

- список сохранённых книг;
- обложки;
- названия и авторов;
- статус чтения;
- кнопки действий.

Конкретные данные и дополнительные функции личной библиотеки методический материал не определяет.

На Desktop элементы можно разместить в несколько колонок, а на Mobile — последовательно, чтобы информация оставалась читаемой.

Требования к компоновке

- **CSS Grid** обязательно применить минимум в одной основной сетке или композиции.
- Каталог книг на Desktop строить с использованием **CSS Grid**.
- **Flexbox** применить минимум в одном самостоятельном блоке интерфейса.
- Строку поиска, метаданные и элементы действий можно организовать с помощью **Flexbox**.
- Книжные карточки на Desktop должны располагаться в несколько колонок.
- На Mobile книжные карточки должны переходить в **одну колонку**.
- Ширина колонок должна быть гибкой.
- Обложки книг не должны искажаться при изменении размеров карточек.
- В Header или Nav добавить ручное переключение **Desktop / Mobile**.
- Реализовать автоматическую адаптацию страницы при изменении ширины окна.
- Визуальная иерархия поиска, категорий, каталога и дополнительных блоков должна сохраняться после перестройки.
- Все элементы должны помещаться в ширину экрана без горизонтального переполнения.

- Для автоматического переключения в практической работе используется breakpoint **768 px**: при ширине менее 768 px страница переходит к мобильной компоновке. Ручное переключение Desktop/Mobile можно реализовать средствами CSS и/или небольшим JavaScript-обработчиком.
- При совместном использовании ручного и автоматического режимов необходимо сохранять состояние содержимого страницы — например, выбранный раздел или фильтр — при перестройке интерфейса.

При создании страницы **не следует использовать весь макет как единое фоновое изображение**. Структура должна быть собрана средствами HTML и CSS, а обложки книг и другие изображения подключаться как отдельные элементы.

Итоговая структура страницы

Desktop:

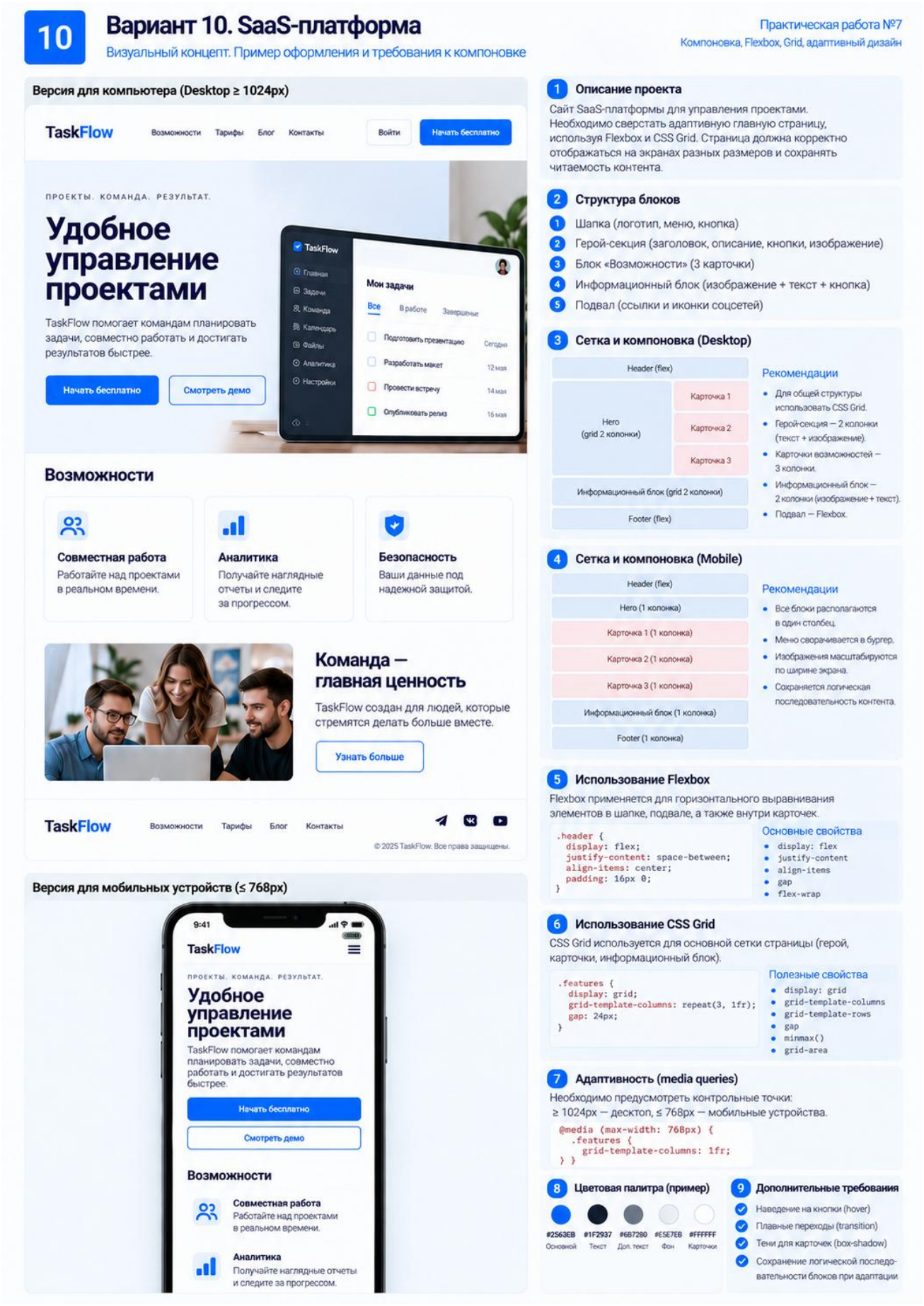
- Header с навигацией и переключателем Desktop/Mobile;
- блок поиска;
- категории;
- каталог популярных книг в CSS Grid;
- новинки;
- авторы;
- личная библиотека;
- Flexbox для строки поиска, метаданных и действий;
- гибкая многоколоночная сетка книг.

Mobile:

- Header с переключателем;
- поиск и фильтры перемещаются вверх;
- категории остаются доступными;
- карточки книг переходят в **одну колонку**;
- обложки сохраняют правильные пропорции;
- новинки, авторы и личная библиотека располагаются последовательно;
- все элементы остаются в пределах ширины экрана.

Именно такой характер перестройки — **каталог Grid на Desktop → одна колонка книг на Mobile, при перемещении поиска и фильтров вверх** — непосредственно задан для Варианта 9 «Электронная библиотека».

Вариант 10. SaaS-платформа



Разработать адаптивную веб-страницу **SaaS-платформы**, предназначенную для представления информации о текущей выставке, экспозициях, коллекциях, расписании и билетах. Страница должна иметь Desktop- и Mobile-представления и сохранять удобную структуру интерфейса при изменении ширины экрана.

Примечание по методичке: в PDF вариант обозначен как «Вариант 10. SaaS-платформа», однако в описании назначения указана **информационная страница музея и выставки**. Ниже структура составлена именно по содержанию задания из методички, без изменения этого несоответствия.

Основная структура страницы согласно заданию: **Header → текущая выставка → экспозиции → коллекции → расписание → билеты**.

Header

В верхней части страницы разместить название или логотип проекта, основную навигацию и элементы управления.

В Header можно расположить:

- название или логотип;
- ссылки на основные разделы страницы;
- кнопку перехода к билетам;
- дополнительные элементы навигации;
- переключатель **Desktop / Mobile**.

Переключатель Desktop/Mobile должен находиться именно в **Header или Nav**.

Шапка должна оставаться отдельным визуальным блоком и не смешиваться с содержимым выставки.

Текущая выставка

Основной визуальный блок страницы, посвящённый текущей выставке.

В нём можно разместить:

- название выставки;
- крупное изображение;
- краткое описание;
- дополнительную информацию;
- кнопку перехода к билетам или расписанию.

Точное содержание этого блока методический материал не определяет, поэтому конкретные тексты, названия и дополнительные элементы можно выбрать самостоятельно.

На Desktop крупный визуальный блок должен занимать заметное место на странице и сохранять свои пропорции при изменении ширины.

На Mobile блок должен уменьшаться по ширине без искажения изображения.

Экспозиции

В этом разделе разместить карточки отдельных экспонатов или частей текущей экспозиции.

Каждая карточка может содержать:

- изображение экспоната;
- название;
- краткое описание;
- дополнительную информацию.

Конкретный состав данных для карточки в методичке не указан, поэтому его можно определить самостоятельно.

На Desktop карточки экспонатов должны формировать **CSS Grid**.

На Mobile карточки экспонатов должны переходить в **одну колонку** и располагаться последовательно.

Коллекции

Отдельный блок предназначен для представления коллекций музея.

В нём можно разместить карточки или небольшие информационные блоки с:

- названием коллекции;
- изображением;
- кратким описанием;
- дополнительной информацией.
- Точное содержимое коллекций методический материал не задаёт.

На Desktop коллекции можно расположить в несколько колонок с использованием Grid. На Mobile элементы должны перестраиваться последовательно, сохраняя читаемость текста и изображений.

Расписание

В данном разделе разместить информацию о времени посещения выставки или отдельных мероприятий.

Можно использовать:

- дату;
- время;
- название мероприятия;
- дополнительную информацию;
- кнопку или ссылку.

Конкретная структура расписания в методичке не указана.

На Desktop расписание может иметь более подробное представление. На Mobile оно должно располагаться ниже основных карточек и сохранять **читаемую иерархию**.

Информация о времени, названии события и дополнительных сведениях должна оставаться визуально связанной после перестройки страницы.

Билеты

Отдельный блок предназначен для информации о билетах.

В нём можно разместить:

- тип билета;
- стоимость;
- краткие условия;
- кнопку покупки или выбора билета.

Точный состав вариантов билетов и их стоимость методический материал не определяет.

На Desktop блок может располагаться рядом с основной информацией или отдельной секцией. На Mobile блок с билетами должен перемещаться **ниже основного содержания**, как предусмотрено заданием.

Требования к компоновке

- **CSS Grid** обязательно применить минимум в одной основной сетке или композиции.
- Экспонаты и выставки на Desktop размещать с использованием **Grid**.
- **Flexbox** применить минимум в одном самостоятельном блоке интерфейса.
- Служебные строки и кнопки организовать с помощью **Flexbox**.
- На Desktop карточки экспонатов и выставок располагаются в несколько колонок.
- На Mobile карточки экспонатов становятся **одноколоночными**.
- Расписание и билеты на Mobile перемещаются ниже основного содержимого.
- Крупный визуальный блок текущей выставки должен сохранять пропорции при изменении ширины.
- Разные карточки и информационные блоки должны иметь согласованные отступы и визуальную сетку.
- В Header или Nav добавить ручное переключение **Desktop / Mobile**.
- Реализовать автоматическую адаптацию при изменении ширины окна.
- Расписание и связанные с ним данные должны сохранять читаемую иерархию на мобильной ширине.
- Основная структура страницы не должна просто масштабироваться — блоки должны **перестраиваться** под мобильный экран.
- Для автоматического переключения используется breakpoint **768 px**. При ширине менее 768 px страница должна переходить к мобильной компоновке. Ручное переключение Desktop/Mobile допускается реализовать средствами CSS и/или небольшим JavaScript-обработчиком.
- Если ручной и автоматический режимы используются одновременно, выбранное состояние содержимого страницы не должно исчезать при изменении ширины окна.

При создании страницы **не следует использовать весь макет как единое фоновое изображение**. Структура должна быть собрана средствами HTML и CSS, а изображения выставки и экспонатов подключаться отдельно.

Итоговая структура страницы

Desktop:

- Header с навигацией и переключателем Desktop/Mobile;
- крупный блок текущей выставки;
- сетка экспозиций;
- сетка коллекций;
- расписание;
- блок билетов;
- Grid для карточек экспонатов и выставок;
- Flexbox для служебных строк и кнопок;
- крупный визуальный блок сохраняет пропорции.

Mobile:

- Header с переключателем;
- текущая выставка адаптируется по ширине без искажения изображения;
- карточки экспонатов переходят в **одну колонку**;
- коллекции располагаются последовательно;
- расписание перемещается ниже основного содержимого;
- билеты перемещаются ниже основного содержимого;
- расписание и билеты сохраняют читаемую иерархию;
- все элементы помещаются в ширину экрана без горизонтального переполнения.

Вариант 11. Новости

Разработать адаптивную веб-страницу **новостного портала**, предназначенную для просмотра новостей по категориям, чтения главной публикации, просмотра ленты материалов, популярных новостей и оформления подписки. Страница должна иметь Desktop- и Mobile-представления и сохранять удобную структуру интерфейса при изменении ширины экрана. Согласно заданию, главная новость должна оставаться визуальным центром страницы, а иерархия Header, Nav, основного контента и дополнительных блоков — сохраняться после перестройки.

Основная структура страницы согласно заданию: **Header → категории → главная новость → лента публикаций → популярное → подписка**.

Header

В верхней части страницы разместить шапку новостного портала.

В Header можно предусмотреть:

- название или логотип новостного портала;
- основную навигацию;
- ссылки на основные разделы;
- кнопку или дополнительный элемент действия;
- переключатель **Desktop / Mobile**.

Переключатель Desktop/Mobile должен находиться в Header или Nav, как предусмотрено заданием.

Категории

Под Header разместить блок категорий новостей.

В нём можно предусмотреть:

- список тематических категорий;
- отдельную активную категорию;
- ссылки или кнопки перехода между категориями;
- дополнительные фильтры, если они предусмотрены макетом.

Точный состав категорий в методичке не указан, поэтому конкретные названия и количество категорий определяются макетом.

На Desktop категории могут располагаться в одну горизонтальную строку с использованием **Flexbox**.

На Mobile блок должен перестраиваться так, чтобы категории оставались доступными и не приводили к горизонтальному переполнению.

Главная новость

После категорий разместить основной визуальный блок с **главной новостью**.

В блоке можно разместить:

- крупное изображение;

- заголовок новости;
- краткое описание или анонс;
- дату публикации;
- категорию;
- кнопку или ссылку для перехода к полной публикации.

Главная новость должна визуально выделяться среди остальных материалов и оставаться **центральным элементом страницы** как в Desktop-, так и в Mobile-представлении. Это отдельно указано в требованиях варианта.

Лента публикаций

Основную часть контента после главной новости занять лентой публикаций.

Каждая новость может быть представлена отдельной карточкой, содержащей:

- изображение;
- заголовок;
- краткое описание;
- дату;
- категорию;
- ссылку на материал.

На Desktop карточки ленты могут располагаться в несколько колонок с помощью **CSS Grid**.

На Mobile карточки новостей должны становиться последовательными и располагаться **одна за другой в одну колонку**.

Популярное

Отдельно разместить блок **популярных новостей**.

В нём можно предусмотреть:

- список наиболее популярных публикаций;
- небольшие карточки;
- заголовки;
- номера или другие элементы, визуально выделяющие порядок материалов;
- дату или дополнительную информацию.

На Desktop данный блок должен входить в боковую область страницы — **Sidebar**.

Вместе с главной новостью и лентой публикаций Sidebar должен формироваться средствами **CSS Grid Areas**.

Подписка

В нижней или дополнительной части страницы разместить блок подписки на новости.

Можно предусмотреть:

- заголовок;
- краткое описание;
- поле для электронной почты;
- кнопку подписки;
- дополнительную информацию об условиях подписки.

Точный состав содержимого блока подписки в методичке не указан, поэтому он определяется визуальным макетом.

На Mobile блок подписки должен оставаться частью общей вертикальной структуры страницы и не нарушать её иерархию.

Требования к компоновке

- **CSS Grid** применить минимум в одной основной сетке или композиции страницы.
- Основную композицию Desktop построить через **Grid Areas**, распределив области для главной новости, ленты публикаций и Sidebar.
- **Flexbox** применить минимум в одном самостоятельном блоке интерфейса.
- Меню Header/Nav реализовать с использованием **Flexbox**.
- На Desktop главная новость, лента публикаций и Sidebar должны образовывать единую Grid-композицию.
- На Mobile Sidebar должен переноситься **ниже ленты публикаций**.
- Карточки новостей на Mobile должны становиться последовательными, то есть располагаться в одну колонку.
- Главная новость должна сохранять визуальный приоритет и оставаться центром страницы.
- Иерархия **Header → Nav → основной контент → дополнительные блоки** должна сохраняться после перестройки.
- Необходимо добавить ручное переключение **Desktop/Mobile** в Header или Nav.
- Необходимо добавить автоматическую адаптацию при изменении ширины окна.
- Для автоматического перехода между представлениями можно использовать CSS media queries. В методических материалах в качестве контрольной границы используется **768 px**: например, при max-width: 768px сетка может переходить к одной колонке.
- При одновременном использовании ручного и автоматического режимов необходимо учитывать сохранение состояния содержимого при переключении.

Макет страницы не следует реализовывать как одно большое фоновое изображение. Структура должна быть собрана средствами HTML и CSS, а изображения новостей использоваться как отдельные элементы страницы.

Итоговая структура страницы

Desktop:

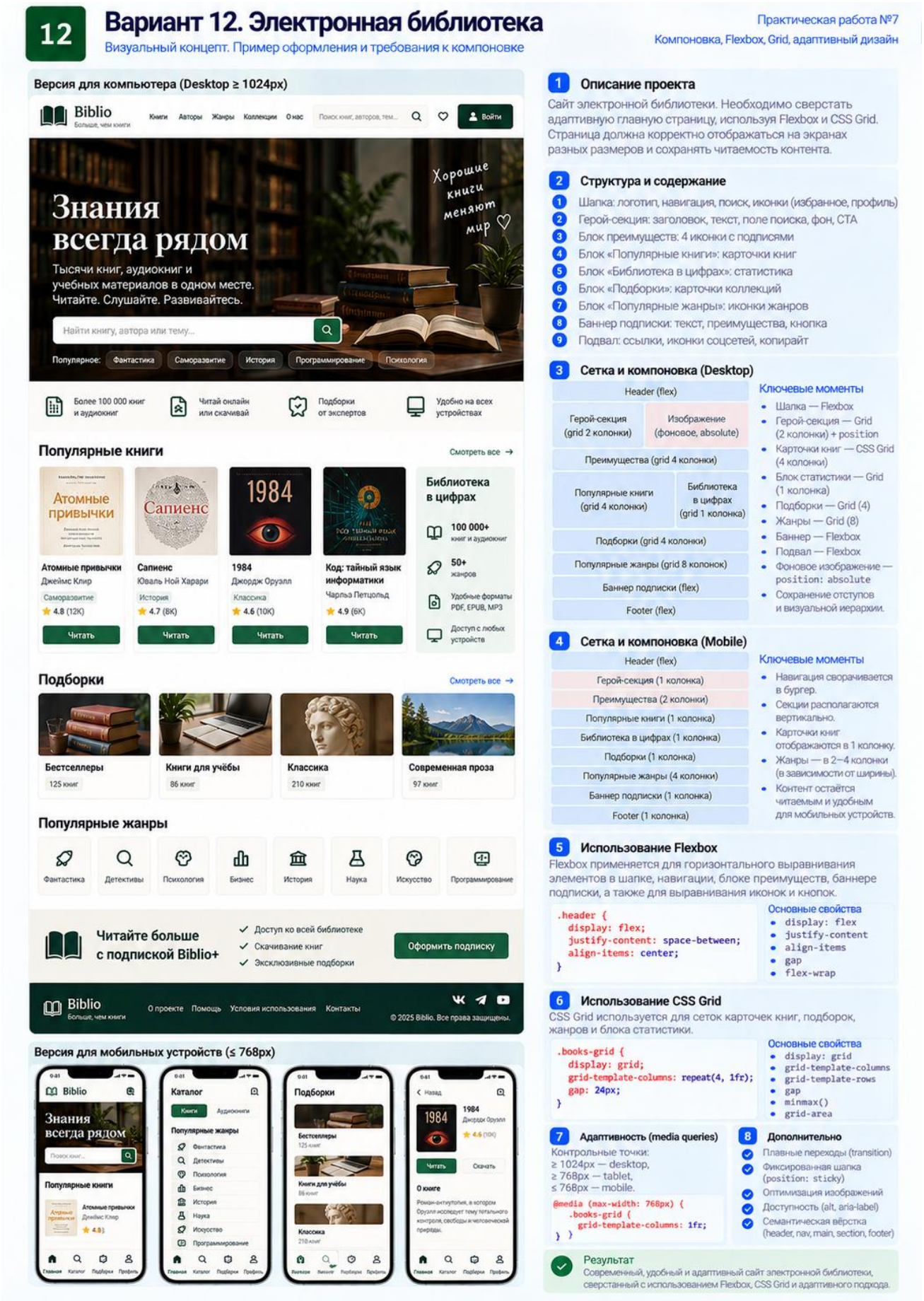
- Header с навигацией и переключателем Desktop/Mobile;
- категории;
- основная Grid-композиция;

- крупная главная новость как визуальный центр;
- лента публикаций;
- Sidebar с популярными новостями;
- блок подписки;
- Grid Areas для расположения основных областей;
- Flexbox для меню и отдельных элементов интерфейса.

Mobile:

- Header и навигация перестраиваются под узкую ширину;
- категории остаются доступными без горизонтального переполнения;
- главная новость сохраняет визуальный приоритет;
- лента публикаций становится одноколоночной;
- Sidebar с популярными новостями переносится **ниже ленты**;
- подписка располагается в общей вертикальной структуре;
- иерархия Header, Nav, основного контента и дополнительных блоков сохраняется.

Вариант 12. Электронная библиотека



Разработать адаптивную веб-страницу **электронной библиотеки**, предназначенную для поиска и просмотра книг, работы с категориями, популярными книгами, новинками, авторами и личной библиотекой. Страница должна иметь Desktop- и Mobile-представления и сохранять удобную структуру интерфейса при изменении ширины экрана. Согласно заданию, обложки книг не должны искажаться при изменении размеров карточек, а сетка должна гибко перестраиваться с Desktop на Mobile.

Основная структура страницы согласно заданию: **поиск → категории → популярные книги → новинки → авторы → личная библиотека**.

Header

В верхней части страницы разместить шапку электронной библиотеки.

В Header можно предусмотреть:

- название или логотип библиотеки;
- основную навигацию;
- ссылки на разделы библиотеки;
- дополнительные элементы управления;
- переключатель **Desktop / Mobile**.

Переключатель Desktop/Mobile должен находиться в Header или Nav, как предусмотрено заданием.

Поиск

Основным элементом верхней части страницы разместить блок поиска книг.

В нём можно предусмотреть:

- поле поиска;
- кнопку поиска;
- дополнительные параметры;
- фильтры по книгам;
- сортировку, если она предусмотрена макетом.

Точный набор полей поиска в методичке не указан, поэтому конкретное содержимое определяется макетом.

На Desktop строка поиска и связанные с ней элементы должны быть организованы с использованием **Flexbox**.

На Mobile поиск и фильтры должны перемещаться **вверх страницы**, чтобы пользователь мог быстро получить доступ к ним.

Категории

После поиска разместить блок категорий книг.

В нём можно предусмотреть:

- список жанров или тематических категорий;

- отдельную активную категорию;
- ссылки или кнопки перехода;
- дополнительные фильтры.

Точный перечень категорий в методичке не указан, поэтому конкретные названия определяются макетом.

На Desktop категории могут располагаться в горизонтальном ряду.

На Mobile блок должен перестраиваться под узкую ширину и не создавать горизонтального переполнения.

Популярные книги

Основную часть каталога начать с блока **популярных книг**.

Каждая книга может быть представлена карточкой, содержащей:

- обложку;
- название;
- автора;
- краткую дополнительную информацию;
- категорию или жанр;
- кнопку или ссылку для просмотра книги.

Каталог книг на Desktop должен строиться с использованием **CSS Grid**. В качестве примера в методичке указана сетка из 3 колонок, которая на Mobile переходит в 1 колонку.

Ширина колонок должна быть гибкой, чтобы карточки корректно изменяли размер вместе с шириной доступной области.

Новинки

Следующим блоком разместить раздел **новинок**.

Карточки новинок могут содержать:

- обложку;
- название книги;
- имя автора;
- краткие сведения;
- дату или отметку о новизне;
- кнопку или ссылку.

Структура карточек должна соответствовать общей визуальной сетке каталога.

На Desktop книги располагаются в несколько колонок с помощью **CSS Grid**.

На Mobile карточки переходят в последовательное одноколоночное расположение. Обложка при этом должна сохранять свои пропорции и не растягиваться.

Авторы

Отдельно разместить блок **авторов**.

В нём можно предусмотреть:

- имя автора;
- фотографию или изображение;
- краткую информацию;
- количество книг;
- ссылку на страницу автора.

Точный состав данных об авторах в методичке не указан, поэтому он определяется макетом.

На Desktop авторы могут располагаться в несколько колонок.

На Mobile элементы должны последовательно перестраиваться под доступную ширину.

Личная библиотека

В нижней части основной структуры разместить раздел **личной библиотеки**.

В нём можно предусмотреть:

- книги пользователя;
- обложки;
- название и автора;
- статус книги;
- дополнительные действия;
- ссылки на просмотр или продолжение чтения.

Точный состав элементов личной библиотеки методичкой не задан, поэтому он определяется визуальным макетом.

На Desktop данный блок должен сохранять общую структуру каталога.

На Mobile содержимое должно переходить в удобную последовательную структуру без горизонтального переполнения.

Требования к компоновке

- **CSS Grid** применить минимум в одной основной сетке или композиции.
- Каталог книг на Desktop построить с помощью **CSS Grid**.
- **Flexbox** применить минимум в одном самостоятельном блоке интерфейса.
- Строку поиска, метаданные книг и элементы действий организовать с помощью **Flexbox**.
- На Desktop карточки книг могут располагаться, например, в **3 колонки**.
- На Mobile книжные карточки должны переходить в **1 колонку**.
- Ширина колонок должна быть гибкой.
- Фильтры и поиск на Mobile должны перемещаться вверх страницы.
- Обложки книг не должны искажаться при изменении размеров карточек.
- Необходимо добавить ручное переключение **Desktop/Mobile** в Header или Nav.

- Необходимо добавить автоматическую адаптацию при изменении ширины окна.
- Для автоматического перехода между представлениями можно использовать CSS media queries. В методических материалах в качестве контрольной границы используется **768 px**, при которой сетка может переходить к одной колонке.
- При одновременном использовании ручного и автоматического режимов необходимо учитывать сохранение состояния содержимого при переключении.

Макет страницы не следует реализовывать как одно большое фоновое изображение. Структура должна быть собрана средствами HTML и CSS, а обложки книг и другие изображения использоваться как отдельные элементы страницы.

Итоговая структура страницы

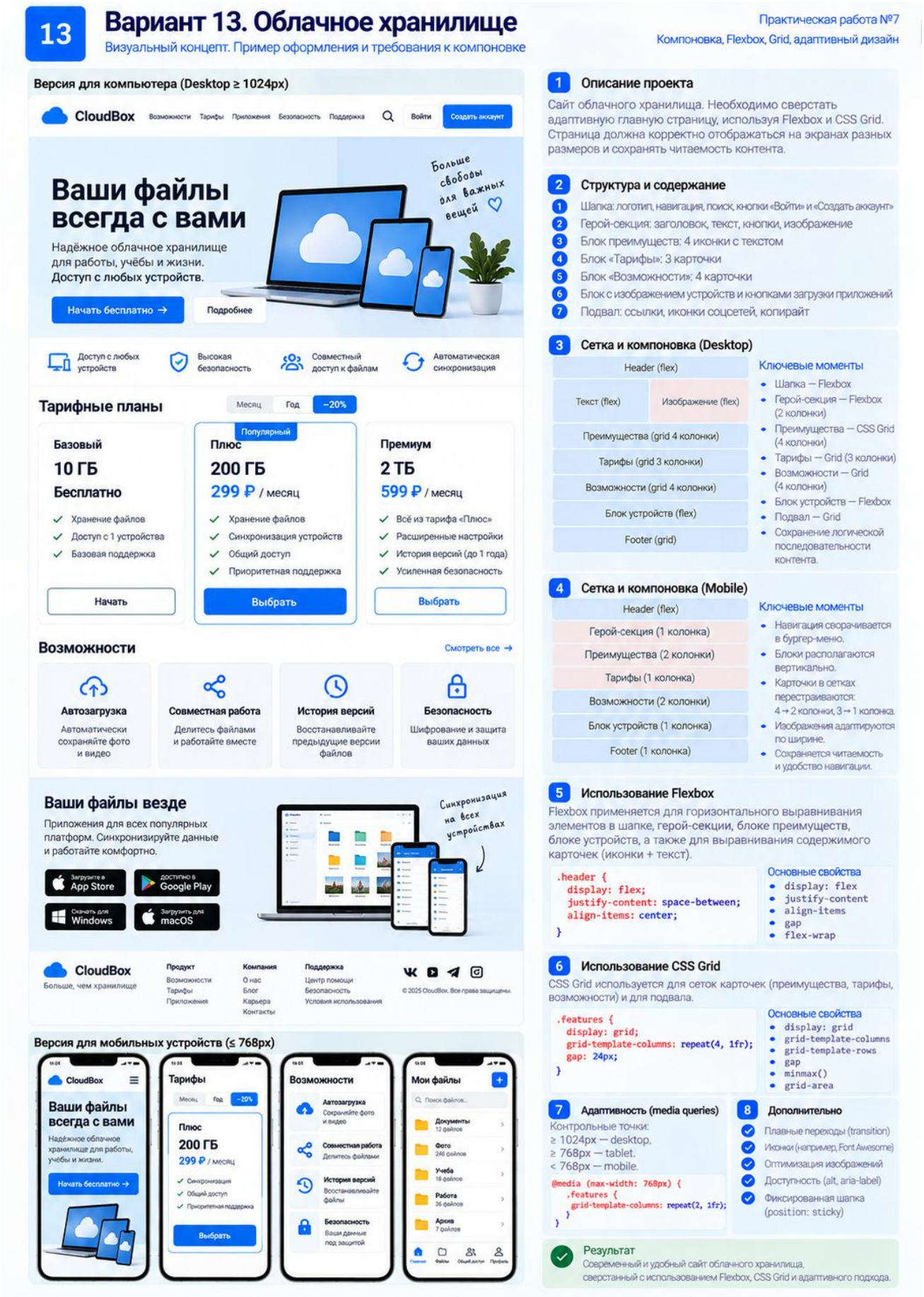
Desktop:

- Header с навигацией и переключателем Desktop/Mobile;
- строка поиска и фильтры;
- категории;
- каталог популярных книг;
- каталог новинок;
- блок авторов;
- личная библиотека;
- CSS Grid для книжных карточек;
- Flexbox для поиска, метаданных и действий;
- гибкая ширина колонок.

Mobile:

- Header и навигация перестраиваются под узкую ширину;
- поиск и фильтры перемещаются вверх;
- категории остаются доступными;
- книжные карточки переходят из нескольких колонок в **одну колонку**;
- популярные книги и новинки располагаются последовательно;
- обложки сохраняют исходные пропорции и не искажаются;
- блок авторов и личная библиотека также перестраиваются под ширину экрана;
- горизонтального переполнения быть не должно.

Вариант 13. Облачное хранилище



Разработать адаптивную веб-страницу **облачного хранилища**, предназначенную для работы с папками и файлами, управления содержимым хранилища и просмотра информации о свободном месте. Страница должна иметь Desktop- и Mobile-представления и сохранять удобную структуру интерфейса при изменении ширины экрана. Согласно заданию, действия над файлами должны оставаться доступными после перестройки страницы.

Основная структура страницы согласно заданию: **Sidebar** → **панель пользователя** → **папки** → **файлы** → **информация о свободном месте**.

Sidebar

Боковая панель навигации по основным разделам облачного хранилища.

В Sidebar можно разместить:

- логотип или название сервиса;
- пункты навигации;
- переход к файлам;
- переход к папкам;
- дополнительные разделы хранилища;
- пункт настроек или профиля.

Точное содержимое пунктов меню в методичке не указано, поэтому конкретные названия можно определить самостоятельно.

На Desktop Sidebar должен быть полноценной боковой частью интерфейса и участвовать в общей Grid Areas-композиции. На Mobile Sidebar превращается в компактную панель.

Панель пользователя

В верхней части основной области должна находиться панель пользователя.

В ней можно разместить:

- имя или аватар пользователя;
- информацию об аккаунте;
- дополнительные действия;
- кнопку профиля;
- элементы управления хранилищем.

Также в Header или Nav необходимо предусмотреть **ручное переключение Desktop/Mobile**.

Папки

Основной блок с папками пользователя.

Каждая папка может быть представлена отдельной карточкой или строкой со следующей информацией:

- название папки;
- иконка;
- количество файлов;
- дополнительная информация;
- кнопка или действие открытия.

Точный состав информации для папок методичкой не задан.

На Desktop папки могут располагаться в несколько колонок. Ширина колонок должна быть гибкой. При уменьшении экрана количество колонок должно сокращаться.

Файлы

Основной рабочий блок страницы — список или сетка файлов.

Для каждого файла можно показать:

- название;
- тип файла;
- иконку или превью;
- размер;
- дату изменения;
- дополнительные сведения;
- действия над файлом.

Конкретный набор полей методичкой не определён.

На Desktop файловая область занимает основную часть страницы. Элементы списка и панели действий должны строиться с использованием **Flexbox**.

На Mobile файловый список должен занимать **всю ширину доступной области**.

Особое внимание необходимо уделить действиям над файлами: после перестройки интерфейса они должны оставаться доступными пользователю.

Информация о свободном месте

Отдельный блок с информацией о состоянии облачного хранилища.

Можно разместить:

- объём занятого пространства;
- общий доступный объём;
- оставшееся свободное место;
- визуальный индикатор заполнения;
- дополнительную информацию о хранилище.

Точная форма блока в методичке не указана, поэтому его внешний вид определяется макетом.

Требования к компоновке

- Основная структура приложения на Desktop должна строиться с использованием **CSS Grid Areas**.
- Панели действий и элементы списка должны использовать **Flexbox**.
- Flexbox необходимо применить минимум в одном самостоятельном блоке интерфейса.
- CSS Grid необходимо применить минимум в одной основной сетке или композиции.
- На Desktop Sidebar должен быть частью основной Grid Areas-композиции.
- На Mobile Sidebar должен превращаться в компактную панель.
- На Mobile файловый список должен занимать всю доступную ширину.
- Карточки и сетки должны иметь гибкую ширину.
- Сетка может перестраиваться, например, из **3 колонок в 1 колонку**.
- Действия над файлами должны оставаться доступными после перестройки.
- Необходимо добавить ручное переключение **Desktop/Mobile** в Header или Nav.
- Необходимо реализовать автоматическую адаптацию при изменении ширины окна.
- При одновременном использовании ручного и автоматического режимов состояние содержимого должно сохраняться.

Адаптация при ширине 768 px

В качестве основной точки перестройки использовать **768 px**.

При ширине больше 768 px:

- отображается полноценный Sidebar;
- основная структура строится через Grid Areas;
- папки и файлы могут располагаться в несколько колонок;
- панели действий и элементы списка используют Flexbox.
- При ширине меньше 768 px:
- Sidebar превращается в компактную панель;
- файловый список занимает всю ширину;
- сетки перестраиваются в меньшее количество колонок;
- действия над файлами остаются доступными и не выходят за пределы экрана.

Переключение должно происходить не только вручную, но и автоматически при изменении ширины окна браузера.

Изображения и фон

Не следует использовать одно изображение как фон всей страницы. Структура должна быть создана средствами **HTML и CSS**, а изображения, иконки и другие визуальные элементы должны подключаться как отдельные элементы интерфейса.

Итоговая структура страницы

Desktop:

- Sidebar;

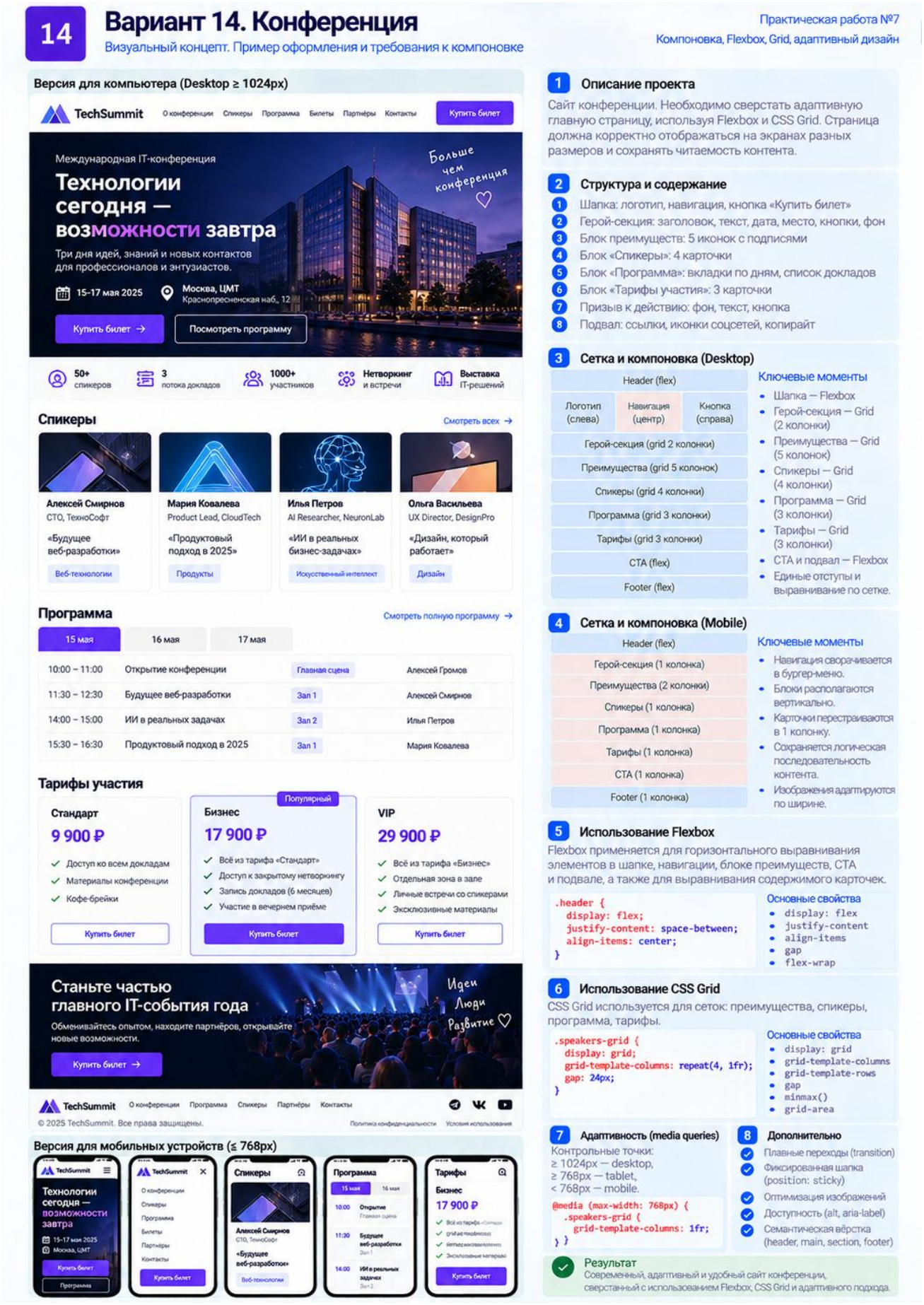
- панель пользователя;
- папки;
- файлы;
- информация о свободном месте;
- основная компоновка через CSS Grid Areas;
- панели действий и элементы списка через Flexbox.

Mobile:

- компактный Sidebar;
- панель пользователя;
- папки;
- файловый список на всю ширину;
- информация о свободном месте;
- сетки перестраиваются в меньшее количество колонок;
- действия над файлами остаются доступными;
- структура страницы сохраняет понятную иерархию.

Именно **Sidebar + панель пользователя + папки + файлы + информация о свободном месте** являются основными обязательными блоками Варианта 13 согласно методичке.

Вариант 14. Конференция



Разработать адаптивную веб-страницу **сайта технической конференции**, предназначенную для представления информации о конференции, её программе, докладах и спикерах, а также для регистрации участников. Страница должна иметь Desktop- и Mobile-представления и сохранять удобную структуру интерфейса при изменении ширины экрана. Согласно заданию, время, название доклада и спикер должны оставаться визуально связанными между собой.

Основная структура страницы согласно заданию: **Hero → дата/место → программа → доклады → спикеры → регистрация**.

Header

В верхней части страницы располагается Header с основной информацией о конференции.

В Header можно разместить:

- название или логотип конференции;
- основные пункты навигации;
- ссылки на программу;
- переход к докладам и спикерам;
- кнопку регистрации;
- дополнительные элементы управления.

Также в Header или Nav необходимо предусмотреть **ручное переключение Desktop/Mobile**.

Hero

Главный визуальный блок страницы конференции.

В Hero можно разместить:

- название конференции;
- краткое описание;
- основную иллюстрацию или фотографию;
- информацию о тематике мероприятия;
- кнопку регистрации.

Точный состав содержимого Hero методичкой не указан, поэтому конкретный текст и дополнительные элементы определяются макетом.

Hero должен сразу обозначать основное назначение страницы и связываться с информацией о дате и месте проведения.

Дата и место

Отдельный информационный блок с основными сведениями о мероприятии.

Можно разместить:

- дату проведения;

- время;
- место проведения;
- адрес;
- дополнительную информацию о формате конференции.

Информацию о дате и месте желательно оформить компактной группой элементов, чтобы она хорошо воспринималась как единый блок.

Программа

Основной блок с расписанием конференции.

Для каждого элемента программы необходимо показать:

- время;
- название доклада;
- имя спикера;
- дополнительную информацию при необходимости.

На Desktop расписание должно строиться с использованием **CSS Grid**.

На Mobile программа должна превращаться в **последовательность временных блоков**.

Особое внимание необходимо уделить тому, чтобы **время, название доклада и спикер оставались визуально связанными** после перестройки страницы.

Доклады

Раздел с карточками докладов конференции.

Каждый доклад можно представить отдельной карточкой, содержащей:

- название доклада;
- краткое описание;
- имя спикера;
- время проведения;
- дополнительную информацию;
- кнопку или ссылку на подробности.

Точный состав информации для карточек докладов в методичке не определён.

На Desktop карточки могут располагаться в несколько колонок. Ширина колонок должна быть гибкой.

На Mobile карточки должны перестраиваться последовательно и не создавать горизонтального переполнения.

Спикеры

Раздел с участниками конференции, которые проводят доклады.

Для каждого спикера можно разместить:

- фотографию;
- имя;
- должность или место работы;
- краткое описание;
- название или список докладов.

Точный состав информации методичкой не задан.

На Desktop карточки спикеров должны строиться с использованием **CSS Grid**.

На Mobile карточки спикеров переходят **в одну колонку**.

Регистрация

Заключительный основной блок страницы для регистрации участника.

В нём можно разместить:

- заголовок;
- краткое описание;
- поля формы;
- имя участника;
- электронную почту;
- дополнительные данные;
- кнопку регистрации.

Точный состав полей формы методичкой не определён.

Строки с элементами формы и CTA-кнопка могут быть организованы с использованием **Flexbox**.

Требования к компоновке

- Расписание и карточки спикеров на Desktop должны строиться с использованием **CSS Grid**.
- Flexbox необходимо применить минимум в одном самостоятельном блоке интерфейса.
- Строки времени и CTA должны использовать **Flexbox**.
- CSS Grid необходимо применить минимум в одной основной сетке или композиции.
- На Desktop программа должна иметь структурированное Grid-представление.
- На Mobile программа должна превращаться в последовательность временных блоков.
- На Desktop карточки спикеров располагаются в сетке.
- На Mobile спикеры переходят в одну колонку.
- Время, название доклада и спикер должны оставаться связанными визуально.
- Расписание должно сохранять читаемую иерархию на мобильной ширине.
- Необходимо добавить ручное переключение **Desktop/Mobile** в Header или Nav.

- Необходимо реализовать автоматическую адаптацию при изменении ширины окна.
- Карточки и сетки должны иметь гибкую ширину и перестраиваться без горизонтального переполнения.

Адаптация при ширине 768 px

В качестве основной точки перестройки использовать **768 px**.

При ширине больше 768 px:

- отображается полноценная версия Header;
- программа располагается в Grid-композиции;
- карточки спикеров располагаются в несколько колонок;
- строки времени и CTA используют Flexbox;
- основные блоки могут занимать несколько колонок.

При ширине меньше 768 px:

- программа превращается в последовательность временных блоков;
- спикеры переходят в одну колонку;
- информация о времени, докладе и спикере остаётся связанной;
- элементы формы и CTA перестраиваются вертикально;
- содержимое не выходит за пределы экрана.

Переключение должно происходить не только вручную, но и автоматически при изменении ширины окна браузера.

Изображения и фон

Не следует использовать одно изображение как фон всей страницы. Структура должна быть создана средствами **HTML и CSS**, а фотографии спикеров, иллюстрации и другие изображения должны подключаться как отдельные элементы интерфейса.

Итоговая структура страницы

Desktop:

- Header;
- Hero;
- дата и место;
- программа в Grid;
- доклады;
- спикеры в Grid;
- регистрация;
- строки времени и CTA через Flexbox.

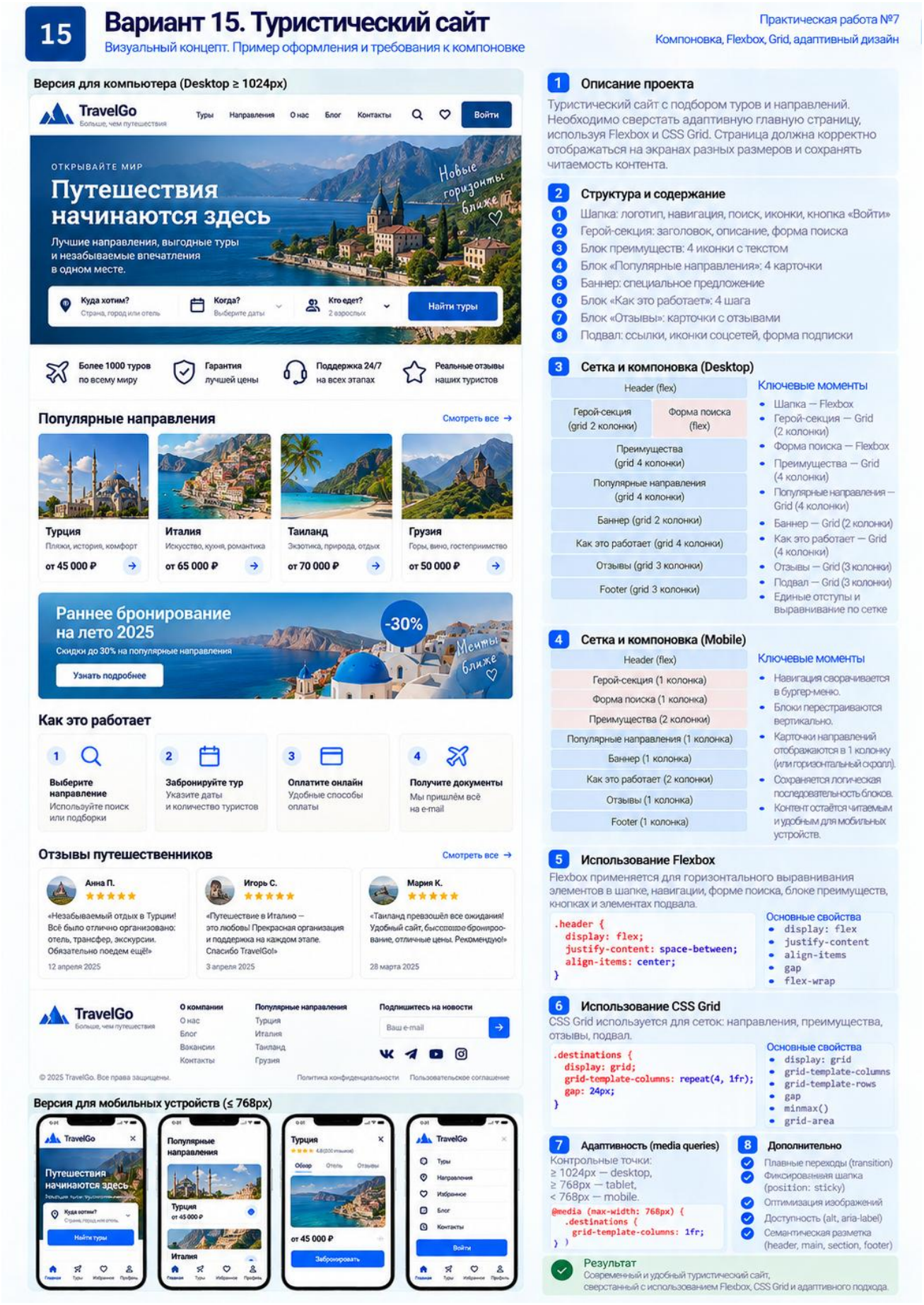
Mobile:

- Header;
- Hero;

- дата и место;
- программа в виде последовательности временных блоков;
- доклады последовательно;
- спикеры в одну колонку;
- регистрация и CTA перестраиваются под узкую ширину;
- время, название доклада и спикер остаются визуально связанными;
- расписание сохраняет читаемую иерархию.

Именно **Hero + дата/место + программа + доклады + спикеры + регистрация** являются основными обязательными блоками Варианта 14 согласно методичке.

Вариант 15. Туристический сайт



Разработать адаптивную веб-страницу презентационного сайта с использованием современных методов CSS-компоновки. Страница должна иметь Desktop- и Mobile-представления и сохранять понятную структуру, визуальную иерархию и удобство взаимодействия при изменении ширины экрана.

Основная структура страницы согласно заданию:

Header → Hero → услуги → проекты → технологии → команда → контакты.

Header

В верхней части страницы располагается Header с основной информацией о сайте.

В Header можно разместить:

- название или логотип;
- основную навигацию;
- ссылки на разделы страницы;
- дополнительные элементы управления;
- кнопку перехода к контактам или целевому действию.

В Header или Nav необходимо предусмотреть **ручное переключение представления Desktop/Mobile**.

Hero

Главный визуальный блок страницы, предназначенный для представления сайта и его основной информации.

В Hero можно разместить:

- название или основное сообщение страницы;
- краткое описание;
- основную иллюстрацию;
- дополнительную информацию;
- кнопку целевого действия (CTA).

Hero должен сразу обозначать назначение страницы и визуально выделяться среди остальных разделов.

Для компоновки элементов Hero необходимо использовать **Flexbox**.

Услуги

Раздел с основными услугами, представленными на странице.

Каждую услугу рекомендуется представить отдельной карточкой, содержащей:

- название услуги;
- краткое описание;
- дополнительную информацию;
- при необходимости — иконку или изображение;

- ссылку или кнопку с дополнительными сведениями.

На Desktop карточки услуг должны располагаться в несколько колонок с использованием **CSS Grid**.

На Mobile карточки должны перестраиваться в последовательность блоков, не создавая горизонтального переполнения.

Проекты

Раздел с примерами выполненных проектов.

Для каждого проекта можно разместить:

- название;
- изображение или иллюстрацию;
- краткое описание;
- используемые технологии;
- дополнительную информацию;
- ссылку или кнопку перехода к проекту.

На Desktop проекты должны формироваться в адаптивную Grid-сетку.

Ширина колонок должна быть гибкой, чтобы блоки равномерно заполняли доступное пространство.

На Mobile проекты должны переходить в последовательное расположение.

Технологии

Раздел, предназначенный для представления используемых технологий.

В нём можно разместить:

- названия технологий;
- логотипы или иконки;
- краткое описание назначения каждой технологии;
- дополнительные информационные элементы.

Элементы раздела должны иметь согласованное визуальное оформление и вписываться в общую ширину контейнера страницы.

При необходимости для расположения элементов можно использовать CSS Grid или Flexbox.

Команда

Раздел с информацией о специалистах.

Для каждого участника команды можно разместить:

- фотографию;
- имя;
- должность;

- краткое описание;
- дополнительную информацию.

На Desktop карточки сотрудников должны располагаться в несколько колонок с использованием **CSS Grid**.

На Mobile карточки команды должны перестраиваться в последовательную структуру, например в одну колонку.

Контакты

Заключительный основной блок страницы.

В нём можно разместить:

- контактную информацию;
- адрес;
- электронную почту;
- телефон;
- форму обратной связи;
- кнопку целевого действия;
- дополнительные ссылки.

Для расположения элементов контактов необходимо использовать **Flexbox**.

На Mobile элементы контактов и CTA должны перестраиваться вертикально, чтобы содержимое полностью помещалось в доступную ширину экрана.

Требования к компоновке

- Услуги, проекты и команда на Desktop должны строиться с использованием **CSS Grid**.
- Hero и блок контактов должны использовать **Flexbox**.
- Flexbox необходимо применить минимум в одном самостоятельном блоке интерфейса.
- CSS Grid необходимо применить минимум в одной основной сетке или композиции.
- На Desktop основные содержательные блоки должны располагаться в несколько колонок там, где это предусмотрено макетом.
- На Mobile сетки должны перестраиваться в последовательные блоки.
- CTA и элементы контактного блока на мобильной ширине должны располагаться вертикально.
- Разные секции страницы должны иметь **единую ширину контейнера и согласованный визуальный ритм**.
- Визуальная иерархия Header, Nav, основного содержимого и дополнительных блоков должна сохраняться после перестройки страницы.
- Необходимо добавить ручное переключение Desktop/Mobile в Header или Nav.
- Необходимо реализовать автоматическую адаптацию страницы при изменении ширины окна браузера.

- Карточки и сетки должны иметь гибкую ширину и не создавать горизонтального переполнения.

Адаптация при ширине 768 px

В качестве основной точки перестройки страницы использовать **768 px**.

При ширине больше 768 px:

- отображается полноценная версия Header;
- услуги располагаются в Grid-сетке;
- проекты располагаются в Grid-сетке;
- карточки команды располагаются в несколько колонок;
- Hero использует Flexbox;
- блок контактов использует Flexbox;
- основные содержательные секции могут занимать несколько колонок.

При ширине меньше 768 px:

- основные Grid-сетки перестраиваются в последовательные блоки;
- услуги располагаются последовательно;
- проекты располагаются последовательно;
- команда переходит в одну колонку;
- CTA перестраивается под узкую ширину;
- контактная информация располагается вертикально;
- содержимое не выходит за пределы экрана;
- общая визуальная иерархия страницы сохраняется.

Переключение должно происходить **не только вручную, но и автоматически** при изменении ширины окна браузера.

Изображения и фон

Не следует использовать одно изображение как фон всей страницы. Структура должна быть создана средствами HTML и CSS, а изображения, фотографии, иконки и другие визуальные элементы должны подключаться как отдельные элементы интерфейса.

Итоговая структура страницы

Desktop:

- Header;
- Hero;
- услуги в Grid;
- проекты в Grid;
- технологии;
- команда в Grid;
- контакты через Flexbox;
- CTA и отдельные строки интерфейса через Flexbox.

Mobile:

- Header;
- Hero;
- услуги последовательно;
- проекты последовательно;
- технологии;
- команда в одну колонку;
- контакты и CTA перестраиваются вертикально;
- все секции сохраняют единую ширину и визуальную иерархию;
- содержимое не выходит за пределы экрана.

Таким образом, основными обязательными блоками Варианта 15 являются Header + Hero + услуги + проекты + технологии + команда + контакты. При этом индивидуальность варианта определяется составом этих блоков и характером их перестройки, а технология компоновки основана на использовании **Flexbox и CSS Grid**.

Задание 2. Создать страницу

Создать страницу самостоятельно; макет нельзя использовать как единое фоновое изображение. Отдельно требуется реализовать ручное переключение Desktop/Mobile и автоматическое переключение при изменении размеров окна; в исходном задании в качестве примера порога приведено 768 px. Проверить страницу на desktop и мобильной ширине.

При разработке руководствоваться спецификацией.

Общие требования к компоновке

- Страница должна иметь отдельные Desktop и Mobile состояния.
- На Desktop должны быть реализованы две или более смысловые зоны, размещённые с использованием Flexbox и/или Grid.
- На Mobile компоновка должна перестраиваться, а не просто масштабироваться.
- Не использовать таблицы HTML для визуального позиционирования блоков.
- Не использовать большое фоновое изображение как замену HTML/CSS-разметке.
- Основные размеры задавать гибко: %, fr, minmax(), min(), max(), clamp(), rem и т. п. по необходимости.
- Стили компоновки должны находиться во внешнем CSS-файле.

Flexbox: обязательные приёмы

- display: flex;
- flex-direction;
- justify-content;
- align-items;

- gap;
- flex-wrap;
- flex / flex-grow / flex-shrink / flex-basis — минимум в одном обоснованном месте;
- при необходимости order для изменения визуального порядка.

```
   .layout {
  display: flex;
  gap: 24px;
  align-items: stretch;
  flex-wrap: wrap;
}

.main {
  flex: 1 1 0;
}

.sidebar {
  flex: 0 1 280px;
}

@media (max-width: 768px) {
  .layout {
    flex-direction: column;
  }
}
```

CSS Grid: обязательные приёмы

- display: grid;
- grid-template-columns;
- grid-template-rows — если необходимо;
- gap;
- grid-area или явное размещение по колонкам/строкам;
- minmax() для адаптивных карточек рекомендуется.

```
   .cards {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 20px;
}

@media (max-width: 768px) {
  .cards {
    grid-template-columns: 1fr;
  }
}
```

Исходный практикум прямо рассматривает CSS Grid как механизм многоосевой компоновки и приводит свойства grid-template-columns, grid-template-rows и grid-area.

Позиционирование и адаптивность

- relative — контекст для абсолютно позиционированных дочерних элементов;
- absolute — локальные бейджи, кнопки, декоративные элементы;
- fixed — элементы, закреплённые относительно окна;
- sticky — при необходимости закрепление в пределах прокручиваемого блока;
- @media — переключение сетки и размеров;
- проверить ширины минимум на desktop и mobile.

```
   .card {
  position: relative;
}

.card__badge {
  position: absolute;
  top: 12px;
  right: 12px;
}

@media (max-width: 768px) {
  .sidebar {
    position: static;
  }
}
```

Механизм Desktop/Mobile: ручное и автоматическое переключение

Исходная постановка требует, чтобы элементы управления для ручного переключения размещались в Header или Nav, а при изменении ширины окна представление менялось автоматически.

Ручное переключение

Для учебной работы допускается реализовать переключатель средствами CSS и/или небольшим JavaScript-обработчиком. Важно, чтобы пользователь видел, какое представление активно.

```
   <button type="button" data-view="desktop">Desktop</button>
<button type="button" data-view="mobile">Mobile</button>
   window.addEventListener("resize", () => {
  const isMobile = window.innerWidth < 768;
  document.body.classList.toggle("is-mobile", isMobile);
});
```

Если ручной и автоматический режимы реализуются совместно, состояние контента (выбранный пункт, фильтр, текущая секция) не должно исчезать при перестройке интерфейса. Это соответствует требованию исходного задания о сохранении состояния.

Требование к структуре проекта

```
  lab-7/
├── index.html
├── css/
│   └── style.css
├── js/
│   └── app.js
├── images/
│   └── ...
└── README.md
```

- index.html — точка входа;
- css/style.css — все основные правила компоновки и оформления;
- js/app.js — только логика ручного/автоматического переключения, если она используется;
- images/ — локальные изображения;
- README.md — ФИО, группа, вариант, описание, URL репозитория и URL опубликованной страницы.

Исходный практикум требует опубликовать страницу на GitHub.io, сохранить доступность до проверки и разместить созданные элементы в каталоге lab-7.

Задание 3. Представление практической работы

Созданную страницы необходимо опубликовать на сервисе **GitHub.io** и предоставить ссылку (вида https://username.github.io) на законченный и рабочий вариант для проверки преподавателем.

Последовательность действий для публикации:

1. Зарегистрируйтесь на https://github.com/, если ещё не сделали этого.
2. Создайте новый репозиторий с именем username.github.io, где username — ваш ник на GitHub.
3. Создайте новый файл. Когда репозиторий не пуст, то в верхнем правом углу есть кнопка "Create New File", но сейчас, когда ещё нет ни одного файла, создать первый можно по прямой ссылке https://github.com/username/username.github.io/new/master, где username — ваш ник.
4. Введите имя index.html и скопируйте туда содержание файла index.html.
5. Нажмите кнопку "Commit new file".
6. Повторите то же самое для файла style.css.
7. Подтвердите публикацию на GitHub Pages. В настройках репозитория: Settings -> Раздел Options -> Секция GitHub Pages.

8. Через несколько мгновений страница станет доступна по вашему адресу https://username.github.io

Примечание. *Страница должна оставаться доступной по ссылке до момента проверки работы в курсе. Затем опубликованные материалы можно удалить или заместить следующими работами.*

Все созданные элементы (исходные материалы) нужно поместить в каталог **lab-7** на GitHub.

Чек-лист самопроверки

- ☐ Показан Desktop-вариант.
- ☐ Показан Mobile-вариант.
- ☐ Продемонстрировано ручное переключение.
- ☐ Продемонстрировано автоматическое переключение при изменении ширины окна.
- ☐ Объяснено применение Flexbox.
- ☐ Объяснено применение Grid.
- ☐ Объяснено хотя бы одно правило position.
- ☐ Показаны media queries.
- ☐ Состояние контента не теряется при перестройке.
- ☐ Исходный код размещён в GitHub.
- ☐ Страница опубликована на GitHub Pages.
- ☐ README заполнен.

Контрольные вопросы

1. Что такое компоновка в CSS и почему она важен для веб-дизайна?
2. Каковы основные методы компоновки элементов в CSS? Приведите примеры.
3. Что такое модель коробки (box model) и как она влияет на компоновку элементов?
4. Как можно использовать свойства display для управления компоновкой элементов?
5. Что такое блочная компоновка и как она работает в CSS?
6. Как использовать свойство float для компоновки элементов? Какие есть ограничения?
7. Что такое clearfix и как его использовать?
8. Как работает свойство position и какие его значения вы знаете?
9. Что такое относительная, абсолютная и фиксированная компоновка? Приведите примеры.
10. Как использовать Flexbox для создания адаптивных макетов?
11. Что такое Flex контейнер и Flex элементы? Как они взаимодействуют?
12. Как задать порядок элементов в Flexbox с помощью свойства order?
13. Что такое CSS Grid и как он отличается от Flexbox?
14. Как создать сетку с помощью CSS Grid? Приведите пример кода.
15. Как использовать медиа-запросы для адаптивной компоновки элементов на странице?

16. Как управлять пространством между элементами с помощью CSS (например, gap в Grid и Flexbox)?
17. Что такое адаптивный дизайн и как CSS3 помогает в его реализации?
18. Как использовать свойства align-items и justify-content в Flexbox?
19. Что такое позиционирование по сетке (grid positioning) и как оно работает?
20. Как можно комбинировать Flexbox и CSS Grid для создания сложных макетов?

ЗАКЛЮЧЕНИЕ

В ходе практикума по веб-разработке студенты получили ценные знания и практические навыки, которые станут основой для их дальнейшего обучения в области веб-технологий. Изучение утилит работы с сетью дало участникам возможность освоить инструменты для диагностики и мониторинга сетевых соединений, что является важным аспектом для понимания работы веб-приложений. Эти навыки помогут им эффективно решать проблемы, связанные с сетью, и обеспечивать стабильность своих проектов.

Понимание протокола HTTP, его методов и статусов ответов стало важной вехой в обучении студентов. Они научились анализировать запросы и ответы, что углубило их понимание взаимодействия клиента и сервера. Эти знания являются необходимыми для разработки высококачественных веб-приложений и подготовки к более сложным темам, связанным с серверной логикой и API.

Настройка рабочего места веб-разработчика позволила студентам создать комфортную и продуктивную среду для работы. Освоив инструменты, такие как редакторы кода и системы контроля версий, они будут готовы к дальнейшему изучению и практике в области разработки.

Изучение HTML и CSS дало участникам возможность создавать и стилизовать веб-страницы, что является основополагающим навыком для любого веб-разработчика. Эти знания откроют двери к более сложным концепциям, таким как JavaScript, фреймворки и библиотеки, которые будут рассмотрены в следующих частях курса. В целом, данный практикум стал важным шагом на пути к профессиональному росту студентов в сфере веб-разработки, подготовив их к дальнейшему изучению современных технологий и инструментов.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Хоган Б. HTML5 и CSS3. Веб-разработка по стандартам нового поколения. 2-е изд. – СПб.: Питер, 2014. – 320 с.
2. Дакетт Д. HTML и CSS. Разработка и дизайн веб-сайтов / Джон Дакетт; [пер. с англ. М.А. Райтмана]. – М.: Эксмо, 2013. – 480 с.
3. Дронов В.А. PHP, MySQL, HTML5 и CSS3. Разработка современных динамических Web-сайтов. – СПб.: БХВ-Петербург, 2016. – 688 с.
4. Дронов В.А. HTML5, CSS3 и Web 2.0. Разработка современных Web-сайтов. – СПб.: БХВ-Петербург, 2011. – 416 с.
5. Полуэктова Н.Р. Разработка веб-приложений: учебное пособие для вузов / Н.Р. Полуэктова. – 2-е изд. – Москва: Издательство Юрайт, 2024. – 204 с.
6. Янцев В.В. Разработка web-страниц на HTML, CSS и JavaScript: учебное пособие для вузов / В.В. Янцев. – Санкт-Петербург: Лань, 2024. – 148 с.